

Vivado Design Suite 用户指南

高层次综合

UG902 (v2019.2) 2020 年 1 月 13 日

条款中英文版本如有歧义，概以英文版本为准。



修订历史

下表列出了本文档的修订历史。

章节	修订综述
2020 年 1 月 13 日 2019.2 版	
更新命令参考信息。	更新命令。
2019 年 10 月 30 日 2019.2 版	
HLS 数学库和定点数学函数	移除 Gamma 函数。
2019 年 7 月 12 日 2019.1 版	
移除假性依赖关系以改善循环流水线化	澄清有关依赖关系的信息。
2019 年 5 月 22 日 2019.1 版	
吞吐量最优化	更新有关数据流和流水线化吞吐量部分的信息。
将数组指定为乒乓缓存或 FIFO	更新乒乓缓存的解释。
Stable 数组、set_directive_stable	新增有关 stable 数组的信息。
在数据流内使用 ap_ctrl_none	新增有关在数据流内使用 ap_ctrl_none 的信息。
RTL 黑盒、RTL 黑盒 JSON 文件和 add_files	新增有关 RTL 黑盒新功能的信息、新增必需 JSON 文件的规格，并更新 add_files 命令以包含 -blackbox 选项。
波形查看器	新增有关波形查看器 (Waveform Viewer) 的信息。
SSR FFT IP 库	新增有关新超采样率 (SSR) FFT 的信息。 新增下列小节： 使用 SSR FFT 定点配置的建议流程 SSR FFT IP 库的使用

目录

修订历史.....	2
第 1 章：高层次综合.....	5
高层次综合的优势.....	5
高层次综合基础.....	5
理解 Vivado HLS.....	10
使用 Vivado HLS.....	15
高效硬件的数据类型.....	59
管理接口.....	65
对设计进行最优化.....	97
验证 RTL.....	143
导出 RTL 设计.....	154
第 2 章：高层次综合 C 语言库.....	160
任意精度数据类型库.....	160
HLS 流传输库.....	172
HLS 数学库.....	179
HLS 视频库.....	188
HLS IP 库.....	188
HLS 线性代数库.....	214
HLS DSP 库.....	223
HLS SQL 库.....	225
第 3 章：高层次综合编码样式.....	226
不受支持的 C 语言构造.....	226
C 语言测试激励文件.....	230
函数.....	236
RTL 黑盒.....	237
循环.....	242
数组.....	248
数据类型.....	255
C 语言内置函数.....	276
硬件高效型 C 语言代码.....	277
C++ 类和模板.....	292
断言.....	299
SystemC 综合.....	301
第 4 章：高层次综合参考指南.....	318
命令参考.....	318

GUI 参考.....	379
接口综合参考.....	383
AXI4-Lite 从接口 C 驱动参考.....	397
HLS 视频函数库.....	409
HLS 线性代数库函数.....	409
HLS DSP 库函数.....	417
HLS SQL 库函数.....	429
C 语言任意精度类型.....	431
C++ 任意精度类型.....	443
C++ 任意精度定点类型.....	460
SystemC 类型与 Vivado HLS 类型之比较.....	479
RTL 黑盒 JSON 文件.....	485
附录 A：附加资源与法律提示.....	488
赛灵思资源.....	488
Documentation Navigator 与设计中心.....	488
参考资料.....	488
请阅读：重要法律提示.....	489

高层次综合

赛灵思 Vivado[®] 高层次综合 (HLS) 工具将 C 语言规格转换为寄存器传输级 (RTL) 实现，供您综合到赛灵思现场可编程门阵列 (FPGA) 中。您可使用 C、C++ 或 SystemC 来编写 C 语言规格，FPGA 可提供大规模并行处理架构，其性能、成本和功耗都比传统处理器更胜一筹。本章提供了高层次综合概述。

注释：如需了解有关 FPGA 架构和 Vivado HLS 基本概念的更多信息，请参阅《采用 Vivado 高层次综合开展 FPGA 设计的简介》(UG998)。

高层次综合的优势

高层次综合将硬件域和软件域桥接在一起，从而提供如下主要优势：

- 提升硬件设计人员的工作效率
创建高性能硬件时，硬件设计人员可以在更高的抽象层次上开展工作。
- 为软件设计人员提升系统性能
软件开发人员可以在新的编译目标（即 FPGA）上加速完成其算法的计算密集型部分操作。

通过使用高层次综合设计方法，您即可：

- 在 C 语言层次开发算法
从占用开发时间的实现细节中抽身
- 在 C 语言层次执行验证
以相比于传统硬件描述语言更快的速度验证设计的功能正确性。
- 通过最优化指令来控制 C 语言综合进程
创建特定的高性能硬件实现。
- 使用最优化指令从 C 语言源代码创建多种实现
浏览设计空间，提升找到最优化实现的可能性。
- 创建可读且可移植的 C 语言源代码
将 C 语言源代码目标调整为其它器件，并将 C 语言源代码整合到新工程中。

高层次综合基础

高层次综合所含阶段如下：

- 调度

根据下列条件判定每个时钟周期内发生的操作：

- 时钟周期或时钟频率长度
- 操作完成所需时间（由目标器件来定义）
- 用户指定的最优化指令

如果时钟周期延长或者目标 FPGA 速度加快，那么在单一时钟周期内可完成更多操作，所有操作都可能在一个时钟周期内完成。反之，如果时钟周期缩短或者目标 FPGA 速度减慢，那么高层次综合会自动将操作调度到更多个时钟周期内完成，部分操作可能需要作为多周期资源来实现。

- 绑定

判定用于实现调度的每项操作所需的硬件资源。高层次综合会使用有关目标器件的信息来实现最优化解决方案。

- 控制逻辑提取

提取控制逻辑以创建有限状态机 (FSM) 以便对 RTL 设计中的操作进行排序。

高层次综合按如下方式综合 C 语言代码：

- 顶层函数实参综合到 RTL I/O 端口内
- C 语言函数综合到 RTL 层级内的块中

如果 C 语言代码包含子函数层级，那么最终 RTL 设计将包含与原始 C 语言函数层级具有一对一对应关系的模块或实体层级。函数的所有实例都使用相同 RTL 实现或时钟。

- C 语言函数中的循环默认情况下全部保持收起状态。

收起循环时，综合会为循环的单次迭代创建逻辑，RTL 设计会为序列中循环的每次迭代都执行此逻辑。通过使用最优化指令，即可展开循环以允许并行执行所有迭代。循环还可通过如下任一方法实现流水线化：通过有限状态机细颗粒度实现（循环流水线化）或者采用基于较粗颗粒度的握手的实现（数据流）。

- C 语言代码中的数组可综合到最终 FPGA 设计中的块 RAM 或 UltraRAM 中

如果数组位于顶层函数接口上，那么高层次综合可将此数组作为端口来实现，以便访问设计外部的块 RAM。

高层次综合可基于默认行为、约束和您指定的任意最优化指令来创建最优化的实现。您可使用最优化指令来修改和控制内部逻辑和 I/O 端口的默认行为。这样您即可从相同 C 语言代码生成不同的硬件实现。

为判定设计是否满足您的要求，您可复查高层次综合生成的综合报告中的性能指标。分析报告后，您可使用最优化指令来对实现进行调优。综合报告包含有关如下性能指标的信息：

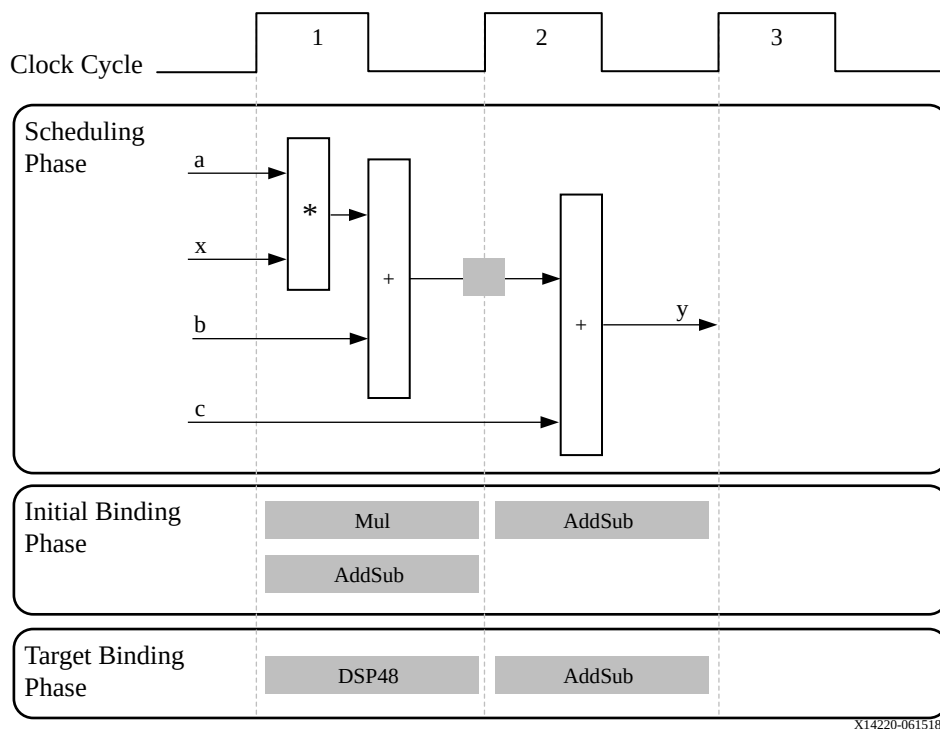
- 面积：根据 FPGA 中可用资源（包括查找表 (LUT)、寄存器、块 RAM 和 DSP48）实现设计所需的硬件资源量。
- 时延：函数计算所有输出值所需的时钟周期数。
- 启动时间间隔 (II)：函数接受新输入数据之前的时钟周期数。
- 循环迭代时延：完成循环的单次迭代所需的时钟周期数。
- 循环启动时间间隔：下一次循环迭代开始处理数据前的时钟周期数。
- 循环时延：执行所有循环迭代的周期数。

调度和绑定示例

下图显示了此代码示例的调度和绑定阶段的示例：

```
int foo(char x, char a, char b, char c) {
    char y;
    y = x*a+b+c;
    return y;
}
```

图 1：调度和绑定示例



在此示例的调度阶段，根据高层次综合的调度，在每个时钟周期内将执行以下操作：

- 第一个时钟周期：乘法和第一次加法
- 第二个时钟周期：第二次加法并生成输出

注释：在上图中，第一个和第二个时钟周期之间的方块指示内部寄存器存储变量的时间。在此示例中，高层次综合仅要求在一个时钟周期内寄存加法的输出。第一个周期读取 *x*、*a* 和 *b* 数据端口。第二个周期读取数据端口 *c* 并生成输出 *y*。

在最终的硬件实现中，高层次综合将顶层函数的实参实现为输入和输出 (I/O) 端口。在此示例中，实参是简单数据端口。因为每个输入变量都是 *char* 类型，所以输入数据端口位宽均为 8 位。*return* 函数为 32 位 *int* 数据类型，而输出数据端口位宽为 32 位。



重要提示！ 在硬件中实现 C 语言代码的优点是所有运算都在更少的时钟周期内完成。在此示例中，完成运算仅需 2 个时钟周期。在中央处理器 (CPU) 中，即使这个简单的代码示例也需要更多时钟周期才能完成。

在此示例的初始绑定阶段，高层次综合使用组合乘法器 (*Mul*) 实现乘法运算，并使用组合加法器/减法器 (*AddSub*) 实现两次加法运算。

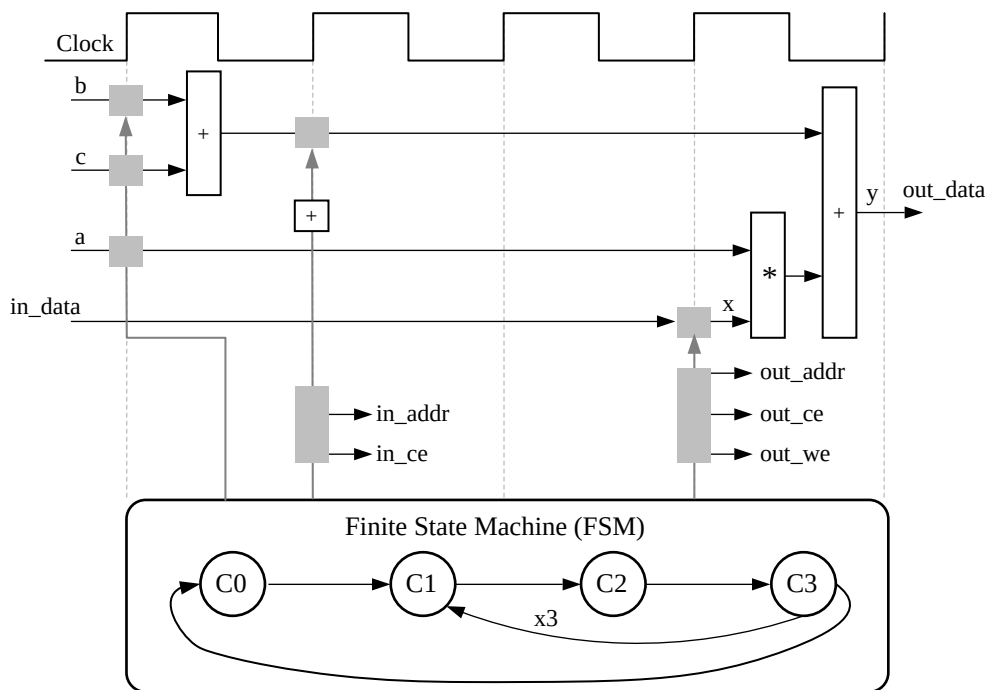
在目标绑定阶段，高层次综合使用 DSP48 资源实现乘法器和其中一项加法运算。DSP48 资源是 FPGA 架构中可用的计算块，可在高性能和高效实现之间达成理想的平衡。

控制逻辑提取和 I/O 端口实现示例

下图显示了对应此代码示例的控制逻辑的提取和 I/O 端口的实现过程：

```
void foo(int in[3], char a, char b, char c, int out[3]) {
    int x,y;
    for(int i = 0; i < 3; i++) {
        x = in[i];
        y = a*x + b + c;
        out[i] = y;
    }
}
```

图 2：控制逻辑提取和 I/O 端口实现示例



X14218

此代码示例执行的运算与前述示例相同。但它在 for 循环内执行运算，且 2 个函数实参为数组。调度此代码后，生成的设计在 for 循环内执行 3 次该逻辑。高层次综合会从 C 语言代码自动提取控制逻辑，并在 RTL 设计中创建 FSM 以对这些运算进行排序。高层次综合在最终 RTL 设计中将顶层函数实参作为端口来实现。类型为 char 的标量变量将映射到标准 8 位数据总线端口。数组实参（如 in 和 out）包含完整数据集。

在高层次综合中，数组默认综合到块 RAM 中，但有其它选项可供选择，例如，FIFO、分布式 RAM 和独立寄存器。使用数组作为顶层函数中的实参时，高层次综合假定块 RAM 位于顶层函数外，并自动创建端口以访问位于设计外的块 RAM，例如，数据端口、地址端口和任何必需的芯片使能信号或写使能信号。

FSM 用于控制寄存器何时存储数据并控制任意 I/O 控制信号的状态。FSM 启动状态为 C0。在下一个时钟上，它会按顺序依次进入 C1 状态、C2 状态和 C3 状态。它会返回 C1、C2 和 C3 状态总计 3 次，最后返回 C0 状态。

注释：这与 C 语言代码中 for 循环的控制结构非常相似。状态的完整顺序为：C0、{C1, C2, C3}、{C1, C2, C3}、{C1, C2, C3}，然后返回 C0。

设计要求仅添加 1 次 b 和 c。高层次综合会将运算移出 for 循环，并移入 C0 状态。每次设计进入 C3 状态时，都会复用加法的结果。

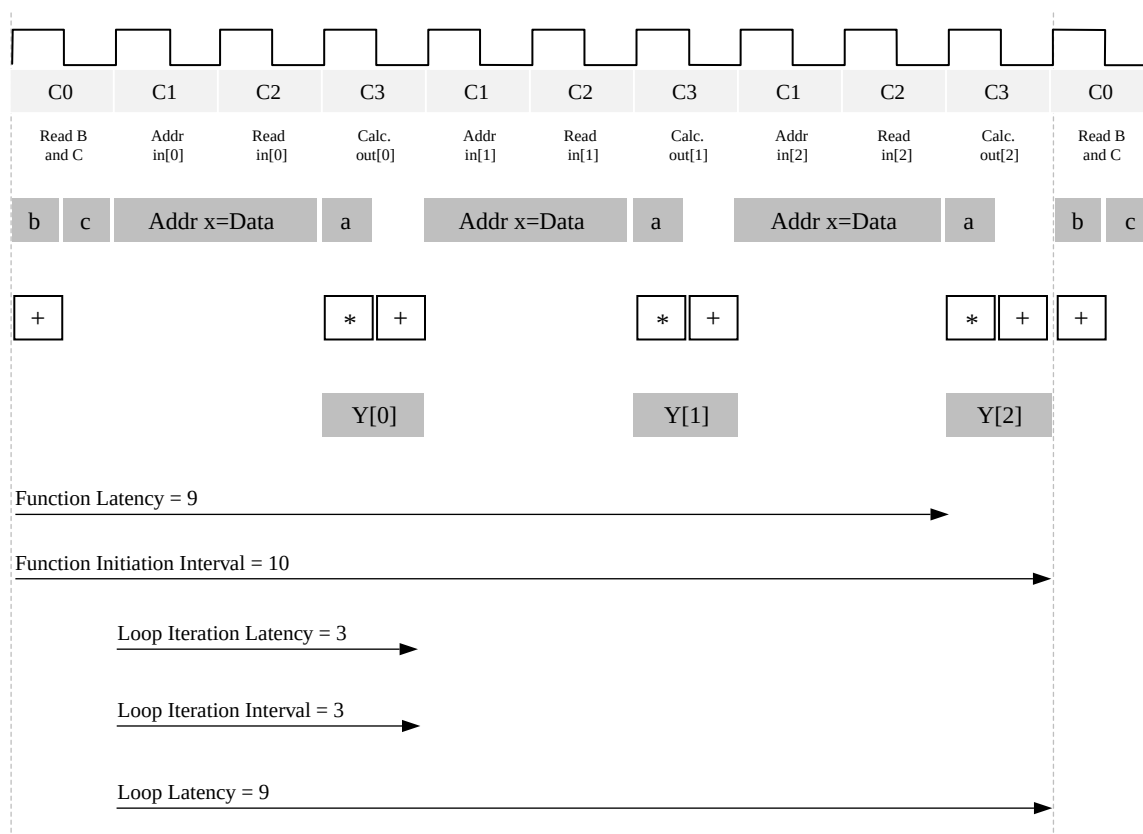
设计会从 in 读取数据，并将数据存储到 x 中。FSM 会为处于 C1 状态的首个元素生成地址。此外，在 C1 状态中，加法器会递增以记录设计必须围绕 C1、C2 和 C3 状态进行迭代的次数。在 C2 状态下，块 RAM 会返回 in 的数据，并将其存储在 x 变量中。

高层次综合会从含其它值的 a 端口读取数据，以执行计算，并生成首个 y 输出。FSM 可确保生成正确的地址和控制信号，以将该值存储在块范围外。随后，设计会返回 C1 状态以从数组/块 RAM in 读取下一个值。此流程将持续直至写完所有输出为止。随后，设计会返回 C0 状态，以读取 b 和 c 的下一个值，以重新开始此流程。

性能指标示例

下图显示了上一个示例中每个周期的代码完整执行过程，包括每个时钟周期的状态、读取操作、计算操作和写入操作。

图 3：时延和启动时间间隔示例



X14219

以下是此示例的性能指标：

- 时延：该函数需要 9 个时钟周期才能输出所有值。

注释：当输出为数组时，时延测量到最后一个数组值输出为止。

- II：II 为 10，这意味着该函数需要 10 个时钟周期后才能启动一组新的输入读取并开始处理下一组输入数据。

注释：函数的一次完整执行的时间称为一项传输事务。在此示例中，该函数需要 11 个时钟周期后才能接受下一项传输事务的数据。

- 循环迭代时延：每次循环迭代的时延为 3 个时钟周期。
- 循环 II：时间间隔为 3。
- 循环时延：时延为 9 个时钟周期。

理解 Vivado HLS

赛灵思 Vivado HLS 工具会将 C 语言函数综合为 IP 块，供您集成到硬件系统中。它与其它赛灵思设计工具紧密集成，提供全方位语言支持和功能，用于为 C 语言算法创建最优化的实现。

以下是 Vivado HLS 设计流程：

1. 编译、执行（仿真）和调试 C 语言算法。
2. 将 C 语言算法综合为 RTL 实现，可选择使用用户最优化指令。
3. 生成综合性报告并分析设计。
4. 使用按钮式流程来验证 RTL 实现。
5. 将 RTL 设计实现封装为一套选定的 IP 格式。

注释：在高层次综合中，运行编译后的 C 语言程序被称为 C 语言仿真。运行 C 语言算法，仿真该函数以验证算法功能是否正确。

输入和输出

以下是 Vivado® HLS 输入：

- 以 C、C++ 或 SystemC 编写的 C 语言函数

这是 Vivado HLS 的主要输入。函数可包含子函数层级。

- 约束

约束为必需，可包含时钟周期、时钟不确定性和 FPGA 目标。时钟不确定性如未指定，则默认设为时钟周期的 12.5%。

- 指令

指令为可选，用于指示综合流程实现特定行为或最优化操作。

- C 语言测试激励文件及所有关联文件

Vivado HLS 使用 C 语言测试激励文件在综合前对 C 语言函数进行仿真并使用 C/RTL 协同仿真来验证 RTL 输出。

您可使用 Vivado HLS 图形用户界面 (GUI) 或者在命令提示符处使用 Tcl 命令以交互方式将 C 输入文件、指令和约束添加到 Vivado HLS 工程中。您还可创建 Tcl 文件并以批处理模式执行命令。

以下是 Vivado HLS 输出：

- 硬件描述语言 (HDL) 格式的 RTL 实现文件

这是来自 Vivado HLS 的主要输出。通过使用 Vivado 综合即可将 RTL 综合到门级实现和 FPGA 比特流文件中。RTL 可采用以下业界标准格式：

- VHDL (IEEE 1076-2000)
- Verilog (IEEE 1364-2001)

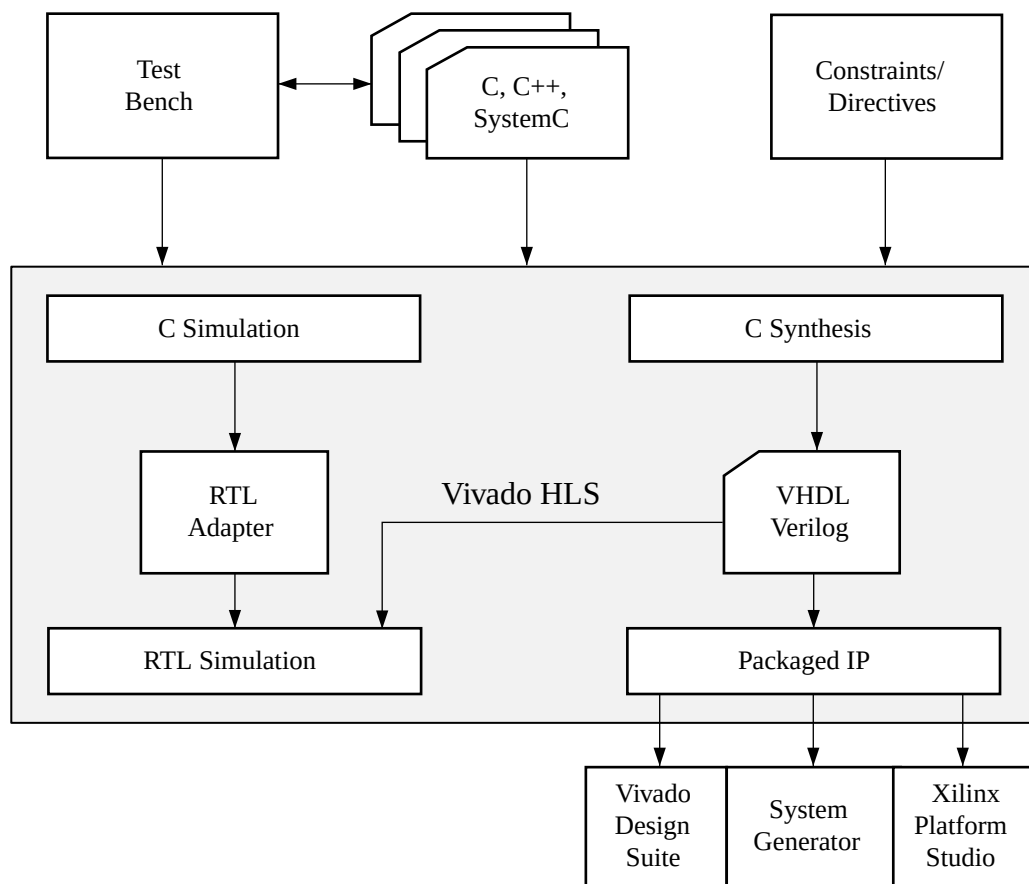
Vivado HLS 将实现文件封装为 IP 块，以便配合其它工具一起在赛灵思设计流程中使用。通过使用逻辑综合即可将已封装的 IP 综合为 FPGA 比特流。

· 报告文件

此输出即综合、C/RTL 协同仿真与 IP 封装的结果。

下图显示了 Vivado HLS 输入和输出文件的概述。

图 4：Vivado HLS 设计流程



X14309

测试激励文件、语言支持和 C 语言库

在任意 C 程序中，顶层函数都称为 `main()`。在 Vivado® HLS 设计流程中，您可将 `main()` 下的任意子函数指定为综合的顶层函数。您无法综合顶层函数 `main()`。其它规则如下：

- 仅允许指定 1 个函数作为综合的顶层函数。

- 层级中综合的顶层函数下的任意子函数也同样会进行综合。
- 如果要综合的函数不包含在综合的顶层函数下的层级内，则必须将此类函数合并到综合的单一顶层函数中。

测试激励文件

使用 Vivado® HLS 设计流程时，对功能错误的 C 语言函数进行综合，然后分析实现详细信息，以判定此函数功能与期望不符的原因，这个过程较为耗时。为提高效率，在综合前可使用测试激励文件来验证 C 语言函数功能是否正确。

C 语言测试激励文件包含 `main()` 函数以及不包含在用于综合的顶层函数下的层级中的所有子函数。这些函数会通过向用于综合的顶层函数提供激励并使用其输出，来验证该函数的功能是否正确。

Vivado HLS 使用测试激励文件来编译和执行 C 语言仿真。在编译流程中，您可选择“Launch Debugger”选项以打开完整的 C 语言调试环境，以便分析 C 语言仿真。



建议：由于 Vivado HLS 可使用测试激励文件在综合前验证 C 语言函数并自动验证 RTL 输出，因此强烈建议使用测试激励文件。

语言支持

Vivado HLS 支持以下 C 编译/仿真标准：

- ANSI-C (GCC 4.6)
- C++ (G++ 4.6)
- SystemC (IEEE 1666-2006 v2.2)

C、C++ 和 SystemC 语言构造

Vivado HLS 支持许多 C、C++ 和 SystemC 语言构造以及每一种语言的所有原生数据类型，包括浮点和双精度类型。但是，针对部分构造不支持综合，包括：

- 动态内存分配

FPGA 具有一组固定资源，不支持动态创建和清空内存资源。

- 操作系统 (OS) 操作

往来 FPGA 的所有数据都必须从输入端口读取或写入输出端口。不支持操作系统操作（例如，文件读写或时间和日期之类的操作系统查询）。改由 C 语言测试激励文件执行这些操作并将数据传递到函数中以便作为函数实参来进行综合。

如需了解有关受支持和不受支持的 C 语言构造的详细信息以及每一种主要构造的示例，请参阅[第 3 章：高层次综合编码样式](#)。

C 语言库

C 语言库包含专为在 FPGA 中实现而最优化的函数和构造。使用这些库有助于确保高质量结果 (QoR)，即最终输出为最优化资源利用率的高性能设计。由于这些库在 C、C++ 或 SystemC 语言中提供，您可将这些库整合到 C 语言函数中，并在综合前对其进行仿真以验证功能正确性。

Vivado® HLS 提供了以下 C 语言库以扩展标准 C 语言：

- 任意精度数据类型
- 半精度（16 位）浮点数据类型

- 数学运算
- 赛灵思 IP 函数，包括快速傅里叶变换 (FFT) 和有限冲激响应 (FIR)
- FPGA 资源致力于帮助最大限度提升移位寄存器 LUT (SRL) 资源的利用率

C 语言库示例

C 语言库可确保 QoR 值高于标准 C 语言类型。标准 C 语言类型基于 8 位边界（8 位、16 位、32 位和 64 位）。但以硬件平台为目标时，通常使用特定宽度的数据类型更高效。

例如，含用于通信协议的筛选函数的设计需要 10 位输入数据和 18 位输出数据以满足数据传输需求。使用标准 C 语言数据类型时，输入数据必须至少为 16 位，输出数据必须至少为 32 位。在最终硬件中，这会在输入和输出之前创建过宽的数据路径、使用过多的资源、延迟过长（例如，32 位乘 32 位的乘法耗时比 18 位乘 18 位的乘法更长），并且需要更多时钟周期才能完成。

如果在此设计中改为使用任意精度数据类型，即可在综合前指定与在 C 语言代码中指定的位大小完全相同的位大小、对更新后的 C 语言代码进行仿真，并使用 C 语言仿真验证输出质量。任意精度数据类型是专为 C 和 C++ 提供的，支持您对 1 位到 1024 位之间的任意位宽的数据类型进行建模。例如，您可对某些高达 32768 位的 C++ 类型进行建模。

注释：任意精度类型通常仅在函数边界处才需要，因为 Vivado HLS 会最优化内部逻辑并移除不扇出到输出端口的数据位和逻辑。

综合、最优化和分析

Vivado® HLS 是围绕工程运行的。每个工程都包含一组 C 语言代码，并且可包含多种解决方案。每个解决方案都有不同约束和最优化指令。您可在 Vivado HLS GUI 中分析和比较每个解决方案的结果。

以下是 Vivado HLS 设计进程中的综合、最优化和分析步骤：

1. 通过初始解决方案创建工程。
2. 验证 C 语言仿真是否能无错执行。
3. 运行综合以获取一组结果。
4. 分析结果。

分析结果后，即可为工程创建含不同约束和最优化指令的新解决方案，并对新解决方案进行综合。您可重复此过程，直至设计达成所期望的性能特性为止。您可使用多种解决方案持续进行开发，同时仍可保留先前的解决方案结果。

最优化

使用 Vivado® HLS 时可以将不同的最优化指令应用于设计，包括：

- 指示任务在流水线中执行，允许在当前任务完成前即开始执行下一项任务。
- 指定完成函数、循环和区域的时延。
- 指定使用的资源数量限制。
- 覆盖代码中的固有或隐含依赖关系，并允许执行指定的操作。例如，如果可以放弃或忽略初始数据值（例如在视频流中），则允许对内存先读取后写入，前提是这样可提高性能。
- 选择 I/O 协议以确保最终设计可以连接到具有相同 I/O 协议的其它硬件块。

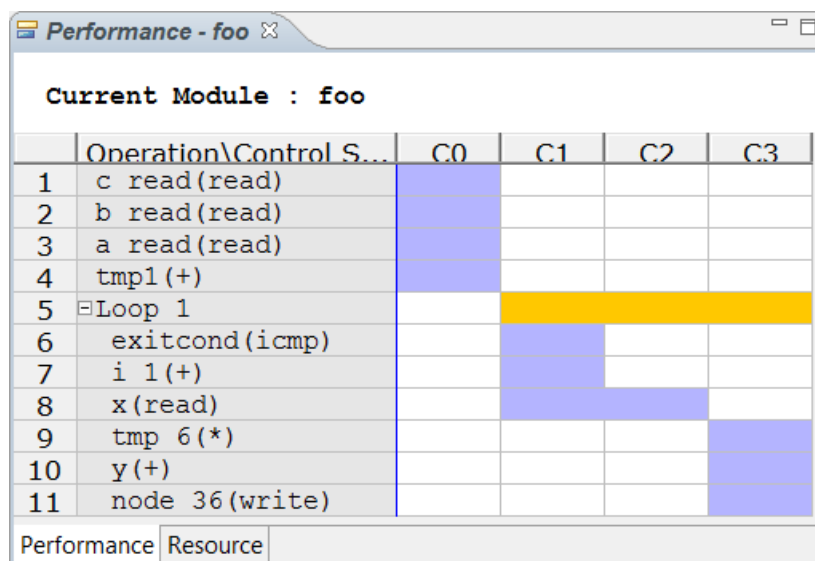
注释：Vivado HLS 自动确定任何子函数使用的 I/O 协议。您不能控制这些端口，只能指定端口是否寄存。

您可以使用 Vivado HLS GUI 将最优化指令直接放入源代码中。另外，您可以使用 Tcl 命令来应用最优化指令。

分析

当综合完成后，Vivado® HLS 会自动创建综合报告以帮助了解实现的性能。在 Vivado HLS GUI 中，“Analysis”透视图包含“Performance”视图，支持您以交互方式详细分析结果。下图显示了控制逻辑提取和 I/O 端口实现示例的“Performance”视图。

图 5: Vivado HLS 分析示例



	Operation\Control S...	C0	C1	C2	C3
1	c read(read)				
2	b read(read)				
3	a read(read)				
4	tmp1(+)				
5	Loop 1				
6	exitcond(icmp)				
7	i 1(+)				
8	x(read)				
9	tmp 6(*)				
10	y(+)				
11	node 36(write)				

“Performance”视图针对每个状态显示以下内容：

- C0：第一种状态包含针对 a、b 和 c 端口的读取操作以及加法运算。
- C1 和 C2：设计进入循环，并检查循环增量计数器和退出条件。随后，设计将数据读取到 x 变量中，此过程需 2 个时钟周期。这是因为设计访问块 RAM 时需 1 个时钟周期来获取地址，还需 1 个时钟周期执行数据读取。
- C3：设计执行计算，并将输出写入 y 端口。然后，循环返回开始位置。

RTL 验证

如果已将 C 语言测试激励文件添加到工程中，则可将其用于验证 RTL 功能与原始 C 是否相同。C 语言测试激励文件会验证综合的顶层函数的输出，如果 RTL 功能相同，则向顶层函数 `main()` 返回 0。Vivado® HLS 会将该返回值用于 C 语言仿真和 C/RTL 协同仿真，以判定结果是否正确。如果 C 语言测试激励文件返回非 0 值，Vivado HLS 会报告仿真失败。如需了解更多信息，请参阅[测试激励文件要求](#)。



提示： Vivado HLS 会自动创建基础架构，以执行 C/RTL 协同仿真，并使用以下受支持的 RTL 仿真器之一来自动执行仿真。

- Vivado 仿真器 (XSim)
- ModelSim 仿真器
- VCS
- NCSim
- Riviera
- Xcelium

如果选择 Verilog 或 VHDL HDL 进行仿真，那么 Vivado HLS 会使用您指定的 HDL 仿真器。赛灵思设计工具包含 Vivado 仿真器。第三方 HDL 仿真器需要来自第三方供应商的许可证。VCS 和 NCSim 仿真器仅在 Linux 操作系统上受支持。

RTL 导出

通过使用 Vivado® HLS 可导出 RTL，并将最终 RTL 输出文件封装为采用以下任一赛灵思 IP 格式的 IP：

- Vivado IP Catalog

导入 Vivado IP 目录，以便在 Vivado Design Suite 中使用。

- System Generator for DSP

将 HLS 设计导入 System Generator。

- Synthesized Checkpoint (.dcp)

按导入任意 Vivado Design Suite 检查点的相同方式直接导入 Vivado Design Suite。

注释：综合后的检查点格式会调用逻辑综合并将 RTL 实现编译为门级实现，以包含在 IP 封装中。

对于除已综合的检查点外的所有 IP 格式，您可选择从 Vivado HLS 内执行逻辑综合，以对 RTL 综合或实现结果进行评估。此可选步骤允许您在交付 IP 封装前确认 Vivado HLS 为时序和面积提供的估算值。这些门级结果不包含在已封装的 IP 中。

注释：Vivado HLS 会根据每个 FPGA 的内置库估算时序和面积资源。使用逻辑综合将 RTL 编译到门级实现中、在 FPGA 的门中执行物理布局，以及执行门间连接布线时，逻辑综合可能需要执行额外的最优化以更改 Vivado HLS 估算值。

使用 Vivado HLS

要在 Windows 平台上打开 Vivado® HLS，请双击桌面按钮，如下图所示。

图 6：Vivado HLS GUI 按钮



要在 Linux 平台上（或者从 Windows 上的 Vivado HLS 命令提示符处）调用 Vivado HLS，请在命令提示符处执行以下命令。

```
$ vivado_hls
```

这将打开 Vivado HLS GUI，如下图所示。

图 7: Vivado HLS GUI Welcome 页面



您可使用“Quick Start”字段来执行以下任务：

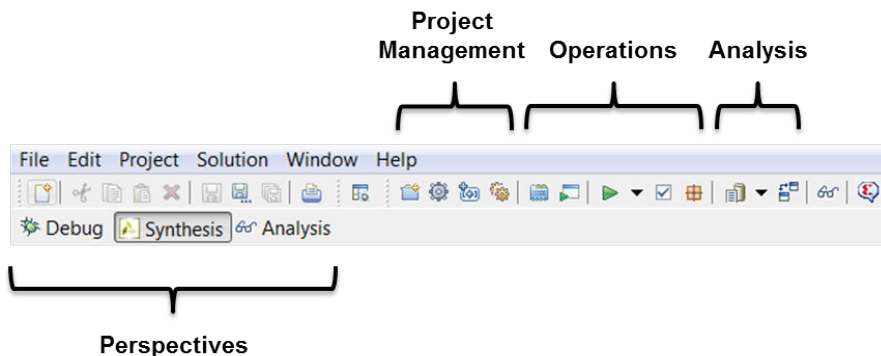
- “Create New Project”：启动工程设置向导。
- “Open Project”：浏览现有工程或者从最近的工程列表中选择。
- “Open Example Project”：打开 Vivado HLS 示例。

您可使用“Documentation”选项来执行以下任务：

- “Tutorials”：打开《Vivado Design Suite 教程：高层次综合》(UG871)。
- “User Guide”：打开《Vivado Design Suite 用户指南：高层次综合》(UG902) 文档。
- “Release Notes Guide”：打开对应最新软件版本的《Vivado Design Suite 用户指南：版本说明、安装和许可》(UG973)。

下图中所示工具栏中显示了用于使用 Vivado HLS 的主要控件。工程控件可确保仅高亮显示当前可执行的命令。例如，必须先执行综合，然后才能执行 C/RTL 协同仿真。“C/RTL co-simulation”工具栏按钮将保持灰显，直至综合完成为止。

图 8: Vivado HLS 控件



在“工程管理 (Project Management)”部分，按钮从左到右依次为：

- “Create New Project”，用于打开新工程向导。
- “Project Settings”，用于修改当前工程设置。
- “New Solution”，用于打开新解决方案对话框。
- “Solution Settings”，用于修改当前解决方案设置。

下一组工具栏按钮用于控制工具操作，从左到右依次为：

- “Index C Source”，用于刷新 C 语言源代码中的注释。
- “Run C Simulation”，用于打开“C 语言仿真 (C Simulation)”对话框。
- “C Synthesis”，用于在 Vivado HLS 中启动 C 语言源代码。
- “Run C/RTL Cosimulation”，用于验证 RTL 输出。
- “Export RTL”，用于将 RTL 封装为期望的 IP 输出格式。

最后一组工具栏按钮用于执行设计分析，从左到右依次为：

- “Open Report”，用于打开 C 语言综合报告，或者下拉以打开其它报告。
- “Compare Reports”，允许对比来自其它解决方案的报告。

工具栏上每个按钮都在菜单中有等效的命令。此外，Vivado HLS GUI 还提供了 3 个透视图。选择透视图时，窗口会自动调整为适应所选任务的布局。

- “Debug”透视图用于打开 C 语言调试器。
- “Synthesis”透视图是默认透视图，用于排列窗口以便执行综合。
- “Analysis”透视图在综合完成后用于对设计进行详细分析。此透视图提供的细节远比综合报告多。

可随时选择所需透视图按钮来切换透视图。

本章剩余部分主要讨论如何使用 Vivado HLS。其中涵盖了下列主题：

- 如何创建 Vivado HLS 综合工程。
- 如何对 C 语言代码进行仿真和调试。
- 如何对设计进行综合、创建新解决方案和添加最优化。

- 如何执行设计分析。
- 如何对 RTL 输出进行验证和封装。
- 如何使用 Vivado HLS Tcl 命令和批处理模式。

本章最后回顾了可提供更多信息的设计示例、教程和资源。

创建新的综合工程

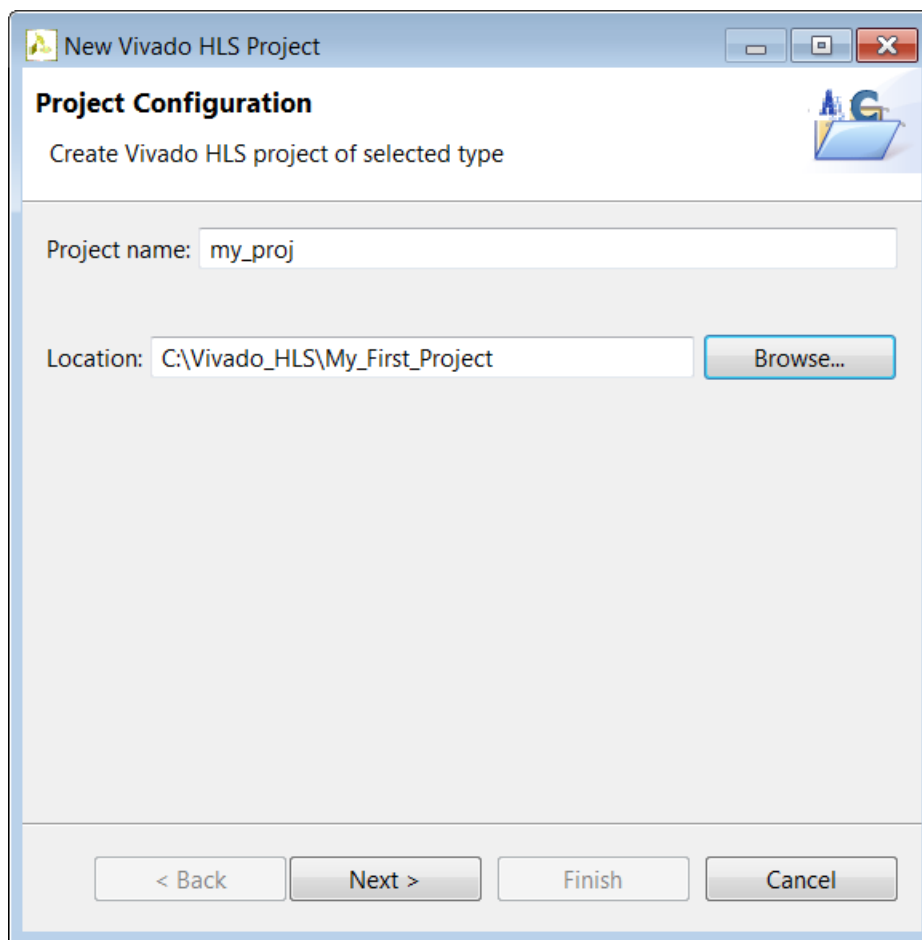
要创建新工程，请在“Welcome”页面上单击“Create New Project”链接，或者选择“File” → “New Project”菜单命令。这样会打开[创建新的综合工程](#)中所示的工程向导，以便您指定以下内容：

- “Project Name”：指定工程名称，与用于存储工程详细信息的目录名称相同。
- “Location”：指定工程存储位置。



注意！ Windows 操作系统对于路径长度存在 260 个字符的限制，这可能影响 Vivado 工具。为避免此问题，创建工程、定义 IP 或托管 IP 工程以及创建块设计时，请尽可能使用较短的名称和目录位置。

图 9：工程规格



选择“Next >”按钮使向导进入第 2 个屏幕，以便您在其中输入工程 C 语言源文件中的详细信息（[创建新的综合工程](#)）。

- “Top Function”：指定要综合的顶层函数的名称。如果您首先添加 C 语言文件，则可使用“Browse”按钮来复查 C 语言层级，然后选择要综合的顶层函数。“Browse”按钮为灰显，直至您添加源文件为止。

注释：当工程指定为 SystemC 时无需执行此步骤，因为 Vivado HLS 会自动识别顶层函数。

使用“Add Files”按钮可向工程添加源代码文件。



重要提示！ 请勿使用“Add Files”按钮（或关联的 `add_files` Tcl 命令）向工程添加头文件（含 `.h` 后缀的文件）。

Vivado HLS 会自动将以下目录添加到搜索路径中：

- 工作目录
注释：工作目录包含 Vivado HLS 工程目录。
- 包含已添加到工程的 C 语言文件的任意目录

驻留在这些目录中的头文件会自动包含在工程中。您必须使用“Edit CFLAGS”按钮指定到所有其它头文件的路径。

“Edit CFLAGS”按钮用于指定编译 C 语言代码所需的 C 语言编译器标记选项。这些编译器标记选项与 gcc 或 g++ 中使用的选项相同。C 语言编译器标记包含头文件、宏规格和编译器指令的路径名称，如以下示例所示：

- “-I/project/source/headers”：提供关联头文件的搜索路径
注释：您必须指定与工作目录（而非工程目录）关联的相对路径名称。
- “-DMACRO_1”：定义编译期间的宏 `MACRO_1`
- “-fnested-functions”：定义包含嵌套函数的任何设计所需的指令

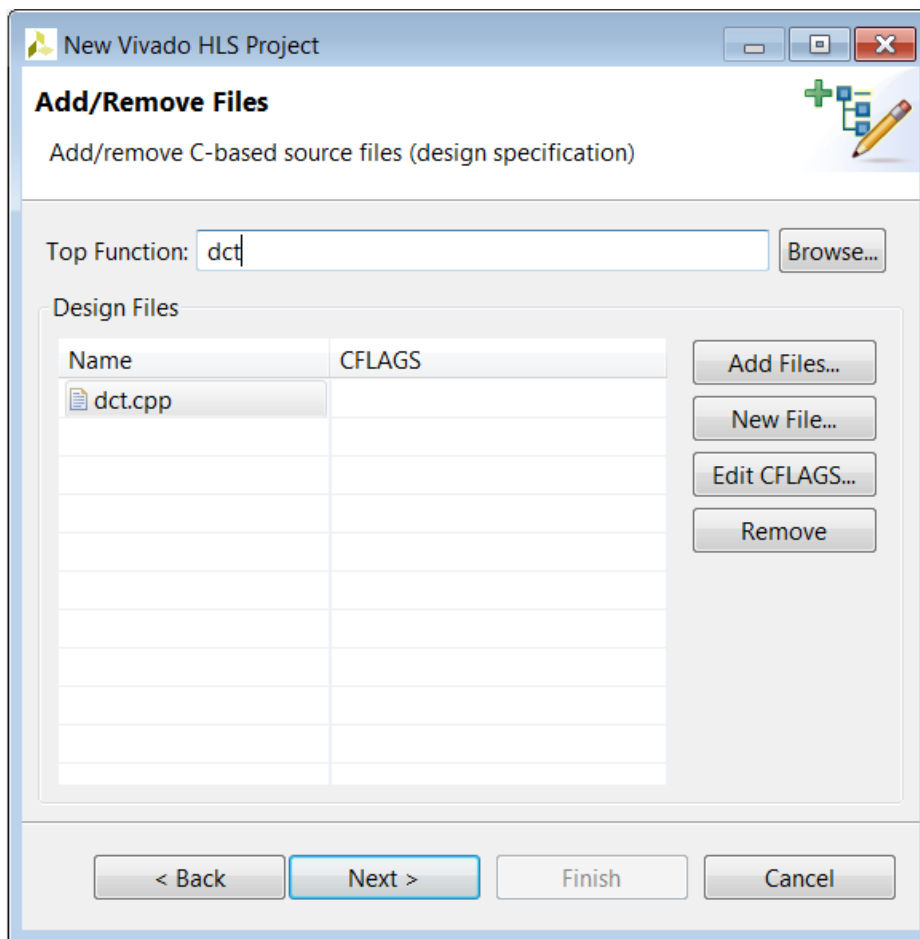


提示：要获取受支持的“Edit CFLAGS”选项的完整列表，请参阅 GNU Compiler Collection (GCC) 网站上的“Option Summary”页面 (<http://gcc.gnu.org/onlinedocs/gcc/Option-Summary.html>)。



提示：您可使用 `$::env(MY_ENV_VAR)` 来指定 CFLAGS 中的环境变量。例如，要在编译中包含目录 `$MY_ENV_VAR/include`，可指定 `-I$::env(MY_ENV_VAR)/include` in CFLAGS。

图 10：工程源文件

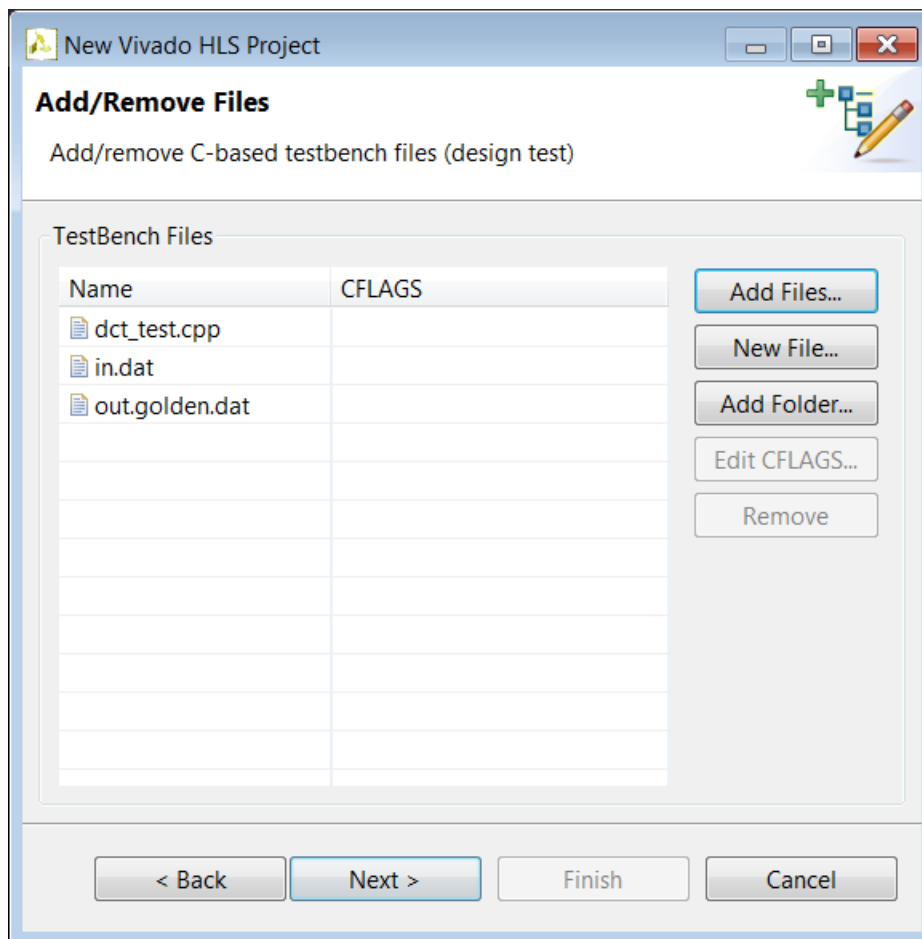


工程向导中的下一个窗口允许您向工程添加测试激励文件关联的文件。

注释：对于包含与测试激励文件关联的头文件但不含设计文件的 SystemC 语言设计，您必须使用“Add Files”按钮向工程添加头文件。

在随 Vivado HLS 提供的大部分设计示例中，测试激励文件与设计文件不同。将测试激励文件与要综合的函数分别保存在不同文件内可便于清晰区分仿真流程与综合流程。如果测试激励文件与要综合的函数保存在相同文件内，则此文件应作为源文件（如下一步中所示）和测试激励文件添加到工程中。

图 11：工程测试激励文件



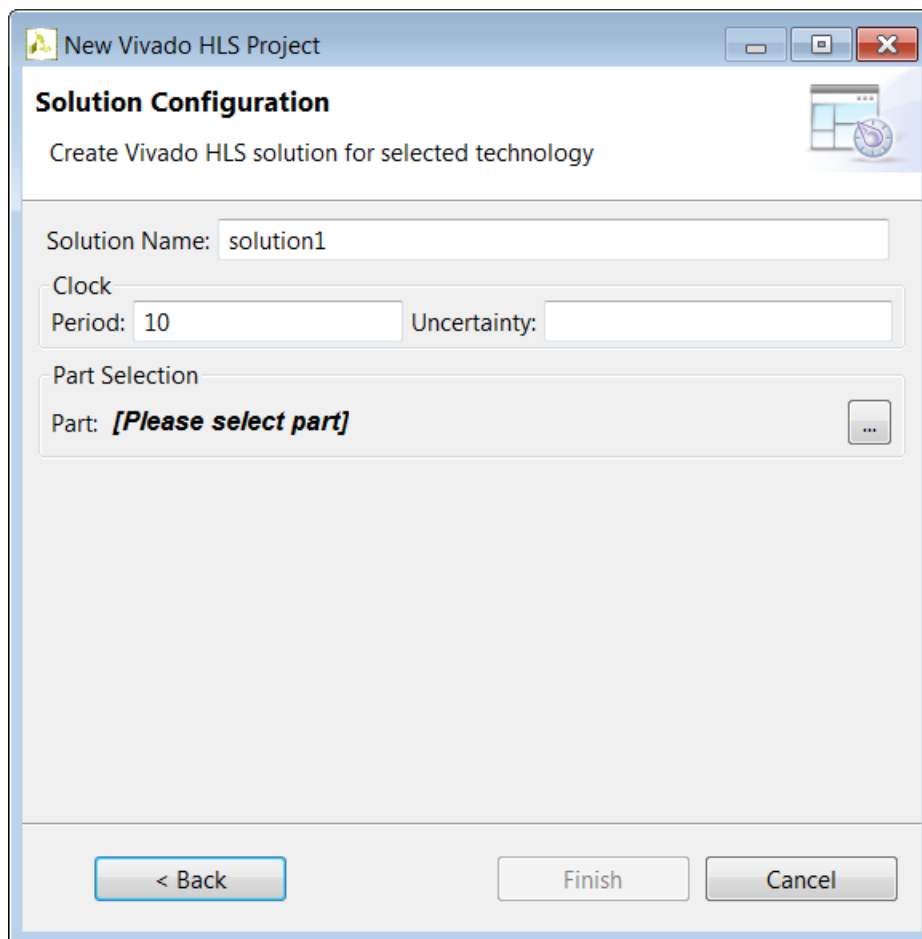
与 C 语言源文件相同，单击“Add Files”按钮以添加 C 语言测试激励文件，单击“Edit CFLAGS”按钮以包含任意 C 语言编译器选项。

除 C 语言源文件外，测试激励文件读取的所有文件都必须添加到工程内。在前图所示示例中，测试激励文件会打开 `in.dat` 文件以向设计提供输入激励，并打开 `out.golden.dat` 文件以读取期望的结果。由于测试激励文件会访问这些文件，因此这 2 个文件都必须包含在工程内。

如果在目录中存在测试激励文件，则可使用“Add Folders”按钮将整个目录（而不是单个文件）添加到工程中。

如果没有 C 语言测试激励文件，则无需在此处输入任何信息，“Next >”按钮会打开工程向导的最后一个窗口，以便您指定首个解决方案的详细信息，如下图所示。

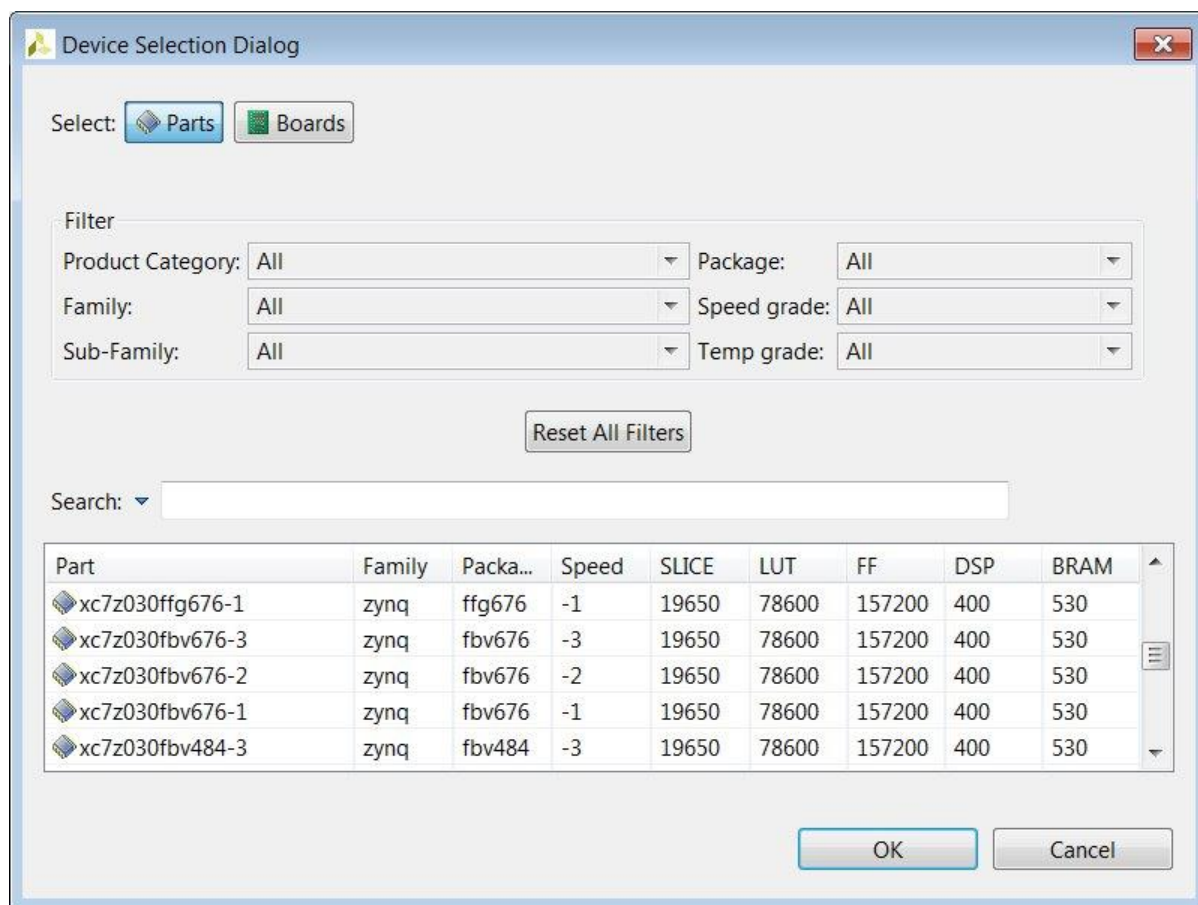
图 12：初始解决方案设置



新建工程向导中的最后一个窗口可供您指定首个解决方案的详细信息：

- “Solution Name”：Vivado HLS 提供初始默认名称 `solution1`，但您可为解决方案指定任意名称。
- “Clock Period”：时钟周期（以 ns 为单位来指定）或频率值（以 MHz 后缀来指定，如 150MHz）。
- “Uncertainty”：用于综合的时钟周期为时钟周期减去时钟不确定性。Vivado HLS 使用内部模型来估算每个 FPGA 的操作延迟。时钟不确定性值可提供可控裕度以防由于 RTL 逻辑综合、布局和布线而导致的信号线延迟增加。如果未以纳秒 (ns) 或百分比为单位来指定，则时钟不确定性默认为时钟周期的 12.5%。
- “Part”：单击此项可选择相应的基数，如下图所示。

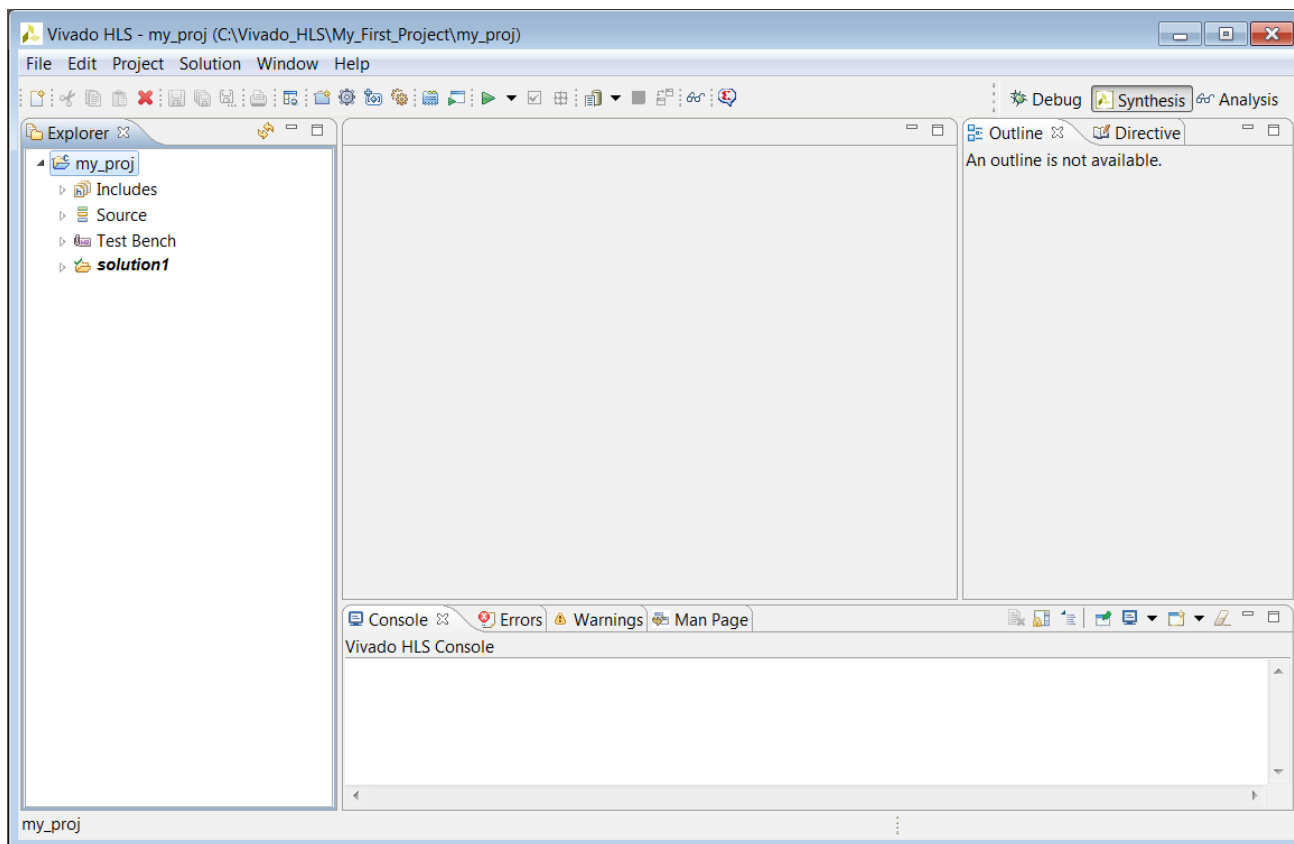
图 13：器件选择



选择 FPGA 目标。您可使用筛选功能来减少器件列表中的器件数量。如果目标为开发板，请在左上角指定开发板，这样器件列表将替换为受支持的开发板列表（并且 Vivado HLS 会自动选择正确的目标器件）。

单击“Finish”即可打开工程，如下图所示。

图 14: Vivado HLS GUI 中的新工程



Vivado HLS GUI 包含 4 个窗格：

- 左侧的 Explorer 窗格支持您浏览工程层级。在硬盘上的工程目录中存在相似的层级。
- 中间的 Information 窗格用于显示文件。可在 Explorer 窗格中双击文件以将其打开。
- 右侧的 Auxiliary 窗格可显示 Information 窗格中打开的任意文件的相关信息。
- 底部的 Console 窗格可显示运行 Vivado HLS 时的输出。

对 C 语言代码进行仿真

Vivado® HLS 流程中的验证可以分为 2 个独立进程。

- 综合前验证用于验证 C 语言程序是否正确实现了所需的功能。
- 综合后验证用于验证 RTL 是否正确。

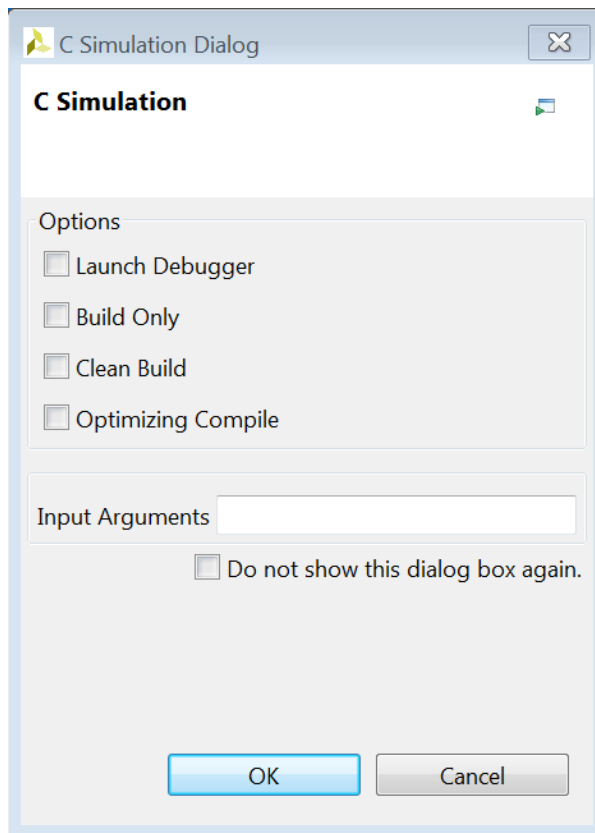
这两个进程都称为仿真：C 语言仿真和 C/RTL 协同仿真。

综合之前，应使用 C 语言仿真通过测试激励文件验证要综合的函数。C 语言测试激励文件包括顶层函数 `main()` 和要综合的函数。它可能包括其它函数。理想的测试激励文件具有以下属性：

- 测试激励文件可以自查，并验证要综合的函数的结果是否正确。
- 如果结果正确，测试激励文件将向 `main()` 返回 0 值。否则，测试激励文件应返回任何非零值

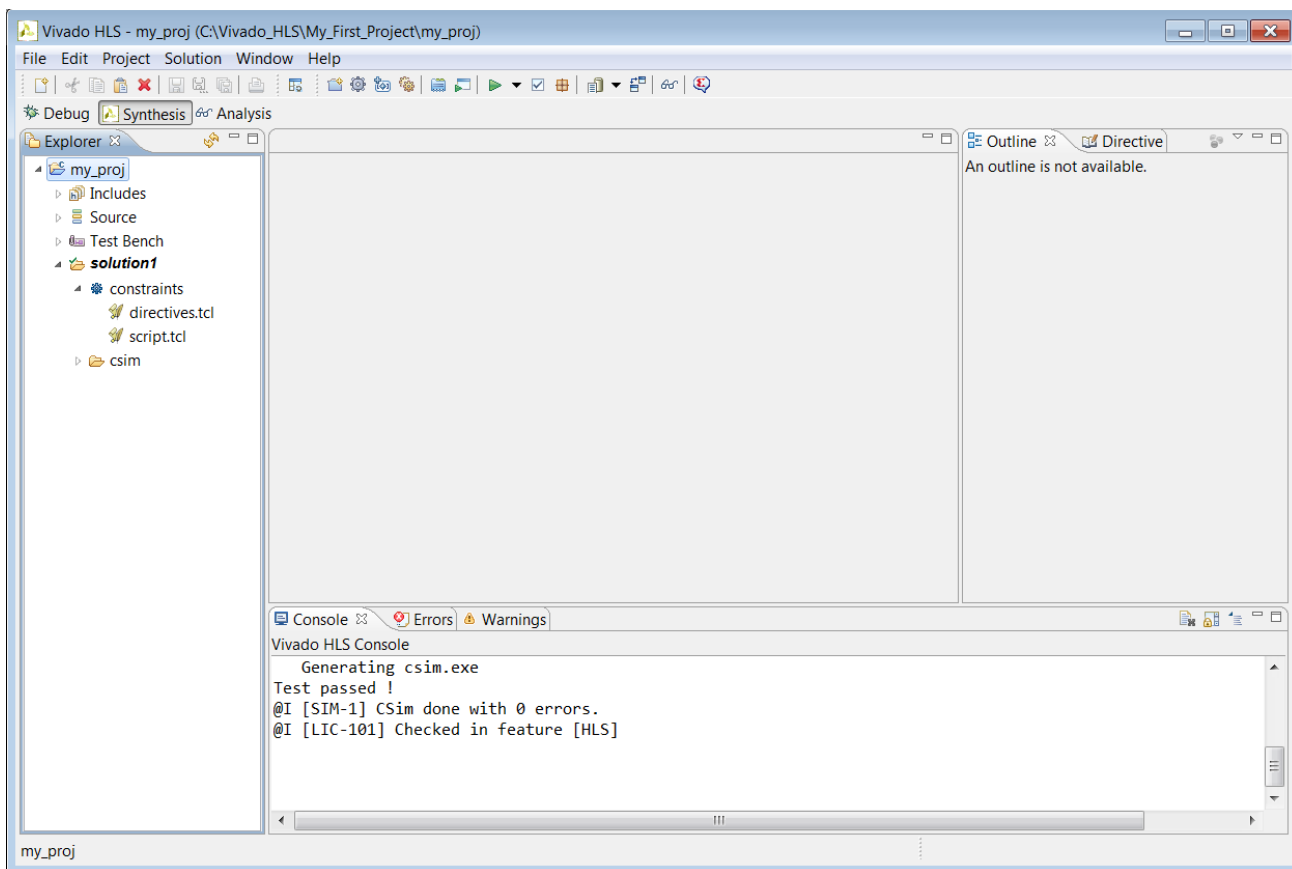
单击“Run C Simulation”工具栏按钮将打开“C Simulation”对话框，如下图所示。

图 15：“C 语言仿真 (C Simulation)”对话框



如果在该对话框中未选中任何选项，则将编译 C 语言代码并自动执行 C 语言仿真。结果如下图所示。C 语言代码成功仿真后，控制台窗口将显示一条消息，如下图所示。使用任何 `printf` 命令后，测试激励文件会在控制台中显示“Test Passed!”消息作为回应。

图 16：C 语言编译结果和构建结果

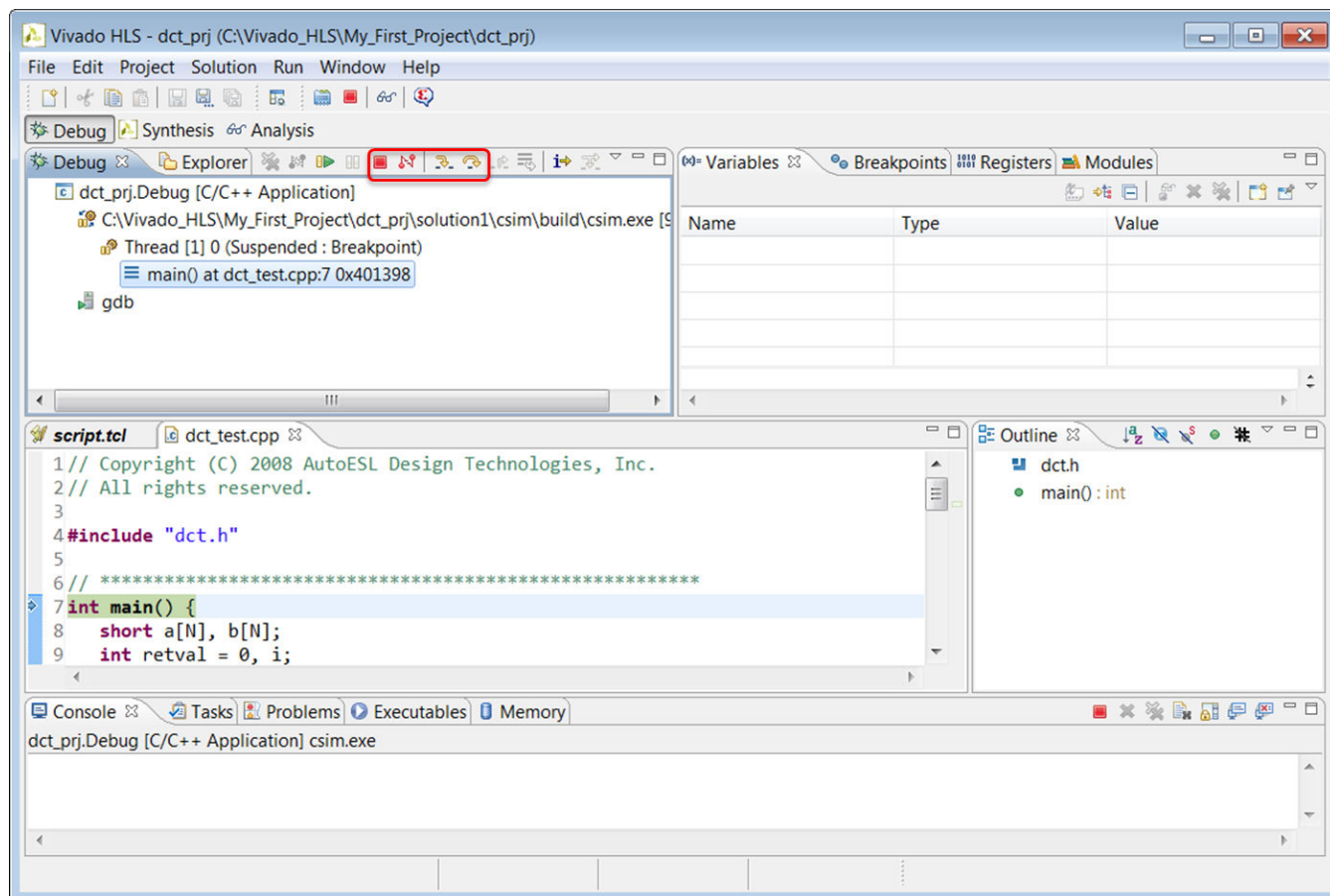


“C Simulation” 对话框中的其它选项包括：

- “Launch Debugger”：编译 C 语言代码并自动打开“Debug”透视图。在“Debug”透视图，可以使用“Synthesis”透视图按钮（左上方）从各窗口返回到“Synthesis”透视图。
- “Build Only”：编译 C 语言代码，但是不运行仿真。
- “Clean Build”：在编译代码之前，从工程中移除所有现有的可执行文件和对象文件。
- “Optimized Compile”：默认情况下，使用调试信息编译设计，以便在“Debug”透视图分析编译。编译设计时，此选项使用更高级别的最优化，但会移除调试器所需的所有信息。这会增加编译时间，但会减少仿真运行时间。
- “Compiler”：支持您选择使用 gcc/g++ 编译代码。

如果选择“Launch Debugger”选项，则窗口将自动切换到“Debug”透视图，并打开调试环境，如下图所示。这是功能齐全的 C 语言调试环境。使用步进按钮（下图中的红色框）可以单步执行代码、设置断点并直接查看变量的值。

图 17: C 语言调试环境

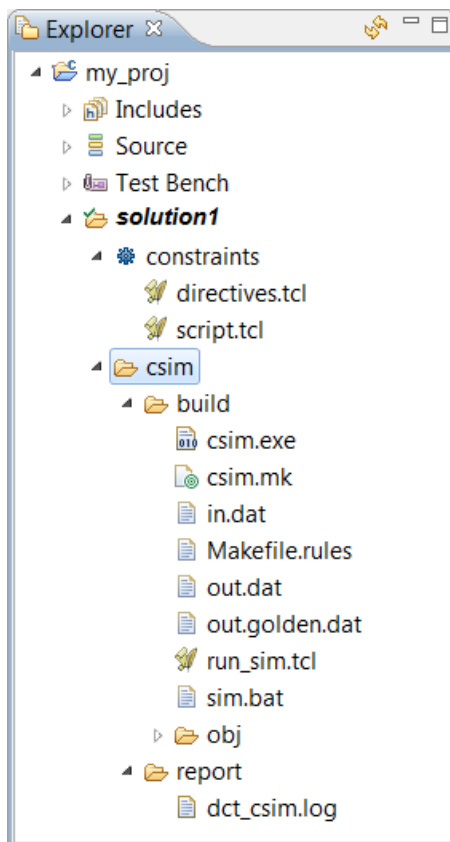


提示：单击“Synthesis”透视图按钮以返回到标准综合窗口。

查看 C 语言仿真输出

完成 C 语言仿真后就会在 solution 文件夹内创建 csim 文件夹，如下所示。

图 18: C 语言仿真输出文件



`csim/build` 文件夹是 C 语言仿真相关的所有文件的主要位置。

- 测试激励文件所读取的所有文件都会复制到此文件夹中。
- 在此文件夹中创建并运行 C 语言可执行文件 `csim.exe`。
- 测试激励文件写入的所有文件都是在此文件夹中创建的。

如果在“C Simulation”对话框中选中“Build Only”选项，那么会在此文件夹中创建 `csim.exe` 文件，但不执行此文件。通过从命令 shell 执行此文件来手动运行 C 语言仿真。在 Windows 上，可通过开始菜单来使用 Vivado® HLS 命令 shell。

文件夹 `csim/report` 包含 C 语言仿真的日志文件。

Vivado HLS 设计流程中的下一步是执行综合。

对 C 语言代码进行综合

本节中探讨了下列主题：

- 创建初始解决方案。
- 复查 C 语言综合的输出。
- 分析综合的结果。
- 创建新解决方案。

- 应用最优化指令。

创建初始解决方案

使用“C Synthesis”工具栏按钮  或菜单“Solution” → “Run C Synthesis”可对设计进行综合以完成 RTL 实现。综合过程中，会向控制台窗口回显消息。

此类消息包括用于显示综合过程进展情况的参考消息：

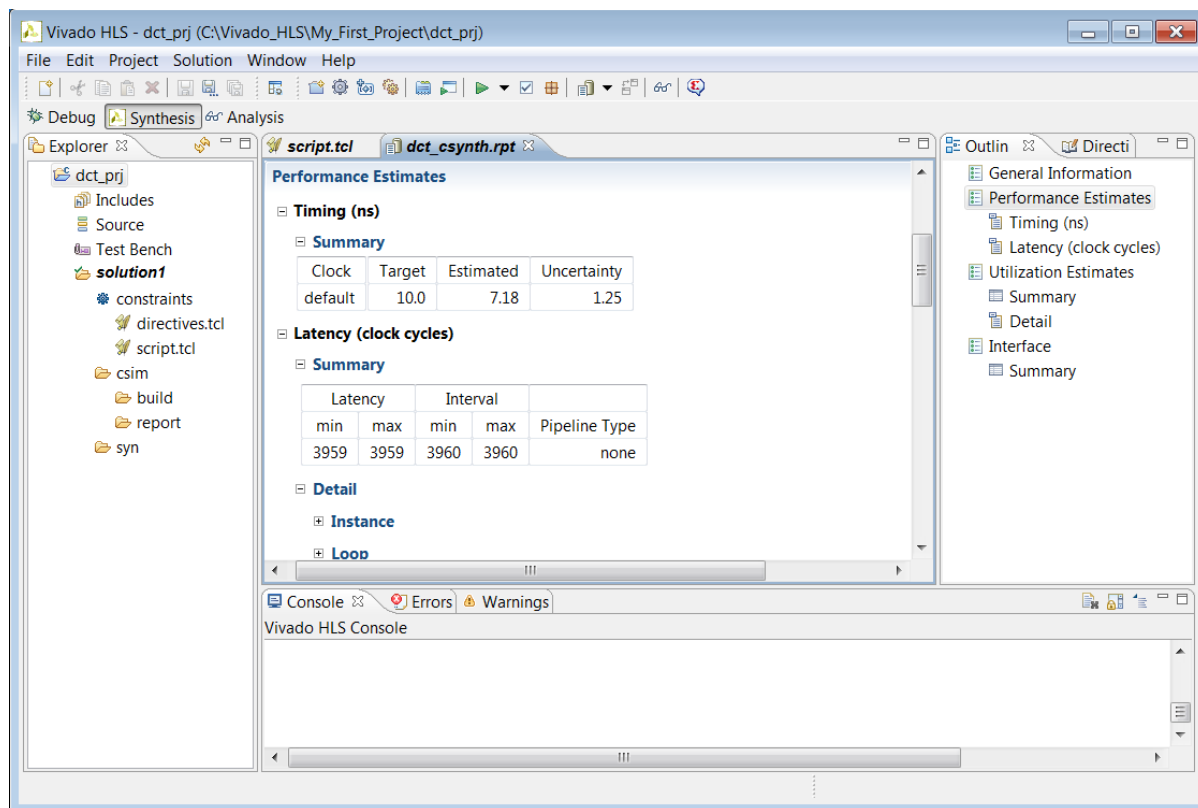
```
INFO: [HLS 200-10] Opening and resetting project
'C:/Vivado_HLS/My_First_Project/proj_dct'.
INFO: [HLS 200-10] Adding design file 'dct.cpp' to the project
INFO: [HLS 200-10] Adding test bench file 'dct_test.cpp' to the project
INFO: [HLS 200-10] Adding test bench file 'in.dat' to the project
INFO: [HLS 200-10] Adding test bench file 'out.golden.dat' to the project
INFO: [HLS 200-10] Opening and resetting solution
'C:/Vivado_HLS/My_First_Project/proj_dct/solution1'.
INFO: [HLS 200-10] Cleaning up the solution database.
INFO: [HLS 200-10] Setting target device to 'xc7k160tfbg484-1'
INFO: [SYN 201-201] Setting up clock 'default' with a period of 4ns.
```

在 GUI 中，部分消息可能包含指向补充信息的链接。在以下示例中，消息 XFORM 203-602 带有下划线，以表示存在超链接。单击此消息可提供有关发出此消息的原因以及可能的解决方法的更多详情。在此情况下，Vivado® HLS 会自动内联小型函数，可将 `INLINE` 指令与 `-off` 选项搭配使用以阻止此自动内联操作。

```
INFO: [XFORM 203-602] Inlining function 'read_data' into 'dct' (dct.cpp:85)
automatically.
INFO: [XFORM 203-602] Inlining function 'write_data' into 'dct'
(dct.cpp:90) automatically.
```

当综合完成后，顶层函数的综合报告会在信息窗格内自动打开，如下图所示。

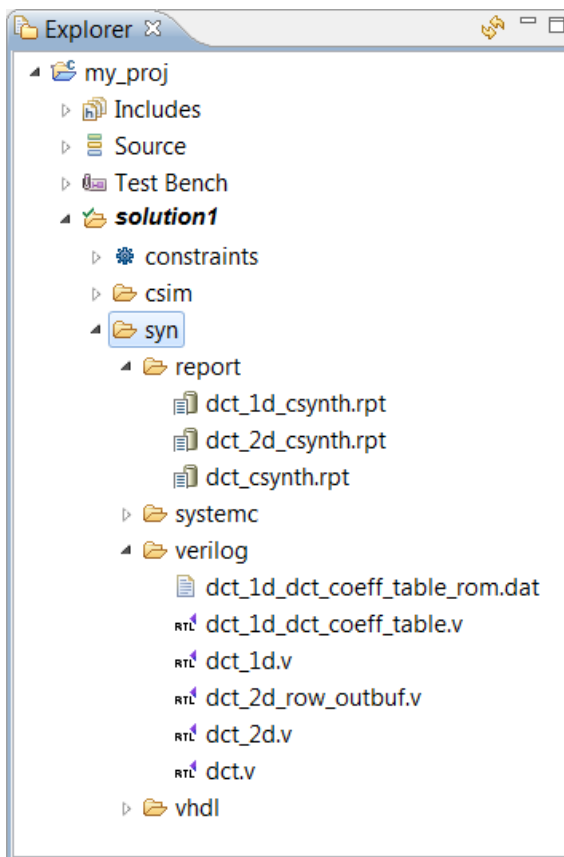
图 19：综合报告



复查 C 语言综合的输出

综合完成后，在 `solution` 文件夹中现已包含 `syn` 文件夹。

图 20: C 语言综合输出文件



`syn` 文件夹包含 4 个子文件夹。1 个 `report` 文件夹，另对应每一种 RTL 输出格式各 1 个文件夹。

`report` 文件夹包含顶层函数的报告文件，以及对应设计中每个子函数包含 1 个报告文件：前提是未使用 `INLINE` 指令内联函数，Vivado® HLS 也未自动内联函数。顶层函数的报告提供了有关整个设计的详细信息。

`verilog`、`vhdl` 和 `systemc` 文件夹包含输出 RTL 文件。前图显示了已展开的 `verilog` 文件夹。顶层文件与综合的顶层函数同名。在 C 语言设计中，针对每个函数（未内联）存在 1 个 RTL 文件。可能还有其它 RTL 文件可用于实现子块（块 RAM、已流水线的乘法器等）。



重要提示！ 赛灵思不建议将这些文件用于 RTL 综合。赛灵思建议改为使用已封装的 IP 输出文件，如本设计流程后文所述。请仔细阅读紧接在本注释后的文本。

如果 Vivado HLS 在设计中使用赛灵思 IP（例如，配合浮点设计使用），那么 RTL 目录包含用于在 RTL 综合期间创建 IP 的脚本。如果 `syn` 文件夹中的文件用于 RTL 综合，那么正确使用这些文件夹中存在的任意脚本文件的责任由您自行承担。如果使用封装 IP，则此进程由赛灵思设计工具自动执行。

分析 C 语言综合的结果

为分析 RTL 设计而提供的 2 项主要功能为：

- 综合报告
- “Analysis” 透视图

此外，如果您更喜欢在 RTL 环境中工作，Vivado® HLS 还会在 IP 封装进程中创建 2 个工程：

- Vivado Design Suite 工程
- Vivado IP integrator 工程

综合报告

当综合完成后，在信息窗格中会自动打开顶层函数的综合报告。此报告可提供有关 RTL 设计的性能和面积的详细信息。右侧的“Outline”选项卡可用于浏览此报告。

下表解释了综合报告中的类别。

表 1：综合报告类别

类别	描述
“General Information”	有关何时生成结果的详情、所使用的软件版本、工程名称、解决方案名称和技术详情。
“Performance Estimates” → “Timing”	目标时钟频率、时钟不确定性和估计可实现的最快时钟频率。
“Performance Estimates” → “Latency” → “Summary”	报告此块和此块中例化的任意子块的时延和启动时间间隔。 在 C 语言源代码中此级别上调用的每个子函数都是此 RTL 块中的实例（除非此函数已内联）。 时延表示生成输出所需的周期数。启动时间间隔是可应用新输入之前的时钟周期数。 如果没有任何 PIPELINE 指令，则时延比启动时间间隔少 1 个周期（写入最终输出时，读取下一个输入）。
“Performance Estimates” → “Latency” → “Detail”	此块中的实例（子函数）和循环的时延和启动时间间隔。如有任何循环包含子循环，则显示循环层级。 最小和最大时延值表示用于执行循环的所有迭代的时延。代码中的条件分支可能使用不同的最小值和最大值。 “迭代时延 (Iteration Latency)”是单次循环迭代的时延。 如果循环具有变量时延，则时延值无法判定并显示为问号 (?)。请参阅该表后的文本。 在达成的实际启动时间间隔旁会显示任何指定的目标启动时间间隔。 行程计数显示的是循环的迭代总数。
“Utilization Estimates” → “Summary”	此部分报告显示了用于实现设计的资源（LUT、触发器、DSP48）。
“Utilization Estimates” → “Details” → “Instance”	此处指定的资源供该层级例化的子块使用。 如果设计不含任何 RTL 层级，则不会报告任何实例。 如果存在任何实例，单击实例名称会打开该实例的综合报告。
“Utilization Estimates” → “Details” → “Memory”	此处所列资源为该层级的内存实现所使用的资源。 Vivado HLS 会将单端口 BRAM 报告为使用 1 个内存 bank，将双端口 BRAM 报告为使用 2 个内存 bank。
“Utilization Estimates” → “Details” → “FIFO”	此处所列资源为该层级实现的任意 FIFO 所使用的资源。
“Utilization Estimates” → “Details” → “Shift Register”	映射到赛灵思 SRL 组件的所有移位寄存器汇总。 在 RTL 综合期间，可能发生更多到 SRL 组件的映射。
“Utilization Estimates” → “Details” → “Expressions”	此类别显示的是供任何表达式（例如，当前层级的乘法器、加法器和比较器）使用的资源。 其中显示了表达式的输入端口的位宽。
“Utilization Estimates” → “Details” → “Multiplexors”	此部分报告显示了用于在此层级实现多路复用器的资源。 其中显示了多路复用器的输入宽度。
“Utilization Estimates” → “Details” → “Register”	此处显示了此层级的所有寄存器列表。此报告包含寄存器位宽。

表 1：综合报告类别 (续)

类别	描述
“Interface Summary” → “Interface”	此部分显示了将函数实参综合到 RTL 端口中的方式。 RTL 端口名称根据其协议和源对象进行分组：这些 RTL 端口是使用有状态的 I/O 协议综合该源对象时创建的。

某些赛灵思器件使用堆叠硅片互联 (SSI) 技术。在此类器件中，全部可用资源被划分为多个超级逻辑区域 (SLR)。选择 SSI 技术器件作为目标技术时，利用率报告将包含有关 SLR 利用率和器件总利用率的详细信息。



重要提示！ 使用 SSI 技术器件时，重要的是确保 Vivado HLS 创建的逻辑适用于单一 SLR 内。

Vivado HLS 新用户常见的综合报告问题如下图所示。时延值全部显示为“?”（问号）。

图 21：综合报告

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
default	8.00	6.75	1.00

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
?	?	?	?	none

Detail

Instance

Loop

Loop Name	Latency		Iteration Latency	Initiation Interval		Trip Count	Pipelined
	min	max		achieved	target		
- YUV_SCALE_LOOP_X	?	?	?	-	-	?	no
+ YUV_SCALE_LOOP_Y	?	?	4	-	-	?	no

Vivado HLS 通过执行分析来判定每个循环的迭代数。如果循环迭代限制为变量，Vivado HLS 将无法判定最大迭代次数上限。

在以下示例中，for 循环的最大迭代次数由输入 `num_samples` 的值来确定。在 C 语言函数中不定义 `num_samples` 的值，该值从外部进入函数。

```
void foo (char num_samples, ...);

void foo (num_samples, ...) {
    int i;
    ...
}
```

```
loop_1: for(i=0;i< num_samples;i++) {  
    ...  
    result = a + b;  
}  
}
```

如果设计的时延或吞吐量从属于含变量索引的循环，那么 Vivado HLS 会将循环时延报告为未知（在报告中以问号“?”来表示）。

TRIPCOUNT 指令可应用于循环以手动指定循环迭代次数，并确保报告包含实用的数值。-max 选项会将循环迭代的最大迭代次数告知 Vivado HLS，-min 选项可指定执行的迭代的最小次数。

注释：TRIPCOUNT 指令不影响综合结果。

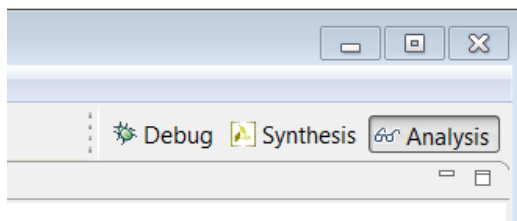
行程计数 (tripcount) 值仅用于报告，以确保 Vivado HLS 所生成的报告可针对时延和时间间隔显示有意义的范围。这样还可对不同解决方案进行有意义的比较。

如果在代码中使用 C 语言 assert 宏，那么 Vivado HLS 可将其用来自动判定循环限制，并根据这些限制创建大小精确的硬件。

分析透视图

除综合报告外，您还可使用“Analysis”透视图来分析结果。要打开“Analysis”透视图，请单击“Analysis”按钮，如下图所示。

图 22：“Analysis”透视图



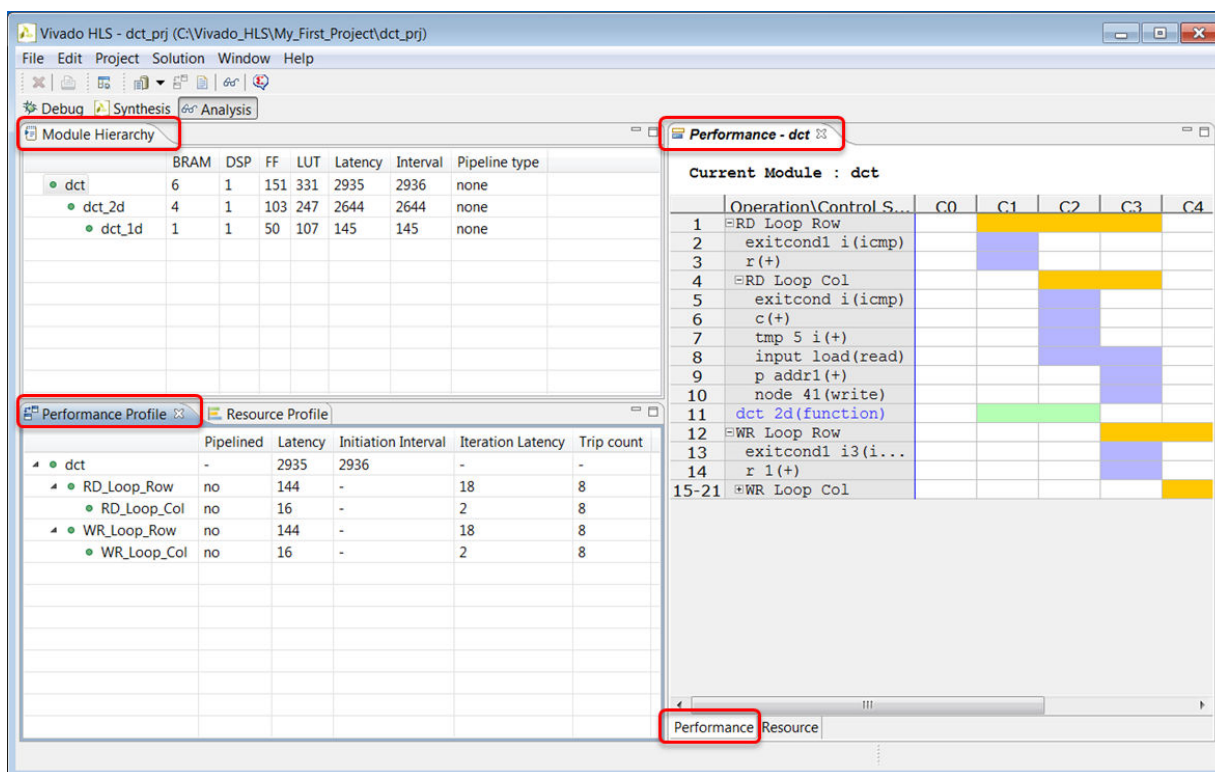
“Analysis”透视图可提供设计性能和资源的表格化视图和图形化视图，并支持 2 个视图之间的交叉引用。下图显示了首次打开“Analysis”透视图时的默认窗口配置。

“Module Hierarchy”窗格提供了整个 RTL 设计的概述。

- 此视图可用于浏览整个设计层级。
- “Module Hierarchy”窗格可显示 RTL 层级中每个块的资源 and 时延份额。

下图显示的 dct 设计使用 6 个块 RAM、约 300 个 LUT 且时延约为 3000 个时钟周期。子块 dct_2b 在总数中占 4 个块 RAM、约 250 个 LUT 以及约 2600 个时延周期。显而易见，此设计中的大部分资源和时延都源于子块 dct_2d，因此应首先分析此块。

图 23: Vivado HLS GUI 中的 “dct_2d” 透视图

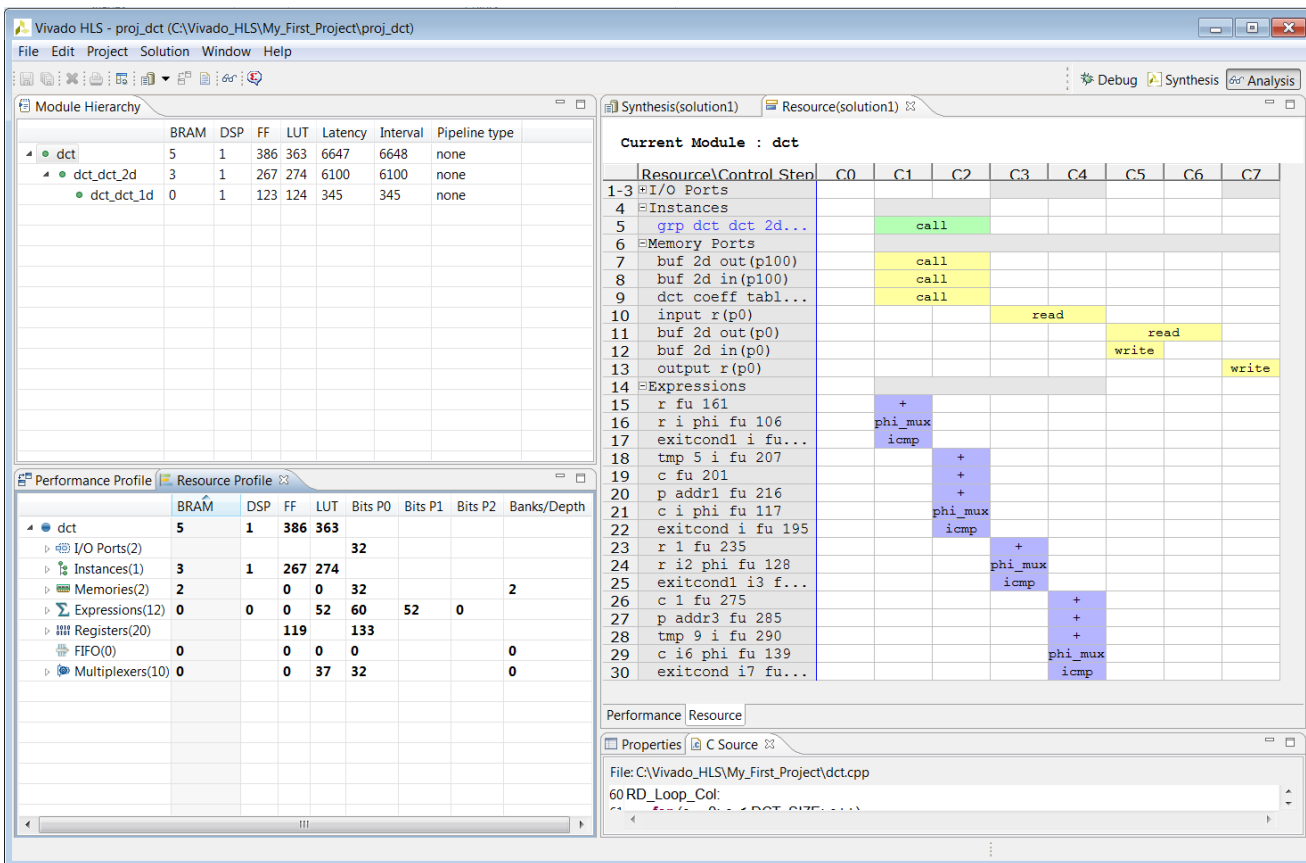


“Performance Profile” 窗格可提供有关当前 “Module Hierarchy” 窗格中选中的块的性能详细信息，在此例中，“Module Hierarchy” 窗格中高亮的是 `dct` 块。

- 此块的性能是其包含的子块以及该层级内包含的任意逻辑的函数。“Performance Profile” 窗格显示了该层级中影响总体性能的各项。
- 性能是按时延和启动时间间隔来测量的。此窗格还包含有关此块是否采用流水线化的详细信息。
- 在此示例中，您可看到有 2 个循环（`RD_Loop_Row` 和 `WR_Loop_Row`）作为该层级的逻辑来实现，这 2 个循环都包含子循环并且在时延中各占 144 个时钟周期。将这 2 个循环的时延与同样位于 `dct` 内的 `dct_2d` 的时延相加即可得到 `dct` 块的总时延。

“Analysis” 透视图还支持您对资源利用率进行分析。下图显示了资源剖析和资源窗格。

图 24：含资源剖析的“Analysis”透视图



“Resource Profile” 窗格可显示在此层级使用的资源。在此示例中，您可看到大部分资源都源于实例：即此块内例化的块。

展开表达式即可看到该层级的资源主要用于实现加法器。

“Resource” 窗格可显示所使用的运算的当前状态。在此示例中，所有加法器运算都与不同加法器资源相关联。不存在加法器共享。每条水平线上存在多个加法运算，这表明在不同状态或不同时钟周期内多次使用了相同资源。

在内存访问的相同周期内使用这些加法器，并且每个内存均有专用的加法器。可通过使用与 C 语言代码的交叉关联来加以确认。

Schedule Viewer

调度查看器 (Schedule Viewer) 可为您提供已综合的 RTL 的详细视图。您可识别妨碍并行化、时序违例和数据依赖关系的任意循环依赖关系。

- 此查看器可通过浏览右侧“Analysis”视图来访问。
- 浏览“Module Hierarchy”窗口、右键单击并选择“Open Schedule Viewer”即可查看每个单独的块的调度。
“Module Hierarchy”可直接指明任意 II 或时序违例。对于时序违例，层级窗口还将显示特定模块中观测到的总体时序余裕量。

注释：使用窗口菜单按钮即可在“Module Hierarchy”中筛选显示 II 或时序违例的块。

在调度查看器主窗口中：

- 纵轴显示的是操作和循环名称。
- 操作按拓扑顺序执行，即行 n 上的操作只能由前一行的操作驱动并且只能驱动后一行中的操作输入。

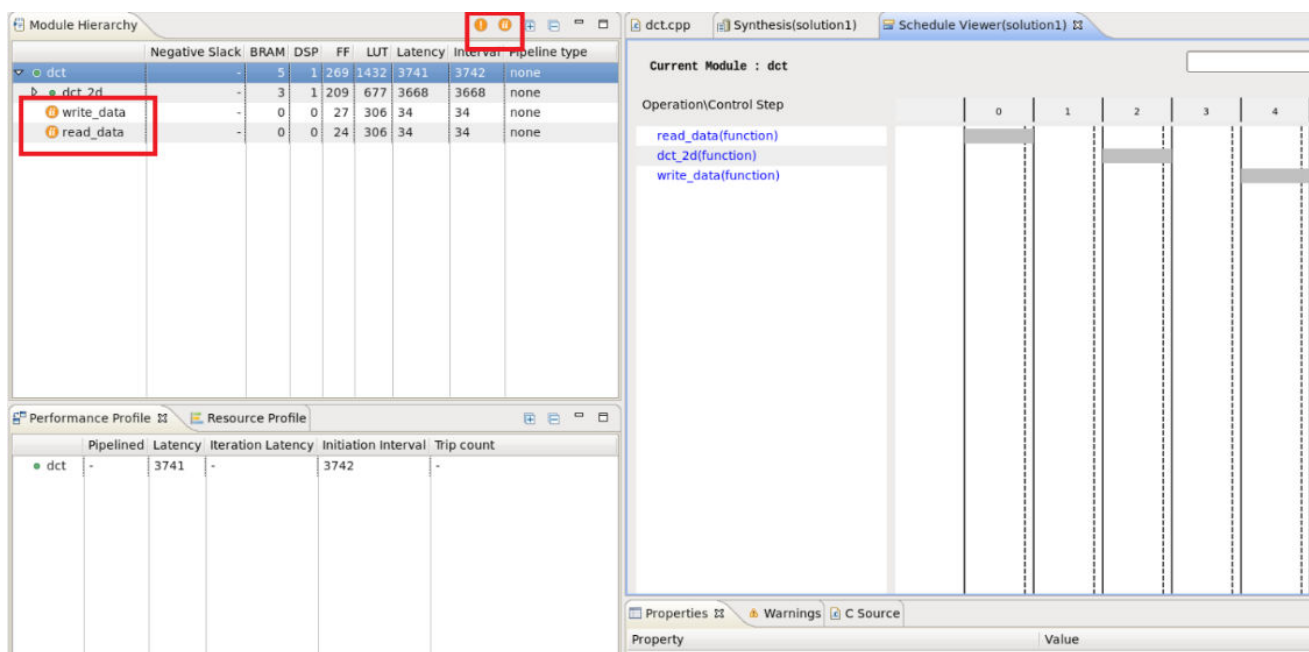
在以下示例中，按如下顺序仅显示顶层函数：

- read_data
- dct_2d
- write_data

- 横轴上的灰色实心块按连续顺序显示周期。
- 竖轴虚线按比例显示由于时钟不确定性而保留的时钟周期部分。此时间即工具保留用于 Vivado 后端进程（如布局布线）的时间。
- 表中对应每项操作显示一个灰色框。一般，根据操作延迟占总时钟周期百分比来限定此框的横向大小。对于函数调用（如此示例中所示），提供的周期信息等同于 op 时延。在此情况下，read_data 函数的 op 时延为 1。
- 通过穿过 op 框的直线来以可视化方式表示多周期操作。在调度查看器菜单右上角的调度查看器“Legend”按钮中列出了所有不同的可视化元素。
- 最重要的是，任意操作都有关联的源位置。双击操作即可在输入源代码中高亮操作源。

对于函数调用，提供的周期信息为 op 时延。在此情况下，read_data 函数的 op 时延为 1，如以下“Properties”选项卡中所示。

图 25：调度查看器



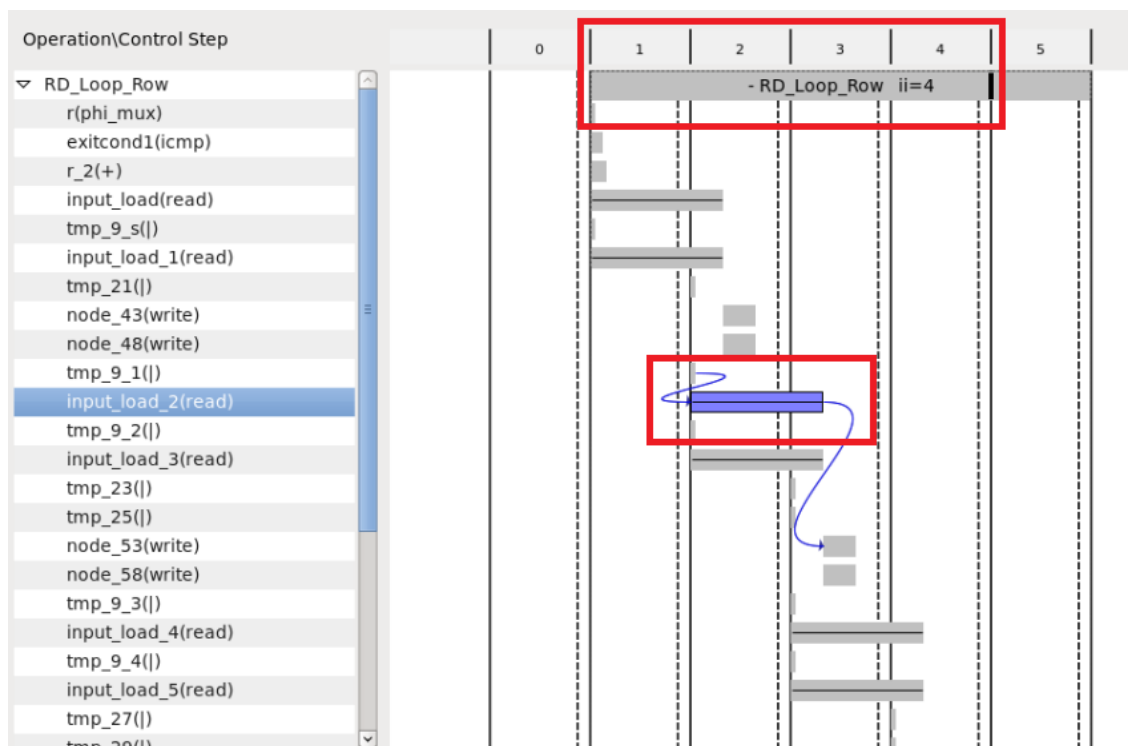
- 浏览至“Module Hierarchy”中的函数，找到名为 read_dataRD_Loop_Row 的循环。
 - 这是流水线化循环，在循环栏中已显式声明启动时间间隔 (II)。所有流水线化循环均处于以可视化方式展开状态，这表示在调度查看器中显示 1 次完整迭代。II 定义的重叠以循环标记上加粗的时钟边界来标记。
 - 单次迭代的总时延等同于循环标记所覆盖的周期数。在此情况下为 5 个周期 (1-5)。
- 时序违例

下图中存在时序违例。可从选定的模块层级入口上下文菜单浏览时序违例视图，或者也可以使用调度查看器菜单中的聚焦下拉菜单来浏览此视图，如下图左侧窗格中所示。

时序违例是一条操作路径，其所需时间超过可用时钟周期。为了以可视化方式直观展示此定义，以下通过一个加长周期图示来直观展示存在问题的周期，其中实际周期边界已移出，并显示一个不透明框，此框全部属于同一个周期。

默认情况下，其中可显示关键时序路径中的每项操作之间的所有依赖关系（蓝色线条）。

图 26：导致违例的操作

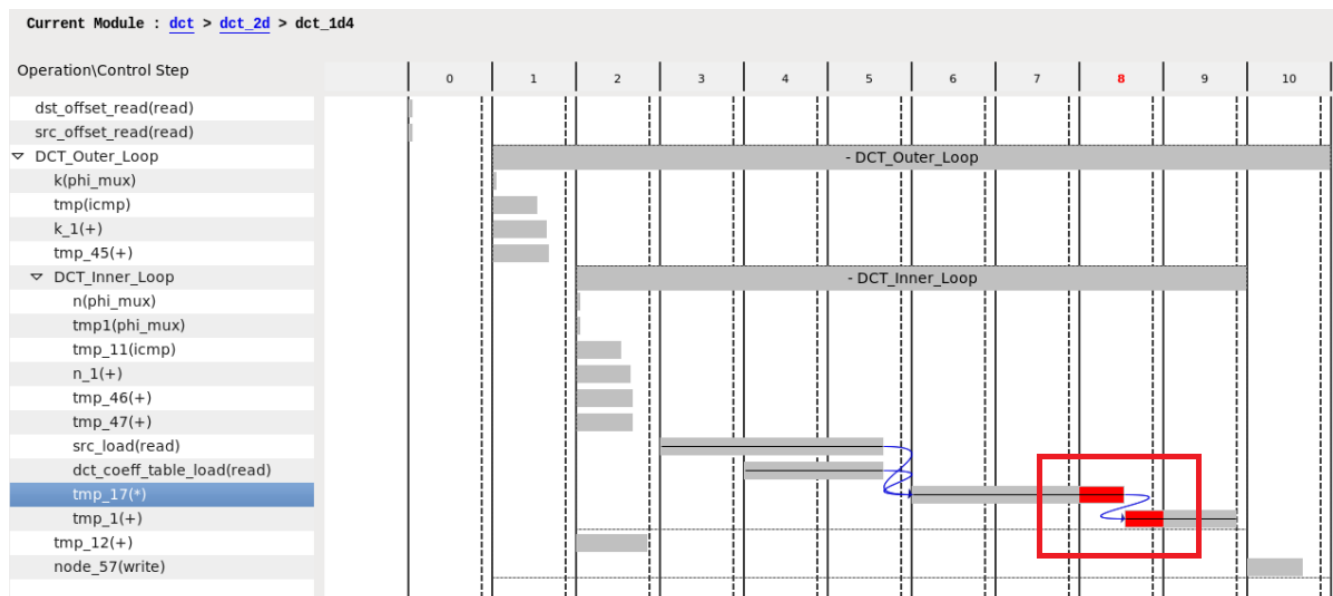


- 此查看器能够显示常见运算符依赖关系。选择某项操作时，其中会显示蓝色箭头，以高亮特定运算符依赖关系。这使您能够对数据依赖关系执行详细分析。
- II 违例分析显示在另一个特殊聚焦视图中。如有模块包含此类违例，调度查看器可通过“Module Hierarchy”中的上下文菜单或者查看器中的聚焦下拉菜单来显示此违例。

如下图所示，有一条生成完整 II 的路径。这说明必须先计算某个值，然后才能开始下一次迭代，并且必须缩短此路径才能降低 II。

为了识别源代码中的操作，请双击此操作，这样源代码查看器就会显示并标识源代码中对象的根。

图 27：时序违例



- 调度查看器菜单栏中的筛选按钮  支持您动态筛选调度查看器中显示的操作。可按类型或者按分群的操作来进行筛选。
 - 按类型筛选可根据操作的功能来限制显示的操作。例如，仅将加法器、乘法器和函数调用可视化会移除“和”与“或”之类的所有小操作。
 - 按群集过滤便于充分利用调度程序对基本操作进行分组然后将其作为单个组件来调度的功能。可启用群集筛选设置来为群集上色，甚至可在查看器内将群集折叠为单一大型操作。这样即可提供更简洁的调度视图。

数据流查看器

如果已向函数应用 `DATAFLOW` 指令，“Analysis”透视图会提供数据流查看器以显示设计结构。此视图可提供数据流图形结构表示法，以显示不同流程和底层生产者/使用者连接。

在下图中，“dct”函数旁的  图标表示数据流视图可供查看。右键单击该函数即可打开数据流视图。

图 28：数据流视图

Module Hierarchy							
	BRAM	DSP	FF	LUT	Latency	Interval	Pipeline type
dct	4	8	1506	620	508	375	dataflow
Open Schedule Viewer				445	374	374	none
Open Dataflow Viewer				62	66	66	none
read_data	0	0	29	54	66	66	none

“Analysis”透视图属于高度交互式功能。如需了解有关“Analysis”透视图的更多信息，请参阅《Vivado Design Suite 教程：高层次综合》(UG871)的“设计分析”部分。



提示：请谨记，即使使用 Tcl 流程来创建设计，仍可在 GUI 中打开工程，并使用“Analysis”透视图来分析设计。

使用“Synthesis”透视图按钮即可返回至综合视图。

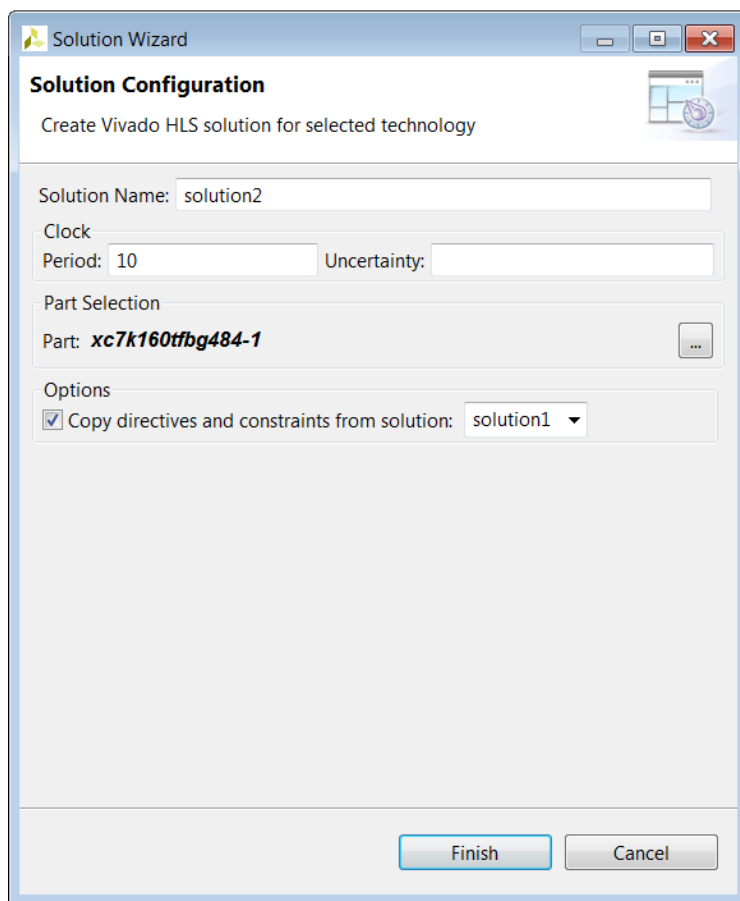
通常完成设计分析后，即可创建新解决方案以应用最优化指令。对此设计使用新解决方案即可对不同解决方案进行比较。

创建新解决方案

Vivado HLS 的最常见用例是用于创建初始设计，然后执行最优化以满足期望的面积和性能目标。解决方案可提供一种便利的方式来确保保留先前综合运行的结果并开展比对。

使用“New Solution”工具栏按钮  或菜单上的“Project” → “New Solution”来创建新解决方案。这样会打开“解决方案向导 (Solution Wizard)”，如下图所示。

图 29：新建解决方案向导



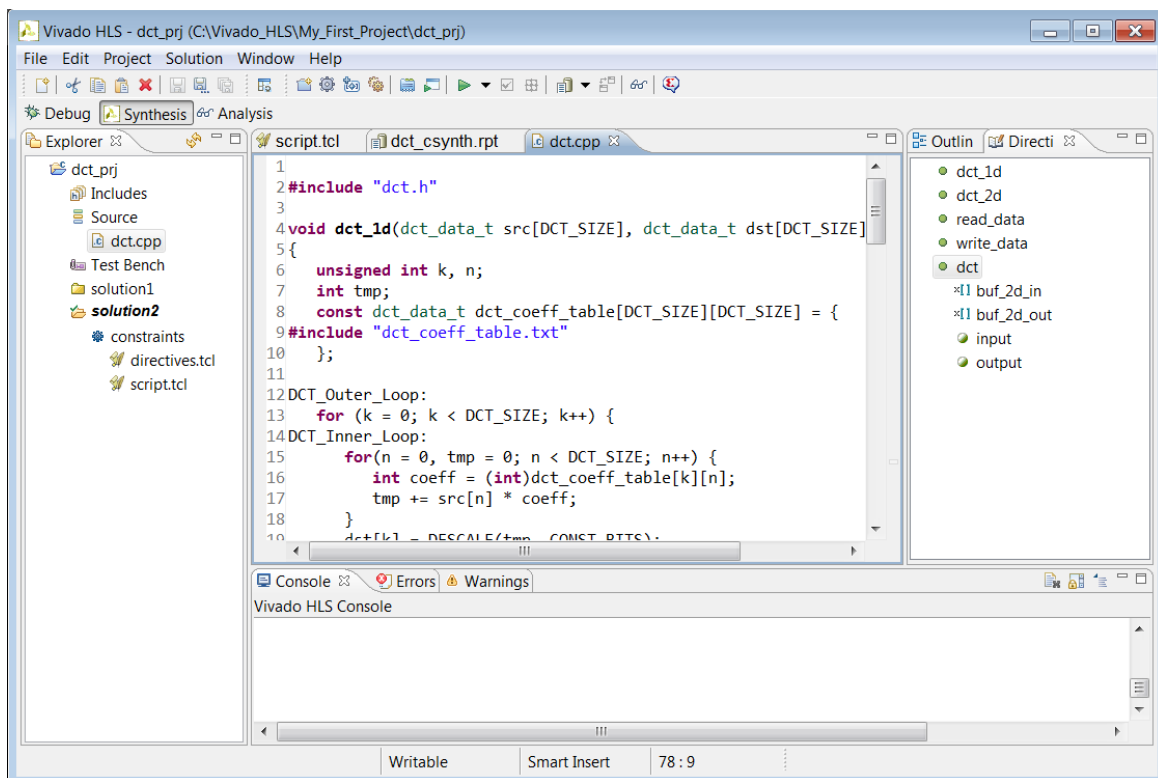
“Solution Wizard”所含选项与“新建工程 (New Project)”向导中最后一个窗口中的选项相同，但附加了一个选项，支持将应用于现有解决方案的任何指令和自定义约束轻松复制到新解决方案，并在其中对其进行修改或删除。

创建新解决方案后，即可添加最优化指令（或者对从先前解决方案复制的指令进行修改）。下一章节解释了如何向解决方案添加指令。使用配置选项可应用自定义约束，欲知详情，请参阅[对设计进行最优化](#)。

应用最优化指令

添加最优化指令的第 1 步是在“信息 (Information)”窗格内打开源代码。如下图所示，展开位于“Explorer”窗格顶部的“Source”容器，双击源文件以将其打开，并在“Information”窗格内对其进行编辑。

图 30：源代码和指令



在“Information”窗格中激活源代码后，选择右侧的“Directives”选项卡来为文件显示并修改指令。“Directives”选项卡包含当前打开的源代码中可对其应用指令的所有对象和作用域。

注释：要对其它 C 语言文件中的对象应用指令，必须打开此文件并在“Information”窗格中将其激活。

虽然可在 Vivado HLS GUI 中选择对象并应用指令，但 Vivado HLS 会将所有指令应用于包含该对象的作用域。例如，您可将 INTERFACE 指令应用于 Vivado HLS GUI 中的接口对象。Vivado HLS 会将该指令应用于顶层函数（作用域）以及该指令中标识的接口端口（对象）。在以下示例中，foo 函数上的 data_in 端口已指定为 AXI4-Lite 接口：

```
set_directive_interface -mode s_axilite "foo" adata_in
```

您可向以下对象和作用域应用最优化指令：

- 接口

向接口应用指令时，Vivado HLS 会将该指令应用于顶层函数，因为顶层函数是包含该接口的作用域。

- 函数

向函数应用指令时，Vivado HLS 会将该指令应用于函数作用域内的所有对象。任意指令的效果都将止于函数的下一个层级。唯一例外是需递归选项的指令，例如，用于以递归方式展开层级中的所有循环的 PIPELINE 指令。

- 循环

对循环应用指令时，Vivado HLS 会将该指令应用于循环作用域内的所有对象。例如，如果对循环应用 LOOP_MERGE 指令，那么 Vivado HLS 会将该指令应用于该循环内的所有子循环，但不会将其应用于该循环本身。

注释：指令应用到的循环不会与相同层级内的同辈循环合并。

- 数组

对数组应用指令时，Vivado HLS 会将该指令应用于包含该数组的作用域。

- 区域

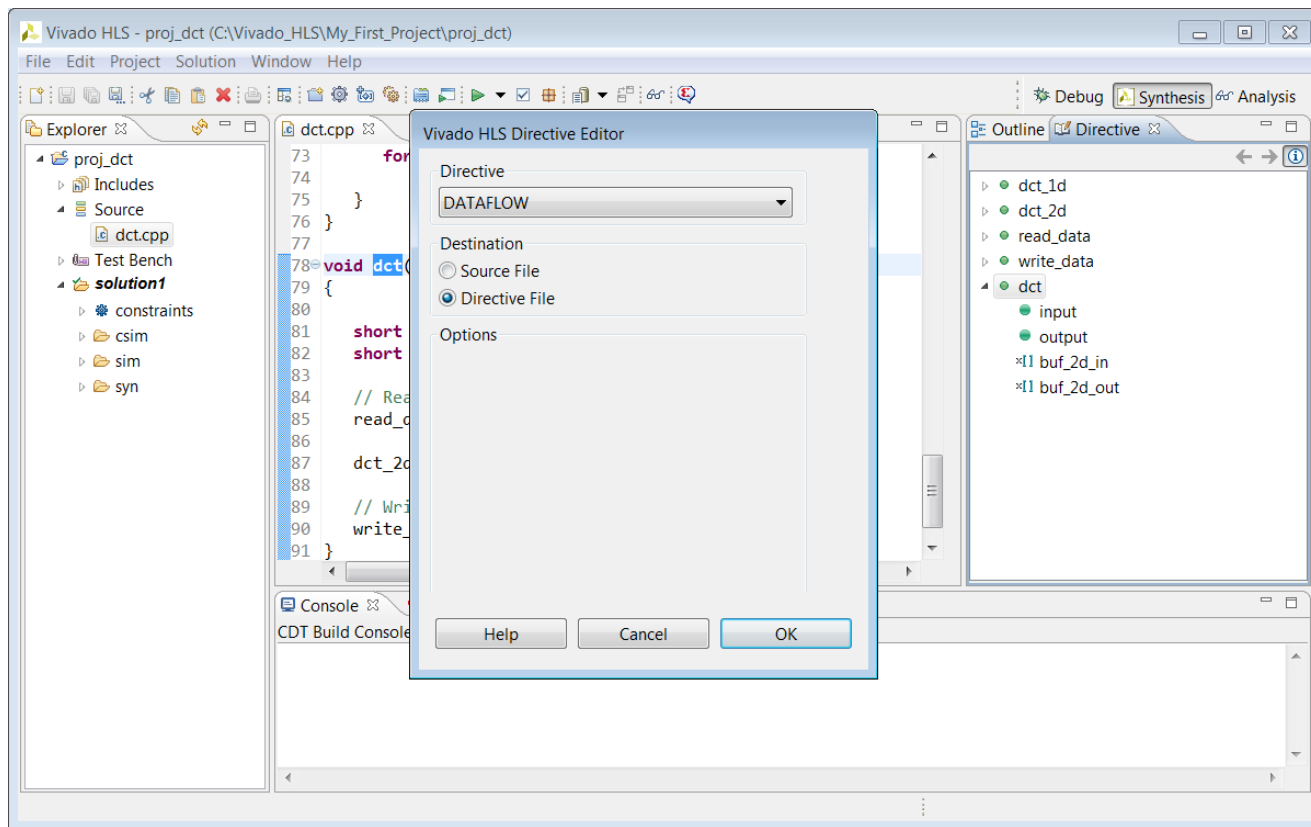
对区域应用指令时，Vivado HLS 会将该指令应用于区域的整个作用域。区域即以 2 个括号扩起的任意面积。例如：

```
{
    the scope between these braces is a region
}
```

注释：您可按向函数和循环应用指令的相同方式来向区域应用指令。

要应用指令，请选中“Directives”选项卡内的对象、右键单击并选择“Insert Directive”以打开“Directives Editor”对话框。从下拉菜单中选择相应的指令。下拉菜单仅显示可供您添加到选定对象或作用域的指令。例如，如果您选择数组对象，那么下拉菜单不会显示 PIPELINE 指令，因为无法对数组进行流水线化。下图显示了向 DCT 函数添加 DATAFLOW 指令的过程。

图 31：添加指令



使用 Tcl 命令或嵌入式编译指示

在 Vivado HLS 的“指令编辑器 (Directive Editor)”对话框中，可指定以下任一“目标 (Destination)”设置：

- “Directive File”：Vivado HLS 将指令作为 Tcl 命令插入解决方案目录中的 directives.tcl 文件。
- “Source File”：Vivado HLS 将指令作为编译指示直接插入 C 语言源文件。

下表描述了这 2 种方法的优缺点。

表 2：Tcl 命令对比编译指示

指令格式	优势	缺点
指令文件 (Tcl 命令)	每个解决方案都包含独立指令。此方法适合设计探索。 如果对任意解决方案进行重新综合，那么仅应用该解决方案中指定的指令。	如果将 C 语言源文件传输给第三方或归档，那么必须包含 directives.tcl 文件。 如果要重现结果，directives.tcl 文件是必需的。
源代码 (编译指示)	最优化指令嵌入 C 语言源代码。 适合将 C 语言源文件作为 C IP 交付给第三方。无需其它文件来重现相同结果。 此方法适用于不太可能改变的指令，如 TRIPCOUNT 和 INTERFACE 等。	如在代码中嵌入最优化指令，则在重新综合时会将其自动应用于每个解决方案。

指定编译指示实参的值时，可使用字面值（如 1、55 或 3.14），或者使用 #define 来传递宏。以下示例显示了含字面值的编译指示。

```
#pragma HLS ARRAY_PARTITION variable = k_matrix_val  cyclic  factor=5
```

该示例使用已定义的宏：

```
#define E 5
#pragma HLS ARRAY_PARTITION variable = k_matrix_val  cyclic  factor=E
```



重要提示！ 请勿使用用户定义的宏来指定编译指示的值，不支持使用此类宏。

编译指示验证

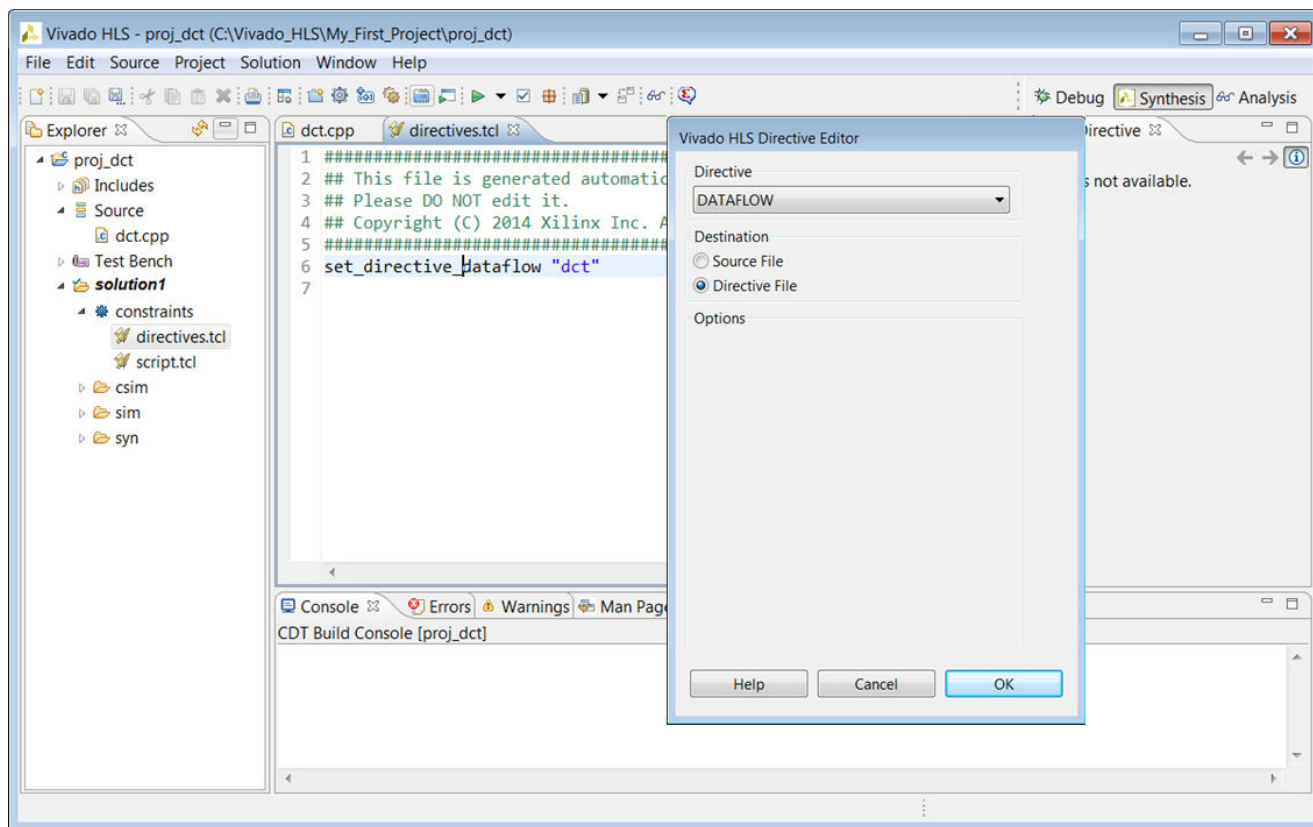
执行 C 语言综合期间，该工具会验证变量、函数和循环上的编译指示。此验证还包含编译指示冲突。

例如，声明数组时，默认情况下它会映射到块 RAM。您可对数组进行分区或重塑，但这些选项互斥。如果您在相同变量上错误指定数组分区和重塑，那么该工具会报错，综合将失败，并显示以下消息：

```
WARNING: [XFORM 203-180] Applying partition directive (core.cpp:12:1) and
reshape
directive (core.cpp:13:1) on the same variable 'A' (core.cpp:11) may lead
to
unexpected synthesis behaviors. INFO: [XFORM 203-131] Reshaping array 'A'
(core.cpp:11) in dimension 1 completely. ERROR: [XFORM 203-103] Cannot
partition
array 'A' (core.cpp:11): variable is not an array. ERROR: [HLS 200-70] Pre-
synthesis
failed. command 'ap_source' returned error code
```

下图显示了向指令文件 (Directive File) 添加 DATAFLOW 指令的过程。directives.tcl 文件位于解决方案 constraints 文件夹内，可使用生成的 Tcl 命令在“信息 (Information)”窗格中打开。

图 32：添加 Tcl 指令



将指令作为 Tcl 命令来应用时，Tcl 命令会指定作用域或者指定作用域和该作用域内的对象。对于循环和区域，Tcl 命令要求标记这些作用域。如果循环或区域当前没有标记，那么会弹出对话框要求提供标记。

以下显示了含标记和不含标记的循环和区域的示例。

```
// Example of a loop with no label
for(i=0; i<3;i++ {
    printf( "This is loop WITHOUT a label \n" );
}

// Example of a loop with a label
My_For_Loop:for(i=0; i<3;i++ {
    printf( "This loop has the label My_For_Loop \n" );
}

// Example of a region with no label
{
    printf( "The scope between these braces has NO label" );
}

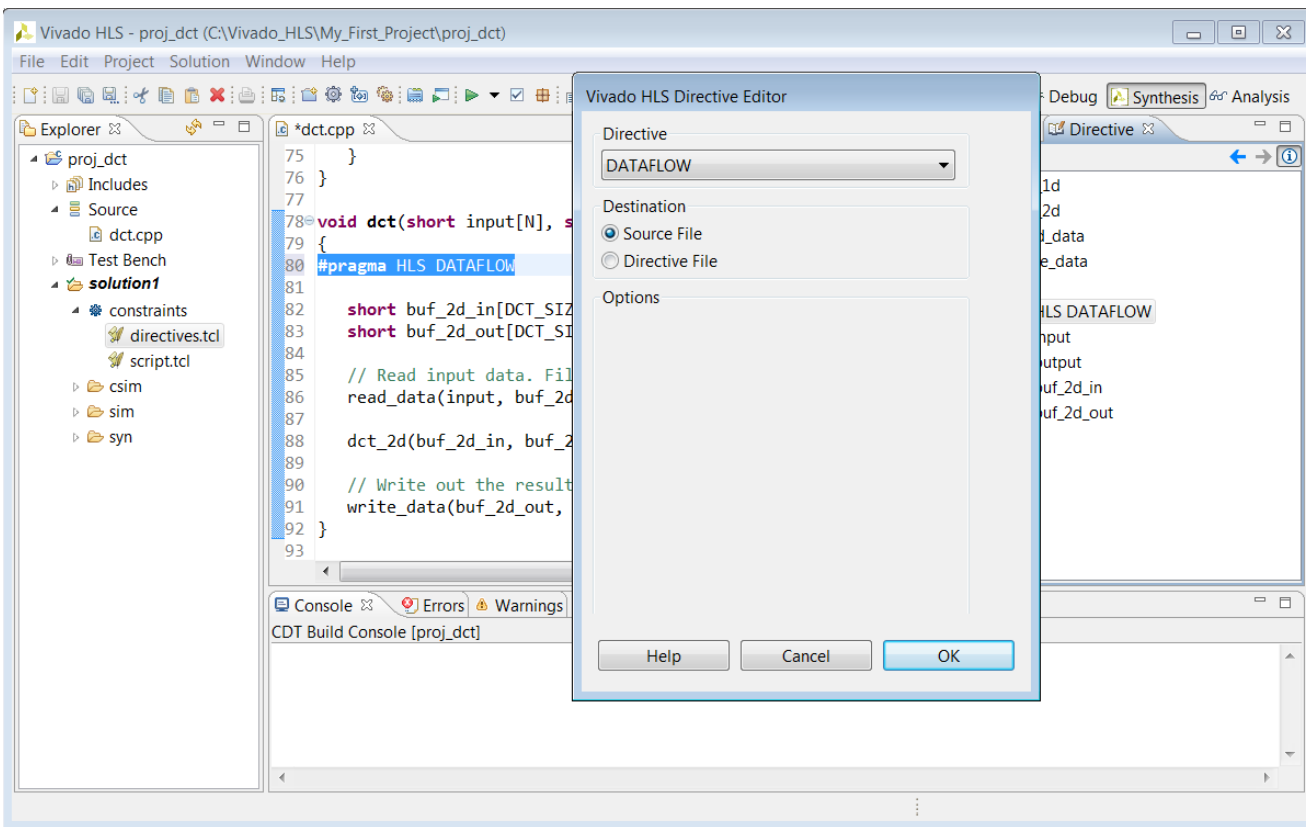
// Example of a NAMED region
My_Region:{
    printf( "The scope between these braces HAS the label My_Region" );
}
```



提示：已命名的循环可便于轻松读取综合报告。对于不含标记的循环，将分配自动生成的标记。

下图显示了向“源文件 (Source File)”添加 DATAFLOW 指令的过程，以及“信息 (Information)”窗格中打开的相关源代码。源代码现在包含用于指定最优化指令的编译指示。

图 33：添加编译指示指令



在这 2 种情况下，执行综合时将应用指令并执行最优化。如果修改了代码（包括插入标记或编译指示），那么会弹出对话框，提醒您在综合前保存代码。

将最优化指令应用于全局变量

指令只能应用于多个作用域或某个作用域内的多个对象。因此，无法将其直接应用于在任意函数作用域外声明的全局变量。

要将指令应用于全局变量，请将指令应用于使用该全局变量的作用域（函数、循环或区域）。在使用变量的作用域上打开“Directives”选项卡，在“HLS Directives Editor”中应用指令并手动输入变量名称。

对类对象应用最优化指令

最优化指令同样可应用于类中定义的对象或作用域。差别通常在于类是在头文件中定义的。使用以下任一操作即可打开头文件：

- 从“Explorer”视图打开 Includes 文件夹、浏览头文件，双击将其打开。
- 在 C 语言源代码中，将光标置于头文件（#include 语句）上，按住“Ctrl”键并单击头文件即可将其打开。

随后，会以头文件中的对象填充“Directives”选项卡，并可应用指令。



注意！ 将指令作为编译指示应用于头文件时应谨慎处理。其他人或其它工程可能使用此文件。每次在设计中包含头文件时，都会应用作为编译指示添加的指令。

对模板应用最优化指令

使用 Tcl 命令时要在模板上手动应用最优化指令，请参阅类方法来指定模板实参和类。例如，给定以下 C++ 代码：

```
template <uint32 SIZE, uint32 RATE>
void DES10<SIZE,RATE>::calcRUN() {}
```

使用以下 Tcl 命令在函数上指定 INLINE 指令：

```
set_directive_inline DES10<SIZE,RATE>::calcRUN
```

将 #Define 与编译指示指令配合使用

编译指示指令原生并不支持使用 define 语句指定的值。以下代码希望通过使用 define 语句来指定流传输深度，而不执行编译。



提示： 请使用显式值指定深度实参。

```
#include <hls_stream.h>
using namespace hls;

#define STREAM_IN_DEPTH 8

void foo (stream<int> &InStream, stream<int> &OutStream) {

// Illegal pragma
#pragma HLS stream depth=STREAM_IN_DEPTH variable=InStream

// Legal pragma
#pragma HLS stream depth=8 variable=OutStream

}
```

如果无需 #define，可使用常量，如，const int。例如：

```
const int MY_DEPTH=1024;
#pragma HLS stream variable=my_var depth=MY_DEPTH
```

在 C 语言代码中可使用宏来实现此功能。使用宏的关键在于使用宏中的层级。这样即可正确执行扩展。代码将按如下方式进行编译：

```
#include <hls_stream.h>
using namespace hls;

#define PRAGMA_SUB(x) _Pragma (#x)
#define PRAGMA_HLS(x) PRAGMA_SUB(x)
#define STREAM_IN_DEPTH 8
```

```
void foo (stream<int> &InStream, stream<int> &OutStream) {

// Legal pragmas
PRAGMA_HLS(HLS stream depth=STREAM_IN_DEPTH variable=InStream)
#pragma HLS stream depth=8 variable=OutStream

}
```

无法满足最优化指令

应用最优化指令时，Vivado HLS 会将详述进程的信息输出至控制台（和日志文件）。在以下示例中，PIPELINE 指令已应用于 C 语言函数且 II=1（迭代时间间隔为 1），但综合未能满足此目标。

```
INFO: [SCHED 11] Starting scheduling ...
INFO: [SCHED 61] Pipelining function 'array_RAM'.
WARNING: [SCHED 63] Unable to schedule the whole 2 cycles 'load' operation
('d_i_load', array_RAM.c:98) on array 'd_i' within the first cycle (II = 1).
WARNING: [SCHED 63] Please consider increasing the target initiation
interval of the
pipeline.
WARNING: [SCHED 69] Unable to schedule 'load' operation ('idx_load_2',
array_RAM.c:98) on array 'idx' due to limited memory ports.
INFO: [SCHED 61] Pipelining result: Target II: 1, Final II: 4, Depth: 6.
INFO: [SCHED 11] Finished scheduling.
```



重要提示！ 如果 Vivado HLS 未能满足最优化指令，它会自动放宽最优化目标，并尝试以更低的性能目标来创建设计。如果无法放宽目标，它将停止并报错。

通过尝试创建满足较低最优化目标的设计，Vivado HLS 可提供 3 种重要信息：

- 使用当前 C 语言代码和最优化指令可达成的目标性能。
- 无法满足更高性能目标的原因列表。
- 可供分析的设计，以提供更多见解并帮助了解失败原因。

在 SCHED-69 消息中，所提供的无法达成目标 II 的原因是由于端口受限。设计必须访问块 RAM，块 RAM 最多只有 2 个端口。

此类失败发生后，下一步是分析问题。在此示例中，即分析代码第 52 行和/或使用“Analysis”透视图来判定瓶颈以及是否可降低需要 2 个以上端口的要求，或者判定如何增加端口的数量。

设计完成最优化并达成期望的性能后，可验证 RTL，并将综合结果封装为 IP。

验证 RTL 正确与否

使用“C/RTL cosimulation”工具栏按钮  或“Solution > Run C/RTL cosimulation”菜单可验证 RTL 结果。

下图所示的 C/RTL 协同仿真对话框允许您选择用于验证的 RTL 输出类型（Verilog 或 VHDL）以及用于仿真的 HDL 仿真器。

图 34: C/RTL 协同仿真对话框



验证完成后，控制台会显示 SIM-1000 消息以确认验证成功。C 语言测试激励文件中的任意 `printf` 命令的结果都会回显到控制台。

```
INFO: [COSIM 316] Starting C post checking ...
Test passed !
INFO: [COSIM 1000] *** C/RTL co-simulation finished: PASS ***
```

在 “Information” 窗格中将自动打开仿真报告并显示成功或失败状态，以及测得的时延和 II 统计数据。



重要提示！ 仅当 C 语言测试激励文件返回 0 值时，才表示 C/RTL 协同仿真成功。协同仿真会对测试激励文件中的场景进行测试，如果返回 True 或 0，则表示测试成功。如果失败，则返回 False 或 1。

复查 C/RTL 协同仿真的输出

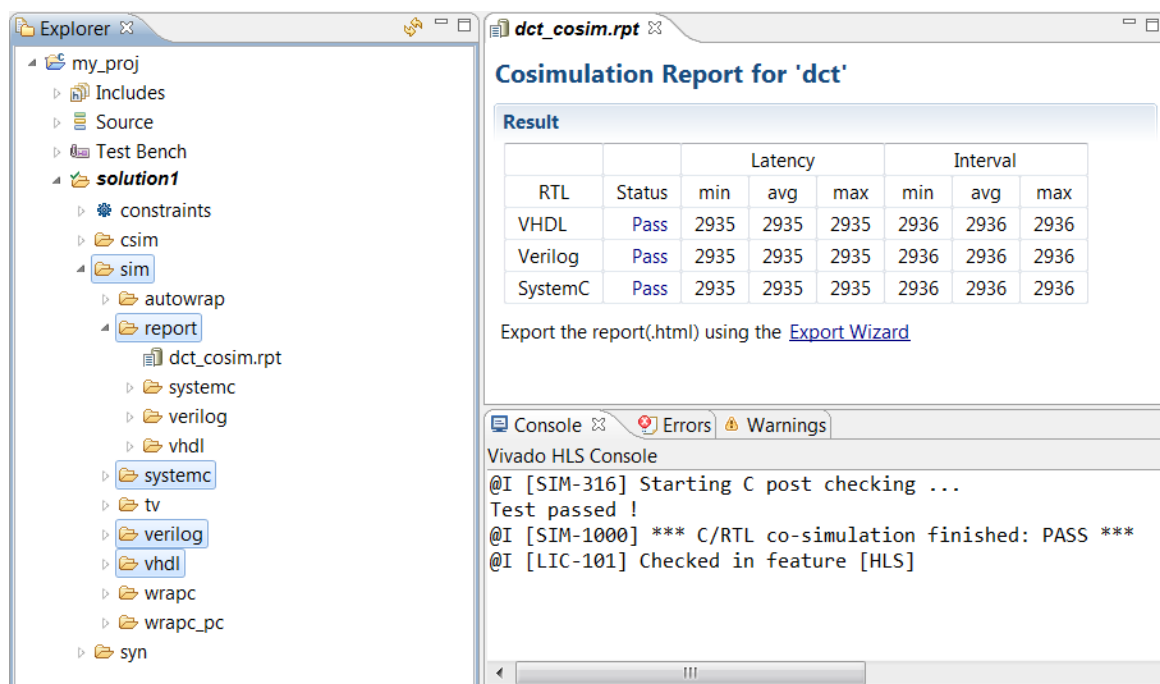
完成 RTL 验证时，会在 solution 文件夹中创建 `sim` 目录。下图显示了创建的子目录。

- 报告目录中包含对应仿真的每一类 RTL 的报告和日志文件。
- 针对验证的每一类 RTL 都会创建 1 个验证文件夹。验证文件夹名为 `verilog` 或 `vhdl`。如果未验证 RTL 格式，则不创建任何文件夹。
- 用于仿真的 RTL 文件存储在验证文件夹内。
- RTL 仿真在验证文件夹内执行。
- 所有输出（如跟踪文件）都写入验证文件夹内。
- `autowrap`、`tv`、`wrap` 和 `wrap_pc` 文件夹是 Vivado HLS 所使用的工作文件夹。这些文件夹内不含任何用户文件。

如果在“C/RTL Co-Simulation”对话框中选中“Setup Only”选项，则会在验证文件夹中创建可执行文件，但不运行仿真。可通过在命令提示符处执行仿真可执行文件来手动运行仿真。

注释：如需了解有关 RTL 验证流程的更多信息，请参阅[验证 RTL](#)。

图 35：RTL 验证输出



Cosimulation Report for 'dct'

Result

RTL	Status	Latency			Interval		
		min	avg	max	min	avg	max
VHDL	Pass	2935	2935	2935	2936	2936	2936
Verilog	Pass	2935	2935	2935	2936	2936	2936
SystemC	Pass	2935	2935	2935	2936	2936	2936

Export the report(.html) using the [Export Wizard](#)

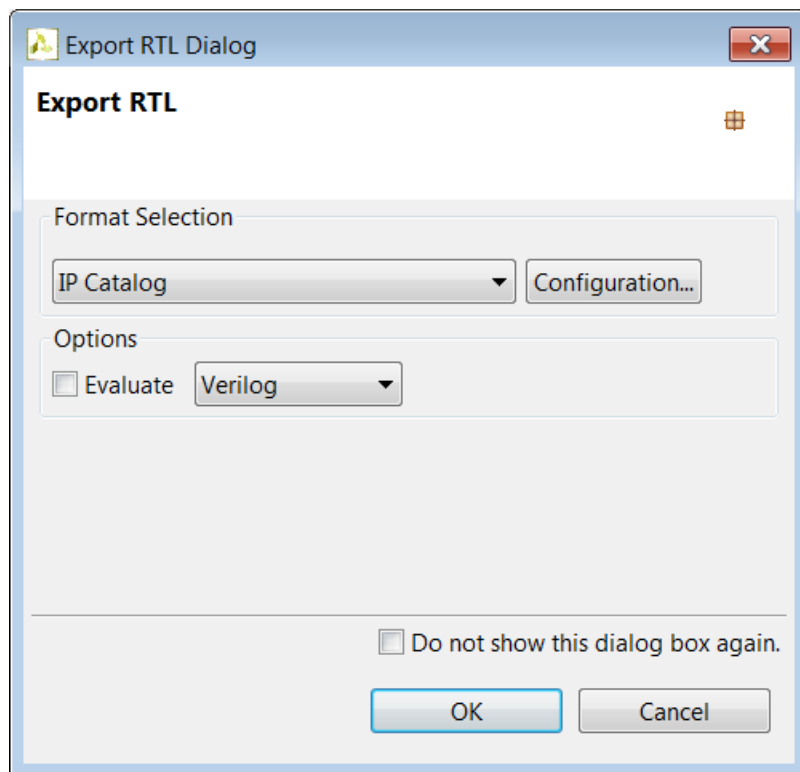
Vivado HLS Console

```
@I [SIM-316] Starting C post checking ...
Test passed !
@I [SIM-1000] *** C/RTL co-simulation finished: PASS ***
@I [LIC-101] Checked in feature [HLS]
```

IP 封装

Vivado HLS 设计流程中最后一步是将 RTL 输出封装为 IP。单击“Export RTL”工具栏按钮 ，或者单击“Solution” → “Export RTL”以打开“导出 RTL (Export RTL)”对话框，如下图所示。

图 36: RTL 导出对话框

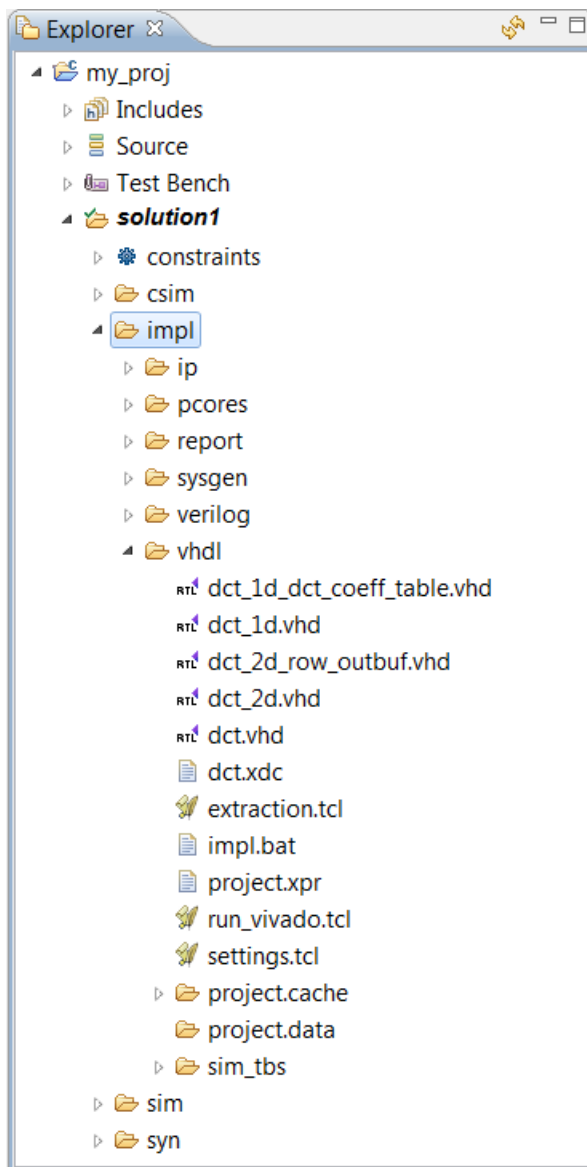


“格式选择 (Format Selection)” 下拉菜单中可用的选项取决于综合的 FPGA 器件目标。

复查 IP 封装的输出

当 “Export RTL” 进程完成后，将在 `solution` 文件夹中创建 `impl` 文件夹。

图 37: “Export RTL” 输出



无论任何情况下，输出都将包含：

- `report` 文件夹。如果选中“Flow”选项，那么 Verilog 和 VHDL 综合或实现的报告将置于此文件夹内。
- `verilog` 文件夹。其中包含 Verilog 格式的 RTL 输出文件。如果选中“Flow”选项，将在此文件夹中执行 RTL 综合或实现。
- `vhdl` 文件夹。其中包含 VHDL 格式的 RTL 输出文件。如果选中“flow”选项，将在此文件夹中执行 RTL 综合或实现。



重要提示！ 赛灵思不建议将 `verilog` 或 `vhdl` 文件夹中的文件直接用于您自己的 RTL 综合工程。赛灵思建议改用已封装的 IP 输出文件，如下文所述。请仔细阅读紧接在本注释后的文本。

如果 Vivado HLS 在设计中使用赛灵思 IP（例如，配合浮点设计使用），那么 RTL 目录包含用于在 RTL 综合期间创建 IP 的脚本。如果将 `verilog` 或 `vhdl` 文件夹中的文件复制出并用于 RTL 综合，那么您自行承担正确使用这些文件夹中存在的任意脚本文件的责任。如果使用封装 IP，那么设计赛灵思工具将自动执行此进程。

“Format Selection” 下拉菜单可判定创建的其它文件夹。已提供下列格式：“IP Catalog”、“System Generator for DSP” 和 “Synthesized Checkpoint (.dcp)”。

表 3: “RTL Export” 选项

格式选择	子文件夹	评述
“IP Catalog”	<code>ip</code>	包含 ZIP 文件，此文件可添加到 Vivado IP 目录中。 <code>ip</code> 文件夹还包含 ZIP 文件的内容（已解压）。 该选项对于早于 7 系列或 Zynq-7000 SoC 的 FPGA 器件不可用。
“System Generator for DSP”	<code>sysgen</code>	此输出可添加到 Vivado 版本的 System Generator for DSP 中。 该选项对于早于 7 系列或 Zynq-7000 SoC 的 FPGA 器件不可用。
“Synthesized Checkpoint (.dcp)”	<code>ip</code>	该选项可创建 Vivado 检查点文件，此文件可直接添加到 Vivado Design Suite 中的设计中。 该选项需执行 RTL 综合。选中该选项时，将自动选中 <code>flow</code> 选项和 <code>syn</code> 设置。 输出包含 HDL 封装，可供您用于将 IP 例化到 HDL 文件中。

Vivado RTL 工程示例

“Export RTL” 进程会自动创建 Vivado RTL 工程。对于更熟悉 RTL 设计并且更熟悉在 Vivado RTL 环境中工作的硬件设计人员，这样可提供一种方便的 RTL 分析方法。

如前图所示，在 `verilog` 文件夹和 `vhdl` 文件夹内会创建 `project.xpr` 文件。此文件可用于在 Vivado Design Suite 内直接打开 RTL 输出。

如果在 Vivado HLS 中已执行 C/RTL 协同仿真，那么 Vivado 工程将包含 RTL 测试激励文件，并且可对设计进行仿真。

Vivado RTL 工程采用来自 Vivado HLS 的 RTL 输出作为顶层设计。通常，此设计应作为 IP 整合到更大的 Vivado RTL 工程中。此 Vivado 工程单独提供以作为设计分析方法，而并非作为实现路径。

IP integrator 工程设计

如果选择 “IP Catalog” 作为输出格式，则会创建输出文件夹 `impl/ip/example`。此文件夹包含可执行文件 (`ipi_example.bat` 或 `ipi_example.csh`)，可用于为 IP integrator 创建工程。

要创建 IP integrator 工程，请在命令提示符处执行 `ipi_example.*` 文件，然后打开由此创建的 Vivado IP integrator 工程文件。

工程归档

要将 Vivado HLS 工程归档为业界标准的 ZIP 文件，请选择 “File > Archive”。使用 “Archive Name” 选项可命名指定的 ZIP 文件。您可按如下方式修改默认设置：

- 默认情况下，仅归档当前处于活动状态的解决方案。要确保将所有解决方案归档，请取消选中 “Active Solution Only” 选项。
- 默认情况下，归档包含归档的解决方案的所有输出结果。如要仅归档输入文件，请取消选中 “Include Run Results” 选项。

使用命令提示符和 Tcl 接口

在 Windows 上，可从开始菜单调用 Vivado HLS 命令提示符：“Xilinx Design Tools” → “Vivado <version>” → “Vivado HLS” → “Vivado HLS <version> Command Prompt”。

在 Windows 和 Linux 上，使用 `-i` 搭配 `vivado_hls` 命令可以交互模式打开 Vivado HLS。随后，Vivado HLS 会等待输入的 Tcl 命令。

```
$ vivado_hls -i [-l <log_file>]
vivado_hls>
```

默认情况下，Vivado HLS 会在当前目录中创建 `vivado_hls.log` 文件。要为日志文件指定其它名称，可使用 `-l <log_file>` 选项。

`help` 命令用于访问有关各项命令的文档。通过以下方式可获取所有命令的完整列表：

```
vivado_hls> help
```

使用命令名称提供有关任一命令的帮助信息。

```
vivado_hls> help <command>
```

使用自动补全功能可补全任意命令或命令选项。指定单一字符后，按 Tab 键可使 Vivado HLS 列出各种选项，以供用于补全命令或命令选项。输入多个字符有助于筛选可用选项。例如，输入“open”后按 Tab 键即可列出以“open”开头的所有命令。

```
vivado_hls> open <press tab key>
open
open_project
open_solution
```

输入 `open_p` 后按 Tab 键可自动补全 `open_project` 命令，因为没有其它可能选项。

输入 `exit` 命令可退出交互模式并返回 shell 提示符：

```
vivado_hls> exit
```

Vivado HLS 的可用选项包括：

- `vivado_hls -p`：打开指定工程
- `vivado_hls -nosplash`：打开不含 Vivado HLS 初始屏幕的 GUI。
- `vivado_hls -r`：返回安装根目录的路径
- `vivado_hls -s`：返回系统类型（例如：Linux 或 Win）
- `vivado_hls -v`：返回发行版版本号。

通过 `-f <script_file>` 选项可批量执行 Tcl 脚本中嵌入的命令。

```
$ vivado_hls -f script.tcl
```

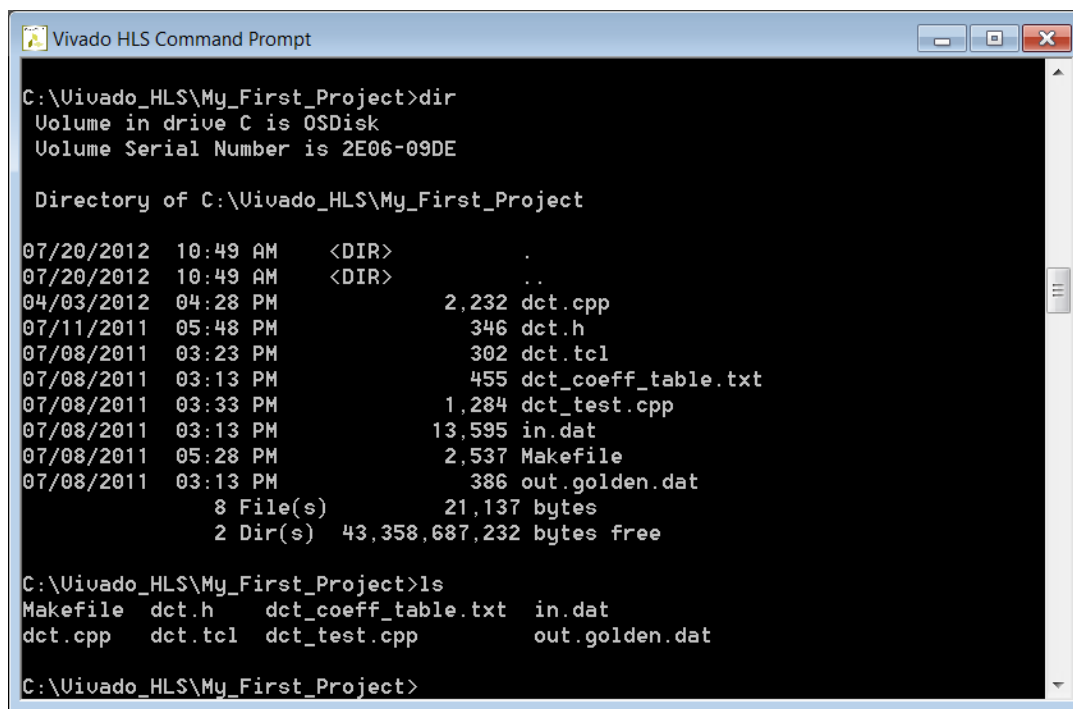
用于在 GUI 中创建工程的所有 Tcl 命令都存储在 solution 文件夹内的 `script.tcl` 文件中。如果要开发 Tcl 批处理脚本，`script.tcl` 文件是一个理想的起点。

了解 Windows 命令提示符

在 Windows 操作系统上，Vivado HLS 命令提示符是使用 Minimalist GNU for Windows (minGW) 环境来实现的，此环境支持使用标准 Windows DOS 命令和少量 Linux 命令。

下图显示了用于列示目录内容的 Linux `ls` 命令和/或 DOS `dir` 命令。

图 38: Vivado HLS 命令提示符



请注意，在 minGW 环境中并不支持所有 Linux 命令和行为。以下显示了部分已知的常见支持差异：

- 不支持 Linux `which` 命令。
- 生成文件 (Makefile) 中的 Linux 路径扩展至 minGW 路径中。在所有 Makefile 文件中，请将任意 Linux 样式的路径名称分配（例如，`FOO := :/`）替换为以双引号扩起的形式（例如，`FOO := ":/"`）以防止发生任何路径替换操作。

改善运行时间和容量

如果 C/RTL 协同仿真存在问题，请参阅[验证 RTL](#) 中的 `reduce_diskpace` 选项。本章下文中回顾了综合运行时间的相关问题。

Vivado HLS 按层级来调度操作。任一循环中的操作将经过调度、然后循环，再调度子函数和函数运算的过程。Vivado HLS 的运行时间会在下列情况下增加：

- 需调度的对象数量增加。
- 有更大自由度和更多可能性可供探索。

Vivado HLS 负责调度对象。无论对象是浮点乘法运算还是单寄存器，它都仍属于要调度的对象。浮点乘法可能需多个周期才能完成，并且使用大量资源来实现，但在调度层次上，它仍是对象。

展开循环和对数组进行分区会创建更多需调度的对象并且可能增加运行时间。函数内联会在此层级创建更多需调度的对象，同样会增加运行时间。这些最优化可能是满足性能要求所必需的，但必须谨慎处理所有数组的分区、所有循环的展开以及所有函数的内联操作：运行时间可能因此增加。请使用先前提提供的最优化策略，并谨慎采用这些最优化。

如果为满足性能必须对数组进行分区，请考虑使用 `config_array_partition` 的 `throughput_driven` 选项，仅根据吞吐量要求来对数组进行分区。

如果必须展开循环，或者如果在上层层级中使用 PIPELINE 指令已自动展开循环，请考虑将循环主体作为独立函数来捕获。这将把所有逻辑捕获到单一函数内，而不是在展开逻辑时创建多个逻辑副本：在已定义的层级内调度一组对象速度更快。谨记，如果在流水线化区域内使用展开的循环，请将此函数流水线化。

代码自由程度同样会影响运行时间。请将 Vivado HLS 视为设计专家，默认负责的任务是查找含最高吞吐量、最慢时延和最小面积的设计。Vivado HLS 所受约束程度越高，可供使用的选项越少，运行速度越快。请考虑对代码中的作用域使用时延约束：循环、函数或区域。设置所含最小值和最大值相同的 LATENCY 指令可以减少该作用域内可能的最优化搜索次数。

最后，`config_schedule` 配置可控制调度期间所使用的工作量。此方法的影响通常小于上述技巧，但值得一试。默认策略设置为 Medium。

如果此设置为 Low，Vivado HLS 将减少用于尝试改进初始结果的时间。在某些情况下，尤其是存在大量运算因而需处理大量组合时，可能值得使用 Low 设置。设计结果可能并不理想，但可能满足要求，并且非常接近理想状态。您可继续借助 Low 设置进行调整，然后使用默认设置，最后再创建最终结果。

通过将运行策略设置为 High，Vivado HLS 会使用额外的 CPU 周期和内存（即使满足约束后也是如此）来判断是否能创建更小或更快的设计。此探索过程有可能会生成更高质量的设计，但需要更多时间和内存才能完成。对于未能满足目标但很接近目标的设计，或者存在大量不同最优化组合的设计，该策略可能很有用。一般情况下，最好保留运行策略的默认设置 Medium。

设计示例与参考

教程

在《Vivado Design Suite 教程：高层次综合》(UG871) 中提供了各种教程。下表显示了教程实践列表。

表 4：Vivado HLS 教程实践

教程实践	描述
Vivado HLS 简介教程	使用 FIR 设计的 Vivado HLS 的运算和主要特性简介。
C 验证	本教程使用 Hamming 窗设计以解释 C 语言仿真，并使用 C 语言调试环境来验证 C 语言算法。
接口综合	有关如何使用接口综合创建各种类型的 RTL 接口端口的实践。
任意精度类型	显示如何使用定点任意精度类型来实现浮点绕组函数以生成更优化的硬件。

表 4: Vivado HLS 教程实践 (续)

教程实践	描述
设计分析	显示如何使用“Analysis”透视图来提高 DCT 块的性能。
设计最优化	使用矩阵乘法示例显示算法的最优化方式。本教程演示了特定硬件实现可能要求更改初始值的方式。
RTL 验证	如何使用 RTL 验证功能并分析 RTL 信号波形。
在 IP integrator 中使用 HLS IP	显示如何使用 IP integrator 将 FFT 的 2 个 HLS 处理前和处理后块连接到 FFT IP 块。
在 Zynq-7000 SoC 处理器设计中使用 HLS IP	显示如何通过 AXI4-Lite 接口使用 CPU 来控制 Vivado HLS 块以及来自往来 Vivado HLS 块的 DDR 内存的 DMA 流传输数据。在 SDK 中包含 CPU 源代码和所需步骤。
在 System Generator for DSP 中使用 HLS IP	有关如何在 System Generator for DSP 设计中使用 HLS 块的教程。

设计示例

要从“Welcome”页面打开 Vivado HLS 设计示例，请单击“Open Example Project”。在“示例 (Examples)”向导中，从“Design Examples”文件夹选择设计。

注释：调用 Vivado HLS GUI 时会显示“Welcome”页面。您可随时选择“Help”→“Welcome”来访问此页面。

您还可从以下 Vivado Design Suite 安装位置直接打开设计示例：Vivado_HLS\<version>\examples\designs。

下表提供了每个设计示例的描述。

表 5: Vivado HLS 设计示例

设计示例	描述
2D_convolution_with_linebuffer	使用 hls::streams 实现的 2D 卷积以及用于保存资源的行缓存。
FFT > fft_ifft	使用 FFT IP 反转 FFT。
FFT > fft_single	含流水线化流传输 I/O 的单个 1024 点正向 FFT。
FIR > fir_2ch_int	含 2 个交织通道的 FIR 滤波器
FIR > fir_3stage	含 3 个串联 FIR 的 FIR 链：半带 FIR 到半带 FIR 到平方根升余弦 (SRRC) FIR。
FIR > fir_config	已使用 FIR CONFIG 通道更新系数的 FIR 滤波器。
FIR > fir_srrc	SRRC FIR 滤波器。
__builtin_ctz	使用 gcc 内置“count trailing zero”函数实现的优先级编码器 (32 位版本和 64 位版本)。
axi_lite	AXI4-Lite 接口。
axi_master	AXI4 主接口。
axi_stream_no_side_channel_data	C 语言代码中不含旁路数据的 AXI4-Stream 接口。
axi_stream_side_channel_data	使用旁路数据的 AXI4-Stream 接口。
dds > dds_mode_fixed	使用相位偏移和相位增量 (固定模式下使用) 创建的 DDS IP。
dds > dds_mode_none	使用相位偏移 (固定模式下使用) 创建的 DDS IP，无相位增量 (mode=none)。
dsp > atan2	来自 HLS DSP 库的 arctan 函数。
dsp > awgn	来自 HLS DSP 库的加性高斯白噪声 (awgn) 函数。

表 5: Vivado HLS 设计示例 (续)

设计示例	描述
dsp > cmpy_complex	使用复数数据类型的定点复数乘法器。
dsp > cmpy_scalar	针对实数和虚数组件使用独立标量数据类型的定点复数乘法器。
dsp > convolution_encoder	来自 HLS DSP 库的 Convolution_encoder 函数，基于用户定义的卷积代码和约束长度执行输入数据流的卷积编码。
dsp > nco	来自 HLS DSP 库的数控振荡器 (NCO) 函数。
dsp > sqrt	来自 HLS DSP 库的平方根函数的定点坐标旋转数字计算机 (CORDIC) 实现。
dsp > viterbi_decoder	来自 HLS DSP 库的 Viterbi 解码器。
fp_mul_pow2	使用 2 次幂的高效（面积和时序）浮点乘法实现，使用小型加法器和某些可选限制检查代替浮点核和 DSP 资源。
fxp_sqrt	以位元串列可完全流水线化方式实现的 ap_fixed 类型的平方根实现。
hls_stream	使用 hls::stream 的多速率数据流（8 位 I/O，32 位数据抽取和抽取）设计。
linear_algebra > cholesky	参数化 Cholesky 函数。
linear_algebra > cholesky_alt	替代 Cholesky 实现。
linear_algebra > cholesky_alt_inverse	Cholesky 函数，含自定义特征类用于选择不同实现。
linear_algebra > cholesky_complex	Cholesky 函数，含复数数据类型。
linear_algebra > cholesky_inverse	参数化 Cholesky 反函数。
linear_algebra > implementation_targets	实现目标示例。 欲知详情，请参阅 线性代数函数最优化 。
linear_algebra > matrix_multiply	参数化矩阵乘法函数。
linear_algebra > matrix_multiply_alt	替代矩阵乘法函数。
linear_algebra > qr_inverse	参数化 QR 反函数。
linear_algebra > qrf	参数化 QRF 函数。
linear_algebra > qrf_alt	替代参数化 QRF 函数。
linear_algebra > svd	参数化 SVD 函数。
linear_algebra > svd_pairs	参数化 SVD 函数，含替代“配对”SVD 实现。
loop_labels > loop_label	含标记的循环。
loop_labels > no_loop_label	不含标记的循环。
memory_porting_and_ii	使用数组分区指令改善的启动时间间隔。
perfect_loop > perfect	完美循环。
perfect_loop > semi_perfect	半完美循环。
rom_init_c	使用子函数编码以保证 ROM 实现的数组。
window_fn_float	单精度浮点窗函数。C++ 模板类示例，含如下编译时间选择：矩形 (Rectangular) 窗（无）、汉宁 (Hann) 窗、汉明 (Hamming) 窗或高斯 (Gaussian) 窗。
window_fn_fxpt	定点窗函数。C++ 模板类示例，含如下编译时间选择：“Rectangular (none)”窗、“Hann”窗、“Hamming”窗或“Gaussian”窗。

编码示例

Vivado HLS 编码示例提供了各种编码技巧示例。这些小型示例旨在突显 Vivado HLS 综合对于各种 C、C++ 和 SystemC 语言构造的结果。

要从“Welcome”页面打开 Vivado HLS 编码示例，请单击“Open Example Project”。在“示例 (Examples)”向导中，从“Coding Style Examples”文件夹选择设计。

注释：调用 Vivado HLS GUI 时会显示“Welcome”页面。您可随时选择“Help”→“Welcome”来访问此页面。

您还可从以下 Vivado Design Suite 安装位置直接打开设计示例：Vivado_HLS\<version>\examples\coding。

下表提供了每个编码示例的描述。

表 6：Vivado HLS 编码示例

编码示例	描述
apint_arith	使用 C ap_cint 类型。
apint_promotion	突显避免 C ap_cint 类型的整数提升问题所需的强制类型转换。
array_arith	使用接口数组中的运算。
array_FIFO	实现 FIFO 接口。
array_mem_bottleneck	演示数组访问如何导致性能瓶颈。
array_mem_perform	array_mem_bottleneck 示例所示的性能瓶颈的解决方案。
array_RAM	实现块 RAM 接口。
array_ROM	演示如何自动推断 ROM 的示例。
array_ROM_math_init	演示如何在更复杂的情况下推断 ROM 的示例。
cpp_ap_fixed	使用 C++ ap_int 类型。
cpp_ap_int_arith	使用 C++ ap_int 类型执行运算。
cpp_FIR	使用面向对象的编码样式的 C++ 设计示例。
cpp_math	浮点运算设计示例，展示对不完全符合 IEEE 标准的运算结果进行比较时，如何在测试激励文件中使用容限。
cpp_template	C++ 模板示例。
func_sized	通过在接口处定义数据宽度来固定运算的大小。
hier_func	添加文件作为测试激励文件和设计文件的示例。
hier_func2	添加文件作为测试激励文件和设计文件的示例。在层级中综合低层块的示例。
hier_func3	将测试激励文件与设计函数组合到同一文件中的示例。
hier_func4	使用预定义的 __SYNTHESIS__ 宏来阻止对代码进行综合。 __SYNTHESIS__ 宏仅限在要综合的代码中使用。请勿在测试激励文件中使用该宏，因为 C 语言仿真或 C RTL 协同仿真不会遵循其指示进行操作。
loop_functions	将循环转换为函数以便并行执行。
loop_imperfect	非完美循环示例。
loop_max_bounds	使用上限来允许将循环展开。
loop_perfect	完美循环示例。
loop_pipeline	循环流水线化示例。
loop_sequential	时序循环。

表 6: Vivado HLS 编码示例 (续)

编码示例	描述
loop_sequential_assert	使用断言语句。
loop_var	含变量边界的循环。
malloc_removed	从代码中移除 mallocs 的示例。
pointer_arith	指针算法示例。
pointer_array	指针数组。
pointer_basic	基本指针示例。
pointer_cast_native	原生 C 类型间的指针强制转换。
pointer_double	指针对指针示例。
pointer_multi	使用多个指针目标的示例。
pointer_stream_better	显示如何在接口上使用易变关键字的示例。
pointer_stream_good	使用显式指针算法的多重读取指针示例。
sc_combo_method	SystemC 组合设计示例。
sc_FIFO_port	SystemC FIFO 端口示例。
sc_multi_clock	含多个时钟的 SystemC 示例。
sc_RAM_port	SystemC 块 RAM 端口示例。
sc_sequ_cthread	SystemC 顺序设计示例。
struct_port	在接口上使用结构体。
sum_io	顶层接口端口示例。
types_composite	复合类型。
types_float_double	浮点类型到双精度类型的转换。
types_global	使用全局变量。
types_standard	标准 C 语言类型示例。
types_union	联合体示例。

高效硬件的数据类型

基于 C 的原生数据类型全部都位于 8 位边界（8 位、16 位、32 位和 64 位）上。RTL 总线（对应于硬件）支持任意数据长度。使用标准 C 语言数据类型可能导致硬件低效。例如，FPGA 中的基本乘法单位是 DSP48 宏。它可提供 18*18 位的乘法器。如果需要 17 位乘法，则不应强制使用 32 位 C 语言数据类型来实现：后者将需要 3 个 DSP48 宏来实现 1 个乘法器，而实际上只需 1 个 DSP48 宏即可。

任意精度数据类型的优势在于允许更新 C 语言代码以使用位宽较小的变量，然后允许重新执行 C 语言仿真以验证功能保持不变或可接受。较小的位宽所生成的硬件运算符也更小且更快。这样即可允许在 FPGA 中布局更多逻辑，并允许以更高的时钟频率来执行逻辑。

硬件有效的数据类型的优势

以下代码用于执行部分基本算术运算：

```
#include "types.h"

void apint_arith(dinA_t inA, dinB_t inB, dinC_t inC, dinD_t inD,
                dout1_t *out1, dout2_t *out2, dout3_t *out3, dout4_t *out4
                ) {

    // Basic arithmetic operations
    *out1 = inA * inB;
    *out2 = inB + inA;
    *out3 = inC / inA;
    *out4 = inD % inA;
}
```

数据类型 `dinA_t`、`dinB_t` 等在 `types.h` 头文件中定义。强烈建议使用适用于全工程的头文件（如 `types.h`），这样可便于从标准 C 语言类型移植到任意精度类型，并且有助于将任意精度类型优化为最优化大小。

如果以上示例中的数据类型定义为：

```
typedef char dinA_t;
typedef short dinB_t;
typedef int dinC_t;
typedef long long dinD_t;
typedef int dout1_t;
typedef unsigned int dout2_t;
typedef int32_t dout3_t;
typedef int64_t dout4_t;
```

综合后，此设计结果如下：

```
+ Timing (ns):
  * Summary:
  +-----+-----+-----+-----+
  | Clock  | Target| Estimated| Uncertainty|
  +-----+-----+-----+-----+
  | default | 4.00 | 3.85 | 0.50 |
  +-----+-----+-----+-----+

+ Latency (clock cycles):
  * Summary:
  +-----+-----+-----+-----+
  | Latency | Interval | Pipeline |
  | min | max | min | max | Type |
  +-----+-----+-----+-----+
  | 66 | 66 | 67 | 67 | none |
  +-----+-----+-----+-----+

* Summary:
+-----+-----+-----+-----+-----+
| Name | BRAM_18K | DSP48E | FF | LUT |
+-----+-----+-----+-----+-----+
| Expression | - | - | 0 | 17 |
| FIFO | - | - | - | - |
| Instance | - | 1 | 17920 | 17152 |
```

Memory	-	-	-	-
Multiplexer	-	-	-	-
Register	-	-	7	-
+-----+				
Total	0	1	17927	17169
+-----+				
Available	650	600	202800	101400
+-----+				
Utilization (%)	0	~0	8	16
+-----+				

如果使用标准 C 语言类型实现的设计并无数据宽度要求，但对于如下数据类型（宽度比标准 C 类型小、而比下一个最小的标准 C 语言类型更大）：

```
typedef int6 dinA_t;
typedef int12 dinB_t;
typedef int22 dinC_t;
typedef int33 dinD_t;
typedef int18 dout1_t;
typedef uint13 dout2_t;
typedef int22 dout3_t;
typedef int6 dout4_t;
```

综合后的结果显示最大时钟频率有所改善、时延改善，且面积利用率显著降低达 75%。

```
+ Timing (ns):
  * Summary:
  +-----+-----+-----+-----+
  | Clock | Target | Estimated | Uncertainty |
  +-----+-----+-----+-----+
  | default | 4.00 | 3.49 | 0.50 |
  +-----+-----+-----+-----+

+ Latency (clock cycles):
  * Summary:
  +-----+-----+-----+-----+
  | Latency | Interval | Pipeline |
  | min | max | min | max | Type |
  +-----+-----+-----+-----+
  | 35 | 35 | 36 | 36 | none |
  +-----+-----+-----+-----+

* Summary:
+-----+-----+-----+-----+-----+
| Name | BRAM_18K | DSP48E | FF | LUT |
+-----+-----+-----+-----+-----+
| Expression | - | - | 0 | 13 |
| FIFO | - | - | - | - |
| Instance | - | 1 | 4764 | 4560 |
| Memory | - | - | - | - |
| Multiplexer | - | - | - | - |
| Register | - | - | 6 | - |
+-----+-----+-----+-----+-----+
| Total | 0 | 1 | 4770 | 4573 |
+-----+-----+-----+-----+-----+
| Available | 650 | 600 | 202800 | 101400 |
+-----+-----+-----+-----+-----+
| Utilization (%) | 0 | ~0 | 2 | 4 |
+-----+-----+-----+-----+-----+
```

这两种设计之间存在的巨大时延差异原因在于除法和余数运算耗费多个周期才完成。使用准确的数据类型而不是强制将设计与标准 C 语言数据类型拟合可以提高 FPGA 实现的质量：相同的准确性，但运行速度更快，耗用资源更少。

任意精度整数数据类型概述

Vivado® HLS 为 C 和 C++ 提供了整数和定点任意精度数据类型，并支持属于 SystemC 的任意精度数据类型。

表 7：任意精度数据类型

语言	整数数据类型	所需头文件
C	[u]int<W> (1024 位)	#include "ap_cint.h"
C++	ap_[u]int<W> (1024 位) 位宽可扩展至 32K 位。	#include "ap_int.h"
C++	ap_[u]fixed<W,I,Q,O,N>	#include "ap_fixed.h"
SystemC	sc_[u]int<W> (64 位) sc_[u]bigint<W> (512 位)	#include "systemc.h"
SystemC	sc_[u]fixed<W,I,Q,O,N>	#define SC_INCLUDE_FX [#define SC_FX_EXCLUDE_OTHER] #include "systemc.h"

Vivado® HLS 也随附定义任意精度类型的头文件作为独立封装，并且您有权在自己的源代码中使用这些任意精度类型。在 Vivado® HLS 安装区域的 include 目录中提供了 xilinx_hls_lib_<release_number>.tgz 封装。此封装不包含 ap_cint.h 中定义的 C 任意精度类型。这些类型不能配合标准 C 语言编译器一起使用 - 只能搭配 Vivado® HLS 使用。

用于 C 的任意精度整数类型

对于 C 语言，头文件 ap_cint.h 用于定义任意精度整数数据类型 [u]int。要在 C 语言函数中使用任意精度整数数据类型，请执行以下操作：

- 将头文件 ap_cint.h 添加到源代码。
- 将位类型更改为 intN 或 uintN，其中 N 是介于 1 到 1024 之间的位大小。

用于 C++ 的任意精度类型

对于 C++ 语言 ap_[u]int 数据类型，头文件 ap_int.h 用于定义任意精度整数数据类型。要在 C++ 函数中使用任意精度整数数据类型，请执行以下操作：

- 将头文件 ap_int.h 添加到源代码。
- 将位类型更改为 ap_int<N> 或 ap_uint<N>，其中 N 是介于 1 到 1024 之间的位大小。

以下示例显示了如何添加头文件并实现 2 个变量来使用 9 位整数和 10 位无符号的整数类型：

```
#include "ap_int.h"

void foo_top () {

    ap_int<9>   var1;           // 9-bit
    ap_uint<10> var2;          // 10-bit unsigned
}
```

针对 `ap_[u]int` 数据类型允许的默认最大位宽为 1024 位。此默认设置可通过在包含 `ap_int.h` 头文件之前定义含正整数（小于或等于 32768）的 `AP_INT_MAX_W` 宏来覆盖。



注意！ `AP_INT_MAX_W` 的值设置过高可能导致软件编译缓慢、运行时间延长。



注意！ 使用 `APFixed` 时，ROM 综合耗时较长。将其更改为 `int` 可加速综合。例如：

```
static ap_fixed<32> a[32][depth] =
```

可更改为：

```
static int a[32][depth] =
```

以下是覆盖 `AP_INT_MAX_W` 的示例：

```
#define AP_INT_MAX_W 4096 // Must be defined before next line
#include "ap_int.h"

ap_int<4096> very_wide_var;
```

用于 SystemC 的任意精度类型

供 SystemC 使用的任意精度类型在 `systemc.h` 头文件中定义，所有 SystemC 语言设计中都必须包含此头文件。此头文件包含 SystemC `sc_int<>`、`sc_uint<>`、`sc_bigint<>` 和 `sc_bignint<>` 类型。

任意精度定点数据类型概述

定点数据类型将数据作为整数位和小数位形式进行建模。在此示例中，Vivado HLS `ap_fixed` 类型用于定义 18 位变量，其中 6 位用于表示二进制小数点前的数值，12 位用于表示小数点后的值。变量指定为有符号，量化模式设置为舍入到正无穷。由于未指定溢出模式，因此针对溢出使用默认卷绕模式。

```
#include <ap_fixed.h>
...
ap_fixed<18,6,AP_RND > my_type;
...
```

执行计算时，如果其中所含变量具有不同位数或不同精度，那么二进制小数点将自动对齐。

使用定点执行的 C++/SystemC 语言仿真行为与生成的硬件相匹配。这使您能够使用 C 语言层次快速仿真来分析位精度、量化和溢出行为。

定点类型适合替代需要多个时钟周期才能完成的浮点类型。除非需要浮点类型的完整范围，否则通常使用定点类型即可实现相同精度，并且硬件更小也更快。

下表中提供了 `ap_fixed` 类型的标识的汇总。

表 8：定点标识汇总

标识	描述
W	字宽（位）
I	用于表示整数值的位数（高于二进制小数点的位数）

表 8：定点标识汇总 (续)

标识	描述		
Q	量化模式。 用于规定当生成的精度大于用于存储结果的变量的最小小数位可定义的精度时的行为。		
	SystemC 类型	ap_fixed 类型	描述
	SC_RND	AP_RND	舍入到正无穷
	SC_RND_ZERO	AP_RND_ZERO	舍入到 0
	SC_RND_MIN_INF	AP_RND_MIN_INF	舍入到负无穷
	SC_RND_INF	AP_RND_INF	舍入到无穷
	SC_RND_CONV	AP_RND_CONV	收敛舍入
	SC_TRN	AP_TRN	截断到负无穷（默认）
	SC_TRN_ZERO	AP_TRN_ZERO	截断到 0
O	溢出模式。 用于规定当运算结果超出结果变量可存储的最大值（或者对于负值为最小值）时的行为。		
	SystemC 类型	ap_fixed 类型	描述
	SC_SAT	AP_SAT	饱和
	SC_SAT_ZERO	AP_SAT_ZERO	饱和到 0
	SC_SAT_SYM	AP_SAT_SYM	对称饱和
	SC_WRAP	AP_WRAP	卷绕（默认）
	SC_WRAP_SM	AP_WRAP_SM	符号量值卷绕
N	定义溢出卷绕模式中的饱和位数。		

针对 `ap_[u]fixed` 数据类型允许的默认最大位宽为 1024 位。此默认设置可通过在包含 `ap_int.h` 头文件之前定义含正整数（小于或等于 32768）的 `AP_INT_MAX_W` 宏来覆盖。



注意！ `AP_INT_MAX_W` 的值设置过高可能导致软件编译缓慢、运行时间延长。



注意！ ROM 综合在以下情况下可能较慢：`static APFixed_2_2 CAcode_sat[32][CACODE_LEN] =`。将 `APFixed` 更改为 `int` 可加速综合：`static int CAcode_sat[32][CACODE_LEN] =`

以下是覆盖 `AP_INT_MAX_W` 的示例：

```
#define AP_INT_MAX_W 4096 // Must be defined before next line
#include "ap_fixed.h"

ap_fixed<4096> very_wide_var;
```

使用 Vivado HLS 时强烈建议采用任意精度数据类型。如先前示例所示，任意精度数据类型通常明显有利于提升硬件实现的质量。

管理接口

在基于 C 语言的设计中，通过正式的函数实参即可立即执行所有输入和输出操作。在 RTL 设计中，同样这些输入和输出操作必须通过设计接口中的端口来执行，并且通常使用特定 I/O（输入/输出）协议来进行操作。

Vivado HLS 支持使用以下解决方案来指定要使用的 I/O 协议类型：

- 接口综合，其中端口接口基于高效的业界标准接口来创建。

接口综合

对顶层函数进行综合时，函数的实参（或参数）将综合到 RTL 端口中。此流程称为“接口综合 (interface synthesis)”。

接口综合概述

以下代码提供了接口综合的完整概述。

```
#include "sum_io.h"

dout_t sum_io(din_t in1, din_t in2, dio_t *sum) {

    dout_t temp;

    *sum = in1 + in2 + *sum;
    temp = in1 + in2;

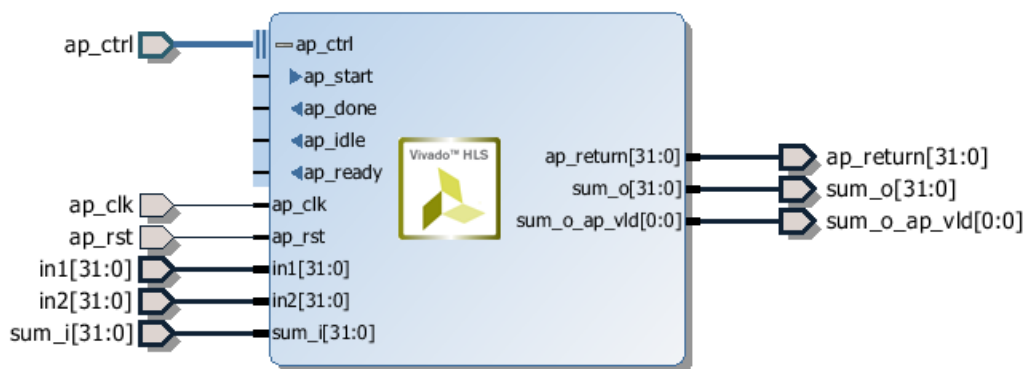
    return temp;
}
```

以上示例包括：

- 2 个按值传递 (pass-by-value) 输入：in1 和 in2。
- 可供读取和写入的指针 sum。
- return 函数，值为 temp。

通过默认接口综合设置，设计将综合到含端口的 RTL 块中，如下图所示。

图 39：默认接口综合后的 RTL 端口



Vivado HLS 会在 RTL 设计上创建 3 种类型的端口：

- 时钟和复位端口：ap_clk 和 ap_rst。
- 块级接口协议。在前图中已显示并展开这些端口：ap_start、ap_done、ap_ready 和 ap_idle。
- 端口级接口协议。这些端口是针对顶层函数和函数返回（如果函数返回值）中的每个实参创建的。在此示例中，这些端口包括：in1、in2、sum_i、sum_o、sum_o_ap_vld 和 ap_return。

时钟和复位端口

如果设计耗时超过 1 个周期才能完成操作。

（可选）可使用“Solution”→“Solution Settings”→“General”和 config_interface 配置将芯片使能端口添加到整个块中。

复位操作由 config_rtl 配置控制。

块级接口协议

默认情况下，块级接口协议会添加到设计中。这些信号用于控制块，与任意端口级 I/O 协议无关。这些端口用于控制块开始处理数据的时间 (ap_start)、指示它是否已准备好开始接受新输入 (ap_ready) 以及指示设计是处于空闲状态 (ap_idle) 还是已完成操作 (ap_done)。

端口级接口协议

最后一组信号是数据端口。创建的 I/O 协议取决于 C 语言实参的类型和默认值。使用块级协议启动块操作后，端口级 I/O 协议用于对进出块的数据进行排序。

默认情况下，输入按值传递 (pass-by-value) 实参和指针作为简单的线型端口来实现，无关联的握手信号。因此在以上示例中，实现的输入端口不含 I/O 协议，仅为数据端口。如果此端口不含默认或按设计指定的 I/O 协议，那么输入数据必须保持稳定直至读取为止。

默认输出指针实现时含关联的输出有效信号，用于指示何时输出数据有效。在上述示例中，输出端口实现时含关联的输出有效端口 (sum_o_ap_vld) 以指示何时端口上的数据有效并且可供读取。如果不存在与输出端口关联的 I/O 协议，那么将难以确定何时读取数据。最好始终在输出上使用 I/O 协议。

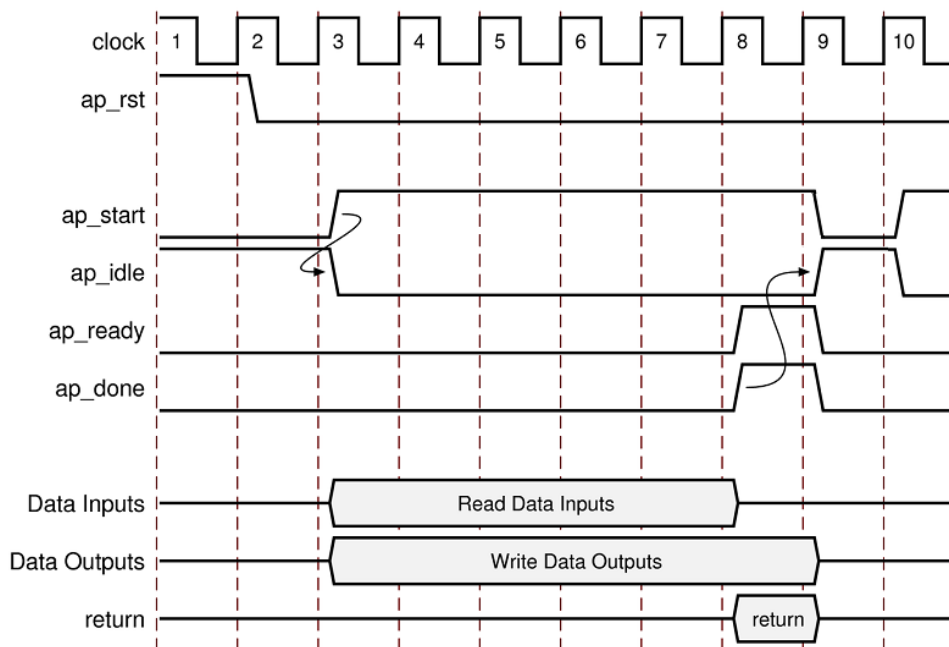
同时支持读取和写入的函数实参将拆分为独立的输入端口和输出端口。在以上示例中，sum 作为输出端口 sum_i 和输出端口 sum_o 来实现，并具有关联的 I/O 协议端口 sum_o_ap_vld。

如果函数具有返回值，则实现输出端口 ap_return 以提供返回值。当设计完成 1 项传输事务时（等同于执行 1 次 C 语言函数），块级协议会以 ap_done 信号来表明函数已完成。这也表示 ap_return 端口上的数据有效且可读。

注释：返回顶层函数的值不能为指针。

对于所示的示例代码，时序行为如下图所示（假定目标技术和时钟频率允许每个时钟周期执行一次加法）。

图 40：含默认综合的 RTL 端口时序



- 当 `ap_start` 断言为 High 时，即开始设计。
- `ap_idle` 信号断言为 Low 以表示设计正在运行。
- 输入数据可在首个周期后的任意时钟内读取。Vivado HLS 会调度读取发生的时间。读取所有输入后，`ap_ready` 信号即断言为高位有效 (high)。
- 计算输出 `sum` 时，关联的输出握手 (`sum_o_ap_vld`) 表示数据有效。
- 当函数完成时，即断言 `ap_done` 有效。这也表示 `ap_return` 上的数据有效。
- 端口 `ap_idle` 断言为 High 以表示设计正在等待再次启动。

接口综合 I/O 协议

由接口综合所创建的接口类型取决于 C 语言实参的类型、默认接口模式以及 INTERFACE 最优化指令。下图显示了您可在每一种 C 语言实参类型上指定的接口协议模式。此图使用以下首字母缩略词：

- D：每一种类型的默认接口模式。

注释：如果指定非法接口，Vivado HLS 会发出 1 条消息，并实现默认接口模式。

- I：输入实参（只读）。
- O：输出实参（只写）。
- I/O：输入/输出实参（可读写）。

图 41：数据类型和接口综合支持

Argument Type	Scalar		Array			Pointer or Reference			HLS:: Stream
Interface Mode	Input	Return	I	I/O	O	I	I/O	O	I and O
ap_ctrl_none									
ap_ctrl_hs		D							
ap_ctrl_chain									
axis									
s_axilite									
m_axi									
ap_none	D					D			
ap_stable									
ap_ack									
ap_vld								D	
ap_ovld							D		
ap_hs									
ap_memory			D	D	D				
bram									
ap_fifo									D
ap_bus									



Supported



D = Default Interface



Not Supported

X14293

在[接口综合参考](#)中包含有关接口协议的完整详细信息（包括波形图）。以下提供了每个接口模式的概述。

块级接口协议

块级接口协议包括 `ap_ctrl_none`、`ap_ctrl_hs` 和 `ap_ctrl_chain`。这些协议在且只能在函数或函数返回时指定。在 GUI 中指定该指令时，会将这些协议应用于函数返回。即使函数不使用返回值，也可在函数返回时指定块级协议。

前述示例中所述的 `ap_ctrl_hs` 模式是默认协议。`ap_ctrl_chain` 协议类似于 `ap_ctrl_hs`，但具有额外的输入端口 `ap_continue` 以提供来自使用此块数据的其它块的回压。如果函数完成时 `ap_continue` 端口为逻辑 0，此块将停止操作，并且不会继续执行下一项传输事务。仅当 `ap_continue` 断言为逻辑 1 时，才会继续执行下一项传输事务。

`ap_ctrl_none` 模式用于实现不含任何块级 I/O 协议的设计。

如果函数返回同时指定为 AXI4-Lite 接口 (`s_axilite`)，那么块级接口中的所有端口都将分组到此 AXI4-Lite 接口中。当使用其它器件（如 CPU）来配置和控制此块的开始和停止操作时间时，常用此方法。

端口级接口协议：AXI4 接口

Vivado HLS 支持的 AXI4 接口包括 AXI4-Stream 接口 (`axis`)、AXI4-Lite 接口 (`s_axilite`) 和 AXI4 主接口 (`m_axi`)，这些接口可按以下方式指定：

- AXI4-Stream 接口：仅在输入实参或输出实参上指定，而不在输入/输出实参上指定。
- AXI4-Lite 接口，在任何类型的实参上指定，但是流传输除外。您可以将多个实参分组到同一 AXI4-Lite 接口中。
- AXI4 主接口：仅在数组和指针（以及 C++ 中的引用）上指定。您可以将多个实参分组到同一 AXI4 接口中。

端口级接口协议：无 I/O 协议

`ap_none` 和 `ap_stable` 模式可指定不向端口添加任何 I/O 协议。指定这些模式时，实参作为不含任何其它关联信号的数据端口来实现。`ap_none` 模式是标量输入的默认模式。`ap_stable` 模式用于仅当器件处于复位模式时才可更改的配置输入。

端口级接口协议：有线握手

接口模式 `ap_hs` 包含与数据端口的双向握手信号。此握手属于业界标准的有效和确认握手。`ap_vld` 模式同样如此，但仅含有效端口，`ap_ack` 仅含确认端口。

`ap_ovld` 模式用于输入输出实参。将输入输出拆分为独立输入端口和输出端口时，`ap_none` 模式适用于输入端口，`ap_vld` 适用于输出端口。这是支持读写的指针实参的默认行为。

`ap_hs` 模式可应用于按顺序读写的数组。如果 Vivado HLS 可判定读访问或写访问为无序访问，它将停止综合并报错。如果无法判定访问顺序，Vivado HLS 将发出警告。

端口级接口协议：内存接口

默认情况下，数组实参作为 `ap_memory` 接口来实现。这是含数据、地址、芯片使能和写使能端口的标准块 RAM 接口。

`ap_memory` 接口可作为单端口接口或双端口接口来实现。如果 Vivado HLS 可判定使用双端口接口将缩短启动时间间隔，那么它将自动实现双端口接口。`RESOURCE` 指令用于指定内存资源，如果在含单端口块 RAM 的数组上指定该指令，那么将实现单端口接口。相反，如果使用 `RESOURCE` 指令指定双端口接口，并且 Vivado HLS 判定此接口并无益处，那么它将自动实现单端口接口。

`bram` 接口模式的运作方式与 `ap_memory` 接口相同。唯一差异是在 Vivado IP integrator 中使用设计时，端口的实现方式。

- `ap_memory` 接口显示为多个独立端口。
- `bram` 接口显示为单个组合端口，可使用单一点对点连接来连接到赛灵思块 RAM。

如果按顺序访问数组，可使用 `ap_fifo` 接口。就像 `ap_hs` 接口一样，如果 Vivado HLS 判定未按顺序进行数据访问，那么它将停止；如果无法判定是否采用顺序访问，则将发出警告；如果判定已采用顺序方式访问，则不发出任何消息。`ap_fifo` 接口只能用于读取或写入，不能用于同时读写。

`ap_bus` 接口可与总线网桥进行通信。此接口不遵循任何特定总线标准，但鉴于其泛用性，可配合总线网桥一起使用，从而与系统总线进行仲裁。总线网桥必须能够将所有突发写操作存入高速缓存。

接口综合与结构体

接口上的结构体默认情况下可分解为其成员元素，针对每个成员元素单独实现各端口。此结构体的每个成员元素都将在不含任何 `INTERFACE` 指令的情况下进行实现。

结构体数组作为多个数组来实现，每个结构体成员都具有独立的数组。

DATA_PACK 最优化指令用于将结构体的所有元素打包到单宽矢量中。这样即可同时读写结构体的所有成员。结构体的成员元素按 C 语言代码中所示顺序置于该矢量内：结构体的第一个元素对齐矢量的 LSB，结构体的最后一个元素对齐矢量的 MSB。结构体中的任意数组都分区到独立数组元素中，并按从低到高的顺序置于矢量内。

注释：DATA_PACK 最优化不支持对包含其它结构体的结构体进行打包。

在含大型数组的结构体上，请谨慎使用 DATA_PACK 最优化。如果某个数组包含 4096 个类型为 `int` 的元素，则将导致矢量（和端口）宽度达 $4096 \times 32 = 131072$ 位。Vivado HLS 可以创建此 RTL 设计，但是逻辑综合在 FPGA 实现期间很可能无法完成其布线。

使用 DATA_PACK 指令创建的单宽矢量支持在单一时钟周期内访问更多数据。前提是结构体包含数组。在单一时钟周期内访问数据时，Vivado HLS 会自动展开使用此数据的所有循环（前提是可提高吞吐量）。循环可全部或部分展开以创建足够的硬件以便在单一时钟周期内使用更多数据。此功能可使用 `config_unroll` 命令和 `tripcount_threshold` 选项来加以控制。在以下示例中，行程计数小于 16 的任何循环都将自动展开（前提是可提高吞吐量）。

```
config_unroll -tripcount_threshold 16
```

注释：对于 AXIM 接口，仅支持使用 DATA_PACK 最优化打包的结构体。

如果使用 DATA_PACK 的结构体端口将使用 AXI4 接口来实现，那么可以考虑使用 DATA_PACK `-byte_pad` 选项。`-byte_pad` 选项用于自动将成员元素对齐到 8 位边界。有时，赛灵思 IP 具有此对齐要求。如果要实现使用 DATA_PACK 的 AXI4 端口，请参阅该端口连接到的赛灵思 IP 文档并确定是否需要字节对齐。

对于以下示例代码，实现结构体端口的选项如下图所示。

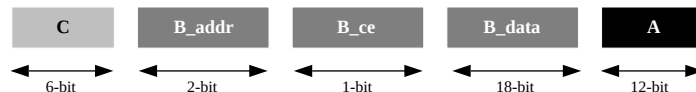
```
typedef struct{
    int12 A;
    int18 B[4];
    int6 C;
} my_data;

void foo(my_data *a )
```

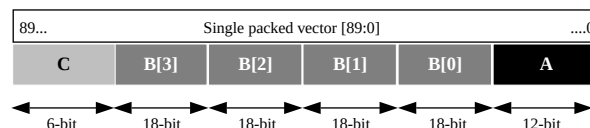
- 默认情况下，成员作为独立端口来实现。数组具有多个端口（数据、地址等）。
- 使用 DATA_PACK 会生成单宽端口。
- 使用含 `struct_level` 字节填充的 DATA_PACK 会将整个结构体对齐到下一个 8 位边界。
- 使用含 `field_level` 字节填充的 DATA_PACK 会将每个结构体成员对齐到下一个 8 位边界。
- 由数据打包创建的任意端口或总线的最大位宽为 8192 位。

图 42：DATA_PACK -byte_pad 对齐选项

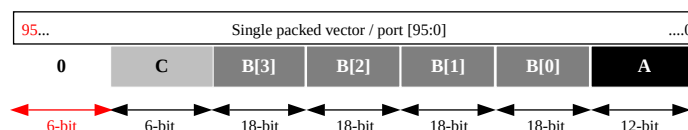
Struct Port Implementation



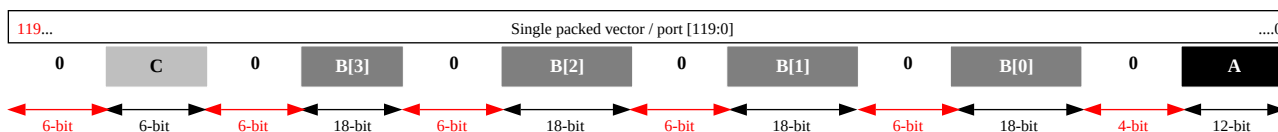
DATA_PACK optimization



DATA_PACK optimization with byte_pad on the struct_level



DATA_PACK optimization with byte_pad on the field_level



X14292

如果结构体包含数组，则可通过如下方式来对这些数组进行最优化：使用 ARRAY_PARTITION 指令对数组进行分区，或者使用 ARRAY_RESHAPE 指令对数组进行分区并将已分区的元素重新组合到更宽的数组中。DATA_PACK 指令执行的操作与 ARRAY_RESHAPE 类似，可将重构的数组与结构体中的其它元素组合在一起。

使用 DATA_PACK 对结构体进行最优化后，无法对该结构体再进行分区或重构。DATA_PACK、ARRAY_PARTITION 与 ARRAY_RESHAPE 指令互斥。

接口综合和多次访问指针

使用多次访问的指针可能会在综合后引发意外行为。在以下示例中，对指针 d_i 执行了 4 次读取，对指针 d_o 执行了 2 次写入：指针执行了多次访问。

```
#include "pointer_stream_bad.h"

void pointer_stream_bad ( dout_t *d_o,  din_t *d_i) {
    din_t acc = 0;

    acc += *d_i;
    acc += *d_i;
    *d_o = acc;
    acc += *d_i;
    acc += *d_i;
    *d_o = acc;
}
```

综合后，此代码产生的 RTL 设计将读取 1 次输入端口，写入 1 次输出端口。与任何标准 C 语言编译器一样，Vivado HLS 将优化掉多余的指针访问。要按“预期”实现上述代码，即对 `d_i` 读取 4 次，对 `d_o` 写入 2 次，必须将指针指定为 `volatile`，如以下示例所示。

```
#include "pointer_stream_better.h"

void pointer_stream_better ( volatile dout_t *d_o,  volatile din_t *d_i) {
    din_t acc = 0;

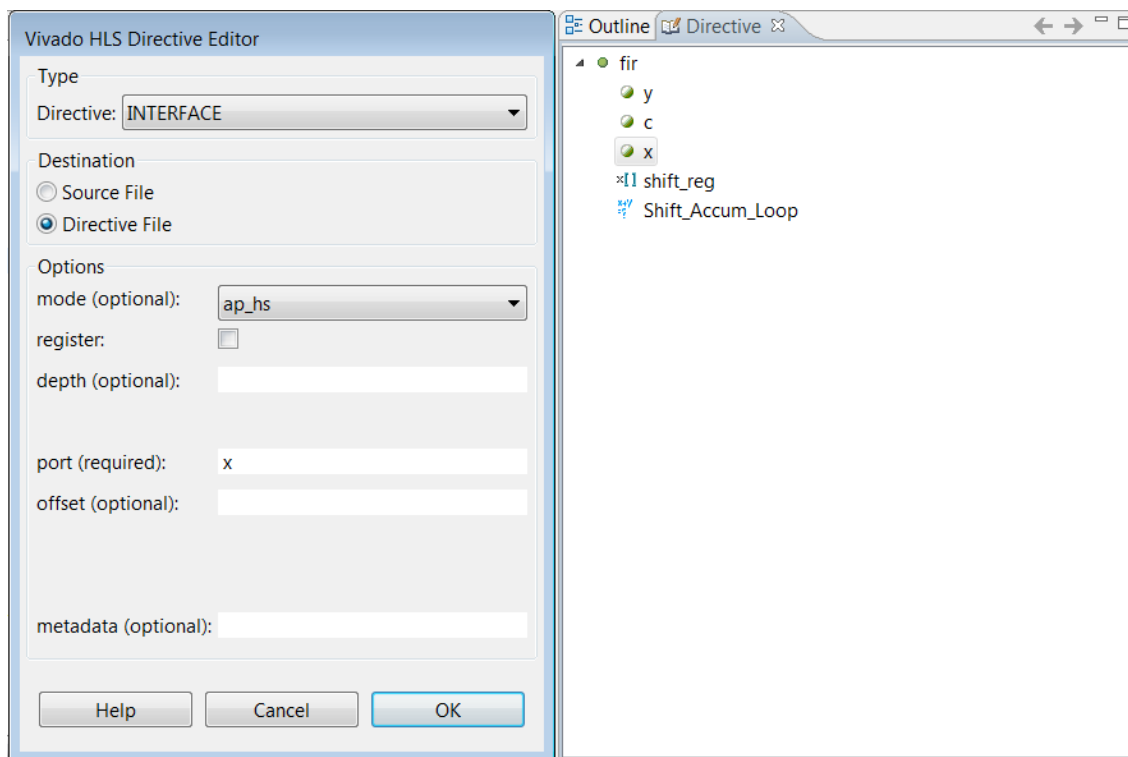
    acc += *d_i;
    acc += *d_i;
    *d_o = acc;
    acc += *d_i;
    acc += *d_i;
    *d_o = acc;
}
```

即使此段 C 语言代码也有问题。实际上，在测试激励文件上，除了为 `d_i` 提供一个值之外，无法执行任何其它操作，或者除了最终写入之外，也不能验证任何对 `d_o` 的写入。尽管支持多次访问指针，但强烈建议使用 `hls::stream` 类实现所需的行为。如需了解有关 `hls::stream` 类的详细信息，请参阅 [HLS 流传输库](#)。

指定接口

接口综合可通过 `INTERFACE` 指令或使用配置设置来加以控制。要在端口上指定接口模式，请在 GUI 的“Directives”选项卡中选择端口，右键单击并选择“Insert Directive”以打开“Vivado HLS Directive Editor”，如下图所示。

图 43：指定端口接口



在“Vivado HLS Directives Editor”中，设置以下选项：

- “mode”

从下拉菜单中选择接口模式。

- “register”

如果选择该选项，则将在首个操作周期内执行所有按值传递读取操作。对于输出端口，“register”选项可保证对输出进行寄存。您可将“register”选项应用于设计中的任意函数。对于内存、FIFO 和 AXI4 接口，“register”选项无效。

- “depth”

该选项用于指定测试激励文件向设计提供的样本数量以及测试激励文件必须存储的输出值数量。以更大的数值为准。

注释：对于在单一传输事务内多次读取或写入的指针，需选中“depth”选项才能执行 C/RTL 协同仿真。对于数组而言或者使用 `hls::stream` 构造时，“depth”选项并不是必需的。仅当在接口上使用指针时，才需选中该选项。

如果“depth”选项设置过小，那么 C/RTL 协同仿真可能出现死锁，如下所示：

1. 输入读取操作可能停滞并等待数据，而测试激励文件无法提供这些数据。
2. 输出写入在尝试写入数据时可能停滞，因为存储空间已满。

- “port”

该选项为必需。默认情况下，Vivado HLS 不寄存端口。

注释：要指定块级 I/O 协议，请在 Vivado HLS GUI 中选择顶层函数，并指定函数返回的端口。

- “offset”

该选项适用于 AXI4 接口。

要设置接口配置，请选择“Solution”→“Solution Settings”→“General”→“config_interface”。您可使用配置设置来执行如下操作：

- 向 RTL 设计添加全局时钟使能。
- 移除无关联的端口，例如，由设计中不使用的结构体元素所创建的端口。
- 为任意全局变量创建 RTL 端口。

任何 C 语言函数均可使用全局变量：即定义的作用域超出任意函数作用域的变量。默认情况下，全局变量不会导致创建 RTL 端口：Vivado HLS 假定全局变量位于最终设计内。`config_interface` 配置设置 `expose_global` 会指示 Vivado HLS 为全局变量创建端口。

对应 SystemC 的接口综合

一般而言，针对 SystemC 语言设计不支持接口综合。SystemC 语言设计的 I/O 端口完全在 `SC_MODULE` 接口内指定，并在源代码中完整描述端口行为。提供接口综合的目的是为了支持：

- 内存块 RAM 接口
- AXI4-Stream 接口
- AXI4-Lite 接口
- AXI4 主接口

在 SystemC 语言设计上执行接口综合的过程不同于将这些接口添加到 C 或 C++ 设计的过程。

- 内存块 RAM 和 AXI4 主接口要求将 SystemC 数据端口替换为 Vivado HLS 端口。
- AXI4-Stream 和 AXI4-Lite 从接口只需使用指令即可，但向 SystemC 语言设计添加指令的流程不同。

对 SystemC 应用接口指令

将指令作为编译指示添加到 SystemC 语言源代码中时，对于 SC_MODULE 声明中指定的端口无法添加编译指示指令，必须在 SC_MODULE 调用的函数内添加这些指令。

使用 GUI 添加指令时：

- 打开 C 语言源代码和“Directives”选项卡。
- 选择需指令的函数。
- 右键单击并单击 INTERFACE 指令以将其应用于函数。

这些指令可应用于 SC_MODULE 的任意成员函数，但最好将其添加到使用变量的函数中。

块 RAM 内存端口

给定接口上含数组端口的 SystemC 语言设计：

```
SC_MODULE(my_design) {
// "RAM" Port
sc_uint<20> my_array[256];
```

端口 my_array 将综合到内部块 RAM 中，而不是块 RAM 接口端口中。

包含 Vivado HLS 头文件 ap_mem_if.h 即可允许将同一个端口指定为 ap_mem_port<data-width, address-bits> 端口。ap_mem_port 数据类型综合到含指定数据和地址总线宽度并使用 ap_memory 端口协议的标准块 RAM 接口中。

```
#include "ap_mem_if.h"
SC_MODULE(my_design) {
// "RAM" Port
ap_mem_port<sc_uint<20>,sc_uint<8>, 256> my_array;
```

将 ap_mem_port 添加到 SystemC 语言设计中时，必须将关联的 ap_mem_chn 添加到 SystemC 语言测试激励文件中，以驱动 ap_mem_port。在测试激励文件中，定义了 1 个 ap_mem_chn 并将其附加到实例，如下所示：

```
#include "ap_mem_if.h"
ap_mem_chn<int,int, 68> bus_mem;

// Instantiate the top-level module
my_design U_dut ( "U_dut" )
U_dut.my_array.bind(bus_mem);
```

头文件 ap_mem_if.h 位于 Vivado HLS 安装区域中的 include 目录内，如果在 Vivado HLS 外部执行仿真，则必须包含此头文件。

SystemC AXI4-Stream 接口

AXI4-Stream 接口可添加到类型为 `sc_fifo_in` 或 `sc_fifo_out` 的任意 SystemC 端口。以下显示了典型 SystemC 语言设计的顶层。通常，在头文件中定义 `SC_MODULE` 和端口：

```
SC_MODULE(sc_FIFO_port)
{
    //Ports
    sc_in<bool>  clock;
    sc_in<bool>  reset;
    sc_in<bool>  start;
    sc_out<bool> done;
    sc_fifo_out<int> dout;
    sc_fifo_in<int> din;

    //Variables
    int share_mem[100];
    bool write_done;

    //Process Declaration
    void Prc1();
    void Prc2();

    //Constructor
    SC_CTOR(sc_FIFO_port)
    {
        //Process Registration
        SC_CTHREAD(Prc1,clock.pos());
        reset_signal_is(reset,true);

        SC_CTHREAD(Prc2,clock.pos());
        reset_signal_is(reset,true);
    }
};
```

要创建 AXI4-Stream 接口，必须使用 `RESOURCE` 指令来指定将端口连接到 AXI4-Stream 资源。对于以上所示接口示例，其中显示在 `SC_MODULE` 调用的函数中添加指令：指定端口 `din` 和 `dout` 包含 AXI4-Stream 资源。

```
#include "sc_FIFO_port.h"

void sc_FIFO_port::Prc1()
{
    //Initialization
    write_done = false;

    wait();
    while(true)
    {
        while (!start.read()) wait();
        write_done = false;

        for(int i=0;i<100; i++)
            share_mem[i] = i;

        write_done = true;
        wait();
    } //end of while(true)
}
```

```
void sc_FIFO_port::Prc2()
{
#pragma HLS resource core=AXI4Stream variable=din
#pragma HLS resource core=AXI4Stream variable=dout
    //Initialization
    done = false;

    wait();

    while(true)
    {
        while (!start.read()) wait();
        wait();
        while (!write_done) wait();
        for(int i=0;i<100; i++)
        {
            dout.write(share_mem[i]+din.read());
        }

        done = true;
        wait();
    } //end of while(true)
}
```

对 SystemC 语言设计进行综合时，会生成含标准 RTL FIFO 端口的 RTL 设计。使用“Export RTL”工具栏按钮  将设计封装为 IP 时，输出即为含 AXI4-Stream 接口的设计。

SystemC AXI4-Lite 接口

AXI4-Lite 从接口可添加到类型为 `sc_in` 或 `sc_out` 的任意 SystemC 端口。以下示例显示了典型 SystemC 语言设计的顶层。在此情况下，`SC_MODULE` 和端口均在头文件内定义（与典型状况相同）：

```
SC_MODULE(sc_sequ_cthread){
    //Ports
    sc_in<bool>  clk;
    sc_in<bool>  reset;
    sc_in<bool>  start;
    sc_in<sc_uint<16>> a;
    sc_in<bool>  en;
    sc_out<sc_uint<16>> sum;
    sc_out<bool> vld;

    //Variables
    sc_uint<16> acc;

    //Process Declaration
    void accum();

    //Constructor
    SC_CTOR(sc_sequ_cthread){

    //Process Registration
    SC_CTHREAD(accum,clk.pos());
    reset_signal_is(reset,true);
    }
};
```

要创建 AXI4-Lite 接口，必须使用 RESOURCE 指令来指定将端口连接到 AXI4-Lite 资源。对于以上所示的示例接口，以下示例显示了如何将 `start`、`a`、`en`、`sum` 和 `vld` 端口分组到同一个 AXI4-Lite 接口内 `slv0`：以相同 `bus_bundle` 名称指定所有端口，并将其分组到同一个 AXI4-Lite 接口内。

```
#include "sc_sequ_cthread.h"

void sc_sequ_cthread::accum(){
//Group ports into AXI4 slave slv0
#pragma HLS resource core=AXI4LiteS metadata="-bus_bundle slv0"
variable=start
#pragma HLS resource core=AXI4LiteS metadata="-bus_bundle slv0" variable=a
#pragma HLS resource core=AXI4LiteS metadata="-bus_bundle slv0" variable=en
#pragma HLS resource core=AXI4LiteS metadata="-bus_bundle slv0" variable=sum
#pragma HLS resource core=AXI4LiteS metadata="-bus_bundle slv0" variable=vld

//Initialization
acc=0;
sum.write(0);
vld.write(false);
wait();

// Process the data
while(true) {
// Wait for start
wait();
while (!start.read()) wait();

// Read if valid input available
if (en) {
acc = acc + a.read();
sum.write(acc);
vld.write(true);
} else {
vld.write(false);
}
}
}
```

对 SystemC 语言设计进行综合时，会生成含标准 RTL 端口的 RTL 设计。使用“Export RTL”工具栏按钮  将设计封装为 IP 时，输出即为含 AXI4-Lite 接口的设计。

SystemC AXI4 主接口

在大部分标准 SystemC 语言设计中，无需指定含 Vivado HLS `ap_bus` I/O 协议行为的端口。但如果设计需要 AXI4 主总线接口，则需要 `ap_bus` I/O 协议。

要在 SystemC 语言设计上指定 AXI4 主接口，请执行以下操作：

- 使用 Vivado HLS 类型 `AXI4M_bus_port` 来创建含 `ap_bus` I/O 协议的接口。
- 将 AXI4M 资源分配到端口。

以下示例显示了如何将称为 `bus_if` 的 `AXI4M_bus_port` 添加到 SystemC 语言设计中。

- 头文件 `AXI4_if.h` 必须添加到设计中。
- 此端口定义为 `AXI4M_bus_port<type>`，其中 `type` 用于指定要使用的数据类型（在此示例中，使用的类型为 `sc_fixed`）。

注释：AXI4M_bus_port 中使用的数据类型必须为 8 位的倍数。此外，此数据类型不支持结构体。

```
#include "systemc.h"
#include "AXI4_if.h"
#include "tlm.h"
using namespace tlm;

#define DT sc_fixed<32, 8>

SC_MODULE(dut)
{
    //Ports
    sc_in<bool> clock; //clock input
    sc_in<bool> reset;
    sc_in<bool> start;
    sc_out<int> dout;
    AXI4M_bus_port<sc_fixed<32, 8> > bus_if;

    //Variables

    //Constructor
    SC_CTOR(dut)
    {
        //:bus_if ("bus_if")
        {
            //Process Registration
            SC_CTHREAD(P1,clock.pos());
            reset_signal_is(reset,true);
        }
    }
}
```

以下显示了如何在 SystemC 语言函数中访问 bus_if 变量以生成标准或突发读写操作。

```
//Process Declaration
void P1() {
    //Initialization
    dout.write(10);
    int addr = 10;
    DT tmp[10];
    wait();
    while(1) {
        tmp[0]=10;
        tmp[1]=11;
        tmp[2]=12;

        while (!start.read()) wait();
        // Port read
        tmp[0] = bus_if->read(addr);

        // Port burst read
        bus_if->burst_read(addr,2,tmp);

        // Port write
        bus_if->write(addr, tmp);

        // Port burst write
        bus_if->burst_write(addr,2,tmp);

        dout.write(tmp[0].to_int());
    }
}
```

```

    addr+=2;
    wait();
  }
}

```

在设计中使用端口类 `AXI4M_bus_port` 时，测试激励文件中必须具有与之匹配的 HLS 总线接口通道 `hls_bus_chn<start_addr >`，如以下示例所示：

```

#include <systemc.h>
#include "tlm.h"
using namespace tlm;

#include "hls_bus_if.h"
#include "AE_clock.h"
#include "driver.h"
#ifdef __RTL_SIMULATION__
#include "dut_rtl_wrapper.h"
#define dut dut_rtl_wrapper
#else
#include "dut.h"
#endif

int sc_main (int argc , char *argv[])
{
    sc_report_handler::set_actions( "/IEEE_Std_1666/deprecated",
    SC_DO_NOTHING);
    sc_report_handler::set_actions( SC_ID_LOGIC_X_TO_BOOL_, SC_LOG);
    sc_report_handler::set_actions( SC_ID_VECTOR_CONTAINS_LOGIC_VALUE_,
    SC_LOG);
    sc_report_handler::set_actions( SC_ID_OBJECT_EXISTS_, SC_LOG);

    // hls_bus_chan<type>
    // bus_variable( "name" , start_addr, end_addr)
    //
    hls_bus_chn<sc_fixed<32, 8> > bus_mem("bus_mem",0,1024);

    sc_signal<bool>          s_clk;
    sc_signal<bool>          reset;
    sc_signal<bool>          start;
    sc_signal<int>           dout;

    AE_Clock    U_AE_Clock("U_AE_Clock", 10);
    dut         U_dut("U_dut");
    driver       U_driver("U_driver");

    U_AE_Clock.reset(reset);
    U_AE_Clock.clk(s_clk);

    U_dut.clock(s_clk);
    U_dut.reset(reset);
    U_dut.start(start);
    U_dut.dout(dout);
    U_dut.bus_if(bus_mem);

    U_driver.clk(s_clk);
    U_driver.start(start);
    U_driver.dout(dout);

    int end_time = 8000;
}

```

```
cout << "INFO: Simulating " << endl;

// start simulation
sc_start(end_time, SC_NS);

return U_driver.ret;
};
```

综合后的 RTL 设计包含具有 ap_bus I/O 协议的接口。

使用 AXI4M_bus_port 类时，会生成含 ap_bus 接口的 RTL 设计。使用“Export RTL”将设计封装为 IP 时，输出即为含有 AXI4 主端口的设计。

使用 AXI4 接口

AXI4-Stream 接口

AXI4-Stream 接口可应用于任意输入实参和任意数组或指针输出实参。由于 AXI4-Stream 接口按顺序流传输方式来传输数据，因此不可用于同时支持读写的实参。AXI4-Stream 接口始终通过符号位扩展方式扩展至下一个字节。例如，12 位数据值通过符号位扩展方式扩展为 16 位。

AXI4-Stream 接口始终作为已寄存的接口来实现，以便在将含有 AXI-Stream 接口的多个 HLS IP 块集成到大型设计中时，确保不会创建组合反馈路径。对于 AXI-Stream 接口，提供了 4 种类型的寄存器模式，以控制 AXI-Stream 接口寄存器的实现方式。

- 正向：仅寄存 TDATA 和 TVALID 信号。
- 反向：仅寄存 TREADY 信号。
- 双向：寄存所有信号（TDATA、TREADY 和 TVALID）。这是默认方式。
- 关：不寄存任何端口信号。

AXI-Stream 旁路信号被视为数据信号，随 TDATA 一起寄存。



建议：将 HLS 生成的 IP 块与 AXI4-Stream 接口相连时，应将至少 1 个接口实现为寄存接口，或者这些块应通过 AXI4-Stream Register Slice 连接。

在设计中可通过 2 种基本方式使用 AXI4-Stream。

- 使用不含旁路的 AXI4-Stream。
- 使用含旁路的 AXI4-Stream。

第 2 种使用模型可提供额外功能，允许将属于 AXI4-Stream 标准的可选旁路直接用于 C 语言代码中。

不含旁路的 AXI4-Stream 接口

当函数实参不包含任何 AXI4 旁路元素时，即可使用不含旁路的 AXI4-Stream。以下示例显示的设计中，数据类型为标准 C 语言 int 类型。在此示例中，2 个接口均使用 AXI4-Stream 来实现。

```
void example(int A[50], int B[50]) {
//Set the HLS native interface types
#pragma HLS INTERFACE axis port=A
#pragma HLS INTERFACE axis port=B
```

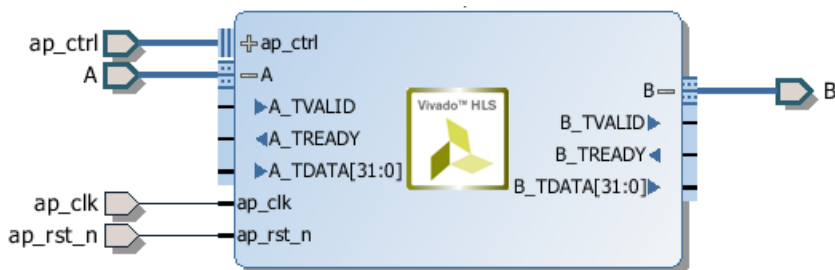


```
int i;

for(i = 0; i < 50; i++){
    B[i] = A[i] + 5;
}
}
```

综合后，2 个实参均使用数据端口以及标准 AXI4-Stream TVALID 和 TREADY 协议端口（如下图所示）来实现。

图 44：不含旁路的 AXI4-Stream 接口



可通过使用结构体和 `DATA_PACK` 指令来将多个变量组合到同一个 AXI4-Stream 接口中。如果顶层函数的实参为结构体，那么默认情况下，Vivado HLS 会将此结构体分区到独立元素中，并将此结构体的每个成员作为独立端口来实现。但是，`DATA_PACK` 指令可用于将结构体的元素封装到单宽矢量中，以便在同一个 AXI4-Stream 接口中实现所有元素。

含旁路的 AXI4-Stream 接口

旁路为属于 AXI4-Stream 标准的一部分的可选信号。在 C 语言代码中可使用结构体直接引用和控制旁路信号，前提是结构体的成员元素与 AXI4-Stream 旁路信号的名称相匹配。AXI-Stream 旁路信号被视为数据信号，随 `TDATA` 一起寄存。Vivado HLS 提供了相关示例。Vivado HLS include 目录包含 `ap_axi_sdata.h` 文件。此头文件包含以下结构体：

```
#include "ap_int.h"
#include "ap_axi_sdata.h"

template<int D,int U,int TI,int TD>
struct ap_axis{
    ap_int<D>    data;
    ap_uint<D/8> keep;
    ap_uint<D/8> strb;
    ap_uint<U>   user;
    ap_uint<1>   last;
    ap_uint<TI>  id;
    ap_uint<TD>  dest;
};

template<int D,int U,int TI,int TD>
struct ap_axiu{
    ap_uint<D>    data;
    ap_uint<D/8> keep;
    ap_uint<D/8> strb;
    ap_uint<U>   user;
    ap_uint<1>   last;
    ap_uint<TI>  id;
    ap_uint<TD>  dest;
};
```

2 个结构体都包含变量作为顶层成员，这些变量的名称与可选 AXI4-Stream 旁路信号的名称相匹配。只要结构体包含具有这些名称的元素，就无需使用提供的头文件。您可创建自己的用户定义的结构体。由于以上所示结构体使用 `ap_int` 类型和模板，因此此头文件仅限用于 C++ 设计。

注释：有效信号和就绪信号在 AXI4-Stream 中属于必需信号，始终由 Vivado HLS 来实现。这些信号无法使用结构体来控制。

以下示例显示了如何在 C 语言代码中直接使用旁路，并在接口上实现这些旁路。在此示例中，使用了有符号的 32 位数据类型。

```
#include "ap_axi_sdata.h"

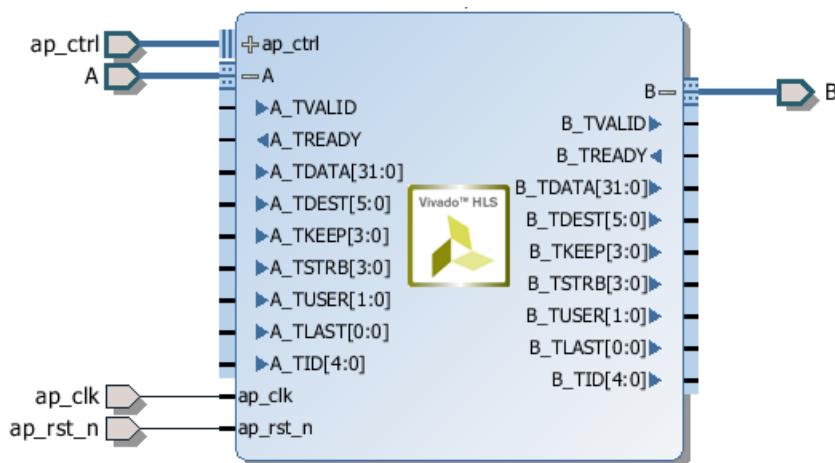
void example(ap_axis<32,2,5,6> A[50], ap_axis<32,2,5,6> B[50]){
//Map ports to Vivado HLS interfaces
#pragma HLS INTERFACE axis port=A
#pragma HLS INTERFACE axis port=B

    int i;

    for(i = 0; i < 50; i++){
        B[i].data = A[i].data.to_int() + 5;
        B[i].keep = A[i].keep;
        B[i].strb = A[i].strb;
        B[i].user = A[i].user;
        B[i].last = A[i].last;
        B[i].id = A[i].id;
        B[i].dest = A[i].dest;
    }
}
```

完成综合后，这 2 个实参都会随数据端口、标准 AXI4-Stream TVALID 和 TREADY 协议端口以及结构体中所描述的所有可选端口一起实现。

图 45：含旁路的 AXI4-Stream 接口



将结构体封装到 AXI4-Stream 接口中

将结构体与 AXI4-Stream 接口配合使用时，默认综合行为会发生改变。如需了解结构体的默认综合行为，请参阅[接口综合与结构体](#)。

使用不含旁路的 AXI4-Stream 接口且函数实参为结构体时：

- Vivado HLS 会自动应用 `DATA_PACK` 指令，且结构体的所有元素都将组合到单宽数据矢量中。此接口将作为含关联 `TVALID` 信号和 `TREADY` 信号的单宽数据矢量来实现。
- 如果将 `DATA_PACK` 指令手动应用于结构体，那么该结构体的所有元素都将组合到单宽数据矢量中，并且可应用对应 `DATA_PACK` 指令的 AXI 对齐选项。此接口将作为含关联 `TVALID` 信号和 `TREADY` 信号的单宽数据矢量来实现。

使用含旁路的 AXI4-Stream 接口时，函数实参本身即结构体（AXI-Stream 结构体）。其中包含本身即为结构体（数据结构体）的数据以及旁路：

- Vivado HLS 会自动将 `DATA_PACK` 指令应用于此数据结构体，并将此数据结构体的所有元素都组合到单宽数据矢量中。此接口将作为含关联旁路、`TVALID` 信号和 `TREADY` 信号的单宽数据矢量来实现。
- 如果将 `DATA_PACK` 指令手动应用于数据结构体，那么该数据结构体的所有元素都将组合到单宽数据矢量中，并且可应用对应 `DATA_PACK` 指令的 AXI 对齐选项。此接口将作为含关联旁路、`TVALID` 信号和 `TREADY` 信号的单宽数据矢量来实现。
- 如果将 `DATA_PACK` 指令应用于 AXI-Stream 结构体，那么函数实参、数据结构体和旁路信号将组合到单宽矢量中。此接口将作为含 `TVALID` 信号和 `TREADY` 信号的单宽数据矢量来实现。

AXI4-Lite 接口

您可使用 AXI4-Lite 接口来允许通过 CPU 或微控制器控制设计。通过使用 Vivado HLS AXI4-Lite 接口可以：

- 将多个端口分组到同一个 AXI4-Lite 接口。
- 输出 C 驱动文件以与处理器上运行的代码配合使用。

注释：这样即可提供一组 C 语言应用编程接口 (API) 函数，以便您轻松通过软件控制硬件。当设计导出到 IP 目录时，此操作很实用。

以下示例演示了 Vivado HLS 将多个实参（包括函数返回）作为 AXI4-Lite 接口来实现的方式。由于针对 `bundle` 选项每条指令都使用相同名称，因此每个端口都分组到同一个 AXI4-Lite 接口中。

```
void example(char *a, char *b, char *c)
{
    #pragma HLS INTERFACE s_axilite port=return bundle=BUS_A
    #pragma HLS INTERFACE s_axilite port=a bundle=BUS_A
    #pragma HLS INTERFACE s_axilite port=b bundle=BUS_A
    #pragma HLS INTERFACE s_axilite port=c bundle=BUS_A offset=0x0400
    #pragma HLS INTERFACE ap_vld port=b

    *c += *a + *b;
}
```

注释：如果不使用 `bundle` 选项，Vivado HLS 会将使用 AXI4-Lite 接口指定的所有实参都分组到同一个默认捆绑中，并自动对端口进行命名。

您还可向分组到同一个 AXI4-Lite 接口的端口分配 I/O 协议。在上述示例中，Vivado HLS 将端口 `b` 作为 `ap_vld` 接口来实现，并将端口 `b` 分组到 AXI4-Lite 接口中。因此，AXI4-Lite 接口包含 3 个寄存器，分别用于端口 `b` 的数据、确认已读取端口 `b` 的输出，以及端口 `b` 的输入有效信号。

每次读取端口 `b` 时，Vivado HLS 都会自动清除输入有效寄存器，并将该寄存器复位为逻辑 0。如果输入有效寄存器未设置为逻辑 1，那么 `b` 数据寄存器中的数据将不会被视为有效，设计将停滞并等待有效寄存器完成置位。



建议：为了便于设计操作，赛灵思建议在已分组到同一个 AXI4-Lite 接口的端口内不要包含额外的 I/O 协议。但是，赛灵思建议您包含与 AXI4-Lite 接口中的 `return` 端口关联的块级 I/O 协议。

您无法使用 `bram` 接口向 AXI4-Lite 接口分配数组。只能使用默认 `ap_memory` 接口向 AXI4-Lite 接口分配数组。您也无法将使用 `ap_stable` I/O 协议指定的任何实参分配到 AXI4-Lite 接口。

由于已分组到同一个 AXI4-Lite 接口的变量均为函数实参，无法在 C 语言代码中为其指定默认值，因此也无法为 AXI4-Lite 接口中的寄存器指定默认值。这些寄存器可通过 `config_rtl` 命令进行复位来实现，但不能为其指定任何其它默认值。

默认情况下，Vivado HLS 会为分组到同一个 AXI4-Lite 接口的每个端口分配地址。Vivado HLS 可在 C 驱动文件中提供已分配的地址。如需了解更多信息，请参阅 [C 驱动文件](#)。要显式定义地址，可使用 `offset` 选项，如以上示例中的实参 `c` 用例所示。



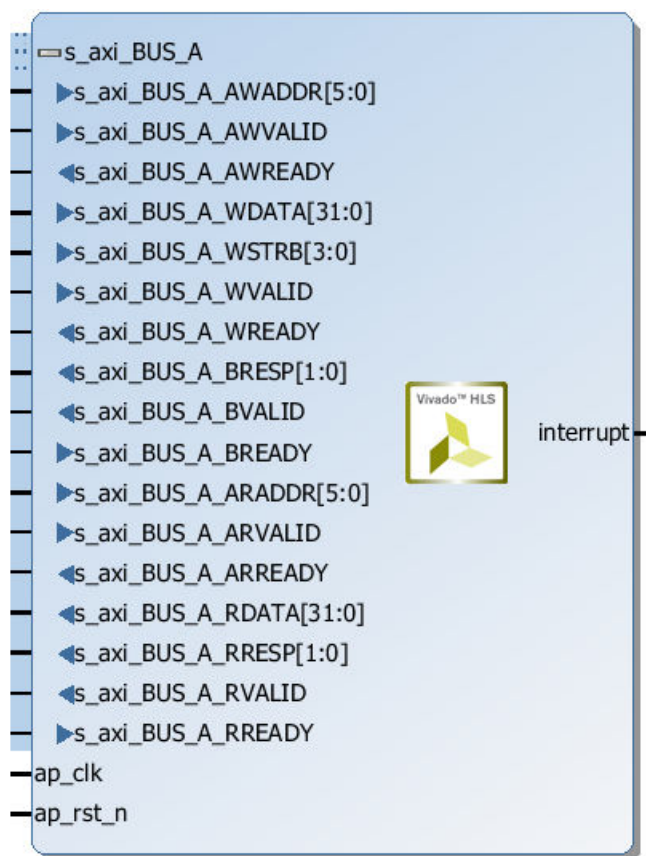
重要提示！ 在 AXI4-Lite 接口中，Vivado HLS 保留地址 0x0000 到 0x000C，用于块级 I/O 协议信号和中断控制。

综合后，Vivado HLS 会实现 AXI4-Lite 接口内的端口，如下图所示。Vivado HLS 会在 AXI4-Lite 接口内包含函数返回来中断端口。您可通过 AXI4-Lite 接口来完成中断编程。您还可通过下列块级协议来驱动中断：

- `ap_done`：指示函数完成所有操作的时间。
- `ap_ready`：指示函数准备好处理新输入数据的时间。

您可使用 C 驱动文件来进行接口编程。

图 46：含已分组的 RTL 端口的 AXI4-Lite 从接口



控制 AXI4-Lite 接口中的时钟和复位

默认情况下，Vivado HLS 针对 AXI4-Lite 接口和已综合的设计使用相同时钟。Vivado HLS 将 AXI4-Lite 接口中的所有寄存器都连接到用于综合逻辑的时钟 (ap_clk)。

(可选) 您可使用 INTERFACE 指令 clock 选项来为每个 AXI4-Lite 端口使用独立的时钟。将时钟连接到 AXI4-Lite 接口时，必须使用以下协议：

- AXI4-Lite 接口时钟必须同步到用于综合逻辑 (ap_clk) 的时钟。即，这 2 个时钟必须衍生自同一个主生成器时钟。
- AXI4-Lite 接口时钟频率必须等于或小于用于综合逻辑 (ap_clk) 的时钟的频率。

如果将 clock 选项与接口指令搭配使用，那么只需针对每个捆绑中的 1 个函数实参指定 clock 选项即可。Vivado HLS 通过相同的时钟和复位来实现捆绑中的所有其它函数实参。Vivado HLS 以 ap_rst_ 作为前缀并后接时钟名称的方式来对生成的复位信号命名。生成的复位信号处于低电平有效状态，与 config_rtl 命令无关。

以下示例显示了 Vivado HLS 如何将函数实参 a 和 b 组合到同一个 AXI4-Lite 端口内，且该端口含名为 AXI_clk1 的时钟和关联的复位端口。

```
// Default AXI-Lite interface implemented with independent clock called
AXI_clk1
#pragma HLS interface s_axilite port=a clock=AXI_clk1
#pragma HLS interface s_axilite port=b
```

在以下示例中，Vivado HLS 将函数实参 c 和 d 组合到 AXI4-Lite 端口 CTRL1 内，该端口含名为 AXI_clk2 的独立时钟和关联的复位端口。

```
// CTRL1 AXI-Lite bundle implemented with a separate clock (called AXI_clk2)
#pragma HLS interface s_axilite port=c bundle=CTRL1 clock=AXI_clk2
#pragma HLS interface s_axilite port=d bundle=CTRL1
```

C 驱动文件

当实现 AXI4-Lite 从接口时，会自动创建一组 C 驱动文件。这些 C 驱动文件可提供一组 API 以集成到 CPU 上运行的任意软件中，并且这些 API 可用于通过 AXI4-Lite 从接口与器件进行通信。

这些 C 驱动文件是在 IP 目录中将设计封装为 IP 时创建的。

针对独立模式和 Linux 模式会创建驱动文件。在独立模式下，驱动的使用方式与任何其它赛灵思独立驱动的使用方式并无不同。在 Linux 模式下，会将所有 C 语言文件 (.c) 和头文件 (.h) 都复制到软件工程中。

驱动文件和 API 函数的名称衍生自综合的顶层函数。在上述示例中，顶层函数称为“example”。如果顶层函数名为“DUT”，那么在以下描述中名称“example”将被替换为“DUT”。驱动文件是在封装的 IP 内创建的（位于 solution 内的 impl 目录中）。

表 9：名为 example 的设计的 C 驱动文件

文件路径	使用模式	描述
data/example.mdd	独立	驱动定义文件。
data/example.tcl	独立	供 SDK 用于将软件集成到 SDK 工程中。
src/xexample_hw.h	两种模式均可	定义所有内部寄存器的地址偏移。
src/xexample.h	两种模式均可	API 定义
src/xexample.c	两种模式均可	独立 API 实现

表 9：名为 `example` 的设计的 C 驱动文件 (续)

文件路径	使用模式	描述
src/xexample_sinit.c	独立	初始化 API 实现
src/xexample_linux.c	Linux	初始化 API 实现
src/Makefile	独立	Makefile

在 `xexample.h` 文件中定义了 2 个结构体。

- `XExample_Config`：用于保存 IP 实例的配置信息（每个 AXI4-Lite 从接口的基址）。
- `XExample`：用于保存 IP 实例指针。大部分 API 使用此实例指针作为首个实参。

标准 API 实现在文件 `xexample.c`、`xexample_sinit.c` 和 `xexample_linux.c` 中提供，可提供用于以下操作的函数。

- 初始化器件
- 控制器件并查询其状态
- 读写寄存器
- 设置、监测和控制中断

下表列出了 C 驱动文件中提供的每个 API 函数。

表 10：C 驱动 API 函数

API 函数	描述
<code>XExample_Initialize</code>	此 API 向 <code>InstancePtr</code> 写入值，以供后续在其它 API 中使用。建议调用此 API 来初始化器件（系统中使用 MMU 时除外）。
<code>XExample_CfgInitialize</code>	初始化器件配置。在系统中使用 MMU 时，调用此函数前请先将 <code>XDut_Config</code> 变量中的基址替换为虚拟基址。不可用于 Linux 系统。
<code>XExample_LookupConfig</code>	用于按 ID 获取器件的配置信息。配置信息包含物理基址。不适用于 Linux 用户。
<code>XExample_Release</code>	在 Linux 中释放 uio 器件。通过 <code>munmap</code> 删除映射：如果终止此进程，将自动删除映射。仅适用于 Linux 系统。
<code>XExample_Start</code>	启动器件。此函数将在器件上断言 <code>ap_start</code> 端口有效。仅当器件上存在 <code>ap_start</code> 端口时才可用。
<code>XExample_IsDone</code>	检查器件是否已完成先前执行：此函数将返回器件上 <code>ap_done</code> 端口的值。仅当器件上存在 <code>ap_done</code> 端口时才可用。
<code>XExample_IsIdle</code>	检查器件是否处于空闲状态：此函数将返回 <code>ap_idle</code> 端口的值。仅当器件上存在 <code>ap_idle</code> 端口时才可用。
<code>XExample_IsReady</code>	检查器件是否已准备好处理下一项输入：此函数将返回 <code>ap_ready</code> 端口的值。仅当器件上存在 <code>ap_ready</code> 端口时才可用。
<code>XExample_Continue</code>	断言 <code>ap_continue</code> 端口有效。仅当器件上存在 <code>ap_continue</code> 端口时才可用。
<code>XExample_EnableAutoRestart</code>	在器件上启用“自动重启 (auto restart)”。设置此项后，当前传输事务完成时，器件将自动启动下一项传输事务。
<code>XExample_DisableAutoRestart</code>	禁用“auto restart”功能。
<code>XExample_Set_ARG</code>	将值写入端口 ARG（顶层函数的标量实参）。仅当 ARG 为输入端口时才可用。

表 10: C 驱动 API 函数 (续)

API 函数	描述
XExample_Set_ARG_vld	断言 ARG_vld 端口有效。仅当 ARG 为输入端口并使用 ap_hs 或 ap_vld 接口协议实现时才可用。
XExample_Set_ARG_ack	断言 ARG_ack 端口有效。仅当 ARG 为输出端口并使用 ap_hs 或 ap_ack 接口协议实现时才可用。
XExample_Get_ARG	从 ARG 读取值。仅当 ARG 端口为器件上的输出端口时才可用。
XExample_Get_ARG_vld	从 ARG_vld 读取值。仅当 ARG 端口为器件上的输出端口并使用 ap_hs 或 ap_vld 接口协议实现时才可用。
XExample_Get_ARG_ack	从 ARG_ack 读取值。仅当 ARG 端口为器件上的输入端口并使用 ap_hs 或 ap_ack 接口协议实现时才可用。
XExample_Get_ARG_BaseAddress	返回接口内的数组基址。仅当 ARG 为分组到 AXI4-Lite 接口内的数组时才可用。
XExample_Get_ARG_HighAddress	返回数组最上方元素的地址。仅当 ARG 为分组到 AXI4-Lite 接口内的数组时才可用。
XExample_Get_ARG_TotalBytes	返回用于存储数组的字节总数。仅当 ARG 为分组到 AXI4-Lite 接口内的数组时才可用。 如果数组中的元素少于 16 位，Vivado HLS 会将多个元素分组到 AXI4-Lite 接口的位宽为 32 位的数据中。如果元素的位宽超过 32 位，Vivado HLS 会将每个元素存储在多个连续地址上。
XExample_Get_ARG_BitWidth	返回数组中每个元素的位宽。仅当 ARG 为分组到 AXI4-Lite 接口内的数组时才可用。 如果数组中的元素少于 16 位，Vivado HLS 会将多个元素分组到 AXI4-Lite 接口的位宽为 32 位的数据中。如果元素的位宽超过 32 位，Vivado HLS 会将每个元素存储在多个连续地址上。
XExample_Get_ARG_Depth	返回数组中元素总数。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。 如果数组中的元素少于 16 位，Vivado HLS 会将多个元素分组到 AXI4-Lite 接口的位宽为 32 位的数据中。如果元素的位宽超过 32 位，Vivado HLS 会将每个元素存储在多个连续地址上。
XExample_Write_ARG_Words	将长度为 32 位的字词写入 AXI4-Lite 接口的指定地址。此 API 需要来自 BaseAddress 的偏移地址以及要存储的数据长度。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。
XExample_Read_ARG_Words	从数组中读取长度为 32 位的字词。此 API 需要数据目标、来自 BaseAddress 的偏移地址以及要存储的数据长度。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。
XExample_Write_ARG_Bytes	将字节长度写入 AXI4-Lite 接口的指定地址。此 API 需要来自 BaseAddress 的偏移地址以及要存储的数据长度。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。
XExample_Read_ARG_Bytes	从数组读取字节长度。此 API 需要数据目标、来自 BaseAddress 的偏移地址以及要加载的数据长度。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。
XExample_InterruptGlobalEnable	启用中断输出。仅当存在 ap_start 时，中断函数才可用。
XExample_InterruptGlobalDisable	禁用中断输出。
XExample_InterruptEnable	启用中断源。可存在最多 2 个中断源 (source 0 对应 ap_done, source 1 对应 ap_ready)
XExample_InterruptDisable	禁用中断源。
XExample_InterruptClear	清除中断状态。
XExample_InterruptGetEnabled	检查启用的中断源。
XExample_InterruptGetStatus	检查触发的中断源。



重要提示！ C 驱动 API 始终使用无符号的 32 位类型 (U32)。您可能需要将 C 语言代码中的数据强制转换为期望的类型。

C 驱动文件和浮点类型

C 驱动文件始终使用 32 位无符号整数 (U32) 数据进行数据传输。在以下示例中，该函数使用浮点类型实参 `a` 和 `r1`。它用于设置 `a` 的值，并返回 `r1` 的值：

```
float caculate(float a, float *r1)
{
#pragma HLS INTERFACE ap_vld register port=r1
#pragma HLS INTERFACE s_axilite port=a
#pragma HLS INTERFACE s_axilite port=r1
#pragma HLS INTERFACE s_axilite port=return

    *r1 = 0.5f*a;
    return (a>0);
}
```

综合后，Vivado HLS 会将所有端口组合到默认 AXI4-Lite 接口，并创建 C 驱动文件。但驱动文件使用的类型为 U32，如以下示例所示：

```
// API to set the value of A
void XCaculate_SetA(XCaculate *InstancePtr, u32 Data) {
    Xil_AssertVoid(InstancePtr != NULL);
    Xil_AssertVoid(InstancePtr->IsReady == XIL_COMPONENT_IS_READY);
    XCaculate_WriteReg(InstancePtr->Hls_periph_bus_BaseAddress,
XCACULATE_HLS_PERIPH_BUS_ADDR_A_DATA, Data);
}

// API to get the value of R1
u32 XCaculate_GetR1(XCaculate *InstancePtr) {
    u32 Data;

    Xil_AssertNonvoid(InstancePtr != NULL);
    Xil_AssertNonvoid(InstancePtr->IsReady == XIL_COMPONENT_IS_READY);

    Data = XCaculate_ReadReg(InstancePtr->Hls_periph_bus_BaseAddress,
XCACULATE_HLS_PERIPH_BUS_ADDR_R1_DATA);
    return Data;
}
```

如果这些函数直接用于处理浮点类型，那么写入和读取值与期望的浮点类型不一致。在软件中使用这些函数时，可在代码中使用以下强制类型转换：

```
float a=3.0f,r1;
u32 ua,url;

// cast float "a" to type U32
XCaculate_SetA(&calculate,*((u32*)&a));
url=XCaculate_GetR1(&calculate);

// cast return type U32 to float type for "r1"
r1=*((float*)&url);
```


控制硬件

此示例中的硬件头文件 `xexample_hw.h` 可为组合到 AXI4-Lite 从接口的端口提供完整的内存映射位置列表。

```
// 0x00 : Control signals
//      bit 0 - ap_start (Read/Write/SC)
//      bit 1 - ap_done (Read/COR)
//      bit 2 - ap_idle (Read)
//      bit 3 - ap_ready (Read)
//      bit 7 - auto_restart (Read/Write)
//      others - reserved
// 0x04 : Global Interrupt Enable Register
//      bit 0 - Global Interrupt Enable (Read/Write)
//      others - reserved
// 0x08 : IP Interrupt Enable Register (Read/Write)
//      bit 0 - Channel 0 (ap_done)
//      bit 1 - Channel 1 (ap_ready)
// 0x0c : IP Interrupt Status Register (Read/TOW)
//      bit 0 - Channel 0 (ap_done)
//      others - reserved
// 0x10 : Data signal of a
//      bit 7~0 - a[7:0] (Read/Write)
//      others - reserved
// 0x14 : reserved
// 0x18 : Data signal of b
//      bit 7~0 - b[7:0] (Read/Write)
//      others - reserved
// 0x1c : reserved
// 0x20 : Data signal of c_i
//      bit 7~0 - c_i[7:0] (Read/Write)
//      others - reserved
// 0x24 : reserved
// 0x28 : Data signal of c_o
//      bit 7~0 - c_o[7:0] (Read)
//      others - reserved
// 0x2c : Control signal of c_o
//      bit 0 - c_o_ap_vld (Read/COR)
//      others - reserved
// (SC = Self Clear, COR = Clear on Read, TOW = Toggle on Write, COH =
// Clear on
// Handshake)
```

要对 AXI4-Lite 从接口中的寄存器进行正确编程，必须了解硬件端口工作方式。此块遵循[接口综合](#)中所述的端口协议来运行。

例如，要启动块操作，`ap_start` 寄存器必须设置为 1。随后，器件将继续从 AXI4-Lite 从接口中的寄存器读取已分组到该接口中的任意输入。当此块完成操作后，`ap_done`、`ap_idle` 和 `ap_ready` 寄存器将由硬件输出端口进行设置，并由相应的寄存器读取已分组到 AXI4-Lite 从接口中的任何输出端口的结果。

以上示例中的 `c` 函数实参的实现还突显了了解硬件端口工作方式的重要性。函数实参 `c` 可供读取和写入，因此作为独立的输入端口和输出端口 `c_i` 和 `c_o` 来实现，如[接口综合](#)中所述。

对 AXI4-Lite 从接口进行编程的第 1 个建议流程对应于函数的一次性执行流程：

- 使用中断函数来判定期望的中断工作方式。
- 为块输入端口加载寄存器值。在以上示例中，此操作是使用 API 函数 `XExample_Set_a`、`XExample_Set_b` 和 `XExample_Set_c_i` 来执行的。

- 使用 `XExample_Start` 将 `ap_start` 位设置为 1，以开始执行函数。此寄存器为自我清除型寄存器，如以上头文件中所述。完成一次传输事务后，此块将暂停操作。
- 允许执行此函数。解决生成的所有中断。
- 读取输出寄存器。在以上示例中，这是使用 API 函数 `XExample_Get_c_o_vld`（用于确认数据有效）和 `XExample_Get_c_o` 来执行的。

注释：AXI4-Lite 从接口中的寄存器遵循的 I/O 协议与端口相同。在此情况下，输出有效设置为逻辑 1，以指示数据是否有效。

- 针对下一项传输事务重复此过程。

第 2 个建议流程对应于块的连续执行流程。在此模式下，AXI4-Lite 从接口中包含的输入端口应仅限于执行配置的端口。通常，此块运行速度必须比 CPU 更快。如果块必须等待输入，那么块的大部分时间将用于等待：

- 使用中断函数来判定期望的中断工作方式。
- 为块输入端口加载寄存器值。在以上示例中，此操作是使用 API 函数 `XExample_Set_a`、`XExample_Set_a` 和 `XExample_Set_c_i` 来执行的。
- 使用 API `XExample_EnableAutoRestart` 设置自动启动函数
- 允许执行此函数。个别端口 I/O 协议将同步通过时钟处理的数据。
- 解决生成的所有中断。执行此操作期间可访问输出寄存器，但数据可能多次发生变化。
- 使用 API 函数 `XExample_DisableAutoRestart` 来停止执行。
- 读取输出寄存器。在以上示例中，此操作是使用 API 函数 `XExample_Get_c_o` 和 `XExample_Set_c_o_vld` 来执行的。

控制软件

在 CPU 上运行的软件中可使用 API 函数来控制硬件块。此流程概述为：

- 创建硬件实例
- 查找器件配置
- 初始化器件
- 设置 HLS 块的输入参数
- 启动器件并读取结果

以下显示此流程的摘要版本。在 Zynq-7000 SoC 教程中提供的软件控制的完整示例。

```
#include "xexample.h" // Device driver for HLS HW block
#include "xparameters.h"

// HLS HW instance
XExample HlsExample;
XExample_Config *ExamplePtr

int main() {
    int res_hw;

    // Look Up the device configuration
    ExamplePtr = XExample_LookupConfig(XPAR_XEXAMPLE_0_DEVICE_ID);
    if (!ExamplePtr) {
        print("ERROR: Lookup of accelerator configuration failed.\n\r");
        return XST_FAILURE;
    }
}
```

```

}

// Initialize the Device
status = XExample_CfgInitialize(&HlsExample, ExamplePtr);
if (status != XST_SUCCESS) {
    print("ERROR: Could not initialize accelerator.\n\r");
    exit(-1);
}

//Set the input parameters of the HLS block
XExample_Set_a(&HlsExample, 42);
XExample_Set_b(&HlsExample, 12);
XExample_Set_c_i(&HlsExample, 1);

// Start the device and read the results
XExample_Start(&HlsExample);
do {
    res_hw = XExample_Get_c_o(&HlsExample);
} while (XExample_Get_c_o(&HlsExample) == 0); // wait for valid data output
print("Detected HLS peripheral complete. Result received.\n\r");
}

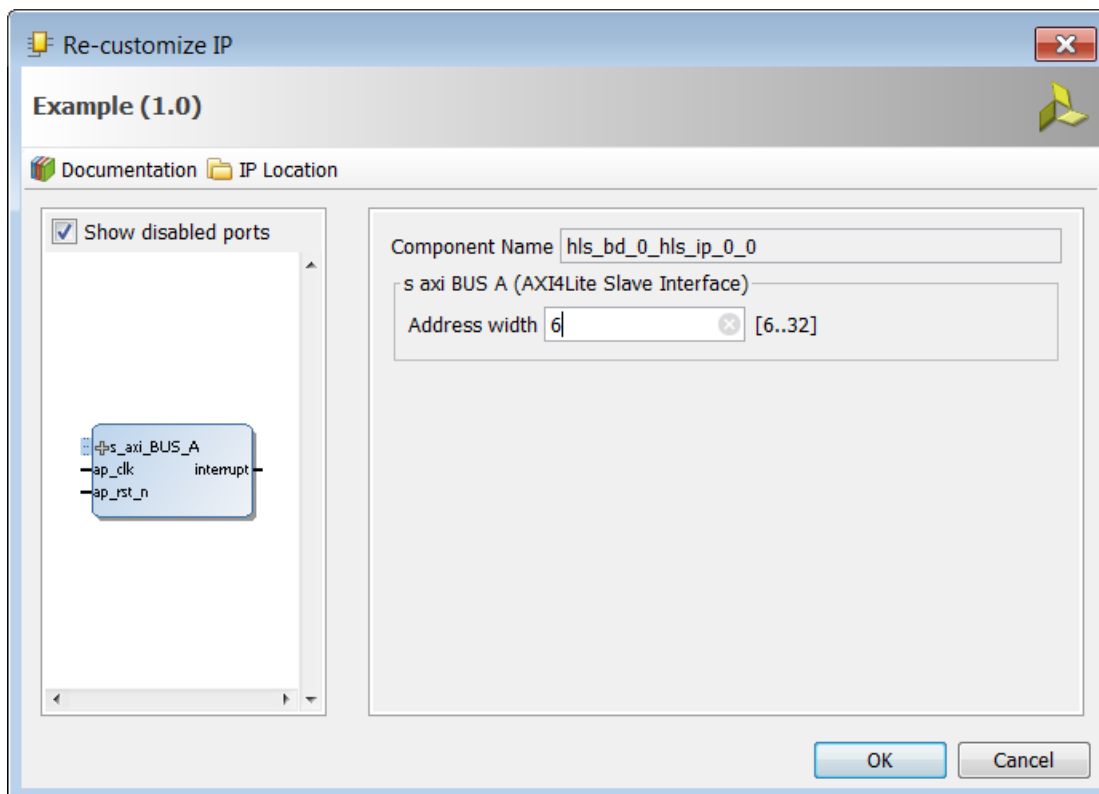
```

在 IP integrator 中自定义 AXI4-Lite 从接口

将使用 AXI4-Lite 从接口的 HLS RTL 设计整合到 Vivado IP integrator 中的设计中时，可对块进行自定义。在 IP integrator 的原理图中，选中 HLS 块、右键单击并选择“Customize Block”。

默认情况下，地址宽度配置为所需的最小大小。对此设置进行修改即可连接到地址大小小于 32 位的块。

图 47：在 IP integrator 中自定义 AXI4-Lite 从接口



AXI4 主接口

您可在数组或指针/参考实参上使用 AXI4 主接口，Vivado HLS 通过以下任一方式来实现该接口：

- 单独数据传输
- 突发模式数据传输

对于单独数据传输，Vivado HLS 会读取或写入每个地址的单个数据元素。以下示例显示了单一读取操作和单一写入操作。在此示例中，Vivado HLS 可在 AXI 接口上生成 2 个地址，1 个用于读取单个数据值，另 1 个用于写入单个数据值。该接口会为每个地址传输 1 个数据值。

```
void bus (int *d) {
    static int acc = 0;

    acc += *d;
    *d = acc;
}
```

对于突发模式传输，Vivado HLS 会使用单一基址读取或写入数据，后接多次顺序数据采样，因此，该模式支持更高的数据吞吐量。使用 `Cmemcpy` 函数或流水线化的 `for` 循环时，即可执行突发模式操作。

注释： 仅当 `Cmemcpy` 函数用于在使用 AXI4 主接口指定的顶层函数实参上执行双向数据出入传输时，才支持对其进行综合。

以下示例显示了使用 `memcpy` 函数的突发模式副本。顶层函数实参 `a` 指定为 AXI4 主接口。

```
void example(volatile int *a){

#pragma HLS INTERFACE m_axi depth=50 port=a
#pragma HLS INTERFACE s_axilite port=return

//Port a is assigned to an AXI4 master interface

    int i;
    int buff[50];

    //memcpy creates a burst access to memory
    memcpy(buff,(const int*)a,50*sizeof(int));

    for(i=0; i < 50; i++){
        buff[i] = buff[i] + 100;
    }

    memcpy((int *)a,buff,50*sizeof(int));
}
```

对此示例进行综合时，会生成如下图所示接口。

注释： 在此图中，AXI4 接口处于折叠状态。

图 48: AXI4 接口



以下示例显示的代码与先前示例相同，但使用 `for` 循环来复制出数据：

```
void example(volatile int *a){

#pragma HLS INTERFACE m_axi depth=50 port=a
#pragma HLS INTERFACE s_axilite port=return

//Port a is assigned to an AXI4 master interface

int i;
int buff[50];

//memcpy creates a burst access to memory
memcpy(buff,(const int*)a,50*sizeof(int));

for(i=0; i < 50; i++){
buff[i] = buff[i] + 100;
}

for(i=0; i < 50; i++){
#pragma HLS PIPELINE
a[i] = buff[i];
}
}
```

使用 `for` 循环实现突发读取或写入时，请遵循如下要求：

- 将循环流水线化
- 按升序访问地址
- 请勿将访问置于条件语句内
- 对于嵌套循环，请勿将循环扁平化，因为这样有碍突发操作

注释：在任一 `for` 循环内只允许 1 次读取和 1 次写入，除非端口捆绑在不同 AXI 端口内。以下示例显示了如何使用不同 AXI 接口以突发模式执行 2 次读取。

在以下示例中，Vivado HLS 将端口读取作为突发传输来实现。指定的端口 `a` 未使用 `bundle` 选项，并在默认 AXI 接口中实现。端口 `b` 是使用已命名的捆绑指定的，在名为 `d2_port` 的独立 AXI 接口中实现。

```
void example(volatile int *a, int *b){

#pragma HLS INTERFACE s_axilite port=return
#pragma HLS INTERFACE m_axi depth=50 port=a
#pragma HLS INTERFACE m_axi depth=50 port=b bundle=d2_port

int i;
int buff[50];
```

```
//copy data in
for(i=0; i < 50; i++){
#pragma HLS PIPELINE
buff[i] = a[i] + b[i];
}
...
}
```

注释：对于 AXIM 接口，仅支持使用 DATA_PACK 最优化打包的结构体。

控制 AXI4 突发行为

当设计在等待访问总线时，从不会在最优化 AXI4 接口上停滞，授予总线访问后，总线等待设计执行读写时从不停滞。为创建最优化 AXI4 接口，在 INTERFACE 指令中提供了以下选项以指定突发的行为并最优化 AXI4 接口的效率。

其中部分选项使用内部存储空间来缓存数据，并且可能影响面积和资源。

- `latency`：指定期望的 AXI4 接口时延，允许设计发起总线请求的时间比执行期望的读取或写入操作早数个周期（时延）。如果该值太低，设计将过早达成就绪状态，可能停滞并等待总线；如果该值太高，则可能授予总线访问权时，总线仍处于停滞状态并等待设计发起访问。
- `max_read_burst_length`：指定突发传输期间读取的数据值的最大数量。
- `num_read_outstanding`：指定在设计停滞前可对 AXI4 总线发出的读取请求的数量（无响应）。此操作暗示设计中的内部存储空间，即 FIFO 大小为：`num_read_outstanding*max_read_burst_length*word_size`。
- `max_write_burst_length`：指定突发传输期间写入的数据值的最大数量。
- `num_write_outstanding`：指定在设计停滞前可对 AXI4 总线发出的写入请求的数量（无响应）。此操作暗示设计中的内部存储空间，即 FIFO 大小为：`num_read_outstanding*max_read_burst_length*word_size`。

以下示例可用于帮助解释这些选项：

```
#pragma HLS interface m_axi port=input offset=slave bundle=gmem0
depth=1024*1024*16/(512/8)
latency=100
num_read_outstanding=32
num_write_outstanding=32
max_read_burst_length=16
max_write_burst_length=16
```

接口时延指定为 100。Vivado HLS 希望将突发访问请求发生时间调度为比设计准备好访问 AXI4 总线的时间早 100 个时钟周期。为了进一步提高总线效率，`num_write_outstanding` 和 `num_read_outstanding` 选项可用于确保设计所含缓存足以存储最多 32 次读写访问。这样即可允许设计持续处理，直至处理总线请求为止。最后，`max_read_burst_length` 和 `max_write_burst_length` 选项可确保最大突发量为 16，并且 AXI4 接口保存总线的时间不超过该设置。

这些选项允许对 AXI4 接口行为进行最优化以满足运行该接口的系统的要求。操作效率取决于设置的这些值的准确性。

创建具有 64 位地址功能的 AXI4 接口

默认情况下，Vivado HLS 实现含 32 位地址总线的 AXI4 端口。您可选择使用 `m_axi_addr64` 接口选项实现含 64 位地址总线的 AXI4 接口，如下所示：

1. 选择“Solution” → “Solution Settings”。
2. 在“解决方案设置 (Solution Settings)”对话框中，单击“General”类别，然后单击“Add”。
3. 在“添加命令 (Add Command)”对话框中，选择“config_interface”，并启用“m_axi_addr64”。



重要提示！ 选择 “m_axi_addr64” 选项时，Vivado HLS 会在含 64 位地址总线的设计中实现所有 AXI4 接口。

控制 AXI4 接口中的地址偏移

默认情况下，AXI4 主接口从地址 0x00000000 启动所有读写操作。例如，给定以下代码时，设计会读取从地址 0x00000000 到 0x000000c7（50 个 32 位字，提供 200 字节）的数据，它表示 50 个地址值。随后，设计会将数据重新写入相同地址。

```
void example(volatile int *a){
    #pragma HLS INTERFACE m_axi depth=50 port=a
    #pragma HLS INTERFACE s_axilite port=return bundle=AXILiteS

    int i;
    int buff[50];

    memcpy(buff, (const int*)a, 50*sizeof(int));

    for(i=0; i < 50; i++){
        buff[i] = buff[i] + 100;
    }
    memcpy((int *)a, buff, 50*sizeof(int));
}
```

要应用地址偏移，请将 `-offset` 选项与 `INTERFACE` 指令搭配使用，并指定以下选项之一：

- `off`：不应用偏移地址。这是默认方式。
- `direct`：向设计添加 32 位端口以应用地址偏移。
- `slave`：在 AXI4-Lite 接口内添加 32 位寄存器以应用地址偏移。

在最终 RTL 中，Vivado HLS 会将地址偏移直接应用于 AXI4 主接口生成的任意读取地址或写入地址。这样即可允许设计访问系统中的任意地址位置。

如果在 AXI 接口中使用 `slave` 选项，则必须在设计接口上使用 AXI4-Lite 端口。赛灵思建议使用以下编译指示来实现 AXI4-Lite 接口：

```
#pragma HLS INTERFACE s_axilite port=return
```

此外，如果使用 `slave` 选项并使用多个 AXI4-Lite 接口，则必须确保 AXI 主端口偏移寄存器已与正确的 AXI4-Lite 接口绑定。在以下示例中，端口 `a` 作为含偏移和 AXI4-Lite 接口（名为 `AXI_Lite_1` 和 `AXI_Lite_2`）的 AXI 主接口来实现：

```
#pragma HLS INTERFACE m_axi port=a depth=50 offset=slave
#pragma HLS INTERFACE s_axilite port=return bundle=AXI_Lite_1
#pragma HLS INTERFACE s_axilite port=b bundle=AXI_Lite_2
```

以下 `INTERFACE` 指令是确保端口 `a` 的偏移寄存器与名为 `AXI_Lite_1` 的 AXI4-Lite 接口绑定所必需的。

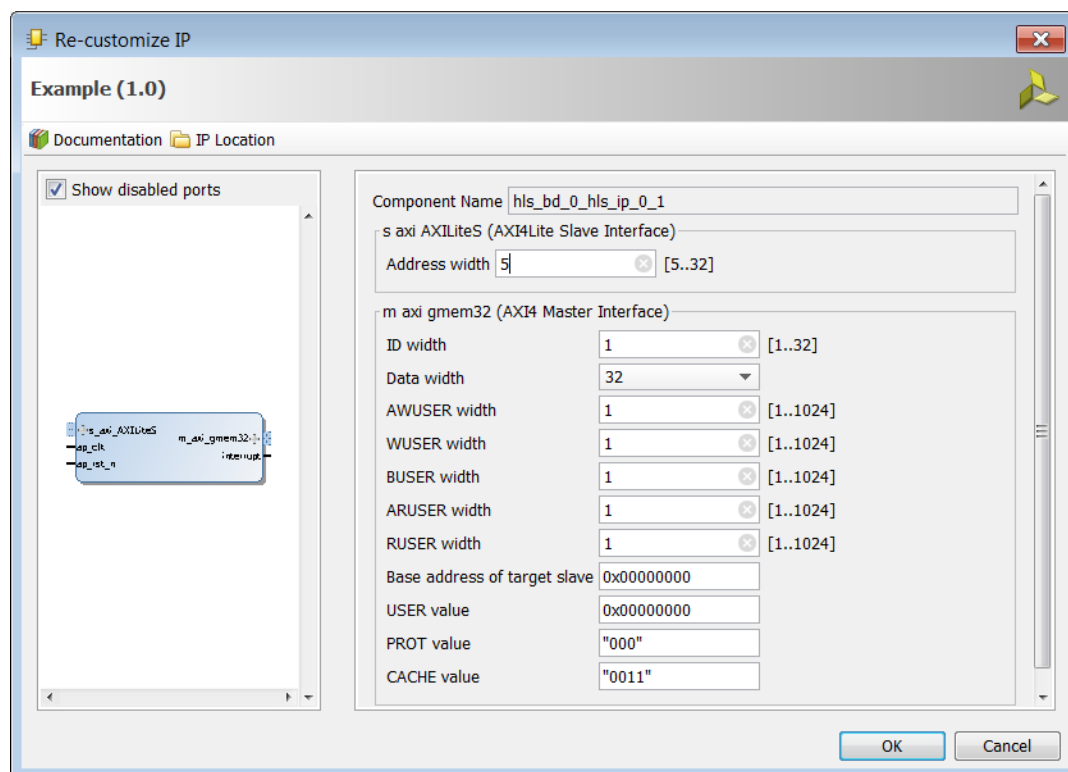
```
#pragma HLS INTERFACE s_axilite port=a bundle=AXI_Lite_1
```

在 IP integrator 中自定义 AXI4 主接口

将使用 AXI4 主接口的 HLS RTL 设计合并到 Vivado IP integrator 中的设计中时，可以自定义块。在 IP integrator 原理图中，选择 HLS 块、右键单击，然后选择“Customize Block”以自定义所提供的任何设置。请访问此[链接](#)以参阅《Vivado Design Suite：AXI 参考指南》(UG1037) 中的相关内容，获取有关 AXI4 参数的完整描述。

下图显示了如下所示设计的“重新自定义 IP (Re-Customize IP)”对话框。此设计包含 1 个 AXI4-Lite 端口。

图 49：在 IP integrator 中自定义 AXI4 主接口



利用 SSI 技术器件管理接口

某些赛灵思器件使用堆叠硅片互联 (SSI) 技术。在此类器件中，全部可用资源被划分为多个超级逻辑区域 (SLR)。SLR 之间的连接使用超长线路 (SLL) 布线。SSL 布线产生的延迟成本通常大于标准 FPGA 布线。为确保设计以最大性能运行，请遵循如下准则：

- 将跨 SLR 的所有信号都寄存在 SLR 输出和 SLR 输入中。
- 通过 I/O 缓存进出 SLR 的信号无需寄存。
- 请确保 Vivado HLS 所创建的逻辑适用于单一 SLR 内。

注释：选择 SSI 技术器件作为目标技术时，利用率报告将包含有关 SLR 利用率和器件总利用率的详细信息。

如果逻辑包含在单一 SLR 器件内，Vivado HLS 会向 `config_interface` 命令提供 `register_io` 选项。该选项支持自动寄存所有块输入和/或输出。仅限标量才需要使用该选项。所有数组端口都自动寄存。

`register_io` 选项的设置包括：

- `off`：不寄存任何输入或输出。

- `scalar_in`: 寄存所有输入。
- `scalar_out`: 寄存所有输出。
- `scalar_all`: 寄存所有输入和输出。

注释：将 `register_io` 选项用于 RTL 的块级布局规划可确保以 SSI 技术器件为目标的逻辑能够以最大时钟速率运行。

对设计进行最优化

本章节概括的各项最优化和技巧可供您用于指导 Vivado HLS 生成可满足所期望的性能和面积目标的宏架构。

下表列出了 Vivado HLS 提供的最优化指令。

表 11: Vivado HLS 最优化指令

指令	描述
ALLOCATION	指定所使用的操作、核或函数的数量限制。这样会强制共享硬件资源并可能增大时延
ARRAY_MAP	把多个较小的数组结合成单个大型数组以帮助减少块 RAM 资源数量。
ARRAY_PARTITION	把大型数组分区为多个较小数组或分区为单独的寄存器，以提升对数据的访问，消除块 RAM 瓶颈。
ARRAY_RESHAPE	把数组从多元型数组重塑为字宽更宽的数组。用于在不增加使用的块 RAM 数量的前提下提升块 RAM 访问。
CLOCK	对于 SystemC 语言设计，可使用 <code>create_clock</code> 命令指定多个已命名的时钟，并且这些时钟可使用该指令应用到各 <code>SC_MODULE</code> 。
DATA_PACK	把结构体的数据字段打包到字宽更宽的单一标量中。
DATAFLOW	实现任务层次的流水线化，让函数和循环并行执行。用于最优化吞吐量和/或时延。
DEPENDENCE	用于提供可克服循环进位依赖关系并允许循环流水线化（或以更低的时间间隔进行流水线化）的其它信息。
EXPRESSION_BALANCE	允许关闭自动表达式平衡。
FUNCTION_INSTANTIATE	允许对相同函数的不同实例进行局部最优化。
INLINE	内联函数，移除该层次的函数层级。用于跨越函数边界实现逻辑最优化，通过减少函数调用开销来改善时延/时间间隔。
INTERFACE	指定如何根据函数描述创建 RTL 端口。
LATENCY	允许指定最小和最大时延约束。
LOOP_FLATTEN	允许把嵌套循环折叠为已改善时延的单一循环。
LOOP_MERGE	合并连续循环，以降低总体时延、增加共享和提升逻辑最优化。
LOOP_TRIPCOUNT	用于含变量边界的循环。提供估算的循环迭代计数。这对综合没有影响，只对报告功能有影响。
OCCURRENCE	在对函数或循环进行流水线化时使用，用于指定某个位置的代码执行速度低于外围函数或循环中的代码执行速度。
PIPELINE	允许在单一循环或函数内重叠执行运算，从而缩短启动时间间隔。
PROTOCOL	此命令可指定代码区域为协议区域。协议区域可用于手动指定接口协议。
RESET	本指令用于在特定状态变量（全局或静态）上添加或移除复位。
RESOURCE	指定将特定的库资源（核）用于实现 RTL 中的变量（数组、算术运算或函数实参）。
STREAM	指定在数据流最优化期间将特定数组作为 FIFO 或 RAM 内存通道来实现。使用 <code>hls::stream</code> 时，STREAM 最优化指令用于覆盖 <code>hls::stream</code> 的配置。

表 11: Vivado HLS 最优化指令 (续)

指令	描述
TOP	综合的顶层函数在工程设置中指定。该指令可用于将任意函数指定为综合的顶层函数。这样即可允许将相同工程内的不同解决方案指定为综合的顶层函数，而无需创建新工程。
UNROLL	展开 for 循环以创建循环主体的多个实例，随后即可独立调度其指令。

除了最优化指令外，Vivado HLS 提供了多种配置设置。配置设置用于更改综合的默认行为。配置设置如下表所示。

表 12: Vivado HLS 配置

GUI 指令	描述
“Config Array Partition”	该配置用于判断包括全局数组在内的数组分区方式，以及分区是否会影响数组端口。
“Config Bind”	判断综合绑定阶段的工作量，可用于在全局层面最大限度减少使用的运算数量。
“Config Compile”	控制综合专用最优化功能，例如自动循环流水线化和浮点数学最优化。
“Config Dataflow”	该配置用于指定数据流最优化中的默认内存通道和 FIFO 深度。
“Config Interface”	此配置用于控制与顶层函数实参无关的 I/O 端口，允许从最终 RTL 中去除未使用的端口。
“Config RTL”	提供对输出 RTL 的控制权，包括文件和模块命名、复位样式和 FSM 编码。
“Config Schedule”	判断综合调度阶段的工作量以及输出消息的详细程度。

在[应用最优化指令](#)中提供了有关如何应用最优化和配置的详细信息。配置访问方式为：使用菜单中的“Solution” → “Solution Settings” → “General”，并使用“Add”按钮选择配置。

根据对设计应用最优化的方式来演示最优化。

时钟、复位和 RTL 输出将结合在一起进行讨论。时钟频率和目标器件是驱动最优化的主要约束。Vivado HLS 努力在每个时钟周期内尽可能放入更多来自目标器件的运算。最终 RTL 中使用的复位样式与设置（如 FSM 编码样式）一起使用 config_rtl 配置来加以控制。

“Optimizing for Throughput”的主要最优化与其典型使用方式一起演示：将任务流水线化，以提升性能、改进任务之间的数据流并最优化结构以改进可能限制性能的地址问题。

“Optimizing for Latency”使用时延约束和移除循环过渡的技巧来减少完成操作所需的时钟周期数。

改善面积的主要技巧集中于运算的实现方式，包括控制运算数量以及在硬件中实现这些运算的方式。

除了编译指示和指令外，Vivado HLS 提供了相应的方法以将经过最优化的现有 RTL 集成到 HLS 设计流程中。如需了解更多信息，请参阅[RTL 黑盒](#)。

时钟、复位和 RTL 输出

指定时钟频率

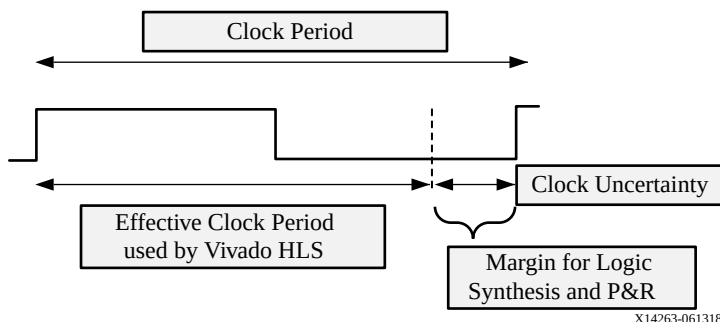
对于 C 和 C++ 设计，仅支持单一时钟。对设计中的所有函数将应用同一个时钟。

对于 SystemC 语言设计，每个 SC_MODULE 都必须使用不同时钟来指定。要在 SystemC 语言设计中指定多个时钟，请使用 create_clock 命令的 -name 选项来创建多个指定时钟，并使用 CLOCK 指令或编译指示来指定哪个函数包含将以指定时钟进行综合的 SC_MODULE。每个 SC_MODULE 都只能使用单一时钟来进行综合：多个时钟可分布于多个函数间（例如，多个时钟从顶层端口连接到各块时），但每个 SC_MODULE 只能受单一时钟影响。

时钟周期（以 ns 为单位）在“Solutions”→“Solutions Setting”中设置。Vivado HLS 使用时钟不确定性概念来提供用户定义的时序裕度。通过使用时钟频率和器件目标信息，Vivado HLS 可估算设计中的操作时序，但无法确定最终组件布局和信号线布线：这些操作由输出 RTL 的逻辑综合来执行。因此，Vivado HLS 无法掌握精确延迟。

为计算用于综合的时钟周期，Vivado HLS 会从时钟周期减去时钟不确定性，如下图所示。

图 50：时钟周期和裕度



这样可为用户提供指定裕度以确保下游进程（例如，逻辑综合与布局布线）具有足够的时序裕度来完成其操作。如果 FPGA 器件利用率过高，那么单元的布局以及用于连接单元的信号线的布线可能无法达成理想状态，并且可能导致设计的时序延迟大于期望值。对于此类情况，增加时序裕度可避免 Vivado HLS 所创建的设计在每个时钟周期内封装过多逻辑，并在布局布线选项不太理想的情况下使 RTL 综合仍能满足时序。

默认时钟不确定性为周期时间的 12.5%。可在时钟周期旁显式指定该值。

Vivado HLS 以满足所有约束为目标：时序、吞吐量和时延。但如果无法满足约束，Vivado HLS 始终会输出 RTL 设计。

如果无法满足时钟周期推断的时序约束，Vivado HLS 会发出 SCHED-644 消息（如下所示）并以可实现的最佳性能来创建设计。

```
@W [SCHED-644] Max operation delay (<operation_name> 2.39ns) exceeds the
effective
cycle time
```

即使 Vivado HLS 无法满足特定路径的时序要求，它仍可在所有其它路径上实现时序。此行为允许您评估提高最优化级别或者由下游逻辑综合对失败路径进行特殊处理是否能够接近并最终满足时序。



重要提示！ 综合后复查约束报告以判定是否满足所有约束至关重要：实际上虽然 Vivado HLS 可生成输出设计，但不能保证设计满足所有性能约束。请复查设计报告的“性能估算 (Performance Estimates)”部分。

config_schedule 命令的 relax_ii_for_timing 选项可用于更改默认时序行为。指定该选项时，Vivado HLS 会在检测到路径无法满足时钟周期时自动放宽任意流水线指令的 II。该选项仅适用于已指定 PIPELINE 指令但未指定 II 值（隐含 II=1）的情况。如果在 PIPELINE 指令中已显式指定 II 值，那么 relax_ii_for_timing 选项无效。

综合完成后，针对层级内每个函数都会生成一份设计报告，并可在解决方案报告文件夹中查看。整个设计的最差情况时序将在各函数报告中报告为最差情况。无需复查层级中的每份报告。

如果时序违例过于严格而无法进一步最优化，且下游进程无法对其加以更正，请先复查指定精确的时延和指定精确的实现核的技巧，然后再考虑是否采用更快的目标技术。

指定复位

通常 RTL 配置中最重要的操作即选择复位行为。对于复位行为，重要的是理解初始化与复位之间的差异。

初始化行为

在 C 中，以静态限定符定义的变量和全局作用域中定义的变量默认都初始化为 0。（可选）对于这些变量，可赋予特定初始值。对于这 2 种类型的变量，C 语言代码中的初始值在编译时（时序为 0 时）进行赋值，并且不再进行赋值。上述 2 种情况下，在 RTL 中实现的初始值相同。

- 在 RTL 仿真期间，为这些变量设置的初始值与 C 语言代码中相同。
- 在用于对 FPGA 进行编程的比特流中同样会对这些变量进行初始化。当器件上电时，变量将以其初始状态启动。

变量启动时的初始状态与 C 语言代码中相同，但无法强制返回此初始状态。要返回初始状态，必须通过复位来实现这些变量。

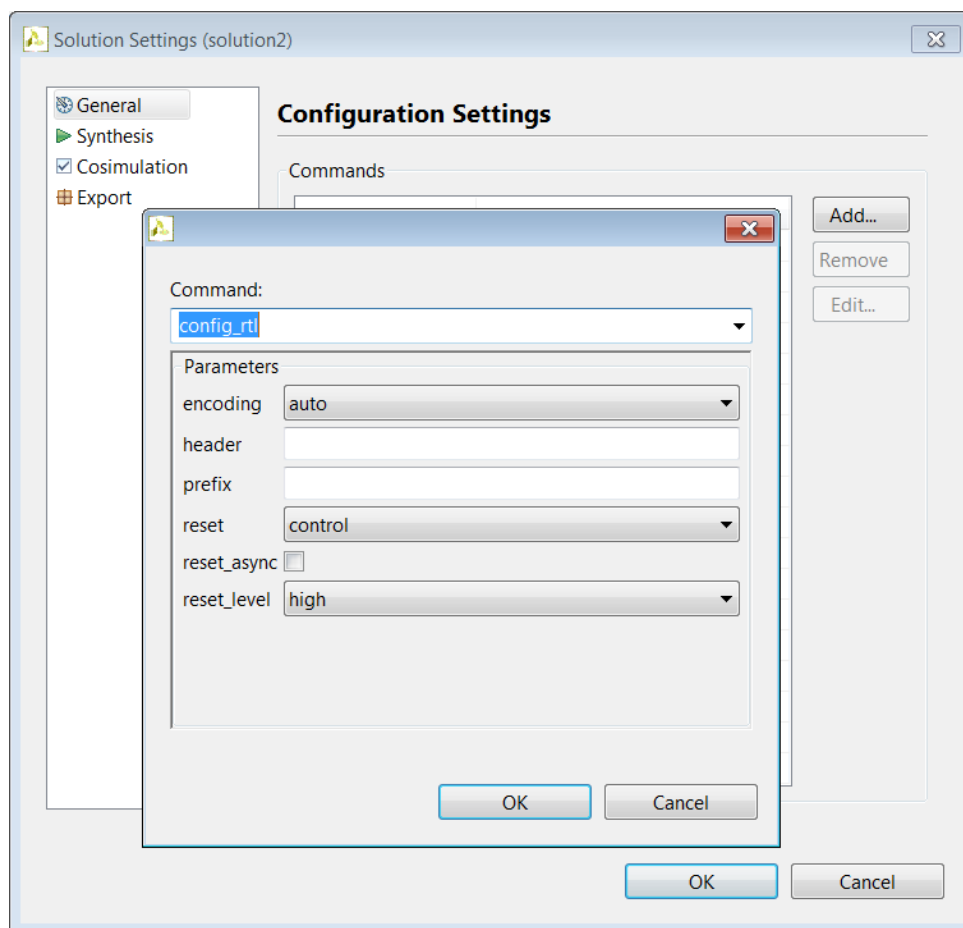


重要提示！ 顶层函数实参可在 AXI4-Lite 接口中实现。由于在 C/C++ 中无法为函数实参提供初始值，因此在 RTL 中无法对这些变量进行初始化，否则由此创建的 RTL 设计将与 C/C++ 代码产生不同的功能行为，从而导致在 C/RTL 协同仿真期间验证失败。

控制复位行为

复位端口在 FPGA 中用于在应用复位信号时，立即将连接到复位端口的寄存器和块 RAM 还原为初始值。`config_rtl` 配置可用于控制 RTL 复位端口是否存在及其行为，如下图所示。要访问此配置，请选择“Solution”→“Solution Settings”→“General”→“Add”→“config_rtl”。

图 51：RTL 配置



复位设置包含设置复位极性以及使用同步复位还是异步复位的功能，但更重要的是，它可通过“reset”选项来控制应用复位信号时要复位的寄存器。



重要提示！ 当在设计上使用 AXI4 接口时，复位极性会自动更改为低电平有效，而与 `config_rtl` 配置中的设置无关。这符合 AXI4 标准的要求。

“reset”选项包含 4 项设置：

- “none”：不向设计添加任何复位。
- “control”：这是默认设置，用于确保将所有控制寄存器复位。控制寄存器即状态机中使用的寄存器，用于生成 I/O 协议信号。此设置可确保设计可立即启动其操作状态。
- “state”：该选项可为控制寄存器添加复位（与“control”设置相同），并且还可为衍生自 C 语言代码中的静态变量和全局变量的任意寄存器或内存添加复位。此设置可确保应用复位后，C 语言代码中初始化的静态变量和全局变量均复位为其初始值。
- “all”：为设计中的所有寄存器和内存添加复位。

通过 RESET 指令可提供更精细的复位控制。对于静态变量或全局变量，RESET 指令可用于为其显式添加复位，或者可将该变量从使用 RESET 指令的 `off` 选项复位的变量中移除。当设计中存在静态数组或全局数组时，该选项非常实用。



重要提示！ 使用 `state` 或 `all` 复位选项时，考量其对于数组的影响至关重要。

数组初始化和复位

数组通常定义为静态变量，这暗示所有元素都将初始化为 0，且数组通常作为块 RAM 来实现。使用复位选项 `state` 或 `all` 时，会强制作为块 RAM 实现的所有数组都在复位后返回初始化状态。这可能导致 RTL 设计中出现 2 个非常不利的条件：

- 不同于上电初始化，显式复位要求 RTL 设计对块 RAM 中的每个地址进行迭代以设置值：如果 N 较大，这可能导致时钟周期数量显著增加并增加实现所需的面积资源。
- 在设计中每个数组中都添加复位。

为防止在每个此类块 RAM 上都放置复位逻辑并避免因复位 RAM 中的所有元素而产生的周期开销，请执行以下操作：

- 使用默认 `control` 复位模式，并使用 RESET 指令来指定要复位的每个静态变量或全局变量。
- 或者，使用 `state` 复位模式，并使用 RESET 指令的 `off` 选项从特定静态变量或全局变量中移除复位。

RTL 输出

可使用 `config_rtl` 配置来控制 Vivado HLS 的各种 RTL 输出特性，如上图所示。

- 指定 RTL 状态机中使用的 FSM 编码类型。
- 使用 `-header` 选项可向所有 RTL 文件添加任意注释字符串（例如，版权声明）。
- 使用 `prefix` 选项可指定要添加到所有 RTL 输出文件名的唯一名称。
- 强制 RTL 端口使用小写名称。

默认 FSM 编码样式为 `onehot`。其它可用选项包括 `auto`、`binary` 和 `gray`。如果选择 `auto`，Vivado HLS 可使用 `onehot` 默认设置来实现编码样式，但 Vivado Design Suite 可在逻辑综合期间提取并重新实现 FSM 样式。如果选择任何其它编码样式（`binary`、`onehot` 或 `gray`），则赛灵思逻辑综合工具无法对编码样式进行重新优化。

RTL 输出文件的名称衍生自综合的顶层函数的名称。如果从相同顶层函数创建不同 RTL 块，则 RTL 文件将采用相同名称，且在相同 RTL 工程内无法组合。`prefix` 选项允许从相同顶层函数生成多个 RTL 文件，且这些文件默认与顶层函数同名，以便于在相同目录内进行组合。`lower_case_name` 选项可用于确保在输出 RTL 中仅使用小写名称。该选项可确保 Vivado HLS 创建的 I/O 协议端口（例如，AXI 接口的 I/O 协议端口）在最终 RTL 中指定为 `s_axis_<port>_tdata` 而不是 `s_axis_<port>_TDATA` 的默认端口名称。

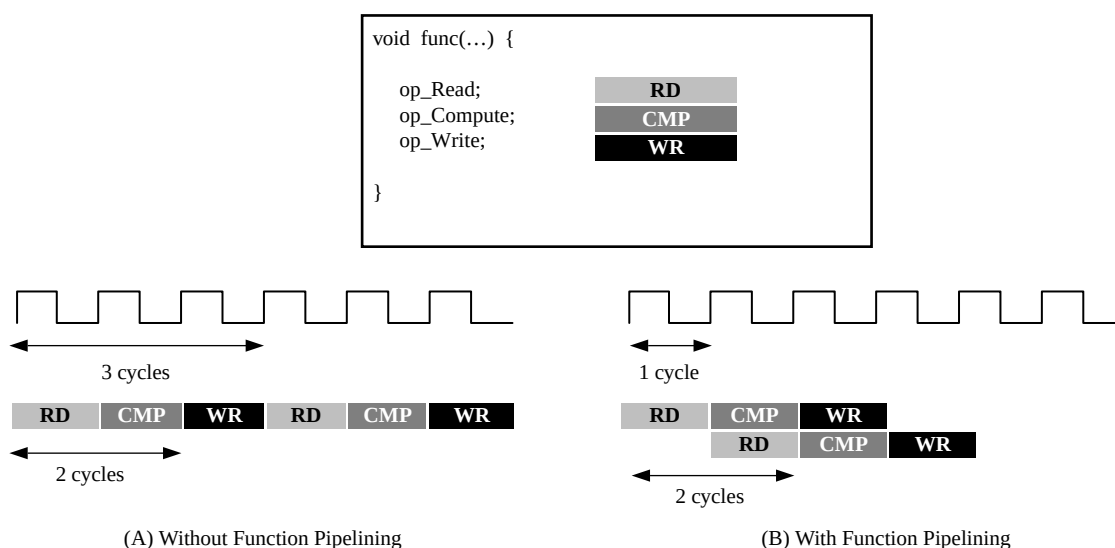
吞吐量最优化

使用以下最优化来提高吞吐量或降低启动时间间隔。

函数与循环流水线化

流水线化允许并行执行操作：每个执行步骤无需等待完成所有操作后再开始下一项操作。流水线化适用于函数和循环。下图显示了通过函数流水线化实现的吞吐量提升。

图 52：函数流水线化行为



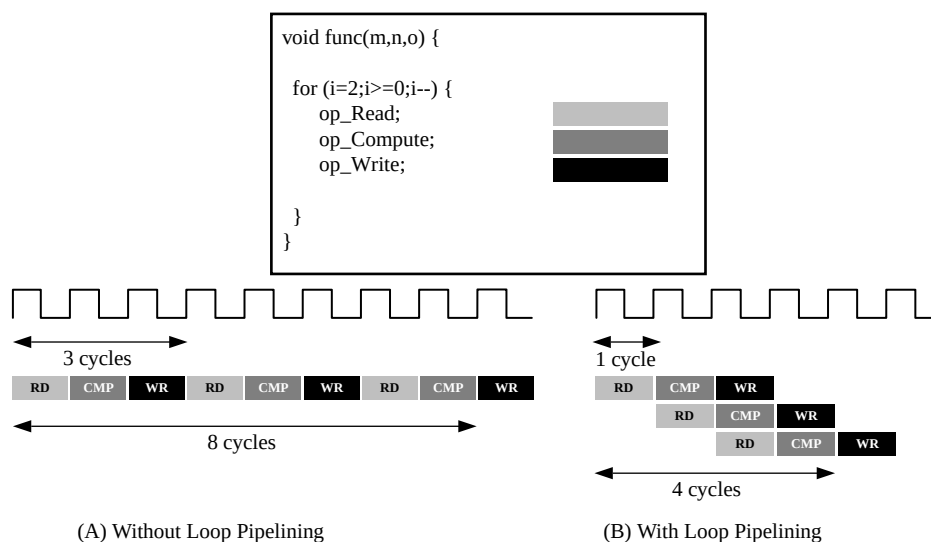
X14269

如果不使用流水线化，上述示例中的函数将每隔 3 个时钟周期读取一次输入，并在 2 个时钟周期后输出值。该函数启动时间间隔 (II) 为 3，时延为 3。使用流水线化后，对于此示例，每个周期都会读取 1 次新输入 (II=1)，且不更改输出时延。

循环流水线化支持以重叠方式来实现循环中的操作。在下图中，(A) 显示了默认顺序操作，每次输入读操作间存在 3 个时钟周期 (II=3)，需要经过 8 个时钟周期才会执行最后一次输出写操作。

在 (B) 所示的循环的流水线版本中，每个周期都会读取一次新输入样本 (II=1)，仅需 4 个时钟周期后即可写入最终输出，在使用相同硬件资源的前提下显著改善 II 和时延。

图 53：循环流水线化



X14277

函数或循环使用 PIPELINE 指令来实现流水线化。该指令是在构成函数或循环主体的区域中指定的。启动时间间隔如果不指定，则默认为 1，但可显式指定其值。

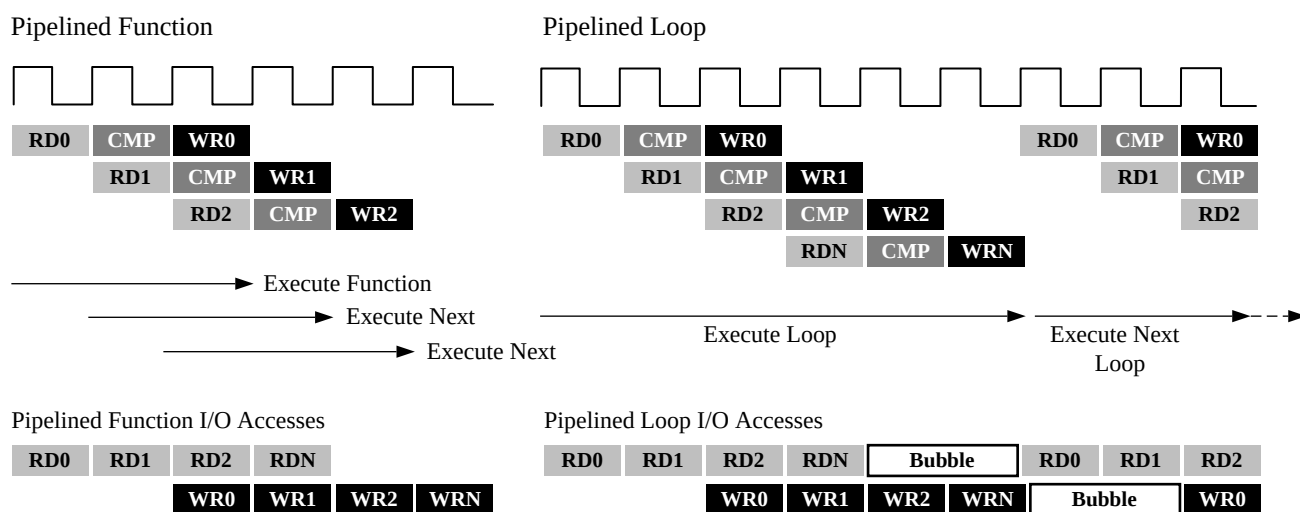
流水线化仅应用于指定区域，而不应用于下层层级。但下层层级内所有循环都会自动展开。指定函数的下层层级内的所有子函数都必须单独流水线化。如果将子函数流水线化，其上层的已流水线化的函数即可充分发挥流水线性能。反之，已流水线化的顶层函数下层的任意子函数如果不采用流水线化，则可能限制流水线性能。

已流水线化的函数与已流水线化的循环之间行为存在差异。

- 对于函数，流水线将永久运行，永不终止。
- 对于循环，流水线将持续执行直至循环的所有迭代完成为止。

在下图中总结了行为差异。

图 54：函数与循环流水线行为



X14302

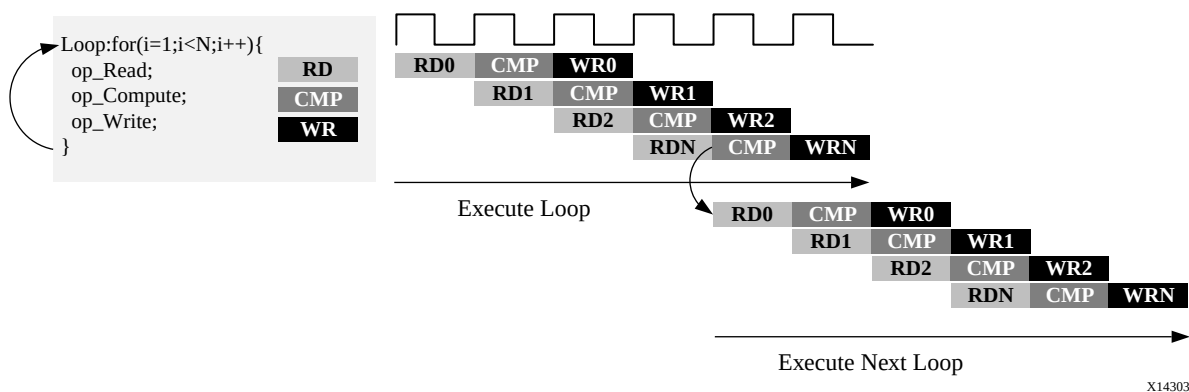
行为差异会影响流水线输入和输出的处理方式。如上图所示，已流水线化的函数将持续读取新输入和写入新输出。相比之下，由于循环必须首先完成循环中的所有操作，然后才能启动下一次循环，已流水线化的循环会导致数据流传输中出现“气泡”；即，当循环完成执行最终迭代后不读取任何新输入的时间点，以及当循环开始新循环迭代时不写入新输出的时间点。

回绕已流水线化的循环以保障性能

为避免前图中所示的问题，PIPELINE 编译指示具有可选 `rewind` 命令。此命令支持将后续调用的迭代重叠到 `rewind` 循环，前提是此循环为数据流进程顶层函数的最外层的构造（并且多次调用数据流区域）。

下图显示了对循环进行流水线化时使用 `rewind` 选项的操作。循环迭代计数结束时，循环将开始重新执行。虽然一般情况下它立即重新执行，但也可能出现延迟，在 GUI 中可显示并描述此延迟。

图 55：含回绕 (Rewind) 选项的循环流水线



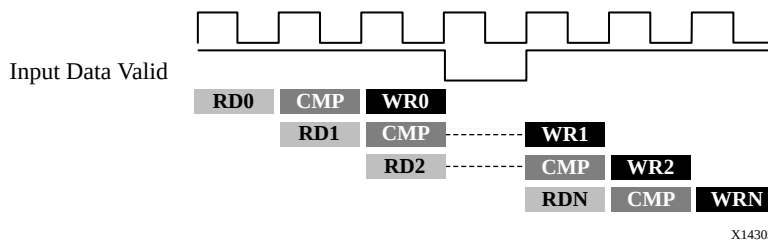
X14303

注释：如果在 DATAFLOW 区域周围使用循环，Vivado HLS 会自动将其实现以允许后续迭代进行重叠。如需了解更多信息，请参阅[利用任务级别并行化：数据流最优化](#)。

清空流水线

只要在流水线输入端数据可用，流水线就会持续执行。如果没有可供处理的数据，流水线就会停滞。如下图所示，其中输入数据 `valid` 信号降低，表示没有更多数据。只要没有可供处理的新数据，流水线就将停止操作。

图 56：含停滞的循环流水线化



X14305

在某些情况下，最好采用可“清空”的流水线。提供 `flush` 选项即可执行此操作。当“清空”流水线时，如果没有新输入可用（由流水线起始位置的数据 `valid` 信号来判定），流水线就会停止读取，但仍会继续处理并关闭后续每个流水线阶段，直至最终输入已处理至流水线输出为止。

自动循环流水线化

`config_compile` 配置支持根据迭代计数对循环进行自动流水线化。此配置可通过菜单“Solution” → “Solution Setting” → “General” → “Add” → “config_compile”访问。

`pipeline_loops` 选项可用于设置迭代限制。低于此限制的单一迭代计数内的所有循环都将自动流水线化。默认值为 0：不执行自动循环流水线化。

给定以下示例代码：

```
for (y = 0; y < 480; y++) {
    for (x = 0; x < 640; x++) {
        for (i = 0; i < 5; i++) {
            // do something 5 times
            ...
        }
    }
}
```

如果 `pipeline_loops` 选项设置为 6，那么以上代码片段中最内层的 `for` 循环将自动流水线化。这等同于以下代码片段：

```
for (y = 0; y < 480; y++) {
    for (x = 0; x < 640; x++) {
        for (i = 0; i < 5; i++) {
            #pragma HLS PIPELINE II=1
            // do something 5 times
            ...
        }
    }
}
```

如果设计中有部分无需使用自动流水线化的循环，请对此类循环应用含 `off` 选项的 `PIPELINE` 指令。`off` 选项可阻止自动循环流水线化。



重要提示！ Vivado HLS 会在执行所有用户指定的指令后应用 `config_compile pipeline_loops` 选项。例如，如果 Vivado HLS 向循环应用用户指定的 `UNROLL` 指令，那么将首先展开该循环，但无法应用自动循环流水线化。

解决流水线化失败的问题

将函数流水线化时，下层层级内所有循环都会自动展开。这是继续执行流水线化的前提。如果循环具有变量边界，则无法展开。这将导致无法对函数进行流水线化。

静态变量

静态变量用于在循环迭代间保留数据，这通常在导致最终实现时生成寄存器。如果在用流水线实现的函数中遇到此变量，`vivado_hls` 可能无法对设计进行充分最优化，这可能导致启动时间间隔超过所需的时间。

以下是此状况的典型示例：

```
function_foo()
{
    static bool change = 0
    if (condition_xyz){
        change = x; // store
    }
    y = change; // load
}
```

如果 `vivado_hls` 无法最优化此代码，则存储操作需要 1 个周期，加载操作也需要 1 个周期。如果此函数包含在流水线中，那么流水线必须以最小启动时间间隔 2 来实现，因为静态变更变量会创建循环附带的依赖关系。

用户可通过重写代码来避免此问题，如下例所示。它可确保在循环的每次迭代中仅存在读操作或写操作，这样即可以 `II=1` 来调度设计。

```
function_readstream()
{
    static bool change = 0
    bool change_temp = 0;
    if (condition_xyz)
    {
        change = x; // store
        change_temp = x;
    }
    else
    {
        change_temp = change; // load
    }
    y = change_temp;
}
```

通过数组分区来改善流水线化

以下消息显示了对函数进行流水线化时常见的问题：

```
INFO: [SCHED 204-61] Pipelining loop 'SUM_LOOP'.
WARNING: [SCHED 204-69] Unable to schedule 'load' operation ('mem_load_2',
bottleneck.c:62) on array 'mem' due to limited memory ports.
WARNING: [SCHED 204-69] The resource limit of core:RAM:mem:p0 is 1, current
assignments:
WARNING: [SCHED 204-69]      'load' operation ('mem_load', bottleneck.c:62)
on array
'mem',
WARNING: [SCHED 204-69] The resource limit of core:RAM:mem:p1 is 1, current
assignments:
WARNING: [SCHED 204-69]      'load' operation ('mem_load_1',
bottleneck.c:62) on array
'mem',
INFO: [SCHED 204-61] Pipelining result: Target II: 1, Final II: 2, Depth: 3.
```

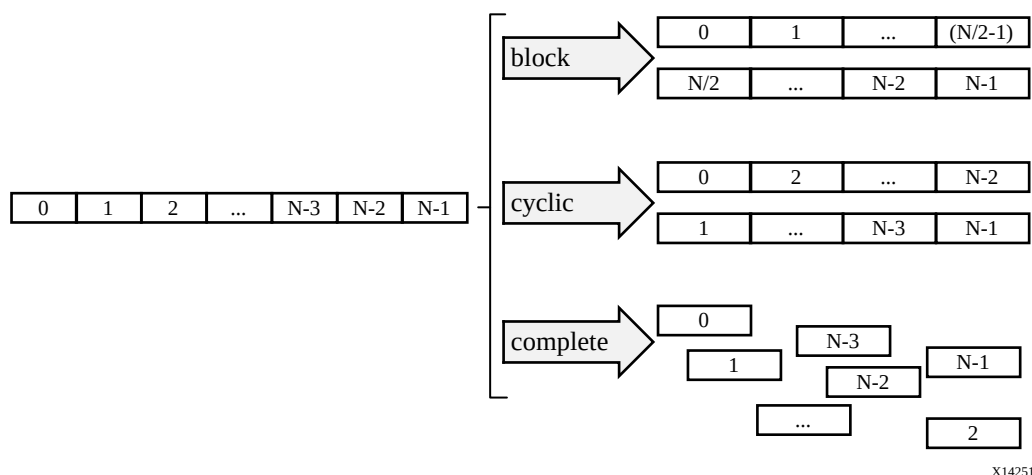
在此示例中，Vivado HLS 声明它无法达成指定的初始时间间隔 (II) 值 1，因为它受内存端口所限，无法在内存上调度 `load`（读取）操作 (`mem_load_2`)。以上消息指出了第 62 行上的 `mem_load` 操作所使用的 “`core:RAM:mem:p0 is 1`” 的资源限制。块 RAM 的第 2 个端口同样仅含 1 项资源，该资源同样供 `mem_load_1` 操作使用。由于存在此内存端口争用，Vivado HLS 报告的最终 II 为 2，而不是所期望的值 1。

此问题通常是由数组所导致的。数组作为最多只含有 2 个数据端口的块 RAM 来实现。这可能限制读写（或加载/存储）密集型算法的吞吐量。通过将该数组（单一块 RAM 资源）拆分为多个更小的数组（多个块 RAM）从而有效增加端口数量，即可改善带宽。

数组可使用 ARRAY_PARTITION 指令来进行分区。Vivado HLS 可提供 3 种类型的数组分区，如下图所示。这 3 种分区样式分别是：

- **block**：原始数组分割为原始数组的连续元素块（大小相同）。
- **cyclic**：原始数组分割多个大小相同的块，这些块交织成原始数组的元素。
- **complete**：默认操作是将数组按其独立元素进行拆分。这对应于将内存解析为寄存器。

图 57：数组分区



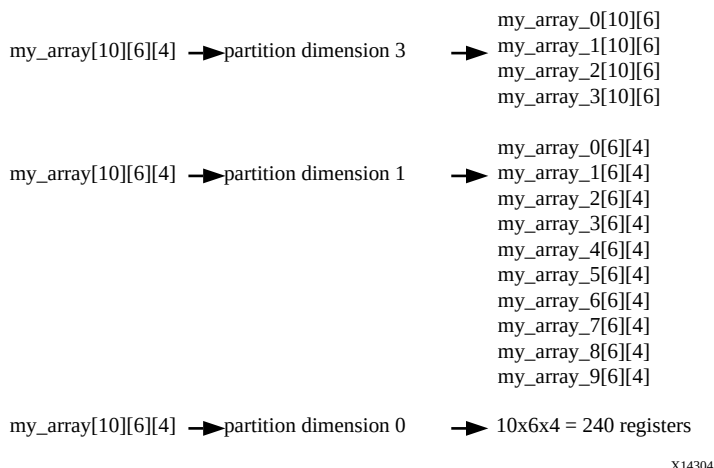
对于 **block** 和 **cyclic** 分区，**factor** 选项可指定要创建的数组数量。在前图中，使用因子 2，即将数组分割为 2 个更小的数字。如果数组的元素数量并非该因子的整数倍，那么最后一个数组所含元素数量较少。

对多维数组进行分区时，**dimension** 选项可用于指定对哪个维度进行分区。下图显示了使用 **dimension** 选项对以下代码示例进行分区的方式：

```
void foo (...) {
    int my_array[10][6][4];
    ...
}
```

此图中的示例演示了如何通过对 **dimension 3** 进行分区来生成 4 个独立数组，以及如何对 **dimension 1** 进行分组以生成 10 个独立分区。如果针对 **dimension** 指定 0，则将对所有维度进行分区。

图 58：对数组维度进行分区



自动数组分区

`config_array_partition` 配置可根据元素数量判定数组的自动分区方式。此配置可通过菜单“Solution” → “Solution Settings” → “General” → “Add” → “config_array_partition”访问。

通过 `throughput_driven` 选项可对分区阈值进行调整，并且可完全实现自动分区。选中 `throughput_driven` 选项时，Vivado HLS 会自动对数组进行分区以实现指定的吞吐量。

与 Vivado HLS 之间的依赖关系

Vivado HLS 会构造对应于 C 语言源代码的硬件数据路径。

如果没有流水线指令，将按顺序执行，并且不考虑任何依赖关系。但对设计进行流水线时，该工具仍需对 Vivado HLS 生成的硬件的处理器架构中法线的依赖关系进行处理。

数据依赖关系或内存依赖关系的典型用例是在完成上一次读操作或写操作后再次发生读操作或写操作。

- 先写后读 (RAW) 操作也称为真性依赖关系，它表示指令（及其读取/使用的数据）从属于前一次操作的结果。

- l1: `t = a * b;`
- l2: `c = t + 1;`

语句 l2 中的读操作取决于语句 l1 中的 t 的写操作。如果对指令进行重新排序，它会使用 t 的前一个值。

- 先读后写 (WAR) 操作也称为反依赖关系，它表示当前一条指令完成数据读取后，下一条指令才能更新寄存器或内存（通过写操作）。

- l1: `b = t + a;`
- l2: `t = 3;`

语句 l2 中的写操作无法在语句 l1 之前执行，否则 b 的结果无效。

- 先写后写 (WAW) 依赖关系表示必须按特定顺序写入寄存器或内存，否则可能破坏其它指令。

- l1: `t = a * b;`
- l2: `c = t + 1;`

- o I3: t = 1;

语句 I3 中的写操作必须晚于语句 I1 中的写操作。否则，语句 I2 结果将出错。

- 先读后读不含任何依赖关系，因为只要变量未声明为易变，即可随意对指令进行重新排序。如果变量声明为易变，则必须保留指令顺序不变。

例如，生成流水线时，工具需确保后续阶段读取的寄存器或内存位置未被先前写操作修改。这属于真性依赖关系或先写后读 (RAW) 依赖关系。具体示例如下：

```
int top(int a, int b) {
    int t,c;
    I1: t = a * b;
    I2: c = t + 1;
    return c;
}
```

在语句 I1 完成前，无法对语句 I2 求值，因为与 t 变量之间存在依赖关系。在硬件中，如果乘法需耗时 3 个时钟周期，那么 I2 将发生等同于此时时间量的延迟。如果对以上函数进行流水线化，那么 VHLS 会将其检测为真性依赖关系，并对操作进行相应调度。它使用数据转发最优化来移除 RAW 依赖关系，因此函数可按 II=1 来运行。

当此示例应用于数组而不仅是变量时，就会出现内存依赖关系。

```
int top(int a) {
    int r=1,rnext,m,i,out;
    static int mem[256];
    L1: for(i=0;i<=254;i++) {
        #pragma HLS PIPELINE II=1
        I1: m = r * a; mem[i+1] = m; // line 7
        I2: rnext = mem[i]; r = rnext; // line 8
    }
    return r;
}
```

在以上示例中，L1 循环的调度导致出现调度警告消息：

```
WARNING: [SCHED 204-68] Unable to enforce a carried dependency constraint
(II = 1,
distance = 1)
between 'store' operation (top.cpp:7) of variable 'm', top.cpp:7 on array
'mem' and
'load' operation ('rnext', top.cpp:8) on array 'mem'.
INFO: [SCHED 204-61] Pipelining result: Target II: 1, Final II: 2, Depth: 3.
```

只要写入的索引不同于读取的索引，那么循环的同一次迭代内就不会发生任何问题。而 2 条指令可同时并行执行。但请观测多次迭代中的读写操作：

```
// Iteration for i=0
I1: m = r * a; mem[1] = m; // line 7
I2: rnext = mem[0]; r = rnext; // line 8
// Iteration for i=1
```

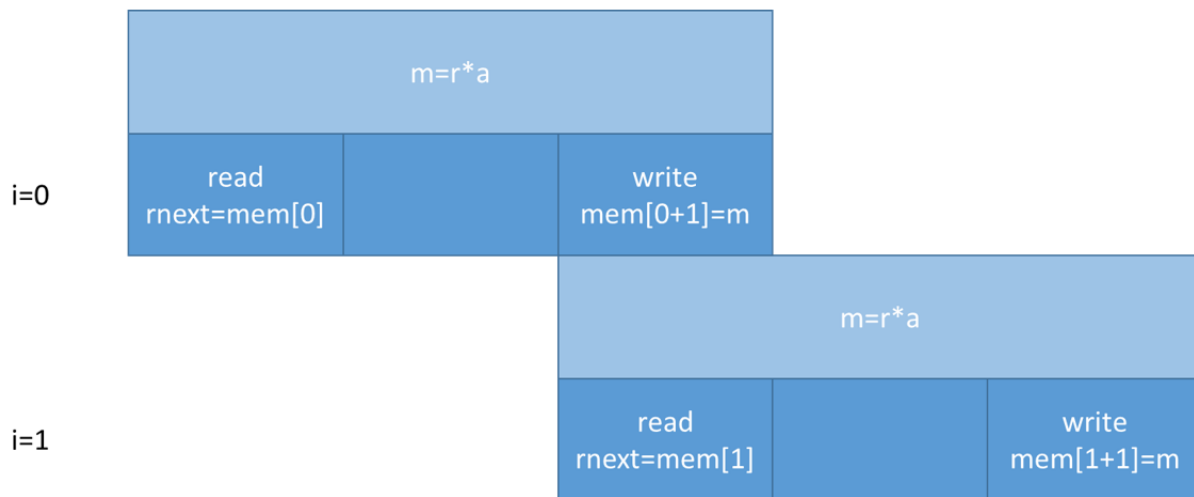
```

I1:    m = r * a; mem[2] = m;      // line 7
I2:    rnext = mem[1]; r = rnext; // line 8
// Iteration for i=2
I1:    m = r * a; mem[3] = m;      // line 7
I2:    rnext = mem[2]; r = rnext; // line 8

```

当考量 2 次连续迭代时，来自 I1 语句的乘法结果 m （时延 = 2）将写入某一位置，而循环的下次迭代的 I2 语句将把位于该位置的结果读取到 $rnext$ 中。在此情况下，存在 RAW 依赖关系，因为上一次计算的写操作完成后，下一次循环迭代才能开始读取 $mem[i]$ 。

图 59：依赖关系示例



请注意，如果增大时钟频率，那么乘法器将需要更多流水线阶段，从而导致时延增加。这也将迫使 II 增大。

请注意以下代码，其中操作已发生交换，导致功能发生改变。

```

int top(int a) {
    int r,m,i;
    static int mem[256];
L1: for(i=0;i<=254;i++) {
#pragma HLS PIPELINE II=1
I1:    r = mem[i];           // line 7
I2:    m = r * a , mem[i+1]=m; // line 8
    }
    return r;
}

```

调度警告为：

```

INFO: [SCHED 204-61] Pipelining loop 'L1'.
WARNING: [SCHED 204-68] Unable to enforce a carried dependency constraint
(II = 1,
distance = 1)
between 'store' operation (top.cpp:8) of variable 'm', top.cpp:8 on array
'mem'
and 'load' operation ('r', top.cpp:7) on array 'mem'.
WARNING: [SCHED 204-68] Unable to enforce a carried dependency constraint
(II = 2,

```

```
distance = 1)
  between 'store' operation (top.cpp:8) of variable 'm', top.cpp:8 on array
'mem'
and 'load' operation ('r', top.cpp:7) on array 'mem'.
WARNING: [SCHED 204-68] Unable to enforce a carried dependency constraint
(II = 3,
distance = 1)
  between 'store' operation (top.cpp:8) of variable 'm', top.cpp:8 on array
'mem'
and 'load' operation ('r', top.cpp:7) on array 'mem'.
INFO: [SCHED 204-61] Pipelining result: Target II: 1, Final II: 4, Depth: 4.
```

请观测多次迭代中的连续读写操作：

```
Iteration with i=0
I1:      r = mem[0];           // line 7
I2:      m = r * a , mem[1]=m; // line 8
Iteration with i=1
I1:      r = mem[1];           // line 7
I2:      m = r * a , mem[2]=m; // line 8
Iteration with i=2
I1:      r = mem[2];           // line 7
I2:      m = r * a , mem[3]=m; // line 8
```

所需 II 延长，因为存在如下 RAW 依赖关系，从 `mem[i]` 读取 `r`、执行乘法并写入 `mem[i+1]`。

移除假性依赖关系以改善循环流水线化

假性依赖关系即编译器过于保守时出现的依赖关系。这些依赖关系在真实代码中并不存在，但无法由编译器来判定。这些依赖关系可能阻碍循环流水线化。

假性依赖关系如下示例所示。在此示例中，针对相同循环迭代内的 2 个不同地址执行读写访问。这 2 个地址均依赖于输入数据，可指向 `hist` 数组中的任一元素。有鉴于此，Vivado HLS 假定这 2 个地址可访问同一个位置。因此，它安排按交替周期对数组执行读写操作，导致循环 II 为 2。但代码显示 `hist[old]` 和 `hist[val]` 永远无法访问相同地址，因为这两者包含在 `if(old == val)` 条件的 `else` 分支中。

```
void histogram(int in[INPUT SIZE], int hist[VALUE SIZE]) f
{
    int acc = 0;
    int i, val;
    int old = in[0];
    for(i = 0; i < INPUT SIZE; i++)
    {
        #pragma HLS PIPELINE II=1
        val = in[i];
        if(old == val)
        {
            acc = acc + 1;
        }
        else
        {
            hist[old] = acc;
            acc = hist[val] + 1;
        }

        old = val;
    }

    hist[old] = acc;
}
```

为克服这一缺陷，您可使用 DEPENDENCE 指令为 Vivado HLS 提供有关这些依赖关系的附加信息。

```
void histogram(int in[INPUT SIZE], int hist[VALUE SIZE]) {
    int acc = 0;
    int i, val;
    int old = in[0];
    #pragma HLS DEPENDENCE variable=hist intra RAW false
    for(i = 0; i < INPUT SIZE; i++)
    {
        #pragma HLS PIPELINE II=1
        val = in[i];
        if(old == val)
        {
            acc = acc + 1;
        }
        else
        {
            hist[old] = acc;
            acc = hist[val] + 1;
        }

        old = val;
    }

    hist[old] = acc;
}
```

注释：在实际上依赖关系并非 FALSE 的情况下指定 FALSE 依赖关系可能导致硬件错误。指定依赖关系前，请确认它是否正确（TRUE 或 FALSE）。

指定依赖关系时，有 2 种主要类型：

- Inter：指定相同循环的不同迭代之间的依赖关系。

如指定为 FALSE，则当循环已流水线化、已展开或已部分展开时，允许 Vivado HLS 并行执行运算，指定为 TRUE 时则阻止此类并行运算。

- Intra：指定循环的相同迭代内的依赖关系，例如，在相同迭代开始和结束时访问的数组。

当 intra 依赖关系指定为 FALSE 时，Vivado HLS 可在循环内自由移动运算、提升运算移动性，从而可能改善性能或面积。当此依赖关系指定为 TRUE 时，必须按指定顺序执行运算。

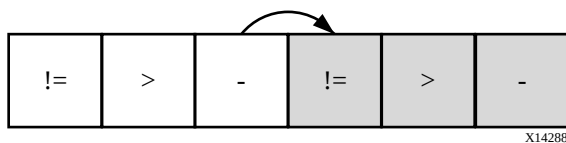
标量依赖关系

部分标量依赖关系较难以解析，且通常需要更改源代码。标量数据依赖关系如下所示：

```
while (a != b) {
    if (a > b) a -= b;
    else b -= a;
}
```

此循环的当前迭代完成 a 和 b 的更新值计算后才能启动下一次迭代，如下图所示。

图 60：标量依赖关系



如果必须得到上一次循环迭代结果后才能开始当前迭代，则无法进行循环流水线化。如果 Vivado HLS 无法以指定的启动时间间隔进行流水线化，那么它会增加启动时间间隔。如果完全无法流水线化，则它会停止流水线化并继续输出非流水线化设计。

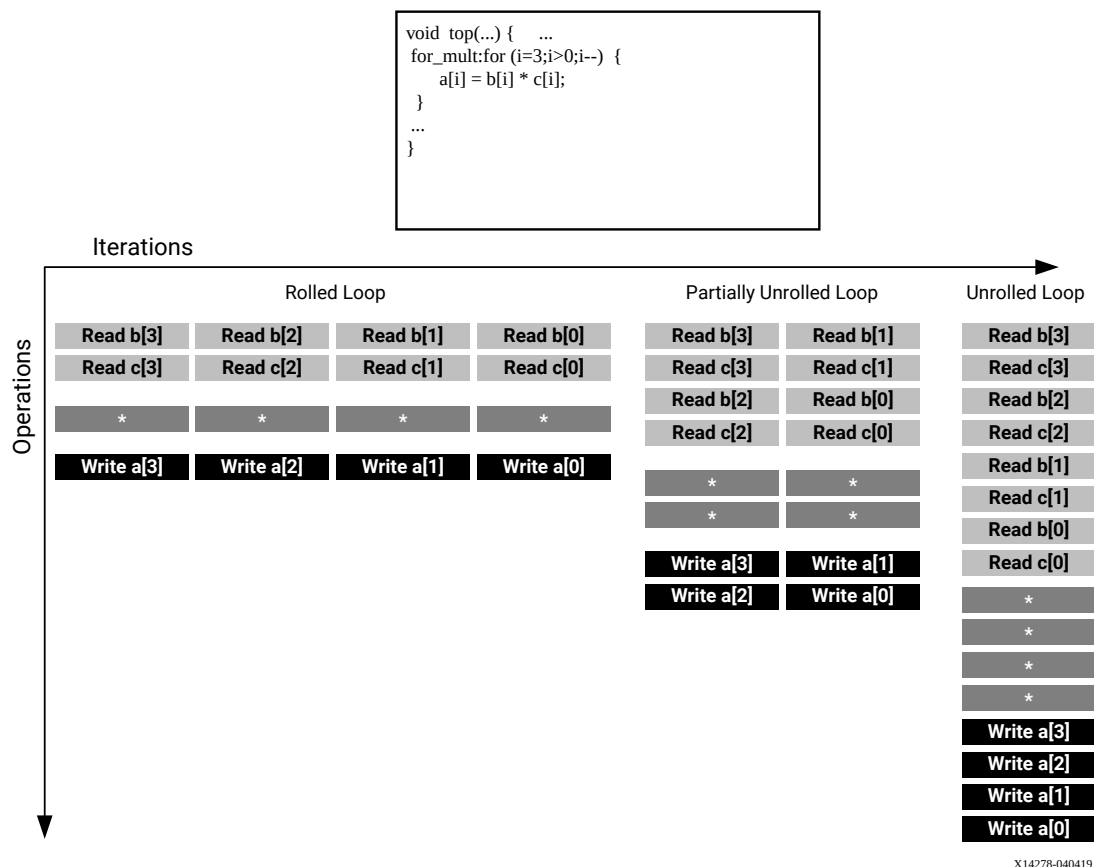
最优化循环展开以改善流水线

默认情况下，在 Vivado HLS 中循环保持处于收起状态。这些收起的循环会生成硬件资源，供循环的每次迭代使用。虽然这样可创建资源节约型块，但有时可能导致性能瓶颈。

Vivado HLS 可提供使用 UNROLL 指令来展开或部分展开 for 循环的功能。

下图显示了循环展开的优势以及展开循环时必须考量的影响。此示例假定 `a[i]`、`b[i]` 和 `c[i]` 数组已映射到块 RAM。此示例显示只需直接应用循环展开即可同时创建大量不同实现。

图 61：循环展开详情



- 循环已收起：当循环已收起时，每次迭代都在单独的时钟周期内执行。此实现需耗时 4 个时钟周期，只需 1 个乘法器并且每个块 RAM 均可作为单端口块 RAM。
- 循环已部分展开：在此示例中，循环已按因子 2 部分展开。此实现需 2 个乘法器和双端口 RAM 以支持在同一个时钟周期内读取或写入每个 RAM。但此实现只需 2 个时钟周期即可完成：相比于循环的收起版本，启动时间间隔和时延均减半。
- 循环已展开：在完全展开的版本中，可在单一时钟周期内执行所有循环操作。但此实现需 4 个乘法器。更重要的是，此实现需在同一个时钟周期内执行 4 次读操作和 4 次写操作的功能。由于块 RAM 最多仅有 2 个端口，因此此实现需对数组进行分区。

要执行循环展开，可向设计中的每个循环应用 UNROLL 指令。或者，可向函数应用 UNROLL 指令，以展开函数作用域内的所有循环。

如果循环已完全展开，那么只要数据依赖关系和资源允许，即可并行执行所有操作。如果某一循环迭代中的操作需要上一次循环的结果，则这两次迭代无法并行执行，但一旦数据可用即可立即执行。完全展开并完全最优化的循环通常涉及循环主体中的多个逻辑副本。

以下示例演示了如何使用循环展开来创建最优化的设计。在此示例中，数据作为交织式通道存储在数组中。如果按 $ll=1$ 来对循环进行流水线化，则每经过 8 个时钟周期才会对每个通道执行依次读取和写入。

```
// Array Order : 0 1 2 3 4 5 6 7 8 9 10 etc. 16
etc...
// Sample Order: A0 B0 C0 D0 E0 F0 G0 H0 A1 B1 C2 etc. A2
etc...
// Output Order: A0 B0 C0 D0 E0 F0 G0 H0 A0+A1 B0+B1 C0+C2 etc. A0+A1+A2
etc...

#define CHANNELS 8
#define SAMPLES 400
#define N CHANNELS * SAMPLES

void foo (dout_t d_out[N], din_t d_in[N]) {
    int i, rem;

    // Store accumulated data
    static dacc_t acc[CHANNELS];

    // Accumulate each channel
    For_Loop: for (i=0;i<N;i++) {
        rem=i%CHANNELS;
        acc[rem] = acc[rem] + d_in[i];
        d_out[i] = acc[rem];
    }
}
```

按 `factor` 为 8 来对循环进行部分展开将允许并行处理每个通道（每 8 个样本为一组），前提是输入数组和输出数组同样按 `cyclic` 方式进行分区，以便在每个时钟周期内进行多次访问。如果此循环同时采用 `rewind` 选项来进行流水线化，那么此设计将持续并行处理全部 8 个通道，前提是以流水线方式（即在顶层或者在数据流区域中）调用这些通道。

```
void foo (dout_t d_out[N], din_t d_in[N]) {
    #pragma HLS ARRAY_PARTITION variable=d_i cyclic factor=8 dim=1 partition
    #pragma HLS ARRAY_PARTITION variable=d_o cyclic factor=8 dim=1 partition

    int i, rem;

    // Store accumulated data
    static dacc_t acc[CHANNELS];

    // Accumulate each channel
    For_Loop: for (i=0;i<N;i++) {
        #pragma HLS PIPELINE rewind
        #pragma HLS UNROLL factor=8

        rem=i%CHANNELS;
        acc[rem] = acc[rem] + d_in[i];
        d_out[i] = acc[rem];
    }
}
```

部分循环展开不要求展开因子为最大迭代计数的整数倍。Vivado HLS 会添加出口检查以确保部分展开的循环的功能与原始循环相同。例如，给定以下代码：

```
for(int i = 0; i < N; i++) {
    a[i] = b[i] + c[i];
}
```

按因子 2 展开的循环可将代码有效变换为如下示例所示形式，其中 `break` 构造器用于确保功能保持不变：

```
for(int i = 0; i < N; i += 2) {
    a[i] = b[i] + c[i];
    if (i+1 >= N) break;
    a[i+1] = b[i+1] + c[i+1];
}
```

由于 `N` 为变量，Vivado HLS 可能无法判定其最大值（它可能受输入端口驱动）。如果展开因子（在此例中为 2）是最大迭代计数 `N` 的整数因子，那么 `skip_exit_check` 选项会移除出口检查和关联的逻辑。展开的效果现在可表示为：

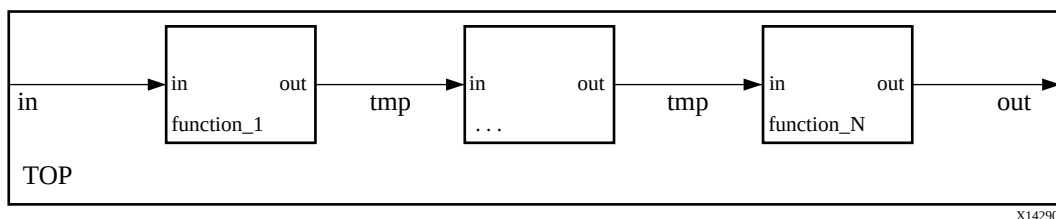
```
for(int i = 0; i < N; i += 2) {
    a[i] = b[i] + c[i];
    a[i+1] = b[i+1] + c[i+1];
}
```

这有助于最大限度降低面积并简化控制逻辑。

利用任务级别并行化：数据流最优化

数据流最优化对于一系列顺序任务（例如，函数和/或循环）很实用，如下图所示。

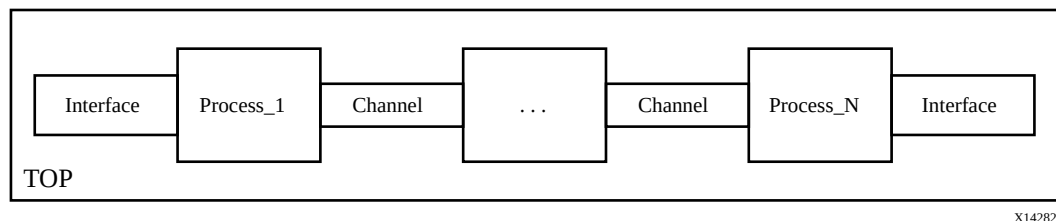
图 62：序列功能描述



上图显示了连续 3 个任务的特定情况，但通信结构比所示情况可能更复杂。

通过使用这一系列顺序任务，数据流最优化可以创建并发进程架构，如下所示。数据流最优化是可用于改进设计吞吐量和时延的强大方法。

图 63：并行进程架构

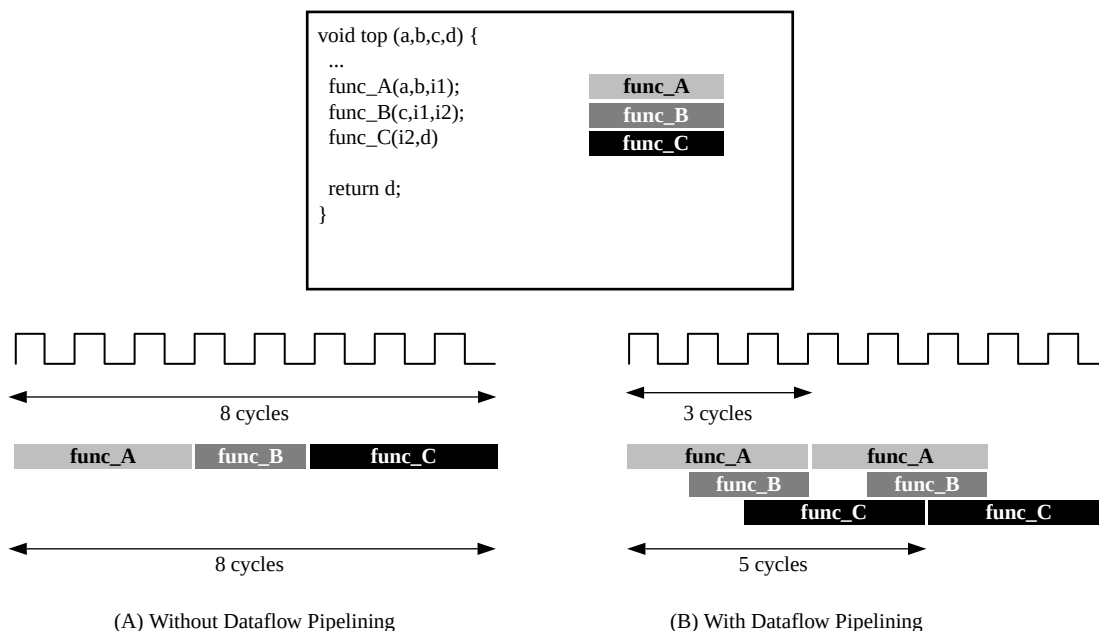


下图显示了数据流最优化允许重叠执行任务的方式，由此可提升总体设计吞吐量并降低时延。

在以下图示和示例中，(A) 表示无数据流最优化的情况。实现需经历 8 个周期后，`func_A` 才能处理新输入，还需要 8 个周期后 `func_C` 才能写入输出。

对于同样的示例，(B) 表示应用数据流最优化的情况。`func_A` 每隔 3 个时钟周期即可开始处理新输入（启动时间间隔更低），只需 5 个时钟即可输出最终值（时延更短）。

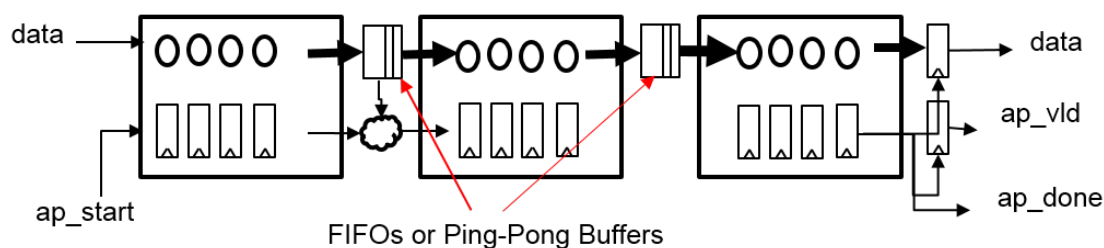
图 64：数据流优化



X14266

这种类型的并行化势必伴随着硬件开销。将某个特定区域（例如，函数主体或循环主体）识别为要应用数据流最优化的区域时，Vivado HLS 会分析此函数主体或循环主体，并创建独立通道以对数据流进行建模，用于将每项任务的结果存储在数据流区域中。这些通道对于标量变量而言可能只是简单的 FIFO，而对于数组之类非标量变量，则可能是乒乓 (PIPO) 缓存。其中每个通道还都包含用于指示 FIFO 或乒乓缓存已满或已空的信号。这些信号表示完全数据驱动的握手接口。通过采用独立 FIFO 和/或乒乓缓存，Vivado HLS 可使每项任务按其自己的步调执行，吞吐量仅受输入和输出缓存的可用性限制。由此实现的任务执行交织比正常流水线化实现更好，但导致增加 FIFO 或块 RAM 寄存器（用于乒乓缓存）成本。前图所示的数据流区域实现的结构与下图中相同示例所示结构相同。

图 65：数据流最优化期间创建的结果



数据流最优化的性能可能比静态流水线化解决方案的性能更好。它将严格集中控制的流水线停滞理念替换为更灵活的分布式握手架构，后者使用 FIFO 和/或乒乓缓存。数据流最优化并不局限于进程链，也可用于任何 DAG 结构。它可生成 2 种不同形式的重叠：在迭代内部（如果使用 FIFO 连接进程）重叠和在不同迭代之间重叠（通过 PIPO 和 FIFO 连接进程）。

规范形式

Vivado HLS 通过转换区域来应用 DATAFLOW 最优化。赛灵思建议在此区域内部（称为“规范区域”）使用规范格式来编写代码。数据流最优化有 2 种主要的规范形式：

1. 适用于未内联的函数的函数规范形式。

```
void dataflow(Input0, Input1, Output0, Output1)
{
    #pragma HLS dataflow
    UserData Type C0, C1, C2;
    func1(read Input0, read Input1, write C0, write C1);
    func2(read C0, read C1, write C2);
    func3(read C2, write Output0, write Output1);
}
```

2. 循环主体内部的数据流。

对于 for 循环（内部函数未内联），整型循环变量应包含：

- 循环头文件中声明初始值（设置为 0）。
- 循环条件为正数常量或常量函数实参。
- 按 1 递增。
- 数据流编译指示需位于循环内。

```
void dataflow(Input0, Input1, Output0, Output1)
{
    for (int i = 0; i < N; i++)
    {
        #pragma HLS dataflow
        UserData Type C0, C1, C2;
        func1(read Input0, read Input1, write C0, write C1);
        func2(read C0, read C0, read C1, write C2);
        func3(read C2, write Output0, write Output1);
    }
}
```

规范主体

在规范区域内，规范主体应遵循如下准则：

- 使用局部非静态标量或数组/指针变量，或者使用局部静态流传输变量。局部变量是在函数主体（对应函数中的数据流）或循环主体（对应循环中的数据流）内声明的。
- 在以下条件下，函数调用序列向前馈送数据（不含反馈）的方式为从某一函数到按词法顺序位居其后的另一函数：
 - 变量（标量除外）只能包含 1 个读进程和 1 个写进程。
 - 如果使用稍后变为通道的局部变量，请先使用写进程，再使用读进程（生产者位于使用者之前）。
 - 如果使用函数实参，请先使用读进程，再使用写进程（使用者位于生产者之前）。设计必须保留所有主体内部反依赖关系。
 - 函数返回类型必须为空。
 - 通过变量的不同进程间不得存在任何循环进位依赖关系。
 - 在规范循环内部（即，某一迭代编写的值由后一迭代读取）。
 - 在连续调用顶层函数之间执行（即，某一迭代编写的输入输出实参由后一迭代读取）。

数据流检查

Vivado HLS 具有数据流检查程序，启用该检查程序后即可检查代码，确认其是否符合建议的规范形式。如不符合，则将向用户发出错误/警告消息。默认情况下，此检查程序设置为 `warning`。您可将此检查程序设置为 `error` 或者在 `config_dataflow` Tcl 命令的严格模式下选择 `off` 以将其禁用。

```
config_dataflow -strict_mode (off | error | warning)
```

数据流最优化限制

DATAFLOW 最优化可对任务（函数和循环）之间的数据流以及按理想方式最优化的函数和循环进行最优化以实现最大性能。这些任务无需逐一链接，但数据传输方式存在某些限制。

以下行为可能阻止或限制 Vivado® HLS 可通过 DATAFLOW 最优化执行的重叠。

- 单一生产者使用者违例
- 绕过任务
- 任务间的反馈
- 任务的有条件执行
- 含多个退出条件的循环



重要提示！ 如果存在上述任一编码样式，Vivado HLS 就会发出一条描述状况的消息。

注释：“Analysis” 透视图中的数据流查看器可用于查看应用 DATAFLOW 指令时的结构。

单一生产者使用者违例

为便于 Vivado HLS 执行 DATAFLOW 最优化，任务间传递的所有元素都必须遵循单一生产者使用者模型。每个变量都必须从单一任务驱动，并且仅限供单一任务使用。在以下代码示例中，`temp1` 将扇出并供 `Loop2` 和 `Loop3` 使用。这违背了单一生产者使用者模型的要求。

```
void foo(int data_in[N], int scale, int data_out1[N], int data_out2[N]) {
    int temp1[N];
    Loop1: for(int i = 0; i < N; i++) {
        temp1[i] = data_in[i] * scale;
    }
    Loop2: for(int j = 0; j < N; j++) {
        data_out1[j] = temp1[j] * 123;
    }
    Loop3: for(int k = 0; k < N; k++) {
        data_out2[k] = temp1[k] * 456;
    }
}
```

此代码的修改后版本使用 Split 函数来创建单一生产者使用者设计。在此情况下，数据流从 Loop1 传输到 Split 函数，然后传输到 Loop2 和 Loop3。现在数据在全部 4 个任务之间流动，这样 Vivado HLS 即可执行 DATAFLOW 最优化。

```
void Split (in[N], out1[N], out2[N]) {
// Duplicated data
  L1:for(int i=1;i<N;i++) {
    out1[i] = in[i];
    out2[i] = in[i];
  }
}
void foo(int data_in[N], int scale, int data_out1[N], int data_out2[N]) {

  int temp1[N], temp2[N], temp3[N];
  Loop1: for(int i = 0; i < N; i++) {
    temp1[i] = data_in[i] * scale;
  }
  Split(temp1, temp2, temp3);
  Loop2: for(int j = 0; j < N; j++) {
    data_out1[j] = temp2[j] * 123;
  }
  Loop3: for(int k = 0; k < N; k++) {
    data_out2[k] = temp3[k] * 456;
  }
}
```

绕过任务

此外，通常数据应在不同任务间流动。如果您绕过任务，则可能降低 DATAFLOW 最优化的性能。在以下示例中，Loop1 会为 temp1 和 temp2 生成值。但下一项任务 Loop2 仅使用 temp1 的值。直至 Loop2 后才会使用 temp2 的值。因此，temp2 会绕过序列中的下一项任务，这可能限制 DATAFLOW 最优化的性能。

```
void foo(int data_in[N], int scale, int data_out1[N], int data_out2[N]) {

  int temp1[N], temp2[N], temp3[N];
  Loop1: for(int i = 0; i < N; i++) {
    temp1[i] = data_in[i] * scale;
    temp2[i] = data_in[i] >> scale;
  }
  Loop2: for(int j = 0; j < N; j++) {
    temp3[j] = temp1[j] + 123;
  }
  Loop3: for(int k = 0; k < N; k++) {
    data_out[k] = temp2[k] + temp3[k];
  }
}
```

由于循环迭代限制在此示例中全都相同，因此您可修改代码，以使 Loop2 使用 temp2 并生成 temp4，如下所示。这样可确保数据从任一任务流至下一任务。

```
void foo(int data_in[N], int scale, int data_out1[N], int data_out2[N]) {

  int temp1[N], temp2[N], temp3[N], temp4[N];
  Loop1: for(int i = 0; i < N; i++) {
    temp1[i] = data_in[i] * scale;
    temp2[i] = data_in[i] >> scale;
  }
}
```



```

Loop2: for(int j = 0; j < N; j++) {
temp3[j] = temp1[j] + 123;
temp4[j] = temp2[j];
}
Loop3: for(int k = 0; k < N; k++) {
data_out[k] = temp4[k] + temp3[k];
}
}

```

任务间的反馈

当任务输出供 DATAFLOW 区域中前一个任务使用时，就会发生反馈。在 DATAFLOW 区域中不允许任务间反馈。当 Vivado HLS 检测到反馈时，它会根据情况发出警告，并且可能不执行 DATAFLOW 最优化。

任务的有条件执行

DATAFLOW 最优化不会对有条件执行的任务进行最优化。以下示例着重演示了此限制。在此示例中，Loop1 和 Loop2 的有条件执行会阻止 Vivado HLS 对这些循环之间的数据流进行最优化，因为在循环之间不发生数据流动。

```

void foo(int data_in1[N], int data_out[N], int sel) {

int temp1[N], temp2[N];

if (sel) {
Loop1: for(int i = 0; i < N; i++) {
temp1[i] = data_in[i] * 123;
temp2[i] = data_in[i];
}
} else {
Loop2: for(int j = 0; j < N; j++) {
temp1[j] = data_in[j] * 321;
temp2[j] = data_in[j];
}
}
Loop3: for(int k = 0; k < N; k++) {
data_out[k] = temp1[k] * temp2[k];
}
}

```

要确保在所有情况下都能执行每个循环，必须按以下示例中所示方式对代码进行转换。在此示例中，条件语句已移至首个循环内。这两个循环都始终执行，数据始终在循环间进行流传输。

```

void foo(int data_in[N], int data_out[N], int sel) {

int temp1[N], temp2[N];

Loop1: for(int i = 0; i < N; i++) {
if (sel) {
temp1[i] = data_in[i] * 123;
} else {
temp1[i] = data_in[i] * 321;
}
}
Loop2: for(int j = 0; j < N; j++) {
temp2[j] = data_in[j];
}
}

```

```

}
Loop3: for(int k = 0; k < N; k++) {
    data_out[k] = temp1[k] * temp2[k];
}
}

```

含多个退出条件的循环

在 DATAFLOW 区域中无法使用含多个退出点的循环。在以下示例中，Loop2 具有 3 项退出条件：

- 由 N 的值定义的出口：当 $k \geq N$ 时循环将退出。
- 由 break 语句定义的出口。
- 由 continue 语句定义的出口。

```

#include "ap_cint.h"
#define N 16

typedef int8 din_t;
typedef int15 dout_t;
typedef uint8 dsc_t;
typedef uint1 dsel_t;

void multi_exit(din_t data_in[N], dsc_t scale, dsel_t select, dout_t
data_out[N]) {
    dout_t temp1[N], temp2[N];
    int i,k;

    Loop1: for(i = 0; i < N; i++) {
        temp1[i] = data_in[i] * scale;
        temp2[i] = data_in[i] >> scale;
    }

    Loop2: for(k = 0; k < N; k++) {
        switch(select) {
            case 0: data_out[k] = temp1[k] + temp2[k];
            case 1: continue;
            default: break;
        }
    }
}

```

由于循环的退出条件始终由循环边界来定义，因此使用 break 或 continue 语句将禁止在 DATAFLOW 区域内使用循环。

最后，DATAFLOW 最优化不含层级实现。如果子函数或循环包含可能受益于 DATAFLOW 最优化的其它任务，那么必须对该函数、子函数或子函数的内联应用 DATAFLOW 最优化。

注释：std::complex 无法直接在 DATAFLOW 区域内使用。应将其定义为原生数据类型和生产者 (producer) 内强制进行转换的类型。

```

#dataflow
float A[N][2];
prod(A, in);
cons(out,A);

Producer(std::complex &)
{
}

```

配置数据流内存通道

根据数据生产者和使用者的访问模式，Vivado HLS 将任务间的通道作为乒乓缓存或 FIFO 缓存来实现。

- 对于标量、指针和引用参数，Vivado HLS 将通道作为 FIFO 来实现。
- 如果参数（生产者或使用者）为数组，那么 Vivado HLS 会将通道作为乒乓缓存或 FIFO 来实现，如下所示：
 - 如果 Vivado HLS 判定数据按顺序访问，那么 Vivado HLS 会将内存通道作为深度为 2 的 FIFO 通道来实现。
 - 如果 Vivado HLS 无法判定数据是否按顺序访问或者判定数据采用随机方式访问，那么 Vivado HLS 会将内存通道作为乒乓缓存来实现，即，将其作为 2 个块 RAM，分别由使用者数组或生产者数组的最大大小来定义。

注释：乒乓缓存可确保通道始终具有保存所有样本（不出现丢失）所需的容量。但这种方法在某些情况下可能过于保守。

要显式指定任务之间使用的默认通道，请使用 `config_dataflow` 配置。此配置可为设计中的所有通道设置默认通道。要减小通道中使用的内存大小并允许在单一迭代内存在重叠，可使用 FIFO。要显式设置 FIFO 中的深度（即，元素数量），请使用 `-fifo_depth` 选项。

指定的 FIFO 通道大小会覆盖默认设置。如果设计中的任何任务生成或使用的样本都可大于指定的 FIFO 大小，那么 FIFO 可能变为空（或满）。在此情况下，设计将暂停操作，因为它无法读取（或写入）。这可能导致停滞或死锁状态。

注释：如果造成死锁状态，那么仅当执行 C/RTL 协同仿真时或者在完整的系统中使用此块时，您才会看到此状态。

设置 FIFO 的深度时，赛灵思建议将初始深度设置为传输的数据值的最大数量（即，任务间传递的数组的大小）、确认设计能够成功通过 C/RTL 协同仿真，然后减小 FIFO 大小并确认 C/RTL 协同仿真仍可成功完成且不会出现任何问题。如果 RTL 协同仿真失败，那么 FIFO 大小可能太小，而无法阻止出现停滞或死锁状况。

将数组指定为乒乓缓存或 FIFO

默认情况下，所有数组均作为乒乓缓存来实现以启用随机访问。这些缓存也可根据需要调整大小。例如，在某些情况下（如绕过任务时），可能会出现性能劣化。为缓解对性能造成的影响，可通过使用 STREAM 指令来调整这些缓存的大小，以向生产者 (producer) 和使用者 (consumer) 提供更多裕量，如下所示。

```
void top ( ... ) {
#pragma HLS dataflow
  int A[1024];
#pragma HLS stream off variable=A depth=3

  producer(A, B, ...); // producer writes A and B
  middle(B, C, ...); // middle reads B and writes C
  consumer(A, C, ...); // consumer reads A and C
}
```

在接口中，如果顶层函数接口上的数组设置为 `ap_fifo`、`axis` 或 `ap_hs` 类型的接口，那么数组会自动指定为流传输。

在设计内，如果实现需要 FIFO，必须使用 STREAM 指令将所有数组指定为流传输。

注释：对数组应用 STREAM 指令时，硬件内实现生成的 FIFO 包含的元素数量与数组同样多。`-depth` 选项可用于指定 FIFO 的大小。

STREAM 指令还用于从 `config_dataflow` 配置指定的默认实现更改 DATAFLOW 区域内的所有数组。

- 如果 `config_dataflowdefault_channel` 设置为乒乓，那么可通过向任意数组应用 STREAM 指令以将该数组作为 FIFO 来实现。

注释：要使用 FIFO 实现，必须以流传输方式来访问数组。

- 如果 `config_dataflowdefault_channel` 设置为 FIFO，或者 Vivado HLS 已自动判定以流传输方式访问 DATAFLOW 区域中的数据，那么仍可通过向数组应用含 `-off` 选项的 STREAM 指令，以便将任意数组作为乒乓实现来加以实现。



重要提示！ 要保留访问，可能需要使用 `volatile` 限定符来阻止编译器最优化（尤其是死码消除）。

当 DATAFLOW 区域中的数组指定为作为 FIFO 来进行流传输和实现时，通常 FIFO 无需保留与原始数组相同数量的元素。当数据采样可用后，DATAFLOW 区域中的任务会立即使用该数据采样。含 `-fifo_depth` 选项的 `config_dataflow` 命令或者含 `-depth` 的 STREAM 指令可用于将 FIFO 大小设置为必需的最小元素数量以确保数据流从不停滞。如果选中 `-off` 选项，`-off` 选项会设置乒乓的深度（块数）。深度应至少为 2。

指定编译器 - FIFO 深度

起始传输

编译器可能自动创建“起始 FIFO”以将“起始标识”传输到内部进程。此类 FIFO 有时可能成为性能瓶颈，在此情况下，可通过以下命令增大默认大小（固定为 2）。

```
config_dataflow -start_fifo_depth <value>
```

如果生产者与使用者之间需无限裕量，并且内部进程可永久运行，并完全由其输入或输出（FIFO 或 PIPO）以安全方式驱动，那么可通过以下编译指示针对给定数据流区域在本地移除这些起始 FIFO（用户自担风险）。

```
#pragma HLS DATAFLOW disable_start_propagation
```

标量传输

编译器通过进程之间的“标量 FIFO”自动从 C/C++ 代码传输部分标量。此类 FIFO 有时可能成为性能瓶颈或者导致死锁，在此情况下，可通过以下命令来设置大小（默认值设置为 `-fifo_depth`）：

```
config_dataflow -scalar_fifo_depth <value>
```

Stable 数组

`stable` 编译指示可用于标记数据流区域的输入变量或输出变量。其作用是移除其对应的同步，假定用户保证此移除行为确实正确无误。

```
void dataflow_region(int A[...], ...
#pragma HLS stable variable=A
#pragma HLS dataflow
    proc1(...);
    proc2(A, ...);
```

如果不使用 `stable` 编译指示，并假定 A 由 `proc2` 读取，那么 `proc2` 会成为其所在位置的数据流区域内执行的初始同步（通过 `ap_start`）的一部分。这意味着仅当 `proc2` 同样准备好重新启动后，`proc1` 才能重新启动，这会阻止数据流迭代重叠，从而可能导致性能下降。`stable` 编译指示表明无需此同步亦可保持操作正确性。

在以上示例中，如无 `stable` 编译指示并假定 A 由 `proc2` 读取（因为 `proc2` 当前绕过任务），将出现性能损失。

通过 `stable` 编译指示，编译器假定：

- 如果 A 由 `proc2` 读取，那么执行 `dataflow_region` 时，任何其它进程或调用上下文都不会覆盖读取的内存位置。

- 如果 A 由 `proc2` 写入，那么执行 `dataflow_region` 时，任何其它进程或调用上下文在定义写入的内存位置前都不会读取这些位置。

典型场景是仅当数据流区域尚未开始执行或者已完成执行时，调用程序才会更新或读取这些变量。

在数据流内使用 `ap_ctrl_none`

`ap_ctrl_none` 块级 I/O 协议会避免使用 `ap_ctrl_hs` 和 `ap_ctrl_chain` 协议所暗示的僵化的同步方案。这些协议要求区域内所有进程的执行次数完全相同以便与 C 语言行为更匹配。

但在某些情况下，例如，如果希望通过某个执行频率更高且更快速的进程将工作分发给多个较慢的进程，

那么对于任何数据流区域（“循环中的数据流”除外），均可指定：

```
#pragma HLS interface ap_ctrl_none port=return
```

前提是满足以下条件：

- 区域及其包含的所有进程（`hls::stream`、流传输的数组、AXIS）均仅通过 FIFO 进行通信；即，排除内存。
- 该区域的所有父级（直至顶层设计）都必须满足以下要求：
 - 必须为数据流区域（不包括“循环中的数据流”）。
 - 必须全部指定 `ap_ctrl_none`。

这意味着层级中包含 `ap_ctrl_none` 的数据流区域的所有父级都不能属于：

- 顺序或流水线 FSM
- `for` 循环内的数据流区域（“循环中的数据流”）

此编译指示会导致不使用 `ap_ctrl_chain` 来同步该区域内的任意进程。这些进程根据其输入 FIFO 中的数据可用性及其输出 FIFO 中的空间来执行或停滞。例如：

```
void region(...) {
#pragma HLS dataflow
#pragma HLS interface ap_ctrl_none port=return
    hls::stream<int> outStream1, outStream2;
    demux(inStream, outStream1, outStream2);
    worker1(outStream1, ...);
    worker2(outStream2, ...);
}
```

在此示例中，`demux` 的执行频率可达 `worker1` 和 `worker2` 的 2 倍。例如，它可设置为 `II=1` 而 `worker1` 和 `worker2` 可设置为 `II=2`，且仍可达成全局 `II=1` 行为。

注释：

- 在执行频率较低的进程内，可能需谨慎使用非阻塞读取，以确保 C 语言仿真有效。
- 此编译指示应用于某个区域，而不是应用于该区域内的个别进程。
- 在协同仿真上必须禁用死锁检测。可通过 `cosim_design` 中的 `-disable_deadlock_detection` 选项来执行此操作。

时延最优化

使用时延约束

Vivado HLS 支持对任意作用域使用时延约束。时延约束是使用 LATENCY 指令来指定的。

对作用域施加最大和/或最小 LATENCY 约束时，Vivado HLS 会尝试确保函数内的所有运算都在指定的时钟周期范围内完成。

应用于循环的时延指令可指定单次循环迭代所需的时延：它指定循环主体的时延，如以下示例所示：

```
Loop_A: for (i=0; i<N; i++) {  
  #pragma HLS latency max=10  
  ..Loop Body...  
}
```

如果要限制所有循环迭代的总时延，应将时延指令应用于包含整个循环的区域，如以下示例所示：

```
Region_All_Loop_A: {  
  #pragma HLS latency max=10  
  Loop_A: for (i=0; i<N; i++)  
  {  
    ..Loop Body...  
  }  
}
```

在此情况下，即使展开循环，时延指令仍会对所有循环操作设置最大限制。

如果 Vivado HLS 无法满足最大时延约束，它会放宽时延约束，并尝试尽可能实现最佳结果。

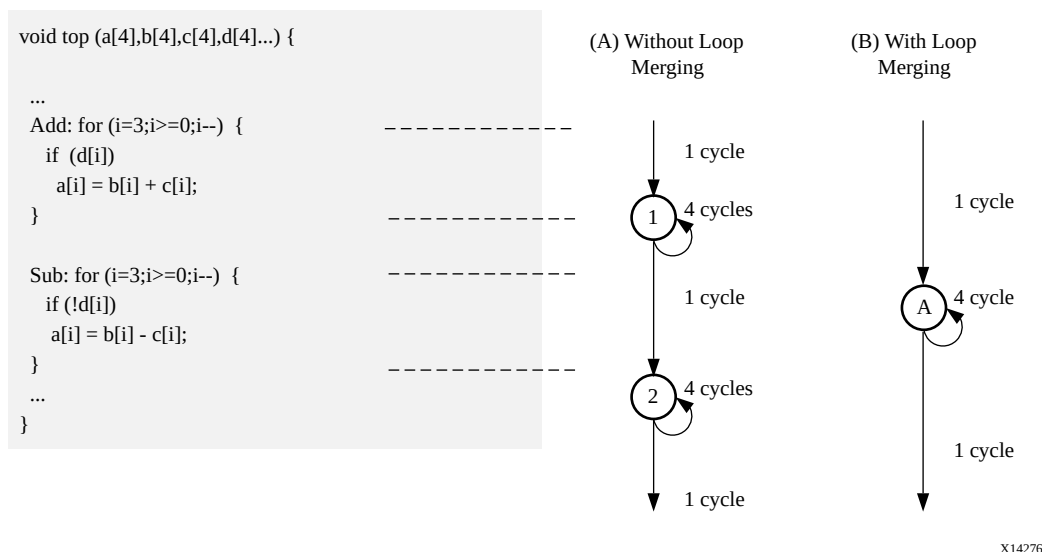
如果设置最小时延约束，并且 Vivado HLS 生成的设计时延低于所需的最小值，它会插入虚拟时钟周期以满足最小时延。

合并顺序循环以减少时延

所有收起的循环都在设计 FSM 中指明并创建至少一种状态。当存在多个顺序循环时，它可能会创建其它不必要的时钟周期，并阻止进一步的最优化。

下图显示了一个简单的示例，其中看似直观的编码样式对 RTL 设计的性能产生了负面影响。

图 66：循环指令



X14276

在上图中，(A) 显示了默认情况下设计中的每个收起的循环如何在 FSM 中创建至少一种状态。在这些状态之间移动需耗费多个时钟周期：假设每个循环迭代需要 1 个时钟周期，则执行两个循环总共需要 11 个周期：

- 需要 1 个时钟周期进入 ADD 循环。
- 需要 4 个时钟周期执行加法循环。
- 需要 1 个时钟周期退出 ADD 并进入 SUB。
- 需要 4 个时钟周期执行 SUB 循环。
- 需要 1 个时钟周期退出 SUB 循环。
- 总共 11 个时钟周期。

在这个简单的示例中，很明显，ADD 循环中的 else 分支也可以解决这个问题，但是在更复杂的示例中，它可能不那么明显，更直观的编码方式可能具有更大的优势。

LOOP_MERGE 最优化指令用于自动合并循环。LOOP_MERGE 指令将尝试合并应用范围内的所有循环。在上面的示例中，合并循环将创建类似于上图 (B) 所示的控制结构，完成它只需要 6 个时钟周期。

合并循环允许将循环内的逻辑组合在一起进行最优化。在上面的示例中，使用双端口块 RAM 可以并行执行加减运算。

当前在 Vivado HLS 中合并循环具有以下限制：

- 如果循环边界都是变量，则它们必须具有相同的值。
- 如果循环边界为常量，那么最大常量值用作为合并循环的边界。
- 具有变量边界和常量边界的循环无法合并。
- 要合并的循环之间的代码不得产生不同结果：多次执行此代码应生成相同的结果（允许使用 $a = b$ ，不允许使用 $a = a + 1$ ）。
- 包含 FIFO 访问的循环无法合并：合并将更改 FIFO 上的读写顺序：这些循环必须始终按顺序进行。

将嵌套循环扁平化以改进时延

已展开的嵌套循环之间的移动需要额外的时钟周期，这与前述章节中所述的连续循环间移动方式相似。从外层循环移至内层循环需要一个时钟周期，从内层循环移至外层循环同样如此。

在此处所示小型示例中，这暗示执行 Outer 循环需 200 个额外时钟周期。

```
void foo_top { a, b, c, d} {  
    ...  
    Outer: while(j<100)  
    Inner: while(i<6) // 1 cycle to enter inner  
    ...  
    LOOP_BODY  
    ...  
} // 1 cycle to exit inner  
}  
...
```

Vivado HLS 提供的 `set_directive_loop_flatten` 命令允许将已标记为完美和半完美的嵌套循环扁平化，这样就无需重新编码来提升硬件性能，并且还可减少执行循环中的运算所需的周期数。

- 完美循环嵌套：仅限最内层的循环才有循环主体内容，逻辑语句之间未指定任何逻辑，所有循环边界均为常量。
- 半完美循环嵌套：仅限最内层的循环才有循环主体内容，逻辑语句之间未指定任何逻辑，所有循环边界均为常量。

对于非完美循环嵌套，即内层循环具有变量边界或者循环主体未完全包含在内层循环内，设计人员应尝试重构代码或者将循环主体中的循环展开以创建完美循环嵌套。

将该指令应用于一组嵌套循环时，应将其应用于包含循环主体的最内层循环。

```
set_directive_loop_flatten top/Inner
```

循环扁平化还可使用 GUI 中的“Vivado HLS Directive Editor”选项卡执行，可将其单独应用于各循环，或者通过在函数级别应用该指令来将其应用于函数中的所有循环。

面积最优化

数据类型和位宽

C 语言函数中的变量位宽会直接影响 RTL 实现中使用的存储元素和运算符的大小。如果变量只需 12 位但指定为整数类型（32 位），它将导致使用更大且更慢的 32 位运算符，从而减少任一时钟周期内可执行的运算数量，并可能增大启动时间间隔和时延。

- 使用适合数据类型的相应精度。
- 确认要作为 RAM 或寄存器来实现的任意数组的大小。任何过大的元素都会影响面积，从而导致硬件资源浪费。
- 请特别注意乘法、除法、取模或其它复杂算术运算。如果这些变量过大，则会对面积和性能都产生负面影响。

函数内联

函数内联会移除函数层级。函数可使用 `INLINE` 指令进行内联。内联函数可支持函数中的组件以更高效的方式与调用函数中的逻辑共享或利用该逻辑进行最优化，从而改善面积。Vivado HLS 也会自动执行此类型的函数内联。小型函数可自动内联。

内联支持对函数共享进行更有效的控制。对于要共享的函数，必须在相同层级内使用。在此代码示例中，`foo_top` 函数会对 `foo` 进行 2 次调用，并调用 `foo_sub` 函数。

```
foo_sub (p, q) {
    int q1 = q + 10;
    foo(p1,q); // foo_3
    ...
}
void foo_top { a, b, c, d} {
    ...
    foo(a,b); //foo_1
    foo(a,c); //foo_2
    foo_sub(a,d);
    ...
}
```

将函数 `foo_sub` 内联并使用 `ALLOCATION` 指令指定仅使用 `foo` 函数的 1 个实例，这样生成的设计仅含 1 个 `foo` 函数的实例：面积为以上示例的三分之一。

```
foo_sub (p, q) {
    #pragma HLS INLINE
    int q1 = q + 10;
    foo(p1,q); // foo_3
    ...
}
void foo_top { a, b, c, d} {
    #pragma HLS ALLOCATION instances=foo limit=1 function
    ...
    foo(a,b); //foo_1
    foo(a,c); //foo_2
    foo_sub(a,d);
    ...
}
```

`INLINE` 指令可选择使用 `recursive` 选项来允许位于指定函数下层的所有函数以递归方式进行内联。如果对顶层函数使用 `recursive` 选项，那么将移除设计中的所有函数层级。

可选择对函数应用 `INLINE off` 选项以阻止对这些函数进行内联。该选项可用于阻止 Vivado HLS 自动进行函数内联。

`INLINE` 指令是大幅修改代码结构而不对源代码执行任何实际修改的强大方法，并可提供强大的架构探索方法。

将大量数组映射到单一大型数组

当 C 语言代码中存在大量小型数组时，将其映射到单一大型数组通常可减少所需的块 RAM 数量。

受器件支持的前提下，每个数组都映射到 1 个块 RAM 或 UltraRAM。任一 FPGA 中提供的基本块 RAM 单元为 18K。如有大量小型数组且占用资源不足 18K，那么为了更有效地利用块 RAM 资源，可将大量小型数组映射到单一大型数组。如果块 RAM 大于 18K，则会自动将其映射到多个 18K 单元。在综合报告中，请复查“Utilization Report” → “Details” → “Memory”，以便详细了解设计中块 RAM 的使用情况。

ARRAY_MAP 指令支持通过 2 种方式将多个小型数组映射到单一大型数组：

- 水平映射：这对应于通过并置原始数组来创建新阵列。实际上，这作为含更多元素的单一数组来实现。
- 垂直映射：这对应于通过并置数组中的字词来创建新阵列。实际上，这作为含较大位宽的单一数组来实现。

水平数组映射

以下代码示例包含 2 个数组，这些数组将生成 2 个 RAM 组件。

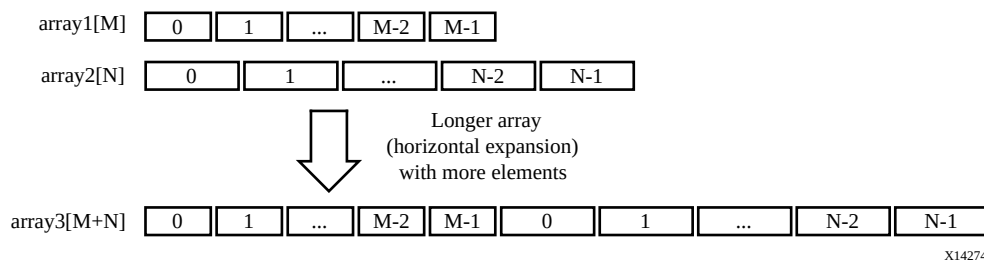
```
void foo (...) {
    int8 array1[M];
    int12 array2[N];
    ...
loop_1: for(i=0;i<M;i++) {
    array1[i] = ...;
    array2[i] = ...;
    ...
}
...
}
```

数组 array1 和 array2 可组合为单一数组，在以下示例中指定为 array3：

```
void foo (...) {
    int8 array1[M];
    int12 array2[N];
    #pragma HLS ARRAY_MAP variable=array1 instance=array3 horizontal
    #pragma HLS ARRAY_MAP variable=array2 instance=array3 horizontal
    ...
loop_1: for(i=0;i<M;i++) {
    array1[i] = ...;
    array2[i] = ...;
    ...
}
...
}
```

在此示例中，ARRAY_MAP 指令按下图所示方式对数组进行变换。

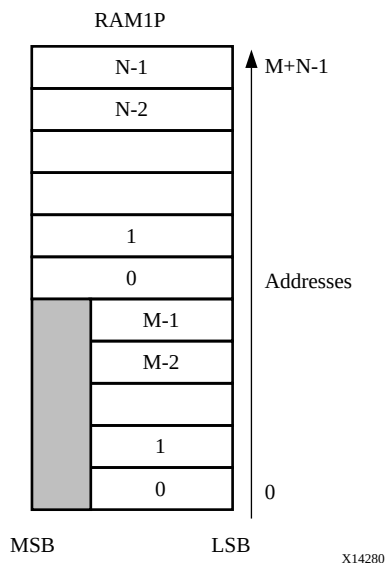
图 67：水平映射



使用水平映射时，多个较小的数组将映射到单一较大的数组。映射从较大的数组中的位置 0 开始，并遵循指定命令的顺序执行映射。在 Vivado HLS GUI 中，此映射操作基于使用菜单命令指定数组的顺序来执行。在 Tcl 环境中，此操作基于发出命令的顺序来执行。

使用下图所示的水平映射时，块 RAM 中的实现如下图所示。

图 68：水平映射的内存

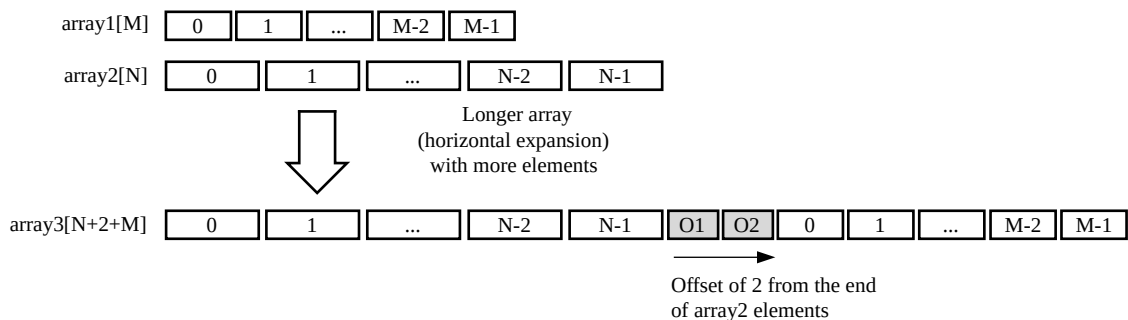


ARRAY_MAP 指令的 `offset` 选项用于指定使用 `horizontal` 选项时后续数组添加到的位置。重复先前示例，但反转命令顺序（先指定 `array2`，然后指定 `array1`）并添加 `offset`，如下所示：

```
void foo (...) {
    int8 array1[M];
    int12 array2[N];
    #pragma HLS ARRAY_MAP variable=array2 instance=array3 horizontal
    #pragma HLS ARRAY_MAP variable=array1 instance=array3 horizontal offset=2
    ...
    loop_1: for(i=0;i<M;i++) {
        array1[i] = ...;
        array2[i] = ...;
        ...
    }
    ...
}
```

这将导致如下图所示的变换。

图 69：含偏移的水平映射



X14273

映射后，新构成的数组（即以上示例中的 `array3`）可通过向映射到新实例的任意变量应用 `RESOURCE` 指令来定向到特定块 RAM 或 UltraRAM。

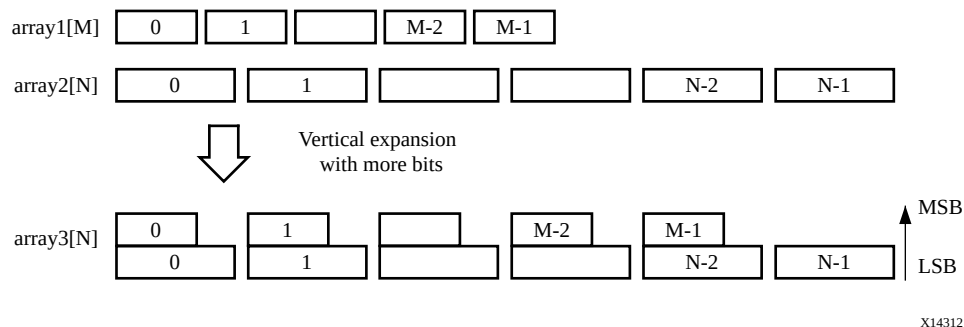
虽然水平映射可能导致使用的块 RAM 组件数量减少从而改善面积，但它确实会影响吞吐量和性能，因为当前块 RAM 端口数量有所减少。为克服此限制，Vivado HLS 还提供了垂直映射。

映射垂直数组

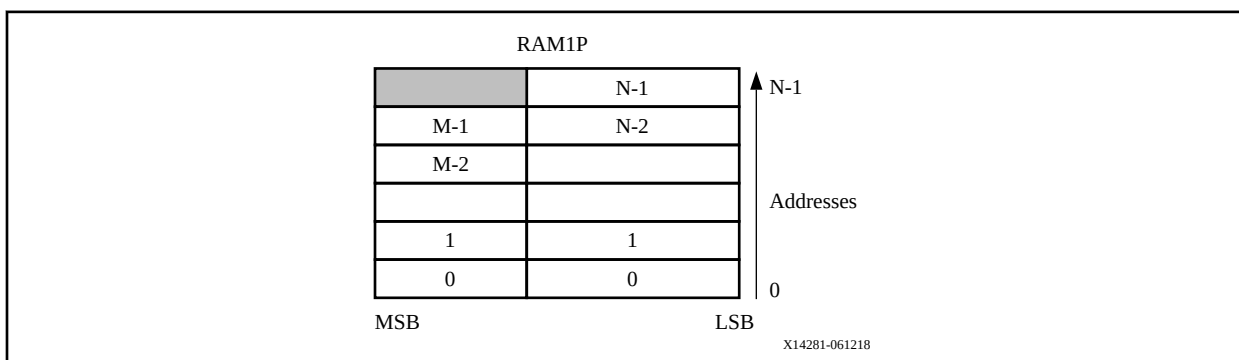
在垂直映射中，通过并置多个数组来生成位宽更高的单个数组。垂直映射通过使用 `vertical` 选项应用于 `INLINE` 指令。下图显示了应用垂直映射模式时，前述示例所发生的变化。

```
void foo (...) {
    int8  array1[M];
    int12 array2[N];
    #pragma HLS ARRAY_MAP variable=array2 instance=array3 vertical
    #pragma HLS ARRAY_MAP variable=array1 instance=array3 vertical
    ...
    loop_1: for(i=0;i<M;i++) {
        array1[i] = ...;
        array2[i] = ...;
        ...
    }
    ...
}
```

图 70：垂直映射



在垂直映射中，按命令指定顺序并置各数组，指定的数组排序顺序为从 LSB 开始到 MSB 结束。完成垂直映射后，新组成的数组将在单一块 RAM 组件内实现，如下图所示。



数组映射以及特殊注意事项



重要提示！ 数组变换的对象必须先包含在源代码中，随后才能应用任何其它指令。

要将元素从已分区的数组映射到含 `horizontal` 映射的单一数组中，必须在 `ARRAY_MAP` 指令中指定数组的每个要分区的元素。例如，以下 Tcl 命令用于对 `accum` 数组进行分区，并将生成的元素重新映射到一起。

```
#pragma HLS array_partition variable=m_accum cyclic factor=2 dim=1
#pragma HLS array_partition variable=v_accum cyclic factor=2 dim=1
#pragma HLS array_map variable=m_accum[0] instance=_accum horizontal
#pragma HLS array_map variable=v_accum[0] instance=mv_accum horizontal
#pragma HLS array_map variable=m_accum[1] instance=mv_accum_1 horizontal
#pragma HLS array_map variable=v_accum[1] instance=mv_accum_1 horizontal
```

可对全局数组进行映射。但生成的数组实例为全局实例，映射到该数组实例的任何局部数组都会变为全局数组。当不同函数的局部数组映射到同一目标数组时，目标数组实例就会变为全局实例。

仅当数组函数实参属于同一函数的实参时，才能对其进行映射。

数组重塑

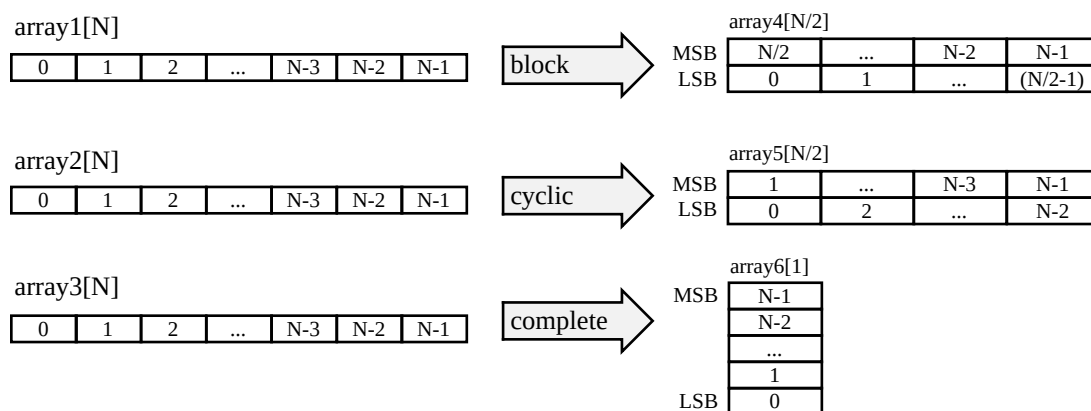
`ARRAY_RESHAPE` 指令将 `ARRAY_PARTITIONING` 与 `ARRAY_MAP` 的垂直模式相结合，用于减少块 RAM 数量，同时仍支持分区的有利特性：并行访问数据。

给定以下示例代码：

```
void foo (...) {
    int array1[N];
    int array2[N];
    int array3[N];
    #pragma HLS ARRAY_RESHAPE variable=array1 block factor=2 dim=1
    #pragma HLS ARRAY_RESHAPE variable=array2 cyclic factor=2 dim=1
    #pragma HLS ARRAY_RESHAPE variable=array3 complete dim=1
    ...
}
```

`ARRAY_RESHAPE` 指令可将数组转换为下图所示形式。

图 71：数组重塑



X14307

ARRAY_RESHAPE 指令支持在单一时钟周期内访问更多数据。只要能在单一时钟周期内访问更多数据，Vivado HLS 即可自动展开使用此数据的所有循环，前提是这有助于提升吞吐量。循环可全部或部分展开以创建足够的硬件以便在单一时钟周期内使用更多数据。此功能可使用 `config_unroll` 命令和 `tripcount_threshold` 选项来加以控制。在以下示例中，行程计数小于 16 的任何循环都将自动展开（前提是可提高吞吐量）。

```
config_unroll -tripcount_threshold 16
```

函数例化

函数例化是一种最优化技巧，不仅具有维持函数层级的面积优势，还可提供另一个强大的选项：在函数的特定实例上执行针对性局部最优化。这样可以简化围绕函数调用的控制逻辑，也可能改进时延和吞吐量。

鉴于调用函数时部分函数输入可能是常量，FUNCTION_INSTANTIATE 指令可藉此简化周围控制结构，并生成进一步优化的、更小的函数块。这可通过示例来细化解释。

给定如下代码：

```
void foo_sub(bool mode){
#pragma HLS FUNCTION_INSTANTIATE variable=mode
if (mode) {
    // code segment 1
} else {
    // code segment 2
}
}

void foo(){
#pragma HLS FUNCTION_INSTANTIATE variable=select
foo_sub(true);
foo_sub(false);
}
```

很明显，函数 `foo_sub` 已写为执行单一重复性运算（根据 `mode` 是否为 `true`）。函数 `foo_sub` 的每个实例均以相同方式实现：这非常适合函数复用和面积最优化，但也意味着函数内部的控制逻辑必须更加复杂。

FUNCTION_INSTANTIATE 最优化允许对每个实例进行独立最优化，从而减少功能和面积。完成 FUNCTION_INSTANTIATE 最优化后，以上代码可有效转换为 2 个独立函数，每个函数都针对模式的不同可能值来完成最优化，如下所示：

```
void foo_sub1() {
    // code segment 1
}

void foo_sub2() {
    // code segment 2
}

void A(){
    B1();
    B2();
}
```

如果在未经大幅内联或代码修改的情况下，在不同层级使用该函数而导致函数难以共享，那么函数例化可提供改进面积的最佳方法：大量小型局部最优化的副本比大量无法共享的大型副本更有效。

控制硬件资源

综合期间，Vivado HLS 会执行以下基本任务：

- 首先，将 C、C++ 或 SystemC 语言源代码细化为包含运算符的内部数据库。
运算符表示 C 语言代码中的运算，如加法、乘法、数组读取和写入等。
- 然后，将运算符映射到实现硬件操作的核上。
核为用于创建设计的特定硬件组件（例如，加法器、乘法器、流水线化的乘法器和块 RAM）。

您可控制其中每个步骤，从而对硬件实现进行极为详细的控制。

限制运算符数量

显式限制运算符的数量以减小某些情况下所需的面积：Vivado HLS 的默认操作是首先最大限度提升性能。限制设计中的运算符数量是一项减小面积的实用技巧：它通过强制共享运算来减小面积。

ALLOCATION 指令允许您限制设计中使用的运算符、核或函数数量。例如，名为 `foo` 的设计包含 317 次乘法，但 FPGA 仅有 256 项乘法器资源 (DSP48)。以下所示 ALLOCATION 指令可指示 Vivado HLS 创建含最多 256 个乘法 (`mul`) 运算符的设计：

```
dout_t array_arith (dio_t d[317]) {
    static int acc;
    int i;
    #pragma HLS ALLOCATION instances=mul limit=256 operation

    for (i=0;i<317;i++) {
        #pragma HLS UNROLL
        acc += acc * d[i];
    }
    rerun acc;
}
```

注释：如果指定的 ALLOCATION 限制超出所需数量，Vivado HLS 会尝试使用此项限制指定的资源数量，导致减少共享量。

您可使用 `type` 选项来指定 ALLOCATION 指令是否限制运算、核和函数数量。下表列出了可使用 ALLOCATION 指令控制的所有运算。

表 14：Vivado HLS 运算符

运算符	描述
<code>add</code>	整数加法
<code>ashr</code>	算术右移
<code>dadd</code>	双精度浮点加法
<code>dcmp</code>	双精度浮点比较
<code>ddiv</code>	双精度浮点除法
<code>dmul</code>	双精度浮点乘法
<code>drecip</code>	双精度浮点倒数
<code>drem</code>	双精度浮点余数
<code>drsqrt</code>	双精度浮点平方根倒数
<code>dsub</code>	双精度浮点减法

表 14: Vivado HLS 运算符 (续)

运算符	描述
dsqrt	双精度浮点平方根
fadd	单精度浮点加法
fcmp	单精度浮点比较
fdiv	单精度浮点除法
fmul	单精度浮点乘法
frecip	单精度浮点倒数
frem	单精度浮点余数
frsqrt	单精度浮点平方根倒数
fsub	单精度浮点减法
fsqrt	单精度浮点平方根
icmp	整数比较
lshr	逻辑右移
mul	乘法
sdiv	有符号的除法器
shl	左移
srem	有符号取余
sub	减法
udiv	无符号除
urem	无符号取余

全局最大限度减少运算符

ALLOCATION 指令与所有其它指令一样，都是在某一作用域内指定的：函数、循环或区域。`config_bind` 配置允许在整个设计中最大限度减少运算符。

在设计中要最大限度减少运算符，可执行 `config_bind` 配置中的 `min_op` 选项。前表中列出的任意运算符均可按此方式加以限制。

应用配置后，它将应用于解决方案中执行的所有综合操作：如果关闭再重新打开解决方案，那么指定的配置仍适用于所有新综合运算。

随 `config_bind` 配置一起应用的任意配置均可使用 `reset` 选项来移除，或者使用 `open_solution -reset` 以打开解决方案。

控制硬件核

执行综合时，Vivado HLS 会使用由时钟指定的时序约束、由目标器件指定的延迟以及由您指定的任意指令来判定使用哪个核来实现运算符。例如，要实现乘法运算，Vivado HLS 可使用组合乘法器核或使用流水线乘法器核。

综合期间映射到运算符的核可采用与运算符相同的方式来加以限制。您无需限制乘法运算总数，而可改为选择限制组合乘法器核的数量以强制使用流水线化乘法器来执行所有剩余乘法（或反之亦然）。这是通过将 ALLOCATION 指令 `type` 选项指定为 `core` 来实现的。

RESOURCE 指令用于显式指定要用于特定操作的核。在以下示例中指定使用 2 阶流水线化乘法器以实现变量的乘法运算。以下命令会告知 Vivado HLS 针对变量 c 使用 2 阶流水线化乘法器。由 Vivado HLS 判定用于变量 d 的核。

```
int foo (int a, int b) {
    int c, d;
    #pragma HLS RESOURCE variable=c latency=2
    c = a*b;
    d = a*c;

    return d;
}
```

在以下示例中，RESOURCE 指令指定变量 temp 的加法运算，并使用 AddSub_DSP 核来实现。这样可确保在最终设计中使用 DSP48 原语来实现此运算 - 默认情况下加法运算是使用 LUT 来实现的。

```
void apint_arith(dinA_t  inA, dinB_t  inB,
                dout1_t *out1
                ) {

    dout2_t temp;
    #pragma HLS RESOURCE variable=temp core=AddSub_DSP

    temp = inB + inA;
    *out1 = temp;
}
```

list_core 命令用于获取有关库中可用的核的详细信息。list_core 只能在 Tcl 命令界面中使用，并且必须使用 set_part 命令指定器件。如果未选中器件，此命令将无效。

list_core 命令的 -operation 选项列出了库中可通过指定运算实现的所有核。下表列出了用于实现标准 RTL 逻辑运算（例如，加法、乘法和比较）的核。

表 15：功能核

核	描述
AddSub	此核用于实现加法器和减法器。
AddSubnS	N 阶流水线化加法器或减法器。Vivado HLS 可判定所需的流水线阶数。
AddSub_DSP	此核用于确保使用 DSP48 实现加法或减法运算（使用 DSP48 内部的加法器或减法器）。
DivnS	N 阶流水线化触发器。
DSP48	乘法，所含位宽允许在单一 DSP48 宏单元内实现。这包括流水线化的乘法和通过预加法器和/或后加法器分组的乘法。该核流水线化时最大时延为 4。高于 4 的值于值为 4 时即达饱和。
Mul	组合乘法，其位宽超出标准 DSP48 宏单元大小。 可通过以单一 DSP48 宏单元实现的乘法器映射到 DSP48 核。
MulnS	N 阶流水线化乘法器，其位宽超出标准 DSP48 宏单元大小。 位宽大于 >= 10 位的乘法在 DSP48 宏单元上实现。低于此限制的乘法使用 LUT 来实现。可通过以单一 DSP48 宏单元实现的乘法器映射到 DSP48 核。

表 15：功能核 (续)

核	描述
Mul_LUT	使用 LUT 实现的乘法器。 注释： 仅适用于 C POD（纯旧数据）类型。不得与 Vivado HLS 类型（ap_int、ap_fixed 等）配合使用。

除标准核外，当运算使用浮点类型时还可使用以下浮点核。请参阅每个器件的文档以判定在器件中是否支持浮点核。

表 16：浮点核

核	描述
FAddSub_nodsp	不使用任何 DSP48 原语实现的浮点加法器或减法器。
FAddSub_fulldsp	仅使用 DSP48 原语实现的浮点加法器或减法器。
FDiv	浮点除法器。
FExp_nodsp	不使用任何 DSP48 原语实现的浮点指数运算。
FExp_meddsp	使用 DSP48 原语余数实现的浮点指数运算。
FExp_fulldsp	仅使用 DSP48 原语实现的浮点指数运算。
FLog_nodsp	不使用任何 DSP48 原语实现的浮点对数运算。
FLog_meddsp	使用 DSP48 原语余数实现的浮点对数运算。
FLog_fulldsp	仅使用 DSP48 原语实现的浮点对数运算。
FMul_nodsp	不使用任何 DSP48 原语实现的浮点乘法器。
FMul_meddsp	使用 DSP48 原语的余数实现的浮点乘法器。
FMul_fulldsp	仅使用 DSP48 原语实现的浮点乘法器。
FMul_maxdsp	使用 DSP48 原语的最大数量实现的浮点乘法器。
FRSqrt_nodsp	不使用任何 DSP48 原语实现的浮点平方根倒数。
FRSqrt_fulldsp	仅使用 DSP48 原语实现的浮点平方根倒数。
FRecip_nodsp	不使用任何 DSP48 原语实现的浮点倒数。
FRecip_fulldsp	仅使用 DSP48 原语实现的浮点倒数。
FSqrt	浮点平方根。
DAddSub_nodsp	不使用任何 DSP48 原语实现的双精度浮点加法器或减法器。
DAddSub_fulldsp	仅使用 DSP48 原语实现的双精度浮点加法器或减法器。
DDiv	双精度浮点除法器。
DExp_nodsp	不使用任何 DSP48 原语实现的双精度浮点指数运算。
DExp_meddsp	使用 DSP48 原语余数实现的双精度浮点指数运算。
DExp_fulldsp	仅使用 DSP48 原语实现的双精度浮点指数运算。
DLog_nodsp	不使用任何 DSP48 原语实现的双精度浮点对数运算。
DLog_meddsp	使用 DSP48 原语余数实现的双精度浮点对数运算。
DLog_fulldsp	仅使用 DSP48 原语实现的双精度浮点对数运算。
DMul_nodsp	不使用任何 DSP48 原语实现的双精度浮点乘法器。
DMul_meddsp	使用 DSP48 原语余数实现的双精度浮点乘法器。
DMul_fulldsp	仅使用 DSP48 原语实现的双精度浮点乘法器。

表 16：浮点核 (续)

核	描述
DMul_maxdsp	使用 DSP48 原语的最大数量实现的双精度浮点乘法器。
DRSqrt	双精度浮点平方根倒数。
DRecip	双精度浮点倒数。
DSqrt	双精度浮点平方根。
HAddSub_nodsp	不使用 DSP48 原语实现的半精度浮点加法器或减法器。
HDiv	半精度浮点除法器。
HMul_nodsp	不使用 DSP48 原语实现的半精度浮点乘法器。
HMul_fulldsp	仅使用 DSP48 原语实现的半精度浮点乘法器。
HMul_maxdsp	使用 DSP48 原语的最大数量实现的半精度浮点乘法器。
HSqrt	半精度浮点平方根。

下表列出了用于实现存储元素（例如，寄存器或内存）的核。

表 17：存储类

核	描述
FIFO	FIFO。Vivado HLS 可判定在 RLT 中使用块 RAM 来实现该核还是将其作为分布式 RAM 来实现。
FIFO_BRAM	使用 RAM 实现的 FIFO。
FIFO_LUTRAM	作为分布式 RAM 实现的 FIFO。
FIFO_SRL	作为 SRL 实现的 FIFO。
RAM_1P	单端口 RAM。Vivado HLS 可判定在 RLT 中使用块 RAM 来实现该核还是将其作为分布式 RAM 来实现。
RAM_1P_BRAM	使用块 RAM 实现的单端口 RAM。
RAM_1P_LUTRAM	作为分布式 RAM 实现的单端口 RAM。
RAM_1P_URAM	使用 Ultra RAM 实现的单端口 RAM。
RAM_2P	双端口 RAM，允许在某一端口上执行读操作并在另一个端口上执行读写操作。Vivado HLS 可判定在 RLT 中使用块 RAM 来实现该核还是将其作为分布式 RAM 来实现。
RAM_2P_BRAM	使用块 RAM 实现的双端口 RAM，允许在某一端口上执行读操作并在另一个端口上执行读写操作。
RAM_2P_LUTRAM	作为分布式 RAM 实现的双端口 RAM，允许在某一端口上执行读操作并在另一个端口上执行读写操作。
RAM_2P_URAM	作为 Ultra RAM 实现的双端口 RAM，允许在某一端口上执行读操作并在另一个端口上执行读写操作。
RAM_S2P_BRAM	使用块 RAM 实现的双端口 RAM，允许在某一端口上执行读操作并在另一个端口上执行写操作。
RAM_S2P_LUTRAM	作为分布式 RAM 实现的双端口 RAM，允许在某一端口上执行读操作并在另一个端口上执行写操作。
RAM_S2P_URAM	使用 Ultra RAM 实现的双端口 RAM，允许在某一端口上执行读操作并在另一个端口上执行写操作。
RAM_T2P_BRAM	真正双端口 RAM，支持使用块 RAM 在 2 个端口上实现读写操作。
RAM_T2P_URAM	真正双端口 RAM，支持使用 Ultra RAM 在 2 个端口上实现读写操作。
ROM_1P	单端口 ROM。Vivado HLS 可判定在 RLT 中使用块 RAM 还是使用 LUT 来实现该核。

表 17: 存储类 (续)

核	描述
ROM_1P_BRAM	使用块 RAM 实现的单端口 ROM。
ROM_nP_BRAM	使用块 RAM 实现的多端口 ROM。Vivado HLS 可自动判定端口数量。
ROM_1P_LUTRAM	使用分布式 RAM 实现的单端口 ROM。
ROM_nP_LUTRAM	使用分布式 RAM 实现的多端口 ROM。Vivado HLS 可自动判定端口数量。
ROM_2P	双端口 ROM。Vivado HLS 可判定在 RLT 中使用块 RAM 来实现该核还是将其作为分布式 ROM 来实现。
ROM_2P_BRAM	使用块 RAM 实现的双端口 ROM。
ROM_2P_LUTRAM	作为分布式 ROM 实现的双端口 ROM。

RESOURCE 指令使用已赋值的变量作为资源目标。给定代码的情况下，RESOURCE 指令可指定 `out1` 的乘法使用 3 阶流水线化乘法器来实现。

```
void foo(...) {
#pragma HLS RESOURCE variable=out1 latency=3

    // Basic arithmetic operations
    *out1 = inA * inB;
    *out2 = inB + inA;
    *out3 = inC / inA;
    *out4 = inD % inA;
}
```

如果赋值指定多个相同的运算符，则必须修改此代码以确保针对要控制的每个运算符都存在单一变量。例如，此示例中，如果仅使用流水线化乘法器来实现首个乘法 (`inA * inB`)：

```
*out1 = inA * inB * inC;
```

那么应使用 `Result_tmp` 变量上指定的指令来将代码更改为：

```
#pragma HLS RESOURCE variable=Result_tmp latency=3
Result_tmp = inA * inB;
*out1 = Result_tmp * inC;
```

全局最优化硬件核

`config_bind` 配置支持控制绑定过程。配置使您能明确将核与运算符绑定时所需的工作量。默认情况下，Vivado HLS 会选择能在时序与面积间实现最佳平衡的核。`config_bind` 能影响所使用的运算符。

```
config_bind -effort [low | medium | high] -min_op <list>
```

`config_bind` 命令只能在处于活动状态的解决方案内发出。绑定运算的默认运行策略是 `medium`。

- **low** 工作量：减少共享时间，运行时间更短，但最终 RTL 可能更大。适用案例为设计人员已知几乎不可能存在运算共享或者几乎没有符合期望的运算共享，并且不希望将 CPU 周期浪费在探寻运算共享可能性上。
- **medium** 工作量：默认设置，Vivado HLS 会尝试共享运算，但尽力在合理时间内完成运算。
- **high** 工作量：尝试最大限度共享运算，不限制运行时间。Vivado HLS 不断尝试直至完成所有可能的运算共享组合为止。

最优化逻辑

控制运算符流水线化

Vivado HLS 会自动判定用于内部运算的流水线化级别。您可将 RESOURCE 指令与 `-latency` 选项配合使用，以显式指定流水线阶段的数量，并覆盖由 Vivado HLS 判定的数量。

RTL 综合可使用多个额外流水线寄存器来帮助改善布局布线后可能导致的时序问题。在运算输出中添加寄存器通常有助于改善输出数据路径中的时序。在运算输入中添加寄存器通常有助于改善输入数据路径和来自 FSM 的控制逻辑中的时序。

添加这些额外的流水线阶段的规则是：

- 如果指定的时延比由 Vivado HLS 判定的时延多 1 个周期，Vivado HLS 会向运算输出添加新的输出寄存器。
- 如果指定的时延比由 Vivado HLS 判定的时延多 2 个周期，Vivado HLS 会向运算输出以及运算的输入侧添加寄存器。
- 如果指定的时延比由 Vivado HLS 判定的时延多 3 个或 3 个以上周期，Vivado HLS 会向运算的输出以及运算的输入侧添加寄存器。Vivado HLS 会自动判定任何附加的寄存器的位置。

您可使用 `config_core` 配置对设计中特定核的具有相同流水线深度的所有实例进行流水线化。要设置此配置，请执行以下操作：

1. 选择“Solutions” → “Solution Settings”。
2. 在“解决方案设置 (Solution Settings)”对话框中，选择“General”类别，然后单击“Add”。
3. 在“添加命令 (Add Command)”对话框中，选择 `config_core` 命令，并指定参数。

例如，以下配置指定随 DSP48 核实现的所有运算均采用流水线化，且时延设置为 3，这是该核允许的最大时延：

```
config_core DSP48 -latency 3
```

以下配置指定随 RAM_1P_BRAM 核实现的所有块 RAM 均采用流水线化，且时延设置为 3：

```
config_core RAM_1P_BRAM -latency 3
```



重要提示！ Vivado HLS 仅将此核配置应用于含显式 RESOURCE 指令的块 RAM，该指令可指定用于实现数组的核。如果使用默认核来实现数组，那么核配置不影响块 RAM。

最优化逻辑表达式

在综合期间会执行多次最优化（例如，强度折减和位宽最小化）。在自动最优化列表中也包括表达式平衡。

表达式平衡会重新排列运算符以构造平衡的树结构并降低时延。

- 对于整数运算，默认情况下表达式平衡处于开启状态，但可将其禁用。
- 对于浮点运算，默认情况下表达式平衡处于关闭状态，但可将其启用。

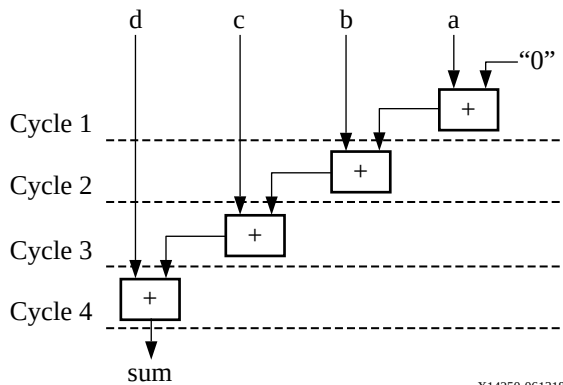
对于使用如下示例所示的 `+=` 和 `*=` 之类的赋值运算符的高度循序代码：

```
data_t foo_top (data_t a, data_t b, data_t c, data_t d)
{
    data_t sum;
    sum = 0;
```

```
sum += a;
sum += b;
sum += c;
sum += d;
return sum;
}
```

如果不使用表达式平衡并且假定每个加法都需要 1 个时钟周期，那么完整计算 `sum` 需要 4 个时钟周期，如下图所示。

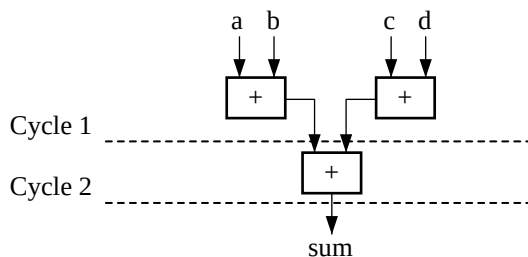
图 72：加法树



X14250-061318

但加法 `a+b` 和 `c+d` 可并行执行以缩减时延。经过平衡后，计算可在 2 个时钟周期内完成，如下图所示。表达式平衡禁止共享，导致面积增加。

图 73：平衡后的加法树



X14249

对于整数，您可使用 `EXPRESSION_BALANCE` 最优化指令配合 `off` 选项来禁用表达式平衡。默认情况下，Vivado HLS 对于 `float` 类型或 `double` 类型的运算不执行 `EXPRESSION_BALANCE` 最优化。对 `float` 和 `double` 类型进行综合时，Vivado HLS 会保留 C 语言代码中执行的运算顺序，以确保结果与 C 语言仿真相同。例如，在以下代码示例中，所有变量类型均为 `float` 或 `double`。O1 和 O2 即使看似执行相同的基本操作，其值也并不同。

```
A=B*C; A=B*F;
D=E*F; D=E*C;
O1=A*D O2=A*D;
```

此行为是 C 语言标准中执行 `float` 或 `double` 类型的运算时饱和和舍入所导致的结果。因此，存在类型为 `float` 或 `double` 的函数时，Vivado HLS 始终保留运算顺序不变，并且默认不执行表达式平衡。

您可使用 `config_compile` 配置选项对 `float` 和 `double` 类型启用表达式平衡，如下所示：

1. 选择 “Solution” → “Solution Settings”。

2. 在“解决方案设置 (Solution Settings)”对话框中，单击“General”类别，然后单击“Add”。
3. 在“添加命令 (Add Command)”对话框中，选择“config_compile”，并启用“unsafe_math_operations”。

启用此设置后，Vivado HLS 即可更改运算顺序，以生成更优化的设计。但 C/RTL 协同仿真的结果可能与 C 语言仿真不同。

unsafe_math_operations 功能还支持 no_signed_zeros 最优化。no_signed_zeros 最优化可确保以下表达式配合浮点类型和双精度类型使用时结果完全相同：

```
x - 0.0 = x;  
x + 0.0 = x;  
0.0 - x = -x;  
x - x = 0.0;  
x*0.0 = 0.0;
```

如果不使用 no_signed_zeros 最优化，由于舍入，以上表达式结果将不同。通过在 config_compile 配置中仅选中该选项，即可选择在不使用表达式平衡的情况下执行此最优化。



提示：使用 unsafe_math_operations 和 no_signed_zero 最优化时，RTL 实现结果将不同于 C 语言仿真。测试激励文件应可忽略结果中的轻微差异：检查范围，不执行精确比对。

验证 RTL

综合后验证是通过 C/RTL 协同仿真功能自动执行的，该功能复用综合前 C 语言测试激励文件来对输出 RTL 执行验证。

自动验证 RTL

C/RTL 协同仿真使用 C 语言测试激励文件来自动验证 RTL 设计。验证流程包含 3 个阶段，如下图所示。

- 执行 C 语言仿真，顶层函数输入或被测器件 (DUT) 保存为“输入矢量”。
- 在 RTL 中，通过 Vivado HLS 创建的 RTL 使用“输入矢量”。RTL 输出保存为“输出矢量”。
- 在应用于综合的函数之后，RTL 仿真的“输出矢量”将应用于 C 语言测试激励文件以验证结果是否正确。C 语言测试激励文件将执行结果验证。

Vivado HLS 会输出以下消息以显示验证进展。

C 语言仿真：

```
[SIM-14] Instrumenting C test bench (wrapc)  
[SIM-302] Generating test vectors(wrapc)
```

在此阶段中，由于已执行 C 语言仿真，C 语言测试激励文件编写的所有消息都将输出到控制台窗口或日志文件中。

RTL 仿真：

```
[SIM-333] Generating C post check test bench
[SIM-12] Generating RTL test bench
[SIM-323] Starting Verilog simulation (Issued when Verilog is the RTL
verified)
[SIM-322] Starting VHDL simulation (Issued when VHDL is the RTL verified)
```

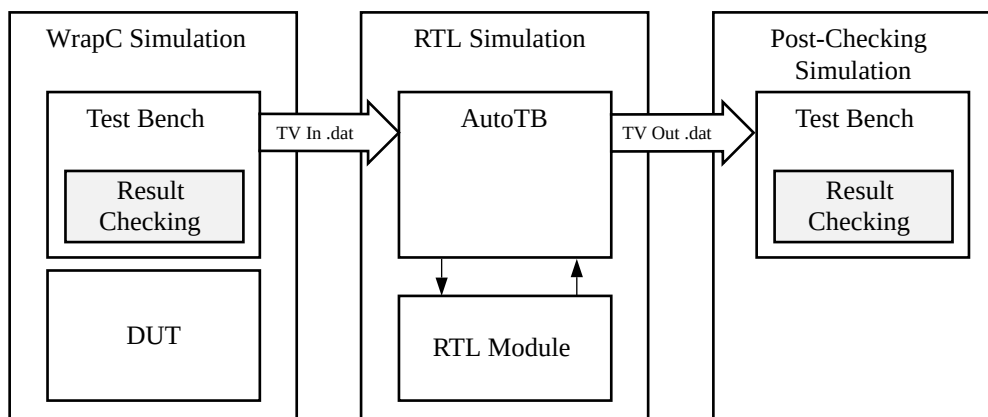
在此阶段中，RTL 仿真生成的所有消息都将输出到控制台窗口或日志文件中。

C 语言测试激励文件结果检查：

```
[SIM-316] Starting C post checking
[SIM-1000] C/RTL co-simulation finished: PASS (If test bench returns a 0)
[SIM-4] C/RTL co-simulation finished: FAIL (If the test bench returns non-
zero)
```

C/RTL 协同仿真流程中 C 语言测试激励文件的重要性如下所述。

图 74：RTL 验证流程



X14311

要成功使用 C/RTL 协同仿真功能，需满足以下条件：

- 测试激励文件必须进行自我检查，如果测试成功，则返回值 0，如果测试失败，则返回非零值。
- 必须选择正确的接口综合选项。
- 搜索路径中必须包含所有第三方仿真器。
- 设计接口上的任意数组或结构体都无法使用[不支持用于协同仿真的最优化](#)中列出的最优化指令或最优化指令组合。

测试激励文件要求

要验证 RTL 设计生成的结果是否与原始 C 语言代码相同，请使用自检测测试激励文件来执行验证。以下代码示例显示了自检测测试激励文件的重要功能：

```
int main () {
    int ret=0;

    // Execute (DUT) Function

    // Write the output results to a file

    // Check the results
    ret = system("diff --brief -w output.dat output.golden.dat");

    if (ret != 0) {
        printf("Test failed   !!!\n");
        ret=1;
    } else {
        printf("Test passed !\n");
    }

    return ret;
}
```

此自检测测试激励文件可将结果与 `output.golden.dat` 文件中已知良好的结果进行比对。

有多种方法可用于执行此检查。以下仅提供 1 个示例。

在 Vivado HLS 设计流程中，`main()` 函数的返回值可指示以下值：

- 0 值：结果正确。
- 非 0 值：结果不正确。

注释：测试激励文件可返回任意非 0 值。复杂的测试激励文件可根据差异或失败类型返回不同的值。如果测试激励文件在 C 语言仿真或 C/RTL 协同仿真后返回非 0 值，那么 Vivado HLS 会报告错误和仿真失败。



建议：由于系统环境（例如，Linux、Windows 或 Tcl）可解读 `main()` 函数的返回值，因此建议将返回值的范围约束为 8 位以保障可移植性和安全性。



注意！您负责确保测试激励文件对结果进行检查。如果测试激励文件不检查结果而返回 0，那么即使实际并未检查结果，Vivado HLS 仍会指示仿真测试成功。即使输出数据正确且有效，只要测试激励文件向 `main()` 函数返回非 0 值，Vivado HLS 仍会报告仿真失败。

接口综合要求

要使用 C/RTL 协同仿真功能来验证 RTL 设计，必须满足以下至少 1 项条件：

- 必须使用 `ap_ctrl_hs` 或 `ap_ctrl_chain` 块级接口综合顶层函数。
- 必须采用纯组合型设计。
- 顶层函数的启动时间间隔必须为 1。
- 接口包含的数组必须全部使用 `ap_hs` 或 `axis` 接口模式来进行流传输和实现。

注释：`hls::stream` 变量作为 `ap_fifo` 接口自动实现。

如果这些条件都未能满足，C/RTL 协同仿真将中止并提示下列消息：

```
@E [SIM-345] Cosim only supports the following 'ap_ctrl_none' designs: (1)
combinational designs; (2) pipelined design with task interval of 1; (3)
designs with
array streaming or hls_stream ports.
@E [SIM-4] *** C/RTL co-simulation finished: FAIL ***
```



重要提示！ 如果设计指定为使用块级 I/O 协议 `ap_ctrl_none` 且设计包含采用非阻塞行为的任意 `hls::stream` 变量，则无法保证完成 C/RTL 协同仿真。

如果任意顶层函数实参指定为 AXI4-Lite 接口，则函数返回也必须指定为 AXI4-Lite 接口。

RTL 仿真器支持

确保满足前述要求后，即可使用 C/RTL 协同仿真来使用 Verilog 或 VHDL 对 RTL 设计进行验证。默认仿真语言为 Verilog。但您也可指定 VHDL。虽然默认仿真器为 Vivado 仿真器 (XSim)，但您可使用以下任一仿真器来运行 C/RTL 协同仿真：

- Vivado 仿真器 (XSim)
- ModelSim 仿真器
- VCS 仿真器
- NC-Sim 仿真器
- Riviera 仿真器
- Xcelium



重要提示！ 要使用第三方仿真器（例如，ModelSim、VCS 或 Riviera）验证 RTL 设计，必须在系统搜索路径中包含该仿真器的可执行文件，并且相应许可证必须可用。请参阅第三方文档以了解有关这些仿真器的配置详细信息。



重要提示！ 验证 SystemC 语言设计时，必须选择 ModelSim 仿真器，确保它包含具有相应许可的 C 语言编译器功能。

不支持用于协同仿真的最优化

自动 RTL 验证不支持对接口上的数组或接口上的结构体内的数组执行多次变换的用例。

为了执行自动验证，函数接口上的数组或者函数接口上的结构体内的数组可使用以下任一最优化，但不得同时使用 2 种或 2 种以上最优化操作：

- 对相同大小的数组执行垂直映射
- 重塑
- 分区
- 结构体上的数据包

对顶层函数接口使用以下最优化时，无法通过 C/RTL 协同仿真执行验证：

- 水平映射

- 对不同大小的数组执行垂直映射
- 结构体上的数据包（包含其它结构体，如成员）
- 不支持对已启用寄存器切片 (slice) 的 AXIS 执行条件访问
- 将数组映射到流传输。

仿真 IP 核

使用浮点核来实现设计时，浮点核的位精确模型必须可供 RTL 仿真器使用。如果使用 Verilog 和 VHDL 执行 RTL 仿真，则会使用赛灵思 Vivado 仿真器自动完成此操作。

对于受支持的 HDL 第三方仿真器，赛灵思浮点库必须预编译并添加到仿真器库中。以下示例步骤演示了如何在 Verilog 中编译浮点库以配合 VCS 仿真器一起使用：

1. 打开 Vivado（而不是 Vivado HLS）并在“Tcl 控制台 (Tcl console)”窗口中发出以下命令：

```
compile_simlib -simulator vcs_mx -family all -language verilog
```

2. 此命令会在当前目录中创建浮点库。
3. 请查看 Vivado 控制台窗口，以确认目录名称，例如，./rev3_1

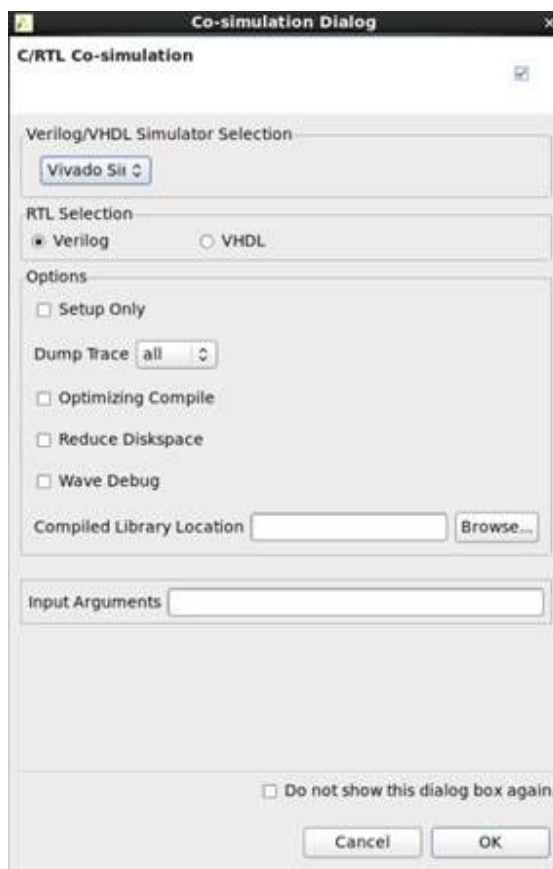
随后，即可从 Vivado HLS 内引用此库：

```
cosim_design -trace_level all -tool vcs -compiled_library_dir/  
<path_to_compile_library>/rev3_1
```

使用 C/RTL 协同仿真

要从 GUI 执行 C/RTL 协同仿真，请单击“C/RTL Cosimulation”工具栏按钮 。这样会打开仿真向导窗口，如下图所示。

图 75：C/RTL 协同仿真向导



选择要仿真的 RTL（Verilog 或 VHDL）。下拉菜单允许选择仿真器。

选项如下：

- “Setup Only”：创建运行仿真（但不执行仿真器）所需的所有文件（封装、适配器和脚本）。仿真可在相应 RTL 仿真文件夹 <solution_name>/sim/<RTL> 中的命令 shell 内运行。
- “Dump Trace”：为每个函数生成追踪文件，并保存到 <solution>/sim/<RTL> 文件夹中。下拉菜单允许您选择要保存到追踪文件的信号。您可选择追踪设计中的所有信号、仅追踪顶层端口或者不追踪任何信号。如需了解有关追踪文件的详细信息，请参阅所选 RTL 仿真器的文档。
- “Optimizing Compile”：该选项可确保使用高层次最优化来编译 C 语言测试激励文件。使用该选项会增加编译时间，但仿真执行速度更快。
- “Reduce Disk Space”：上述流程会在执行 RTL 仿真前保存所有传输事务的结果。在某些情况下，这可能导致数据文件过大。reduce_diskspace 选项可用于指定每次执行一项传输事务，从而减少文件所需的磁盘空间量。如果在 C 语言测试激励文件中该函数执行 N 次，则 reduce_diskspace 选项可确保执行 N 次独立 RTL 仿真。这可能导致仿真运行速度变慢。
- “Compiled Library Location”：该选项用于指定第三方 RTL 仿真器的编译库的位置。

如果要对第三方 RTL 仿真器进行仿真，并且设计使用 IP，那么必须使用对应此 IP 的 RTL 仿真模型才能执行 RTL 仿真。要创建或获取 RTL 仿真模型，请联系 IP 提供商。

- “Input Arguments”：该选项允许指定测试激励文件所需的任意实参。

执行 RTL 仿真

Vivado HLS 在工程子目录中执行 RTL 仿真：<SOLUTION>/sim/<RTL>

其中

- SOLUTION 是解决方案名称。
- RTL 是针对仿真所选的 RTL 类型。

协同仿真期间由 C 语言测试激励文件所写入的任何文件以及由仿真器生成的任何跟踪文件都将写入此目录。例如，如果 C 语言测试激励文件保存输出结果以供比对，请复查此目录中的输出文件，将其与期望的结果进行比对。

指令验证

C/RTL 协同仿真会自动验证 DEPENDENCE 和 DATAFLOW 指令的各方面。

如果 DATAFLOW 指令用于将任务流水线化，那么它会在任务间插入通道以改善任务间的数据流。通常，通道是以使用 STREAM 指令或 config_dataflow 命令指定的 FIFO 和 FIFO 深度来实现的。如果 FIFO 深度太小，RTL 仿真可能停滞。例如，如果指定的 FIFO 深度为 2，但在使用者任务读取任意数据值之前，生产者任务写入 3 个值，那么 FIFO 会阻止生产者操作。在某些情况下，这可能导致整个流程停滞。

C/RTL 协同仿真会发出如下消息，以表明当前 DATAFLOW 区域中的通道导致 RTL 仿真停滞。

```

////////////////////////////////////
//
// ERROR!!! DEADLOCK DETECTED at 1292000 ns! SIMULATION WILL BE STOPPED! //
////////////////////////////////////
//
////////////////////////////////////
// Dependence cycle 1:
// (1): Process: hls_fft_1kxburst.fft_rank_rad2_nr_man_9_U0
//      Channel: hls_fft_1kxburst.stage_chan_in1_0_V_s_U, FULL
//      Channel: hls_fft_1kxburst.stage_chan_in1_1_V_s_U, FULL
//      Channel: hls_fft_1kxburst.stage_chan_in1_0_V_1_U, FULL
//      Channel: hls_fft_1kxburst.stage_chan_in1_1_V_1_U, FULL
// (2): Process: hls_fft_1kxburst.fft_rank_rad2_nr_man_6_U0
//      Channel: hls_fft_1kxburst.stage_chan_in1_2_V_s_U, EMPTY
//      Channel: hls_fft_1kxburst.stage_chan_in1_2_V_1_U, EMPTY
////////////////////////////////////
// Totally 1 cycles detected!
////////////////////////////////////

```

在此情况下，请复查任务间通道的实现，确保所有 FIFO 大小足够保存所生成的数据。

RTL 测试激励文件也可采用类似方式来配置，以自动确认使用 DEPENDENCE 指令指定的假性依赖关系。这表明依赖关系并非假性依赖关系，必须将其移除以实现功能有效的设计。

分析 RTL 仿真

当 C/RTL 协同仿真完成后，会打开仿真报告并显示测量所得时延和 II。这些结果可能不同于 HLS 综合后报告的值，后者为基于穿过设计的绝对最短路径和绝对最长路径计算所得。C/RTL 协同仿真后提供的结果会显示给定仿真数据集的时延和 II 的实际值（如果改变使用的输入激励，则结果可能发生改变）。

在非流水线设计中，C/RTL 协同仿真会测量 ap_start 与 ap_done 信号之间的时延。II 比时延多 1，因为设计会在所有操作完成 1 个周期后读取新输入。设计仅在当前传输事务完成后才启动下一项传输事务。

在流水线设计中，设计可能在第一项传输事务完成之前读取新输入，并且在传输事务完成前可能存在多个 `ap_start` 和 `ap_ready` 信号。在此情况下，C/RTL 协同仿真会测量数据输入值与数据输出值之间的周期数作为时延。II 是 `ap_ready` 信号之间的周期数，设计会使用该值来请求新输入。

注释：对于流水线设计，仅当设计针对多个传输事务进行仿真时，C/RTL 协同仿真的 II 值才有效。

（可选）您可使用“Open Wave Viewer”工具栏按钮复查来自 C/RTL 协同仿真的波形。要查看 RTL 波形，您必须在执行 C/RTL 协同仿真前选择以下选项：

- “Verilog/VHDL Simulator Selection”：选择“Vivado Simulator”。对于赛灵思 7 系列和后续器件，您可选择“Auto”。
- “Dump Trace”：选择“all”或“port”。

当 C/RTL 协同仿真完成后，“Open Wave Viewer”工具栏按钮会在 Vivado IDE 中打开 RTL 波形。

注释：使用此方法打开 Vivado IDE 后，只能使用波形分析功能，例如，缩放、平移和波形基数。

波形查看器

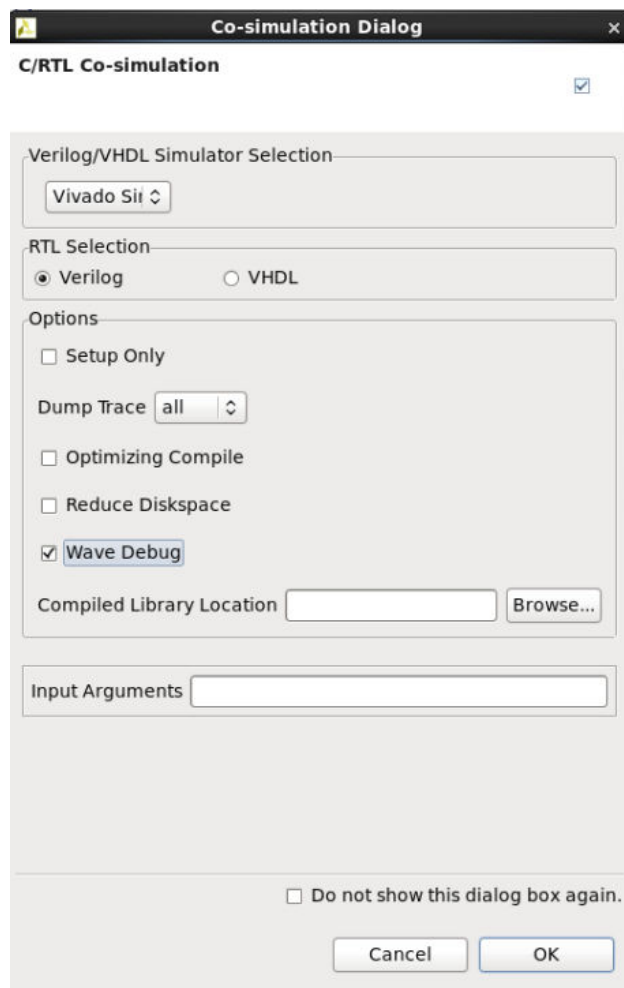
“波形查看器 (Waveform Viewer)”可将设计中的所有进程可视化。此可视化分为 2 个部分：

- HLS 进程汇总
 - 包含所有进程的活动报告的层级展示。例如，数据流和顺序进程都包含在生成的 RTL 内。
- 数据流分析
 - 提供有关数据流区域内的任务的详细活动信息。

将 HLS 设计内的活动进程可视化，以便对顶部模块每次激活期间的进程活动和长度进行详细剖析。这些可视化有助于对各进程的性能以及独立进程的总体并行执行情况进行分析。

只要能够缩短进程执行时间，主导总体执行的进程就极有可能提升性能。在 Vivado 仿真器协同仿真期间，可使用此可视化。选中“Co-simulation Dialog”窗口中的“Wave Debug”选项即可将其启用。

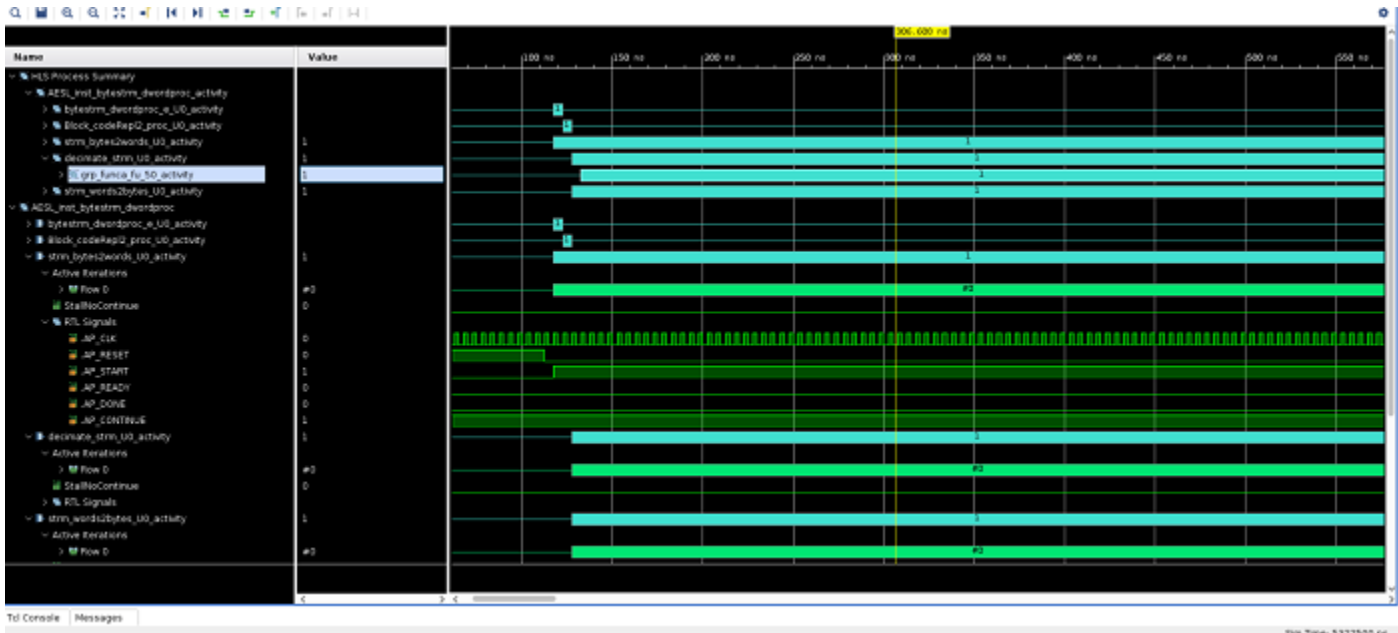
图 76：启用波形调试



查看器分为以下几个部分：

- HLS 进程汇总
 - “DUT name”：<名称>
 - “Function”：<函数名称>
- 数据流分析
 - “DUT name”：<名称>
 - “Function”：<函数名称>
 - “Dataflow/Pipeline Activity”：显示作为数据流进程实现时，函数的并行执行数。
 - “Active Iterations”：显示数据流的当前活动的迭代。行数将动态递增以适应任意并行执行的可视化。
 - “StallNoContinue”：这是 1 个停滞信号，表示数据流进程是否遇到任何输出停滞（函数已执行，但并未收到相邻数据流进程的继续执行信号）。
 - “RTL Signals”：底层 RTL 控制信号，用于解读数据流进程的传输事务视图。

图 77：波形查看器



调试 C/RTL 协同仿真

当 C/RTL 协同仿真完成时，Vivado HLS 通常会指示仿真成功且 RTL 设计的功能与初始 C 语言代码相匹配。当 C/RTL 协同仿真失败时，Vivado HLS 会发出以下消息：

```
@E [SIM-4] *** C/RTL co-simulation finished: FAIL ***
```

以下是 C/RTL 协同仿真失败的主要原因：

- 环境设置错误
- 应用的最优化指令不受支持或不正确
- C 语言测试激励文件或 C 语言源代码问题

要对 C/RTL 协同仿真失败进行调试，请运行如下章节中所述的检查。如果无法解决 C/RTL 协同仿真失败问题，请访问[赛灵思技术支持](#)，以获取支持资源，如答复记录、文档、下载和论坛。

设置环境

检查环境设置是否如下表中所示。

表 18：调试环境设置

问题	需采取的行动
当前是否使用第三方仿真器？	确认在系统搜索路径中已指定指向仿真器可执行文件的路径。 使用 Vivado 仿真器时，无需指定搜索路径。

表 18：调试环境设置 (续)

问题	需采取的行动
当前是否正在运行 Linux?	确保设置文件（例如， <code>.cshrc</code> 或 <code>.bashrc</code> ）不含更改目录命令。当 C/RTL 协同仿真开始后，会生成新的 shell 进程。如果在设置文件中存在 <code>cd</code> 命令，它会导致更改 shell 运行位置，并最终导致 C/RTL 协同仿真失败。

最优化指令

检查最优化指令是否如下图所示。

表 19：调试最优化指令

问题	需采取的行动
您当前使用的是 DEPENDENCE 指令吗?	从设计中移除 DEPENDENCE 指令，查看 C/RTL 协同仿真是否成功。如果协同仿真成功，则可能表明 DEPENDENCE 的 TRUE 或 FALSE 设置不正确。
设计在顶层接口上是否使用 <code>volatile</code> 指针?	确保在 INTERFACE 指令上指定 DEPTH 选项。在接口上使用 <code>volatile</code> 指针时，必须指定在每个传输事务中或者每次执行 C 语言函数中，在端口上执行的读写次数。
是否正在对 FIFO 使用 DATAFLOW 最优化?	检查确认按标准乒乓缓存 C/RTL 协同仿真是否成功。 检查确认不指定 FIFO 通道大小的情况下，C/RTL 协同仿真是否成功。这样可确保通道默认设置为 C 语言代码中的数组大小。 减小 FIFO 通道大小，直至 C/RTL 协同仿真停滞为止。停滞表明通道大小太小。复查设计，判定 FIFO 的最优大小。您可使用 STREAM 指令来指定各 FIFO 的大小。
您当前使用的接口是否受支持?	确保您使用受支持的接口模式。欲知详情，请参阅 接口综合要求 。
是否正在向接口上的数组应用多项最优化指令?	确保使用设计为可协同运行的最优化。欲知详情，请参阅 不支持用于协同仿真的最优化 。
当前是否正在映射到流传输的接口上使用数组?	要使用接口级流传输（DUT 的顶层函数），请使用 <code>hls::stream</code> 。

C 语言测试激励文件和 C 语言源代码

请查看下表中所示的 C 语言测试激励文件和 C 语言源代码。

表 20：对 C 语言测试激励文件和 C 语言源代码进行调试

问题	需采取的行动
C 语言测试激励文件是否检查结果并在结果正确时返回 0（零）值?	确保 C 语言测试激励文件返回 0 值以便进行 C/RTL 协同仿真。即使结果正确，如果 C 语言测试激励文件不返回 0 值，那么 C/RTL 协同仿真功能仍会报告失败。
C 语言测试激励文件当前是否基于随机数字创建输入数据?	将测试激励文件更改为使用固定种子进行任意随机数字生成。如果用于随机数字生成的种子基于变量（例如，基于时间的种子），那么每次执行测试激励文件时用于仿真的数据都将不同，故而导致结果不同。
当前是否正在多次访问的顶层接口上使用指针?	对于单一传输事务（执行 1 次 C 语言函数）内多次访问的任意指针，请使用 <code>volatile</code> 指针。如果不使用 <code>volatile</code> 指针，那么会根据 C 语言标准将除首次读取和最后一次写入外的所有一切都优化掉。

表 20：对 C 语言测试激励文件和 C 语言源代码进行调试 (续)

问题	需采取的行动
C 语言代码是否包含未定义的值或者执行出界数组访问？	确认所有数组大小都设置正确，以匹配所有访问。超出数组大小的循环边界常引发各种问题（例如，针对大小为 N-1 的数组执行 N 次访问）。 确认 C 语言仿真结果符合预期，并且未针对输出值指定随机数据值。 请考虑在 Vivado HLS 设计环境外使用业界标准 Valgrind 应用来确认 C 语言代码不含未定义或出界问题。 即使某些变量未定义或者出界，C 语言函数仍可执行并完成。在 C 语言仿真中，将为未定义的值赋予随机数字。在 RTL 仿真中，将为未定义的值赋予未知值或 X 值。
当前在设计中是否使用浮点数学运算？	请检查 C 语言测试激励文件结果是否在可接受的误差范围内，而不是执行精确比较。对于某些浮点数学运算，RTL 实现不同于 C。欲知详情，请参阅验证和数学函数。 请确保向第三方仿真器提供适用于浮点核的 RTL 仿真模型。欲知详情，请参阅仿真 IP 核。
当前是否使用赛灵思 IP 块和第三方仿真器？	确保已向第三方仿真器提供指向赛灵思 IP HDL 模型的路径。
当前设计中使用的 <code>hls::stream</code> 构造是否会更改数据速率（例如，抽取或内插）？	分析设计，并使用 STREAM 指令增大用于实现 <code>hls::stream</code> 的 FIFO 的大小。 默认情况下，<code>hls::stream</code> 作为 FIFO 来实现，深度为 2。如果设计导致数据速率增加（例如，内插操作），那么默认 FIFO 大小 2 可能太小，并导致 C/RTL 协同仿真停滞。
当前仿真中使用的数据集是否过大？	执行 C/RTL 仿真时使用 <code>reduce_diskspace</code> 选项。在此模式下，Vivado HLS 每次仅执行 1 个传输事务。仿真可能运行稍慢，但这会导致存储空间受限，从而导致系统容量问题。 C/RTL 协同仿真功能可同时验证所有传输事务。如果多次调用顶层函数（例如，用于对多个视频帧进行仿真），那么所有仿真输入和输出数据都存储在磁盘上。根据机器设置和操作系统，这可能导致性能问题或执行问题。

导出 RTL 设计

Vivado HLS 流程的最后一步是导出 RTL 设计作为 IP 块，以供赛灵思设计流程中的其它工具使用。RTL 设计可封装为以下输出格式：

- IP Catalog 格式的 IP，可搭配 Vivado Design Suite 使用
- System Generator for DSP IP，可搭配 Vivado System Generator for DSP 使用
- Synthesized Checkpoint (.dcp)

下表显示了可导出的格式以及每个格式的详细信息。

表 21：“RTL Export” 选项

格式选择	子文件夹	评述
“IP Catalog”	ip	包含 ZIP 文件，此文件可添加到 Vivado IP 目录中。ip 文件夹还包含 ZIP 文件的内容（已解压）。 该选项对于早于 7 系列或 Zynq-7000 SoC 的 FPGA 器件不可用。

表 21: “RTL Export” 选项 (续)

格式选择	子文件夹	评述
“System Generator for DSP”	<code>sysgen</code>	此输出可添加到 Vivado 版本的 System Generator for DSP 中。 该选项对于早于 7 系列或 Zynq-7000 SoC 的 FPGA 器件不可用。
“Synthesized Checkpoint (.dcp)”	<code>ip</code>	该选项可创建 Vivado 检查点文件，此文件可直接添加到 Vivado Design Suite 中的设计中。 该选项需执行 RTL 综合。选中该选项时，将自动选中含 <code>syn</code> 设置的 <code>flow</code> 选项。 输出包含 HDL 封装，可供您用于将 IP 例化到 HDL 文件中。

除封装输出格式外，RTL 文件还可作为独立文件（不属于已封装的格式的一部分）在 `verilog` 和 `vhdl` 目录中使用，这 2 个目录位于 `<project_name>/<solution_name>/impl` 实现目录内。

除了 RTL 文件，这些目录还包含 Vivado Design Suite 的工程文件。打开 `project.xpr` 文件会导致在 Vivado 工程内打开设计（Verilog 或 VHDL），并在其中对设计进行分析。如果在 Vivado HLS 工程中已执行 C/RTL 协同仿真，那么 C/RTL 协同仿真文件位于 Vivado 工程内。

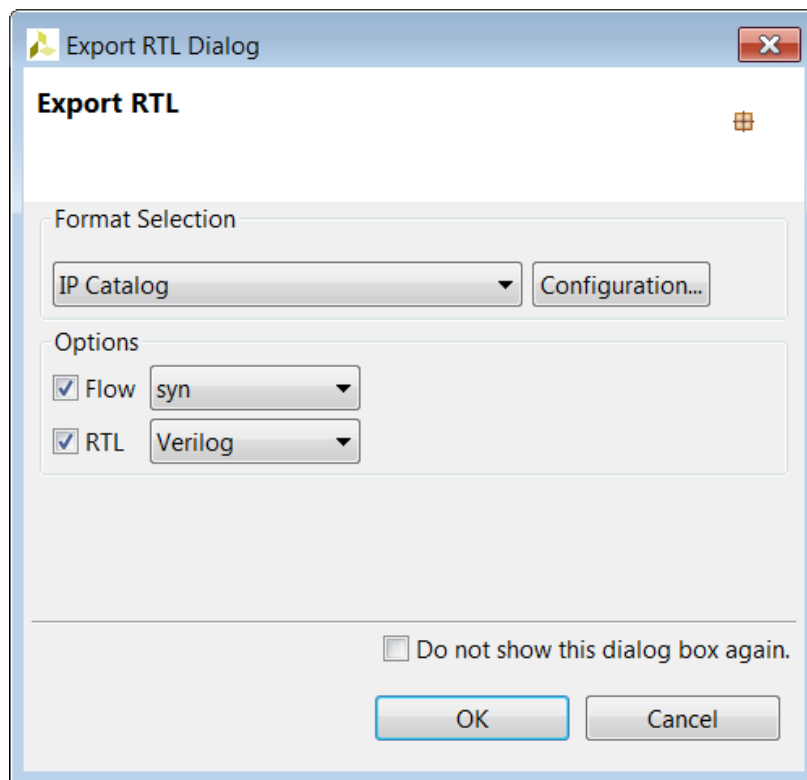
综合 RTL

当 Vivado HLS 报告综合结果时，它将提供 RTL 综合后预期的结果估算值：预期的时钟频率、预期的寄存器、LUT 和块 RAM 的数量。这些结果均为估算值，因为 Vivado HLS 无法明确 RTL 综合执行的确切最优化内容或实际的布线延迟，因此无法明确最终的面积和时序值。

在导出设计之前，您将有机会执行逻辑综合并确认估算值的准确性。下图所示的“Flow”选项在导出过程中使用 `syn` 选项调用 RTL 综合，或者使用 `impl` 选项调用 RTL 综合和实现，并将 RTL 设计综合到门电路或已布局布线的实现中。

所提供的 RTL 综合选项可用于确认报告的估算值。在大多数情况下，这些 RTL 结果不包含在封装的 IP 中。

图 78: “导出 RTL (Export RTL)” 对话框



对于大多数导出格式，根据上图中使用下拉菜单所选的用于 RTL 综合的 HDL，将在 `verilog` 或 `vhdl` 目录中执行 RTL 综合，但 RTL 综合结果不包含在封装的 IP 中。

注释：已综合的检查点 (`.dcp`) 为设计检查点，它始终作为综合 RTL 导出。“Flow” 选项可用于评估综合或实现的结果，但是导出的封装始终包含已综合的网表。

封装 IP 目录格式

完成综合和 RTL 验证后，单击“Export RTL”工具栏按钮  打开“Export RTL”对话框。

选择“Format Selection”部分中的“IP Catalog”格式。

“Configuration” 选项允许将以下标识标签嵌入到导出的封装中。这些字段可用于识别 Vivado IP Catalog 内封装的 RTL。

“Configuration” 信息用于在将设计加载到 IP Catalog 时区分相同设计的多个实例。例如，如果针对 IP Catalog 封装某个实现，然后创建新解决方案并将其封装为 IP，那么默认情况下新解决方案包含相同的名称和配置信息。如果将新解决方案也添加到 IP Catalog 中，那么 IP Catalog 会将其识别为相同 IP 的更新版本，并且将使用添加到 IP Catalog 的最新版本。

另一种方法是使用 `config_rtl` 配置中的 `prefix` 选项来以专用前缀重命名输出设计和文件。

如果“Configuration” 设置中未提供任何值，将使用以下值：

- “Vendor” : `xilinx.com`

- “Library”：hls
- “Version”：1.0
- “Description”：Vivado HLS 生成的 IP
- “Display Name”：该字段默认留空
- “Taxonomy”：该字段默认留空

完成封装流程后，<project_name>/<solution_name>/impl/ip 目录中的 .zip 文件归档可导入 Vivado IP 目录以供在任意 Vivado 设计（RTL 或 IP integrator）内使用。

软件驱动文件

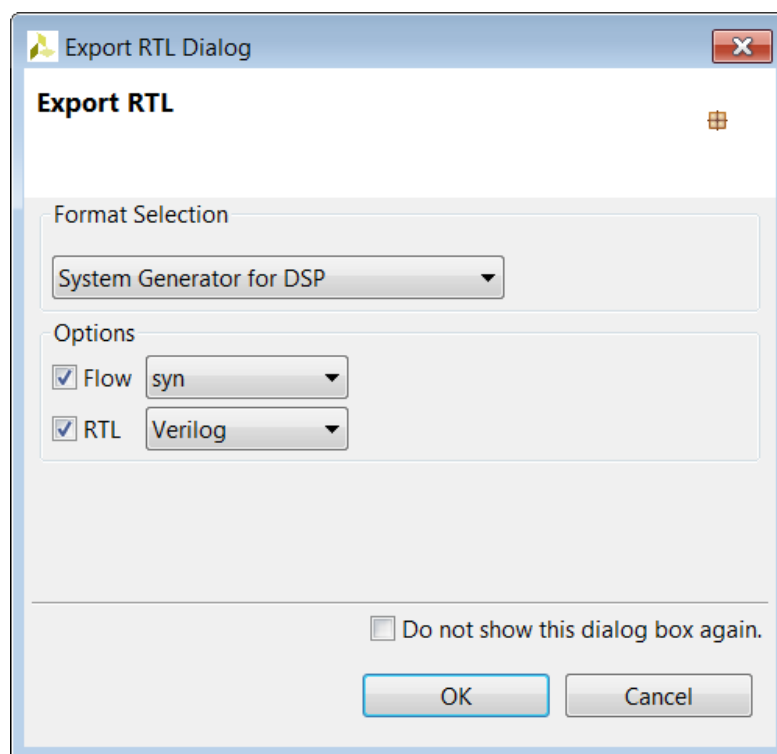
对于包含 AXI4-Lite 从接口的设计，导出流程中将创建一组软件驱动文件。这些 C 驱动文件可包含在 SDK C 工程中，并用于访问 AXI4-Lite 从端口。

软件驱动文件将写入 <project_name>/<solution_name>/impl/ip/drivers 目录，并包含在封装的 .zip 归档中。请参阅 [AXI4-Lite 接口](#)，以获取有关 C 驱动文件的详细信息。

将 IP 导出至 System Generator

完成综合与 RTL 验证后，单击“Export RTL”工具栏按钮  打开“导出 RTL (Export RTL)”对话框。

图 79：将 RTL 导出至 System Generator



如需 IP 块的布局布线后资源和时序统计数据，请选中“Flow”选项，并选择所需的 RTL 语言。

按“确定 (OK)”即可生成 IP 封装。此封装将写入 `<project_name>/<solution_name>/impl/sysgen` 目录，并且其中包含要将设计导入 System Generator 所需的一切。

如果已选中“Flow”选项，则将执行 RTL 综合，并报告最终时序和资源，但这些时序和资源不包含在 IP 封装中。请参阅上文 RTL 综合部分，以获取有关此过程的更多详细信息。

将 RTL 导入 System Generator

可使用下列步骤将 Vivado HLS 生成的 System Generator 封装导入 System Generator：

1. 在 System Generator 设计中，右键单击并使用“XilinxBlockAdd”选项来例化新的块。
2. 在此对话框中向下滚动列表，并选中“Vivado HLS”。
3. 双击新例化的 Vivado HLS 块以打开“Block Parameters”对话框。
4. 浏览 Vivado HLS 块导出到的解决方案目录。使用 `<project_name>/<solution_name>/impl/sysgen` 示例，浏览到 `<project_name>/<solution_name>` 目录并选择“应用 (Apply)”。

端口最优化

如果在综合流程期间将任意顶层函数实参变换为复合端口，则该端口的类型信息无法判定，且无法包含在 System Generator IP 块中。

此限制意味着在端口上使用重塑、映射或数据打包最优化的任意设计都必须包含端口类型信息，对于这些复合端口，请在 System Generator 中手动指定这些信息。

要在 System Generator 中手动指定类型信息，您应掌握复合端口的创建方式，然后在 System Generator 内部使用切片 (slice) 和重新解读块来将 Vivado HLS 块连接到系统中的其它块。

例如：

- 如果将 3 个 8 位输入输出 (in-out) 端口 R、G 和 B 打包到 1 个 24 位输入端口 (RGB_in) 和 1 个 24 位输出端口 (RGB_out)，

那么在 System Generator 中已包含此 IP 块后：

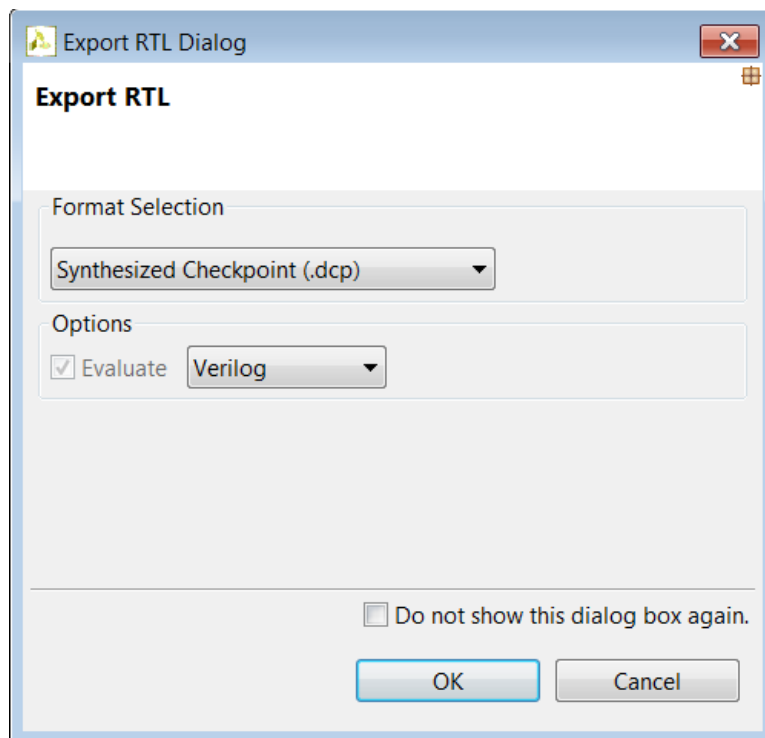
- 24 位输入端口 (RGB_in) 需由 System Generator 块驱动，后者可将 3 个 8 位输入信号 (Rin、Gin 和 Bin) 正确组合为 1 个 24 位输入总线。
- 而 24 位输出总线 (RGB_out) 则需正确拆分为 3 个 8 位信号 (Rout、Bout 和 Gout)。

请参阅 System Generator 文档以获取有关如何使用切片 (slice) 和重新解读块来连接到复合类型端口的详细信息。

导出已综合的检查点

完成综合与 RTL 验证后，单击“Export RTL”工具栏按钮  打开“导出 RTL (Export RTL)”对话框。

图 80：将 RTL 导出至已综合的检查点



当设计封装为设计检查点 IP 时，设计首先进行综合，然后再封装。

选择“OK”即可生成设计检查点封装。此封装将写入 `<project_name>/<solution_name>/impl/ip` 目录，在 Vivado Design Suite 工程中，设计检查点文件的使用方式与任何其它设计检查点相同。

高层次综合 C 语言库

Vivado® HLS C 语言库支持在 C 语言中对常用硬件设计构造和函数轻松建模并将其综合到 RTL 中。Vivado HLS 提供了以下 C 语言库：

- 任意精度数据类型库
- HLS 流传输库
- HLS 数学库
- HLS 视频库
- HLS IP 库
- HLS 线性代数库
- HLS DSP 库

您可在设计中包含 C 语言库头文件来使用每一种 C 语言库。这些头文件位于 Vivado HLS 安装区域内的 `include` 目录中。



重要提示！ 如果在 Vivado HLS 中使用设计，那么 Vivado HLS C 语言库的头文件无需包含在 `include` 路径中。指向库头文件的路径将自动添加。

任意精度数据类型库

基于 C 的原生数据类型位于 8 位边界（8 位、16 位、32 位和 64 位）上。RTL 总线（对应于硬件）支持任意长度。HLS 需要相应的机制以支持任意精度位宽规格，并且不依赖于原生 C 数据类型的人工边界：如需 17 位乘法器，则不应强制使用 32 位乘法器来实现。

Vivado® HLS 为 C 和 C++ 提供了整数和定点任意精度数据类型，并支持属于 SystemC 的任意精度数据类型。

任意精度数据类型的优势在于允许更新 C 语言代码以使用位宽较小的变量，然后允许重新执行 C 语言仿真以验证功能保持不变或可接受。

相关信息

[浮点与双精度](#)

使用任意精度数据类型

Vivado® HLS 提供了任意精度整数数据类型用于管理指定宽度边界内整数的值，如下表所示。

表 22：整数数据类型

语言	整数数据类型	所需头文件
C	[u]int<precision> (1024 位)	gcc #include "ap_cint.h"
C++	ap_[u]int<W> (1024 位)	#include "ap_int.h"
SystemC	sc_[u]int<W> (64 位) sc_[u]bigint<W> (512 位)	#include "systemc.h"

Vivado HLS 也随附定义任意精度类型的头文件作为独立封装，并且您有权在自己的源代码中使用这些任意精度类型。在 Vivado 安装区域的 `include` 目录中提供了 `xilinx_hls_lib_<release_number>.tgz` 封装。

用于 C 的任意整数精度类型

对于 C 语言，头文件 `ap_cint.h` 用于定义任意精度整数数据类型 `[u]int`。

注释： `xilinx_hls_lib_<release_number>.tgz` 封装不包含 `ap_cint.h` 中定义的 C 任意精度类型。这些类型不能配合标准 C 语言编译器一起使用，只能搭配 Vivado HLS `gcc` 编译器使用。

要在 C 语言函数中使用任意精度整数数据类型，请执行以下操作：

- 将头文件 `ap_cint.h` 添加到源代码。
- 针对有符号的类型将位类型更改为 `intN`，或者针对无符号的类型使用 `uintN`，其中 `N` 表示范围介于 1 到 1024 之间的位大小。

以下示例显示了如何添加头文件并实现 2 个变量来使用 9 位整数和 10 位无符号的整数类型：

```
#include "ap_cint.h"

void foo_top () {

    int9    var1;           // 9-bit
    uint10  var2;           // 10-bit unsigned
}
```

用于 C++ 的任意整数精度类型

头文件 `ap_int.h` 用于为 C++ `ap_[u]int` 数据类型定义任意精度整数数据类型。要在 C++ 函数中使用任意精度整数数据类型，请执行以下操作：

- 将头文件 `ap_int.h` 添加到源代码。
- 针对有符号的类型将位类型更改为 `ap_int<N>`，或者针对无符号的类型使用 `ap_uint<N>`，其中 `N` 表示范围介于 1 到 1024 之间的位大小。

以下示例显示了如何添加头文件并实现 2 个变量来使用 9 位整数和 10 位无符号的整数类型：

```
#include "ap_int.h"

void foo_top () {

    ap_int<9>    var1;           // 9-bit
    ap_uint<10>  var2;           // 10-bit unsigned
}
```

用于 SystemC 的任意精度整数类型

供 SystemC 使用的任意精度类型在 `systemc.h` 头文件中定义，所有 SystemC 语言设计中都必须包含此头文件。此头文件包含 SystemC `sc_int<>`、`sc_uint<>`、`sc_bigint<>` 和 `sc_bignint<>` 类型。

任意精度定点数据类型

在 Vivado HLS 中，使用定点数据类型至关重要，因为使用定点数据类型执行的 C++/SystemC 语言仿真的行为与综合创建的硬件的行为相匹配。这使您能够使用 C 语言层次快速仿真来分析位精度、量化和溢出的影响。

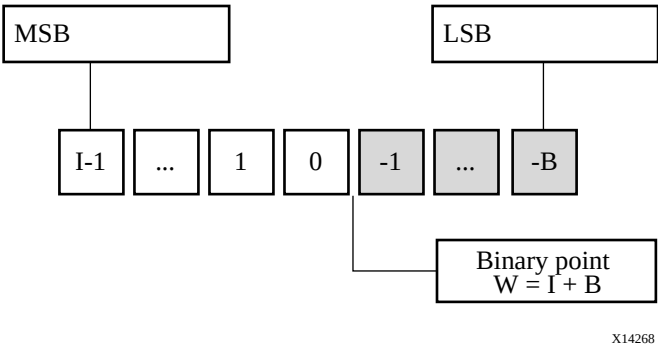
Vivado HLS 可提供任意精度的定点数据类型，以配合 C++ 和 SystemC 语言函数一起使用，如下表所示。

表 23：定点数据类型

语言	定点数据类型	所需头文件
C	-- 不适用 --	-- 不适用 --
C++	<code>ap_[u]fixed<W,I,Q,O,N></code>	<code>#include "ap_fixed.h"</code>
SystemC	<code>sc_[u]fixed<W,I,Q,O,N></code>	<code>#define SC_INCLUDE_FX</code> <code>[#define SC_FX_EXCLUDE_OTHER]</code> <code>#include "systemc.h"</code>

这些数据类型用于管理指定总宽度和整数宽度边界内的实数（非整数）的值，如下图所示。

图 81：定点数据类型



定点标识汇总

下表提供了定点类型支持的运算的简要概述。

表 24：定点标识汇总

标识	描述
W I	“Word length in bits”：用于表示整数值的位数（高于小数点的位数）

表 24：定点标识汇总 (续)

标识	描述		
Q	“Quantization mode”：用于规定当生成的精度大于用于存储结果的变量的最小小数位可定义的精度时的行为。		
	SystemC 类型	ap_fixed 类型	描述
	SC_RND	AP_RND	舍入到正无穷
	SC_RND_ZERO	AP_RND_ZERO	舍入到 0
	SC_RND_MIN_INF	AP_RND_MIN_INF	舍入到负无穷
	SC_RND_INF	AP_RND_INF	舍入到无穷
	SC_RND_CONV	AP_RND_CONV	收敛舍入
	SC_TRN	AP_TRN	截断到负无穷（默认）
	SC_TRN_ZERO	AP_TRN_ZERO	截断到 0
O	“Overflow mode”：用于规定当运算结果超出用于存储结果的变量可存储的最大值（或者对于负值为最小值）时的行为。		
	SystemC 类型	ap_fixed 类型	描述
	SC_SAT	AP_SAT	饱和
	SC_SAT_ZERO	AP_SAT_ZERO	饱和到 0
	SC_SAT_SYM	AP_SAT_SYM	对称饱和
	SC_WRAP	AP_WRAP	卷绕（默认）
	SC_WRAP_SM	AP_WRAP_SM	符号量值卷绕
N	定义溢出卷绕模式中的饱和位数。		

使用 ap_fixed 的示例

在此示例中，Vivado HLS `ap_fixed` 类型用于定义 18 位变量，其中 6 位用于表示小数点前的数值，12 位用于表示小数点后的值。此变量指定为有符号变量，量化模式设置为舍入到正无穷，针对溢出使用默认卷绕模式。

```
#include <ap_fixed.h>
...
ap_fixed<18,6,AP_RND > my_type;
...
```

使用 sc_fixed 的示例

在此 `sc_fixed` 示例中，显示 1 个 22 位变量，其中 21 位表示小数点前的数字，仅支持最低精确度 0.5。支持使用舍入到 0，因此小于 0.5 的所有结果都舍入到 0，并指定饱和度。

```
#define SC_INCLUDE_FX
#define SC_FX_EXCLUDE_OTHER
#include <systemc.h>
...
sc_fixed<22,21,SC_RND_ZERO,SC_SAT> my_type;
...
```

C 任意精度整数数据类型

C 语言中的原生数据类型位于 8 位边界（8 位、16 位、32 位和 64 位）上。RTL 信号和运算支持任意位长。Vivado HLS 为 C 提供任意精度数据类型，允许以任意位宽指定 C 语言代码中的变量和运算：例如，6 位、17 位、234 位以及（最高）1024 位。

Vivado HLS 还为 C++ 提供任意精度数据类型，并支持 SystemC 中的部分任意精度数据类型。在相应的 C++ 和 SystemC 编码中对这些类型进行了探讨。

C 任意精度数据类型的优势

任意精度数据类型的主要优势在于：

- 更高质量的硬件

例如，如需 17 位乘法器，可使用任意精度类型来要求在运算中精确使用 17 位。

如不使用任意精度数据类型，那么像 17 位这样的乘法则必须使用 32 位整数数据类型才能实现。这导致实现的乘法包含多个 DSP48 组件。

- 准确的 C 语言仿真和分析

C 语言代码中的任意精度数据类型允许使用准确位宽来执行 C 语言仿真，并且允许 C 语言仿真在进行综合前验证算法的功能（和准确性）。

对于 C 语言，头文件 `ap_cint.h` 用于定义任意精度整数数据类型 `[u]int#W`。例如：

- `int8` 表示 8 位有符号的整数数据类型。
- `uint234` 表示 234 位无符号的整数类型。

`ap_cint.h` 文件位于 `$HLS_ROOT/include` 目录中，其中 `$HLS_ROOT` 表示 Vivado HLS 安装目录。

以下示例中所示的代码与[标准类型](#)中所示“基本算术”代码示例相同。在这 2 个示例中，顶层函数中要综合的数据类型指定为 `dinA_t`、`dinB_t` 等。

```
#include "apint_arith.h"

void apint_arith(din_A inA, din_B inB, din_C inC, din_D inD,
out_1 *out1, dout_2 *out2, dout_3 *out3, dout_4 *out4
) {

    // Basic arithmetic operations
    *out1 = inA * inB;
    *out2 = inB + inA;
    *out3 = inC / inA;
    *out4 = inD % inA;

}
```

2 个示例中的真正差异在于数据类型的定义方式。要在 C 语言函数中使用任意精度整数数据类型，请执行以下操作：

- 将头文件 `ap_cint.h` 添加到源代码。
- 将原生 C 类型更改为任意精度类型：`intN` 或 `uintN`，其中，`N` 是介于 1 到 1024 之间的位大小。

数据类型在 `apint_arith.h` 头文件中定义。请参阅以下示例与[标准类型](#)中的“基本算术”示例的比较结果：

- 输入数据类型已减缩，以表示实际输入数据的最大大小。例如，8 位输入 `inA` 已减缩至 6 位输入。
- 输出类型已优化，以提升准确性。例如，`out2` (`inA` 与 `inB` 之和) 只需 13 位，而不是 32 位。

```
#include <stdio.h>
#include ap_cint.h

// Previous data types
//typedef char dinA_t;
//typedef short dinB_t;
//typedef int dinC_t;
//typedef long long dinD_t;
//typedef int dout1_t;
//typedef unsigned int dout2_t;
//typedef int32_t dout3_t;
//typedef int64_t dout4_t;

typedef int6 dinA_t;
typedef int12 dinB_t;
typedef int22 dinC_t;
typedef int33 dinD_t;

typedef int18 dout1_t;
typedef uint13 dout2_t;
typedef int22 dout3_t;
typedef int6 dout4_t;

void apint_arith(dinA_t inA,dinB_t inB,dinC_t inC,dinD_t inD,dout1_t
*out1,dout2_t *out2,dout3_t *out3,dout4_t *out4);
```

对前述示例进行综合即可生成功能与[标准类型](#)中所示“基本算术”示例相同的设计（数据范围与前述示例中指定的范围相同）。最终 RTL 设计面积更小且时钟速度更快，因为更小的位宽导致更小的逻辑。

该函数必须先编译并验证，然后才能进行综合。

在 C 中验证任意精度类型

为创建任意精度类型，在 `ap_cint.h` 文件中添加了用于定义位大小的属性。标准 C 语言编译器（如 `gcc`）可编译头文件中使用的属性，但并不了解这些属性的含义。这导致计算不反映代码的位精确行为。例如，`gcc`（或任何其它第三方 C 语言编译器）将采用二进制表示法 100 的 3 位整数值作为含十进制值 4（而不是 -4）来处理。

注释： 仅当使用 C 任意精度类型时，才会出现此问题。对于 C++ 或 SystemC 任意精度类型，不存在此类问题。

Vivado HLS 通过自动使用自带内置 C 语言编译器 `apcc` 来识别所使用的任意精度 C 类型，以解决此问题。此编译器与 `gcc` 兼容，但可正确解释任意精度类型和算法。您可在命令提示符处通过将“`gcc`”替换为“`apcc`”来调用 `apcc` 编译器。

```
$ apcc -o foo_top foo_top.c tb_foo_top.c
$ ./foo_top
```

在 C 语言中使用任意精度类型时，无法再使用 Vivado HLS C 语言调试器来分析设计。如果需要调试设计，赛灵思建议采用以下任一方法：

- 使用 `printf` 或 `fprintf` 函数输出数据值以供分析。
- 将任意精度类型替换为原生 C 类型（`int`、`char`、`short` 等）。此方法有助于对算术运算本身进行调试，但如果必须分析算法的位精确结果，则无效。

- 将 C 语言函数更改为 C++ 并使用不存在调试器限制的 C++ 任意精度类型。

整数提升

如果任意精度运算跨越原生 8 位、16 位、32 位和 64 位边界，请谨慎处理。在以下示例中，通过对 2 个 18 位值执行乘法，将结果存储到 36 位的数值中：

```
#include "ap_cint.h"

int18  a,b;
int36  tmp;

tmp = a * b;
```

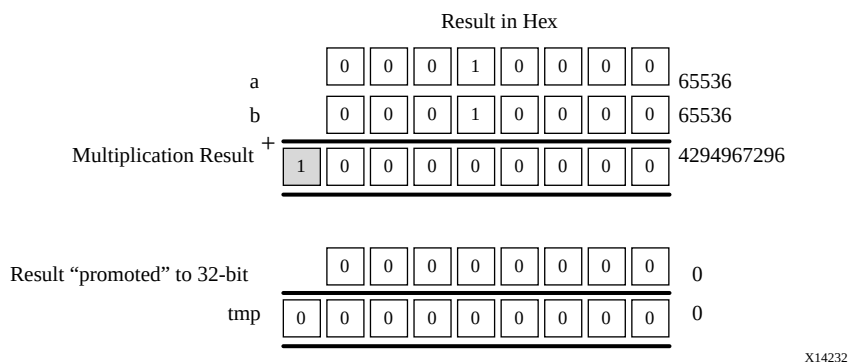
使用此方法时会发生整数提升。结果可能与期望不符。

在整数提升中，C 语言编译器会执行以下操作：

- 将乘法输入提升到原生整数大小（32 位）。
- 执行乘法，并生成 32 位结果。
- 将结果赋值给 36 位可变 tmp。

这将产生如下图所示的行为和错误结果。

图 82：整数提升



由于 Vivado HLS 生成的结果与 C 语言仿真相同，在 Vivado HLS 创建的硬件中生成的 32 位乘法结果的符号位将扩展至 36 位结果。

为解决整数提升问题，请将运算符输入强制转换为输出大小。以下示例显示的乘法器输入已在执行乘法前强制转换为 36 位值。这样在 C 语言仿真期间即可生成正确（符合期望）的结果，并在 RTL 中按期望方式执行 36 位乘法。

以下示例显示为避免整数提升所执行的强制转换。

```
#include "ap_cint.h"

typedef int18  din_t;
typedef int36  dout_t;

dout_t apint_promotion(din_t a,din_t b) {
```

```
dout_t tmp;  
  
tmp = (dout_t)a * (dout_t)b;  
return tmp;  
}
```

仅当运算结果大于下一个原生边界（8、16、32 或 64）时，才需执行强制转换来避免整数提升问题。相比于加减法运算，此行为在乘法运算中更常见。

使用 C++ 或 SystemC 任意精度类型时，不会出现整数提升。

C 任意精度整数类型：参考信息

C 语言任意精度类型提供以下相关信息：

- 用于将常量和初始化值赋值给任意精度整数（包括大于 64 位的值）的方法。
- 描述 Vivado HLS 帮助程序函数，包括打印、串联、位切片 (bit-slicing) 和范围选择函数。
- 描述运算符行为，包括移位操作（即导致反向移位的负移位值）的描述。

C++ 任意精度整数类型

C++ 中的原生数据类型位于 8 位边界（8 位、16 位、32 位和 64 位）上。RTL 信号和运算支持任意位长。

Vivado HLS 为 C++ 提供任意精度数据类型，允许以任意位宽指定 C++ 代码中的变量和运算：6 位、17 位、234 位以及（最高）1024 位。



提示：允许的默认最大宽度为 1024 位。可通过在包含 `ap_int.h` 头文件之前定义含正整数（小于或等于 32768）的 `AP_INT_MAX_W` 宏来覆盖此默认值。

C++ 支持使用 SystemC 标准中定义的任意精度类型。包含 SystemC 头文件 `systemc.h`，使用 SystemC 数据类型。

任意精度数据类型相比于原生 C++ 类型，存在 2 项主要优势：

- 硬件质量更高：例如，如需 17 位乘法器，任意精度类型可指定在计算中精确使用 17 位。
如无任意精度数据类型，则必须使用 32 位整数数据类型实现 17 位乘法，从而导致以多个 DSP48 组件实现乘法。
- 准确的 C++ 仿真/分析：C++ 代码中的任意精度数据类型允许使用精确位宽执行 C++ 仿真，并允许 C++ 仿真在执行综合前验证算法的功能（和准确性）。

C++ 中的任意精度类型不具有 C 的任何劣势：

- C++ 任意类型可使用标准 C++ 编译器来进行编译（C++ 不含等效于 `apcc` 的命令）。
- C++ 任意精度类型不存在“整数提升问题”。

用户经常将文件扩展名从 `.c` 更改为 `.cpp`，因此此类文件可作为 C++ 来编译，并且不存在上述问题。

对于 C++ 语言，头文件 `ap_int.h` 用于定义任意精度整数数据类型 `ap_(u)int<W>`。例如，`ap_int<8>` 表示 8 位有符号的整数数据类型，而 `ap_uint<234>` 表示 234 位无符号的整数类型。

`ap_int.h` 文件位于 `$HLS_ROOT/include` 目录中，其中 `$HLS_ROOT` 表示 Vivado HLS 安装目录。

以下示例中所示的代码与[标准类型](#)中的“基本算术”示例中所示的代码相同。在此示例中，要综合的顶层函数中的数据
类型指定为 `dinA_t, dinB_t ...`

```
#include "cpp_ap_int_arith.h"

void cpp_ap_int_arith(din_A inA, din_B inB, din_C inC, din_D inD,
    dout_1 *out1, dout_2 *out2, dout_3 *out3, dout_4 *out4
) {

    // Basic arithmetic operations
    *out1 = inA * inB;
    *out2 = inB + inA;
    *out3 = inC / inA;
    *out4 = inD % inA;

}
```

在此示例的最新更新中，使用的是 C++ 任意精度类型：

- 将头文件 `ap_int.h` 添加到源代码。
- 将原生 C++ 类型更改为任意精度类型 `ap_int<N>` 或 `ap_uint<N>`，其中 `N` 表示介于 1 到 1024 之间的位大小（如上所述，可根据需要扩展至 32K 位）。

数据类型在 `cpp_ap_int_arith.h` 头文件中定义。

通过与[标准类型](#)中的“基本算术”示例比较可以发现，输入数据类型已减缩，以表示实际输入数据的最大大小（例如，8 位输入 `inA` 减缩至 6 位输入）。输出类型已优化，准确性更高，例如，`out2`（`inA` 与 `inB` 之和）只需 13 位，而不是 32 位。

以下示例显示了 C++ 任意精度类型的基本算术。

```
#ifndef _CPP_AP_INT_ARITH_H_
#define _CPP_AP_INT_ARITH_H_

#include <stdio.h>
#include "ap_int.h"

#define N 9

// Old data types
//typedef char dinA_t;
//typedef short dinB_t;
//typedef int dinC_t;
//typedef long long dinD_t;
//typedef int dout1_t;
//typedef unsigned int dout2_t;
//typedef int32_t dout3_t;
//typedef int64_t dout4_t;

typedef ap_int<6> dinA_t;
typedef ap_int<12> dinB_t;
typedef ap_int<22> dinC_t;
typedef ap_int<33> dinD_t;

typedef ap_int<18> dout1_t;
typedef ap_uint<13> dout2_t;
typedef ap_int<22> dout3_t;
typedef ap_int<6> dout4_t;
```



```
void cpp_ap_int_arith(dinA_t inA,dinB_t inB,dinC_t inC,dinD_t inD,dout1_t
*out1,dout2_t *out2,dout3_t *out3,dout4_t *out4);

#endif
```

C++ 任意精度整数类型综合后可生成与**标准类型**和**C 任意精度数据类型的优势**中功能相同的设计。它会使测试激励文件与**C 任意精度数据类型的优势**中尽可能保持相似，而不是使用 C++ cout 运算符将结果输出到文件，内置 ap_int 方法 .to_int() 用于将 ap_int 结果转换为整数类型，以配合标准 fprintf 函数使用。

```
fprintf(fp, "%d*%d=%d; %d+%d=%d; %d/%d=%d; %d mod %d=%d;\n",
inA.to_int(), inB.to_int(), out1.to_int(),
inB.to_int(), inA.to_int(), out2.to_int(),
inC.to_int(), inA.to_int(), out3.to_int(),
inD.to_int(), inA.to_int(), out4.to_int());
```

C++ 任意精度整数类型：参考信息

要获取有关使用 ap_(u)int<N> 任意精度数据类型的方法、综合行为和各方面全面信息，请参阅 **C++ 任意精度类型**。本章节包含以下内容：

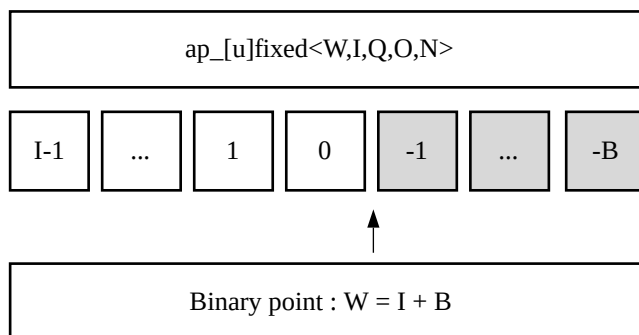
- 用于将常量和初始化值赋值给任意精度整数（包括大于 1024 位的值）的方法。
- 描述 Vivado HLS 帮助程序方法，包括打印、串联、位切片 (bit-slicing) 和范围选择函数。
- 描述运算符行为，包括移位操作（即导致反向移位的负移位值）的描述。

C++ 任意精度定点类型

C++ 函数可利用 Vivado HLS 所含的任意精度定点类型。下图汇总了这些定点类型的基本功能：

- 字型可采用有符号 (ap_fixed) 或无符号 (ap_ufixed) 类型。
- 可定义任意大小 W 的字。
- 小数点 I 前的位数还可定义字词的小数位数 W-I（在下图中以 B 来表示）。
- 可选择舍入或量化 (Q) 类型。
- 可选择溢出行为 (O 和 N)。

图 83：任意精度定点类型



X14233



提示：当代码中包含 `ap_fixed.h` 头文件时，可使用任意精度定点类型。

任意精度定点类型在 C 语言仿真期间会占用更多内存。如果使用 `ap_[u]fixed` 类型的超大数组，请参阅[数组](#)中所述的 C 语言仿真。

使用定点类型的优势包括：

- 支持轻松显示小数。
- 当变量包含的整数位数和小数位数不同时，可处理小数点的对齐。
- 当小数位数太少无法准确表示结果精度时，有多个选项可用于处理舍入方式。
- 当结果大于可显示的整数位数时，有多个选项可用于处理变量溢出方式。

请查看如下示例中的代码，其中汇总了各项属性。首先，包含 `ap_fixed.h` 头文件。随后使用 `typedef` 语句定义 `ap_fixed` 类型：

- 10 位输入：8 位整数值和 2 个小数位。
- 6 位输入：3 位整数值和 3 个小数位。
- 用于累加的 22 位变量：17 位整数值和 5 个小数位。
- 用于表示结果的 36 位变量：30 位整数值和 6 个小数位。

函数不含用于管理执行运算后的小数点对齐的代码。对齐自动执行。

以下代码样本显示的类型为 `ap_fixed`。

```
#include "ap_fixed.h"

typedef ap_ufixed<10,8, AP_RND, AP_SAT> din1_t;
typedef ap_fixed<6,3, AP_RND, AP_WRAP> din2_t;
typedef ap_fixed<22,17, AP_TRN, AP_SAT> dint_t;
typedef ap_fixed<36,30> dout_t;

dout_t cpp_ap_fixed(din1_t d_in1, din2_t d_in2) {

    static dint_t sum;
    sum += d_in1;
    return sum * d_in2;
}
```

通过使用 `ap_(u)fixed` 类型，C++ 仿真可保证位精度。快速仿真可验证算法及其准确性。综合后，RTL 会显示完全相同的位精确行为。

在此代码中可为任意精度定点类型自由赋予字面值。如以上示例中使用的测试激励文件中所示（请参阅以下示例），其中已声明 `in1` 和 `in2` 的值，并对其赋予常量值。

赋予含运算符的字面值时，字面值必须首先强制转换为 `ap_(u)fixed` 类型。否则，C 语言编译器和 Vivado HLS 会将字面值解释为整数或 `float/double` 类型，并且可能无法找到合适的运算符。如以下示例所示，在 `in1 = in1 + din1_t(0.25)` 的赋值中，字面值 0.25 强制转换为 `ap_fixed` 类型。

```
#include <cmath>
#include <fstream>
#include <iostream>
#include <iomanip>
```

```
#include <cstdlib>
using namespace std;
#include "ap_fixed.h"

typedef ap_ufixed<10,8, AP_RND, AP_SAT> din1_t;
typedef ap_fixed<6,3, AP_RND, AP_WRAP> din2_t;
typedef ap_fixed<22,17, AP_TRN, AP_SAT> dint_t;
typedef ap_fixed<36,30> dout_t;

dout_t cpp_ap_fixed(din1_t d_in1, din2_t d_in2);
int main()
{
    ofstream result;
    din1_t in1 = 0.25;
    din2_t in2 = 2.125;
    dout_t output;
    int retval=0;

    result.open(result.dat);
    // Persistent manipulators
    result << right << fixed << setbase(10) << setprecision(15);

    for (int i = 0; i <= 250; i++)
    {
        output = cpp_ap_fixed(in1,in2);

        result << setw(10) << i;
        result << setw(20) << in1;
        result << setw(20) << in2;
        result << setw(20) << output;
        result << endl;

        in1 = in1 + din1_t(0.25);
        in2 = in2 - din2_t(0.125);
    }
    result.close();

    // Compare the results file with the golden results
    retval = system(diff --brief -w result.dat result.golden.dat);
    if (retval != 0) {
        printf(Test failed !!!\n);
        retval=1;
    } else {
        printf(Test passed !\n);
    }

    // Return 0 if the test passes
    return retval;
}
```

定点标识汇总

下表显示了量化和溢出模式：



提示：不超过标准硬件算术（卷绕和截断）的默认行为的量化和溢出模式可导致运算符含更多关联硬件。要实现更高级的模式需耗用逻辑（LUT），例如，舍入到负无穷或对称饱和。

表 25：定点标识汇总

标识	描述	
W	字宽（位）	
I	用于表示整数值的位数（高于小数点的位数）	
Q	量化模式用于规定当生成的精度大于用于存储结果的变量的最小小数位可定义的精度时的行为。	
	模式	描述
	AP_RND	舍入到正无穷
	AP_RND_ZERO	舍入到 0
	AP_RND_MIN_INF	舍入到负无穷
	AP_RND_INF	舍入到无穷
	AP_RND_CONV	收敛舍入
	AP_TRN	截断到负无穷（默认）
	AP_TRN_ZERO	截断到 0
O	溢出模式用于规定生成的位数超过用于存储结果的变量所含位数时的行为。	
	模式	描述
	AP_SAT	饱和
	AP_SAT_ZERO	饱和到 0
	AP_SAT_SYM	对称饱和
	AP_WRAP	卷绕（默认）
	AP_WRAP_SM	符号量值卷绕
N	卷绕模式中的饱和位数。	

C++ 任意精度定点类型：参考信息

要获取有关使用 `ap_(u)fixed<N>` 任意精度定点数据类型的方法、综合行为和各方面全面信息，请参阅 [C++ 任意精度定点类型](#)。本章节包含以下内容：

- 用于将常量和初始化值赋值给任意精度整数（包括大于 1024 位的值）的方法。
- 溢出和饱和模式的详细描述。
- 描述 Vivado HLS 帮助程序方法，包括打印、串联、位切片 (bit-slicing) 和范围选择函数。
- 描述运算符行为，包括移位操作（即导致反向移位的负移位值）的描述。



重要提示！ 要使编译器继续处理，必须使用对应于语言的相应头文件。

HLS 流传输库

流传输数据是一种数据传输形式，其中数据样本从第一个样本开始按顺序发送。流传输不需要地址管理。

在 C 中很难对使用流传输数据的设计进行建模。使用指针执行多次读取和/或写入访问的方法可能会引发问题，因为其中隐含了类型限定符以及测试激励文件构造方式的要求。

Vivado HLS 提供了 C++ 模板类 `hls::stream<>`，用于对流传输数据结构进行建模。使用 `hls::stream<>` 类实现的流传输具有如下属性。

- 在 C 语言代码中，`hls::stream<>` 的行为与无限深度的 FIFO 相似。无需定义 `hls::stream<>` 的大小。
- 按顺序对其执行读取和写入。即，从 `hls::stream<>` 读取数据之后，无法再次对其进行读取。
- 顶层接口上的 `hls::stream<>` 默认情况下使用 `ap_fifo` 接口来实现。
- 设计内部的 `hls::stream<>` 作为 FIFO 来实现，深度为 2。最优化指令 `STREAM` 用于更改此默认大小。

本节显示了如何使用 `hls::stream<>` 类来对含流传输数据的设计进行轻松建模。本节中的主题包括：

- 流传输建模以及流传输的 RTL 实现概述。
- 全局流传输变量规则。
- 流传输使用方式。
- 阻塞读写。
- 非阻塞读写。
- 控制 FIFO 深度。

注释：`hls::stream` 类应始终在函数之间作为 C++ 参考实参来进行传递。例如，`&my_stream`。



重要提示！ `hls::stream` 类仅限在 C++ 设计中使用。不支持流传输数组。

C 语言建模与 RTL 实现

数据流在软件中（以及在测试激励文件中进行 RTL 协同仿真期间）作为无限队列来建模。在 C++ 中对数据流进行仿真无需满足任意深度。数据流可在函数内部使用，也可在函数接口上使用。内部数据流可作为函数参数来进行传递。

数据流仅限在基于 C++ 的设计中才能使用。每个 `hls::stream<>` 对象都仅限单一进程写入，且仅限单一进程读取。

如果在顶层接口上使用 `hls::stream`，默认在 RTL 中会将其作为 FIFO 接口 (`ap_fifo`) 来实现，但可选择将其作为握手接口 (`ap_hs`) 或作为 AXI-Stream 接口 (`axis`) 来实现。

如果在设计函数中使用 `hls::stream` 并将其综合到硬件中，那么会将其作为 FIFO 来实现，且默认深度为 2。在某些情况下（如使用内插时），可能必须增大 FIFO 深度以确保 FIFO 可保存硬件生成的所有元素。如果无法确保 FIFO 大小足以保存硬件生成的所有数据样本，则可能导致设计停滞（可见于 C/RTL 协同仿真和硬件实现中）。FIFO 深度可使用 `STREAM` 指令搭配 `depth` 选项来进行调整。在示例设计 `hls_stream` 中提供了相关示例。



重要提示！ 确保 `hls::stream` 变量在默认非 DATAFLOW 区域内使用时的大小设置正确。

如果使用 `hls::stream` 在不同任务（子函数或循环）间传输数据，应立即考量在不同任务间存在数据流传输的 DATAFLOW 区域内实现这些任务。默认（非 DATAFLOW）行为是完成每一项任务后再开始下一项任务，在此情况下必须确保用于实现 `hls::stream` 变量的 FIFO 大小足以保存生产者任务所生成的所有数据采样。不增大 `hls::stream` 变量的大小会导致如下错误：

```
ERROR: [XFORM 203-733] An internal stream xxxx.xxxx.V.user.V' with default
size is
used in a non-dataflow region, which may result in deadlock. Please
consider to
resize the stream using the directive 'set_directive_stream' or the 'HLS
stream'
pragma.
```

此错误表示在非 DATAFLOW 区域内（默认 FIFO 深度为 2），其大小可能不足以保存生产者任务写入 FIFO 的所有数据采样。

全局和局部流传输

流传输可定义为局部或全局。局部流传输始终作为内部 FIFO 来实现。全局流传输可作为内部 FIFO 或端口来实现：

- 仅读取或者仅写入的全局定义的流传输推断为顶层 RTL 块的外部端口。
- 支持读取和写入的全局定义的流传输（位于顶层函数下的层级内）作为内部 FIFO 来实现。

全局作用域内定义的流传输遵循的规则与任何其它全局变量相同。

使用 HLS 流传输

要使用 `hls::stream<>` 对象，请包含 `hls_stream.h` 头文件。数据对象流传输是通过指定类型和变量名称来定义的。在此示例中，已定义 1 个 128 位无符号整数类型，用于创建名为 `my_wide_stream` 的流传输变量。

```
#include "ap_int.h"
#include "hls_stream.h"

typedef ap_uint<128> uint128_t; // 128-bit user defined type
hls::stream<uint128_t> my_wide_stream; // A stream declaration
```

流传输必须使用限定作用域的命名。赛灵思建议使用限定作用域的 `hls::` 命名，如上示例所示。但如要使用 `hls` 名称空间，可将前述示例重写为：

```
#include <ap_int.h>
#include <hls_stream.h>
using namespace hls;

typedef ap_uint<128> uint128_t; // 128-bit user defined type
stream<uint128_t> my_wide_stream; // hls:: no longer required
```

给定指定为 `hls::stream<T>` 的流传输的情况下，类型 `T` 可以是：

- 任何 C++ 原生数据类型
- Vivado HLS 任意精度类型（例如，`ap_int<>` 或 `ap_ufixed<>`）
- 用户定义的结构体（包含以上任意类型）

注释：用户定义的通用类（或结构），所含方法（成员函数）不应用作为流传输变量的类型 (T)。

流传输命名为可选操作。为流传输提供名称即可在报告中使用此名称。例如，Vivado HLS 会自动检查以确保仿真期间读取来自输入流传输的所有元素。给定以下 2 个流传输的情况下：

```
stream<uint8_t> bytestr_in1;
stream<uint8_t> bytestr_in2("input_stream2");
```

有关流传输中剩余元素的任何警告均按如下方式报告，其中明确标明与 bytestr_in2 的消息：

```
WARNING: Hls::stream 'hls::stream<unsigned char>.1' contains leftover data,
which
may result in RTL simulation hanging.
WARNING: Hls::stream 'input_stream2' contains leftover data, which may
result in RTL
simulation hanging.
```

传入或传出函数的流传输必须按引用传递，如以下示例所示：

```
void stream_function (
    hls::stream<uint8_t> &strm_out,
    hls::stream<uint8_t> &strm_in,
    uint16_t strm_len
)
```

Vivado HLS 支持阻塞访问方式和非阻塞访问方式。

- 非阻塞访问只能作为 FIFO 接口来实现。
- 作为 ap_fifo 端口实现并以 AXI4-Stream 资源定义的流传输端口不得使用非阻塞访问。

在 Vivado HLS 示例中提供了使用流传输的完整设计示例。请参阅 GUI Welcome 屏幕中提供的设计示例中的 hls_stream 示例。

阻塞读写

对应 hls::stream<> 对象的基本访问为阻塞读取和阻塞写入。这些访问是使用类方法完成的。这些方法会停止（阻止）对空的流传输 FIFO 尝试执行的读操作以及对满的流传输 FIFO 尝试执行的写操作，直至针对映射到 ap_hs 接口协议的流传输完成完整握手为止。

在 C/RTL 协同仿真中可观测到如下停滞现象：持续执行仿真器，但传输事务毫无进展。以下显示了停滞状况的经典示例，其中 RTL 仿真时间不断增加，但传输事务间或事务内部毫无进展。

```
// RTL Simulation : "Inter-Transaction Progress" ["Intra-Transaction
Progress"] @
"Simulation Time"
////////////////////////////////////
// RTL Simulation : 0 / 1 [0.00%] @ "110000"
// RTL Simulation : 0 / 1 [0.00%] @ "202000"
// RTL Simulation : 0 / 1 [0.00%] @ "404000"
```

阻塞写入方法

在此示例中，变量 `src_var` 的值推送到数据流中。

```
// Usage of void write(const T & wdata)

hls::stream<int> my_stream;
int src_var = 42;

my_stream.write(src_var);
```

“<<”运算符已重载，可按相似方式应用于 C++ 流传输（例如，`iostreams` 和 `filestreams`）的流插入运算符。要写入的 `hls::stream<>` 对象将作为左侧实参提供，而变量则作为右侧实参写入。

```
// Usage of void operator << (T & wdata)

hls::stream<int> my_stream;
int src_var = 42;

my_stream << src_var;
```

阻塞读取方法

此方法从流传输头部读取并向 `dst_var` 变量赋值。

```
// Usage of void read(T &rdata)

hls::stream<int> my_stream;
int dst_var;

my_stream.read(dst_var);
```

或者，通过将流传输赋值（例如，使用 `=` 或 `+=`）给左侧对象，可读取流传输中的下一个对象。

```
// Usage of T read(void)

hls::stream<int> my_stream;

int dst_var = my_stream.read();
```

“>>”运算符已重载，支持以相似方式用于 C++ 流传输（例如，`iostreams` 和 `filestreams`）的流提取运算符。`hls::stream` 作为 LHS 实参和目标变量 RHS 提供。

```
// Usage of void operator >> (T & rdata)

hls::stream<int> my_stream;
int dst_var;

my_stream >> dst_var;
```


非阻塞读写

非阻塞读写方法同样可用。这些方法允许在空的流传输上尝试读取或者尝试写入已满的流传输时仍可继续执行。

这些方法会返回布尔值以指示访问状态（`true` 表示成功，`false` 表示失败）。此外还包含其它方法用于测试 `hls::stream<>` 流传输的状态。



重要提示！ 仅在使用 `ap_fifo` 协议的接口上支持非阻塞方法。更确切地说，AXI-Stream 标准和赛灵思 `ap_hs` I/O 协议不支持非阻塞访问。

在 C 语言仿真期间，流传输大小无限。因此，无法对 C 语言仿真验证流传输是否已满。只能在 RTL 仿真期间已定义 FIFO 大小（默认大小 1。或者通过 `STREAM` 指令定义的任意大小）的情况下验证这些方法。



重要提示！ 如果设计指定为使用块级 I/O 协议 `ap_ctrl_none` 且设计包含采用非阻塞行为的任意 `hls::stream` 变量，则无法保证完成 C/RTL 协同仿真。

非阻塞写操作

此方法会尝试将 `src_var` 变量推送到 `my_stream` 流传输，如果成功，则返回布尔值 `true`。否则返回 `false`，队列不受影响。

```
// Usage of void write_nb(const T & wdata)

hls::stream<int> my_stream;
int src_var = 42;

if (my_stream.write_nb(src_var)) {
    // Perform standard operations
    ...
} else {
    // Write did not occur
    return;
}
```

满盈度测试

```
bool full(void)
```

仅当 `hls::stream<>` 对象已满时返回 `true`。

```
// Usage of bool full(void)

hls::stream<int> my_stream;
int src_var = 42;
bool stream_full;

stream_full = my_stream.full();
```

非阻塞读取

```
bool read_nb(T & rdata)
```

此方法会尝试从流传输中读取值，成功则返回 `true`。否则返回 `false`，队列不受影响。

```
// Usage of void read_nb(const T & wdata)

hls::stream<int> my_stream;
int dst_var;

if (my_stream.read_nb(dst_var)) {
    // Perform standard operations
    ...
} else {
    // Read did not occur
    return;
}
```

空状态测试

```
bool empty(void)
```

如果 `hls::stream<>` 为空，则返回 `true`。

```
// Usage of bool empty(void)

hls::stream<int> my_stream;
int dst_var;
bool stream_empty;

stream_empty = my_stream.empty();
```

以下示例显示了 RTL FIFO 已满或已空时，如何通过非阻塞访问与满/空测试的组合来提供错误处理功能。

```
#include "hls_stream.h"
using namespace hls;

typedef struct {
    short    data;
    bool     valid;
    bool     invert;
} input_interface;

bool invert(stream<input_interface>& in_data_1,
            stream<input_interface>& in_data_2,
            stream<short>& output
) {
    input_interface in;
    bool full_n;

    // Read an input value or return
    if (!in_data_1.read_nb(in))
        if (!in_data_2.read_nb(in))
            return false;

    // If the valid data is written, return not-full (full_n) as true
    if (in.valid) {
        if (in.invert)
            full_n = output.write_nb(~in.data);
```

```
    else
        full_n = output.write_nb(in.data);
    }
    return full_n;
}
```

控制 RTL FIFO 深度

对于使用流传输数据的大部分设计，默认 RTL FIFO 深度 2 足矣。每次仅处理一个采样的流传输数据。

对于执行实现所需的 FIFO 深度大于 2 的多速率设计，必须判定（并使用 STREAM 指令设置）完成 RTL 仿真所需的深度。如果 FIFO 深度不足，RTL 协同仿真将停滞。

由于在 GUI 指令窗格内无法查看流传输对象，因此在该窗格内无法直接应用 STREAM 指令。

对于包含已声明的 `hls::stream<>` 对象（或者实参列表中已使用或包含该对象）的函数，右键单击并执行以下操作：

- 选中 STREAM 指令。
- 使用流传输变量的名称手动填充 `variable` 字段。

或者可以：

- 在 `directives.tcl` 文件中手动指定 STREAM 指令，或
- 在 `source` 中添加该指令作为编译指示。

C/RTL 协同仿真支持

Vivado HLS C/RTL 协同仿真功能不支持在顶层接口中包含 `hls::stream<>` 成员的结构或类。Vivado HLS 支持对这些结构或类进行综合。

```
typedef struct {
    hls::stream<uint8_t> a;
    hls::stream<uint16_t> b;
} strm_struct_t;

void dut_top(strm_struct_t indata, strm_struct_t outdata) { }
```

上述限制适用于顶层函数实参和全局声明的对象。如果使用流传输结构体进行综合，则必须使用外部 RTL 仿真器和用户创建的 HDL 测试激励文件来验证设计。对于仅含内部链接的 `hls::stream<>` 对象并无此类限制。

HLS 数学库

Vivado HLS 数学库 (`hls_math.h`) 可为标准 C 语言库 (`math.h`) 和 C++ 库 (`cmath.h`) 提供支持，并自动用于指定综合期间的数学运算。此支持包含对应所有函数的浮点支持（单精度、双精度和半精度）以及对应部分函数的定点支持。

（可选）`hls_math.h` 库可在 C++ 源代码中用于代替标准 C++ 数学库 (`cmath.h`)，但它不能在 C 语言源代码中使用。Vivado HLS 将使用相应的仿真实现来避免 C 语言仿真与 C/RTL 协同仿真之间出现准确性差异。

HLS 数学库精度

HLS 数学函数由 `hls_math.h` 库实现为可综合的位近似函数。位近似 HLS 数学库函数无法提供与标准 C 语言函数相同的精度。为了获得期望的结果，位近似的实现可能使用与标准 C 语言数学库版本不同的底层算法。该函数的精度以 ULP（最小精度单位）的形式指定。精度上的差异对 C 语言仿真和 C/RTL 协同仿真都有影响。

ULP 差异通常在 1-4 ULP 范围内。

- 如果在 C 语言源代码中使用标准 C 语言数学库，则由于某些函数与标准 C 语言数学库表现出 ULP 差异，因此 C 语言仿真与 C/RTL 协同仿真之间可能会有差异。
- 如果在 C 语言源代码中使用 HLS 数学库，则 C 语言仿真与 C/RTL 协同仿真之间不会有任何差异。但是，使用 HLS 数学库的 C 语言仿真可能与使用标准 C 语言数学库的 C 语言仿真之间存在差异。

此外，以下 7 个函数可能会显示出一些差异，具体取决于用于编译和运行 C 语言仿真的 C 语言标准：

- `copysign`
- `fpclassify`
- `isinf`
- `isfinite`
- `isnan`
- `isnormal`
- `signbit`

C90 模式

系统通常仅提供 `isinf`、`isnan` 和 `copysign`，并且这些函数仅适用双精度类型。特别是，`copysign` 始终返回双精度结果。如果必须返回浮点数，则在综合后可能会导致意外结果，这是因为在硬件中引入了双精度至浮点数转换块。

C99 模式 (-std=c99)

在此模式下通常系统可提供全部 7 个函数，并且期望系统头文件会将其重定向到 `__isnan(double)` 和 `__isnan(float)`。常用的 GCC 头文件不会重定向 `isnormal`，而是根据 `fpclassify` 来将其实现。

使用 math.h 的 C++

系统可提供全部 7 个函数，并且这些函数都适用于双精度类型。

`copysign` 始终返回双精度结果。如果必须返回浮点数，则在综合后可能会导致意外结果，这是因为在硬件中引入了双精度至浮点数转换块。

使用 cmath 的 C++

与 C99 mode (-std=c99) 类似，以下情况除外：

- 系统头文件通常不同。
- 这些函数可正确重载，以用于：
 - `float().snan(double)`
 - `isinf(double)`

`copysign` 和 `copysignf` 作为内置函数来处理，即使使用 `namespace std;` 时也是如此。

使用 `cmath` 和名称空间 `std` 的 C++

没有任何问题。赛灵思建议使用以下方法以获得最佳效果：

- 对于 C，使用 `-std=c99`
- 对于 C 和 C++，使用 `-fno-builtin`

注释：要指定 `-std=c99` 等 C 语言编译选项，请使用 Tcl 命令 `add_files` 搭配 `-cflags` 选项。还可以在“工程设置 (Project Settings)”对话框中使用“Edit CFLAGS”按钮。

HLS 数学库

在 HLS 数学库中提供了以下函数。其中每个函数都支持半精度 (`half` 类型)、单精度 (`float` 类型) 和双精度 (`double` 类型)。



重要提示！ 对于下列每个 `func` 函数，库中还提供了 1 个关联的仅限半精度函数（名为 `half_func`）和 1 个仅限单精度函数（名为 `funcf`）。

混用半精度、单精度和双精度数据类型时，请检查常见综合错误，防止在最终 FPGA 实现中引入类型转换硬件。

三角函数

<code>acos</code>	<code>acospi</code>	<code>asin</code>	<code>asinpi</code>
<code>atan</code>	<code>atan2</code>	<code>atan2pi</code>	<code>cos</code>
<code>cospi</code>	<code>sin</code>	<code>sincos</code>	<code>sinpi</code>
<code>tan</code>	<code>tanpi</code>		

双曲函数

<code>acosh</code>	<code>asinh</code>	<code>atanh</code>	<code>cosh</code>
<code>sinh</code>	<code>tanh</code>		

指数函数

<code>exp</code>	<code>exp10</code>	<code>exp2</code>	<code>expm1</code>
<code>frexp</code>	<code>ldexp</code>	<code>modf</code>	

对数函数

<code>ilogb</code>	<code>log</code>	<code>log10</code>	<code>log1p</code>
--------------------	------------------	--------------------	--------------------

幂函数

<code>cbrt</code>	<code>hypot</code>	<code>pow</code>	<code>rsqrt</code>
<code>sqrt</code>			

误差函数

erf erfc

舍入函数

ceil	floor	llrint	llround
lrint	lround	nearbyint	rint
round	trunc		

余数函数

fmod remainder remquo

浮点

copysign nan nextafter nexttoward

差分函数

fdim	fmax	fmin	maxmag
minmag			

其它函数

abs	divide	fabs	fma
fract	mad	recip	

分类函数

fpclassify	isfinite	isinf	isnan
isnormal	signbit		

比较函数

isgreater	isgreaterequal	isless	islessequal
islessgreater	isunordered		

关系函数

all	any	bitselect	isequal
isnotequal	isordered	select	

定点数学函数

针对以下数学函数还提供了定点实现。

所有定点匹配函数都支持 ap__[u]fixed 类型和 ap__[u]int 数据类型，并采用以下位宽规格。

1. `ap_fixed<W, I>` (当 $I \leq 33$ 且 $W-I \leq 32$ 时)
2. `ap_ufixed<W, I>` (当 $I \leq 32$ 且 $W-I \leq 32$ 时)
3. `ap_int<I>` (当 $I \leq 33$ 时)
4. `ap_uint<I>` (当 $I \leq 32$ 时)

三角函数

<code>cos</code>	<code>sin</code>	<code>tan</code>	<code>acos</code>	<code>asin</code>	<code>atan</code>	<code>atan2</code>	<code>sincos</code>
<code>cospi</code>	<code>sinpi</code>						

双曲函数

<code>cosh</code>	<code>sinh</code>	<code>tanh</code>	<code>acosh</code>	<code>asinh</code>	<code>atanh</code>
-------------------	-------------------	-------------------	--------------------	--------------------	--------------------

指数函数

<code>exp</code>	<code>frexp</code>	<code>modf</code>	<code>exp2</code>	<code>expm1</code>
------------------	--------------------	-------------------	-------------------	--------------------

对数函数

<code>log</code>	<code>log10</code>	<code>ilogb</code>	<code>log1p</code>
------------------	--------------------	--------------------	--------------------

幂函数

<code>pow</code>	<code>sqrt</code>	<code>rsqrt</code>	<code>cbrt</code>	<code>hypot</code>
------------------	-------------------	--------------------	-------------------	--------------------

误差函数

<code>erf</code>	<code>erfc</code>
------------------	-------------------

舍入函数

<code>ceil</code>	<code>floor</code>	<code>trunc</code>	<code>round</code>	<code>rint</code>	<code>nearbyint</code>
-------------------	--------------------	--------------------	--------------------	-------------------	------------------------

浮点

<code>nextafter</code>	<code>nexttoward</code>
------------------------	-------------------------

差分函数

<code>erf</code>	<code>erfc</code>	<code>fdim</code>	<code>fmax</code>	<code>fmin</code>	<code>maxmag</code>	<code>minmag</code>
------------------	-------------------	-------------------	-------------------	-------------------	---------------------	---------------------

其它函数

<code>fabs</code>	<code>recip</code>	<code>abs</code>	<code>fract</code>	<code>divide</code>
-------------------	--------------------	------------------	--------------------	---------------------

分类函数

signbit

比较函数

isgreater

isgreaterequal

isless

islessequal

islessgreater

关系函数

isequal

isnotequal

any

all

bitselect

定点类型提供的函数值版本准确性略低，但 RTL 实现更小且更快。

使用定点数据类型实现数学函数的方法是：

1. 判定是否支持定点实现。
2. 将数学函数更新为使用 `ap_fixed` 类型。
3. 执行 C 语言仿真来验证设计按所需精度是否仍可正常操作。执行 C 语言仿真时所使用的位精度类型与 RTL 实现相同。
4. 对设计执行综合。

例如，使用含如下数学函数的定点类型来指定函数 `sin` 的定点实现：

```
#include "hls_math.h"
#include "ap_fixed.h"

ap_fixed<32,2> my_input, my_output;

my_input = 24.675;
my_output = sin(my_input);
```

使用定点数学函数时，结果类型的宽度和整数位数必须与输入相同。

验证和数学函数

如果在 C 语言源代码中使用标准 C 语言数学库，C 语言仿真结果与 C/RTL 协同仿真结果可能产生差异：如果源代码中的任一数学函数与标准 C 语言数学库存在 ULP 差异，则可能导致 RTL 仿真结果不同。

如果在 C 语言源代码中使用 `hls_math.h` 库，那么 C 语言仿真与 C/RTL 协同仿真结果可能不同。但使用 `hls_math.h` 时，C 语言仿真结果与使用标准 C 语言库时的结果不同。`hls_math.h` 库只确保 C 语言仿真与 C/RTL 协同仿真结果不同。在上述 2 种情况下，创建的 RTL 实现是相同的。以下解释了使用数学函数时用于执行验证的每一种可能的选项。

验证选项 1：标准数学库和验证差异

在此选项中，源代码中使用的是标准 C 语言数学库。如果已综合的任意函数的准确度与此完全相同，那么 C/RTL 协同仿真与 C 语言仿真不同。以下示例着重演示了此方法。

```
#include <cmath>
#include <fstream>
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

typedef float data_t;

data_t cpp_math(data_t angle) {
    data_t s = sinf(angle);
    data_t c = cosf(angle);
    return sqrtf(s*s+c*c);
}
```

在此情况下，C 语言仿真与 C/RTL 协同仿真之间的结果不同。比较仿真输出时请谨记，从测试激励文件写出的任意结果都将写入执行仿真的工作目录：

- C 语言仿真：<project>/<solution>/csim/build 文件夹
- C/RTL 协同仿真：<project>/<solution>/sim/<RTL> 文件夹

其中，<project> 是工程文件夹，<solution> 是解决方案文件夹的名称，<RTL> 是验证的 RTL 的类型（verilog 或 vhd）。下图显示了综合前的结果文件（左侧）与分析后的 RTL 结果文件（右侧）的典型比较。输出显示在第 3 列中。

图 84：分析前和分析后仿真差异

	result.dat					proj_cpp_math.prj/solution1/sim/systemc/result.dat			
1	0.0000000000000000	0.009999999776483	1.0000000000000000		1	0.0000000000000000	0.009999999776483	1.0000000000000000	
2	1.0000000000000000	0.109999999403954	1.0000000000000000		2	1.0000000000000000	0.109999999403954	1.0000000000000000	
3	2.0000000000000000	0.2099999993443489	1.0000000000000000		3	2.0000000000000000	0.2099999993443489	1.0000000000000000	
4	3.0000000000000000	0.310000002384186	1.0000000000000000		4	3.0000000000000000	0.310000002384186	1.0000000000000000	
5	4.0000000000000000	0.409999996423721	1.0000000000000000		5	4.0000000000000000	0.409999996423721	1.0000000000000000	
6	5.0000000000000000	0.509999990463257	1.0000000000000000		6	5.0000000000000000	0.509999990463257	1.0000000000000000	
7	6.0000000000000000	0.610000014305115	0.999999940395355		7	6.0000000000000000	0.610000014305115	1.0000000000000000	
8	7.0000000000000000	0.710000038146973	1.0000000000000000		8	7.0000000000000000	0.710000038146973	1.0000000000000000	
9	8.0000000000000000	0.810000061988831	1.0000000000000000		9	8.0000000000000000	0.810000061988831	1.0000000000000000	
10	9.0000000000000000	0.910000085830688	1.0000000000000000		10	9.0000000000000000	0.910000085830688	1.0000000000000000	
11	10.0000000000000000	1.010000109672546	1.0000000000000000		11	10.0000000000000000	1.010000109672546	1.0000000000000000	
12	11.0000000000000000	1.110000133514404	1.0000000000000000		12	11.0000000000000000	1.110000133514404	0.999999940395355	
13	12.0000000000000000	1.210000157356262	0.999999940395355		13	12.0000000000000000	1.210000157356262	1.0000000000000000	
14	13.0000000000000000	1.310000181198120	0.999999940395355		14	13.0000000000000000	1.310000181198120	0.999999940395355	
15	14.0000000000000000	1.410000205039978	1.0000000000000000		15	14.0000000000000000	1.410000205039978	1.0000000000000000	
16	15.0000000000000000	1.510000228881836	1.0000000000000000		16	15.0000000000000000	1.510000228881836	1.0000000000000000	
17	16.0000000000000000	1.610000252723694	1.0000000000000000		17	16.0000000000000000	1.610000252723694	1.0000000000000000	
18	17.0000000000000000	1.710000276565552	1.0000000000000000		18	17.0000000000000000	1.710000276565552	1.0000000000000000	
19	18.0000000000000000	1.810000300407410	1.0000000000000000		19	18.0000000000000000	1.810000300407410	1.0000000000000000	
20	19.0000000000000000	1.910000324249268	0.999999940395355		20	19.0000000000000000	1.910000324249268	1.0000000000000000	
21	20.0000000000000000	2.010000228881836	0.999999940395355		21	20.0000000000000000	2.010000228881836	0.999999940395355	
22	21.0000000000000000	2.110000133514404	1.0000000000000000		22	21.0000000000000000	2.110000133514404	1.0000000000000000	
23	22.0000000000000000	2.210000038146973	1.0000000000000000		23	22.0000000000000000	2.210000038146973	1.0000000000000000	
24	23.0000000000000000	2.309999942779541	1.0000000000000000		24	23.0000000000000000	2.309999942779541	1.0000000000000000	
25	24.0000000000000000	2.4099999847412109	1.0000000000000000		25	24.0000000000000000	2.4099999847412109	1.0000000000000000	
26	25.0000000000000000	2.509999752044678	1.0000000000000000		26	25.0000000000000000	2.509999752044678	1.0000000000000000	
27	26.0000000000000000	2.609999656677246	1.0000000000000000		27	26.0000000000000000	2.609999656677246	1.0000000000000000	
28	27.0000000000000000	2.709999561309814	0.999999940395355		28	27.0000000000000000	2.709999561309814	1.0000000000000000	
29	28.0000000000000000	2.809999465942383	1.0000000000000000		29	28.0000000000000000	2.809999465942383	1.0000000000000000	
30	29.0000000000000000	2.909999370574951	1.0000000000000000		30	29.0000000000000000	2.909999370574951	1.0000000000000000	

分析前仿真与分析后仿真结果存在少量差异。您必须判定在最终 RTL 实现中此少量差异是否可接受。

处理此类差异的建议流程是使用测试激励文件检查结果，以确认结果处于可接受的误差范围内。这可通过为同一函数创建 2 个版本来完成，其中 1 个版本用于综合，另 1 个版本用作为参考版本。在此示例中，仅对 `cpp_math` 函数进行综合。

```
#include <cmath>
#include <fstream>
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

typedef float data_t;

data_t cpp_math(data_t angle) {
    data_t s = sinf(angle);
    data_t c = cosf(angle);
    return sqrtf(s*s+c*c);
}

data_t cpp_math_sw(data_t angle) {
    data_t s = sinf(angle);
    data_t c = cosf(angle);
    return sqrtf(s*s+c*c);
}
```

用于验证设计的测试激励文件会使用 `diff` 函数对 2 个函数的输出进行比较以判定差异，如以下示例所示。在 C 语言仿真期间，这 2 个函数会生成完全相同的输出。在 C/RTL 协同仿真期间，`cpp_math` 函数会生成不同结果，并检查结果的差异。

```
int main() {
    data_t angle = 0.01;
    data_t output, exp_output, diff;
    int retval=0;

    for (data_t i = 0; i <= 250; i++) {
        output = cpp_math(angle);
        exp_output = cpp_math_sw(angle);

        // Check for differences
        diff = ( (exp_output > output) ? exp_output - output : output -
exp_output);
        if (diff > 0.0000005) {
            printf("Difference %.10f exceeds tolerance at angle %.10f \n", diff,
angle);
            retval=1;
        }

        angle = angle + .1;
    }

    if (retval != 0) {
        printf("Test failed !!!\n");
        retval=1;
    } else {
        printf("Test passed !\n");
    }
    // Return 0 if the test passes
    return retval;
}
```

如果差异裕度降低到 0.00000005，那么此测试激励文件会高亮 C/RTL 协同仿真期间的误差裕度。

```
Difference 0.0000000596 at angle 1.1100001335
Difference 0.0000000596 at angle 1.2100001574
Difference 0.0000000596 at angle 1.5100002289
Difference 0.0000000596 at angle 1.6100002527
etc..
```

使用标准 C 语言数学库 (`math.h` 和 `cmath.h`) 时，请创建一个“智能”测试激励文件以验证准确性差异是否可接受。

验证选项 2：HLS 函数库和验证差异

另一个验证选项是将源代码转换为使用 HLS 函数库。对于该选项，C 语言仿真与 C/RTL 协同仿真结果之间并无差异。以下示例显示了如何将以上代码修改为使用 `hls_math.h` 库。

注释：该选项仅限在 C++ 中才可用。

- 包含 `hls_math.h` 头文件。
- 将数学函数替换为等效的 `hls::` 函数。

```
#include <cmath>
#include "hls_math.h"
#include <fstream>
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

typedef float data_t;

data_t cpp_math(data_t angle) {
    data_t s = hls::sinf(angle);
    data_t c = hls::cosf(angle);
    return hls::sqrtf(s*s+c*c);
}
```

验证选项 3：HLS 数学库文件和验证差异

包含 HLS 数学库文件 `lib_hlsm.cpp` 作为设计文件可确保 Vivado HLS 使用 HLS 数学库进行 C 语言仿真。该选项与选项 2 相同，但无需修改 C 语言代码。

HLS 数学库文件位于 Vivado HLS 安装区域内的 `src` 目录中。只需将此文件复制到本地文件夹并将其添加为标准设计文件即可。

注释：该选项仅限在 C++ 中才可用。

正如选项 2 一样，该选项使用 HLS 数学库文件进行 C 语言仿真的结果与先前不添加此文件时获取的结果不同。应使用类似选项 1 中的“智能”测试激励文件对 C 语言仿真验证这些差异。

常见综合错误

以下是综合数学函数时常见的用法错误。这些错误通常是由于将 C 语言函数转换为 C++ 以便利用数学函数综合所导致的，但这并非唯一原因。

C++ cmath.h

如果使用 C++ `cmath.h` 头文件，则可使用浮点函数，如 `sinf` 和 `cosf`。这样可在硬件中生成 32 位运算。`cmath.h` 头文件还会导致标准函数（如 `sin` 和 `cos`）重载，以使其可用于浮点类型和双精度类型。

C math.h

如果使用 `C math.h` 库，则需要单精度函数（如 `sinf` 和 `cosf`）才能综合 32 位浮点运算。所有标准函数调用（如 `sin` 和 `cos`）都会导致对双精度和 64 位双精度运算进行综合。

警告

将 C 语言函数转换为 C++ 以利用 `math.h` 支持时，请确保在使用 Vivado HLS 综合前正确编译新 C++ 代码。例如，如果在代码中配合 `math.h` 使用 `sqrtf()`，则需将以下外部代码添加到 C++ 代码中以便为其提供支持：

```
#include <math.h>
extern "C" float sqrtf(float);
```

为避免因类型转换而导致产生不必要的硬件，请遵循“浮点与双精度”中所述的混合双精度与浮点类型进行操作。

HLS 视频库



重要提示！ Vivado® HLS 视频库已迁移到赛灵思 GitHub，可在如下位置找到：<https://github.com/Xilinx/xfopencv>

HLS IP 库

Vivado HLS 可提供 C++ 库以实现多个赛灵思 IP 块。C 语言库允许从 C++ 源代码直接推断以下赛灵思 IP 块，以确保 FPGA 内的高质量实现。

表 26：HLS IP 库

库头文件	描述
<code>hls_fft.h</code>	允许在 C 语言中对赛灵思 LogiCORE IP FFT 进行仿真并使用赛灵思 LogiCORE 块来实现。
<code>hls_ssrlib.h</code>	允许完全可综合的超采样数据率 (SSR) FFT 在每个时钟周期内处理多个输入样本。
<code>hls_fir.h</code>	允许在 C 语言中对赛灵思 LogiCORE IP FIR 进行仿真并使用赛灵思 LogiCORE 块来实现。
<code>hls_dds.h</code>	允许在 C 语言中对赛灵思 LogiCORE IP DDS 进行仿真并使用赛灵思 LogiCORE 块来实现。
<code>ap_shift_reg.h</code>	提供 C++ 类用于实现移位寄存器，该移位寄存器是使用赛灵思 SRL 原语直接实现的。

FFT IP 库

在 C++ 设计中可使用 `hls_fft.h` 库调用赛灵思 FFT IP 块。本节解释了在 C++ 代码中配置 FFT 的方式。



建议：赛灵思强烈建议您阅读《Fast Fourier Transform LogiCORE IP 产品指南》(PG109)，以获取有关如何实现和使用此 IP 的功能的信息。

要在 C++ 代码中使用 FFT，请执行以下操作：

1. 在代码中包含 `hls_fft.h` 库
2. 使用预定义的 `hls::ip_fft::params_t` 结构体来设置默认参数
3. 定义运行时间配置
4. 调用 FFT 函数
5. (可选) 检查运行时间状态

以下代码示例提供了每个步骤的执行方式的摘要信息。后文对每个步骤进行了更详细的探讨。

首先，在源代码中包含 FFT 库。此头文件位于 Vivado HLS 安装区域内的 `include` 目录中，执行 Vivado HLS 时会自动搜索该目录。

```
#include "hls_fft.h"
```

定义 FFT 的静态参数。这包括诸如输入宽度、通道数、架构类型等不会发生动态更改的参数。FFT 库包含参数化 `hls::ip_fft::params_t` 结构体，可用于以默认值初始化所有静态参数。

在此示例中，使用用户基于预定义结构体定义的 `param1` 结构体来覆盖输出排序以及配置端口和状态端口宽度的默认值。

```
struct param1 : hls::ip_fft::params_t {
    static const unsigned ordering_opt = hls::ip_fft::natural_order;
    static const unsigned config_width = FFT_CONFIG_WIDTH;
    static const unsigned status_width = FFT_STATUS_WIDTH;
};
```

为运行时间配置和运行时间状态定义类型和变量。这些值可采用动态值，因此在 C 语言代码中定义为可通过 API 来更改和访问的变量。

```
typedef hls::ip_fft::config_t<param1> config_t;
typedef hls::ip_fft::status_t<param1> status_t;
config_t fft_config1;
status_t fft_status1;
```

下一步，设置运行时间配置。此示例基于“`direction`”变量的值设置 FFT 的方向（正向或反向），并设置了缩放调度的值。

```
fft_config1.setDir(direction);
fft_config1.setSch(0x2AB);
```

使用具有已定义的静态配置（即此示例中的 `param1`）的 HLS 名称空间来调用 FFT 函数。函数参数按顺序为输入数据、输出数据、输出状态和输入配置。

```
hls::fft<param1> (xn1, xk1, &fft_status1, &fft_config1);
```

最后，检查输出状态。此示例检查了溢出标记，并将结果存储在“ovflo”变量中。

```
*ovflo = fft_status1->getOvflo();
```

在 Vivado HLS 示例中提供了使用 FFT C 语言库的设计示例，可使用以下菜单选项来访问这些示例：“Help” → “Welcome” → “Open Example Project” → “Design Examples” → “FFT”。

FFT 静态参数

FFT 的静态参数用于定义 FFT 的配置方式，并指定固定参数，例如，FFT 的大小、大小是否可动态更改以及实现采用流水线化还是 radix_4_burst_io。

hls_fft.h 头文件用于定义 hls::ip_fft::params_t 结构体，该结构体可用于为静态参数设置默认值。如果要使用默认值，那么参数化结构体可直接搭配 FFT 函数一起使用。

```
hls::fft<hls::ip_fft::params_t >
(xn1, xk1, &fft_status1, &fft_config1);
```

更常见的用例是将部分参数更改为非默认值。执行此操作的方式是基于默认参数化结构体创建用户定义的新参数化结构体并更改部分默认值。

在以下示例中，定义了 1 个新的用户结构体 my_fft_config，并为输出排序定义了新的值（更改为 natural_order）。FFT 的所有其它静态参数均使用默认值。

```
struct my_fft_config : hls::ip_fft::params_t {
    static const unsigned ordering_opt = hls::ip_fft::natural_order;
};

hls::fft<my_fft_config >
(xn1, xk1, &fft_status1, &fft_config1);
```

如需了解用于 hls::ip_fft::params_t 参数化结构体的解释，请参阅 [FFT 结构体参数](#)。在 [FFT 结构体参数值](#) 中提供了参数默认值以及可能的值的列表。



建议：赛灵思强烈建议您参阅《LogiCORE IP Fast Fourier Transform 产品指南》(PG109)，以了解有关参数及其设置的影响的详细信息。

FFT 结构体参数

表 27：FFT 结构体参数

参数	描述
input_width	数据输入端口宽度。
output_width	数据输出端口宽度。
status_width	输出状态端口宽度。
config_width	输入配置端口宽度。
max_nfft	FFT 数据集大小指定为 $1 \ll \text{max_nfft}$ 。
has_nfft	判定在运行时间 FFT 大小是否可配置。
channels	通道数。

表 27: FFT 结构体参数 (续)

参数	描述
arch_opt	实现架构。
phase_factor_width	配置内部相位因子精度。
ordering_opt	输出排序模式。
ovflo	启用溢出模式。
scaling_opt	定义缩放选项。
rounding_opt	定义舍入模式。
mem_data	指定使用块 RAM 还是分布式 RAM 作为数据内存。
mem_phase_factors	指定使用块 RAM 还是分布式 RAM 作为相位因子内存。
mem_reorder	指定使用块 RAM 还是分布式 RAM 作为输出重新排序内存。
stages_block_ram	定义实现中使用的块 RAM 阶段数。
mem_hybrid	针对数据、相位因子或重新排序缓存指定使用块 RAM 时，mem_hybrid 用于指定在何处混用块 RAM 与分布式 RAM 来减少某些配置中的块 RAM 计数，或者指定不混用这两者。
complex_mult_type	定义用于复数乘法的乘法器类型。
butterfly_type	定义用于蝶形 FFT 的实现。

如果指定的参数值并非整数或布尔值，则应使用 HLS FFT 名称空间。

例如，下表中参数 butterfly_type 的可能值为 use_luts 和 use_xtremedsp_slices。C 程序中使用的值应为 butterfly_type = hls::ip_fft::use_luts 和 butterfly_type = hls::ip_fft::use_xtremedsp_slices。

FFT 结构体参数值

下表涵盖了 FFT IP 的所有特性和功能。本表中未描述的特性和功能在 Vivado HLS 实现中均不予支持。

表 28: FFT 结构体参数值

参数	C 语言类型	默认值	有效值
input_width	无符号	16	8-34
output_width	无符号	16	介于 input_width 到 (input_width + max_nfft + 1) 之间
status_width	无符号	8	取决于 FFT 配置
config_width	无符号	16	取决于 FFT 配置
max_nfft	无符号	10	3-16
has_nfft	布尔值	false	True, False
channels	无符号	1	1-12
arch_opt	无符号	pipelined_streaming_io	automatically_select pipelined_streaming_io radix_4_burst_io radix_2_burst_io radix_2_lite_burst_io
phase_factor_width	无符号	16	8-34

表 28: FFT 结构体参数值 (续)

参数	C 语言类型	默认值	有效值
ordering_opt	无符号	bit_reversed_order	bit_reversed_order natural_order
ovflo	布尔值	true	false true
scaling_opt	无符号	scaled	scaled unscaled block_floating_point
rounding_opt	无符号	truncation	truncation convergent_rounding
mem_data	无符号	block_ram	block_ram distributed_ram
mem_phase_factors	无符号	block_ram	block_ram distributed_ram
mem_reorder	无符号	block_ram	block_ram distributed_ram
stages_block_ram	无符号	$(\text{max_nfft} < 10) ? 0 :$ $(\text{max_nfft} - 9)$	0-11
mem_hybrid	布尔值	false	false true
complex_mult_type	无符号	use_mults_resources	use_luts use_mults_resources use_mults_performance
butterfly_type	无符号	use_luts	use_luts use_xtremesp_slices

FFT 运行时间配置与状态

FFT 支持通过配置端口和状态端口进行运行时间配置与运行时间状态监控。这些端口定义为 FFT 函数的实参，此处分别显示为 `fft_status1` 变量和 `fft_config1` 变量：

```
hls::fft<param1> (xn1, xk1, &fft_status1, &fft_config1);
```

运行时间配置和状态可使用来自 FFT C 语言库的预定义结构体来访问：

- `hls::ip_fft::config_t<param1>`
- `hls::ip_fft::status_t<param1>`

注释：在这 2 种情况下，此结构体需获取静态参数化结构体的名称，在此处示例中显示的名称为 `param1`。请参阅上一章节，以获取有关定义静态参数化结构体的详细信息。

运行时间配置结构体允许在 C 语言代码中执行以下操作：

- 设置 FFT 长度，前提是已启用运行时间配置
- 将 FFT 方向设置为正向或反向
- 设置缩放调度

FFT 长度可按如下方式进行设置：

```
typedef hls::ip_fft::config_t<param1> config_t;
config_t fft_config1;
// Set FFT length to 512 => log2(512) =>9
fft_config1->setNfft(9);
```



重要提示！ 运行时间期间指定的长度不能超过静态配置中的 `max_nfft` 定义的大小。

FFT 方向可按如下方式进行设置：

```
typedef hls::ip_fft::config_t<param1> config_t;
config_t fft_config1;
// Forward FFT
fft_config1->setDir(1);
// Inverse FFT
fft_config1->setDir(0);
```

FFT 缩放调度可按如下方式进行设置：

```
typedef hls::ip_fft::config_t<param1> config_t;
config_t fft_config1;
fft_config1->setSch(0x2AB);
```

可使用预定义结构体来访问输出状态端口以判定：

- FFT 期间是否发生任何溢出
- 块指数的值

FFT 溢出模式可按如下方式进行检查：

```
typedef hls::ip_fft::status_t<param1> status_t;
status_t fft_status1;
// Check the overflow flag
bool *ovflo = fft_status1->getOvflo();
```



重要提示！ 完成每项传输事务后，检查溢出状态以确认 FFT 的正确操作。

可使用以下方式获取块指数值：

```
typedef hls::ip_fft::status_t<param1> status_t;
status_t fft_status1;
// Obtain the block exponent
unsigned int *blk_exp = fft_status1->getBlkExp();
```

使用 FFT 函数

FFT 函数在 HLS 名称空间内定义，可按如下方式调用：

```
hls::fft<STATIC_PARAM> (
    INPUT_DATA_ARRAY,
    OUTPUT_DATA_ARRAY,
    OUTPUT_STATUS,
    INPUT_RUN_TIME_CONFIGURATION);
```

STATIC_PARAM 属于静态参数化结构体，用于为 FFT 定义静态参数。

输入数据和输出数据都作为数组（INPUT_DATA_ARRAY 和 OUTPUT_DATA_ARRAY）提供给函数。在最终实现中，FFT RTL 块上的端口将作为 AXI4-Stream 端口来实现。赛灵思建议始终在使用数据流最优化 (set_directive_dataflow) 的区域内使用 FFT 函数，因为这样可确保数组作为流传输数组来实现。另一种方法是使用 set_directive_stream 命令将这两个数组都指定为流传输数组。



重要提示！ FFT 无法在流水线化的区域内使用。如需高性能运算，请在使用 FFT 前后分别将循环或函数流水线化，然后对区域内的所有循环和函数使用数据流最优化。

数组的数据类型可采用 float 或 ap_fixed。

```
typedef float data_t;
complex<data_t> xn[FFT_LENGTH];
complex<data_t> xk[FFT_LENGTH];
```

要使用定点数据类型，应使用 Vivado HLS 任意精度类型 ap_fixed。

```
#include "ap_fixed.h"
typedef ap_fixed<FFT_INPUT_WIDTH,1> data_in_t;
typedef ap_fixed<FFT_OUTPUT_WIDTH,FFT_OUTPUT_WIDTH-FFT_INPUT_WIDTH+1>
data_out_t;
#include <complex>
typedef hls::x_complex<data_in_t> cmpxData;
typedef hls::x_complex<data_out_t> cmpxDataOut;
```

使用这 2 种类型时，都应使用相同且正确的数据大小来将 FFT 参数化。对于浮点数据，数据宽度始终为 32 位，指定任何其它大小都将被视为无效。



重要提示！ FFT 的输入和输出宽度可配置为受支持范围内的任意值。连接到输入参数和输出参数的变量必须按 8 位增量来定义。例如，如果输出宽度配置为 32 位，那么输出变量必须定义为 40 位变量。

可通过针对输入数据和输出数据使用 2 维数组来使用 FFT 的多通道功能。在此情况下，数组数据的配置方法应为第 1 个维度表示每个通道，且第 2 个维度表示 FFT 数据。

```
typedef float data_t;
static complex<data_t> xn[CHANNEL][FFT_LENGTH];
static complex<data_t> xk[CHANNEL][FFT_LENGTH];
```

FFT 核使用并生成数据作为交织式通道（例如，ch0-data0、ch1-data0、ch2-data0 等，ch0-data1、ch1-data1、ch2-data2 等，以此类推）。因此，要按读取或写入数据的顺序来对 FFT 的输入或输出数组进行流传输，必须首先迭代通道索引，为多个通道填满或清空 2 维数组，如下示例所示：

```
cmpxDat  in_fft[FFT_CHANNELS][FFT_LENGTH];
cmpxDat  out_fft[FFT_CHANNELS][FFT_LENGTH];

// Write to FFT Input Array
for (unsigned i = 0; i < FFT_LENGTH; i++) {
    for (unsigned j = 0; j < FFT_CHANNELS; ++j) {
        in_fft[j][i] = in.read().data;
    }
}

// Read from FFT Output Array
for (unsigned i = 0; i < FFT_LENGTH; i++) {
    for (unsigned j = 0; j < FFT_CHANNELS; ++j) {
        out.data = out_fft[j][i];
    }
}
```

在 Vivado HLS 示例中提供了使用 FFT C 语言库的设计示例，可使用以下菜单选项来访问这些示例：“Help” → “Welcome” → “Open Example Project” → “Design Examples” → “FFT”。

SSR FFT IP 库

简介

Vivado HLS 可提供完全可综合的超采样数据率 (SSR) FFT，其中具有脉动架构，用于处理每个时钟周期的多个输入样本。每个周期内并行处理的样本数量以 SSR 因子来表示。此 FFT 作为 C++ 模板函数来实现，该函数的结构可通过模板参数来加以参数化，此模板参数属于 `ssr_fft_default_params` 类型的 C++ 结构体。可通过扩展默认结构并覆盖成员常量来定义新结构体，如下所示：

```
struct ssr_fft_fix_params:ssr_fft_default_params
{
    static const int N=1024;
    static const int R=4;
    static const scaling_mode_enum scaling_mode=SSR_FFT_GROW_TO_MAX_WIDTH;
    static const fft_output_order_enum output_data_order=SSR_FFT_NATURAL;
    static const int twiddle_table_word_length=18;
    static const int twiddle_table_intger_part_length=2;
};
```

以上结构可定义：

- N：变换的大小或长度
- R：要并行处理的样本数量
- scaling_mode：枚举类型的缩放模式
- output_data_order：输出数据顺序，用于决定按自然顺序处理数据还是按数字反转转置顺序来处理数据
- twiddle_table_word_length：定义要用于存储旋转表因子的总位数
- twiddle_table_intger_part_length：用于存储旋转的整数部分的整数位数

调用 FFT 时，定义 C++ 结构体的用户可作为模板参数，如下所示：

```
hls::ssr_fft::fft<ssr_fft_fix_params>(...);
```

性能

FFT 吞吐量（启动时间间隔）可采用 L/R 的方式来计算，其中 R 表示 SSR 值，而 L 则表示要变换的样本数。R 的可能值（SSR 值）包括：2、4、8 和 16。以最慢的 UltraScale+ 速度等级器件为目标时，这些值支持的 Fmax 范围为 300-550 MHz。

数据类型

FFT 基于用于综合和实现的定点数据类型 (`std::complex<ap_fixed<>>`)。否则，可使用浮点类型进行仿真。

为实现最佳结果，请将数据位宽限制为 27 位（整数 + 小数），因为这样可直接映射到单一 DSP 块。可使用更大的输入，但可能导致 Fmax 更慢，利用率更低。最后，请注意，复指数/旋转因子存储于第 18 位上（16F+2I 位）。选择第 18 位的原因是为了监视具有 18x27 位乘法器的赛灵思 FPGA 上的 DSP 块可用的乘法器。

在 FFT 阶段中管理数据位增长：

FFT 支持采用 3 种不同模式来管理 FFT 阶段间的位增长。这 3 种模式可用于允许每个阶段内的位增长、允许在每个阶段内使用无位增长的缩放，或者允许位增长直至 27 位，然后开始使用缩放。详细描述如下所述：

- **SSR_FFT_GROW_TO_MAX_WIDTH**：当参数结构中的 `scaling_mode` 常量设置为 `SSR_FFT_GROW_TO_MAX_WIDTH` 时，可指定各阶段间的增长量，从第一阶段开始直至指定的最大位宽为止。输出位宽增长至 27 位，然后达到饱和。每个阶段的输出位宽增长 $\log_2(R)$ 位，然后达到最大值 27 位时终止，以继续将蝶形操作映射至 DSP。当初始输入位宽小于 27 位时，适用该选项。
- **SSR_FFT_SCALE**：当参数结构中的 `scaling_mode` 常量设置为 `SSR_FFT_SCALE` 时，支持在每个阶段内对输出进行缩放。输出在每个阶段内进行缩放，并损失精度。大小为 L 且 $\text{Radix}=\text{SSR}=\text{R}$ 的 FFT 所含阶段数量为 $\log_2(L)$ 。当输入位宽已接近 27 位并且要求输出增长不超过 27 位以将乘法映射到 DSP 时，适用该选项。
- **SSR_FFT_NO_SCALE**：当参数结构中的 `scaling_mode` 常量设置为 `SSR_FFT_NO_SCALE` 时，每个阶段中都允许位增长，并且每个阶段内输出增长都不受 $\log_2(R)$ 限制。如需高精度，可使用此设置。但如果输出位宽超过 27 位，乘法可能不止映射到 DSP，还会开始与 FPGA 结构逻辑配合使用；这可能导致对时钟速度和资源利用率产生不利影响。

使用 SSR FFT 定点配置的建议流程

SSR FFT 支持多种缩放模式，并提供了定义输入位宽和存储指数值（查找表中的正弦/余弦）所需的位宽的相应选项。用于定义输出信号质量的信噪比取决于这些不同参数的选择以及用于将实值连续信号或浮点信号转换为定点信号的量化方案。信号的范围和解析（尤其是整数位和小数位）应谨慎选择，以在 FFT 输出处达成良好的信噪比 (SNR)。处理 SSR FFT HLS IP 的建议流程如下所述。

首先从 SSR FFT 的浮动模型开始

当前，SSR FFT 可搭配 `ap_fixed<>`、`float` 和 `double` 类型使用。下表列出了对应综合与仿真的支持。

表 29：SSR FFT 类型支持

类型	支持综合	支持仿真
<code>std::complex < ap_fixed <> ></code>	支持	支持
<code>std::complex<float></code>	不支持	支持

表 29: SSR FFT 类型支持 (续)

类型	支持综合	支持仿真
<code>std::complex<double></code>	不支持	支持

建议的起点是首先在 `std::complex<>` 中使用 `float/double` 内部类型，针对使用参考模型（例如，Matlab/Python/Octave/Simulink 等通过生成黄金测试矢量使用的任意建模语言或工具）验证 SNR。SSR FFT 的可综合版本当前仅支持 `ap_fixed<>` 内部类型，因此下一步是开始试验定点模型。

定点建模与实现

开始处理定点模型

处理定点模型时，建议的起始缩放模式为 `SSR_FFT_NO_SCALING`。输入位宽应按如下方式选择。

创建 `ap_fixed<WL, IL>` 类型的初始定点模型。整体输入类型为 `std::complex <ap_fixed<WL, IL>`，实质上它用于存储输入的实数部分和虚数部分。

这些部分分别为：

- `IL`：整数位，基于输入范围来选择
- `WL`：字长 (Word Length) = `IL` + `FL`，其中 `FL` 表示小数位宽 (Fraction Bit Width)，基于输入解析度来选择

在此例中，由于缩放模式选择，SSR FFT 内部不使用缩放；因此，输出中不会出现缩放错误。当缩放模式设置为无缩放时，可试用其它定点参数，例如用于表示输入样本的整数位和小数位。简单方法是基于输入范围和解析度来选择表示输入所需的位数，但用户可根据其它输入特征来对此位宽进行最优化。

选择输入位宽

输入位宽的选择取决于输入数据特征以及所需的解析度，其选择与数据相关，本质上取决于测试数据的范围和解析度。为便于仿真，您可选择任意较大的位宽值来表示整数位数和小数位数。为便于实现，您必须根据所需 SNR 来作出最优选择。

建议的策略是执行以下任一操作：

- 保持缩放模式固定为 `SSR_FFT_NO_SCALING`。
- 观察位于 SSR FFT 输出处的噪声比率，修改整数和小数表示法的输入位数。
- 减少位宽，以所需的最低位数来满足输出 SNR 要求。

满足 SNR 要求后，即可继续执行其它定点最优化，例如，存储复指数表和 SSR FFT 输出缩放选项所需的位数。

旋转因子或正弦/余弦查找表量化

您可更改用于对正弦/余弦表（旋转因子/复指数）进行量化的位数。建议的设置总计 18 位，小数部分 2 位。此设置可确保在执行乘法期间，旋转因子/正弦/余弦输入可映射到赛灵思 FPGA 中的 DSP 块的 18 位输入。此模型可执行综合，并且适用于任何大型位宽，但性能可能下降，因为乘法运算未映射到单一 DSP 块，并使用多个 DSP 块和/或 FPGA 结构来实现。

如果旋转因子存储空间的初始设置大于 18 位，那么可适用旋转因子减宽。默认情况下，它设置为使用 18 位，预留 2 位用于有符号的整数部分。这 2 位是准确表示表中的 -1 值所必需的。

选择最佳缩放模式

在没有缩放的情况下选择输入位宽度和旋转因子，在定点 SSR FFT 输出端给出可接受的 SNR 或均方根 (RMS) 误差后，即可开始尝试选择缩放模式。SSR FFT 提供 3 种不同缩放模式。推荐的策略是从 `SSR_FFT_NO_SCALING` 开始。如果输出的 SNR/RMS 误差可接受，请切换至 `SSR_FFT_GROW_TO_MAX_WIDTH`。如果仍然存在可接受的 SNR/RMS 误差，请切换至 `SSR_FFT_SCALE`。

SSR_FFT_NO_SCALING

建议从此模式开始。它不执行缩放，但是输出位宽度在每个阶段都将增加 $\log_2(R=SSR)$ 。例如，如果 FFT 的大小为 $N = 64$ ，并且选择了 $SSR = R = 4$ ，则 SSR FFT 包含的阶段数为 $\log_4(64) = 3$ 。如果输入位宽度为 W ，则输出位宽为 $W + 3 \times 2 = W + 6$ 。因此，输出将增加 $\log_2(N) \times \log_2(R)$ 位。

`SSR_FFT_NO_SCALING` 保留了计算的准确性，但代价是所需硬件成本最高。SSR FFT 计算是分阶段进行的，上一个阶段为下一个阶段提供输入，因此从本质上讲，它属于链式阶段。

不可控制的位增长的缺点之一是，在某一点上，当某一阶段的输出宽度增加超过一个极限，即乘法开始不再映射到 FPGA 上的 DSP 块时，在速度方面，设计性能可能会大幅下降。例如，如果给定设计的 $\log_2(N) \times \log_2(R) + \text{Input Bit Width}(\text{IL} + \text{FL}) > \max(\text{DSP Block Multiplier Inputs})$ ，则您可以考虑使用另 2 个可用的缩放方案之一。如果将具有 18x27 个乘法器的赛灵思 DSP48 块用于具有 DSP48 块的 FPGA 器件，则条件为 $\log_2(N) \times \log_2(R) + \text{Input Bit Width} > 27$ 。

SSR_FFT_GROW_TO_MAX_WIDTH

在这种模式下，将使用混合方法。最初，如果有增长空间，则允许位增长。如果在开始的 FFT 阶段中输出的位宽小于可映射到 DSP 块的位宽，则允许位增长。当位宽超过可映射到 DSP 块的位宽时，它将开始缩放输出。

SSR_FFT_SCALE

当已知指定的 FFT 大小 N 和 SSR 因子时，输出将超出限制，导致 DSP 乘法器块无法在指定的 FPGA 器件上对其进行处理，此时即可选择 `SSR_FFT_SCALE` 选项开启每个阶段的缩放。该选项通过在每个阶段将输出右移 $\log_2(SSR=R)$ 来缩放每个阶段的输出。

推荐流程仅提供创建定点模型的指南，并讨论了 SSR FFT 中可用的选项。根据设计的 SNR/RMS 要求，要求用户仔细选择所有这些参数，同时考虑具体应用的不同性能和 SNR/RMS 要求。

SSR FFT IP 库的使用

SSR FFT 可在 C++ 设计中使用 `library hls_ssr_lib.h` 库来使用。本节解释了有关在基于 C++ 的 HLS 设计中使用的其它接口级详细信息，并提供了使用示例。

要使用 SSR FFT IP 库，请执行以下操作：

1. 包含“`hls_ssr_lib.h`”头文件：

```
#include <hls_ssr_lib.h>
```

2. 定义 C++ 结构体以扩展 `ssr_fft_default_params`：

```
struct ssr_fft_params:ssr_fft_default_params
{
    static const int N=SSR_FFT_L;
    static const int R=SSR_FFT_R;
    static const scaling_mode_enum
        scaling_mode=SSR_FFT_GROW_TO_MAX_WIDTH;
```

```
static const fft_output_order_enum
    output_data_order=SSR_FFT_NATURAL;
static const int twiddle_table_word_length=18;
static const int twiddle_table_intger_part_length=2;

};
```

3. 调用 SSR FFT，如下所示：

```
hls::ssr_fft::fft<ssr_fft_params>(inD,outD);
```

其中 inD 和 outD 是二维复数数组（类型为 ap_fixed、float 或 double），在前表中已解释了综合和仿真用法。I/O 数组可按如下方式声明：

- 定点类型：首先定义输入类型，然后使用类型特征基于 ssr_fft_params 结构体计算输出类型（输出类型计算考量缩放模式的依据是位增长率和输入位宽）

```
typedef std::complex< ap_fixed<16,8> > I_TYPE;
typedef
hls::ssr_fft::ssr_fft_output_type<ssr_fft_params,I_TYPE>:t_ssr_fft_out
O_TYPE;
I_TYPE inD[SSR_FFT_R][SSR_FFT_L/SSR_FFT_R];
O_TYPE outD [R][L/R];
```

此处 SSR_FFT_R 用于定义 SSR 因子，SSR_FFT_L 定义 FFT 变换大小。

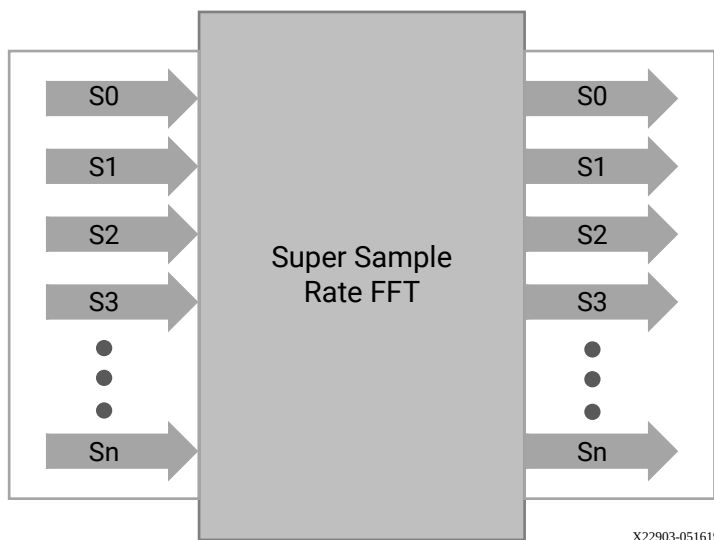
- 浮点类型/双精度类型：首先定义双精度/浮点输入类型，然后使用类型特征基于 ssr_fft_params 结构体计算输出类型。对于浮点类型，输出类型计算返回的类型与输入相同。

```
typedef std::complex< float/double > I_TYPE;
typedef
hls::ssr_fft::ssr_fft_output_type<ssr_fft_params,I_TYPE>:t_ssr_fft_out
O_TYPE;
I_TYPE inD[SSR_FFT_R][SSR_FFT_L/SSR_FFT_R];
O_TYPE outD[SSR_FFT_R][SSR_FFT_L/SSR_FFT_R];
```

SSR FFT 输入数组读写注意事项

综合后，SSR FFT HLS IP 会映射到输入和输出两端都含 FIFO 接口的流传输块，如下图所示：

图 85：综合后的 SSR FFT HLS IP



综合期间，置于 IP 描述内部 HLS 编译指示将把 I/O 数组内的 2 维映射到时间和宽流传输。针对第 2 个维度，它使用 HLS STREAM 编译指示。针对第 1 个维度，它使用编译指示进行数据打包、分区和重塑，以创建单宽流传输。

如果输入和输出数组按如下方式声明：

```
I_TYPE inD[R][L/R];
O_TYPE outD[R][L/R];
```

大小为 L/R 的维度将映射到时间，大小为 R 的维度将映射到宽度为 R 的单个流传输。此映射会施加约束以限制使用者和生产者使用 SSR FFT 编写 C++ 设计时读写这些数组的方式。这些约束源于从数组维度到时间和并行宽度访问的物理映射。SSR FFT I/O 数组上的读写可按如下方式执行：

1. 应按如下方式在嵌套循环中写入输入，访问第 1 个维度的循环为内层循环。外层循环应访问时间维度/第 2 个维度。

```
for( int t=0;t<L/R;t++)
{
    for (int r=0; r <R : r++)
    {
        inD[r][t] = ..... ;
    }
}
```

2. 输出应按如下方式读取：

```
for( int t=0;t<L/R;t++)
{
    for (int r=0; r <R : r++)
    {
        ..... = outD[r][t] ;
    }
}
```

3. 如果 SSR FFT IP 面向输入链或输出链中的另一个 HLS IP，那么应将执行读写的内层循环展开。

在数据流区域中使用 SSR FFT 进行流传输连接和非流传输连接

SSR FFT 内部严重依赖于 HLS 数据流最优化。SSR FFT 的可能用例包括通过以下 2 种方式与 FFT 输入或输出进行互连：

- 流传输连接
- 非流传输连接

流传输连接

输入时的流传输连接如以下代码片段所示：

```
#pragma HLS DATAFLOW

in_dummy_proc (... , fft_in);
hls:ssr_fft::fft<ssr_fft_params>(fft_in, fft_out)
out_dummy_proc(fft_out, ....)
...
...
...
```

输入生产者的约束为，它应生成宽流传输。输出使用者的约束为，它应使用宽流传输。在前文中同样对这些约束进行了描述。

非流传输连接

当前版本的 SSR FFT 不支持在输出和输入端的非流传输连接。但可通过根据需要在输入/输出端放置适配器来启用此支持，这样即可将流传输转换为不同接口。例如，以下代码片段即将流传输接口映射到基于内存的接口的输入适配器：

```
template < type name TYPE, int R, int L >
void fft_input_adapter (TYPE inData[R][L/R], TYPE outDataStream[R][L/R])
{
#pragma HLS INLINE off
#pragma HLS DATA_PACK variable=inData
#pragma HLS ARRAY_RESHAPE variable=inData complete dim=1
    for(int t=0; t<L/R; t++)
    {
#pragma HLS PIPELINE II=1
        for (int r = 0; r< R; ++r)
        {
            outDataStream [r][t] = inData[r][t];
        }
    }
}

.
.
. // Usage of Adapter at input side:

#pragma HLS DATAFLOW
in_proc_memory_based(...,in_data_mem_based)
fft_input_adapter<TYPE_NAME,R,L>( in_data_mem_based,
fft_in_stream_based);
hls:ssr_fft::fft<ssr_fft_params>(fft_in_stream_based,
fft_out_stream_based)
out_dummy_proc(fft_out_stream_based, ....)
...
...
...
```

注释：可使用相似方法来构造输出端的适配器。

FIR 滤波器 IP 库

在 C++ 设计中可使用 `hls_fir.h` 库调用赛灵思 FIR IP 块。本节解释了在 C++ 代码中配置 FIR 的方式。



建议：赛灵思强烈建议您阅读《FIR Compiler LogiCORE IP 产品指南》(PG149)，以获取有关如何实现和使用此 IP 的功能的信息。

要在 C++ 代码中使用 FIR，请执行以下操作：

1. 在代码中包含 `hls_fir.h` 库。
2. 使用预定义的 `hls::ip_fir::params_t` 结构体设置静态参数。
3. 调用 FIR 函数。
4. （可选）定义运行时间输入配置以动态修改部分参数。

以下代码示例提供了每个步骤的执行方式的摘要信息。后文对每个步骤进行了更详细的探讨。

首先，在源代码中包含 FIR 库。此头文件位于 Vivado HLS 安装区域内的 `include` 目录中。执行 Vivado HLS 时，会自动搜索此目录。在 Vivado HLS 内编译时无需指定到该目录的路径。

```
#include "hls_fir.h"
```

定义 FIR 的静态参数。包括诸如输入宽度、系数和滤波率 (`single`、`decimation` 和 `hilbert`) 等静态属性。FIR 库包含参数化 `hls::ip_fir::params_t` 结构体，可用于以默认值初始化所有静态参数。

在此示例中，系数定义为驻留在数组 `coeff_vec` 中，并使用用户定义的 `myconfig` 结构体（基于预定义结构体）来覆盖系数的数量、输入宽度和量化模式的默认值。

```
struct myconfig : hls::ip_fir::params_t {
    static const double coeff_vec[sg_fir_srcc_coeffs_len];
    static const unsigned num_coeffs = sg_fir_srcc_coeffs_len;
    static const unsigned input_width = INPUT_WIDTH;
    static const unsigned quantization = hls::ip_fir::quantize_only;
};
```

使用含已定义的静态参数（此示例中为 `myconfig`）的 HLS 名称空间来创建 FIR 函数实例，然后使用 `run` 方法调用并执行该函数。函数实参按顺序为输入数据和输出数据。

```
static hls::FIR<param1> fir1;
fir1.run(fir_in, fir_out);
```

（可选）可使用运行时间输入配置。在 FIR 的某些模式下，此输入上的数据用于判定交织通道期间或者需要重新加载系数时的系数使用方式。此配置可采用动态配置，因而将其定义为变量。要获取有关哪些模式需使用此输入配置的完整说明，请参阅《FIR Compiler LogiCORE IP 产品指南》(PG149)。

使用运行时间输入配置时，将使用以下 3 个实参来调用 FIR 函数：输入数据、输出数据和输入配置。

```
// Define the configuration type
typedef ap_uint<8> config_t;
// Define the configuration variable
config_t fir_config = 8;
// Use the configuration in the FFT
static hls::FIR<param1> fir1;
fir1.run(fir_in, fir_out, &fir_config);
```

在 Vivado HLS 示例中提供了使用 FIR C 语言库的设计示例，可使用以下菜单选项来访问这些示例：“Help” → “Welcome” → “Open Example Project” → “Design Examples” → “FIR”。

FIR 静态参数

FIR 的静态参数可定义 FIR IP 的参数化方式，并指定非动态项，如输入和输出宽度、小数位数、系数值和内插与抽取速率。以上大部分配置都具有默认值：系数无默认值。

hls_fir.h 头文件可定义用于设置大部分静态参数的默认值的 struct hls::ip_fir::params_t。



重要提示！ 系数不含定义的默认值。因此，赛灵思不建议使用预定义的结构体来直接初始化 FIR。应始终使用用户定义的用于指定系数的新结构体来执行静态参数化。

在此示例中，用户定义了 1 个新的 struct my_config，其中包含用作为系数的新值。这些系数指定为驻留在 coeff_vec 数组内。FIR 的所有其它参数均使用默认值。

```
struct myconfig : hls::ip_fir::params_t {
    static const double coeff_vec[sg_fir_srcc_coeffs_len];
};
static hls::FIR<myconfig> fir1;
fir1.run(fir_in, fir_out);
```

FIR 静态参数描述了用于参数化结构体 hls::ip_fir::params_t 的参数。**FIR 构造体参数值**提供了参数默认值和可能的值的列表。



建议：赛灵思强烈建议您参阅《FIR Compiler LogiCORE IP 产品指南》(PG149)，以获取有关参数及其设置的影响的详细信息。

FIR 结构体参数

表 30：FIR 结构体参数

参数	描述
input_width	数据输入端口宽度
input_fractional_bits	输入端口的小数位数
output_width	数据输出端口宽度
output_fractional_bits	输出端口的小数位数
coeff_width	系数位宽
coeff_fractional_bits	系数的小数位数
num_coeffs	系数的数量
coeff_sets	系数集的数量
input_length	输入数据中的样本数

表 30: FIR 结构体参数 (续)

参数	描述
output_length	输出数据中的样本数
num_channels	指定要处理的数据通道数
total_num_coeff	系数总数
coeff_vec[total_num_coeff]	系数数组
filter_type	用于筛选的实现类型
rate_change	指定整数或小数速率变化
interp_rate	内插速率
decim_rate	抽取速率
zero_pack_factor	内插中使用的 0 系数数量
rate_specification	指定以频率或周期形式来表示速率
hardware_oversampling_rate	指定过采样速率
sample_period	硬件过采样周期
sample_frequency	硬件过采样频率
quantization	要使用的量化方法
best_precision	启用或禁用最佳精度
coeff_structure	要使用的系数结构类型
output_rounding_mode	输出上使用的舍入类型
filter_arch	选择脉动或转置架构
optimization_goal	指定最优化的速度或面积目标
inter_column_pipe_length	DSP 列之间所需流水线长度
column_config	指定 DSP48 列的数量
config_method	指定 DSP48 列的配置方式
coeff_padding	添加到滤波器前的填充 0 的数量

如果指定的参数值并非整数或布尔值，则应使用 HLS FIR 名称空间。

例如，下表中显示 `rate_change` 的可能值为 `integer` 和 `fixed_fractional`。C 程序中使用的值应为 `rate_change = hls::ip_fir::integer` 和 `rate_change = hls::ip_fir::fixed_fractional`。

FIR 构造体参数值

下表涵盖了 FIR IP 的所有特性和功能。本表中未描述的特性和功能在 Vivado HLS 实现中均不予支持。

表 31: FIR 构造体参数值

参数	C 语言类型	默认值	有效值
input_width	无符号	16	无限制
input_fractional_bits	无符号	0	受 input_width 大小所限
output_width	无符号	24	无限制
output_fractional_bits	无符号	0	受 output_width 大小所限
coeff_width	无符号	16	无限制
coeff_fractional_bits	无符号	0	受 coeff_width 大小所限

表 31：FIR 构造体参数值 (续)

参数	C 语言类型	默认值	有效值
num_coeffs	bool	21	Full
coeff_sets	无符号	1	1-1024
input_length	无符号	21	无限制
output_length	无符号	21	无限制
num_channels	无符号	1	1-1024
total_num_coeff	无符号	21	num_coeffs * coeff_sets
coeff_vec[total_num_coeff]	双精度数组	无	不适用
filter_type	无符号	single_rate	single_rate、interpolation、decimation、hilbert_filter 或 interpolated
rate_change	无符号	integer	integer 或 fixed_fractional
interp_rate	无符号	1	1-1024
decim_rate	无符号	1	1-1024
zero_pack_factor	无符号	1	1-8
rate_specification	无符号	period	frequency 或 period
hardware_oversampling_rate	无符号	1	无限制
sample_period	bool	1	无限制
sample_frequency	无符号	0.001	无限制
quantization	无符号	integer_coefficients	integer_coefficients、quantize_only 或 maximize_dynamic_range
best_precision	无符号	false	false true
coeff_structure	无符号	non_symmetric	inferred、non_symmetric、symmetric、negative_symmetric、half_band 或 hilbert
output_rounding_mode	无符号	full_precision	full_precision、truncate_lsbs、non_symmetric_rounding_down、non_symmetric_rounding_up、symmetric_rounding_to_zero、symmetric_rounding_to_infinity、convergent_rounding_to_even 或 convergent_rounding_to_odd
filter_arch	无符号	systolic_multiply_accumulate	systolic_multiply_accumulate 或 transpose_multiply_accumulate
optimization_goal	无符号	area	area 或 speed
inter_column_pipe_length	无符号	4	1-16
column_config	无符号	1	受所用 DSP48 数量所限
config_method	无符号	single	single 或 by_channel

表 31：FIR 构造体参数值 (续)

参数	C 语言类型	默认值	有效值
coeff_padding	bool	false	false true

使用 FIR 函数

FIR 函数在 HLS 名称空间内定义，可按如下方式调用：

```
// Create an instance of the FIR
static hls::FIR<STATIC_PARAM> fir1;
// Execute the FIR instance fir1
fir1.run(INPUT_DATA_ARRAY, OUTPUT_DATA_ARRAY);
```

STATIC_PARAM 属于静态参数化结构体，用于为 FIR 定义大部分静态参数。

输入数据和输出数据都作为数组（INPUT_DATA_ARRAY 和 OUTPUT_DATA_ARRAY）提供给函数。在最终实现中，FIR IP 上的这些端口将作为 AXI4-Stream 端口来实现。赛灵思建议始终在使用数据流最优化（set_directive_dataflow）的区域内使用 FIR 函数，因为这样可确保数组作为流传输数组来实现。另一种方法是使用 set_directive_stream 命令将这两个数组都指定为流传输数组。



重要提示！ FIR 无法在流水线化的区域内使用。如需高性能运算，请在使用 FIR 前后分别将循环或函数流水线化，然后对区域内的所有循环和函数使用数据流最优化。

通过在单一输入和单一输出数组内进行数据交织来支持 FIR 的多通道功能。

- 输入数组的大小应足以保存所有样本：num_channels * input_length。
- 输出数据大小应指定为可包含所有输出样本：num_channels * output_length。

以下代码示例用于演示 2 个通道的数据交织方式。在此示例中，顶层函数包含 2 个通道的输入数据（din_i 和 din_q）以及 2 个通道的输出数据（dout_i 和 dout_q）。位于前端 (fe) 和后端 (be) 的 2 个函数用于对 FIR 输入数组中的数据进行正确排序并从 FIR 输出数组中抽取该数据。

```
void dummy_fe(din_t din_i[LENGTH], din_t din_q[LENGTH], din_t
out[FIR_LENGTH]) {
    for (unsigned i = 0; i < LENGTH; ++i) {
        out[2*i] = din_i[i];
        out[2*i + 1] = din_q[i];
    }
}
void dummy_be(dout_t in[FIR_LENGTH], dout_t dout_i[LENGTH], dout_t
dout_q[LENGTH]) {
    for(unsigned i = 0; i < LENGTH; ++i) {
        dout_i[i] = in[2*i];
        dout_q[i] = in[2*i+1];
    }
}
void fir_top(din_t din_i[LENGTH], din_t din_q[LENGTH],
dout_t dout_i[LENGTH], dout_t dout_q[LENGTH]) {

    din_t fir_in[FIR_LENGTH];
    dout_t fir_out[FIR_LENGTH];
    static hls::FIR<myconfig> fir1;
```

```
dummy_fe(din_i, din_q, fir_in);
firl.run(fir_in, fir_out);
dummy_be(fir_out, dout_i, dout_q);
}
```

可选 FIR 运行时间配置

在某些操作模式下，FIR 需要额外输入以配置系数使用方式。要获取有关哪些模式需使用此输入配置的完整说明，请参阅《FIR Compiler LogiCORE IP 产品指南》(PG149)。

此输入配置可在 C 语言代码中通过使用标准 `ap_int.h` 8 位数据类型来执行。在此示例中，头文件 `fir_top.h` 可指定使用 FIR 和 `ap_fixed` 库、定义设计参数值的数量，然后基于以下条件定义部分定点类型：

```
#include "ap_fixed.h"
#include "hls_fir.h"

const unsigned FIR_LENGTH = 21;
const unsigned INPUT_WIDTH = 16;
const unsigned INPUT_FRACTIONAL_BITS = 0;
const unsigned OUTPUT_WIDTH = 24;
const unsigned OUTPUT_FRACTIONAL_BITS = 0;
const unsigned COEFF_WIDTH = 16;
const unsigned COEFF_FRACTIONAL_BITS = 0;
const unsigned COEFF_NUM = 7;
const unsigned COEFF_SETS = 3;
const unsigned INPUT_LENGTH = FIR_LENGTH;
const unsigned OUTPUT_LENGTH = FIR_LENGTH;
const unsigned CHAN_NUM = 1;
typedef ap_fixed<INPUT_WIDTH, INPUT_WIDTH - INPUT_FRACTIONAL_BITS> s_data_t;
typedef ap_fixed<OUTPUT_WIDTH, OUTPUT_WIDTH - OUTPUT_FRACTIONAL_BITS>
m_data_t;
typedef ap_uint<8> config_t;
```

在顶层代码中，已包含头文件中的信息，使用用于指定位宽的常量值来创建静态参数化结构体，从而确保 C 语言代码与 FIR 配置相匹配并且已指定系数。在顶层代码中，头文件中定义为 8 位数据的输入配置将传递到 FIR 中。

```
#include "fir_top.h"

struct param1 : hls::ip_fir::params_t {
    static const double coeff_vec[total_num_coeff];
    static const unsigned input_length = INPUT_LENGTH;
    static const unsigned output_length = OUTPUT_LENGTH;
    static const unsigned num_coeffs = COEFF_NUM;
    static const unsigned coeff_sets = COEFF_SETS;
};

const double param1::coeff_vec[total_num_coeff] =
    {6,0,-4,-3,5,6,-6,-13,7,44,64,44,7,-13,-6,6,5,-3,-4,0,6};

void dummy_fe(s_data_t in[INPUT_LENGTH], s_data_t out[INPUT_LENGTH],
              config_t* config_in, config_t* config_out)
{
    *config_out = *config_in;
    for(unsigned i = 0; i < INPUT_LENGTH; ++i)
        out[i] = in[i];
}

void dummy_be(m_data_t in[OUTPUT_LENGTH], m_data_t out[OUTPUT_LENGTH])
```

```

{
    for(unsigned i = 0; i < OUTPUT_LENGTH; ++i)
        out[i] = in[i];
}

// DUT
void fir_top(s_data_t in[INPUT_LENGTH],
            m_data_t out[OUTPUT_LENGTH],
            config_t* config)
{
    s_data_t fir_in[INPUT_LENGTH];
    m_data_t fir_out[OUTPUT_LENGTH];
    config_t fir_config;
    // Create struct for config
    static hls::FIR<param1> fir1;

    //=====
    // Dataflow process
    dummy_fe(in, fir_in, config, &fir_config);
    fir1.run(fir_in, fir_out, &fir_config);
    dummy_be(fir_out, out);
    //=====
}

```

在 Vivado HLS 示例中提供了使用 FIR C 语言库的设计示例，可使用以下菜单选项来访问这些示例：“Help” → “Welcome” → “Open Example Project” → “Design Examples” → “FIR”。

DDS IP 库

您可在 C++ 设计内通过使用 `hls_dds.h` 库来使用赛灵思 Direct Digital Synthesizer (DDS) IP 块。本节解释了如何在 C++ 代码中配置 DDS IP。



建议：赛灵思强烈建议您参阅《LogiCORE IP DDS Compiler 产品指南》(PG141)，以获取有关如何实现和使用此 IP 的功能的信息。



重要提示！ DDS IP 核的 C IP 实现支持针对 `Phase_Increment` 与 `Phase_Offset` 参数采用混合模式，并且支持针对 `Phase_Offset` 采用 `none` 模式，但不支持针对这些参数采用 `programmable` 和 `streaming` 模式。

要在 C++ 代码中使用 DDS，请执行以下操作：

1. 在代码中包含 `hls_dds.h` 库。
2. 使用预定义的 `hls::ip_dds::params_t` 结构体来设置默认参数。
3. 调用 DDS 函数。

首先，在源代码中包含 DDS 库。此头文件位于 Vivado HLS 安装区域内的 `include` 目录中，执行 Vivado HLS 时会自动搜索该目录。

```
#include "hls_dds.h"
```

定义 DDS 的静态参数。例如，定义相位宽度、时钟速率和相位与增量偏移。DDS C 语言库包含 `hls::ip_dds::params_t` 参数化结构体，它可用于初始化含默认值的所有静态参数。通过重新定义此结构体中的任意值，即可自定义实现。

以下示例显示了如何使用用户定义的 `param1` 结构体覆盖相位宽度、时钟速率、相位偏移以及通道数量的默认值，此结构基于现有预定义的 `hls::ip_dds::params_t` 结构体。

```
struct param1 : hls::ip_dds::params_t {
    static const unsigned Phase_Width = PHASEWIDTH;
    static const double DDS_Clock_Rate = 25.0;
    static const double PINC[16];
    static const double POFF[16];
};
```

使用含已定义的静态参数（例如，`param1`）的 HLS 名称空间创建 DDS 函数实例。然后，使用运行方法调用并执行该函数。以下是按顺序显示的数据和相位函数实参：

```
static hls::DDS<config1> dds1;
dds1.run(data_channel, phase_channel);
```

要访问使用 DDS C 语言库的设计示例，请选择 “Help” → “Welcome” → “Open Example Project” → “Design Examples” → “DDS”。

DDS 静态参数

DDS 的静态参数定义了如何配置 DDS，例如时钟速率、相位间隔和模式。`hls_dds.h` 头文件可定义 `hls::ip_dds::params_t` 结构体，用于设置静态参数的默认值。要使用默认值，可以将参数化结构体直接与 DDS 函数一起使用。

```
static hls::DDS< hls::ip_dds::params_t > dds1;
dds1.run(data_channel, phase_channel);
```

下表描述了 `hls::ip_dds::params_t` 参数化结构体的参数。



建议：赛灵思强烈建议您查看《DDS 编译器 LogiCORE IP 产品指南》(PG141)，以了解有关参数和值的详细信息。

表 32：DDS 结构体参数

参数	描述
<code>DDS_Clock_Rate</code>	指定 DDS 输出的时钟速率。
<code>Channels</code>	指定通道数。DDS 和相位发生器最多可支持 16 个通道。这些通道属时分复用通道，可降低每个通道的有效时钟频率。
<code>Mode_of_Operation</code>	指定以下操作模式之一： 标准模式，当累加的相位用于访问 SIN/COS LUT 之前可被截断的情况下使用。 栅格化模式，当所需频率和系统时钟通过有理分式关联时使用。
<code>Modulus</code>	描述系统时钟频率和所需频率之间的关系。 仅在栅格化模式下使用此参数。
<code>Spurious_Free_Dynamic_Range</code>	指定 DDS 所产生音调的目标纯度。
<code>Frequency_Resolution</code>	以 Hz 为单位指定最小频率分辨率，并确定相位累加器使用的相位宽度，包括相关的相位增量 (PINC) 和相位偏移 (POFF) 值。
<code>Noise_Shaping</code>	用于控制是使用相位截断、抖动还是泰勒级数校正。

表 32: DDS 结构体参数 (续)

参数	描述
Phase_Width	设置以下各项的宽度： m_axis_phase_tdata 中的“PHASE_OUT”字段 s_axis_phase_tdata 中的“Phase”字段（当 DDS 配置为仅 SIN/COS LUT 时） 相位累加器 相关的相位增量和偏移寄存器 s_axis_config_tdata 中的“Phase”字段 对于栅格化模式，相位宽度固定为描述有效输入范围 $[0, \text{Modulus}-1]$ 所需的位数，即 $\log_2 (\text{Modulus}-1)$ 取整。
Output_Width	设置 m_axis_data_tdata 中的“SINE”和“COSINE”字段的宽度。此参数提供的 SFDR 取决于所选的“Noise Shaping”选项。
Phase_Increment	选择相位增量值。
Phase_Offset	选择相位偏移值。
Output_Selection	在 m_axis_data_tdata 中将输出选项设置为“SINE”和/或“COSINE”。
Negative_Sine	在运行时间对“SINE”字段取反。
Negative_Cosine	在运行时间对“COSINE”字段取反。
Amplitude_Mode	将波幅设置为全幅或单位圆。
Memory_Type	用于控制 SIN/COS LUT 的实现。
Optimization_Goal	用于控制实现决策的目标是最高速度还是最低资源。
DSP48_Use	用于控制相位偏移和/或抖动噪声加法的相位累加器和解决方案阶段的实现。
Latency_Configuration	根据最优化目标设置核达到最佳值的时延。
Latency	指定手动时延值。
Output_Form	将输出形式设置为 2 的补码或符号和量级。通常，SINE 和 COSINE 的输出为 2 的补码形式。但是，使用象限对称时，可以将输出形式更改为符号和量级。
PINC[XIP_DDS_CHANNELS_MAX]	设置每个输出通道的相位增量值。
POFF[XIP_DDS_CHANNELS_MAX]	设置每个输出通道的相位偏移值。

DDS 接口参数值

下表显示了 hls::ip_dds::params_t 参数化结构体参数的可能值。

表 33: DDS 接口参数值

参数	C 语言类型	默认值	有效值
DDS_Clock_Rate	双精度	20.0	任意双精度值
Channels	无符号	1	1 - 16
Mode_of_Operation	无符号	XIP_DDS_MOO_CONVENTIONAL	XIP_DDS_MOO_CONVENTIONAL 用于截断累加的相位。 XIP_DDS_MOO_RASTERIZED 用于选择栅格化模式。
Modulus	无符号	200	129 - 256

表 33: DDS 接口参数值 (续)

参数	C 语言类型	默认值	有效值
Spurious_Free_Dynamic_Range	双精度	20.0	18.0 - 150.0
Frequency_Resolution	双精度	10.0	0.000000001 - 125000000
Noise_Shaping	无符号	XIP_DDS_NS_NONE	XIP_DDS_NS_NONE 生成相位截断 DDS。 XIP_DDS_NS_DITHER 使用相位抖动来改进 SFDR，但会造成噪声本底增加。 XIP_DDS_NS_TAYLOR 使用其它情况下通过相位截断丢弃的位来插入正弦/余弦值 XIP_DDS_NS_AUTO 自动判定噪声成形。
Phase_Width	无符号	16	必须为 8 的倍数（整数）
Output_Width	无符号	16	必须为 8 的倍数（整数）
Phase_Increment	无符号	XIP_DDS_PINCPOFF_FIXED	XIP_DDS_PINCPOFF_FIXED 在生成时修复 PINC，而 PINC 在运行时间无法更改。 这是受支持的唯一值。
Phase_Offset	无符号	XIP_DDS_PINCPOFF_NONE	XIP_DDS_PINCPOFF_NONE 不生成相位偏移。 XIP_DDS_PINCPOFF_FIXED 在生成时修复 POFF，而 POFF 在运行时间无法更改。
Output_Selection	无符号	XIP_DDS_OUT_SIN_AND_COS	XIP_DDS_OUT_SIN_ONLY 仅生成正弦输出。 XIP_DDS_OUT_COS_ONLY 仅生成余弦输出。 XIP_DDS_OUT_SIN_AND_COS 可生成正弦输出和余弦输出。
Negative_Sine	无符号	XIP_DDS_ABSENT	XIP_DDS_ABSENT 用于生成标准正弦波。 XIP_DDS_PRESENT 用于抵消正弦波。
Negative_Cosine	bool	XIP_DDS_ABSENT	XIP_DDS_ABSENT 用于生成标准正弦波。 XIP_DDS_PRESENT 用于抵消正弦波。
Amplitude_Mode	无符号	XIP_DDS_FULL_RANGE	XIP_DDS_FULL_RANGE 首先使用二进制小数点将波幅归一化为输出宽度。例如，8 位输出的二进制波幅为 100000000 - 10（对应 01111110 与 11111110 之间值），分别对应于小于 1 和大于 -1。 XIP_DDS_UNIT_CIRCLE 将波幅归一化到半满范围，即值范围为从 01000 ..(+0.5). 到 110000 .. (-0.5).

表 33: DDS 接口参数值 (续)

参数	C 语言类型	默认值	有效值
Memory_Type	无符号	XIP_DDS_MEM_AUTO	对于小规模案例（例如，如果该表可包含在单层内存中），XIP_DDS_MEM_AUTO 会选择分布式 ROM，对于大型案例则选择块 ROM。 XIP_DDS_MEM_BLOCK 始终使用块 RAM。 XIP_DDS_MEM_DIST 始终使用分布式 RAM。
Optimization_Goal	无符号	XIP_DDS_OPTGOAL_AUTO	XIP_DDS_OPTGOAL_AUTO 会自动选择最优优化目标。 XIP_DDS_OPTGOAL_AREA 针对面积进行最优优化。 XIP_DDS_OPTGOAL_SPEED 针对性能进行最优优化。
DSP48_Use	无符号	XIP_DDS_DSP_MIN	XIP_DDS_DSP_MIN 用于在 FPGA 逻辑内实现相位累加器以及相位偏移和/或抖动噪声添加阶段。 XIP_DDS_DSP_MAX 用于使用 DSP 切片来实现相位累加器以及相位偏移和/或抖动噪声添加阶段。对于单通道，DSP 切片还可提供寄存器用于存储可编程相位增量和/或相位偏移，从而节省更多结构资源。
Latency_Configuration	无符号	XIP_DDS_LATENCY_AUTO	XIP_DDS_LATENCY_AUTO 自动判定时延。 XIP_DDS_LATENCY_MANUAL 使用“时延 (Latency)”选项手动指定时延。
Latency	无符号	5	任意值
Output_Form	无符号	XIP_DDS_OUTPUT_TWOS	XIP_DDS_OUTPUT_TWOS 输出二进制补码。 XIP_DDS_OUTPUT_SIGN_MAG 输出有符号的量级。
PINC[XIP_DDS_CHANNELS_MAX]	无符号的数组	{0}	对应每个通道的相位增量的任意值
POFF[XIP_DDS_CHANNELS_MAX]	无符号的数组	{0}	对应每个通道的相位偏移的任意值

SRL IP 库

使用 C 语言编写代码可满足几种不同的要求：复用性、可读性和性能。但在目前情况下，使用 C 语言编写代码难以在进行高层次综合之后获得最理想的硬件。

与对复用性、可读性和性能的要求一样，某些编码技巧或预定义的结构可以确保综合输出产生更优化的硬件，或者以 C 语言更好地对硬件建模，从而使算法更易于验证。

直接映射到 SRL 资源中

许多 C 语言算法都会在数组间按顺序移动数据。此类算法会在数组开始处添加新的值、在数组中移动现有数据，并删除最早的数据值。此操作在硬件中是通过移位寄存器来实现的。

这种将移位寄存器从 C 实现到硬件中的最常用的方式会将数组完全分区为各元素，并支持 RTL 中元素间的数据依赖关系暗示需使用移位寄存器。

逻辑综合通常会将 RTL 移位寄存器实现到赛灵思 SRL 资源中，这样即可有效实现移位寄存器。问题在于有时逻辑综合不使用 SRL 组件来实现 RTL 移位寄存器。

- 在移位寄存器中间访问数据时，逻辑综合无法直接推断 SRL。
- 有时，即使 SRL 处于理想状态，逻辑综合也可能因其它因素而在触发器中实现移位寄存器。（逻辑综合同样是一个复杂的过程）。

Vivado HLS 可提供了 1 个 C++ 类 (`ap_shift_reg`) 用于确保 C 语言代码中定义的移位寄存器始终使用 SRL 资源来实现。`ap_shift_reg` 类采用 2 种方法来执行 SRL 组件支持的各种读写访问。

从移位器中读取

读取方法允许从移位器寄存器读取指定的位置。

Vivado HLS 还包含 `ap_shift_reg.h` 头文件作为独立封装，用于定义 `ap_shift_reg` 类。您有权在自己的源代码中使用此头文件。软件包 `xilinx_hls_lib_<release_number>.tgz` 位于 Vivado HLS 安装位置的 `include` 目录中。

```
// Include the Class
#include "ap_shift_reg.h"

// Define a variable of type ap_shift_reg<type, depth>
// - Sreg must use the static qualifier
// - Sreg will hold integer data types
// - Sreg will hold 4 data values
static ap_shift_reg<int, 4> Sreg;
int var1;

// Read location 2 of Sreg into var1
var1 = Sreg.read(2);
```

数据的读取、写入和移位

`shift` 方法允许执行读取、写入和移位操作。

```
// Include the Class
#include "ap_shift_reg.h"

// Define a variable of type ap_shift_reg<type, depth>
// - Sreg must use the static qualifier
// - Sreg will hold integer data types
// - Sreg will hold 4 data values
static ap_shift_reg<int, 4> Sreg;
int var1;
```

```
// Read location 3 of Sreg into var1
// THEN shift all values up one and load In1 into location 0
var1 = Sreg.shift(In1,3);
```

读、写和使能移位

shift 方法还支持使能输入，以支持通过变量来对移位进程进行控制和使能。

```
// Include the Class
#include "ap_shift_reg.h"

// Define a variable of type ap_shift_reg<type, depth>
// - Sreg must use the static qualifier
// - Sreg will hold integer data types
// - Sreg will hold 4 data values
static ap_shift_reg<int, 4> Sreg;
int var1, In1;
bool En;

// Read location 3 of Sreg into var1
// THEN if En=1
// Shift all values up one and load In1 into location 0
var1 = Sreg.shift(In1,3,En);
```

使用 ap_shift_reg 类时，Vivado HLS 会为每个移位器创建唯一的 RTL 组件。执行逻辑综合时，该组件会综合到 SRL 资源内。

HLS 线性代数库

HLS 线性代数库可提供多个常用的 C++ 线性代数函数。HLS 线性代数库中的函数全都使用二维数组来表示矩阵，下表中列出了这些函数。

表 34：HLS 线性代数库

函数	数据类型	实现样式
cholesky	float ap_fixed x_complex<float> x_complex<ap_fixed>	已综合
cholesky_inverse	float ap_fixed x_complex<float> x_complex<ap_fixed>	已综合
matrix_multiply	float ap_fixed x_complex<float> x_complex<ap_fixed>	已综合

表 34: HLS 线性代数库 (续)

函数	数据类型	实现样式
qrf	float x_complex<float>	已综合
qr_inverse	float x_complex<float>	已综合
svd	float x_complex<float>	已综合

线性代数函数全都使用二维数组来表示矩阵。针对实数和复数数据，所有函数都支持单精度浮点 (float) 输入。针对实数和复数数据，一小部分函数支持 `ap_fixed` (定点) 输入。`ap_fixed` 类型的精度和舍入行为必须由用户根据需要定义。

使用线性代数库

您可以通过以下任一方法来引用 HLS 线性代数函数：

- 使用限定作用域的命名：

```
#include "hls_linear_algebra.h"

hls::cholesky(In_Array, Out_Array);
```

- 使用 `hls` 名称空间：

```
#include "hls_linear_algebra.h"
using namespace hls; // Namespace specified after the header files

cholesky(In_Array, Out_Array);
```

线性代数函数最优化

使用线性代数函数时，必须判定 RTL 实现的最优化级别。最优化的级别和类型取决于 C 语言代码的编写方式以及对 C 语言代码应用 Vivado HLS 指令的方式。

为简化最优化流程，Vivado HLS 提供了线性代数库函数，其中包含多个 C 语言代码架构和嵌入式最优化指令。通过使用 C++ 配置类，您可选择要使用的 C 语言代码和要应用的最优化指令。

虽然具体最优化操作因函数而异，但配置类通常允许您为 RTL 实现指定最优化级别，如下所示：

- “Small”：少量资源，吞吐量较低
- “Balanced”：折衷平衡资源和吞吐量
- “Fast”：吞吐量更高，占用更多资源

Vivado HLS 提供了工程示例用于展示针对线性代数库中的每个函数如何使用配置类。您可将这些示例用作为模板，以了解如何为每个函数配置 Vivado HLS 以达成具体实现目标。每个示例都提供 1 个 C++ 源文件，其中包含多个 C 语言代码架构作为不同的 C++ 函数。

注释：为识别顶层 C++ 函数，请在 `directives.tcl` 文件或 Vivado HLS GUI 的“Directive”选项卡中查找 TOP 指令。

您可从 Vivado HLS Welcome 屏幕中打开这些示例：

1. 单击“Open Example Project”。
2. 在“Examples”对话框中，展开“Design Examples” → “linear_algebra” → “implementation_targets”。

注释：调用 Vivado HLS GUI 时会显示“Welcome”页面。您可随时选择“Help” → “Welcome”来访问此页面。

为判定最适合您的设计的最优化方法，您可使用 Vivado HLS 的“Compare Reports”功能来比较每个解决方案的性能和利用率估算结果。为比较估算结果，您必须选择“Solution” → “Run C Synthesis” → “All Solutions”来为所有工程解决方案运行综合。然后，使用“”工具栏按钮。

Cholesky

实现控制

下表汇总了影响资源利用率、函数吞吐量（启动时间间隔）和函数时延的主要因子。值“低”、“中”和“高”均为相对于其它关键因子的值。

表 35: Cholesky 关键因子汇总

关键因子	值	资源	吞吐量	时延
架构 (ARCH)	0	低	低	高
	1	中	中	中
	2	高	高	低
内部循环流水线化 (INNER_LI)	1	高	高	低
	>1	低	低	高
展开内部循环 (UNROLL_FACTOR)	1	低	低	高
	>1	高	高	低

关键因子

- 架构
 - 0: 使用最低 DSP 利用率和最低吞吐量。
 - 1: 使用较高的 DSP 利用率，增大吞吐量但最大限度降低内存利用率。该值不支持展开内部循环以进一步增大吞吐量。
 - 2: 使用最高 DSP 和内存利用率。该值支持展开内部循环以提高整体吞吐量，少量增加 DSP 资源。这是适合设计探索的最灵活的架构。
- 内部循环流水线化
 - >1: 对于 ARCH 2，支持 Vivado HLS 进行资源共享并降低 DSP 利用率。使用复数浮点数据类型时，将值设置为 2 或 4 可显著降低 DSP 利用率。
- 展开内部循环
 - 对于 ARCH 2，按指定因子复制实现循环处理所需的硬件、并行执行对应数量的循环迭代，增大吞吐量，同时增大 DSP 和内存利用率。

规格

您可使用衍生自以下 `hls::cholesky_traits` 基本类的配置类并重新定义相应的类成员，以指定所有因子：

```
struct MY_CONFIG :
hls::cholesky_traits<LOWER_TRIANGULAR, ROWS_COLS_A, MAT_IN_T, MAT_OUT_T>{
    static const int ARCH = 2;
    static const int INNER_II = 2;
    static const int UNROLL_FACTOR = 1;
};
```

此配置类将作为模板参数提供给 `hls::cholesky_top` 函数，如下所示：

```
hls::cholesky_top<LOWER_TRIANGULAR, ROWS_COLS_A, MY_CONFIG, MAT_IN_T, MAT_OUT_T>
>(A, L);
```

`hls::cholesky` 函数使用以下默认配置：

```
hls::cholesky<LOWER_TRIANGULAR, ROWS_COLS_A, MAT_IN_T, MAT_OUT_T>(A, L);
```

Cholesky 求逆和 QR 求逆

实现控制

下表汇总了影响资源利用率、函数吞吐量（启动时间间隔）和函数时延的主要因子。值“低”、“中”和“高”均为相对于其它关键因子的值。

表 36：反转关键因子汇总

关键因子	值	资源	吞吐量	时延
子函数实现目标 (Cholesky/QRF 和矩阵乘法)	Small	低	低	高
	Balanced	中	中	中
	Fast	高	高	低
回代内部循环和对角线 循环流水线	1	高	高	低
	>1	低	低	高
DATAFLOW 指令	支持	中	高	高
INLINE 指令	支持	低	低	高

关键因子

以下是有关上表中显示的关键因子的其它信息：

- 子函数实现
 - 利用下列子函数（按顺序执行）：Cholesky 或 QRF、回代和矩阵乘法。为这些子函数选择的实现决定了反函数的资源利用率和函数吞吐量/时延。
- 回代内部循环和对角线循环流水线
 - >1：支持 Vivado HLS 进行资源共享并降低 DSP 利用率。

- DATAFLOW 指令
 - 用于将顺序任务流水线化，根据最大子函数时延（而不是各子函数时延的总和）将函数吞吐量增大至启动时间间隔。函数吞吐量随着总体时延的增加而大幅增加。需要其它内存资源。
- INLINE 指令
 - 用于移除子函数层级结构，使 Vivado HLS 可以更好地共享资源，并可以降低 DSP 和内存利用率。



提示：您可以通过将 DATAFLOW 指令与适当的子函数实现组合来调整反函数的资源和吞吐量以满足特定要求。

规格

DATAFLOW 指令应用于 `hls::cholesky_inverse_top` 或 `hls::qr_inverse_top` 函数，如下所示：

```
set_directive_dataflow "cholesky_inverse_top"
```

INLINE 指令的应用方式相同：

```
set_directive_inline -recursive "cholesky_inverse_top"
```

您可使用衍生自以下 `hls::cholesky_inverse_traits` 或 `hls::qr_inverse_traits` 基本类的配置类并重新定义相应的类成员，以指定各子函数实现：

```
typedef hls::cholesky_inverse_traits<ROWS_COLS_A,
    MAT_IN_T,
    MAT_OUT_T> MY_DFLT_CFG;

struct MY_CONFIG : MY_DFLT_CFG {
    struct CHOLESKY_TRAITS :
        hls::cholesky_traits<false,
            ROWS_COLS_A,
            MAT_IN_T,
            MY_DFLT_CFG::CHOLESKY_OUT> {
        static const int ARCH = 1;
    };
    struct BACK_SUB_CONFIG :
        hls::back_substitute_traits<ROWS_COLS_A,
            MY_DFLT_CFG::CHOLESKY_OUT,
            MY_DFLT_CFG::BACK_SUBSTITUTE_OUT> {
        static const int INNER_II = 2;
        static const int DIAG_II = 2;
    };
    struct MULTIPLIER_CONFIG :
        hls::matrix_multiply_traits<hls::NoTranspose,
            hls::ConjugateTranspose,
            ROWS_COLS_A,
            ROWS_COLS_A,
            ROWS_COLS_A,
            ROWS_COLS_A,
            MY_DFLT_CFG::BACK_SUBSTITUTE_OUT,
            MAT_OUT_T> {
        static const int INNER_II = 2;
    };
};
```

此配置类将作为模板参数提供给 `hls::cholesky_inverse_top` 或 `hls::qr_inverse_top` 函数，如下所示：

```
hls::cholesky_inverse_top<ROWS_COLS_A, MY_CONFIG, MAT_IN_T, MAT_OUT_T>(A, INVERSE_A, inverse_OK);
```

`hls::cholesky_inverse` 或 `hls::qr_inverse` 函数使用以下默认配置：

```
hls::cholesky_inverse<ROWS_COLS_A, MAT_IN_T, MAT_OUT_T>(A, INVERSE_A, inverse_OK);
```

矩阵乘法

实现控制

下表汇总了影响资源利用率、函数吞吐量（启动时间间隔）和函数时延的主要因子。值“低”、“中”和“高”均为相对于其它关键因子的值。

表 37：矩阵乘法关键因子汇总

关键因子	值	资源	吞吐量	时延
架构 (ARCH)	2（浮点）	低	低	高
	3（浮点）	高	高	低
	0（定点）	低	低	高
	2（定点）	中	中	中
	4（定点）	高	高	低
内部循环流水线化 (INNER_II)	1	高	高	低
	>1	低	低	高
展开内部循环 (UNROLL_FACTOR)	1	低	低	高
	>1	高	高	低
资源指令 (RESOURCE)	LUTRAM	中	不适用	不适用

关键因子

· 架构

ARCH 关键因子可根据实现数据类型来选择架构。

○ 浮点数据类型

- 2：确保内层累加循环以 II 值 1 即可达成最大吞吐量。该值支持部分展开内部循环以提高整体吞吐量，少量增加 DSP 资源。
- 3：实现完全展开的内层累加循环，此循环使用最大数量的 DSP 资源和最高的吞吐量。

○ 定点数据类型

- 0：使用最少量资源和最低吞吐量。
- 2：支持部分展开内部循环以提高整体吞吐量，少量增加 DSP 资源。
- 4：实现完全展开的内层累加循环，此循环使用最大数量的 DSP 资源和最高的吞吐量。

- 内部循环流水线化
 - >1: 使用复数浮点数据类型时，共享资源并降低 DSP 利用率。将该值设置为 2 或 4 可显著降低 DSP 利用率。
- 展开内部循环
 - 对于 ARCH 2，按指定因子复制实现循环处理所需的硬件、并行执行对应数量的循环迭代，增大吞吐量，同时增大 DSP 和内存利用率。
 - 对于 ARCH 3 或 4，完全展开累加循环。
- 资源指令

默认情况下，Vivado HLS 使用块 RAM 来实现数组。

 - 对于 ARCH 2，部分展开累加循环会导致 Vivado HLS 将 `sum_mult` 数组拆分到多个块 RAM 中。
 - 当分区大小无需使用一整个块 RAM 时，请使用 `RESOURCE` 指令来指定 LUTRAM。

规格

除 `RESOURCE` 指令外，您可使用衍生自以下 `hls::matrix_multiply_traits` 基本类的配置类并重定义相应的类成员，以指定所有因子：

```
struct MY_CONFIG: hls::matrix_multiply_traits<hls::NoTranspose,
hls::NoTranspose,
A_ROWS,
A_COLS,
B_ROWS,
B_COLS,
MATRIX_T,
MATRIX_T>{
static const int ARCH          = 2;
static const int INNER_II     = 1;
static const int UNROLL_FACTOR = 2;
};
```

此配置类将作为模板参数提供给 `hls::matrix_multiply_top` 函数，如下所示：

```
hls::matrix_multiply_top<hls::NoTranspose,hls::NoTranspose,A_ROWS,A_COLS,B_ROWS,B_COLS,C_ROWS,C_COLS,MY_CONFIG,MATRIX_T,MATRIX_T>(A,B,C);
```

`hls::matrix_multiply` 函数使用以下默认配置：

```
hls::matrix_multiply<hls::NoTranspose,hls::NoTranspose,A_ROWS,A_COLS,B_ROWS,B_COLS,C_ROWS,C_COLS,MATRIX_T,MATRIX_T>(A,B,C);
```

如果选择 ARCH 2，`RESOURCE` 指令将应用于 `hls::matrix_multiply_alt2` 函数中的 `sum_mult` 数组，如下所示：

```
set_directive_resource -core RAM_S2P_LUTRAM "matrix_multiply_alt2" sum_mult
```

QRF

实现控制

下表汇总了影响资源利用率、函数吞吐量（启动时间间隔）和函数时延的主要因子。值“低”、“中”和“高”均为相对于其它关键因子的值。

表 38：QRF 关键因子汇总

关键因子	值	资源	吞吐量	时延
Q 和 R 更新循环流水线化 (UPDATE_II)	2	高	高	低
	>2	低	低	高
Q 和 R 更新循环展开 (UNROLL_FACTOR)	1	低	低	高
	>1	高	高	低
旋转循环流水线化 (CALC_ROT_II)	1	高	高	低
	>1	低	低	高

关键因子

以下是有关前表中关键因子的附加信息：

- Q 和 R 更新循环流水线化
 - 2：将可实现的最低启动时间间隔 (II) 设置为 2，这满足 Q 和 R 矩阵数组要求每个更新循环迭代 2 次写入的要求。
 - >2：支持 Vivado HLS 进一步进行资源共享并降低 DSP 利用率。使用复数浮点数据类型时，将值设置为 4 或 8 可显著降低 DSP 利用率。
- Q 和 R 更新循环展开
 - 按指定因子复制实现循环处理所需的硬件、并行执行对应数量的循环迭代，增大吞吐量，同时增大 DSP 和内存利用率。
- 旋转循环流水线化
 - 支持 Vivado HLS 进行资源共享并降低 DSP 利用率。

规格

您可使用衍生自以下 `hls::qrf_traits` 基本类的配置类并重新定义相应的类成员，以指定所有因子：

```
struct MY_CONFIG : hls::qrf_traits<A_ROWS,A_COLS,MAT_IN_T,MAT_OUT_T>{
    static const int CALC_ROT_II = 4;
    static const int UPDATE_II= 4;
    static const int UNROLL_FACTOR= 2;
};
```

此配置类将作为模板参数提供给 `hls::qrf_top` 函数，如下所示：

```
hls::qrf_top<TRANPOSED_Q,A_ROWS,A_COLS,MY_CONFIG,MAT_IN_T,MAT_OUT_T>(A,Q,R)
;
```

hls::qrf 函数使用以下默认配置：

```
hls::qrf<TRANPOSED_Q,A_ROWS,A_COLS,MAT_IN_T,MAT_OUT_T>(A,Q,R);
```

SVD

实现控制

下表汇总了影响资源利用率、函数吞吐量（启动时间间隔）和函数时延的主要因子。值“低”、“中”和“高”均为相对于其它关键因子的值。

表 39：SVD 关键因子汇总

关键因子	值	资源	吞吐量	时延
ALLOCATION 指令 (vm2x1_base 限制)	1	低	低	高
	>1	高	高	低
非对角线循环流水线化 (OFF_DIAG_II)	4	高	高	低
	>4	低	低	高
对角线循环流水线化 (DIAG_II)	1	高	高	低
	>1	低	低	高
迭代 (NUM_SWEEP)	<10	不适用	高	低
倒数平方根运算符	组合运算符	中	高	低

关键因子

以下是有关上表中的关键因子的其它信息：

- ALLOCATION 指令
 - 限制已实现的 2x1 矢量点积的数量。Vivado HLS 调度 SVD 函数以使用指定数量的 2x1 矢量点积内核。
 - 注释：**SVD 算法为计算密集型算法，对于复数数据类型尤其如此。ALLOCATION 指令是平衡资源利用率和吞吐量的最有效方法。
- 非对角线循环流水线化
 - 4：将最小可实现的启动间隔 (II) 设置为 4，它满足非对角线循环中每次迭代 4 次写入的 S、U 和 V 数组要求。
 - >4：支持 Vivado HLS 进一步共享资源并降低 DSP 利用率。
- 对角线循环流水线化
 - >1：支持 Vivado HLS 共享资源。
- 迭代

SVD 函数使用双侧 Jacobi 迭代法。

 - 10：设置默认的迭代次数。
 - <10：通过设置满足期望性能的最小迭代次数来最大化函数吞吐量。

- 倒数平方根运算符

- 确保时延远低于离散运算符的时延。

注释：默认情况下，Vivado HLS 不使用 `rsqrt` 组合运算符，而使用离散除法和 `sqrt` 运算符。选择 `-unsafe_math_optimizations` 编译器选项以支持使用 `rsqrt` 运算符。

规格

您可将 `ALLOCATION` 指令与 `INLINE` 指令组合在一起应用于 `hls::svd_pairs` 函数，如下所示：

```
set_directive_inline -off "vm2x1_base"
set_directive_allocation -limit 1 -type function "svd_pairs" vm2x1_base
```

您可选择 `-unsafe_math_optimizations` 编译器选项，如下所示：

```
config_compile -unsafe_math_optimizations
```

您可使用衍生自以下 `hls::svd_traits` 基本类的配置类并重新定义相应的类成员，以指定所有其它因子：

```
struct MY_CONFIG : hls::svd_traits<A_ROWS,A_COLS,MATRIX_IN_T,MATRIX_OUT_T>{
    static const int NUM_SWEEPS = 6;
    static const int DIAG_II = 4;
    static const int OFF_DIAG_II = 4;
};
```

此配置类将作为模板参数提供给 `hls::svd_top` 函数，如下所示：

```
hls::svd_top<A_ROWS,A_COLS,MY_CONFIG,MATRIX_IN_T,MATRIX_OUT_T>(A,S,U,V);
```

`hls::svd` 函数使用以下默认配置：

```
hls::svd<A_ROWS,A_COLS,MATRIX_IN_T,MATRIX_OUT_T>(A,S,U,V);
```

HLS DSP 库

HLS DSP 库包含构建块函数，以供在 C++ 内进行 DSP 系统建模，其中重点是 SDR 应用中使用的函数。下表显示了 HLS DSP 库中的函数。

表 40: HLS DSP 库

函数	数据类型	实现样式
atan2	输入: <code>std::complex< ap_fixed ></code> 输出: <code>ap_ufixed</code>	已综合
awgn	输入: <code>ap_ufixed</code> 输出: <code>ap_int</code>	已综合
cmpy	输入: <code>std::complex< ap_fixed ></code> 输出: <code>std::complex< ap_fixed ></code>	已综合
convolution_encoder	输入: <code>ap_uint</code> 输出: <code>ap_uint</code>	已综合

表 40: HLS DSP 库 (续)

函数	数据类型	实现样式
nco	输入: ap_uint 输出: std::complex< ap_int >	已综合
sqrt	输入: ap_ufixed, ap_int 输出: ap_ufixed, ap_int	已综合
viterbi_decoder	输入: ap_uint 输出: ap_uint	已综合

这些函数使用 Vivado HLS 固定精度类型 `ap_uint` 和 `ap_ufixed` 来根据需要描述输入和输出数据。这些函数具有最小可变接口类型用于最大限度提升灵活性。例如，具有简单吞吐量模型的函数（例如，输出 1 个采样，则同样输入 1 个采样）使用指针接口。执行速率变更的函数（如 `viterbi_decoder`）在接口上使用 `hls::stream` 类型。

您可复制现有库并使接口更复杂，例如，为任意函数的指针接口和 AXI4-Stream 接口创建 `hls::streams`。但复杂接口需要更多资源。

Vivado HLS 以模板化 C++ 类的形式提供大部分库元素，在头文件 (`hls_dsp.h`) 中通过构造函数、析构函数和运算符访问函数对这些类进行了完整的描述。

使用 DSP 库

您可以使用以下任一方法引用 DSP 函数：

- 使用限定作用域的命名：

```
#include <hls_dsp.h>
static hls::awgn<output_width> my_awgn(seed);
my_awgn(snr, noise);
```

- 使用 `hls` 名称空间：

```
#include <hls_dsp.h>
using namespace hls;
static awgn<output_width> my_awgn(seed);
my_awgn(snr, noise);
```

DSP 库中的函数包括作为源代码中编译指示的综合指令，这些指令用于指示 Vivado HLS 对函数进行综合以满足典型要求。这些函数已针对最大吞吐量进行了最优化，这是最常见的用例。例如，数组可能已完全分区以确保启动时间间隔始终为 1，而与模板参数配置无关。

您可移除现有的最优化或应用其它最优化，如下所示：

- 要对 DSP 函数应用最优化，请在 Vivado HLS GUI 中打开头文件 `hls_dsp.h`，然后执行以下操作之一：
 - 按 **Ctrl** 键并单击 “`#include "hls_dsp.h"`”
 - 使用 “Explorer” 窗格导航到使用 `Includes` 文件夹的文件。
- 要添加或移除最优化指令，请在 “Information” 窗格中打开头文件，然后使用 “Directives” 选项卡。

注释：如果将最优化添加为编译指示，则 Vivado HLS 会将最优化放置在库中，并在每次将头文件添加到设计时都应用该最优化。可能需要文件写入权限才能将最优化添加为编译指示。



提示：如果要修改函数以修改其 RTL 实现，请在库源代码中查找带有 `TIP` 前缀的注释，这些注释指示放置编译指示或应用指令的位置。

HLS SQL 库

SQL 库包含 C++ 中的 SQL 构建块函数。下表显示了 HLS SQL 库中的函数。

表 41：HLS SQL 库

函数	数据类型	注释
hls_alg::sha224	输入： hls::stream<unsigned char> 输出： hls::stream<unsigned char>	实现来自 SHA-2 系列的 SHA-224 算法。
hls_alg::sha256	输入： hls::stream<unsigned char> unsigned long long 输出： hls::stream<unsigned char>	实现来自 SHA-2 系列的 SHA-256 算法。
hls_alg::sort	输入： hls::stream<T> 输出： hls::stream<T>	实现双调排序 (Bitonic Sort) 算法。 T 为数据类型。

Vivado HLS 在 hls_db 名称空间中提供这些库元素作为 C++ 模板函数。要获取所有 SQL 函数的完整描述，请参阅第 4 章中的“HLS SQL 库函数”。

使用 SQL 库

您可使用以下方法来引用 SQL 函数：

```
#include <hls_alg.h>
hls_alg::sha256(in_stream, in_stream_depth, out_stream);
```

SQL 库中的函数包含综合指令作为源代码中的编译指示，用于指导 Vivado HLS 对函数进行综合，以便满足典型要求。

高层次综合编码样式

本章解释了如何将 C、C++ 和 SystemC 的各种构造综合到 FPGA 硬件实现中。



重要提示！ 术语“C 语言代码”在本指南中用于表示以 C、C++ 和 SystemC 语言编写的代码（除非另有特别说明）。

本指南中的编码示例包含在 Vivado® HLS 发布中。请使用以下任一方法来访问编码示例：

- 从 Welcome 屏幕上单击“Open Example Project”。
- **注释：**选择“Help”→“Welcome”即可随时查看 Welcome 屏幕。
- 进入 Vivado HLS 安装区域的 `examples/coding` 目录。

不受支持的 C 语言构造

虽然 Vivado® HLS 支持各种 C 语言，但部分构造不可综合，或者可能导致设计流程下游出错。本节探讨了为了对器件中的函数进行综合和实现而必须执行编码更改。

要执行综合：

- C 语言函数必须包含设计的完整功能。
- 无法通过对操作系统执行系统调用来执行任何功能。
- C 语言构造必须采用固定大小或受限大小。
- 这些构造必须以无歧义方式来实现。

系统调用

系统调用无法综合，因为系统调用操作与在运行 C 程序的操作系统上执行的某些任务有关。

Vivado® HLS 会忽略仅显示数据而对算法执行没有影响的常用系统调用，如 `printf()` 和 `fprintf(stdout,)`。一般情况下，系统调用无法综合，应在综合前从函数中移除。其它此类调用示例包括 `getc()`、`time()` 和 `sleep()`，这些操作都会对操作系统执行调用。

Vivado HLS 会在执行综合时定义 `__SYNTHESIS__` 宏。这样 `__SYNTHESIS__` 宏即可从设计中排除不可综合的代码。

注释：`__SYNTHESIS__` 宏仅限在要综合的代码中使用。请勿在测试激励文件中使用该宏，因为 C 语言仿真或 C RTL 协同仿真不会遵循其指示进行操作。



注意！ 您不得在代码中或者通过编译器选项来定义或取消定义 `__SYNTHESIS__` 宏，否则编译可能失败。

在以下代码示例中，来自子函数的中间结果将保存到硬盘上。__SYNTHESIS__ 宏用于确保综合期间忽略不可综合的文件写入。

```
#include "hier_func4.h"

int sumsub_func(dint_t *in1, dint_t *in2, dint_t *outSum, dint_t *outSub)
{
    *outSum = *in1 + *in2;
    *outSub = *in1 - *in2;
}

int shift_func(dint_t *in1, dint_t *in2, dout_t *outA, dout_t *outB)
{
    *outA = *in1 >> 1;
    *outB = *in2 >> 2;
}

void hier_func4(dint_t A, dint_t B, dout_t *C, dout_t *D)
{
    dint_t apb, amb;

    sumsub_func(&A,&B,&apb,&amb);
#ifdef __SYNTHESIS__
    FILE *fp1; // The following code is ignored for synthesis
    char filename[255];
    sprintf(filename,Out_apb_%03d.dat,apb);
    fp1=fopen(filename,w);
    fprintf(fp1, %d \n, apb);
    fclose(fp1);
#endif
    shift_func(&apb,&amb,C,D);
}
```

使用 __SYNTHESIS__ 宏即可轻松排除不可综合的代码，而无需从 C 语言函数中移除代码本身。使用此类宏并不意味着用于仿真的 C 语言代码与用于综合的 C 语言代码当前存在差异。



注意！ 如果使用 __SYNTHESIS__ 宏来更改 C 语言代码的功能，那么它可能导致 C 语言仿真与 C 语言综合之间出现不同结果。此类代码中的错误本身非常难以调试。请勿使用 __SYNTHESIS__ 宏来更改功能。

动态内存使用

管理系统内的内存分配的任何系统调用（如，malloc()、alloc() 和 free()）均使用操作系统的内存中存在的资源，并在运行时间内创建和发布：要对硬件实现进行综合，设计必须完全独立并指定所有必需的资源。

必须从设计代码中移除内存分配系统调用后才能进行综合。由于动态内存操作用于定义设计功能，因此必须转换为受同等限制的表示法。以下代码示例显示了如何将使用 malloc() 的设计变换为可综合版本，并突出显示了 2 项实用的编码样式技巧：

- 此设计不使用 __SYNTHESIS__ 宏。

用户定义的 NO_SYNTH 宏用于选择可综合版本或不可综合版本。这样可确保在 C 语言中仿真的代码与 Vivado® HLS 中综合的代码相同。

- 使用 malloc() 的原始设计中的指针无需重写即可配合固定大小的元素一起使用。

可创建固定大小的资源，并且可将现有指针直接设置为指向此固定大小的资源。此技巧可避免对现有设计进行手动重新编码。

```
#include "malloc_removed.h"
#include <stdlib.h>
//#define NO_SYNT

dout_t malloc_removed(din_t din[N], dsel_t width) {

#ifdef NO_SYNT
    long long *out_accum = malloc (sizeof(long long));
    int* array_local = malloc (64 * sizeof(int));
#else
    long long _out_accum;
    long long *out_accum = &_out_accum;
    int _array_local[64];
    int* array_local = &_array_local[0];
#endif
    int i,j;

    LOOP_SHIFT:for (i=0;i<N-1; i++) {
        if (i<width)
            *(array_local+i)=din[i];
        else
            *(array_local+i)=din[i]>>2;
    }

    *out_accum=0;
    LOOP_ACCUM:for (j=0;j<N-1; j++) {
        *out_accum += *(array_local+j);
    }

    return *out_accum;
}
```

由于在此处执行编码更改会影响设计功能，因此赛灵思不建议使用 `__SYNTHESIS__` 宏。赛灵思建议您执行以下步骤：

1. 将用户定义的 `NO_SYNT` 宏添加到代码中，并对代码进行修改。
2. 启用 `NO_SYNT` 宏、执行 C 语言仿真，并保存结果。
3. 禁用 `NO_SYNT` 宏，执行 C 语言仿真以验证结果是否相同。
4. 在禁用用户定义的宏的情况下执行综合。

此方法可确保通过 C 语言仿真来验证更新后的代码，然后对相同的代码进行综合。就像 C 中的动态内存使用限制一样，Vivado HLS 也不支持动态创建或删除 C++ 对象（用于综合）。这包括动态多态性和动态虚拟函数调用。

以下代码无法进行综合，因为它会在运行时间创建新函数。

```
Class A {
public:
    virtual void bar() {â{}};
};

void fun(A* a) {
    a->bar();
}
A* a = 0;
```

```
if (base)
    a = new A();
else
    a = new B();

foo(a);
```

指针限制

常用指针强制转换

Vivado HLS 不支持常用指针强制转换，但支持原生 C 类型之间的指针强制转换。

指针数组

Vivado HLS 支持对指针数组进行综合，前提是每个指针都指向 1 个标量或标量数组。指针数组不能指向其他指针。

函数指针

不支持函数指针。

递归函数

递归函数无法综合。这是适用于可形成无穷递归的函数，其中无穷表示：

```
unsigned foo (unsigned n)
{
    if (n == 0 || n == 1) return 1;
    return (foo(n-2) + foo(n-1));
}
```

Vivado® HLS 不支持其中包含有限数量的函数调用的尾递归。

```
unsigned foo (unsigned m, unsigned n)
{
    if (m == 0) return n;
    if (n == 0) return m;
    return foo(n, m%n);
}
```

在 C++ 中，模板可实现尾递归。下文将详解 C++。

标准模板库

许多 C++ 标准模板库 (STL) 都包含函数递归并使用动态内存分配。因此，无法对 STL 进行综合。对于 STL 的解决方案是创建局部函数，该函数具有相同功能但不呈现这些递归、动态内存分配或动态创建和解构对象特征。

注释：支持对标准数据类型（例如 `std::complex`）进行综合。

C 语言测试激励文件

对任何块进行综合的第一步是验证 C 语言函数正确无误。此步骤由测试激励文件执行。准确编写测试激励文件有助于显著提升工作效率。

C 语言函数的执行速度比 RTL 仿真快。与在 RTL 进行开发相比，在综合之前使用 C 语言来开发和验证算法的效率更高。

- 利用 C 语言开发的关键是使用测试激励文件函数结果与已知良好的结果进行比对检查。因为已知该算法正确，所以可以在综合之前验证所有代码更改。
- Vivado® HLS 会复用 C 语言测试激励文件来验证 RTL 设计。使用 Vivado HLS 时，无需创建 RTL 测试激励文件。如果测试激励文件检查顶层函数的结果，则可通过仿真来验证 RTL。

注释：要为测试激励文件提供输入实参，请选择“Project” → “Project Settings”，单击“Simulation”，并使用“Input Arguments”选项。测试激励文件不得要求执行交互式用户输入。Vivado HLS GUI 没有命令控制台，并且在执行测试激励文件时不能接受用户输入。

赛灵思建议您将顶层综合函数与测试激励文件分开，并使用头文件。以下代码示例显示的设计中，`hier_func` 函数调用 2 个子函数：

- `sumsub_func` 执行加法和减法。
- `shift_func` 执行移位。

数据类型在头文件 (`hier_func.h`) 中定义，该文件还描述了：

```
#include "hier_func.h"

int sumsub_func(dint_t *in1, dint_t *in2, dint_t *outSum, dint_t *outSub)
{
    *outSum = *in1 + *in2;
    *outSub = *in1 - *in2;
}

int shift_func(dint_t *in1, dint_t *in2, dout_t *outA, dout_t *outB)
{
    *outA = *in1 >> 1;
    *outB = *in2 >> 2;
}

void hier_func(dint_t A, dint_t B, dout_t *C, dout_t *D)
{
    dint_t apb, amb;

    sumsub_func(&A,&B,&apb,&amb);
    shift_func(&apb,&amb,C,D);
}
```

顶层函数可以包含多个子函数。仅限对单个顶层函数进行综合。要对多个函数进行综合，请将这些函数组合为 1 个顶层函数。

要对 `hier_func` 函数进行综合，请执行以下操作：

1. 将以上示例所示的文件作为设计文件添加到 Vivado HLS 工程中。
2. 将顶层函数指定为 `hier_func`。

综合后：

- 传递到顶层函数中的实参（上例中的 A、B、C 和 D）被综合到 RTL 端口中。
- 顶层中的函数（上例中的 sumsub_func 和 shift_func）被综合到层级块中。

上例中的头文件 (hier_func.h) 显示如何使用宏以及如何通过 typedef 语句提升代码可移植性和可读性。后续章节将说明如何使用 typedef 语句来优化变量类型从而优化变量位宽，在最终的 FPGA 实现中改善面积和性能。

```
#ifndef _HIER_FUNC_H_
#define _HIER_FUNC_H_

#include <stdio.h>

#define NUM_TRANS 40

typedef int din_t;
typedef int dint_t;
typedef int dout_t;

void hier_func(din_t A, din_t B, dout_t *C, dout_t *D);

#endif
```

本示例中的头文件包含设计文件中不需要的一些定义（例如 NUM_TRANS）。这些定义供同样包含这些头文件的测试激励文件使用。

以下代码示例显示了第一个示例中所示设计的测试激励文件。

```
#include "hier_func.h"

int main() {
    // Data storage
    int a[NUM_TRANS], b[NUM_TRANS];
    int c_expected[NUM_TRANS], d_expected[NUM_TRANS];
    int c[NUM_TRANS], d[NUM_TRANS];

    //Function data (to/from function)
    int a_actual, b_actual;
    int c_actual, d_actual;

    // Misc
    int retval=0, i, i_trans, tmp;
    FILE *fp;

    // Load input data from files
    fp=fopen(tb_data/inA.dat,r);
    for (i=0; i<NUM_TRANS; i++){
        fscanf(fp, %d, &tmp);
        a[i] = tmp;
    }
    fclose(fp);

    fp=fopen(tb_data/inB.dat,r);
    for (i=0; i<NUM_TRANS; i++){
        fscanf(fp, %d, &tmp);
        b[i] = tmp;
    }
    fclose(fp);
```

```
// Execute the function multiple times (multiple transactions)
for(i_trans=0; i_trans<NUM_TRANS-1; i_trans++){

//Apply next data values
a_actual = a[i_trans];
b_actual = b[i_trans];

    hier_func(a_actual, b_actual, &c_actual, &d_actual);

//Store outputs
c[i_trans] = c_actual;
d[i_trans] = d_actual;
}

// Load expected output data from files
fp=fopen(tb_data/outC.golden.dat,r);
for (i=0; i<NUM_TRANS; i++){
fscanf(fp, %d, &tmp);
c_expected[i] = tmp;
}
fclose(fp);

fp=fopen(tb_data/outD.golden.dat,r);
for (i=0; i<NUM_TRANS; i++){
fscanf(fp, %d, &tmp);
d_expected[i] = tmp;
}
fclose(fp);

// Check outputs against expected
for (i = 0; i < NUM_TRANS-1; ++i) {
if(c[i] != c_expected[i]){
retval = 1;
}
if(d[i] != d_expected[i]){
retval = 1;
}
}

// Print Results
if(retval == 0){
printf(    *** *** *** *** \n);
printf(    Results are good \n);
printf(    *** *** *** *** \n);
} else {
printf(    *** *** *** *** \n);
printf(    Mismatch: retval=%d \n, retval);
printf(    *** *** *** *** \n);
}

// Return 0 if outputs are corre
return retval;
}
```

高效的测试激励文件

测试激励文件示例着重介绍了高效的测试激励文件的属性，例如：

- 根据 NUM_TRANS 宏的定义，将针对多项传输事务执行综合的顶层函数 (hier_func)。此执行操作允许应用并验证大量不同数据值。测试激励文件的效率受到所执行的测试类型的限制。

- 函数输出将与已知有效的值进行比较。已知有效的值将从该示例中的文件读取，但也可在测试激励文件使用过程中进行计算。
- `main()` 函数的返回值设置为：
 - 0 值：结果正确。
 - 非 0 值：结果不正确。

注释：测试激励文件可返回任意非 0 值。复杂的测试激励文件可根据差异或失败类型返回不同的值。如果测试激励文件在 C 语言仿真或 C/RTL 协同仿真后返回非 0 值，那么 Vivado® HLS 会报告错误和仿真失败。



建议：由于系统环境（例如，Linux、Windows 或 Tcl）可解读 `main()` 函数的返回值，因此赛灵思建议将返回值的范围约束为 8 位以保障可移植性和安全性。



注意！您负责确保测试激励文件对结果进行检查。如果测试激励文件不检查结果而返回 0，那么即使实际并未检查结果，Vivado HLS 仍会指示仿真测试成功。即使输出数据正确且有效，只要测试激励文件向 `main()` 函数返回非 0 值，Vivado HLS 仍会报告仿真失败。

含有这些属性的测试激励文件会在综合前快速测试并验证对 C 语言函数进行的所有更改，在 RTL 中可复用这些测试激励文件以便于简化 RTL 的验证。

设计文件和测试激励文件

由于 Vivado® HLS 复用 C 语言测试激励文件进行 RTL 验证，因此要求在将测试激励文件和任何关联文件添加到 Vivado HLS 工程时，将这些文件全部标记为测试激励文件。符合下列条件的任意文件均为测试激励文件的关联文件：

- 供测试激励文件访问的文件
- 测试激励文件正常运行所需的文件。

测试激励文件示例中，此类文件示例包括数据文件 `inA.dat` 和 `inB.dat`。您必须将这些文件作为测试激励文件添加到 Vivado HLS 工程中。

在 Vivado HLS 工程中识别测试激励文件的要求并不强制要求设计文件与测试激励文件分为不同文件（虽然推荐采用不同文件）。

在以下示例中重复采用了来自 [C 语言测试激励文件](#) 的相同设计。唯一差异在于顶层函数已重命名为 `hier_func2` 以区分这 2 个示例。

使用的头文件和测试激励文件不变（并非将 `hier_func` 更改为 `hier_func2`），在 Vivado HLS 中将 `sumsub_func` 函数综合为顶层函数所需的唯一更改为：

- 在 Vivado HLS 工程中将 `sumsub_func` 设置为顶层函数。
- 添加以下示例中的文件作为设计文件和工程文件。高于 `sumsub_func` 的层次（`hier_func2` 函数）现已属于测试激励文件的一部分。它必须包含在 RTL 实现中。

即使 `sumsub_func` 函数在 `main()` 函数中并未显式例化，但其它函数（`hier_func2` 和 `shift_func`）可确认该函数正常运行，并且属于测试激励文件的一部分。

```
#include "hier_func2.h"

int sumsub_func(dint_t *in1, dint_t *in2, dint_t *outSum, dint_t *outSub)
{
    *outSum = *in1 + *in2;
    *outSub = *in1 - *in2;
}
```

```

}

int shift_func(dint_t *in1, dint_t *in2, dout_t *outA, dout_t *outB)
{
    *outA = *in1 >> 1;
    *outB = *in2 >> 2;
}

void hier_func2(din_t A, din_t B, dout_t *C, dout_t *D)
{
    dint_t apb, amb;

    sumsub_func(&A,&B,&apb,&amb);
    shift_func(&apb,&amb,C,D);
}

```

组合测试激励文件与设计文件

您还可将设计与测试激励文件包含到同一个设计文件内。以下示例所示功能与 [C 语言测试激励文件](#) 到 [C 语言测试激励文件](#) 相同，但所有一切都是在单一文件内捕获的。hier_func 函数已重命名为 hier_func3，以确保示例的唯一性。



重要提示！ 如果测试激励文件与设计包含在同一个文件内，那么您必须将此文件同时作为设计文件和测试激励文件添加到 Vivado® HLS 工程内。

```

#include <stdio.h>

#define NUM_TRANS 40

typedef int din_t;
typedef int dint_t;
typedef int dout_t;

int sumsub_func(din_t *in1, din_t *in2, dint_t *outSum, dint_t *outSub)
{
    *outSum = *in1 + *in2;
    *outSub = *in1 - *in2;
}

int shift_func(dint_t *in1, dint_t *in2, dout_t *outA, dout_t *outB)
{
    *outA = *in1 >> 1;
    *outB = *in2 >> 2;
}

void hier_func3(din_t A, din_t B, dout_t *C, dout_t *D)
{
    dint_t apb, amb;

    sumsub_func(&A,&B,&apb,&amb);
    shift_func(&apb,&amb,C,D);
}

int main() {
    // Data storage
    int a[NUM_TRANS], b[NUM_TRANS];
    int c_expected[NUM_TRANS], d_expected[NUM_TRANS];
    int c[NUM_TRANS], d[NUM_TRANS];
}

```

```
//Function data (to/from function)
int a_actual, b_actual;
int c_actual, d_actual;

// Misc
int retval=0, i, i_trans, tmp;
FILE *fp;
// Load input data from files
fp=fopen(tb_data/inA.dat,r);
for (i=0; i<NUM_TRANS; i++){
fscanf(fp, %d, &tmp);
a[i] = tmp;
}
fclose(fp);

fp=fopen(tb_data/inB.dat,r);
for (i=0; i<NUM_TRANS; i++){
fscanf(fp, %d, &tmp);
b[i] = tmp;
}
fclose(fp);

// Execute the function multiple times (multiple transactions)
for(i_trans=0; i_trans<NUM_TRANS-1; i_trans++){

//Apply next data values
a_actual = a[i_trans];
b_actual = b[i_trans];

hier_func3(a_actual, b_actual, &c_actual, &d_actual);

//Store outputs
c[i_trans] = c_actual;
d[i_trans] = d_actual;
}

// Load expected output data from files
fp=fopen(tb_data/outC.golden.dat,r);
for (i=0; i<NUM_TRANS; i++){
fscanf(fp, %d, &tmp);
c_expected[i] = tmp;
}
fclose(fp);

fp=fopen(tb_data/outD.golden.dat,r);
for (i=0; i<NUM_TRANS; i++){
fscanf(fp, %d, &tmp);
d_expected[i] = tmp;
}
fclose(fp);

// Check outputs against expected
for (i = 0; i < NUM_TRANS-1; ++i) {
if(c[i] != c_expected[i]){
retval = 1;
}
if(d[i] != d_expected[i]){
retval = 1;
}
}

// Print Results
if(retval == 0){
```

```
printf(    *** *** *** *** \n);
printf(    Results are good \n);
printf(    *** *** *** *** \n);
} else {
printf(    *** *** *** *** \n);
printf(    Mismatch: retval=%d \n, retval);
printf(    *** *** *** *** \n);
}

// Return 0 if outputs are correct
return retval;
}
```

函数

顶层函数综合后即变为 RTL 设计的顶层。子函数综合后则变为 RTL 设计中的块。



重要提示！ 顶层函数不得采用静态函数。

综合后，设计中每个函数都有其自己的综合报告和 RTL HDL 文件（Verilog 和 VHDL）。

函数内联

（可选）子函数可通过内联来将其逻辑与周围函数的逻辑合并。虽然内联函数可实现更好的最优化，但也会导致运行时间增加。这导致在内存中必须保留和分析更多逻辑和更多可能性。



提示： Vivado® HLS 可执行小型函数的自动内联。要禁用小型函数的自动内联，请针对该函数将 `inline` 指令设置为 `off`。

如果对某个函数进行内联，则该函数不含报告或独立 RTL 文件。其逻辑和循环将与层级中位于其上层的函数合并。

编码样式的影响

编码样式对于函数的主要影响在于影响函数实参和接口。

如果准确调整函数实参大小，Vivado® HLS 即可在整个设计中传输此信息。无需为每个变量创建任意精度类型。在以下示例中，有 2 个整数相乘，但仅后 24 位用于生成结果。

```
#include "ap_cint.h"

int24 foo(int x, int y) {
    int tmp;

    tmp = (x * y);
    return tmp;
}
```

对此代码进行综合时，可生成 32 位乘法器，输出截断至 24 位。

如果将输入大小正确调整为 12 位类型 (int12) (如下代码示例所示)，则最终 RTL 使用 24 位乘法器。

```
#include "ap_cint.h"
typedef int12 din_t;
typedef int24 dout_t;

dout_t func_sized(din_t x, din_t y) {
    int tmp;

    tmp = (x * y);
    return tmp
}
```

针对 2 项函数输入使用任意精度类型即可足以确保 Vivado HLS 使用 24 位乘法器创建设计。这样即可在整个设计中传输 12 位类型。赛灵思建议您正确调整层级中所有函数实参的大小。

总而言之，从函数接口（尤其是从顶层函数接口）直接驱动变量时，即可阻止执行最优化。常见用例是使用输入作为循环索引上限。

RTL 黑盒

RTL 支持将预先存在的 RTL IP 集成到 HLS 设计中，从而生成可通过 HLS 设计流程运行的设计。此 RTL IP 可在顺序区域、流水线区域或数据流区域内使用。以下文件是将 RTL IP 集成到 HLS 中所必需的：

1. 黑盒描述文件
2. RTL IP 文件
3. RTL 的 C 语言实现

要将 RTL IP 集成到 HLS 设计中，请执行以下操作：

1. 创建 RTL IP 的 C 语言实现函数。
2. 在 HLS 设计中调用此 C 语言实现函数。
3. 创建含必要字段的 JSON 文件。在 [RTL 黑盒 JSON 文件](#) 中提供了 JSON 文件示例和有关格式的信息。
4. 请使用 `add_files` 选项将 JSON 文件添加到 `script.tcl` 文件中。

```
add_files -blackbox my_file.json
```

5. 运行 HLS 设计流程；即，C 仿真、综合和协同仿真。

要求与限制

- 在 HLS 内部，仅限 C++ 才支持 RTL 黑盒。
- 在 HLS 内部，RTL 黑盒无法连接到顶层接口 I/O 信号。
- 在 HLS 内部，RTL 黑盒无法直接充当 DUT。
- 在 HLS 内部，RTL 黑盒不支持接口类型为结构体或类。
- 在 HLS 内部，RTL 黑盒支持以下接口协议：
 - `hls::stream`：RTL 黑盒 IP 支持 `hls::stream` 接口。在 C 语言中使用此特定数据类型时，请针对 RTL 黑盒 IP 中的该实参使用 FIFO 接口。

- 数组：RTL 黑盒 IP 支持 RAM（数组）接口。在 C 语言中使用此构造时，请针对 RTL 黑盒 IP 中的对应实参使用下列 RAM 接口之一：
 - 单端口 RAM - RAM_1P
 - 双端口 RAM - RAM_T2P
- C 语言标量和输入指针：RTL 黑盒 IP 仅支持在顺序区域和流水线区域中使用 C 语言标量和输入指针，不支持在数据流区域内使用。在 C 语言中使用此构造时，请在 RTL IP 中使用 wire。
- 输入输出指针和输出指针：RTL 黑盒 IP 仅支持在顺序区域和流水线区域中使用输入输出指针和输出指针，不支持在数据流区域内使用。在 C 语言中使用此构造时，RTL IP 应使用 ap_vld 作为输出指针，使用 ap_ovld 作为输入输出指针。
- 提供给 HLS 的 RTL IP 文件应采用 Verilog (.v) 格式。
- RTL IP 模块必须具有专用时钟信号和专用复位信号（处于正电平高位）。
- RTL IP 模块必须具有用于启用或停用 RTL IP 的 CE 信号。
- RTL IP 必须使用 ap_ctrl_chain 协议。如需了解更多信息，请参阅[块级 I/O 协议](#)。

JSON 文件限制：

- “c_function_name” 字段必须与 C 语言函数模型一致。
- “rtl_top_module_name” 必须与 “c_function_name” 一致
- 未使用的 “c_parameters” 字段应从模板删除。
- 每个 “c_parameter” 字段都应 与 1 个 “rtl_port” 字段关联。

注释：使用 RTL 黑盒时，仍适用所有其它 HLS 设计限制。

JSON 文件格式

下表描述了 JSON 文件格式：

表 42：JSON 文件格式

项目	属性	描述
c_function_name		黑盒的 C++ 函数名称
rtl_top_module_name		黑盒的 RTL 函数名称
c_files	c_file	指定要用于黑盒模块的 C 语言文件。
	cflag	提供对应 C 语言文件所需的任何编译选项。
rtl_files		指定要用于黑盒模块的 RTL 文件。

表 42: JSON 文件格式 (续)

项目	属性	描述
c_parameters	c_name	指定用于黑盒 C++ 函数的实参名称。
	c_port_direction	对应 C 语言实参的访问方向。 · in: 仅供黑盒 C++ 函数读取。 · out: 仅供黑盒 C++ 函数写入。 · inout: 将可供黑盒 C++ 函数读写。
	RAM_type	指定对应 C 语言实参使用 RTL 的 RAM 协议时要使用的 RAM 类型。可使用 2 种类型的 RAM: · RAM_1P: 用于 1 端口 RAM 模块 · RAM_T2P: 用于 2 端口 RAM 模块 当对应 C 语言实参不使用 RTL 的 RAM 协议时, 省略该属性。
	rtl_ports	指定对应 C 语言实参的 RTL 端口协议信号。可使用 5 种类型的 RTL 端口协议: · wire: C 语言实参使用标量或含输入方向的指针时, 可映射到 wire。 · ap_vld: C 语言实参使用含输出方向的指针时, 可映射到 ap_vld。 · ap_ovld: C 语言实参使用含输入输出方向的指针时, 可映射到 ap_ovld。 · FIFO: C 语言实参使用 hls::stream 数据类型时, 可映射到 FIFO。 · RAM: C 语言实参使用数组类型时, 可映射到 RAM。数组类型支持输入输出方向。 以上指定的 RTL 端口协议具有关联的控制信号, 这些关联信号需在 JSON 文件内指定。请参阅下表以获取有关使用方法的更多详细信息。
c_return	c_port_direction	必须设置为 out。
	rtl_ports	指定 RTL 黑盒 IP 中使用的对应 RTL 端口名称。

表 42: JSON 文件格式 (续)

项目	属性	描述
rtl_common_signal	module_clock	RTL 黑盒模块的专用时钟信号。
	module_reset	指定 RTL 黑盒模块的复位信号。复位信号必须处于高电平有效或正电平有效状态。
	module_clock_enable	指定 RTL 黑盒模块的时钟使能信号。使能信号必须处于高电平有效或正电平有效状态。
	ap_ctrl_chain_protocol_idle	RTL 黑盒模块的 ap_ctrl_chain 协议中的 ap_idle 信号。
	ap_ctrl_chain_protocol_start	RTL 黑盒模块的 ap_ctrl_chain 协议中的 ap_start 信号。
	ap_ctrl_chain_protocol_ready	RTL 黑盒 IP 的 ap_ctrl_chain 协议中的 ap_ready 信号。
	ap_ctrl_chain_protocol_done	RTL 黑盒模块的 ap_ctrl_chain 协议中的“ap_done”信号。
	ap_ctrl_chain_protocol_continue	RTL 黑盒模块的 ap_ctrl_chain 协议中的 ap_continue 信号。
rtl_performance	latency	指定 RTL 黑盒模块的时延。该值必须为非负整数值。对于组合型 RTL IP，请指定 0，否则请指定 RTL 模块的精确时延。
	II	函数接受新输入数据之前的时钟周期数。该值必须为非负整数值。0 表示无法将此黑盒流水线化。否则，它表示黑盒模块可流水线化。
rtl_resource_usage	FF	指定 RTL 黑盒模块的寄存器利用率。
	LUT	指定 RTL 黑盒模块的 LUT 利用率。
	BRAM	指定 RTL 黑盒模块的块 RAM 利用率。
	URAM	指定 RTL 黑盒模块的 URAM 利用率。
	DSP	指定 RTL 黑盒模块的 DSP 利用率。

表 43：RTL 端口协议

RTL 端口协议	RAM 类型	C 端口方向	属性	用户定义的名称	注
wire		in	data_read_in	指定 RTL 黑盒 IP 中使用的用户定义的名称。作为对应 wire 的示例，如果 RTL 端口名称为“flag”，则 JSON 文件格式为 "data_read-in" : "flag"	
ap_vld		out	data_write_out		
			data_write_valid		
ap_ovld		inout	data_read_in		
			data_write_out		
			data_write_valid		
FIFO		in	FIFO_empty_flag		必须处于负电平有效状态。
			FIFO_read_enable		
			FIFO_data_read_in		
		out	FIFO_full_flag		必须处于负电平有效状态。
			FIFO_write_enable		
			FIFO_data_write_out		
RAM	RAM_1P	in	RAM_address		
			RAM_clock_enable		
			RAM_data_read_in		
		out	RAM_address		
			RAM_clock_enable		
			RAM_write_enable		
			RAM_data_write_out		
		inout	RAM_address		
			RAM_clock_enable		
			RAM_write_enable		
			RAM_data_write_out		
			RAM_data_read_in		

表 43: RTL 端口协议 (续)

RTL 端口协议	RAM 类型	C 端口方向	属性	用户定义的名称	注
RAM	RAM_T2P	in	RAM_address	指定 RTL 黑盒 IP 中使用的用户定义的名称。作为对应 wire 的示例，如果 RTL 端口名称为“flag”，则 JSON 文件格式为 "data_read-in": "flag"	含 _snd 的信号属于 RAM 的第 2 端口。不含 _snd 的信号属于第 1 端口。
			RAM_clock_enable		
			RAM_data_read_in		
			RAM_address_snd		
			RAM_clock_enable_snd		
			RAM_data_read_in_snd		
		out	RAM_address		
			RAM_clock_enable		
			RAM_write_enable		
			RAM_data_write_out		
			RAM_address_snd		
			RAM_clock_enable_snd		
			RAM_write_enable_snd		
			RAM_data_write_out_snd		
		inout	RAM_address		
			RAM_clock_enable		
			RAM_write_enable		
			RAM_data_write_out		
			RAM_data_read_in		
			RAM_address_snd		
			RAM_clock_enable_snd		
			RAM_write_enable_snd		
			RAM_data_write_out_snd		
			RAM_data_read_in_snd		

注释：RTL 黑盒的 C 语言函数行为模型必须遵循建议的 HLS 编码样式。

循环

循环可提供非常直观且简单的方式来捕获算法行为，常用于 C 语言代码。循环受到综合的全面支持：循环可流水线化、展开、部分展开和扁平化。

最优化展开、部分展开、扁平化与合并可对循环结构进行有效更改，与更改代码的效果无异。这些最优化可确保对循环进行最优化时只需最低限度的编码更改即可。部分最优化仅限在某些条件下应用。可能需要仅限适当编码更改。



建议：对于循环索引变量，请避免使用全局变量，因为这可能不利于某些最优化。

变量循环边界

当循环包含变量边界时，将禁止执行 Vivado® HLS 可应用的某些最优化操作。在以下代码示例中，循环边界由顶层输入驱动的变量 `width` 来判定。在此情况下，循环被视为包含变量边界，因为 Vivado HLS 无从知晓循环将何时完成。

```
#include "ap_cint.h"
#define N 32

typedef int8 din_t;
typedef int13 dout_t;
typedef uint5 dsel_t;

dout_t code028(din_t A[N], dsel_t width) {

    dout_t out_accum=0;
    dsel_t x;

    LOOP_X:for (x=0;x<width; x++) {
        out_accum += A[x];
    }

    return out_accum;
}
```

尝试对上述示例中的设计进行最优化时，会报告由变量循环边界造成的问题。变量循环边界的第一个问题是阻止 Vivado HLS 判定循环时延。Vivado HLS 可判定完成一次循环迭代所需的时延，但由于它无法静态判定变量宽度的精确值，因此无从知晓执行的迭代次数，因而无法报告循环时延（即完全执行循环的每次迭代的周期数）。

存在变量循环边界时，Vivado HLS 会将时延报告为问号 (?) 而不是使用精确值。以下显示了完成上述示例的综合后的结果。

```
+ Summary of overall latency (clock cycles):
* Best-case latency:      ?
* Worst-case latency:    ?
+ Summary of loop latency (clock cycles):
+ LOOP_X:
* Trip count: ?
* Latency:    ?
```

变量循环边界的另一个问题是设计性能未知。可通过如下 2 种方法解决此问题：

- 使用 `Tripcount` 指令。下文解释了有关此方法的详细信息。
- 在 C 语言代码中使用 `assert` 宏。

`tripcount` 指令允许为循环指定 `tripcount` 最小值和/或最大值。行程计数 (`tripcount`) 表示循环的迭代次数。如果对第 1 个示例中的 `LOOP_X` 应用 `tripcount` 最大值 32，那么报告将更新为显示如下内容：

```
+ Summary of overall latency (clock cycles):
* Best-case latency:      2
* Worst-case latency:    34
+ Summary of loop latency (clock cycles):
+ LOOP_X:
* Trip count: 0 ~ 32
* Latency:    0 ~ 32
```

Tripcount 指令不影响综合结果，仅影响报告结果。用户为 Tripcount 指令提供的值仅用于报告。Tripcount 值允许 Vivado HLS 显示报告中的数值，并支持与来自其它解决方案的报告进行对比。要将此循环边界信息用于综合，必须更新 C 语言代码。

Tripcount 指令不影响综合结果，仅影响报告结果。

要对第 1 个示例进行最优化以降低启动时间间隔，请执行下列后续步骤：

- 展开循环，允许并行执行累加。
- 通过单一内存端口对数组输入进行分区或者限制并行累加。

应用这些最优化操作后，Vivado HLS 的输出会高亮显示变量边界循环的最显著的问题：

```
@W [XFORM-503] Cannot unroll loop 'LOOP_X' in function 'code028': cannot
completely
unroll a loop with a variable trip count.
```

由于变量边界循环无法展开，因此不仅阻止应用展开 (unroll) 指令，而且还会阻止将循环上层的层次流水线化。



重要提示！ 当循环或函数实现流水线化后，Vivado HLS 即可将函数或循环下层层级中的所有循环展开。如果在此层级中存在具有变量边界的循环，它就会阻止流水线化。

对于含变量边界的循环，解决方案是通过循环内部的条件执行来将循环迭代次数设置为固定值。可重写变量循环边界示例中的代码，如以下代码示例所示。此处循环边界显式设置为变量宽度的最大值，循环主体则以有条件方式来执行：

```
#include "ap_cint.h"
#define N 32

typedef int8 din_t;
typedef int13 dout_t;
typedef uint5 dsel_t;

dout_t loop_max_bounds(din_t A[N], dsel_t width) {

    dout_t out_accum=0;
    dsel_t x;

    LOOP_X:for (x=0;x<N; x++) {
        if (x<width) {
            out_accum += A[x];
        }
    }

    return out_accum;
}
```

以上示例中的 for 循环 (LOOP_X) 可展开。由于此循环的上限固定，因此 Vivado HLS 知晓需创建的硬件数量。在 RTL 设计中包含 N(32) 份循环主体副本。每份循环主体副本都包含与之关联的条件逻辑，并根据变量宽度值来执行。

循环流水线化

对循环进行流水线化时，通常可通过将最内层的循环流水线化来实现面积与性能之间的最优平衡。这样同时也可达成最短的运行时间。以下代码示例演示了循环和函数流水线化的优缺点取舍。

```
#include "loop_pipeline.h"

dout_t loop_pipeline(din_t A[N]) {

    int i,j;
    static dout_t acc;

    LOOP_I:for(i=0; i < 20; i++){
    LOOP_J: for(j=0; j < 20; j++){
        acc += A[i] * j;
    }
    }

    return acc;
}
```

如果将最内层循环 (LOOP_J) 流水线化，那么硬件中包含 1 份 LOOP_J 副本（单次乘法）。Vivado® HLS 会尽可能自动将循环扁平化（如此例所示），并有效创建 1 个含 20*20 次迭代的新循环。这样只需调度 1 次乘法操作并访问 1 个数组，即可将循环迭代调度为单一循环主体实体（20x20 次循环迭代）。



提示：当循环或函数流水线化时，所在层级比流水线化的循环或函数层级更低的所有循环都必须展开。

如果流水线化外部循环 (LOOP_I)，则内部循环 (LOOP_J) 将展开，以创建 20 份循环主体副本：这样必须调度 20 次乘法和 20 次数组访问。然后，LOOP_I 的每次迭代都必须作为单一实体来调度。

如果顶层函数流水线化，则 2 个循环都必须展开：这样就必须调度 400 次乘法和 400 次数组访问。Vivado HLS 生成含 400 次乘法的设计的可能性很低，因为在大部分设计中，数据依赖关系通常会阻止最大程度并行化，例如，即使针对 A[N] 使用双端口 RAM，设计也只能在任意时钟周期内访问 2 个 A[N] 值。

选择适合流水线化的层级时应遵循的概念是：对最内层的循环进行流水线化所涉及的硬件量最少，并且产生的吞吐量对大部分应用都可接受。对较上层的层级进行流水线化则会将所有下层循环全部展开，从而导致显著增加要调度的操作数量（可能影响运行时间和内存容量），但在吞吐量和时延方面通常可产生最高性能的设计。

总结上述选项：

- 流水线化 LOOP_J

时延约为 400 个周期 (20x20)，需少于 100 个 LUT 和寄存器（I/O 控制和 FSM 始终存在）。

- 流水线化 LOOP_I

时延约 20 个周期，但需要数百个 LUT 和寄存器。比第 1 个选项高约 20 倍，扣除可执行的所有逻辑最优化。

- 流水线化 loop_pipeline 函数

时延约为 10（20 次双端口访问），但需要数千个 LUT 和寄存器（约为第 1 个选项的循环的 400 倍，扣除可执行的所有最优化）。

非完美嵌套矩阵

对循环层级的内层循环进行流水线化时，Vivado® HLS 会将嵌套循环扁平化，以缩短时延并改进总体吞吐量，方法是移除因循环过渡（进入和退出循环时，对循环索引执行的检查）导致的任何周期。此类检查在循环间过渡（进入和/或退出）时可能导致时钟延迟。

非完美循环嵌套或者无法对循环进行扁平化会导致进入和退出循环时额外增加时钟周期。当设计包含嵌套循环时，请分析结果以确认已尽可能将更多嵌套循环扁平化：复查日志文件或者查看综合报告中是否存在如上所示已合并循环标签的案例（LOOP_I 和 LOOP_J 现已报告为 LOOP_I-LOOP_J）。

循环并行化

Vivado® HLS 将逻辑和函数调度为尽早运行以降低时延。为此，它会将尽可能多的逻辑运算和函数调度为并行运行。但它并不会将循环也调度为并行执行。

如果对以下代码示例进行综合，则先调度 SUM_X 循环，然后调度 SUM_Y 循环，即使 SUM_Y 循环无需等待 SUM_X 循环完成后再开始运算，也仍调度为在 SUM_X 之后运行。

```
#include "loop_sequential.h"

void loop_sequential(din_t A[N], din_t B[N], dout_t X[N], dout_t Y[N],
    dsel_t xlimit, dsel_t ylimit) {

    dout_t X_accum=0;
    dout_t Y_accum=0;
    int i,j;

    SUM_X:for (i=0;i<xlmit; i++) {
        X_accum += A[i];
        X[i] = X_accum;
    }

    SUM_Y:for (i=0;i<ylimit; i++) {
        Y_accum += B[i];
        Y[i] = Y_accum;
    }
}
```

由于循环边界（xlimit 和 ylimit）不同，因此无法合并。通过将循环置于不同函数内（如以下代码示例所示），即可实现相同功能，并且可并行调度这 2 个循环（在函数内部）。

```
#include "loop_functions.h"

void sub_func(din_t I[N], dout_t O[N], dsel_t limit) {
    int i;
    dout_t accum=0;

    SUM:for (i=0;i<limit; i++) {
        accum += I[i];
        O[i] = accum;
    }
}

void loop_functions(din_t A[N], din_t B[N], dout_t X[N], dout_t Y[N],
```

```

dse1_t xlimit, dse1_t ylimit) {

    sub_func(A,X,xlimit);
    sub_func(B,Y,ylimit);
}

```

如果对前述示例进行综合，则时延为顺序循环示例的一半，因为现在这些循环（作为函数）可并行执行。

dataflow 最优化同样可在顺序循环示例中使用。此处演示了在函数中捕获循环以发挥其并行性的原理，在此处示例中无法使用 dataflow 最优化。例如，在较大的示例中，对位于顶层的所有循环和函数应用 dataflow 最优化，内存置于每个顶层循环和函数之间。

循环依赖关系

循环依赖关系即阻止对循环进行最优化（通常为流水线化）的数据依赖关系。此类依赖关系可出现在任一循环的单一迭代内或者也可出现在任一循环的不同迭代内。

了解循环依赖关系的最简单的方法是检验极端示例。在以下示例中，循环结果用作为循环持续条件或循环退出条件。循环的每次迭代必须完成后才能开始下一次迭代。

```

Minim_Loop: while (a != b) {
    if (a > b)
        a -= b;
    else
        b -= a;
}

```

此循环无法流水线化。前一次迭代结束后，下一次循环迭代才能开始。并非所有循环依赖关系都如此极端，但此示例着重展示了部分操作在某些其它操作完成后才能开始的情况。解决方案为尝试确保尽早执行初始运算。

循环依赖关系可能发生于所有类型的数据。使用数组时尤其常见。

展开 C++ 类中的循环

在 C++ 类中使用循环时，请确保循环感应变量的数据成员，因为这会导致展开此循环。

在此示例中，循环感应变量的 `k` 是类 `foo_class` 的成员。

```

template <typename T0, typename T1, typename T2, typename T3, int N>
class foo_class {
private:
    pe_mac<T0, T1, T2> mac;
public:
    T0 areg;
    T0 breg;
    T2 mreg;
    T1 preg;
    T0 shift[N];
    int k; // Class Member
    T0 shift_output;
    void exec(T1 *pcout, T0 *dataOut, T1 pcin, T3 coeff, T0 data, int col)
    {
        Function_label0::
        #pragma HLS inline off
        SRL:for (k = N-1; k >= 0; --k) {

```

```
#pragma HLS unroll // Loop will fail UNROLL
if (k > 0)
    shift[k] = shift[k-1];
else
    shift[k] = data;
}

    *dataOut = shift_output;
    shift_output = shift[N-1];
}

*pcout = mac.exec1(shift[4*col], coeff, pcin);
};
```

为了使 Vivado® HLS 能够按 UNROLL 编译指示指令指定的方式展开此循环，应重写此代码，从类成员中移除 `k`。

```
template <typename T0, typename T1, typename T2, typename T3, int N>
class foo_class {
private:
    pe_mac<T0, T1, T2> mac;
public:
    T0 areg;
    T0 breg;
    T2 mreg;
    T1 preg;
    T0 shift[N];
    T0 shift_output;
    void exec(T1 *pcout, T0 *dataOut, T1 pcin, T3 coeff, T0 data, int col)
    {
Function_label0::
        int k; // Local variable
#pragma HLS inline off
        SRL:for (k = N-1; k >= 0; --k) {
#pragma HLS unroll // Loop will unroll
            if (k > 0)
                shift[k] = shift[k-1];
            else
                shift[k] = data;
            }

            *dataOut = shift_output;
            shift_output = shift[N-1];
        }

        *pcout = mac.exec1(shift[4*col], coeff, pcin);
    };
};
```

数组

在探讨编码样式对综合后的数组实现的影响之前，有必要探讨数组可能引发问题（即使在综合之前）的情况，例如，在 C 语言仿真期间。

如果您指定的数组过大，可能导致 C 语言仿真因内存不足而失败，如以下示例所示：

```
#include "ap_cint.h"

int i, acc;
// Use an arbitrary precision type
int32 la0[100000000], la1[100000000];

for (i=0 ; i < 100000000; i++) {
    acc = acc + la0[i] + la1[i];
}
```

仿真可能因内存不足而失败，因为数组置于内存中存在的堆栈上，而不是置于由操作系统管理并且可使用本地磁盘空间来扩展的堆上。

这可能导致设计运行时出现内存不足，某些问题还会提高出现此问题的概率：

- 在 PC 上，可用内存通常小于大型 Linux 系统，并且可用内存可能进一步减少。
- 而使用如上所示任意精度类型则可能使此问题更严重，因为这些类型所需内存比标准 C 语言类型所需内存更多。
- 使用 C++ 和 SystemC 语言中的更复杂的定点任意精度类型可能还会提高出现此问题的概率，因为这些类型所需内存更多。

增加 C/C++ 代码开发中的内存资源的标准方法是使用连接器选项增大堆栈大小，例如以下选项将堆栈大小显式设置为 -Wl,--stack,10485760。在 Vivado® HLS 中可转至“Project Settings” → “Simulation” → “Linker flags”来应用此类选项，或者还可作为 Tcl 命令的选项来提供此类选项：

```
cosim_design -ldflags {-Wl,--stack,10485760}
cosim_design -ldflags {-Wl,--stack,10485760}
```

在某些情况下，机器可能没有足够内存，增大堆栈大小无效。

解决方案是使用动态内存分配执行仿真，但使用固定大小的数组执行综合，如以下示例所示。这意味着此操作所需内存将在由操作系统管理并可使用本地磁盘空间来扩展的堆上进行分配。

对代码进行此类更改并非理想方式，因为仿真的代码将不同于综合的代码，但有时这是推动设计进程继续进行的唯一途径。如果采用此方法，请务必确认 C 语言测试激励文件涵盖数组访问的各方面。由 cosim_design 执行的 RTL 仿真将验证内存访问是否正确。

```
#include "ap_cint.h"

int i, acc;
#ifdef __SYNTHESIS__
    // Use an arbitrary precision type & array for synthesis
    int32 la0[100000000], la1[100000000];
#else
    // Use an arbitrary precision type & dynamic memory for simulation
    int32 *la0 = malloc(100000000 * sizeof(int32));
    int32 *la1 = malloc(100000000 * sizeof(int32));
#endif
for (i=0 ; i < 100000000; i++) {
    acc = acc + la0[i] + la1[i];
}
```

注释：__SYNTHESIS__ 宏仅限在要综合的代码中使用。请勿在测试激励文件中使用该宏，因为 C 语言仿真或 C RTL 协同仿真不会遵循其指示进行操作。

综合后，数组通常作为内存（RAM、ROM 或 FIFO）来实现。顶层函数接口上的数组作为访问外部内存的 RTL 端口来进行综合。而在设计内部，大小小于 1024 的数组将作为 SRL 来综合。大小大于 1024 的数组将综合到块 RAM、LUTRAM 或 UltraRAM 中（取决于最优化设置）。

就像循环一样，数组是直观的编码构造，通常可在 C 程序中找到。同样像循环一样，Vivado HLS 包含最优化和指令，可通过应用这些最优化和指令来最优化数组在 RTL 中的实现，而无需修改代码。

在下列情况下，数组在 RTL 中可能引发问题：

- 数组访问通常会造成性能瓶颈。作为内存来实现时，内存端口的数量会限制对数据的访问。数组初始化如未谨慎执行，则可能导致 RTL 中不必要的长时间复位和初始化。
- 请确保在 RTL 中将只需读访问权的数组作为 ROM 来实现。

Vivado HLS 支持指针数组。每个指针都只能指向 1 个标量或 1 个标量数组。

注释：数组大小必须固定。例如，支持固定大小的数组，如：Array[10]；。但不支持非固定大小的数组，如：Array[]；。

数组访问和性能

以下代码示例显示了访问数组导致最终 RTL 设计性能受限的案例。在此示例中，对 mem[N] 数组执行了 3 次访问以创建求和结果。

```
#include "array_mem_bottleneck.h"

dout_t array_mem_bottleneck(din_t mem[N]) {

    dout_t sum=0;
    int i;

    SUM_LOOP: for(i=2; i<N; ++i)
        sum += mem[i] + mem[i-1] + mem[i-2];

    return sum;
}
```

在综合期间，该数组作为 RAM 来实现。如果将此 RAM 指定为单端口 RAM，那么将无法通过对 SUM_LOOP 循环进行流水线化来实现在每个时钟周期内处理新的循环迭代的目标。

尝试以值为 1 的启动时间间隔对 SUM_LOOP 进行流水线化将生成如下消息（无法达成吞吐量 1 后，Vivado® HLS 会放宽约束）：

```
INFO: [SCHED 61] Pipelining loop 'SUM_LOOP'.
WARNING: [SCHED 69] Unable to schedule 'load' operation ('mem_load_2',
bottleneck.c:62) on array 'mem' due to limited memory ports.
INFO: [SCHED 61] Pipelining result: Target II: 1, Final II: 2, Depth: 3.
```

此处的问题是单端口 RAM 只有 1 个数据端口：每个时钟周期内只能执行 1 次读取（和 1 次写入）。

- SUM_LOOP Cycle1：读取 mem[i]；
- SUM_LOOP Cycle2：读取 mem[i-1]，对值求和；
- SUM_LOOP Cycle3：读取 mem[i-2]，对值求和；

可使用双端口 RAM，但这样仅支持每个时钟周期 2 次访问。要计算总和值，需读取 3 次，因此要将循环流水线化，每个时钟周期需访问 3 次，即每个时钟周期内新增 1 次迭代。



注意！ 作为内存或内存端口实现的数组通常会成为性能瓶颈。

以上示例中的代码可按以下代码示例中所示方式重写，以便按吞吐量为 1 来对代码进行流水线化。在以下代码示例中，通过执行预读取并手动对数据访问进行流水线化，即可在循环每次迭代中仅指定 1 次数组读取。这样可确保只需 1 个单端口 RAM 即可实现性能目标。

```
#include "array_mem_perform.h"

dout_t array_mem_perform(din_t mem[N]) {

    din_t tmp0, tmp1, tmp2;
    dout_t sum=0;
    int i;

    tmp0 = mem[0];
    tmp1 = mem[1];
    SUM_LOOP:for (i = 2; i < N; i++) {
        tmp2 = mem[i];
        sum += tmp2 + tmp1 + tmp0;
        tmp0 = tmp1;
        tmp1 = tmp2;
    }

    return sum;
}
```

Vivado HLS 包含用于更改数组实现和访问方式的最优化指令。通常可直接使用这些指令，而无需更改任何代码。数组可分区到块中或者分区到其各元素中。在某些情况下，Vivado HLS 将数组分区到各独立元素中。这可使用自动分区配置设置来加以控制。

将数组分区到多个块中时，单一阵列将作为多个 RTL RAM 块来实现。分区到元素中时，每个元素都作为 RTL 中的 1 个寄存器来实现。在这 2 种情况下，分区允许并行访问更多元素，并且有助于提升性能；设计将在性能与实现性能所需的 RAM 或寄存器数量之间加以权衡取舍。

FIFO 访问

数组作为 FIFO 实现时应留意数组访问。使用数据流最优化时尤其如此。

必须从位置 0 开始按顺序访问 FIFO。此外，如果在多个位置读取某一数组，则代码必须严格实施 FIFO 访问顺序。含多个扇出的数组如无附加代码用于严格实施访问顺序，则无法作为 FIFO 来实现。

接口上的数组

默认情况下，Vivado® HLS 会将数组综合到内存元素中。使用数组作为顶层函数的实参时，Vivado HLS 假定：

- 内存位于片外。
Vivado HLS 会综合接口端口以访问内存。
- 内存为标准块 RAM，时延为 1。
提供地址后经过 1 个时钟周期后，数据即处于就绪状态。

要配置 Vivado HLS 创建这些端口的方式，请执行以下操作：

- 使用 INTERFACE 指令将接口指定为 RAM 或 FIFO 接口。
- 使用 RESOURCE 指令将 RAM 指定为单端口或双端口 RAM。
- 使用 RESOURCE 指令指定 RAM 时延。
- 使用数组最优化指令（Array_Partition、Array_Map 或 Array_Reshape）来重新配置数组结构，从而重新配置 I/O 端口数量。



提示：由于通过内存（RAM 或 FIFO）端口限制对数据的访问，接口上的数组可能造成性能瓶颈。通常您可使用指令来克服这些瓶颈。

在可综合的代码中使用数组时，必须调整数组大小。例如，如果数组接口中的声明 `d_i[4]` 更改为 `d_i[]`，那么 Vivado HLS 会发出消息称无法综合此设计：

```
@E [SYNCHK-61] array_RAM.c:52: unsupported memory access on variable 'd_i'
which is (or contains) an array with unknown size at compile time.
```

数组接口

`resource` 指令可显式指定使用的 RAM 类型，以及据此创建的 RAM 端口（单端口或双端口）。如果不指定任何资源，Vivado® HLS 会使用：

- 单端口 RAM（默认情况下）。
- 双端口 RAM，前提是这样可缩短启动时间间隔或减少时延。

`partition`、`map` 和 `reshape` 指令可重配置接口上的数组。数组可分区为多个较小的数组，并在其各自接口上单独实现。这包括能够将数组的每个元素分区到其自己的标量元素中。在函数接口上，这导致数组中每个元素都有专用的端口。由此可提供最大限度并行访问，但会造成端口数量剧增，可能在上层层级中导致布线问题。

同样，较小的数组可组合为单一较大的数组，从而生成单一接口。虽然这样更便于映射到片外块 RAM，但也可能导致性能瓶颈。可通过使用 Vivado HLS 最优化指令来权衡这些取舍，而不会影响编码。

默认情况下，以下代码示例中所示的函数中的数组实参将综合到单端口 RAM 接口中。

```
#include "array_RAM.h"

void array_RAM (dout_t d_o[4], din_t d_i[4], didx_t idx[4]) {
    int i;

    For_Loop: for (i=0;i<4;i++) {
        d_o[i] = d_i[idx[i]];
    }
}
```

使用单端口 RAM 接口的原因是因为 `for-loop` 可确保在每个时钟周期内仅读写 1 个元素。使用双端口 RAM 接口并无任何优势。

如果展开 `for` 循环，那么 Vivado HLS 会使用双端口。这样即可支持同时读取多个元素，从而缩短启动时间间隔。通过应用资源指令即可显式设置 RAM 接口类型。

接口上数组相关问题通常与吞吐量有关。可通过最优化指令来处理这些问题。例如，如果以上示例中的数组分区到各独立元素中，并且展开 `for-loop`，那么即可同时访问每个数组中的全部 4 个元素。

您还可使用 RESOURCE 指令来指定 RAM 的时延。这样 Vivado HLS 即可对接口处时延大于 1 的外部 SRAM 进行建模。

FIFO 接口

Vivado® HLS 允许在 RTL 中将数组实参作为 FIFO 端口来实现。如果要使用 FIFO 端口，请务必确保往来该数组的访问均为顺序访问。Vivado HLS 会判定访问是否为顺序访问。

表 44：Vivado HLS 顺序访问分析

是否为顺序访问?	Vivado HLS 操作
是	实现 FIFO 端口。
否	1. 发出错误消息。 2. 暂停综合。
无法判定	1. 发出警告。 2. 实现 FIFO 端口。

注释：如果实际上并非顺序访问，则存在 RTL 仿真不匹配。

以下代码示例中显示了 Vivado HLS 无法判定访问是否为顺序访问时的情况。在此示例中，d_i 和 d_o 均已指定为在综合期间作为 FIFO 接口来实现。

```
#include "array_FIFO.h"

void array_FIFO (dout_t d_o[4], din_t d_i[4], didx_t idx[4]) {
    int i;
    #pragma HLS INTERFACE ap_fifo port=d_i
    #pragma HLS INTERFACE ap_fifo port=d_o
    // Breaks FIFO interface d_o[3] = d_i[2];
    For_Loop: for (i=0;i<4;i++) {
        d_o[i] = d_i[idx[i]];
    }
}
```

在此情况下，idx 变量的行为将判定是否可成功创建 FIFO 接口。

- 如果 idx 按序递增，则可创建 FIFO 接口。
- 如果针对 idx 使用随机值，则在 RTL 中实现 FIFO 接口时将失败。

由于此接口可能无效，Vivado HLS 会在综合期间发出一条消息并创建 FIFO 接口。

```
@W [XFORM-124] Array 'd_i': may have improper streaming access(es).
```

如果移除以上示例中的 //Breaks FIFO interface 注释，Vivado HLS 即可判定对数组的访问并非顺序访问，如果已指定 FIFO 接口，那么它将暂停并发出一条错误消息。

注释：针对要读取和写入的数组，FIFO 端口无法进行综合。必须创建独立的输入数组和输出数组（如上示例所示）。

以下通用规则适用于使用流传输接口（代替 FIFO 接口）实现的数组：

- 该数组只能在 1 个循环或函数内读取和写入。这可变换为匹配 FIFO 链接特征的点对点连接。
- 数组读取与数组写入的顺序必须相同。由于针对 FIFO 通道不支持随机访问，因此必须在遵循先入先出语义的程序内使用该数组。
- 用于读取或写入 FIFO 的索引必须可供在编译时分析。对于基于运行时间计算的数组寻址，无法分析其 FIFO 语义，这导致该工具无法将数组转换为 FIFO。

通常在顶层接口中实现或最优化数组时，无需更改代码。仅当数组属于结构体的一部分时，才可能需要对接口上的数组进行编码更改。

数组初始化



建议：赛灵思建议（但并不强制）将数组指定为作为含静态 (static) 限定符的内存来实现。这不仅可确保 Vivado® HLS 以 RTL 中的内存来实现数组，还允许使用静态类型的初始化行为。

在以下代码中，数组利用一组值来进行初始化。每次执行函数时，都会为 `coeff` 数组分配这些值。综合后，每次执行设计时，用于实现 `coeff` 的 RAM 都会随这些值一起加载。对于单端口 RAM，此操作耗时 8 个时钟周期。对于 1024 数组，当然也就需要 1024 个时钟周期，在此期间无法执行任何依赖于 `coeff` 的运算。

```
int coeff[8] = {-2, 8, -4, 10, 14, 10, -4, 8, -2};
```

以下代码使用 `static` 限定符来定义 `coeff` 数组。该数组开始执行时会使用指定的值进行初始化。每次执行该函数时，`coeff` 数组都会保留上次执行的值。静态数组在 C 语言代码中的行为与 RTL 中的内存行为相同。

```
static int coeff[8] = {-2, 8, -4, 10, 14, 10, -4, 8, -2};
```

此外，如果变量包含 `static` 限定符，那么 Vivado HLS 会对 RTL 设计和 FPGA 比特流中的变量进行初始化。因此，无需经历多个时钟周期来初始化内存，并且可确保大型内存初始化不会产生任何运算开销。

RTL 配置命令可指定应用复位后，静态变量是否返回其初始状态（非缺省设置）。如果执行复位操作后内存将返回初始状态，则会导致运算开销，并且需要经历多个周期才能完成值的复位。每个值都必须写入每个内存地址。

实现 ROM

Vivado® HLS 不要求通过使用 `static` 限定符指定的数组来对内存进行综合，也不要求使用 `const` 限定符来推断内存是否应为 ROM。Vivado HLS 会分析设计并尝试创建最优化的硬件。

赛灵思强烈建议针对旨在用作为内存的数组使用静态限定符。如“数组初始化”中所述，静态类型与 RTL 中的内存的行为方式几乎完全相同。

如果数组为只读，则也建议对其使用 `const` 限定符，因为 Vivado HLS 并不总是推断设计分析是否应使用 ROM。ROM 的自动推断的一般规则是局部静态（非全局）数组先写入后读取。在代码中遵循下列实践有助于推断 ROM：

- 在使用数组的函数中尽早初始化该数组。
- 将写操作组合在一起。
- 勿将 `array` (ROM) 初始化写操作与非初始化代码交织在一起。
- 勿将不同值存储到相同数组元素中（将所有写操作一起组合到代码中）。
- 元素值计算不得依赖于除初始化循环计数器变量以外的任何非常量（在编译时）设计变量。

如果使用复数赋值来初始化 ROM（例如，来自 `math.h` 库的函数），将数组初始化置于独立函数中即可允许推断 ROM。在以下示例中，完成 RTL 综合后，将 `sin_table[256]` 数组推断为内存，并作为 ROM 来实现。

```
#include "array_ROM_math_init.h"
#include <math.h>

void init_sin_table(din1_t sin_table[256])
{
    int i;
    for (i = 0; i < 256; i++) {
        dint_t real_val = sin(M_PI * (dint_t)(i - 128) / 256.0);
        sin_table[i] = (din1_t)(32768.0 * real_val);
    }
}

dout_t array_ROM_math_init(din1_t inval, din2_t idx)
{
    short sin_table[256];
    init_sin_table(sin_table);
    return (int)inval * (int)sin_table[idx];
}
```



提示：由于 `sin()` 函数的结果生成了常量值，因此 RTL 设计中无需任何核即可实现 `sin()` 函数。

数据类型

编译为可执行文件的 C 语言函数中使用的数据类型会影响结果的准确性以及内存要求，并且可能影响性能。

- 32 位整数 `int` 数据类型可包含更多数据，因此精度比 8 位 `char` 类型更高，但需更多存储空间。
- 如果在 32 位系统上使用 64 位 `long long` 类型，则会影响运行时间，因为通常需要多次访问以读取和写入相应的值。

同样，将 C 语言函数综合到 RTL 实现中时，类型会影响 RTL 设计的精度、面积和性能。用于变量的数据类型可判定所需的运算符大小，因此可判定 RTL 的面积和性能。

Vivado HLS 支持对所有标准 C 语言类型进行综合，包括精确宽度整数类型。

- `(unsigned) char`、`(unsigned) short` 和 `(unsigned) int`
- `(unsigned) long` 和 `(unsigned) long long`
- `(unsigned) intN_t`（其中，N 为 8、16、32 或 64，在 `stdint.h` 中定义）
- `float` 和 `double`

精确宽度整数类型可用于确保设计在各种类型的系统之间的可移植性。

C 语言标准规定 `(unsigned)long` 整数类型在 64 位操作系统上作为 64 位类型来实现，在 32 位操作系统上作为 32 位类型来实现。综合可根据运行 Vivado HLS 的操作系统类型来匹配此行为并生成不同大小的运算符，从而生成不同 RTL 设计。在 Windows 操作系统上，Microsoft 将 `long` 类型定义为 32 位，与操作系统无关。

- 针对 32 位，使用 `(unsigned)int` 或 `(unsigned)int32_t` 数据类型代替 `(unsigned)long` 类型。
- 针对 64 位，使用 `(unsigned)long long` 或 `(unsigned)int64_t` 数据类型代替 `(unsigned)long` 类型。

注释：C/C++ 编译选项 `-m32` 可用于指定按对应 C 语言仿真的方式来编译此代码，并按 32 位架构规范来对此代码进行综合。这样可确保将 long 数据类型作为 32 位值来实现。该选项可通过使用 `-CFLAGS` 选项来应用于 `add_files` 命令。

赛灵思强烈建议在单一公用头文件中定义所有变量的数据类型，该头文件可包含在所有源文件中。

- 在一般 Vivado HLS 工程中，可对部分数据类型进行调优，例如，用于减小其大小和支持更高效的硬件实现。
- 提高工作的抽象层次的主要作用之一在于能够快速创建新设计实现。通常在后续工程中使用的文件是相同的，但可能使用的数据类型不同（更小、更大或更准确）。

在单一位置更改数据类型更便于完成这 2 项任务：替代方法是对多个文件进行编辑。



重要提示！ 在头文件中使用宏时，请始终使用专有名称。例如，如果头文件中已定义 1 个名为 `_TYPES_H` 的宏，那么在其它系统文件中也可定义此相同名称，这可能启用或禁用某些其它代码，导致不可预见的意外结果。

标准类型

以下代码示例显示了所执行的部分基本算术运算。

```
#include "types_standard.h"

void types_standard(din_A inA, din_B inB, din_C inC, din_D inD,
  dout_1 *out1, dout_2 *out2, dout_3 *out3, dout_4 *out4
) {

  // Basic arithmetic operations
  *out1 = inA * inB;
  *out2 = inB + inA;
  *out3 = inC / inA;
  *out4 = inD % inA;

}
```

以上示例中的数据类型是在 `types_standard.h` 头文件中定义的，如下代码示例所示。其中显示了以下类型的使用方式：

- 标准有符号类型
- 无符号类型
- 精确宽度的整数类型（包含 `stdint.h` 头文件）

```
#include <stdio.h>
#include <stdint.h>

#define N 9

typedef char din_A;
typedef short din_B;
typedef int din_C;
typedef long long din_D;

typedef int dout_1;
typedef unsigned char dout_2;
typedef int32_t dout_3;
```



```
typedef int64_t dout_4;

void types_standard(din_A inA,din_B inB,din_C inC,din_D inD,dout_1
*out1,dout_2 *out2,dout_3 *out3,dout_4 *out4);
```

这些不同类型经过综合可生成以下运算符和端口大小：

- 用于计算结果 `out1` 的乘法器为 24 位乘法器。8 位 `char` 类型乘以 16 位 `short` 类型需要 24 位乘法器。结果是符号位将扩展为 32 位，以与输出端口宽度匹配。
- 用于 `out2` 的加法器为 8 位。由于输出为 8 位 `unsigned char` 类型，因此仅将底部 8 位 `inB`（16 位 `short`）与 8 位 `char` 类型的 `inA` 相加。
- 对于 `out3` 输出（32 位精确宽度类型），8 位 `char` 类型的 `inA` 的符号位将扩展为 32 位值，并使用 32 位（`int` 类型）`inC` 输入执行 32 位除法运算。
- 使用 64 位 `long long` 类型的 `inD` 来执行 64 位模数运算，8 位 `char` 类型的 `inA` 的符号位将扩展至 64 位，以创建 64 位输出 `out4` 的结果。

根据 `out1` 结果指示，Vivado HLS 使用可用的最小运算符，并扩展结果以与所需输出位宽相匹配。对于 `out2` 结果，即使其中一项输入为 16 位，仍可使用 8 位加法器，因为只需 8 位输出。正如 `out3` 和 `out4` 的结果所示，如需所有位元，则将对完整大小的运算符进行综合。

浮点与双精度

Vivado HLS 支持 `float` 和 `double` 类型的综合。这两种数据类型均依据 IEEE-754 标准来进行综合。

- 单精度 32 位
 - 24 位小数
 - 8 位指数
- 双精度 64 位
 - 53 位小数
 - 11 位指数



建议：使用浮点数据类型时，赛灵思强烈建议您参阅《使用 Vivado HLS 进行浮点设计》(XAPP599)

除了将浮点和双精度类型用于标准算术运算（例如，+、-、* 等）外，浮点和双精度通常还可配合 `math.h` 一起使用（针对 C++，这 2 个类型还可配合 `cmath.h` 一起使用）。本节讨论了为标准运算符提供的支持。

以下代码示例显示的头文件可配合更新后的[标准类型](#)一起使用，以将数据类型定义为 `double` 和 `float` 类型。

```
#include <stdio.h>
#include <stdint.h>
#include <math.h>

#define N 9

typedef double din_A;
typedef double din_B;
typedef double din_C;
typedef float din_D;

typedef double dout_1;
```

```
typedef double dout_2;
typedef double dout_3;
typedef float dout_4;

void types_float_double(din_A inA,din_B inB,din_C inC,din_D inD,dout_1
*out1,dout_2 *out2,dout_3 *out3,dout_4 *out4);
```

此更新后的头文件可配合以下代码示例一起使用，此代码中使用了 `sqrtf()` 函数。

```
#include "types_float_double.h"

void types_float_double(
    din_A inA,
    din_B inB,
    din_C inC,
    din_D inD,
    dout_1 *out1,
    dout_2 *out2,
    dout_3 *out3,
    dout_4 *out4
) {

    // Basic arithmetic & math.h sqrtf()
    *out1 = inA * inB;
    *out2 = inB + inA;
    *out3 = inC / inA;
    *out4 = sqrtf(inD);

}
```

对以上示例进行综合后，可生成 64 位双精度乘法器、加法器和除法器运算符。这些运算符可由相应的浮点赛灵思 IP 目录核来实现。

使用 `sqrtf()` 的平方根函数是使用 32 位单精度浮点核来实现的。

如果使用双精度平方根函数 `sqrt()`，则将生成额外逻辑以在用于 `inD` 的 32 位单精度浮点类型与其它类型之间进行双向强制转换，`out4: sqrt()` 为双精度 (double) 函数，`sqrtf()` 为单精度 (float) 函数。

在 C 语言函数中，混用浮点类型和双精度类型时请谨慎处理，因为浮点到双精度和双精度到浮点转换单位是在硬件中推断所得的。

```
float foo_f      = 3.1459;
float var_f = sqrt(foo_f);
```

以上代码可生成如下硬件：

```
wire(foo_t)
-> Float-to-Double Converter unit
-> Double-Precision Square Root unit
-> Double-to-Float Converter unit
-> wire (var_f)
```

使用 `sqrtf()` 函数即可：

- 使硬件无需再使用类型转换器。

- 节省面积。
- 改进时序。

对浮点和双精度类型进行综合时，Vivado HLS 会保留 C 语言代码中执行的运算顺序，以确保结果与 C 语言仿真结果相同。由于执行的饱和与截断操作，在单精度运算和双精度运算中不保证：

```
A=B*C; A=B*F;  
D=E*F; D=E*C;  
O1=A*D O2=A*D;
```

对于 `float` 和 `double` 类型，不保证 `O1` 和 `O2` 相同。



提示：在某些情况下（视设计而定），展开或部分展开循环等最优化可能无法充分利用并行计算，因为 Vivado HLS 对浮点和双精度类型进行综合时会严格保留运算顺序。

对于 C++ 设计，Vivado HLS 可提供最常用的数学函数的位近似实现。

任意精度数据类型

Vivado HLS 可提供任意精度数据类型，如[浮点与双精度](#)中所述。

复合数据类型

Vivado HLS 针对综合支持以下复合数据类型：

- 结构体 (struct)
- 枚举 (enum)
- 联合体 (union)

结构体

使用结构体作为顶层函数实参时，综合创建的端口即结构体成员的直接反映。标量成员是作为标准标量端口来实现的，而数组默认则是作为内存端口来实现的。

在此设计示例中，`struct data_t` 是在头文件中定义的，如以下代码示例所示。此结构体包含 2 个数据成员：

- 无符号的矢量 `A`，类型为 `short`（16 位）。
- 数组 `B`，含 4 种 `unsigned char` 类型（8 位）。

```
typedef struct {  
    unsigned short A;  
    unsigned char B[4];  
} data_t;  
  
data_t struct_port(data_t i_val, data_t *i_pt, data_t *o_pt);
```

- 在以下代码示例中，结构体用作按值传递实参（从 `i_val` 到返回的 `o_val`）并作为指针（`*i_pt` 到 `*o_pt`）。

```
#include "struct_port.h"

data_t struct_port(
    data_t i_val,
    data_t *i_pt,
    data_t *o_pt
) {

    data_t o_val;
    int i;

    // Transfer pass-by-value structs
    o_val.A = i_val.A+2;
    for (i=0;i<4;i++) {
        o_val.B[i] = i_val.B[i]+2;
    }

    // Transfer pointer structs
    o_pt->A = i_pt->A+3;
    for (i=0;i<4;i++) {
        o_pt->B[i] = i_pt->B[i]+3;
    }

    return o_val;
}
```

所有函数实参和函数返回都综合到端口中，如下所示：

- Struct 元素 A 生成 16 位端口。
- Struct 元素 B 生成访问 4 个元素的 RAM 端口。

对于 Vivado HLS 可综合的结构体的大小或复杂性并无限制。结构体中可包含所需的任意数量的数组维度和成员。仅当数组作为流传输（例如，FIFO 接口）来实现时，才会出现结构体实现的唯一限制。在此情况下，请遵循适用于接口（FIFO 接口）上的数组的常规规则进行操作。

结构体上的元素可通过数据打包最优化来打包到单一矢量中。如需了解更多信息，请参阅用于执行此最优化的 `set_directive_data_pack` 命令相关信息。此外，通过 `config-interface` 命令的 `-trim_dangling_ports` 选项可从接口移除结构体中未使用的元素。

枚举类型

以下代码示例中的头文件定义了一些 `enum` 类型，在 `struct` 中使用。随后在其它 `struct` 中使用了该 `struct`。这样可以直观描述要采集的复杂类型。

以下代码示例演示如何指定和综合复杂定义 (`MAD_NSBSAMPLES`) 语句。

```
#include <stdio.h>

enum mad_layer {
    MAD_LAYER_I    = 1,
    MAD_LAYER_II   = 2,
    MAD_LAYER_III  = 3
};

enum mad_mode {
    MAD_MODE_SINGLE_CHANNEL = 0,
```

```

MAD_MODE_DUAL_CHANNEL = 1,
MAD_MODE_JOINT_STEREO = 2,
MAD_MODE_STEREO = 3
};

enum mad_emphasis {
    MAD_EMPHASIS_NONE = 0,
    MAD_EMPHASIS_50_15_US = 1,
    MAD_EMPHASIS_CCITT_J_17 = 3
};

typedef    signed int mad_fixed_t;

typedef struct mad_header {
    enum mad_layer layer;
        enum mad_mode mode;
    int mode_extension;
    enum mad_emphasis emphasis;

    unsigned long long bitrate;
    unsigned int samplerate;

    unsigned short crc_check;
    unsigned short crc_target;

    int flags;
    int private_bits;
} header_t;

typedef struct mad_frame {
    header_t header;
    int options;
    mad_fixed_t sbsample[2][36][32];
} frame_t;

# define MAD_NSBSAMPLES(header) \
    ((header)->layer == MAD_LAYER_I ? 12 : \
    (((header)->layer == MAD_LAYER_III && \
    ((header)->flags & 17)) ? 18 : 36))

void types_composite(frame_t *frame);

```

下例中使用了上例中定义的 `struct` 和 `enum` 类型。如果在顶层函数的实参中使用 `enum`，则会将枚举综合为 32 位值以符合标准 C 语言编译行为。如果枚举类型在设计内部，则 Vivado HLS 会将其最优化到必需的最小位数。

以下代码示例显示在综合期间如何忽略 `printf` 语句。

```

#include "types_composite.h"

void types_composite(frame_t *frame)
{
    if (frame->header.mode != MAD_MODE_SINGLE_CHANNEL) {
        unsigned int ns, s, sb;
        mad_fixed_t left, right;

        ns = MAD_NSBSAMPLES(&frame->header);
        printf("Samples from header %d \n", ns);

        for (s = 0; s < ns; ++s) {

```

```
for (sb = 0; sb < 32; ++sb) {
    left = frame->sbsample[0][s][sb];
    right = frame->sbsample[1][s][sb];
    frame->sbsample[0][s][sb] = (left + right) / 2;
}
}
frame->header.mode = MAD_MODE_SINGLE_CHANNEL;
}
}
```

联合体

在以下代码示例中，已创建含 `double` 和 `struct` 的联合体 (`union`)。不同于 C 编译，综合不保证针对 `union` 中的所有字段使用相同内存（对于综合，则不保证使用相同寄存器）。Vivado HLS 通过执行最优化来提供最优硬件。

```
#include "types_union.h"

dout_t types_union(din_t N, dinfp_t F)
{
    union {
        struct {int a; int b; } intval;
        double fpval;
    } intfp;
    unsigned long long one, exp;

    // Set a floating-point value in union intfp
    intfp.fpval = F;

    // Slice out lower bits and add to shifted input
    one = intfp.intval.a;
    exp = (N & 0x7FF);

    return ((exp << 52) + one) & (0x7fffffffffffffffffLL);
}
```

Vivado HLS 不支持：

- 顶层函数接口上的联合体。
- 重新解读指针以供综合。因此，联合体不能保存不同类型的指针，也不能保存指向不同类型的数组的指针。
- 通过其它变量访问联合体。同样使用上述示例中的联合体，不支持：

```
for (int i = 0; i < 6; ++i)
if (i<3)
    A[i] = intfp.intval.a + B[i];
else
    A[i] = intfp.intval.b + B[i];
}
```

- 但可将其显式重编码为：

```
A[0] = intfp.intval.a + B[0];
A[1] = intfp.intval.a + B[1];
A[2] = intfp.intval.a + B[2];
A[3] = intfp.intval.b + B[3];
A[4] = intfp.intval.b + B[4];
A[5] = intfp.intval.b + B[5];
```

对联合体进行综合不支持原生 C 类型与用户定义的类型之间进行强制类型转换。

在 VHLS 设计中，联合体常用于转换原始位元的数据类型。通常在顶层端口接口处使用浮点值时，需使用此原始位元转换。请参阅以下示例：

```
typedef float T;
unsigned int value; // the "input" of the conversion
T myhalfvalue; // the "output" of the conversion
union
{
    unsigned int as_uint32;
    T as_floatingpoint;
} my_converter;
my_converter.as_uint32 = value;
myhalfvalue = my_converter.as_floatingpoint;
```

这种类型的代码适用于 C 语言的 float 数据类型，通过修改还可用于 double 数据类型。将 typedef 和 int 更改为 short 对于 half 数据类型无效，但由于 half 属于类，因此在联合体中无法使用。可改用以下代码：

```
typedef half T;
short value;
T myhalfvalue = static_cast<T>(value);
```

同样，相反方式的转换使用的是 value=static_cast<ap_uint<16>>(myhalfvalue) 或 static_cast<unsigned short>(myhalfvalue)。

```
ap_fixed<16,4> afix = 1.5;
ap_fixed<20,6> bfix = 1.25;
half ahlf = afix.to_half();
half bhlf = bfix.to_half();
```

另一种方法是使用帮助程序类 fp_struct<half> 来通过 data() 或 to_int() 方法进行转换。使用 hls/utils/x_hls_utils.h 头文件。

类型限定符

类型 (type) 限定符可直接影响高层次综合创建的硬件。一般情况下，此类限定符会以可预测的方式来影响综合结果，如下所述。Vivado HLS 仅受限定符解读的限制，因为它会影响功能行为，并且可执行最优化以创建更优化的硬件设计。在每个限定符概述后提供了此行为的示例。

Volatile

在函数接口上多次访问指针时，`volatile` 限定符会影响 RTL 中执行的读取或写入操作次数。虽然 `volatile` 限定符会影响层级内所有函数中的此行为，但 `volatile` 限定符的影响主要常见于顶层接口的讨论中。

任意精度类型不支持使用易变 (`volatile`) 限定符执行算术运算。对于使用 `volatile` 限定符的所有任意精度数据类型，必须将其指定为非易变数据类型，才可在算术表达式中使用。

相关信息

[理解易变 \(Volatile\) 数据](#)

静态

函数中的静态 (`static`) 类型在函数调用之间保留其值。硬件设计中的等效行为是寄存变量（触发器或内存）。如果 C 语言函数正确执行时要求变量是静态类型，则在最终的 RTL 设计中它肯定是寄存器。该值必须在函数和设计的调用之间保持不变。

综合后并不是只有 `static` 类型会实现为寄存器。Vivado HLS 会判定在 RTL 设计中需要将哪些变量实现为寄存器。例如，如果必须在多个周期内保持变量赋值不变，则即使 C 语言函数中的原始变量不是静态类型，Vivado HLS 也会创建一个寄存器来保存该值。

Vivado HLS 遵守静态函数的初始化行为，并在初始化期间对寄存器赋值 0（或任何显式初始化的值）。这意味着 `static` 变量在 RTL 代码和 FPGA 比特流中初始化。但这并不意味着每次复位信号时都会重新初始化该变量。

请参阅 RTL 配置 (`config_rtl` 命令) 以确定有关系统复位的静态初始化值的实现方式。

Const

`const` 类型用于指定变量值从不更新。该变量仅供读取，而无法写入，因此必须初始化。这通常意味着在 RTL 设计中，大部分 `const` 变量都会减缩为常量。Vivado HLS 会执行常量传输并移除任何不必要的硬件。

对于数组，`const` 变量在最终 RTL 设计中作为 ROM 来实现（Vivado HLS 不会像对小型数组一样对此类变量执行自动分区）。使用 `const` 限定符指定的数组与静态限定符一样在 RTL 和 FPGA 比特流中进行初始化。无需将其复位，因为从不对其执行写入。

Vivado HLS 最优化

以下代码示例显示了即使指定数组不含 `static` 或 `const` 限定符，但 Vivado HLS 仍实现 ROM 的状况。其中着重讲解了 Vivado HLS 分析设计和判定最优化实现的方式。限定符的有无会影响但并不支配最终 RTL。

```
#include "array_ROM.h"

dout_t array_ROM(din1_t inval, din2_t idx)
{
    din1_t lookup_table[256];
    dint_t i;

    for (i = 0; i < 256; i++) {
        lookup_table[i] = 256 * (i - 128);
    }

    return (dout_t)inval * (dout_t)lookup_table[idx];
}
```


对于前述示例，Vivado HLS 能够判定采用变量 `lookup_table` 作为最终 RTL 中的内存元素的情况下实现结果最佳。

全局变量

全局变量可在代码中自由使用并且完全可综合。默认情况下，全局变量在 RTL 接口上不公开为端口。

以下代码示例显示了全局变量的默认综合行为。其中使用了 3 个全局变量。虽然此示例使用数组，但 Vivado HLS 支持所有类型的全局变量。

- 从 `Ain` 数组读取值。
- `Aint` 数组用于将值从 `Ain` 变换并传输到 `Aout`。
- 输出写入 `Aout` 数组。

```
din_t Ain[N];
din_t Aint[N];
dout_t Aout[N/2];

void types_global(din1_t idx) {
    int i,lidx;

    // Move elements in the input array
    for (i=0; i<N; ++i) {
        lidx=i;
        if(lidx+idx>N-1)
            lidx=i-N;
        Aint[lidx] = Ain[lidx+idx] + Ain[lidx];
    }

    // Sum to half the elements
    for (i=0; i<(N/2); i++) {
        Aout[i] = (Aint[i] + Aint[i+1])/2;
    }
}
```

默认情况下，综合后，RTL 设计上的唯一端口即 `idx` 端口。默认情况下，全局变量不公开为 RTL 端口。默认情况下：

- `Ain` 数组为可供读取的内部 RAM。
- `Aout` 数组为可供写入的内部 RAM。

将全局变量作为 I/O 端口公开

虽然默认情况下，全局变量并不作为 I/O 端口公开，但可通过使用 `expose_global` 选项来将其作为 I/O 端口予以公开。接口配置中的 `expose_global` 选项可将所有全局变量作为 RTL 接口上的端口予以公开。接口配置可通过如下方式来设置：

- “Solution Settings” → “General”，或者
- 使用 `config_interface Tcl` 命令

使用接口配置公开全局变量时，设计中的所有全局变量都作为 I/O 端口公开，包括仅在设计内部访问的全局变量。

最后，如果以静态限定符指定任意全局变量，则无法将此类全局变量综合为 I/O 端口。

总之，虽然 Vivado HLS 支持对全局变量进行综合，但赛灵思不建议广泛使用全局变量的编码样式。

指针

指针在 C 语言代码中使用广泛，并且受到妥善支持，可用于综合。使用指针时，请注意如下用例：

- 在同一函数内多次访问（读取或写入）指针时。
- 使用指针数组时，每个指针都必须指向 1 个标量或标量数组（而不是另一指针）。
- 仅支持标准 C 语言类型间的指针强制转换，如下所示。

以下代码示例显示支持针对指向多个对象的指针执行综合。

```
#include "pointer_multi.h"

dout_t pointer_multi (sel_t sel, din_t pos) {
    static const dout_t a[8] = {1, 2, 3, 4, 5, 6, 7, 8};
    static const dout_t b[8] = {8, 7, 6, 5, 4, 3, 2, 1};

    dout_t* ptr;
    if (sel)
        ptr = a;
    else
        ptr = b;

    return ptr[pos];
}
```

Vivado HLS 支持指针到指针类型的综合，但不支持在顶层接口上执行，即不支持将其作为顶层函数的实参来进行综合。如果在多个函数中使用指针到指针，那么 Vivado HLS 会将使用指针到指针的所有函数内联。内联多个函数可能导致运行时间增加。

```
#include "pointer_double.h"

data_t sub(data_t ptr[10], data_t size, data_t**flagPtr)
{
    data_t x, i;

    x = 0;
    // Sum x if AND of local index and pointer to pointer index is true
    for(i=0; i<size; ++i)
        if (**flagPtr & i)
            x += *(ptr+i);
    return x;
}

data_t pointer_double(data_t pos, data_t x, data_t* flag)
{
    data_t array[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    data_t* ptrFlag;
    data_t i;

    ptrFlag = flag;

    // Write x into index position pos
    if (pos >=0 & pos < 10)
```

```

*(array+pos) = x;

// Pass same index (as pos) as pointer to another function
return sub(array, 10, &ptrFlag);
}

```

指针数组同样可进行综合。请参阅以下代码示例，其中使用指针数组来存储全局数组的第二维度的起始位置。指针数组中的指针只能指向标量或标量数组，不能指向其它指针。

```

#include "pointer_array.h"

data_t A[N][10];

data_t pointer_array(data_t B[N*10]) {
    data_t i,j;
    data_t sum1;

    // Array of pointers
    data_t* PtrA[N];

    // Store global array locations in temp pointer array
    for (i=0; i<N; ++i)
        PtrA[i] = &(A[i][0]);

    // Copy input array using pointers
    for(i=0; i<N; ++i)
        for(j=0; j<10; ++j)
            *(PtrA[i]+j) = B[i*10 + j];

    // Sum input array
    sum1 = 0;
    for(i=0; i<N; ++i)
        for(j=0; j<10; ++j)
            sum1 += *(PtrA[i] + j);

    return sum1;
}

```

如果使用原生 C 语言类型，则支持对指针强制转换进行综合。在以下代码示例中，`int` 类型将强制转换为 `char` 类型。

```

#define N 1024

typedef int data_t;
typedef char dint_t;

data_t pointer_cast_native (data_t index, data_t A[N]) {
    dint_t* ptr;
    data_t i =0, result = 0;
    ptr = (dint_t*)&A[index];

    // Sum from the indexed value as a different type
    for (i = 0; i < 4*(N/10); ++i) {
        result += *ptr;
        ptr+=1;
    }
    return result;
}

```

Vivado HLS 不支持一般类型间的指针强制转换。例如，如果创建 (struct) 复合类型的有符号值，那么无法通过对该指针进行强制转换来赋值无符号值。

```
struct {
    short first;
    short second;
} pair;

// Not supported for synthesis
*(unsigned*)&pair = -1U;
```

在此类情况下，必须使用原生类型对这些值进行赋值。

```
struct {
    short first;
    short second;
} pair;

// Assigned value
pair.first = -1U;
pair.second = -1U;
```

接口上的指针

指针可作为顶层函数的实参。了解综合期间指针的实现方式至关重要，因为有时指针可能在综合后实现期望的 RTL 接口与设计时引发问题。

基本指针

如以下代码示例所示，顶层接口上具有基本指针的函数不会对 Vivado HLS 产生影响。指针可综合为简单的有线接口或使用握手的接口协议。



提示： 仅限只读或只写指针才能综合为 FIFO 接口。

```
#include "pointer_basic.h"

void pointer_basic (dio_t *d) {
    static dio_t acc = 0;

    acc += *d;
    *d = acc;
}
```

每个函数调用仅对接口上的指针执行一次读取或写入。以下代码示例中显示了测试激励文件。

```
#include "pointer_basic.h"

int main () {
    dio_t d;
    int i, retval=0;
    FILE *fp;

    // Save the results to a file
```

```
fp=fopen(result.dat,w);
printf( Din Dout\n, i, d);

// Create input data
// Call the function to operate on the data
for (i=0;i<4;i++) {
    d = i;
    pointer_basic(&d);
    fprintf(fp, %d \n, d);
    printf( %d %d\n, i, d);
}
fclose(fp);

// Compare the results file with the golden results
retval = system(diff --brief -w result.dat result.golden.dat);
if (retval != 0) {
    printf(Test failed!!!\n);
    retval=1;
} else {
    printf(Test passed!\n);
}

// Return 0 if the test
return retval;
}
```

C 和 RTL 仿真通过以下简单数据集验证运算是否正确（尽管并未列出所有可能的情况）：

```
Din Dout
0 0
1 1
2 3
3 6
Test passed!
```

指针算法

引入指针算法会限制 RTL 中可综合的接口。以下代码示例显示了相同的代码，但在此实例中使用简单的指针算法来累加数据值（从第 2 个值开始）。

```
#include "pointer_arith.h"

void pointer_arith (dio_t *d) {
    static int acc = 0;
    int i;

    for (i=0;i<4;i++) {
        acc += *(d+i+1);
        *(d+i) = acc;
    }
}
```

以下代码示例显示了支持此示例的测试激励文件。由于执行累加的循环当前位于 `pointer_arith` 函数内部，测试激励文件会填充含相应值的 `d[5]` 数组所指定的地址空间。

```
#include "pointer_arith.h"

int main () {
    dio_t d[5], ref[5];
    int i, retval=0;
    FILE      *fp;

    // Create input data
    for (i=0;i<5;i++) {
        d[i] = i;
        ref[i] = i;
    }

    // Call the function to operate on the data
    pointer_arith(d);

    // Save the results to a file
    fp=fopen(result.dat,w);
    printf( Din Dout\n, i, d);
    for (i=0;i<4;i++) {
        fprintf(fp, %d \n, d[i]);
        printf(  %d  %d\n, ref[i], d[i]);
    }
    fclose(fp);

    // Compare the results file with the golden results
    retval = system(diff --brief -w result.dat result.golden.dat);
    if (retval != 0) {
        printf(Test failed!!!\n);
        retval=1;
    } else {
        printf(Test passed!\n);
    }

    // Return 0 if the test
    return retval;
}
```

仿真后，会生成如下输出：

```
Din Dout
0    1
1    3
2    6
3   10
Test passed!
```

指针算法并不会按顺序访问指针数据。连线、握手或 FIFO 接口无法访问无序数据：

- 当设计准备好使用数据或写入数据（当数据就绪后）时，连线接口会读取数据。
- 当控制信号允许继续操作时，握手和 FIFO 接口会执行读写操作。

在上述 2 种情况下，数据都必须按顺序到达（并写入），从元素 0 开始。在“含指针算法的接口”示例中，代码声明从索引 1 读取第 1 个数据值（`i` 从 0 开始，`0+1=1`）。这是来自测试激励文件中的 `d[5]` 数组的第 2 个元素。

在硬件中实现此示例后，需要某种形式的数据索引。Vivado HLS 不支持对连线、握手或 FIFO 接口建立数据索引。

“含指针算法的接口”示例中的代码只能通过 `ap_bus` 接口来进行综合。访问（读取或写入）数据时，此接口可提供用于建立数据索引的地址。

或者，必须修改此代码，在接口上使用数组代替指针，如以下示例所示。在综合中可通过 RAM (`ap_memory`) 接口来实现此代码。此接口可使用地址来建立数据索引，并执行乱序或无序访问。

连线、握手或 FIFO 接口只能用于流传输数据，不能搭配指针算法使用，除非它从 0 位数据开始建立数据索引，然后按顺序进行操作。

```
#include "array_arith.h"

void array_arith (dio_t d[5]) {
    static int acc = 0;
    int i;

    for (i=0;i<4;i++) {
        acc += d[i+1];
        d[i] = acc;
    }
}
```

多重访问指针接口：数据流传输

在顶层函数的实参列表中使用指针的设计在使用指针执行多重访问时需考量一些特殊注意事项。当任一指针以相同方式多次执行读取或写入时，就会发生多重访问。

- 您必须对多次访问的任意函数实参使用 `volatile` 限定符。
- 在顶层函数上，如果在 Vivado HLS 中使用协同仿真来验证 RTL，则必须指定任意此类实参对端口接口的访问次数。
- 执行综合前，请务必验证 C 语言代码以确认其意图以及 C 模型是否正确。

如果对设计进行建模的过程需多次访问某一函数实参，赛灵思建议您使用流传输对设计进行建模。使用流传输可确保您不会遇到本节中所述的问题。下表中的设计使用的是[编码示例](#)。

表 45：设计场景示例

示例设计	显示
<code>pointer_stream_bad</code>	多次访问同一函数内的指针时需要 <code>volatile</code> 限定符的原因。
<code>pointer_stream_better</code>	使用 C 语言测试激励文件来验证顶层接口上包含此类指针的设计以确保对目标行为进行正确建模的原因。

在以下代码示例中，对输入指针 `d_i` 执行 4 次读取，对输出 `d_o` 执行 2 次写入，目标是通过 FIFO 接口来实现访问（对流入和流出最终 RTL 实现的数据进行流传输）。

```
#include "pointer_stream_bad.h"

void pointer_stream_bad ( dout_t *d_o,  din_t *d_i) {
    din_t acc = 0;

    acc += *d_i;
    acc += *d_i;
```

```
*d_o = acc;
acc += *d_i;
acc += *d_i;
*d_o = acc;
}
```

用于验证此设计的测试激励文件如下代码示例所示。

```
#include "pointer_stream_bad.h"

int main () {
    din_t d_i;
    dout_t d_o;
    int retval=0;
    FILE *fp;

    // Open a file for the output results
    fp=fopen(result.dat,w);

    // Call the function to operate on the data
    for (d_i=0;d_i<4;d_i++) {
        pointer_stream_bad(&d_o,&d_i);
        fprintf(fp, "%d %d\n", d_i, d_o);
    }
    fclose(fp);

    // Compare the results file with the golden results
    retval = system(diff --brief -w result.dat result.golden.dat);
    if (retval != 0) {
        printf(Test failed !!!\n);
        retval=1;
    } else {
        printf(Test passed !\n);
    }

    // Return 0 if the test
    return retval;
}
```

理解易变 (Volatile) 数据

多重访问指针接口示例中编写的代码中，输入指针 `d_i` 和输出指针 `d_o` 在 RTL 中是作为 FIFO（或握手）接口实现的，目的是为了确：

- 每次在 RTL 端口 `d_i` 上执行读取时，上游生产者块都可提供新数据。
- 每次写入 RTL 端口 `d_o` 时，下游使用者块都可接受新数据。

通过标准 C 语言编译器来编译此代码时，针对每个指针的多重访问会减少为单次访问。就编译器而言，并无指示表明 `d_i` 上的数据在函数执行期间发生更改，仅限最终写入 `d_o` 的数据才相关。其它写入在函数完成时被覆盖。

Vivado HLS 与 `gcc` 编译器的行为相符，将这些读写最优化到单一读操作和单一写操作中。检验 RTL 时，每个端口上只有单一读写操作。

此设计的基本问题在于测试激励文件和设计无法对您期望的 RTL 端口实现方式进行建模。

- 您期望 RTL 端口在每次传输事务期间进行多次读写（并且可通过流传输来输入和输出数据）。

- 测试激励文件仅提供单一输入值，并且仅返回单一输出值。按[多重访问指针接口：数据流传输](#)所示进行 C 语言仿真可显示如下结果，证明每次输入累加 4 次。同样的值读取一次，且每次都累加。这并非 4 次不同的读取操作。

Din	Dout
0	0
1	4
2	8
3	12

- 为了使该设计能够多次读写 RTL 端口，请使用 `volatile` 限定符。请参阅以下代码示例。

`volatile` 限定符会告知 C 语言编译器（和 Vivado HLS）不假定指针访问次数。即数据为易变数据，可能更改。



提示：请勿对指针访问次数进行优化。

```
#include "pointer_stream_better.h"

void pointer_stream_better ( volatile dout_t *d_o,  volatile din_t *d_i) {
    din_t acc = 0;

    acc += *d_i;
    acc += *d_i;
    *d_o = acc;
    acc += *d_i;
    acc += *d_i;
    *d_o = acc;
}
```

以上示例按[多重访问指针接口：数据流传输](#)方式进行仿真，但会使用 `volatile` 限定符。

- 阻止指针访问最优化。
- 生成的 RTL 设计会对输入端口 `d_i` 执行期望的 4 次读取，对输出端口 `d_o` 执行 2 次写入。

即使使用 `volatile` 关键字，此编码样式（多次访问指针）仍存在函数和测试激励文件无法对多次独立的读写操作进行充分建模的问题。

在此情况下，将执行 4 次读取，但只对相同数据读取 4 次。存在 2 次独立的写入，每次写入数据正确，但测试激励文件仅捕获最终写入的数据。

注释：要查看中间访问，请启用 `cosim_design` 以在 RTL 仿真期间创建追踪文件，并在相应的查看器中查看此追踪文件。

以上多重访问 Volatile 指针接口示例可通过连线接口来实现。如果指定 FIFO 接口，Vivado HLS 会创建 RTL 测试激励文件以在每次读取时对新数据执行流传输。由于测试激励文件未提供新数据，因此 RTL 验证失败。测试激励文件也就无法对读写操作进行正确建模。

对流传输数据接口进行建模

不同于软件，硬件系统的并行性质使其能够充分利用流传输数据。数据可持续不断提供给设计，设计可持续不断输出数据。RTL 设计可在设计完成处理现有数据之前接受新数据。

如[理解易变 \(Volatile\) 数据](#)所示，对软件中的流传输数据进行建模是很重要的，编写软件以对现有硬件实现进行建模（即并行/流传输性质已存在，且需加以建模）时尤其如此。

有多种方法可供使用：

- 添加 `volatile` 限定符，如“多重访问 `volatile` 指针接口”示例中所示。测试激励文件不会对专用读写进行建模，使用原 C 语言测试激励文件的 RTL 仿真可能失败，但通过查看追踪文件波形发现执行的读写操作正确。
- 修改代码以对显式专用读写进行建模。请参阅以下示例。
- 修改代码以使用流传输数据类型。流传输数据类型支持对使用流传输数据的硬件进行准确建模。

以下代码示例已更新，以确保它可从测试激励文件读取 4 个唯一值并写入 2 个唯一值。由于指针访问按顺序进行并从位置 0 开始，因此综合期间可使用流传输接口类型。

```
#include "pointer_stream_good.h"

void pointer_stream_good ( volatile dout_t *d_o,  volatile din_t *d_i) {
    din_t acc = 0;

    acc += *d_i;
    acc += *(d_i+1);
    *d_o = acc;
    acc += *(d_i+2);
    acc += *(d_i+3);
    *(d_o+1) = acc;
}
```

测试激励文件已更新，可对函数在每项传输事务内读取 4 个唯一值的行为进行建模。此新测试激励文件仅对单一传输事务进行建模。要对多项传输事务进行建模，必须增大输入数据集并多次调用该函数。

```
#include "pointer_stream_good.h"

int main () {
    din_t d_i[4];
    dout_t d_o[4];
    int i, retval=0;
    FILE *fp;

    // Create input data
    for (i=0;i<4;i++) {
        d_i[i] = i;
    }

    // Call the function to operate on the data
    pointer_stream_good(d_o,d_i);

    // Save the results to a file
    fp=fopen(result.dat,w);
    for (i=0;i<4;i++) {
        if (i<2)
            fprintf(fp, %d %d\n, d_i[i], d_o[i]);
        else
            fprintf(fp, %d \n, d_i[i]);
    }
    fclose(fp);

    // Compare the results file with the golden results
    retval = system(diff --brief -w result.dat result.golden.dat);
    if (retval != 0) {
        printf(Test failed  !!!\n);
        retval=1;
    } else {
        printf(Test passed !\n);
    }
}
```

```

}

// Return 0 if the test
return retval;
}

```

测试激励文件验证算法是否包含如下结果，结果显示：

- 单一传输事务有 2 项输出。
- 输出分别为前 2 次输入读取的累加，以及后 2 次输入读取的累加，再加上前 2 者的累加。

```

Din Dout
0      1
1      6
2
3

```

- 多次访问位于函数接口处的指针时需要注意的最后一个问题是 RTL 仿真建模。

多重访问指针和 RTL 仿真

当多次访问接口上的指针以进行读写时，Vivado HLS 无法从函数接口判定执行的读写次数。函数接口中的实参数可告知 Vivado HLS 读写的值的数量。

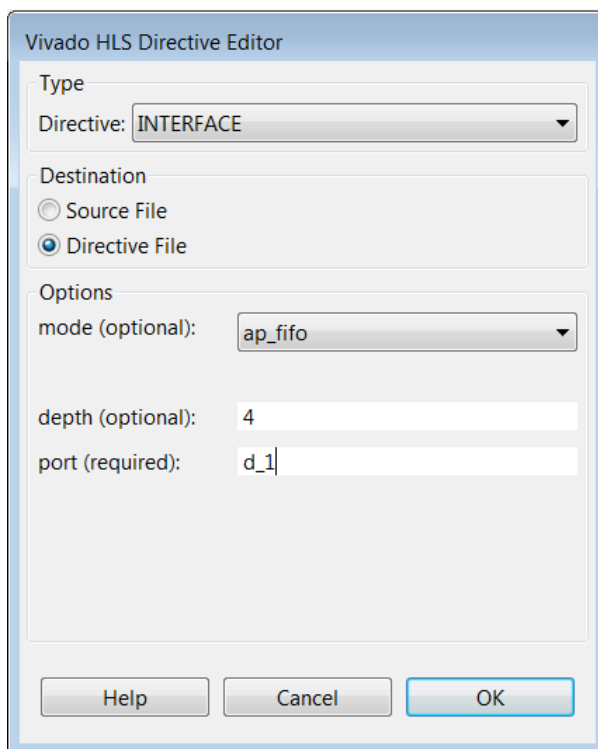
```
void pointer_stream_good (volatile dout_t *d_o, volatile din_t *d_i)
```

除非接口告知 Vivado HLS 所需的值的数量（例如，数组的最大大小），否则 Vivado HLS 假定仅含单个值，并仅为单一输入和单一输出创建 C/RTL 协同仿真。

如果 RTL 端口实际读写多个值，那么 RTL 协同仿真将中止运行。RTL 协同仿真会对连接到 RTL 设计的生产者和使用者块进行建模。如果需要多个值，RTL 设计会在尝试读写多个值时中止运行（因为当前没有要读取的值，或者没有可供写入的空间）。

在接口上使用多重访问指针时，必须将接口上的读写最大次数告知 Vivado HLS。指定接口时，请选择“INTERFACE”指令并设置“depth”选项，如下图所示。

图 86：含深度 (Depth) 选项的“Vivado HLS 指令编辑器 (Directive Editor)”对话框



在上例中，实参或端口 `d_i` 的 FIFO 接口深度设置为 4。这样可确保 RTL 协同仿真提供足够的值以正确验证 RTL。

C 语言内置函数

Vivado HLS 支持以下 C 语言内置函数：

- `__builtin_clz(unsigned int x)`：返回前置 0 位数量（以 `x` 表示），从最高有效位的位置开始。如果 `x` 为 0，则结果为未定义。
- `__builtin_ctz(unsigned int x)`：返回后置 0 位数量（以 `x` 表示），从最不显著的位的位置开始。如果 `x` 为 0，则结果为未定义。

以下示例显示这些函数的使用方式。此示例返回 `in0` 中前置 0 的数量和 `in1` 中后置 0 的数量之和：

```
int foo (int in0, int in1) {  
    int ldz0 = __builtin_clz(in0);  
    int ldz1 = __builtin_ctz(in1);  
    return (ldz0 + ldz1);  
}
```

硬件高效型 C 语言代码

针对 CPU 进行 C 语言代码编译时，编译器会将 C 语言代码转换并优化为一组 CPU 机器指令。大部分情况下，开发者的工作在此阶段完成。但如果存在性能需求，那么开发者将执行以下部分或全部操作：

- 了解是否可通过编译器执行任何其它最优化。
- 尝试进一步了解处理器架构并修改代码以充分利用任何特定于架构的行为（例如，减少条件分支以改进指令流水线）
- 修改 C 语言代码以使用特定于 CPU 的内部函数来并行执行关键运算。（例如，Arm NEON 内部函数）

同样的方法也适用于针对 DSP 或 GPU 编写的代码，使用 FPGA 时也适用：FPGA 器件只是另一个目标而已。

由 Vivado HLS 综合的 C 语言代码将在 FPGA 上执行，提供与 C 语言仿真相同的功能。某些情况下，开发者的工作在此阶段完成。

但通常由于 FPGA 器件的卓越性能而选择使用 FPGA 来实现 C 语言代码，FPGA 的大规模并行架构使其执行运算的速度远高于处理器的固有顺序运算，而用户通常都希望能够充分利用此性能。

此处重点在于了解 C 语言代码对于可达成的结果的影响以及如何通过修改 C 语言代码来充分发挥该列表的前三项的优势。

用于卷积函数的典型 C 语言代码

此处使用应用于图像的标准卷积函数来演示 C 语言代码对于 FPGA 性能产生的负面影响。在此示例中，将按顺序对数据执行横向卷积，然后执行纵向卷积。由于图像边缘的数据位于卷积窗口外，最后一步是解决边界周围的数据。

算法结构概括如下：

```
template<typename T, int K>
static void convolution_orig(
    int width,
    int height,
    const T *src,
    T *dst,
    const T *hcoeff,
    const T *vcoeff) {

    T local[MAX_IMG_ROWS*MAX_IMG_COLS];

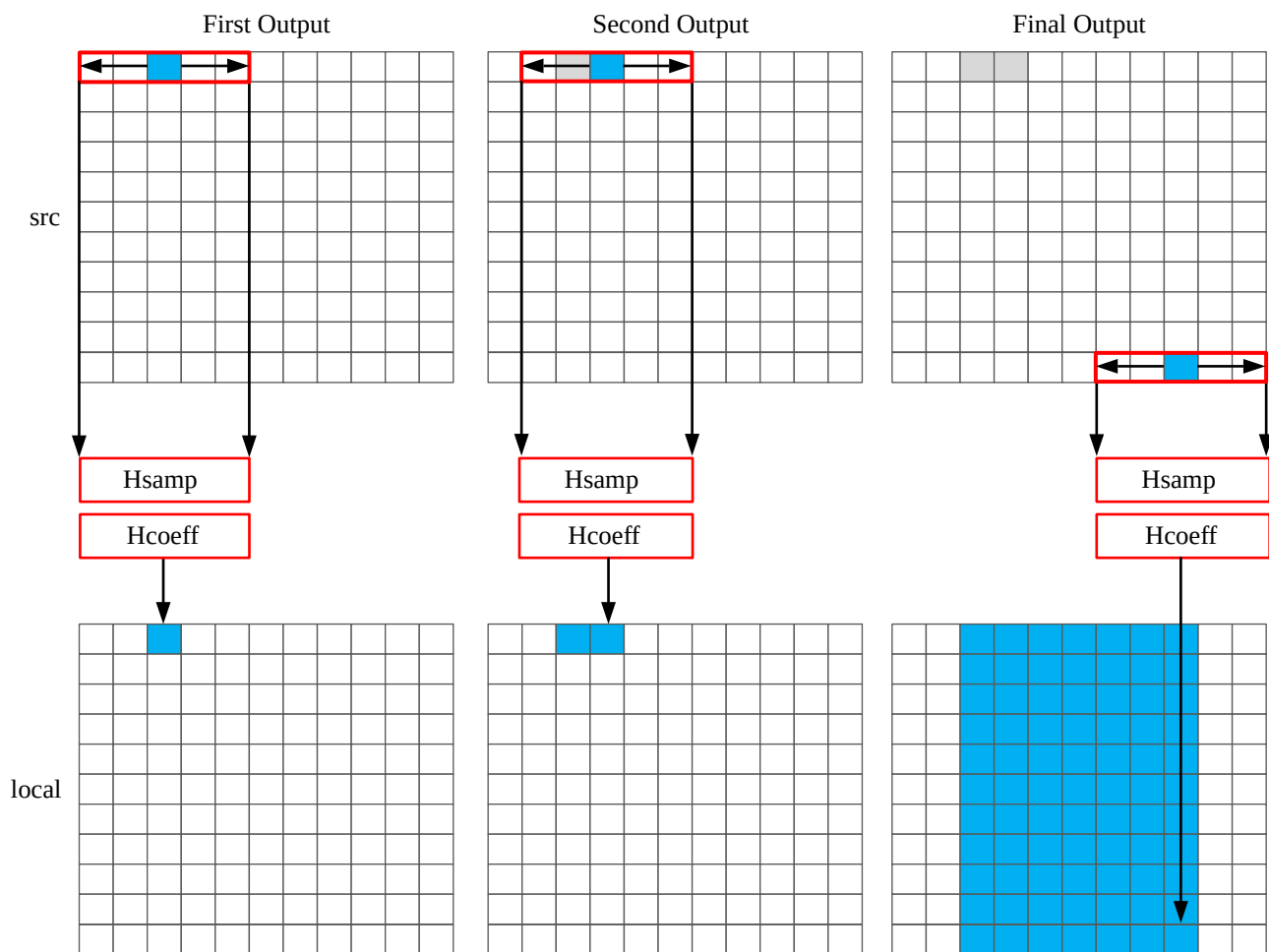
    // Horizontal convolution
    HconvH:for(int col = 0; col < height; col++){
        HconvW:for(int row = border_width; row < width - border_width; row++){
            Hconv:for(int i = - border_width; i <= border_width; i++){
            }
        }
    }
    // Vertical convolution
    VconvH:for(int col = border_width; col < height - border_width; col++){
        VconvW:for(int row = 0; row < width; row++){
            Vconv:for(int i = - border_width; i <= border_width; i++){
            }
        }
    }
    // Border pixels
    Top_Border:for(int col = 0; col < border_width; col++){
    }
}
```

```
Side_Border:for(int col = border_width; col < height - border_width; col++){
}
Bottom_Border:for(int col = height - border_width; col < height; col++){
}
}
```

水平卷积

第一步是在水平方向上进行卷积，如下图所示。

图 87：水平卷积



X14296

使用 K 个数据样本和 K 个卷积系数执行卷积。在上图中， K 显示为 5，但是 K 的值在代码中定义。为了进行卷积，至少需要 K 个数据样本。卷积窗口不能从第一个像素开始，因为该窗口将需要包含图像外部的像素。

通过执行对称卷积，可以将来自输入 `src` 的前 K 个数据样本与水平系数进行卷积，然后计算出第一个输出。接着使用下一组 K 个数据样本计算第二个输出。逐行进行此计算，直到写入最终输出。

最终结果是一个较小的图像，在上图中以蓝色显示。沿垂直边界的像素将在以后处理。

下面显示了用于执行此操作的 C 语言代码。

```
const int conv_size = K;
const int border_width = int(conv_size / 2);

#ifdef __SYNTHESIS__
    T * const local = new T[MAX_IMG_ROWS*MAX_IMG_COLS];
#else // Static storage allocation for HLS, dynamic otherwise
    T local[MAX_IMG_ROWS*MAX_IMG_COLS];
#endif

Clear_Local:for(int i = 0; i < height * width; i++){
    local[i]=0;
}
// Horizontal convolution
HconvH:for(int col = 0; col < height; col++){
    HconvWfor(int row = border_width; row < width - border_width; row++){
        int pixel = col * width + row;
        Hconv:for(int i = - border_width; i <= border_width; i++){
            local[pixel] += src[pixel + i] * hcoeff[i + border_width];
        }
    }
}
```

注释：只需在要综合的代码中使用 `__SYNTHESIS__` 宏即可。请勿在测试激励文件中使用该宏，因为 C 语言仿真或 RTL 协同仿真不会遵循其指示进行操作。

这段代码简单直观。但是，此段 C 语言代码中存在一些问题，其中 3 个问题将对硬件结果的质量产生负面影响。

第一个问题是需要 2 个单独的存储需求。结果存储在内部 `local` 数组中。这需要一个 `HEIGHT * WIDTH` 的数组，对于 `1920 * 1080` 的标准视频图像，将有 2,073,600 个值。在某些 Windows 系统上，如此大量的本地存储会引起问题。`local` 数组的数据放置在堆栈上，而不是由操作系统管理的堆上。

避免此类问题的一种有效方法是使用 `__SYNTHESIS__` 宏。执行综合时会自动定义此宏。上面显示的代码将在 C 语言仿真期间使用动态内存分配，以避免任何编译问题，并且仅在综合期间使用静态存储。使用此宏的缺点是，通过 C 语言仿真验证的代码与综合的代码不同。但是，在这种情况下，代码并不复杂，并且行为是相同的。

第一个影响 FPGA 实现质量的问题是 `local` 数组。由于这是 1 个数组，因此将使用内部 FPGA 块 RAM 来实现。这是要在 FPGA 内部实现的超大存储。它可能需要更大且更昂贵的 FPGA 器件。通过使用 DATAFLOW 最优化并通过小型高效 FIFO 流传输数据，可以最大程度减少块 RAM 的使用，但这将要求以流传输方式使用数据。

下一个问题是 `local` 数组的初始化。使用循环 `Clear_Local` 将 `local` 数组中的值设置为 0。即使已流水线化此循环，该操作也将需要大约 2 百万个时钟周期 (`HEIGHT*WIDTH`) 来实现。可以使用循环 `HConv` 中的临时变量在写入之前初始化累加，从而获得相同的数据初始化结果。

最后，数据的吞吐量受数据访问模式的限制。

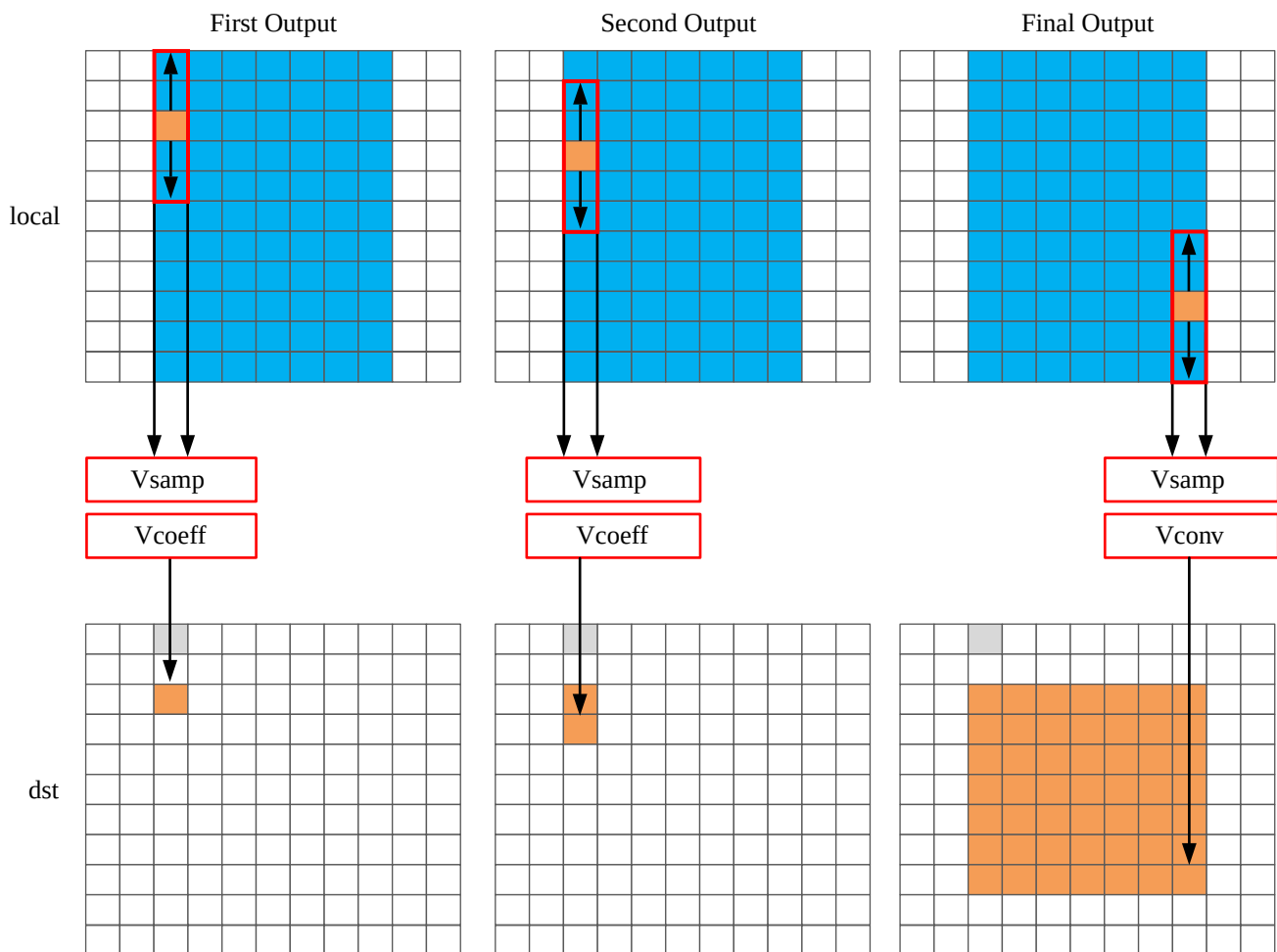
- 对于第一个输出，从输入中读取前 K 个值。
- 为了计算第二个输出，通过数据输入端口重新读取相同的 K-1 个值。
- 为了处理整个图像，将重复执行重新读取数据的过程。

实现高性能 FPGA 的关键之一是最大程度地减少对往来顶层函数实参的访问。顶层函数实参会成为 RTL 块上的数据端口。对于以上所示代码，由于需要反复读取数据，因此无法使用 DMA 操作直接从处理器流传输数据。重新读取输入还会限制 FPGA 处理样本的速率。

垂直卷积

下一步是执行下图中的垂直卷积。

图 88：垂直卷积



X14299

垂直卷积的过程与水平卷积的过程相似。需要一组 K 个数据样本通过卷积系数（在此例中为 `Vcoeff`）来执行卷积。使用垂直方向的前 K 个样本创建首个输出后，下一组 K 个值将用于创建第 2 个输出。此进程将沿每个列向下持续执行，直至创建最终输出为止。

完成垂直卷积后，由于水平和垂直边界作用，当前图像比源图像 `src` 更小。

执行这些操作的代码如下所示：

```
Clear_Dst:for(int i = 0; i < height * width; i++){
    dst[i]=0;
}
// Vertical convolution
VconvH:for(int col = border_width; col < height - border_width; col++){
    VconvW:for(int row = 0; row < width; row++){
```



```
int pixel = col * width + row;
Vconv:for(int i = - border_width; i <= border_width; i++){
    int offset = i * width;
    dst[pixel] += local[pixel + offset] * vcoeff[i + border_width];
}
}
```

此代码突显了水平卷积代码中所述的相似问题。

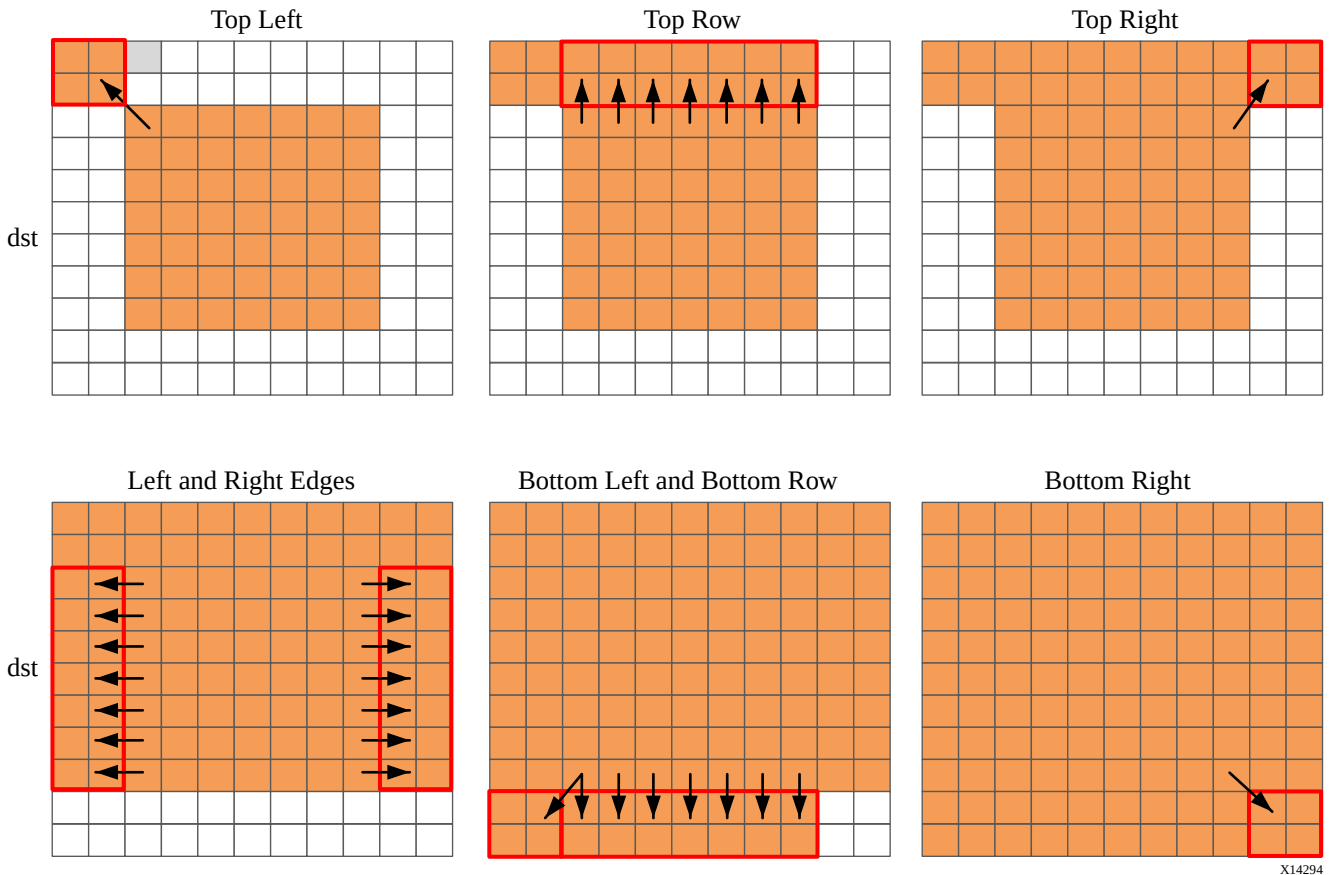
- 其中耗用了大量时钟周期来将输出图像 `dst` 中的值设置为 0。在此情况下，针对 1920*1080 图像大小，大约额外需要 2 百万个周期。
- 每个像素都需访问多次以重复读取 `local` 数组中的数据。
- 每个像素都需多次写入输出数组/端口 `dst`。

以上代码的另一个问题是 `local` 数组的访问模式。该算法要求 K 行上的数据可用于执行首次计算。首先沿行方向向下处理护具，然后再转至下一列，这需要将整个图像存储在本地。此外，由于数据未流传输至 `local` 数组外，无法使用 FIFO 来实现由 DATAFLOW 最优化所创建的内存通道。如果在此设计中使用 DATAFLOW 最优化，那么此内存通道需要乒乓缓存：这将导致实现的内存要求翻倍，在 FPGA 上需本地存储的数据样本数量将达到近 4 百万个。

边界像素

执行卷积的最后一步是在边界附近创建数据。这些像素可以通过直接复用卷积输出中最近的像素来创建。下图显示了如何实现此操作。

图 89：卷积边界样本



已使用最近的有效值填充边界区域。以下代码执行图中所示的操作。

```
int border_width_offset = border_width * width;
int border_height_offset = (height - border_width - 1) * width;
// Border pixels
Top_Border:for(int col = 0; col < border_width; col++){
    int offset = col * width;
    for(int row = 0; row < border_width; row++){
        int pixel = offset + row;
        dst[pixel] = dst[border_width_offset + border_width];
    }
    for(int row = border_width; row < width - border_width; row++){
        int pixel = offset + row;
        dst[pixel] = dst[border_width_offset + row];
    }
    for(int row = width - border_width; row < width; row++){
        int pixel = offset + row;
        dst[pixel] = dst[border_width_offset + width - border_width - 1];
    }
}

Side_Border:for(int col = border_width; col < height - border_width; col++){
    int offset = col * width;
    for(int row = 0; row < border_width; row++){
```

```

    int pixel = offset + row;
    dst[pixel] = dst[offset + border_width];
}
for(int row = width - border_width; row < width; row++){
    int pixel = offset + row;
    dst[pixel] = dst[offset + width - border_width - 1];
}
}

Bottom_Border:for(int col = height - border_width; col < height; col++){
    int offset = col * width;
    for(int row = 0; row < border_width; row++){
        int pixel = offset + row;
        dst[pixel] = dst[border_height_offset + border_width];
    }
    for(int row = border_width; row < width - border_width; row++){
        int pixel = offset + row;
        dst[pixel] = dst[border_height_offset + row];
    }
    for(int row = width - border_width; row < width; row++){
        int pixel = offset + row;
        dst[pixel] = dst[border_height_offset + width - border_width - 1];
    }
}
}

```

这段代码需要多次重复访问数据。现在，存储在 FPGA 外部 `dst` 数组中的数据必须可作为输入数据以供多次重复读取。即使在第一个循环中，也将多次读取 `dst[border_width_offset + border_width]`，但是 `border_width_offset` 和 `border_width` 的值不会发生改变。

这种编码样式对 FPGA 实现的性能和质量产生负面影响的最后一个方面在于不同条件的处理方式结构。每个 `for` 循环负责处理一种条件的操作：左上角、顶部行等。此处的最优化选择是：

在这种情况下，无法对顶层循环（`Top_Border`、`Side_Border`、`Bottom_Border`）进行流水线化处理，因为某些子循环具有变量边界（基于输入 `width` 宽度的值）。在这种情况下，您必须对子循环进行流水线化处理并依次执行每组流水线化循环。

判断是对顶层循环进行流水线化并展开子循环还是单独流水线化子循环的依据是由循环限制以及 FPGA 器件上有多少可用资源决定的。如果顶层循环限制很小，则可展开循环以复制硬件并满足性能要求。如果顶层循环限制较大，则将较低层次的循环流水线化，并在单一循环（`Top_Border`、`Side_Border`、`Bottom_Border`）中依次执行这些低层循环，从而会导致部分性能损失。

如标准卷积算法的回顾中所示，以下编码样式会对 FPGA 实现的性能和规模产生负面影响：

- 在数组中设置默认值会增加时钟周期且降低性能。
- 多次访问以反复读取数据会增加时钟周期且降低性能。
- 以任意或随机访问方式访问数据需将数据存储在本地图组中，这会占用资源。

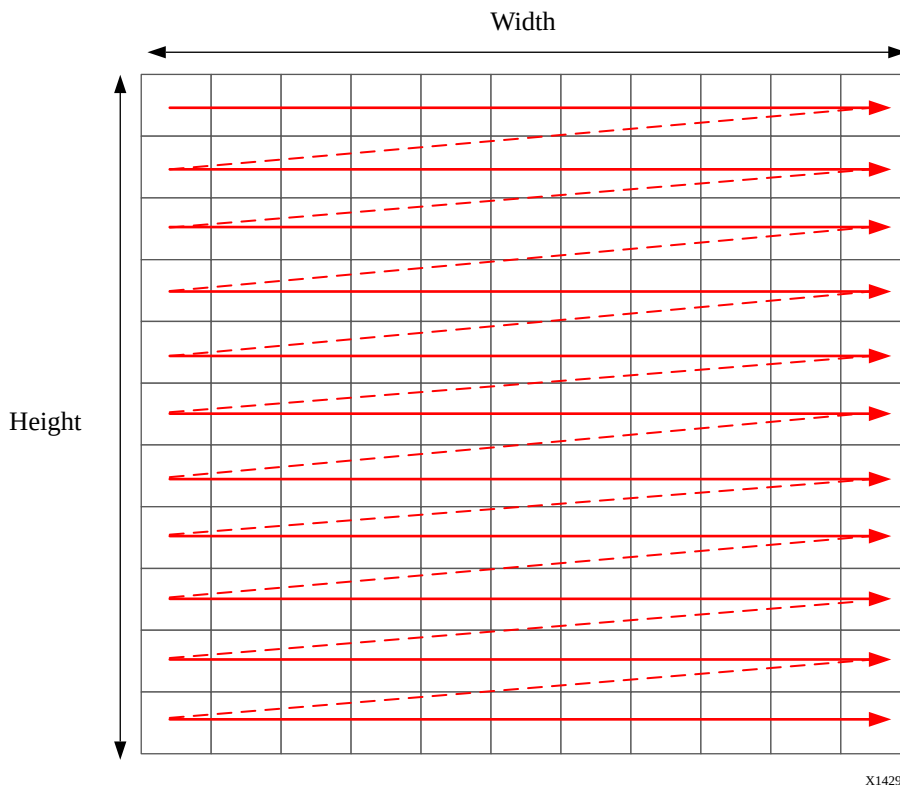
确保数据连续流动和数据复用

以最少量的资源将前一章节中所示的卷积示例实现为高性能设计的关键在于考量在总体系统中使用 FPGA 实现的方式。理想行为是使数据样本持续流经 FPGA。

- 最大限度改善系统内的数据流动。限制使用任何阻碍数据流动的编码方法或算法行为。
- 最大程度提升数据复用。使用本地高速缓存来确保无需重复读取数据，并且传入数据可保持流动。

第一步是确保对进出 FPGA 的 I/O 操作执行最优化。卷积算法在镜像上执行。生成并使用来自镜像的数据时，会按标准光栅扫描方式来传输这些数据，如下图所示。

图 90：光栅扫描顺序



如果数据从 CPU 或系统内存传输到 FPGA，那么通常将以此流传输方式进行传输。从 FPGA 传回系统的数据同样将以此方式执行传输。

使用 HLS 流进行数据流传输

对早期代码可执行的首要改进措施之一是使用 HLS 流传输构造（通常称为 `hls::stream`）。`hls::stream` 对象可采用与数组相同的方式来存储数据采样。`hls::stream` 中的数据只能按顺序访问。在 C 语言代码中，`hls::stream` 的行为与无限深度的 FIFO 相似。

使用 `hls::streams` 编写的代码通常用于在 FPGA 中创建设计，此类设计性能高且占用资源极少，因为 `hls::stream` 会强制使用最适合 FPGA 中实现的编码样式。

无法从 `hls::stream` 多次读取相同数据。从 `hls::stream` 中读取数据后，该数据在流传输中就不再存在。这有助于消除此编码实践。

如果需要再次使用 `hls::stream` 中的数据，则需将其置于高速缓存中。对于编写将在 FPGA 上进行综合的代码，这是另一项非常实用的实践。

`hls::stream` 会强制以适合 FPGA 实现的方式来开发 C 语言代码。

对 `hls::stream` 进行综合后，它会作为 FIFO 通道来自动实现，深度为 1 个元素。这是适合连接已流水线化的任务的理想硬件。

无需使用 `hls::streams` 并且可使用 C 语言代码中的数组来执行相同的实现。`hls::stream` 构造有助于强制实施最实用的编码实践。

对于 `hls::stream` 构造，经过最优化的新代码大纲如下所示：

```
template<typename T, int K>
static void convolution_strm(
    int width,
    int height,
    hls::stream<T> &src,
    hls::stream<T> &dst,
    const T *hcoeff,
    const T *vcoeff)
{

    hls::stream<T> hconv("hconv");
    hls::stream<T> vconv("vconv");
    // These assertions let HLS know the upper bounds of loops
    assert(height < MAX_IMG_ROWS);
    assert(width < MAX_IMG_COLS);
    assert(vconv_xlim < MAX_IMG_COLS - (K - 1));

    // Horizontal convolution
    HConvH:for(int col = 0; col < height; col++) {
        HConvW:for(int row = 0; row < width; row++) {
            HConv:for(int i = 0; i < K; i++) {
                }
            }
        }
    // Vertical convolution
    VConvH:for(int col = 0; col < height; col++) {
        VConvW:for(int row = 0; row < vconv_xlim; row++) {
            VConv:for(int i = 0; i < K; i++) {
                }
            }
        }

    Border:for (int i = 0; i < height; i++) {
        for (int j = 0; j < width; j++) {
            }
        }
    }
}
```

相比于早期代码，值得注意的明显差异包括：

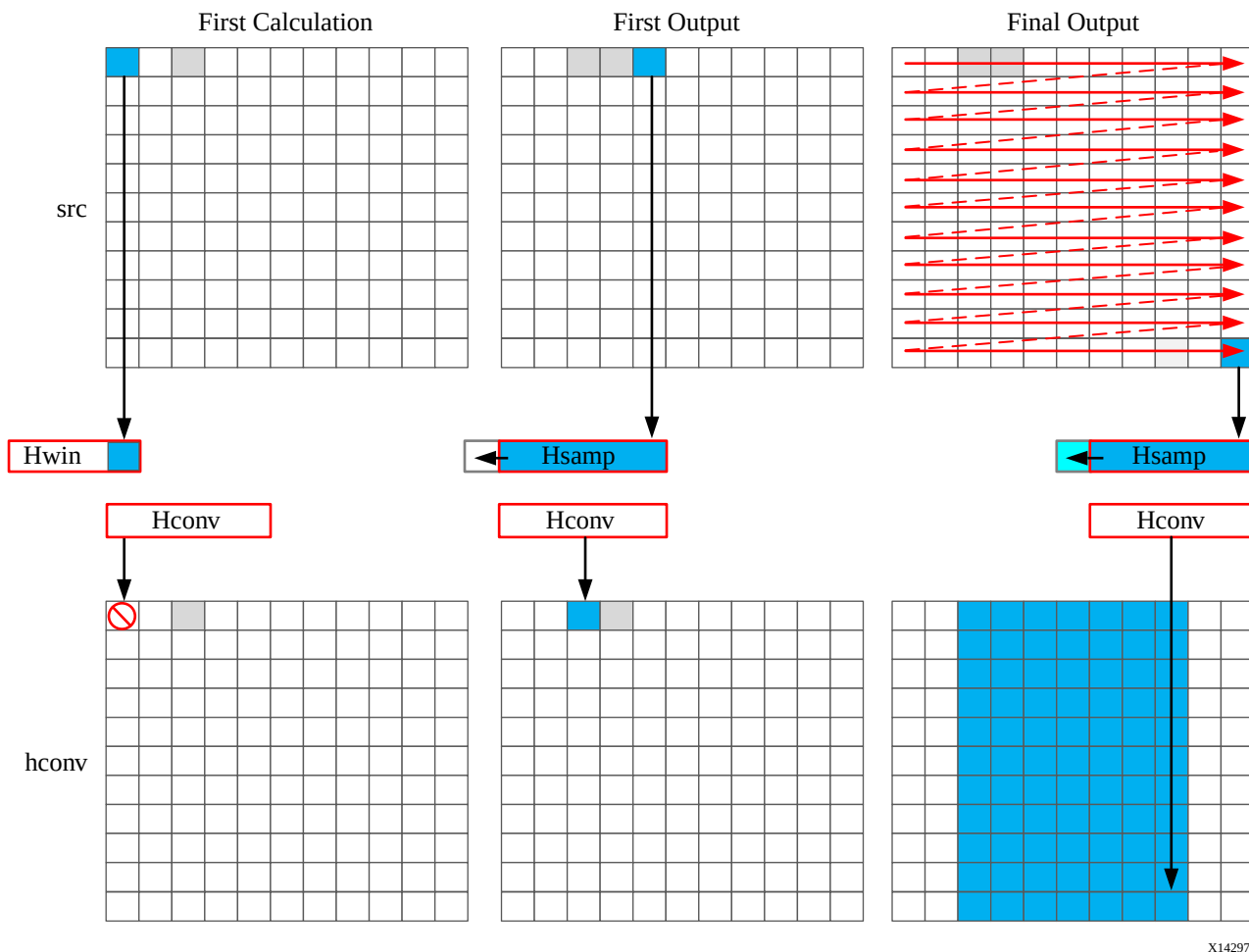
- 输入和输出数据现已采用 `hls::streams` 来建模。
- 其中并不采用大小为 `HEIGHT*WDITH` 的单一局部数组，而是改为使用 2 个内部 `hls::streams` 来保存水平和垂直卷积的输出。

此外，部分 `assert` 语句用于指定循环边界的最大值。这是非常实用的编码样式，允许 HLS 自动报告变量边界循环的时延并最优化循环边界。

水平卷积

为了提升 FPGA 实现的计算执行效率，采用下图所示方式来计算水平卷积。

图 91：水平卷积流传输



X14297

使用 `hls::stream` 可强制采用高效的算法实践，迫使您首先读取第一个样本，而不是对数据执行随机访问。该算法必须使用前 K 个样本来计算卷积结果，因此它将该样本复制到临时高速缓存 `hwin` 中。对于第一次计算，`hwin` 中没有足够的值用于计算结果，因此不写入任何输出值。

该算法会持续读取输入样本，并将其缓存到 `hwin` 中。读取每个新样本时，都会从 `hwin` 移除不需要的样本。读取第 K 个输入后，才能首次写入输出值。现在即可写入输出值。

该算法会沿各行按此方式继续操作，直至读取最后一个样本为止。届时，在 `hwin` 中仅存储最后 K 个样本：即计算卷积所需的全部样本。

执行这些操作的代码如下所示：

```
// Horizontal convolution
HConvW:for(int row = 0; row < width; row++) {
  HconvW:for(int row = border_width; row < width - border_width; row++){
    T in_val = src.read();
    T out_val = 0;
    HConv:for(int i = 0; i < K; i++) {
      hwin[i] = i < K - 1 ? hwin[i + 1] : in_val;
      out_val += hwin[i] * hcoeff[i];
    }
  }
}
```

```
}  
if (row >= K - 1)  
    hconv << out_val;  
}  
}
```

以上代码中值得注意的是，它使用临时变量 `out_val` 来执行卷积计算。执行计算前，此变量设置为 0，无需耗费 2 百万个时钟周期来复位这些值，如前例中所示。

在整个流程中，`src` 输入中的样本以光栅流传输方式来处理。逐一读取每个样本。任务输出将被放弃或使用，但任务将持续运行计算。这表示此代码与专为在 CPU 上执行而编写的代码不同。

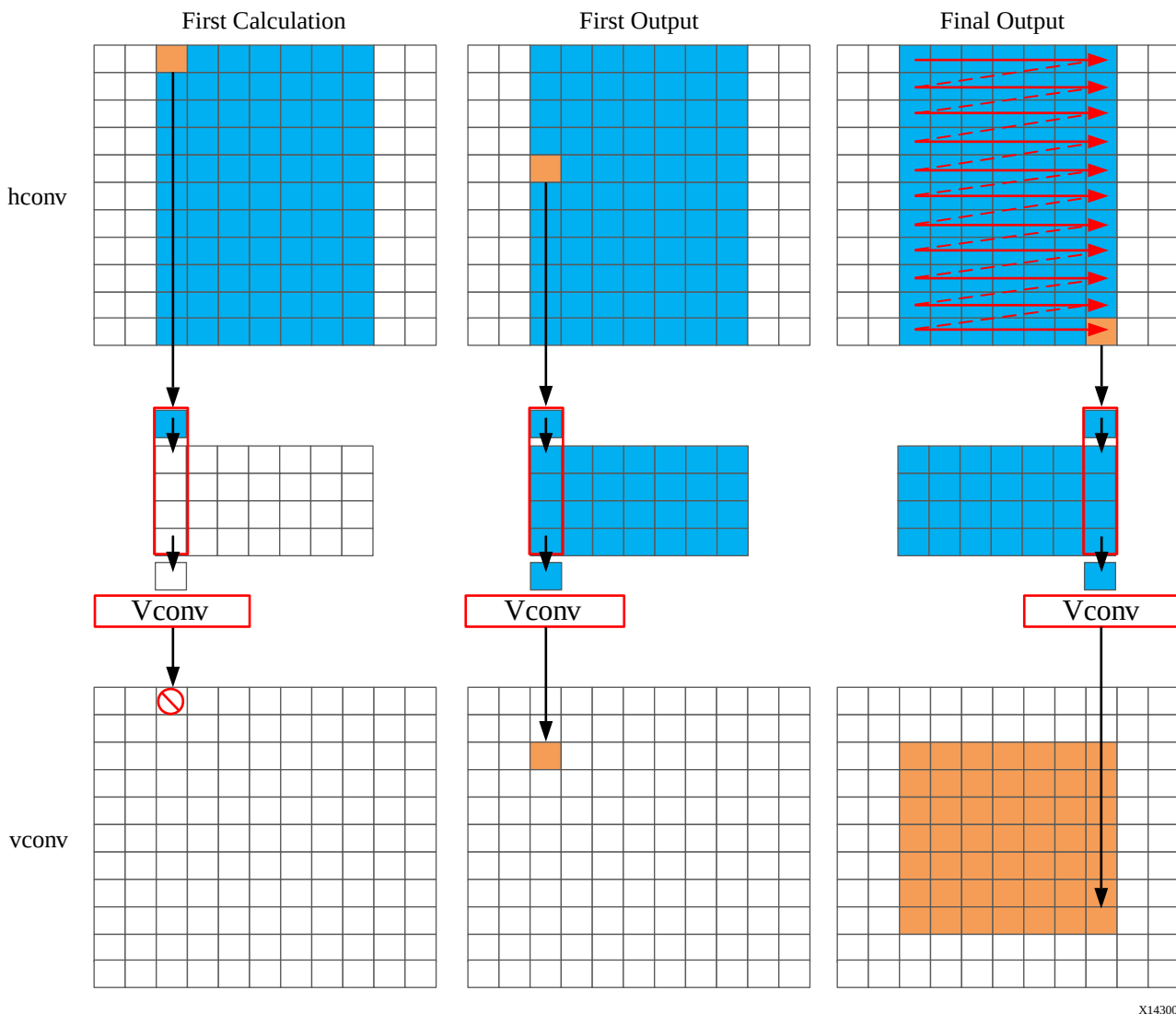
在 CPU 架构中，通常会避免条件操作或分支操作。当程序需要分支时，会丢失 CPU 访存流水线中存储的所有指令。在 FPGA 架构中，硬件中针对每个条件分支已存在一条独立路径，并且在流水线任务内不存在与分支相关联的性能惩罚。只需选择要使用哪个分支即可。

输出存储在 `hls::stream hconv` 中以供垂直卷积循环使用。

垂直卷积

垂直卷积给 FPGA 首选的流传输数据模型带来了难题。数据必须按列访问，但存储整张图像并非上策。解决方案是使用行缓存，如下图所示。

图 92：垂直卷积的流传输



X14300

此处样本同样按流传输方式来读取，但这次从 `hls::stream hconv` 读取。算法要求读取至少 $K-1$ 行数据，然后才能处理首个样本。在此之前执行的所有计算都将被放弃。

行缓存允许存储 $K-1$ 行数据。每次读取新样本时，都会将另一个样本挤出行缓存。此处值得注意的是在计算中使用最新样本，然后将此样本存储到行缓存中，并将旧样本排出。这样即可确保只需将 $K-1$ 行数据保存在高速缓存中，而不是 K 行数据。虽然行缓存确实要求将多行数据存储在本地，但卷积内核大小 K 始终小于完整的视频图像中的 1080 行。

读取第 K 行上的首个样本时即可执行首次计算。随后，该算法会继续输出值，直至读取最后一个像素为止。

```
// Vertical convolution
VConvH:for(int col = 0; col < height; col++) {
    VConvW:for(int row = 0; row < vconv_xlim; row++) {
        #pragma HLS DEPENDENCE variable=linebuf inter false
        #pragma HLS PIPELINE
        T in_val = hconv.read();
```



```
T out_val = 0;
VConv:for(int i = 0; i < K; i++) {
    T vwin_val = i < K - 1 ? linebuf[i][row] : in_val;
    out_val += vwin_val * vcoeff[i];
    if (i > 0)
        linebuf[i - 1][row] = vwin_val;
}
if (col >= K - 1)
    vconv << out_val;
}
```

以上代码会按流传输方式再次处理设计中的所有样本。此任务持续运行。使用 `hls::stream` 构造会强制将数据保存在本地高速缓存中。这是适合用于 FPGA 的理想策略。

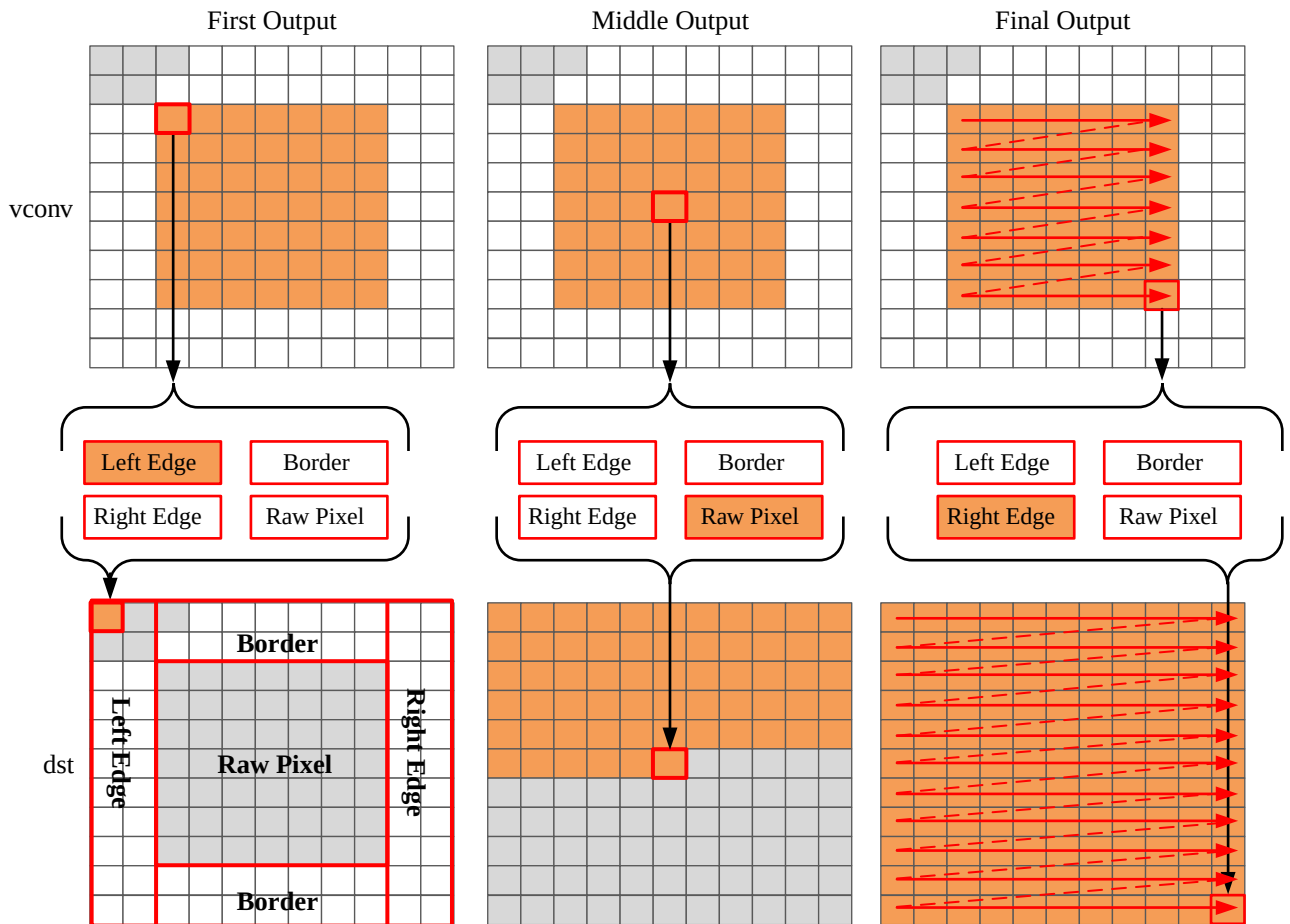
边界像素

算法的最后一步是将边缘像素复制到边界区域内。同样，为确保数据流不中断和数据复用，算法使用 `hls::stream` 和高速缓存。

下图显示了边界采样与图像的对齐方式。

- 每个采样均从垂直卷积的 `vconv` 输出读取。
- 随后，采样将缓存为 4 种可能的像素类型之一。
- 随后，采样将写入输出流。

图 93：流传输边界采样



X14295

判定边界像素的位置的代码为：

```
Border:for (int i = 0; i < height; i++) {
    for (int j = 0; j < width; j++) {
        T pix_in, l_edge_pix, r_edge_pix, pix_out;
        #pragma HLS PIPELINE
        if (i == 0 || (i > border_width && i < height - border_width)) {
            if (j < width - (K - 1)) {
                pix_in = vconv.read();
                borderbuf[j] = pix_in;
            }
            if (j == 0) {
                l_edge_pix = pix_in;
            }
            if (j == width - K) {
                r_edge_pix = pix_in;
            }
        }
        if (j <= border_width) {
            pix_out = l_edge_pix;
        } else if (j >= width - border_width - 1) {
            pix_out = r_edge_pix;
        }
    }
}
```

```

    } else {
        pix_out = borderbuf[j - border_width];
    }
    dst << pix_out;
}
}
}
}

```

此新代码的一个明显差异是在任务内广泛使用条件。这使任务在流水线化后可持续不断处理数据，且条件结果不会影响流水线的执行：结果将影响输出值，但只要输入采样可用，流水线就会持续处理。

这种有利于 FPGA 的算法的最终代码使用了以下最优化指令。

```

template<typename T, int K>
static void convolution_strm(
int width,
int height,
hls::stream<T> &src,
hls::stream<T> &dst,
const T *hcoeff,
const T *vcoeff)
{
#pragma HLS DATAFLOW
#pragma HLS ARRAY_PARTITION variable=linebuf dim=1 complete

hls::stream<T> hconv("hconv");
hls::stream<T> vconv("vconv");
// These assertions let HLS know the upper bounds of loops
assert(height < MAX_IMG_ROWS);
assert(width < MAX_IMG_COLS);
assert(vconv_xlim < MAX_IMG_COLS - (K - 1));

// Horizontal convolution
HConvH:for(int col = 0; col < height; col++) {
    HConvW:for(int row = 0; row < width; row++) {
#pragma HLS PIPELINE
        HConv:for(int i = 0; i < K; i++) {
        }
    }
}
// Vertical convolution
VConvH:for(int col = 0; col < height; col++) {
    VConvW:for(int row = 0; row < vconv_xlim; row++) {
#pragma HLS PIPELINE
#pragma HLS DEPENDENCE variable=linebuf inter false
        VConv:for(int i = 0; i < K; i++) {
        }
    }
}

Border:for (int i = 0; i < height; i++) {
    for (int j = 0; j < width; j++) {
#pragma HLS PIPELINE
    }
}
}

```

按采样级别对每项任务进行流水线化。行缓存完全分区到寄存器中，以确保不存在因块 RAM 不足而导致的读写限制。行缓存还需要依赖关系指令。所有任务都在数据流区域中执行，这将确保任务可并行运行。hls::streams 将作为含 1 个元素的 FIFO 自动实现。

高效硬件的 C 语言摘要

最大限度减少数据输入读取。将数据读入块后，它可以轻松馈送许多并行路径，但是输入端口可能成为性能瓶颈。如果必须复用数据，请一次性读取数据并使用本地高速缓存。

最大限度减少对数组的访问，尤其是大型数组。数组是在块 RAM 中实现的，就像 I/O 端口一样，数组中的端口数量有限，这可能成为性能瓶颈。可以将数组分区为较小的数组，甚至可以分区为单个寄存器，但是对较大的数组进行分区会导致使用大量寄存器。使用小规模的本地高速缓存来保存诸如累加之类的结果，然后将最终结果写入数组。

力求在流水线任务内部执行条件分支，而不是有条件地执行任务，甚至是流水线任务。条件将在流水线中作为单独的路径来实现。允许将一个任务中的数据流入下一个任务，并在下一个任务中执行条件，可实现性能更高的系统。

应尽量减少输出写入，原因与输入读取相同，即端口将成为瓶颈。复制更多端口只是将问题进一步推到系统中。

对于以流传输方式处理数据的 C 语言代码，请考虑使用 `hls::streams`，因为它们将强制要求采用良好的编码实践。用 C 语言设计算法以获得高性能的 FPGA 实现的效率远高于调试 FPGA 以寻找它无法按所需性能运行的原因。

C++ 类和模板

Vivado HLS 完全支持对 C++ 类进行综合，但仅支持对顶层函数进行综合，而不支持对顶层的类进行综合。要对类成员函数进行综合，请将类本身例化为函数。请勿直接将顶层类例化为测试激励文件。以下代码示例显示了如何在 `cpp_FIR` 顶层函数中对后文所述头文件中定义的 `CFir` 类进行例化，以及如何使用该类实现 FIR 滤波器。

```
#include "cpp_FIR.h"

// Top-level function with class instantiated
data_t cpp_FIR(data_t x)
{
    static CFir<coef_t, data_t, acc_t> fir1;

    cout << fir1;

    return fir1(x);
}
```



重要提示！ 不支持对位于顶层的类和类成员函数进行综合。请例化顶层函数中的类。

检验以上 C++ FIR 滤波器示例中用于实现设计的类之前，值得注意的是，Vivado HLS 在综合期间会忽略标准输出流传输 `cout`。完成综合后，Vivado HLS 会发出以下警告：

```
INFO [SYNCHK-101] Discarding unsynthesizable system call:
'std::ostream::operator<<' (cpp_FIR.h:108)
INFO [SYNCHK-101] Discarding unsynthesizable system call:
'std::ostream::operator<<' (cpp_FIR.h:108)
INFO [SYNCHK-101] Discarding unsynthesizable system call: 'std::operator<<
<std::char_traits<char> >' (cpp_FIR.h:110)
```

以下代码示例显示了 `cpp_FIR.h` 头文件，包括 `CFir` 类的定义及其关联的成员函数。在此示例中，运算符成员函数 `()` 和 `<<` 为重载运算符，分别用于执行主算法以及配合 `cout` 用于数据格式化以便在 C 语言仿真期间显示。

```
#include <fstream>
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

#define N 85

typedef int coef_t;
typedef int data_t;
typedef int acc_t;

// Class CFir definition
template<class coef_T, class data_T, class acc_T>
class CFir {
protected:
    static const coef_T c[N];
    data_T shift_reg[N-1];
private:
public:
    data_T operator()(data_T x);
    template<class coef_TT, class data_TT, class acc_TT>
    friend ostream&
    operator<<(ostream& o, const CFir<coef_TT, data_TT, acc_TT> &f);
};

// Load FIR coefficients
template<class coef_T, class data_T, class acc_T>
const coef_T CFir<coef_T, data_T, acc_T>::c[N] = {
    #include "cpp_FIR.h"
};

// FIR main algorithm
template<class coef_T, class data_T, class acc_T>
data_T CFir<coef_T, data_T, acc_T>::operator()(data_T x) {
    int i;
    acc_t acc = 0;
    data_t m;

    loop: for (i = N-1; i >= 0; i--) {
        if (i == 0) {
            m = x;
            shift_reg[0] = x;
        } else {
            m = shift_reg[i-1];
            if (i != (N-1))
                shift_reg[i] = shift_reg[i - 1];
        }
        acc += m * c[i];
    }
    return acc;
}

// Operator for displaying results
template<class coef_T, class data_T, class acc_T>
ostream& operator<<(ostream& o, const CFir<coef_T, data_T, acc_T> &f) {
    for (int i = 0; i < (sizeof(f.shift_reg)/sizeof(data_T)); i++) {
        o << shift_reg[ << i << ] = << f.shift_reg[i] << endl;
    }
}
```

```
o << ----- << endl;
return o;
}

data_t cpp_FIR(data_t x);
```

以下代码示例中显示的 C++ FIR 滤波器示例中的测试激励文件用于演示对顶层函数 `cpp_FIR` 进行调用和验证的方式。此示例突出显示了适合 Vivado HLS 综合的标准测试激励文件的部分重要属性：

- 根据已知有效值对输出结果进行检查。
- 如果结果确认正确，测试激励文件会返回 0。

```
#include "cpp_FIR.h"

int main() {
    ofstream result;
    data_t output;
    int retval=0;

    // Open a file to saves the results
    result.open(result.dat);

    // Apply stimuli, call the top-level function and saves the results
    for (int i = 0; i <= 250; i++)
    {
        output = cpp_FIR(i);

        result << setw(10) << i;
        result << setw(20) << output;
        result << endl;
    }
    result.close();

    // Compare the results file with the golden results
    retval = system(diff --brief -w result.dat result.golden.dat);
    if (retval != 0) {
        printf(Test failed !!!\n);
        retval=1;
    } else {
        printf(Test passed !\n);
    }

    // Return 0 if the test
    return retval;
}
```

适用于 `cpp_FIR` 的 C++ 测试激励文件

要将指令应用于类中定义的对象，请执行以下操作：

1. 打开定义类的文件（通常为头文件）。
2. 使用“Directives”选项卡应用指令。

就像函数一样，针对类的所有实例将应用相同的最优化操作。

构造函数、析构函数和虚拟函数

只要声明类对象，就会包含类构造函数和析构函数并对其进行综合。

Vivado HLS 支持对虚拟函数（包括抽象函数）进行综合，前提是它可静态判定细化期间的函数。Vivado HLS 不支持在以下情况下对虚拟函数进行综合：

- 虚拟函数可在多层继承类层级中定义，但仅限含单一继承的层级。
- 仅当可在编译时判定指针对象时，才支持动态多态性。例如，在 if-else 或循环构造中无法使用此类指针。
- STL 容器不能包含对象指针，也不能调用多态函数。例如：

```
vector<base *> base_ptrs(10);

//Push_back some base ptrs to vector.
for (int i = 0; i < base_ptrs.size(); ++i) {
    //Static elaboration cannot resolve base_ptrs[i] to actual data type.
    base_ptrs[i]->virtual_function();
}
```

- Vivado HLS 不支持基本对象指针为全局变量的情况。例如：

```
Base *base_ptr;

void func()
{
    ...
    base_ptr->virtual_function();
    ...
}
```

- 基本对象指针不能作为类定义中的成员变量。例如：

```
// Static elaboration cannot bind base object pointer with correct data
type.
class A
{
    ...
    Base *base_ptr;
    void set_base(Base *base_ptr);
    void some_func();
    ...
};

void A::set_base(Base *ptr)
{
    this.base_ptr = ptr;
}

void A::some_func()
{
    â!
    base_ptr->virtual_function();
    â!
}
```

- 如果基本对象指针或参考包含在构造函数的函数参数列表中，则 Vivado HLS 无法对其进行转换。ISO C++ 标准第 12.7 章中已描述了此问题：有时此行为未定义。

```
class A {
    A(Base *b) {
        b-> virtual _ function ();
    }
};
```

全局变量和类

赛灵思不建议在类中使用全局变量。此类变量可能阻止执行某些最优化。在以下代码示例中，类用于创建筛选工具组件（polyd_cell 类用作执行移位、乘法和累加运算的组件）。

```
typedef long long acc_t;
typedef int mult_t;
typedef char data_t;
typedef char coef_t;

#define TAPS 3
#define PHASES 4
#define DATA_SAMPLES 256
#define CELL_SAMPLES 12

// Use k on line 73 static int k;

template <typename T0, typename T1, typename T2, typename T3, int N>
class polyd_cell {
private:
public:
    T0 areg;
    T0 breg;
    T2 mreg;
    T1 preg;
    T0 shift[N];
    int k; //line 73
    T0 shift_output;
    void exec(T1 *pcout, T0 *dataOut, T1 pcin, T3 coeff, T0 data, int col)
    {
        Function_label0::

        if (col==0) {
            SHIFT:for (k = N-1; k >= 0; --k) {
                if (k > 0)
                    shift[k] = shift[k-1];
                else
                    shift[k] = data;
            }
            *dataOut = shift_output;
            shift_output = shift[N-1];
        }
        *pcout = (shift[4*col]* coeff) + pcin;
    }
};

// Top-level function with class instantiated
void cpp_class_data (
    acc_t *dataOut,
```



```

coef_t coeff1[PHASES][TAPS],
coef_t coeff2[PHASES][TAPS],
data_t dataIn[DATA_SAMPLES],
int row
) {

    acc_t pcin0 = 0;
    acc_t pcout0, pcout1;
    data_t dout0, dout1;
    int col;
    static acc_t accum=0;
    static int sample_count = 0;
    static polyd_cell<data_t, acc_t, mult_t, coef_t, CELL_SAMPLES>
polyd_cell0;
    static polyd_cell<data_t, acc_t, mult_t, coef_t, CELL_SAMPLES>
polyd_cell1;

    COL:for (col = 0; col <= TAPS-1; ++col) {

        polyd_cell0.exec(&pcout0,&dout0,pcin0,coeff1[row]
[col],dataIn[sample_count],
col);

        polyd_cell1.exec(&pcout1,&dout1,pcout0,coeff2[row][col],dout0,col);

        if ((row==0) && (col==2)) {
            *dataOut = accum;
            accum = pcout1;
        } else {
            accum = pcout1 + accum;
        }
    }
    sample_count++;
}

```

在 `polyd_cell` 类中包含用于数据移位的 `SHIFT` 循环。如果 `SHIFT` 循环中使用的循环索引 `k` 已被移除并替换为对应 `k` 的全局索引（如上示例所示，但带有 `static int k` 注释），则 Vivado HLS 无法对使用 `polyd_cell` 类的任何循环或函数进行流水线化。Vivado HLS 将发出如下消息：

```

@W [XFORM-503] Cannot unroll loop 'SHIFT' in function 'polyd_cell<char,
long long,
int, char, 12>::exec' completely: variable loop bound.

```

使用局部非全局变量来建立循环索引即可确保 Vivado HLS 能执行所有最优化操作。

模板

Vivado HLS 支持在 C++ 中使用模板进行综合。Vivado HLS 不支持对顶层函数使用模板。



重要提示！ 不能使用模板作为顶层函数。

使用模板创建专用实例

针对模板实参的每个不同值将复制模板函数中的静态变量。

```
template<int NC, int K>
void startK(int* dout) {
    static int acc=0;
    acc += K;
    *dout = acc;
}

void foo(int* dout) {
    startK<0,1> (dout);
}

void goo(int* dout) {
    startK<1,1> (dout);
}

int main() {
    int dout0,dout1;
    for (int i=0;i<10;i++) {
        foo(&dout0);
        goo(&dout1);
        cout <<"dout0/1 = "<<dout0<<" / "<<dout1<<endl;
    }
    return 0;
}
```

使用模板执行递归

模板还可用于实现标准 C 语言综合中不支持的某种形式的递归（递归函数）。

以下代码示例显示了使用模板化 `struct` 来实现尾递归斐波那契算法。执行综合的关键是用于在递归中实现最终调用的终端类，其中使用的模板大小为 1。

```
//Tail recursive call
template<data_t N> struct fibon_s {
    template<typename T>
    static T fibon_f(T a, T b) {
        return fibon_s<N-1>::fibon_f(b, (a+b));
    }
};

// Termination condition
template<> struct fibon_s<1> {
    template<typename T>
    static T fibon_f(T a, T b) {
        return b;
    }
};

void cpp_template(data_t a, data_t b, data_t &dout){
    dout = fibon_s<FIB_N>::fibon_f(a,b);
}
```

断言

支持对 C 语言中用于断言范围信息的 `assert` 宏执行综合。例如，变量和循环边界的上限。

存在变量循环边界时，Vivado HLS 无法判定循环的所有迭代的时延，其报告的时延显示为问号。`tripcount` 指令可将循环边界告知 Vivado HLS，但此信息仅用于报告目的，不影响综合结果（无论是否使用 `tripcount` 指令，创建的硬件大小都相同）。

以下代码示例显示了断言如何将变量最大范围告知 Vivado HLS 以及如何使用这些断言来生成更优化的硬件。

使用断言前，必须包含用于定义 `assert` 宏的头文件。在此示例中，头文件中包含以下内容。

```
#ifndef _loop_sequential_assert_H_
#define _loop_sequential_assert_H_

#include <stdio.h>
#include <assert.h>
#include ap_cint.h
#define N 32

typedef int8 din_t;
typedef int13 dout_t;
typedef uint8 dsel_t;

void loop_sequential_assert(din_t A[N], din_t B[N], dout_t X[N], dout_t
Y[N], dsel_t
xlimit, dsel_t ylimit);

#endif
```

在 `main` 代码中，每个循环前都有 2 条 `assert` 语句。

```
assert(xlimit<32);
...
assert(ylimit<16);
...
```

这些断言用于：

- 保证如果断言为 `false`，并且值大于声明的值，则 C 语言仿真将失败。由此可见在综合前执行 C 语言代码仿真的重要性：在综合前确认设计有效。
- 告知 Vivado HLS，该变量的范围将不超出该值，这样即可最优化 RTL 中变量大小，在此情况下即最优化循环迭代计数。

以下代码示例显示了这些断言。

```
#include "loop_sequential_assert.h"

void loop_sequential_assert(din_t A[N], din_t B[N], dout_t X[N], dout_t
Y[N], dsel_t
xlimit, dsel_t ylimit) {
    dout_t X_accum=0;
```

```
dout_t Y_accum=0;
int i,j;

assert(xlimit<32);
SUM_X:for (i=0;i<=xlimit; i++) {
    X_accum += A[i];
    X[i] = X_accum;
}

assert(ylimit<16);
SUM_Y:for (i=0;i<=ylimit; i++) {
    Y_accum += B[i];
    Y[i] = Y_accum;
}
}
```

除 `assert` 宏外，此代码与循环并行化中所示代码相同。综合后，综合报告中存在 2 个重要差异。

不使用 `assert` 宏的情况下，报告如下所示，显示循环行程计数范围为 1 到 256，因为循环边界的变量数据类型为 `d_sel` (8 位变量)。

```
* Loop Latency:
+-----+-----+-----+
|Target II |Trip Count |Pipelined |
+-----+-----+-----+
|- SUM_X   |1 ~ 256   |no        |
|- SUM_Y   |1 ~ 256   |no        |
+-----+-----+-----+
```

在使用 `assert` 宏的版本中，报告显示针对 `SUM_X` 循环和 `SUM_Y` 循环报告的行程计数分别为 32 和 16。由于断言称这些值永不大于 32 和 16，因此 Vivado HLS 可在报告中使用此结果。

```
* Loop Latency:
+-----+-----+-----+
|Target II |Trip Count |Pipelined |
+-----+-----+-----+
|- SUM_X   |1 ~ 32    |no        |
|- SUM_Y   |1 ~ 16    |no        |
+-----+-----+-----+
```

此外，不同于使用 `Tripcount` 指令，`assert` 语句可提供更优化的硬件。不使用断言的情况下，最终硬件使用的变量和计数器大小设为最多 256 次循环迭代。

```
* Expression:
+-----+-----+-----+-----+-----+
|Operation |Variable Name          |DSP48E |FF |LUT |
+-----+-----+-----+-----+-----+
|+         |X_accum_1_fu_182_p2    |0      |0  |13  |
|+         |Y_accum_1_fu_209_p2    |0      |0  |13  |
|+         |indvar_next6_fu_158_p2 |0      |0  |9   |
|+         |indvar_next_fu_194_p2  |0      |0  |9   |
|+         |tmp1_fu_172_p2         |0      |0  |9   |
|+         |tmp_fu_147_p2          |0      |0  |9   |
```

icmp	exitcond1_fu_189_p2	0	0	9	
icmp	exitcond_fu_153_p2	0	0	9	
+-----+-----+-----+-----+					
Total		0	0	80	
+-----+-----+-----+-----+					

断言变量范围的代码如果小于可能的范围的最大值，则可能导致更小的 RTL 设计。

```
* Expression:
```

+-----+-----+-----+-----+					
Operation	Variable Name	DSP48E	FF	LUT	
+-----+-----+-----+-----+					
+	X_accum_1_fu_176_p2	0	0	13	
+	Y_accum_1_fu_207_p2	0	0	13	
+	i_2_fu_158_p2	0	0	6	
+	i_3_fu_192_p2	0	0	5	
icmp	tmp_2_fu_153_p2	0	0	7	
icmp	tmp_9_fu_187_p2	0	0	6	
+-----+-----+-----+-----+					
Total		0	0	50	
+-----+-----+-----+-----+					

断言可用于指示设计中任意变量的范围。使用断言时，执行涵盖所有可能情况的 C 语言仿真至关重要。这将确认 Vivado HLS 所使用的断言有效。

SystemC 综合

Vivado HLS 支持 SystemC (IEEE 1666 标准)，它是用于对硬件进行建模的 C++ 类库。此库可通过 Accellera 网站 (www.accellera.org) 获取。对于综合，Vivado HLS 支持适用于 SystemC v2.1 的 SystemC Synthesizable Subset (Draft 1.3)。

本节提供了有关使用 Vivado HLS 对 SystemC 语言函数执行综合的信息。此信息是对先前章节“C 语言综合”和“C++ 语言综合”中信息的补充。赛灵思建议您阅读这些章节以充分了解适用于综合的编码基本规则。



重要提示！就像 C 和 C++ 设计一样，要用于综合的顶层函数只能位于顶层 C 编译 `sc_main()` 函数的下层。不支持对作为顶层函数的 `sc_main()` 函数进行综合。

设计建模

综合的顶层必须为 `SC_MODULE`。如果使用 SystemC 语言构造函数进程 `SC_METHOD`、`SC_CTHREAD` 和 `SC_HAS_PROCESS` 宏或者如果 `SC_MODULES` 在其它 `SC_MODULES` 内进行例化，那么可对设计进行综合。

不能使用模板作为设计中的顶层 `SC_MODULE`。模板只能在子模块上使用。

模块构造函数只能定义或例化模块，它不能包含任何功能。

`SC_MODULE` 不能在另一个 `SC_MODULE` 内定义。（虽然可对其进行例化，请参阅后文）。

使用 SC_MODULE

层级模块定义不受支持。当某个模块在另一个模块内定义（以下第 1 个 SC_MODULE 示例）时，必须将其转换为非嵌套模块版本（以下第 2 个 SC_MODULE 示例）。

```
SC_MODULE(nested1)
{
    SC_MODULE(nested2)
    {
        sc_in<int> in0;
        sc_out<int> out0;
        SC_CTOR(nested2)
        {
            SC_METHOD(process);
            sensitive<<in0;
        }
        void process()
        {
            int var =10;
            out0.write(in0.read()+var);
        }
    };

    sc_in<int> in0;
    sc_out<int> out0;
    nested2 nd;
    SC_CTOR(nested1)
    :nd(nested2)
    {
        nd.in0(in0);
        nd.out0(out0);
    }
};
```

```
SC_MODULE(nested2)
{
    sc_in<int> in0;
    sc_out<int> out0;
    SC_CTOR(nested2)
    {
        SC_METHOD(process);
        sensitive<<in0;
    }
    void process()
    {
        int var =10;
        out0.write(in0.read()+var);
    }
};

SC_MODULE(nested1)
{
    sc_in<int> in0;
    sc_out<int> out0;
    nested2 nd;
    SC_CTOR(nested1)
    :nd(nested2)
```

```
{
nd.in0(in0);
nd.out0(out0);
}
};
```

此外，SC_MODULE 不得衍生自另一个 SC_MODULE，如以下示例所示：

```
SC_MODULE( BASE)
{
sc_in<bool> clock; //clock input
sc_in<bool> reset;
SC_CTOR( BASE) {}

};

class DUT: public BASE
{
public:
sc_in<bool> start;
sc_in<sc_uint<8> > din;
};
```



建议：在模块内部定义模块构造函数。

像以下第 1 个 SC_MODULE 示例这样的案例应按以下第 2 个 SC_MODULE 示例所示进行变换。

```
SC_MODULE(dut) {
sc_in<int> in0;
sc_out<int>out0;
SC_HAS_PROCESS(dut);
dut(sc_module_name nm);
...
};

dut::dut(sc_module_name nm)
{
SC_METHOD(process);
sensitive<<in0;
}
```

```
SC_MODULE(dut) {
sc_in<int> in0;
sc_out<int>out0;

SC_HAS_PROCESS(dut);
dut(sc_module_name nm)
:sc_module(nm)
{
SC_METHOD(process);
sensitive<<in0;
}
};
```

Vivado HLS 不支持对 SC_THREAD 进行综合。

使用 SC_METHOD

以下代码示例显示了小型组合设计的头文件 (sc_combo_method.h)，此设计采用 SC_METHOD 来对半加器进行建模。顶层设计名称 (c_combo_method) 是在 SC_MODULE 中指定的。

```
#include <systemc.h>

SC_MODULE(sc_combo_method){
    //Ports
    sc_in<sc_uint<1> > a,b;
    sc_out<sc_uint<1> > sum,carry;

    //Process Declaration
    void half_adder();

    //Constructor
    SC_CTOR(sc_combo_method){

        //Process Registration
        SC_METHOD(half_adder);
        sensitive<<a<<b;
    }
};
```

此设计具有 2 个单位输入端口 (a 和 b)。SC_METHOD 会随任一输入端口中的任何状态变更而变，并执行 half_adder 函数。half_adder 函数是在 sc_combo_method.cpp 文件中指定的，如下代码示例所示。此代码用于计算输出端口进位的值。

```
#include "sc_combo_method.h"

void sc_combo_method::half_adder(){
    bool s,c;
    s=a.read() ^ b.read();
    c=a.read() & b.read();
    sum.write(s);
    carry.write(c);

#ifdef __SYNTHESIS__
    cout << Sum is << a << ^ << b << = << s << : <<
    sc_time_stamp() <<endl;
    cout << Car is << a << & << b << = << c << : <<
    sc_time_stamp() <<endl;
#endif
}
```

注释：以上示例显示了如何使用 __SYNTHESIS__ 宏来防止对 C 语言仿真期间用于显示值的任意 cout 语句执行综合。

只需在要综合的代码中使用 __SYNTHESIS__ 宏即可。请勿在测试激励文件中使用该宏，因为 C 语言仿真或 C RTL 协同仿真不会遵循其指示进行操作。

以下代码示例显示了前述示例的测试激励文件。此测试激励文件显示了使用 Vivado HLS 时所需的几个重要属性。

```
#ifdef __RTL_SIMULATION__
#include "sc_combo_method_rtl_wrapper.h"
#define sc_combo_method sc_combo_method_RTL_wrapper
#else
#include "sc_combo_method.h"
```



```
#endif
#include "tb_init.h"
#include "tb_driver.h"

int sc_main (int argc , char *argv[])
{
    sc_report_handler::set_actions(/IEEE-Std-1666/deprecated, SC_DO_NOTHING);
    sc_report_handler::set_actions( SC_ID_LOGIC_X_TO_BOOL_, SC_LOG);
    sc_report_handler::set_actions( SC_ID_VECTOR_CONTAINS_LOGIC_VALUE_, SC_LOG);
    sc_report_handler::set_actions( SC_ID_OBJECT_EXISTS_, SC_LOG);

    sc_signal<bool> s_reset;
    sc_signal<sc_uint<1>> s_a;
    sc_signal<sc_uint<1>> s_b;
    sc_signal<sc_uint<1>> s_sum;
    sc_signal<sc_uint<1>> s_carry;

    // Create a 10ns period clock signal
    sc_clock s_clk(s_clk,10,SC_NS);

    tb_init U_tb_init(U_tb_init);
    sc_combo_method U_dut(U_dut);
    tb_driver U_tb_driver(U_tb_driver);

    // Generate a clock and reset to drive the sim
    U_tb_init.clk(s_clk);
    U_tb_init.reset(s_reset);

    // Connect the DUT
    U_dut.a(s_a);
    U_dut.b(s_b);
    U_dut.sum(s_sum);
    U_dut.carry(s_carry);

    // Drive stimuli from dat* ports
    // Capture results at out* ports
    U_tb_driver.clk(s_clk);
    U_tb_driver.reset(s_reset);
    U_tb_driver.dat_a(s_a);
    U_tb_driver.dat_b(s_b);
    U_tb_driver.out_sum(s_sum);
    U_tb_driver.out_carry(s_carry);

    // Sim for 200
    int end_time = 200;

    cout << INFO: Simulating << endl;

    // start simulation
    sc_start(end_time, SC_NS);

    if (U_tb_driver.retval != 0) {
        printf(Test failed !!!\n);
    } else {
        printf(Test passed !\n);
    }
    return U_tb_driver.retval;
};
```

要在 Vivado HLS 中使用 `cosim_design` 功能执行 RTL 仿真，测试激励文件必须包含上述示例顶部显示的宏。对于名为 DUT 的设计，必须使用以下代码，其中 DUT 替换为实际设计名称。

```
#ifndef __RTL_SIMULATION__
#include "DUT_rtl_wrapper.h"
#define DUT DUT_RTL_wrapper
#else
#include "DUT.h" //Original unmodified code
#endif
```

您必须将此代码添加到包含设计头文件的测试激励文件中。否则，`cosim_design` RTL 仿真将失败。



建议：请将以上示例中所示的报告处理程序函数添加到配合 Vivado HLS 一起使用的所有 SystemC 语言测试激励文件中。

```
sc_report_handler::set_actions(/IEEE_Std_1666/deprecated, SC_DO_NOTHING);
sc_report_handler::set_actions( SC_ID_LOGIC_X_TO_BOOL_, SC_LOG);
sc_report_handler::set_actions( SC_ID_VECTOR_CONTAINS_LOGIC_VALUE_, SC_LOG);
sc_report_handler::set_actions( SC_ID_OBJECT_EXISTS_, SC_LOG);
```

这些设置可阻止在 RTL 仿真期间打印无关消息。

这些消息中最重要的部分是如下警告：

```
Warning: (W212) sc_logic value 'X' cannot be converted to bool
```

这表明布局在已综合的设计周围的适配器以未知 (X) 值开头。并非所有 SystemC 类型都支持未知 (X) 值。当对不支持未知 (X) 值的类型应用未知 (X) 值时，就会发出此警告，通常在从测试激励文件应用激励前会发生这种情况，并且通常可忽略该警告。

最后，以上示例中的测试激励文件会对结果执行检查。

如果结果正确，则返回零值。在此情况下，将在 `tb_driver` 函数内验证结果，但在顶层测试激励文件中检查返回值并返回这些值。

```
if (U_tb_driver.retval != 0) {
    printf(Test failed !!!\n);
} else {
    printf(Test passed !\n);
}
return U_tb_driver.retval;
```

例化 SC_MODULES

如下代码示例所示，可对 `SC_MODULE` 的层级例化进行综合。在此代码示例中，已将使用 `SC_METHOD` 中的半加器设计 (`sc_combo_method`) 的 2 个实例例化，以创建 1 个全加器设计。

```
#include <systemc.h>
#include "sc_combo_method.h"

SC_MODULE(sc_hier_inst){
//Ports
```

```

sc_in<sc_uint<1> > a, b, carry_in;
sc_out<sc_uint<1> > sum, carry_out;

//Variables
sc_signal<sc_uint<1> > carry1, sum_int, carry2;

//Process Declaration
void full_adder();

//Half-Adder Instances
sc_combo_method U_1, U_2;

//Constructor
SC_CTOR(sc_hier_inst)
:U_1(U_1)
,U_2(U_2)
{
    // Half-adder inst 1
    U_1.a(a);
    U_1.b(b);
    U_1.sum(sum_int);
    U_1.carry(carry1);

    // Half-adder inst 2
    U_2.a(sum_int);
    U_2.b(carry_in);
    U_2.sum(sum);
    U_2.carry(carry2);

    //Process Registration
    SC_METHOD(full_adder);
    sensitive<<carry1<<carry2;
}
};

```

full_adder 函数用于为 carry_out 信号创建逻辑，如下代码示例所示。

```

#include "sc_hier_inst.h"

void sc_hier_inst::full_adder(){
    carry_out= carry1.read() | carry2.read();
}

```

使用 SC_CTHREAD

构造函数进程 SC_CTHREAD 用于对已时控的进程（线程）进行建模，这是对顺序设计进行建模的主要方式。以下代码示例显示了高亮顺序设计的主要属性的示例。

- 此数据具有关联的握手信号，使其能够在综合前后使用相同的测试激励文件来进行运算。
- 时钟上的 SC_CTHREAD 敏感信号可用于对执行函数的时间进行建模。
- SC_CTHREAD 支持复位行为。

```

#include <systemc.h>

SC_MODULE(sc_sequ_cthread){
    //Ports

```

```

sc_in <bool>   clk;
sc_in <bool>   reset;
sc_in <bool>   start;
sc_in<sc_uint<16> > a;
sc_in<bool>   en;
sc_out<sc_uint<16> > sum;
sc_out<bool>   vld;

//Variables
sc_uint<16> acc;

//Process Declaration
void accum();

//Constructor
SC_CTOR(sc_sequ_cthread){

    //Process Registration
    SC_CTHREAD(accum,clk.pos());
    reset_signal_is(reset,true);
}
};

```

- 以下代码示例显示了 accum 函数。此示例用于演示：
- 核建模流程是包含 wait() 语句的无限 while() 循环。
- 变量的任意初始化均在无限 while() 循环之前执行。当 SC_CTHREAD 识别出复位时，即执行此代码。
- 数据读取和写入由握手协议来加以限定。

```

#include "sc_sequ_cthread.h"

void sc_sequ_cthread::accum(){

    //Initialization
    acc=0;
    sum.write(0);
    vld.write(false);
    wait();

    // Process the data
    while(true) {
        // Wait for start
        while (!start.read()) wait();

        // Read if valid input available
        if (en) {
            acc = acc + a.read();
            sum.write(acc);
            vld.write(true);
        } else {
            vld.write(false);
        }
        wait();
    }
}
}

```

循环综合

对循环进行编码时，必须考虑 Vivado HLS SystemC 调度规则，即 Vivado HLS 必须始终以新状态开始对循环进行综合。例如，给定以下设计：

注释：针对此示例仅显示最少量的代码。

```
sc_in<bool> start;
sc_in<bool> enable;

process code:
  unsigned count = 0;
  while (!start.read()) wait();
  for(int i=0;i<100; i++)
  {
    if(enable.read()) count++;
    wait();
  }
```

添加以下测试激励文件：

```
start = true;
enable=true;
wait(1);
start = false;
wait(99);
enable=false;
```

此设计在 C 语言仿真期间执行并对使能信号进行采样。然后，计数达到 100。综合后，SystemC 循环调度规则要求循环以新状态启动，并执行循环中调度为此时间点后开始执行的所有操作。例如，以下代码显示的 `wait` 语句调用了 First Loop Clock：

```
sc_in<bool> start;
sc_in<bool> enable;

process code:
  unsigned count = 0;
  while (!start.read()) wait();
  for(int i=0;i<100; i++)
  {
    wait(); //First Loop Clock
    if(enable.read()) count++;
    wait();
  }
```

当初时钟对启动信号进行采样后，在新时钟首次对 `enable` 信号进行采样前存在 2 个时钟周期延迟。此新时钟与测试激励文件中的第 2 个时钟（即连续 99 个时钟中的首个时钟）同时发生。在第 3 个测试激励文件时钟（即连续 99 个时钟中的第 2 个时钟）上，该时钟将首次对 `enable` 信号进行采样。在此情况下，RTL 设计仅计数至 99，然后将 `enable` 设置为 `false`。



建议：在 SystemC 中对循环进行编码前，赛灵思强烈建议您将 `wait()` 语句置于循环中首个项所在位置。

在以下示例中，wait() 语句为综合后循环中的首个时钟或状态。

```
sc_in<bool> start;
sc_in<bool> enable;

process code:
    unsigned count = 0;
    while (!start.read()) wait();
        for(int i=0;i<100; i++)
        {
            wait(); // Put the 'wait()' at the beginning of the loop
            if(enable.read()) count++;
        }
    }
```

使用多个时钟进行综合

不同于 C 和 C++ 综合，SystemC 支持使用多个时钟的设计。在多时钟设计中，必须在 SC_CTHREAD 中捕获与每个时钟关联的功能。

以下示例代码显示了含 2 个时钟的设计（clock 和 clock2）。

- 其中 1 个时钟用于激活 SC_CTHREAD 以执行 Prc1 函数。
- 另 1 个时钟用于激活 SC_CTHREAD 以执行 Prc2 函数。

综合后，与 Prc1 函数关联的顺序逻辑均由 clock 进行时钟，而 clock2 则用于驱动 Prc2 函数的所有时序逻辑。

```
#includesystemc.h
#includeetlm.h
using namespace tlm;

SC_MODULE(sc_multi_clock)
{
    //Ports
    sc_in <bool>  clock;
    sc_in <bool>  clock2;
    sc_in <bool>  reset;
    sc_in <bool>  start;
    sc_out<bool>  done;
    sc_fifo_out<int> dout;
    sc_fifo_in<int> din;

    //Variables
    int share_mem[100];
    bool write_done;

    //Process Declaration
    void Prc1();
    void Prc2();

    //Constructor
    SC_CTOR(sc_multi_clock)
    {
        //Process Registration
        SC_CTHREAD(Prc1,clock.pos());
        reset_signal_is(reset,true);
    }
}
```

```
SC_CTHREAD(Prc2,clock2.pos());
reset_signal_is(reset,true);
}
};
```

通信通道

线程、方法和模块（本身包含线程和方法）之间的通信应仅限使用通道来执行。请勿针对线程间通信使用简单变量。

赛灵思建议使用 `sc_buffer` 或 `sc_signal` 在不同进程（线程或方法）之间进行通信。如果可先写入多个值然后再读取首个值，则可使用 `sc_fifo` 和 `tlm_fifo`。

对于 `sc_fifo` 和 `tlm_fifo`，支持采用以下方法进行综合：

- 非阻塞读写
- 阻塞读写
- `num_available()/num_free()`
- `nb_can_put()/nb_can_get()`

顶层 SystemC 端口

SystemC 语言设计中的端口在源代码中指定。不同于 C 和 C++ 函数，在 SystemC 语言中，Vivado HLS 仅在受支持的内存接口上执行接口综合。

顶层接口上的所有端口都必须为以下任一类型：

- `sc_in_clk`
- `sc_in`
- `sc_out`
- `sc_inout`
- `sc_fifo_in`
- `sc_fifo_out`
- `ap_mem_if`
- `AXI4M_bus_port`

除受支持的内存接口外，设计与测试激励文件之间的所有握手都必须在 SystemC 语言函数中显式建模。受支持的内存接口包括：

- `sc_fifo_in`
- `sc_fifo_out`
- `ap_mem_if`

Vivado HLS 可能会为 SystemC 语言设计增加额外时钟周期以满足时序所需。由于综合后的时钟周期数可能产生变化，SystemC 语言设计应将所有数据传输与测试激励文件握手。

Vivado HLS 不支持采用传输事务级建模（使用 TLM 2.0）和基于事件的建模进行综合。

SystemC 接口综合

通常，Vivado HLS 不会在 SystemC 上执行接口综合。它确实支持某些内存接口的接口综合，例如 RAM 和 FIFO 端口。

RAM 端口综合

不同于 C 和 C++ 语言的综合，Vivado HLS 不会将数组端口变换为 RTL RAM 端口。在以下 SystemC 语言代码中，您必须使用 Vivado HLS 指令来将数组端口分区到各元素中。

否则，此示例代码将无法完成综合：

```
SC_MODULE( dut )
{
    sc_in<T> in0[N];
    sc_out<T> out0[N];

    ...
    SC_CTOR( dut )
    {
        ...
    }
};
```

RAM 端口综合编码示例

用于将这些数组分区到各元素的指令是：

```
set_directive_array_partition dut in0 -type complete
set_directive_array_partition dut out0 -type complete
```

如果 N 值较大，则会导致 RTL 接口上出现大量独立标量端口。

以下代码示例显示了如何在 SystemC 语言仿真中对 RAM 接口进行建模并通过 Vivado HLS 对其进行完整综合。在此代码示例中，数组类型替换为可综合到 RAM 端口中的 `ap_mem_if` 类型。

- 要使用 `ap_mem_port` 类型，必须包含来自 Vivado HLS 安装的 `include/ap_sysc` 目录中的头文件 `ap_mem_if.h`。

注释：在 Vivado HLS 环境内，包含 `include/ap_sysc` 目录。

- `din` 和 `dout` 的数组类型替换为 `ap_mem_port` 类型。在代码示例下对各字段进行了解释。

```
#includesystemc.h
#include "ap_mem_if.h"

SC_MODULE(sc_RAM_port)
{
    //Ports
    sc_in <bool>  clock;
    sc_in <bool>  reset;
    sc_in <bool>  start;
    sc_out<bool>  done;
    //sc_out<int> dout[100];
    //sc_in<int>  din[100];
```



```

ap_mem_port<int, int, 100, RAM_2P> dout;
ap_mem_port<int, int, 100, RAM_2P> din;

//Variables
int share_mem[100];
sc_signal<bool> write_done;

//Process Declaration
void Prc1();
void Prc2();

//Constructor
SC_CTOR(sc_RAM_port)
: dout (dout),
  din (din)
{
    //Process Registration
    SC_CTHREAD(Prc1,clock.pos());
    reset_signal_is(reset,true);

    SC_CTHREAD(Prc2,clock.pos());
    reset_signal_is(reset,true);
}
};

```

- ap_mem_port 类型的格式为：

```

ap_mem_port (<data_type>, < address_type>, <number_of_elements>,
<Mem_Target>)

```

- data_type 是用于存储的数据元素的类型。在以上示例中，这些均为标准 int 类型。
- address_type 是用于地址总线的类型。此类型所含数据位数应足以对数组中所有元素进行寻址，否则 C 语言仿真将失败。
- number_of_elements 用于指定要建模的数组中的元素数量。
- Mem_Target 用于指定此端口将连接到的内存，因而可判定最终 RTL 上的 I/O 端口。要获取可用目标列表，请参阅下表。

下表中所述内存目标会影响综合所创建的端口以及在设计中调度操作的方式。例如，双端口 RAM：

- 生成的 I/O 端口数量为单端口 RAM 的 2 倍。
- 可允许并行调度内部操作（前提是循环和数据依赖关系等代码构造允许此操作）。

表 46: SystemC ap_mem_port 内存目标

目标 RAM	描述
RAM_1P	单端口 RAM
RAM_2P	双端口 RAM
RAM_T2P	真正的双端口 RAM，支持在输入和输出两端执行读写操作
ROM_1P	单端口 ROM
ROM_2P	双端口 ROM

在接口上定义 `ap_mem_port` 后，即可按任何其它数组中相同的方式来访问代码中的变量：

```
dout[i] = share_mem[i] + din[i];
```

以下代码示例中显示了支持上述示例的测试激励文件。测试激励文件中的 `ap_mem_chn` 类型必须支持 `ap_mem_port` 类型。`ap_mem_chn` 类型在头文件 `ap_mem_if.h` 中定义，支持的字段与 `ap_mem_port` 相同。

```
#ifdef __RTL_SIMULATION__
#include "sc_RAM_port_rtl_wrapper.h"
#define sc_RAM_port sc_RAM_port_RTL_wrapper
#else
#include "sc_RAM_port.h"
#endif
#include "tb_init.h"
#include "tb_driver.h"
#include "ap_mem_if.h"

int sc_main (int argc , char *argv[])
{
    sc_report_handler::set_actions(/IEEE Std 1666/deprecated, SC_DO_NOTHING);
    sc_report_handler::set_actions( SC_ID_LOGIC_X_TO_BOOL_, SC_LOG);
    sc_report_handler::set_actions( SC_ID_VECTOR_CONTAINS_LOGIC_VALUE_, SC_LOG);
    sc_report_handler::set_actions( SC_ID_OBJECT_EXISTS_, SC_LOG);

    sc_signal<bool> s_reset;
    sc_signal<bool> s_start;
    sc_signal<bool> s_done;
    ap_mem_chn<int,int, 100, RAM_2P> dout;
    ap_mem_chn<int,int, 100, RAM_2P> din;

    // Create a 10ns period clock signal
    sc_clock s_clk(s_clk,10,SC_NS);

    tb_init U_tb_init(U_tb_init);
    sc_RAM_port U_dut(U_dut);
    tb_driver U_tb_driver(U_tb_driver);

    // Generate a clock and reset to drive the sim
    U_tb_init.clk(s_clk);
    U_tb_init.reset(s_reset);
    U_tb_init.done(s_done);
    U_tb_init.start(s_start);

    // Connect the DUT
    U_dut.clock(s_clk);
    U_dut.reset(s_reset);
    U_dut.done(s_done);
    U_dut.start(s_start);
    U_dut.dout(dout);
    U_dut.din(din);

    // Drive inputs and Capture outputs
    U_tb_driver.clk(s_clk);
    U_tb_driver.reset(s_reset);
    U_tb_driver.start(s_start);
    U_tb_driver.done(s_done);
    U_tb_driver.dout(dout);
    U_tb_driver.din(din);

    // Sim
```

```
int end_time = 1100;

cout << INFO: Simulating << endl;

// start simulation
sc_start(end_time, SC_NS);

if (U_tb_driver.retval != 0) {
    printf(Test failed !!!\n);
} else {
    printf(Test passed !\n);
}
return U_tb_driver.retval;
};
```

FIFO 端口综合

可以从标准 SystemC `sc_fifo_in` 和 `sc_fifo_out` 端口直接综合顶层接口上的 FIFO 端口。有关在接口上使用 FIFO 端口的示例，请参阅以下代码示例。

综合后，每个 FIFO 端口都有一个数据端口和关联的 FIFO 控制信号。

- 输入有 `empty` 和 `read` 端口。
- 输出有 `full` 和 `write` 端口。

通过使用 FIFO 端口，在 RTL 测试激励文件中会添加同步数据传输所需的握手。

```
#includesystemc.h
#include tlm.h
using namespace tlm;

SC_MODULE(sc_FIFO_port)
{
    //Ports
    sc_in<bool> clock;
    sc_in<bool> reset;
    sc_in<bool> start;
    sc_out<bool> done;
    sc_fifo_out<int> dout;
    sc_fifo_in<int> din;

    //Variables
    int share_mem[100];
    bool write_done;

    //Process Declaration
    void Prc1();
    void Prc2();

    //Constructor
    SC_CTOR(sc_FIFO_port)
    {
        //Process Registration
        SC_CTHREAD(Prc1, clock.pos());
        reset_signal_is(reset, true);
    }
}
```

```
SC_CTHREAD(Prc2,clock.pos());
reset_signal_is(reset,true);
}
};
```

不受支持的 SystemC 语言构造

模块和构造函数

- SC_MODULE 不能嵌套在另一个 SC_MODULE 内。
- SC_MODULE 不能衍生自另一个 SC_MODULE。
- Vivado HLS 不支持 SC_THREAD。
- Vivado HLS 支持 SC_CTHREAD 的时控版本。

模块例化

SC_MODULE 无法使用 new 进行例化。以下示例中所示代码 (SC_MODULE(TOP)) 必须按其下方示例中所示方式来进行变换。

```
{
  sc_in<T> din;
  sc_out<T> dout;

  M1 *t0;

  SC_CTOR(TOP){
    t0 = new M1(t0);
    t0->din(din);
    t0->dout(dout);
  }
}
```

```
SC_MODULE(TOP)
{
  sc_in<T> din;
  sc_out<T> dout;

  M1 t0;

  SC_CTOR(TOP)
  : t0("t0")
  {
    t0.din(din);
    t0.dout(dout);
  }
}
```

模块构造函数

仅限名称参数才能搭配模块构造函数使用。不允许传递类型为 `int` 的 `temp` 变量。请参阅以下示例。

```
SC_MODULE(dut) {
    sc_in<int> in0;
    sc_out<int> out0;
    int var;
    SC_HAS_PROCESS(dut);
    dut(sc_module_name nm, int temp)
:sc_module(nm),var(temp)
    { ... }
};
```

模块构造函数代码示例

虚拟函数

Vivado HLS 不支持虚拟函数。由于以下代码使用虚拟函数，因此无法进行综合。

```
SC_MODULE(DUT)
{
    sc_in<int> in0;
    sc_out<int> out0;

    virtual int foo(int var1)
    {
        return var1+10;
    }

    void process()
    {
        int var=foo(in0.read());
        out0.write(var);
    }
    ...
};
```

顶层接口端口

Vivado HLS 不支持读取 `sc_out` 端口。以下代码不受支持，因为它读取 `out0`。

```
SC_MODULE(DUT)
{
    sc_in<T> in0;
    sc_out<T> out0;
    ...
    void process()
    {
        int var=in0.read()+out0.read();
        out0.write(var);
    }
};
```

高层次综合参考指南

命令参考

add_files

描述

将设计源文件添加到当前工程中。

该工具会搜索当前目录中是否存在设计源文件中包含的任何头文件。要使用存储在其它目录中的头文件，请使用 `-cflags` 选项将这些目录添加到搜索路径中。

语法

```
add_files [OPTIONS] <src_files>
```

- `<src_files>` 用于列出含设计描述的源文件。

选项

```
-tb
```

指定设计测试激励文件使用的任意文件。

这些文件不进行综合。通过 `cosim_design` 命令执行综合后验证时会使用这些文件。

该选项不允许在源文件列表中包含设计文件。请单独使用 `add_files` 命令来添加设计文件和测试激励文件文件。

```
-cflags <string>
```

含任意期望的 GCC 编译选项的字符串。

```
-blackbox <file_name.json>
```

指定用于 RTL 黑盒的 JSON 文件。此文件中的信息供 HLS 编译器在综合期间以及运行 C 语言仿真和协同仿真时使用。如需了解更多信息，请参阅 [RTL 黑盒](#)。

```
- csimflags <string>
```

含任意期望的仿真编译选项的字符串。使用该选项指定的标记仅适用于仿真编译（包括 C 语言仿真和 RTL 协同仿真），不适用于综合编译。该选项不影响 `-cflags` 选项。

编译指示

没有等效的编译指示。

示例

将 3 个设计文件添加到工程中。

```
add_files a.cpp
add_files b.cpp
add_files c.cpp
```

使用单一命令行添加多个文件。

```
add_files "a.cpp b.cpp c.cpp"
```

添加含编译器标记的 SystemC 语言文件，以启用 USE_RANDOM 宏，并为头文件指定一条额外的搜索路径：子目录 ./lib_functions。

```
add_files top.cpp -cflags "-DUSE_RANDOM -I./lib_functions"
```

使用 -tb 选项将测试激励文件文件添加到工程中。此示例使用一条命令添加多个文件，包括：

- 测试激励文件 a_test.cpp
- 测试激励文件读取的所有数据文件：
 - input_stimuli.dat
 - out.gold.dat

```
add_files -tb "a_test.cpp input_stimuli.dat out.gold.dat"
```

如果先前示例中的测试激励文件数据文件存储在单独的目录（例如，test_data）中，可将此目录添加到工程中，以代替添加个别数据文件。

```
add_files -tb a_test.cpp
add_files -tb test_data
```

close_project

描述

关闭当前工程。在 Vivado® HLS 会话中，此工程不再处于活动状态。

close_project 命令：

- 可阻止您输入任何特定于工程的命令或特定于解决方案的命令。
- 并非必需命令。打开或创建新工程即可关闭当前工程。

语法

```
close_project
```

选项

此命令不含任何选项。

编译指示

没有等效的编译指示。

示例

```
close_project
```

- 关闭当前工程。
- 保存所有结果。

close_solution

描述

关闭当前解决方案。在 Vivado HLS 会话中，当前解决方案不再处于活动状态。

`close_solution` 命令：

- 可阻止您输入任何特定于解决方案的命令。
- 并非必需命令。打开或创建新解决方案即可关闭当前解决方案。

语法

```
close_solution
```

选项

此命令不含任何选项。

编译指示

没有等效的编译指示。

示例

```
close_solution
```

- 关闭当前工程。
- 保存所有结果。

config_array_partition

描述

指定数组分区的默认行为。

语法

```
config_array_partition [OPTIONS]
```

选项

```
-auto_partition_threshold <int>
```

设置数组分区的阈值（包括不含常量索引的数组）。

所含元素少于指定阈值限制的数组将被分区为独立元素，除非在此类数组上应用接口或核规格。默认值为 4。

```
-auto_promotion_threshold <int>
```

设置含常量索引的数组分区阈值。

所含元素数量少于指定阈值限制并且包含常量索引（索引不可变）的数组将被分区为独立元素。默认值为 64。

```
-include_extern_globals
```

包含来自受吞吐量驱动自动分区的外部全局数组。

```
-include_ports
```

启用 I/O 数组的自动分区。

这样可将单个数组 I/O 端口减缩为多个端口。每个端口大小都与各数组元素大小相同。

```
-maximum_size <int>
```

指定数组分区的最大大小。

```
-scalarize_all
```

将设计中的所有数组分区到其各自的元素中。

```
-throughput_driven
```

基于吞吐量启用数组自动分区。

Vivado HLS 会判定将数组分区到独立元素中是否能使其满足任意指定的吞吐量要求。

编译指示

没有等效的编译指示。

示例

将设计中包含不足 12 个元素的所有数组（不包括全局数组）分区到各元素中。

```
_config_array_partition -auto_partition_threshold 12 -  
_include_extern_globals_
```

指示 Vivado HLS 判定要对哪些数组进行分区以改进吞吐量。

```
config_array_partition -throughput_driven -include_ports
```

将设计中的所有数组（包括全局数组）分区到各元素中。

```
config_array_partition -scalarize_all
```

config_bind

描述

设置微架构绑定的默认选项。

绑定是将运算符（如、加法、乘法和位移）映射到特定 RTL 实现的过程。例如，作为组合或流水线化 RTL 乘法器来实现的 `mult` 运算。

语法

```
config_bind [OPTIONS]
```

选项

```
-effort (low|medium|high)
```

最优化工作量可控制运行时间与最优化之间的权衡取舍。

- 默认工作量为 `Medium`。
- 最优化工作量为 `Low` 时可改善运行时间，适用于只需少量最优化的场景。例如，当所有 `if-else` 语句的每个分支都包含互斥运算符并且无法达成运算符共享时。
- 最优化工作量为 `High` 时可能增加运行时间，但通常可产生更好的结果。

```
-min_op <string>
```

请最大限度减少特定运算符的实例数。如果代码中包含多个此类运算符，则必须最大限度减少共享这些运算符的 RTL 资源（核）数量。

以下运算符可指定为实参：

- `add` - 加法
- `sub` - 减法

- mul - 乘法
- icmp - 整数比较
- sdiv - 有符号除
- udiv - 无符号除
- srem - 有符号取余
- urem - 无符号取余
- lshr - 逻辑右移
- ashr - 算术右移
- shl - 左移

编译指示

没有等效的编译指示。

示例

指示 Vivado HLS 执行下列操作：

- 增加绑定过程中的工作量。
- 尝试采用其它选项来实现运算符。
- 尝试生成资源利用率更高的设计。

```
config_bind -effort high
```

最大限度减少乘法运算符的数量，最大限度减少生成的 RTL 的乘法器数量。

```
config_bind -min_op mul
```

config_compile

描述

配置前端编译的默认行为。

语法

```
config_compile [OPTIONS]
```

选项

```
-ignore_long_run_time
```

跳过因大量负载或存储指令导致的运行时间过长警告。

```
-name_max_length <threshold>
```

指定函数名称的最大长度。如果名称长度超出该阈值，则名称最后一部分将被截断。默认值为 80。

```
-no_signed_zeros
```

忽略浮点 0 值的符号类型，以便编译器对浮点运算执行主动最优化。默认设置为关 (off)。

注释：使用该选项可能改变任意浮点运算的结果，导致 C/RTL 协同仿真不匹配。请确保测试激励文件容许差异，并检查差异的裕度，而不是精确值。请参阅[编码示例](#)中表 1-6 的 `cpp_math` 示例，以获取在测试激励文件中的裕度和容限使用示例。

```
-pipeline_loops <threshold>
```

指定自动执行循环流水线化时，使用较低的阈值。默认不自动执行循环流水线化。

如果应用该选项，则将对所含行程计数高于阈值的最内层循环进行流水线化，或者如果最内层循环的行程计数小于或等于阈值，则对其父循环执行流水线化。如果最内层的循环无父循环，则忽略最内层的循环的行程计数，并对其进行流水线化。

阈值越高，对父循环进行流水线化的可能性更高，并且运行时间将增加。

```
-unsafe_math_optimizations
```

忽略浮点 0 值的符号类型并启用关联的浮点运算，以便编译器对浮点运算执行主动最优化。默认设置为关 (off)。

注释：使用该选项可能改变任意浮点运算的结果，导致 C/RTL 协同仿真不匹配。请确保测试激励文件容许差异，并检查差异的裕度，而不是精确值。请参阅[编码示例](#)中表 1-6 的 `cpp_math` 示例，以获取在测试激励文件中的裕度和容限使用示例。

编译指示

没有等效的编译指示。

示例

对所含行程计数高于 30 的最内层循环进行流水线化，或者如果最内层循环的行程计数小于或等于 30，则对其父循环进行流水线化。

```
config_compile -pipeline_loops 30
```

忽略浮点 0 值的符号类型：

```
config_compile -no_signed_zeros
```

忽略浮点 0 值的符号类型并启用关联的浮点运算：

```
config_compile -unsafe_math_optimizations
```

config_core

描述

用于全局配置指定的核。

语法

```
config_core [OPTIONS] <core>
```

选项

- <core> <string>

用于指定核的名称。

- -latency <int>

用于指定调度期间要使用的核的新默认时延。

编译指示

config_core 命令没有等效的编译指示。

示例

更改核 DSP48 的默认时延。

```
config_core DSP48 -latency 4
```

config_dataflow

描述

- 指定数据流流水线化（通过 set_directive_dataflow 命令实现）的默认行为。
- 允许您指定默认通道内存类型和深度。

语法

```
config_dataflow [OPTIONS]
```

选项

```
-default_channel [fifo|pingpong]
```

默认情况下，在使用数据流流水线化时，以 pingpong 方式配置的 RAM 内存用于缓存函数或循环之间的数据。使用流传输数据时（即数据始终连续读写），FIFO 内存更高效且可选作为默认内存类型。



提示：使用 set_directive_stream 命令将数组设置为流传输以执行 FIFO 访问。

```
-fifo_depth <integer>
```

指定 FIFO 的默认深度。默认深度为 2。

使用乒乓内存时，该选项无效。如果不指定该选项，默认深度为 2，或者如果这是转换为 FIFO 的数组，则默认大小为原始数组的大小。在某些情况下，这可能过于保守，导致 FIFO 过大。当您确认 FIFO 过大时，可使用该选项。



注意！ 请谨慎使用该选项。FIFO 深度不足可能导致死锁。

```
-scalar_fifo_depth
```

针对标量传输 FIFO 请指定最小值。

通过标量传输，编译器会将标量从 C 语言代码转换为 FIFO。可通过 `-start_fifo_depth` 设置这些 FIFO 的最小大小。如果不提供该选项，则使用 `-fifo_depth` 的值。

```
-start_fifo_depth
```

指定起始传输 FIFO 的最小深度。

仅当生产者与使用者间通道为 FIFO 时，该选项才有效。该选项使用的默认值与 `-fifo_depth` 选项同为 2。此 FIFO 有时可能导致死锁，在此情况下您可使用该选项来增加 FIFO 的深度。

编译指示

没有等效的编译指示。

示例

将默认通道从乒乓内存更改为 FIFO。

```
config_dataflow -default_channel
```

将默认通道从乒乓内存更改为深度为 6 的 FIFO。

```
config_dataflow -default_channel fifo -fifo_depth 6
```



注意！ 如果设计实现需要包含超过 6 个元素的 FIFO，此设置会导致设计的 RTL 验证失败。请谨慎使用该选项，因为它属于用户覆盖选项。

config_export

描述

配置 `export_design` 的选项，它可用于运行下游工具或者封装 Vivado IP 或 Vitis 工程 XO。

语法

```
config_export [OPTIONS]
```

选项

```
-description <string>
```

为生成的 IP 目录 IP 提供描述。

```
-display_name
```

为生成的 IP 提供显示名称。

```
-flow (syn|impl)
```

为使用 RTL 综合指定的 HDL 获取更准确的时序和利用率数据。`syn` 选项可执行 RTL 综合，`impl` 选项可执行 RTL 综合和实现，包括已综合的门控的详细布局布线。在 Vivado HLS GUI 中，这些选项显示为复选框，并分别标记为“Vivado Synthesis”和“Vivado Synthesis, place and route stage”。

```
-format (ipcatalog|sysgen|syn_dcp)
```

指定 IP 封装格式。受支持的格式为：

- `sysgen`

采用面向赛灵思 Design Suite 的 System Generator for DSP（仅限 7 系列器件）接受的格式

- `ip_catalog`

采用适合添加到赛灵思 IP 目录的格式（7 系列器件的默认格式）

- `syn_dcp`

适用于 Vivado Design Suite 的已综合的检查点文件。如果使用该选项，则将自动执行 RTL 综合。

```
-ip_name
```

为生成的 IP 提供 IP 名称。

```
-library
```

指定生成的 IP 目录 IP 的库名称。

```
-rtl (verilog |VHDL)
```

选择执行 `-flow` 选项时使用的 HDL。如果不指定此项，那么默认语言为 Verilog。

```
-vitis_tcl
```

控制输出 Tcl 文件的位置。

```
-taxonomy
```

该选项用于封装 IP。

```
-vendor
```

指定用于生成的 IP 目录 IP 的供应商字符串。

```
-version
```

指定用于生成的 IP 目录的版本字符串。

```
-vivado_ip_cache <path-to-ip-cache>
```

添加到 OOC Vivado 工程的 IP 高速缓存路径。缩短 RTL 综合的运行时间（如果综合达到高速缓存）。默认值为 none。

```
-vivado_imp_strategy {default|<strategy>}
```

控制 export_design -evaluate Vivado 运行中使用的实现策略。

该选项的值应为 default 或有效的 Vivado 实现策略的名称。

```
-vivado_phy_opt {none|place|route|all}
```

控制在 export_design -evaluate Vivado 运行中是否启用物理最优化。

该选项的有效值包括：

- none：不启用任何物理最优化。
- place：布局后运行。这是默认方式。
- route：布线后运行。
- all：布局和布线后都运行。

```
-vivado_synth_design_args {args...}
```

默认值为 -directive sdx_optimization_effort_high。

该选项的值将传递到 export_design -evaluate Vivado 综合运行内的 synth_design。

```
-vivado_synth_strategy {default|<strategy>}
```

控制 export_design -evaluate Vivado 运行中使用的综合策略。

该选项的值应为 default 或有效的 Vivado 综合策略的名称。

```
-vivado_report_level
```

该选项可创建利用率报告和时序报告。默认模式设置为 0。

- 0：在综合后和布局布线后都创建利用率报告和时序报告。
- 1：在综合后和布局布线后都创建利用率报告、时序报告和分析报告。
- 2：在综合后和布局布线后都创建利用率报告、时序报告、分析报告和 Failfast 报告。

编译指示

没有等效的编译指示。

示例

config_interface

描述

指定用于在接口综合期间实现每个函数的 RTL 端口的默认接口选项。

语法

```
config_interface [OPTIONS]
```

选项

```
-clock_enable
```

向设计添加时钟使能端口 (ap_ce)。

时钟使能处于低电平有效状态时会阻止所有时钟操作。它会禁用所有后续操作

```
-expose_global
```

将全局变量作为 I/O 端口公开。

如果创建的变量为全局变量，但所有读写访问均为设计本地操作，那么会在设计中创建该资源。RTL 中无需 I/O 端口。



建议：如果您预计该全局变量为 RTL 块的外部源或目标，请使用该选项创建端口。

```
-m_axi_addr64
```

全局启用设计中的所有 M_AXI 端口的 64 位寻址。

```
-m_axi_offset (off|direct|slave)
```

全局控制设计中的所有 M_AXI 接口的偏移端口。

- off (默认)

不生成偏移端口。

- direct

生成标量输入偏移端口。

- slave

生成偏移端口并自动将其映射到 AXI4-Lite 从接口。

```
-register_io (off|scalar_in|scalar_out|scalar_all)
```

全局控制为顶层函数上的所有输入/输出开启寄存器的操作。默认设置为关 (off)。

```
-trim_dangling_port
```

基于结构体覆盖接口的默认行为。

默认情况下，块接口上已解包的结构体的所有成员都会成为 RTL 端口，与设计块是否使用这些成员无关。将此开关设置为“开 (on)”会移除生成的块不使用的所有接口端口。

编译指示

没有等效的编译指示。

示例

- 将全局变量作为 I/O 端口公开。
- 添加时钟使能端口。

```
config_interface -expose_global -clock_enable
```

config_rtl

描述

配置输出 RTL 的各属性、所使用的复位类型以及状态机的编码。它还允许您在 RTL 中使用特定标识。

默认情况下，这些选项适用于顶层设计以及设计中的所有 RTL 块。（可选）您可指定特定 RTL 模型。

语法

```
config_rtl [OPTIONS] <model_name>
```

选项

```
-header <string>
```

将文件 <string> 的内容置于所有输出 RTL 和仿真文件的顶部（作为注释）。



提示：该选项可用于确保输出 RTL 文件包含用户指定的标识。

```
-auto_prefix
```

指定顶层函数名称作为前缀值。如果同时指定 config_rtl -prefix，则忽略该选项。

```
-prefix <string>
```

指定要添加到所有 RTL 实体/模块名称的前缀。

```
-enable_maxiConservative
```

此模式可告知 AXI 主接口在写通道缓存中有足够数据后再发出写请求。

```
-reset (none|control|state|all)
```

C 语言代码中变量的初始化值与 RTL 中初始化的值始终相同，因此与比特流中初始化值也始终相同。此初始化在上电时执行，对设计应用复位时不会重复此操作。

通过 `-reset` 选项应用的置位操作用于判定寄存器和内存的复位方式。

- `none`

在设计中不添加复位。

- `control` (默认)

复位控制寄存器，例如，状态机中使用的寄存器和用于生成 I/O 协议信号的寄存器。

- `state`

复位控制寄存器和从 C 语言代码中的静态变量或全局变量衍生的寄存器或内存。C 语言代码中已初始化的任意静态变量或全局变量都复位为其初始值。

- `all`

复位设计中的所有寄存器和内存。C 语言代码中已初始化的任意静态变量或全局变量都复位为其初始值。

```
-reset_async
```

这导致所有寄存器都使用异步复位。

如果不指定该选项，则使用同步复位。

```
-reset_level (low|high)
```

允许复位信号的极性设置为低电平有效或高电平有效。

默认值为 `High`。

```
-encoding (binary|onehot|gray)
```

指定供设计的状态机使用的编码样式。

默认值为 `onehot`。

使用 `auto` 编码时，Vivado HLS 可判定编码样式。但 Vivado 中的赛灵思逻辑综合工具在逻辑综合期间可提取并重新实现 FSM 样式。如果选择任何其它编码样式，则赛灵思逻辑综合工具无法重新对编码样式进行最优优化。

编译指示

没有等效的编译指示。

示例

将输出 RTL 配置为以异步低电平有效复位对所有寄存器进行复位。

```
config_rtl -reset all -reset_async -reset_level low
```

将 `my_message.txt` 的内容作为注释添加到所有 RTL 输出文件中。

```
config_rtl -header my_message.txt
```

config_schedule

描述

配置 Vivado HLS 执行的调度的默认类型。

语法

```
config_schedule [OPTIONS]
```

选项

```
-effort (high|medium|low)
```

指定调度操作期间使用的工作量。

- 默认工作量为 Medium。
- Low 工作量最优化可改进运行时间，当设计实现选择较少时，此设置很有用。
- High 工作量最优化会增加时间，但通常可提供更好的结果。

```
-verbose
```

当调度失败时打印出关键路径以满足任意指令或约束。

```
-relax_ii_for_timing
```

该选项允许调度放宽流水线化循环或函数的 II，以满足时序要求。一般，调度创建的设计可能无法满足时序，这样即可使用逻辑综合来确保满足时序要求。该选项会指示调度始终满足时序并放宽吞吐量目标 (II) 以确保设计可满足其时序要求。

编译指示

没有等效的编译指示。

示例

将默认调度工作量更改为 Low 可缩短运行时间。

```
config_schedule -effort low
```

config_sdx

描述

以 HLS 或 XOCC 模式运行工具的编译器。

语法

```
config_sdx [OPTIONS]
```

选项

```
-target (none|xocc|vitis)
```

- none

以 HLS 单机模式运行工具。

- xocc

以 XOCC 模式运行工具，这将启用特定于 XOCC 的检查。

- vitis

在 Vitis 核开发套件中运行工具。

```
-optimization_level (none|0|1|2|3)
```

控制 HLS Vivado OOC 运行时间和工作量。该值越高，所需工作量越大。

```
-profile (true|false)
```

启用和禁用对生成的 HLS IP 或 XO 进行剖析的功能。

config_unroll

描述

基于循环索引限制（或行程计数）自动展开循环。

语法

```
config_unroll -tripcount_threshold <value>
```

选项

```
-tripcount_threshold
```

所含迭代数少于指定值的所有循环都将自动展开。

示例

以下命令可确保所含迭代数少于 18 的所有循环在调度期间都自动展开。

```
config_unroll -tripcount_threshold 18
```

cosim_design

描述

利用原先基于 C 的测试激励文件来为已综合的 RTL 执行综合后协同仿真。

要指定测试激励文件的文件，请运行以下命令：

```
add_files -tb
```

仿真在处于活动状态的解决方案的 `sim/<HDL>` 子目录内运行。

- `<HDL>` 由 `-rtl` 选项指定。

对于要使用 `cosim_design` 来验证的设计：

- 此设计必须使用接口模式 `ap_ctrl_hs`。
- 每个输出端口都必须使用以下接口模式之一：
 - `ap_vld`
 - `ap_ovld`
 - `ap_hs`
 - `ap_memory`
 - `ap_fifo`
 - `ap_bus`

接口模式使用写有效信号来指定写入输出的时间。

语法

```
cosim_design [OPTIONS]
```

选项

```
-argv <string>
```

`<string>` 传递到主 C 语言函数。

指定行为测试激励文件的实参列表。

```
-compiled_library_dir <string>
```

指定使用第三方仿真器进行仿真期间已编译的库目录。`<string>` 是已编译的库目录的路径名称。

```
-coverage
```

启用使用 VCS 仿真器进行仿真期间的覆盖率功能。

```
-disable_deadlock_detection
```

禁用协同仿真中的死锁检测功能。

```
-ignore_init <integer>
```

禁用前 `<integer>` 个时钟周期的对比检查。

当已知 RTL 最初将以未知 ('hX) 值启动时，该选项很有用。

```
-ldflags <string>
```

指定传递到连接器用于协同仿真的选项。

该选项通常用于为 C 语言测试激励文件传递包含路径信息或库信息。

```
-O
```

启用 C 语言测试激励文件和 RTL 封装的编译最优化。

```
-reduce_diskspace
```

该选项可启用节省磁盘空间流程。它有助于减少仿真期间所使用的磁盘空间量，但可能导致运行时间和内存利用率增加。

```
-rtl (vhdl|verilog)
```

指定用于 C/RTL 协同仿真的 RTL。默认为 Verilog。您可使用 `-tool` 选项来选择 HDL 仿真器。默认值为 `xsim`。

```
-setup
```

创建在处于活动状态的解决方案的 `sim/<HDL>` 目录中创建的所有仿真文件。不执行仿真。

```
-tool (*auto*|vcs|modelsim|riviera|isim|xsim|ncsim|xcelium)
```

指定用于将 RTL 与 C 语言测试激励文件一起协同仿真的仿真器。

```
-trace_level (*none*|all|port)
```

判定执行的追踪文件输出的级别。

判定 C/RTL 协同仿真期间的波形追踪级别。选项 “all” 会导致将所有端口和信号波形都保存到追踪文件，选项 “port” 仅保存顶层端口的波形追踪。执行仿真时，追踪文件保存在当前解决方案的 `sim/<RTL>` 目录中。<RTL> 目录取决于配合 `-rtl` 选项使用的选项：verilog 或 vhdL。

默认值为 none。

未经最优化时，`cosim_design` 会尽快编译测试激励文件。

请尽可能启用最优化以提升运行时间性能，代价是牺牲编译时间。虽然生成的可执行结果可能运行速度显著提升，但运行时间改善的效果因设计而异。运行时间最优化可能需要大量内存（用于大量函数）。

```
-wave_debug
```

在生成的 RTL 中启用所有进程的可视化，就像在数据流和顺序进程中一样。仅当使用 Vivado 仿真器进行协同仿真时，才支持该选项。

编译指示

没有等效的编译指示。

示例

使用 Vivado 仿真器执行验证：

```
cosim_design
```

使用 VCS 仿真器验证 Verilog RTL，并启用波形追踪文件保存功能：

```
cosim_design -tool VCS -rtl verilog -coverage -trace_level all
```

使用 ModelSim 验证 VHDL RTL。值 5 和 1 会传递到测试激励文件函数，以供在 RTL 验证中使用：

```
cosim_design -tool modelsim -rtl vhdl -argv "5 1"
```

create_clock

描述

针对当前解决方案创建 1 个虚拟时钟。

此命令只能在处于活动状态的解决方案上下文中执行。时钟周期是用于驱动最优化的约束（在给定时钟周期内将尽可能多的可行运算链接起来）。

C 和 C++ 设计仅支持单个时钟。对于 SystemC 语言设计，您可创建多个已命名的时钟并使用 `set_directive_clock` 命令将其应用于不同 SC_MODULE。

语法

```
create_clock -period <number> [OPTIONS]
```

选项

```
-name <string>
```

指定时钟名称。

如果不提供名称，将使用默认名称。

```
-period <number>
```

指定时钟周期（以 ns 或 MHz 为单位）。

- 如果不指定单位，将使用 ns。
- 如果不指定周期，将使用默认周期 10 ns。

编译指示

没有等效的编译指示。

示例

指定时钟周期为 50 ns。

```
create_clock -period 50
```

使用默认周期 10 ns 来指定时钟。

```
create_clock
```

对于 SystemC 语言设计，可使用 `set_directive_clock` 创建并应用多个已命名的时钟。

```
create_clock -period 15 fast_clk
create_clock -period 60 slow_clk
```

指定以 MHz 为单位的时钟频率。

```
create_clock -period 100MHz
```

csim_design

描述

使用提供的 C 语言测试激励文件编译并运行综合前的 C 语言仿真。

要指定测试激励文件，请使用 `add_file -tb`。仿真工作目录位于活动解决方案内的 `csim` 中。

语法

```
csim_design [OPTIONS]
```

选项

```
-o
```

启用最优化编译。

默认情况下，以调试模式执行编译以启用调试。

```
-argv <string>
```

指定 C 语言测试激励文件的实参列表。

在 C 语言测试激励文件中将 `<string>` 传递到 `<main>` 函数。

```
-clean
```

启用纯净构建。

如果不选择此选项，`csim_design` 将进行增量编译。

```
-ldflags <string>
```

指定传递给连接器以进行 C 语言仿真的选项。

此选项通常用于传递 C 语言测试激励文件和设计的库信息。

```
-compiler (*gcc*)
```

此选项选择用于 C 语言仿真的编译器。默认的编译器是 gcc（对于 C++ 为 g++）。

```
-mflags <string>
```

指定传递给编译器以进行 C 语言仿真的选项。

此选项通常用于加快编译速度。

```
-setup
```

在活动解决方案的 `csim` 目录中创建 C 语言仿真二进制文件。不执行仿真。

编译指示

没有等效的编译指示。

示例

编译并运行 C 语言仿真：

```
csim_design
```

编译源代码设计和测试激励文件以生成仿真二进制文件。不执行二进制文件。要运行仿真，请执行活动解决方案的 `csim/build` 目录中的 `run.sh`：

```
csim_design -O -setup
```

csynth_design

描述

为处于活动状态的解决方案综合 Vivado HLS 数据库。

此命令只能在处于活动状态的解决方案上下文中执行。数据库中细化后的设计将根据已设置的任意约束进行调度并映射到 RTL 上。

语法

```
csynth_design
```

选项

此命令不含任何选项。

编译指示

没有等效的编译指示。

示例

在顶层设计上运行 Vivado HLS。

```
csynth_design
```

delete_project

描述

删除与工程关联的目录。

`delete_project` 命令用于检查对应的工程目录 `<project>`，以在删除该目录前确保它是有效的 Vivado HLS 工程。如果在当前工作目录中不存在任何 `<project>` 目录，那么此命令无效。

语法

```
delete_project <project>
```

- `<project>` 是工程名称。

选项

此命令不含任何选项。

编译指示

没有等效的编译指示。

示例

移除 `Project_1` 目录及其所有内容即可删除 `Project_1`。

```
delete_project Project_1
```

delete_solution

语法

```
delete_solution <solution>
```

- `<solution>` 表示要删除的解决方案。

描述

从处于活动状态的工程中移除解决方案，并从工程目录中删除 `<solution>` 子目录。

如果解决方案在工程目录中不存在，则此命令无效。

编译指示

没有等效的编译指示。

示例

从处于活动状态的工程目录中移除 `Solution_1` 子目录，以从该工程中删除 `Solution_1` 解决方案。

```
delete_solution Solution_1
```

export_design

描述

导出并封装 RTL 中已综合的设计作为 IP，以供下游工具使用。

受支持的 IP 格式为：

- Vivado IP 目录
- DCP 格式
- System Generator

已封装的设计位于处于活动状态的 `impl` 目录的以下某一子目录中：

- `ip`
- `sysgen`

语法

```
export_design [OPTIONS]
```

选项

```
-flow (syn|impl)
```

为使用 RTL 综合指定的 HDL 获取更准确的时序和利用率数据。`syn` 选项用于执行 RTL 综合，`impl` 选项用于执行 RTL 综合和实现（已综合的门电路的详细布局布线）。

在 Vivado HLS GUI 中，这些选项显示为复选框，并分别标记为“Vivado Synthesis”和“Vivado Synthesis, place and route stage”。

```
-format (sysgen|ip_catalog|syn_dcp)
```

指定 IP 封装格式。

受支持的格式为：

- `sysgen`

采用面向 Vivado Design Suite 的 System Generator for DSP（仅限赛灵思 7 系列器件）接受的格式

- `ip_catalog`

采用适合添加到 Vivado IP 目录的格式（赛灵思 7 系列器件的默认格式）

- `syn_dcp`

适用于 Vivado Design Suite 的已综合的检查点文件。如果使用该选项，则将自动执行 RTL 综合。

```
-rtl (verilog|vhdl)
```

选择执行 `-flow` 选项时使用的 HDL。如果不指定此项，那么默认语言为 Verilog。

```
-xo <path-to-output-xo>
```

指定 XO 文件的直接输出。

编译指示

没有等效的编译指示。

示例

导出 RTL 以供 System Generator 使用：

```
export_design -format sysgen
```

导出 IP 目录中的 RTL。评估 VHDL 以获取更好的时序和利用率数据（使用 Vivado 工具）：

```
export_design -flow syn -rtl vhdl -format ip_catalog
```

help

描述

- 使用时如果不含任何 `<cmd>` 实参，则列出所有 Vivado HLS Tcl 命令。
- 使用时如果包含 Vivado HLS Tcl 命令作为实参，则提供有关指定命令的信息。

对于合法的 Vivado HLS 命令，输入命令实参时支持使用 Tab 键自动补全。

语法

```
help [OPTIONS] <cmd>
```

- `<cmd>` 是要显示相关帮助的命令。

选项

此命令不含任何选项。

编译指示

没有等效的编译指示。

示例

显示所有命令和指令的帮助。

```
help
```

显示 `add_files` 命令的帮助。

```
help add_files
```

list_core

描述

列出当前加载的库中的所有核。

核为用于在输出 RTL（例如，加法器、乘法器和内存）中实现运算的组件。

经细化后，RTL 中的运算在内部数据库中显示为运算符。调度期间，运算符将从库映射到核，以实现 RTL 设计。可将多个运算符映射到共享相同 RTL 资源的同一个核实例。

`list_core` 命令支持通过使用相关选项来列出可用的运算符与核：

- “Operation”

显示库中哪些核可实现每一项运算。

- “Type”

按类型列出可用的核，例如，实现函数运算的核或者实现内存或存储器操作的核。

如果不指定任何选项，那么此命令会列出库中的所有核。



提示：将 `list_core` 命令提供的信息与 `set_directive_resource` 命令搭配使用即可在特定核上实现特定运算。

语法

```
list_core [OPTIONS]
```

选项

```
-operation (opers)
```

列出库中可实现指定运算的核。运算包括：

- `add` - 加法
- `sub` - 减法
- `mul` - 乘法
- `udiv` - 无符号除
- `urem` - 无符号取余（模数运算符）

- srem - 有符号取余（模数运算符）
- icmp - 整数比较
- shl - 左移
- lshr - 逻辑右移
- ashr - 算术右移
- mux - 多路复用器
- load - 内存读取
- store - 内存写入
- fiforead - FIFO 读取
- fifowrite - FIFO 写入
- fifonbread - 非阻塞 FIFO 读取
- fifonbwrite - 非阻塞 FIFO 写入

```
-type (functional_unit|storage|connector|adapter|ip_block)
```

仅列出指定类型的核。

- “Function Units”

实现标准 RTL 运算（例如，加法、乘法或比较）的核。

- “Storage”

实现存储元件（例如，寄存器或内存）的核。

- “Connectors”

用于在设计中实现连接的核，包括直接连接和存储元件的流传输。

- “Adapter”

实现接口的核，这些接口用于在生成 IP 时连接顶层设计。这些接口是在 IP 生成流程（赛灵思 EDK）中使用的 RTL 封装中实现的。

- “IP Blocks”

您添加的任意 IP 核。

编译指示

没有等效的编译指示。

示例

列出当前加载的库中可实现 add 运算的所有核。

```
list_core -operation add
```

列出库中所有可用的内存（存储）核。

```
list_core -type storage
```



提示：使用 `set_directive_resource` 命令通过以下任一可用内存来实现数组。

list_part

描述

- 如果已指定系列，则返回受支持的器件系列或该系列中受支持的器件。
- 如果未指定系列，则返回所有受支持的系列。



提示：要返回某个系列的器件，运行该命令时如果未指定任何系列，则请指定某一个受支持的系列。

语法

```
list_part [OPTIONS]
```

编译指示

没有等效的编译指示。

示例

返回所有受支持的系列。

```
list_part
```

返回所有受支持的 Virtex®-6 器件。

```
list_part virtex6
```

open_project

描述

打开现有工程或者创建新工程。

在 Vivado HLS 会话中，任意给定时间仅限 1 个工程处于活动状态。任一工程均可包含多个解决方案。

要关闭工程，请运行以下操作：

- 使用 `close_project` 命令，或者
- 使用 `open_project` 命令启动另一个工程。

使用 `delete_project` 命令可完整删除工程目录（将其从硬盘中移除）以及与之关联的所有解决方案。

语法

```
open_project [OPTIONS] <project>
```

- <project> 是工程名称。

选项

```
-reset
```

- 移除已存在的所有工程数据即可将工程复位。
- 移除设计源文件上的所有原有工程信息、头文件搜索路径和顶层函数。关联的解决方案目录和文件都予以保留，但可能包含无效的结果。

`delete_project` 命令的作用与 `-reset` 选项相同，但移除所有解决方案数据。



建议：使用 Tcl 脚本执行 Vivado HLS 时请使用该选项。否则，每条新的 `add_files` 命令都会向现有数据添加更多文件。

编译指示

没有等效的编译指示。

示例

打开名为 `Project_1` 的新工程或现有工程。

```
open_project Project_1
```

打开工程，并移除所有现有数据。

```
open_project -reset Project_2
```



建议：将此命令与 Tcl 脚本搭配使用可防止将源文件或库文件添加到现有工程数据中。

open_solution

描述

在当前活动的工程内打开现有解决方案或创建新解决方案。



注意！没有任何工程处于活动状态的前提下尝试打开或创建解决方案将导致出错。在 Vivado HLS 会话中，任意给定时间仅限存在 1 个处于活动状态的解决方案。

在当前工程目录中，每个解决方案都对应 1 个子目录。如果当前工作目录中不存在解决方案，则会创建新解决方案。

要关闭解决方案，请运行以下操作：

- 运行 `close_solution` 命令，或者
- 使用 `open_solution` 命令打开另一个解决方案。

使用 `delete_solution` 命令从工程中移除解决方案并删除对应的子目录。

语法

```
open_solution [OPTIONS] <solution>
```

- <solution> 为解决方案名称。

选项

```
-reset
```

- 如果解决方案已存在，则复位解决方案数据。这样将删除有关库、约束和指令的所有先前的解决方案信息。
- 移除综合、验证和实现。

编译指示

没有等效的编译指示。

示例

在名为 Solution_1 的活动工程内打开新解决方案或现有解决方案。

```
open_solution Solution_1
```

在活动工程内打开解决方案。移除所有现有数据。

```
open_solution -reset Solution_2
```



建议：将此命令与 Tcl 脚本配合使用可阻止向现有解决方案数据添加更多内容。

set_clock_uncertainty

描述

在 create_clock 定义的时钟周期上设置裕度。

裕度将从时钟周期中扣除以创建有效的时钟周期。如果未以 ns 为单位或者未以百分比格式来定义时钟不确定性，则默认为时钟周期的 12.5%。

Vivado HLS 会根据有效时钟周期对设计进行最优化，为下游工具提供裕度以便进行逻辑综合和布线。此命令只能在处于活动状态的解决方案上下文中执行。Vivado HLS 仍使用所有输出文件中指定的时钟周期进行验证和实现。

如果 SystemC 语言设计中以 create_clock 命令指定了多个已命名的时钟，那么可在每个已命名的时钟上通过指定已命名的时钟来指定不同的时钟不确定性。

语法

```
set_clock_uncertainty <uncertainty> <clock_list>
```

- <uncertainty> 是表示时钟周期中用作裕度的量的值（以 ns 为单位）。
- <clock_list> 是对其应用不确定性的时钟的列表。如果未提供任何时钟，则表示对所有时钟应用不确定性。

编译指示

没有等效的编译指示。

示例

在时钟上指定不确定性或裕度为 0.5 ns。这样可将 Vivado HLS 可使用的时钟周期有效缩短 0.5 ns。

```
set_clock_uncertainty 0.5
```

在此 SystemC 示例中，创建 2 个时钟域。在每个域上指定不同的时钟不确定性。

```
create_clock -period 15 fast_clk  
create_clock -period 60 slow_clk  
set_clock_uncertainty 0.5 fast_clock  
set_clock_uncertainty 1.5 slow_clock
```



提示：SystemC 语言设计支持多个时钟。使用 `set_directive_clock` 命令将时钟应用于相应的函数。

set_directive_allocation

描述

指定资源分配的实例限制。

定义并可限制用于实现特定函数或运算的 RTL 实例的数量。例如，如果 C 语言源代码的 `foo_sub` 函数有 4 个实例，则 `set_directive_allocation` 命令可以确保在最终 RTL 中 `foo_sub` 只有一个实例。全部 4 个实例均使用相同的 RTL 块来实现。

语法

```
set_directive_allocation [OPTIONS] <location> <instances>
```

- `<location>` 是位置字符串，格式为 `function[/label]`。
- `<instances>` 是函数或运算符。

该函数可以是原始 C 语言代码中未由 `set_directive_inline` 命令内联或未由 Vivado HLS 自动内联的任何函数。

运算符列表如下（前提是在 C 语言源代码中有此类运算的 1 个实例）：

- `add`：加法
- `sub`：减法
- `mul`：乘法
- `icmp`：整数比较
- `sdiv`：有符号除
- `udiv`：无符号除
- `srem`：有符号取余

- urem：无符号取余
- lshr：逻辑右移
- shl：左移

选项

```
-limit <integer>
```

设置要在 RTL 设计中使用的实例（类型由 `-type` 选项定义）数量上限。

```
-type [function|operation]
```

实例类型可以是 `function`（默认）或 `operation`。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS allocation \  
    instances=<Instance Name List> \  
    limit=<Integer Value> \  
    <operation, function>
```

示例

给定 `foo_top` 设计中有多个 `foo` 函数的实例，在 RTL 中将 `foo` 的实例数限制为 2。

```
set_directive_allocation -limit 2 -type function foo_top foo  
#pragma HLS allocation instances=foo limit=2 function
```

将 `My_func` 实现中使用的乘法器数限制为 1。此限制不适用于可能驻留在 `My_func` 的子函数中的任何乘法器。要限制任何子函数实现过程中使用的乘法器数量，请在子函数上指定分配指令，或将子函数内联到 `My_func` 函数中。

```
set_directive_allocation -limit 1 -type operation My_func mul  
#pragma HLS allocation instances=mul limit=1 operation
```

set_directive_array_map

描述

将较小的数组映射到较大的数组。

设计人员通常使用 `set_directive_array_map` 命令（含相同 `-instance` 目标）来将多个较小的数组映射到单个较大的数组。随后，此大型数组将定向到单一大型内存（RAM 或 FIFO）资源。

`-mode` 选项可用于判定新目标是否是由以下项串联而成：

- 元素（水平映射）或
- 位宽（垂直映射）

数组按发出 `set_directive_array_map` 命令的顺序进行串联，开始位置为：

- 水平映射中的目标元素 0
- 垂直映射中的位 0

语法

```
set_directive_array_map [OPTIONS] <location> <array>
```

<location> 是包含数组变量的位置（格式为 `function[/label]`），<variable> 是要映射到新目标数组实例的数组变量。

选项

```
-instance <string>
```

指定当前数组变量要映射到的新数组实例名称。

```
-mode (horizontal|vertical)
```

- 水平映射（默认）将数组串联起来以构成含更多元素的目标。
- 垂直映射将数组串联起来以构成含更长的字词的目标。

```
-offset <integer>
```



重要提示！ 仅限水平映射。

指定整数值，以指示当前映射操作的目标实例的绝对偏移量。例如：

- 数组变量的元素 0 映射到新目标的元素 <int>。
- 其它元素映射到新目标的 <int+1>、<int+2>...。

如果不指定值，Vivado HLS 会自动计算所需的偏移，以避免出现任何重叠。示例：从目标中下一个未使用的元素开始串联数组。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS array_map \
    variable=<variable> \
    instance=<instance> \
    <horizontal, vertical> \
    offset=<int>
```

示例

这些命令将 `foo` 函数中的数组 `A[10]` 和 `B[15]` 映射到单个新数组 `AB[25]`。

- 元素 `AB[0]` 将与 `A[0]` 相同。
- 元素 `AB[10]` 将与 `B[0]` 相同（因为未使用 `-offset` 选项）。

- 数组 AB[25] 的位宽将为 A[10] 或 B[15] 的位宽最大值。

```
set_directive_array_map -instance AB -mode horizontal foo A
set_directive_array_map -instance AB -mode horizontal foo B
#pragma HLS array_map variable=A instance=AB horizontal
#pragma HLS array_map variable=B instance=AB horizontal
```

将数组 C 和 D 串联到新数组 CD 中，新数组所含位数与数组 C 和 D 组合的位数相同。CD 中元素数量为 C 或 D 的最大值

```
set_directive_array_map -instance CD -mode vertical foo C
set_directive_array_map -instance CD -mode vertical foo D
#pragma HLS array_map variable=C instance=CD vertical
#pragma HLS array_map variable=D instance=CD vertical
```

set_directive_array_partition

描述

将数组分区为更小的数组或者独立元素。

这种分区方式可以：

- 生成包含多个小型内存或多个寄存器（而不是一个大型内存）的 RTL。
- 有效增加存储器读写端口数量。
- 可能改善设计吞吐量。
- 需要更多内存实例或寄存器。

语法

```
set_directive_array_partition [OPTIONS] <location> <array>
```

- <location> 是包含数组变量的位置（格式为 function[/label]）。
- <array> 是要分区的数组变量。

选项

```
-dim <integer>
```

注释：仅与多维数组相关。

指定要分区的数组的维度。

- 如果使用 0 值，则使用指定选项对所有维度进行分区。
- 设置任何其它值都仅对该值对应的维度进行分区。例如，如果使用的值为 1，则仅对第 1 个维度进行分区。

```
-factor <integer>
```

注释：仅与 block 或 cyclic 分区类型有关。

指定要创建的更小的数组数量。

```
-type (block|cyclic|complete)
```

- `block` 型分区会从原始数组的连续块创建更小组。这样可将数组有效分区为 `N` 个相等的块，其中 `N` 为 `-factor` 选项定义的整数。
- `cyclic` 型分区会通过交织来自原始数组的元素来创建更小的数组。例如，如果使用的 `-factor` 为 3：
 - 向第 1 个新数组分配元素 0。
 - 向第 2 个新数组分配元素 1。
 - 向第 3 个新数组分配元素 2。
 - 向第 4 个新数组分配元素 3。
- `complete` 分区可将数组分解为多个独立元素。对于一维数组，这对应于将内存解析为独立寄存器。对于多维数组，请指定每个维度的分区方式，或者使用 `-dim 0` 选项来对所有维度进行分区。

默认值为 `complete`。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS array_partition \
    variable=<variable> \
    <block, cyclic, complete> \
    factor=<int> \
    dim=<int>
```

示例

将 `foo` 函数中的 `AB[13]` 数组分区为 4 个数组。由于 4 不是 13 的整数因子，因此：

- 有 3 个数组各包含 3 个元素。
- 另 1 个数组包含 4 个元素 (`AB[9:12]`)。

```
set_directive_array_partition -type block -factor 4 foo AB
#pragma HLS array_partition variable=AB block factor=4
```

将 `foo` 函数中的 `AB[6][4]` 数组分区为 2 个数组，每个数组维度为 `[6][2]`。

```
set_directive_array_partition -type block -factor 2 -dim 2 foo AB
#pragma HLS array_partition variable=AB block factor=2 dim=2
```

将 `foo` 函数中的 `AB[4][10][6]` 的所有维度分区为独立元素。

```
set_directive_array_partition -type complete -dim 0 foo AB
#pragma HLS array_partition variable=AB complete dim=0
```

set_directive_array_reshape

描述

将数组分区与垂直数组映射相结合以创建所含元素更少但字宽更宽的新数组。

set_directive_array_reshape 命令可执行如下操作：

1. 将数组拆分为多个数组（方式与 set_directive_array_partition 相同）
2. 自动重新按垂直方式组合数组（根据 set_directive_array_map-type 垂直方式）以创建含字宽更宽的新数组。

语法

```
set_directive_array_reshape [OPTIONS] <location> <array>
```

- <location> 是包含数组变量的位置（格式为 function[/label]）。
- <array> 是要重塑的数组变量。

选项

```
-dim <integer>integer>
```

注释：仅与多维数组相关。

指定要重塑的数组的维度。

- 如果 value = 0，则使用指定选项对所有维度进行分区。
- 设置任何其它值都仅对该值对应的维度进行分区。例如，如果 value = 1，则仅对第 1 个维度进行分区。

```
-factor <integer>
```

注释：仅与 block 或 cyclic 类型的水塑有关。

指定要临时创建的更小的数组的数量。

```
-type (block|cyclic|complete)
```

- block 型水塑会从原始数组的连续块创建更小数组。这样即可将数组有效等分为 N 个块，其中 N 为 -factor 选项定义的整数，然后使用 word-width*N 将 N 个块整合到单一数组中。默认值为 complete。
- cyclic 型水塑会通过交织来自原始数组的元素来创建更小的数组。例如，如果使用 -factor 3，则将向第 1 个新数组分配元素 0，向第 3 个新数组分配元素 2，然后再向第 1 个新数组分配元素 3。最后一个数组是将新数组垂直串联（字词串联以创建更长的字词）成单个数组。
- complete 型水塑可将数组分解为临时独立元素，然后将其重新组合为含字宽更宽的单个数组。对于一维数组，这等同于创建一个超宽寄存器（如果原始数组为 N 个 M 位的元素，结果可生成含 N*M 位的寄存器）。

```
-object
```

注释：仅与容器数组相关。

对容器内的对象应用水塑。如果指定该选项，则将水塑对象的所有维度，但将保留容器的所有维度。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS array_reshape \
    variable=<variable> \
    <block, cyclic, complete> \
    factor=<int> \
    dim=<int>
```

示例

将函数 `foo` 中的 8 位数组 `AB[17]` 重塑为含 5 个元素的新的 32 位数组。

由于 4 不是 13 的整数因子，因此：

- `AB[17]` 位于第 5 个元素的下 8 位中。
- 第 5 个元素的其它位无需使用。

```
set_directive_array_reshape -type block -factor 4 foo AB
#pragma HLS array_reshape variable=AB block factor=4
```

将函数 `foo` 中的数组 `AB[6][4]` 分区到维度 `[6][2]` 的新数组中，其中维度 2 为 2 倍宽度。

```
set_directive_array_reshape -type block -factor 2 -dim 2 foo AB
#pragma HLS array_reshape variable=AB block factor=2 dim=2
```

将函数 `foo` 中的 8 位数组 `AB[4][2][2]` 重塑到新的单元数组（寄存器）中，其位宽为 $4 \times 2 \times 8 (=128)$ 。

```
set_directive_array_reshape -type complete -dim 0 foo AB
#pragma HLS array_reshape variable=AB complete dim=0
```

set_directive_clock

描述

将指定的时钟应用于指定函数。

C 和 C++ 设计仅支持单个时钟。由 `create_clock` 指定的时钟周期将应用于设计中的所有函数。

SystemC 语言设计支持多个时钟。可使用 `create_clock` 命令指定多个已命名的时钟，并可使用 `set_directive_clock` 命令将这些时钟应用于各 `SC_MODULE`。每个 `SC_MODULE` 均使用单个时钟来进行综合。

语法

```
set_directive_clock <location> <domain>
```

- `<location>` 表示应用已命名的时钟的函数。
- `<domain>` 表示由 `create_clock` 命令的 `-name` 选项指定的时钟名称。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS clock domain=<string>
```

示例

假定 SystemC 语言设计中：

- 顶层 `foo_top` 含时钟端口 `fast_clock` 和 `slow_clock`。
- 它在其函数内仅使用 `fast_clock`。
- 子块 `foo` 仅使用 `slow_clock`。

在此情况下，以下所示命令将执行下列操作：

- 创建这 2 个时钟。
- 将 `fast_clock` 应用于 `foo_top`。
- 将 `slow_clock` 应用于子块 `foo`。

```
create_clock -period 15 fast_clk
create_clock -period 60 slow_clk
set_directive_clock foo_top fast_clock
set_directive_clock foo slow_clock
#pragma HLS clock domain=fast_clock
#pragma HLS clock domain=slow_clock
```

注释： `create_clock` 不存在等效的编译指示。

set_directive_dataflow

描述

指定要对函数或循环执行的数据流最优化，用于提高 RTL 实现的并发。

在 C 语言描述中，所有操作均按顺序执行。如无限制资源的任何指令（如 `set_directive_allocation`），则 Vivado HLS 会尝试最大限度减小时延并提高并发。

数据依赖关系可对此加以限制。例如，访问数组的函数或循环必须先完成对数组的所有读写访问后才能完成操作。这样会阻止下一个使用该数据的函数或循环开始操作。

函数或循环中的操作可在前一个函数或循环尚未完成其所有操作时就开始操作。

指定数据流最优化时，Vivado HLS 可执行如下操作：

- 分析顺序函数或循环之间的数据流。
- 尝试创建通道（根据乒乓 RAM 或 FIFO）以允许使用者函数或循环在生产者函数或循环完成前即开始操作。

这使函数或循环可并行操作，从而：

- 减小时延
- 提升 RTL 设计的吞吐量

如果未指定启动时间间隔（任一函数或循环与下一个函数或循环的开始时间间隔的周期数），那么 Vivado HLS 会尝试最大限度减小启动时间间隔，且数据一旦可用即开始操作。

语法

```
set_directive_dataflow <location>
```

- <location> 是数据流最优化的执行位置（格式为 function[/label]）。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS dataflow
```

示例

指定数据流最优化在函数 foo 内执行。

```
set_directive_dataflow foo  
#pragma HLS dataflow
```

set_directive_data_pack

描述

把结构体的数据字段打包到字宽更宽的单一标量中。

在该结构体内部声明的所有数组都将全部分区并重构为单一大宽度标量，与其它标量字段封装在一起。

可根据结构体字段的声明顺序推断出生成的新字宽标量的位对齐方式。第一个字段占用该字词的最低有效位，依此类推，直到映射完成所有字段为止。

注释：DATA_PACK 最优化不支持包含其它结构体的打包结构。

语法

```
set_directive_data_pack [OPTIONS] <location> <variable>
```

- <location> 是包含要打包的变量的位置（格式为 function[/label]）。
- <variable> 是要打包的变量。

选项

```
-instance <string>
```

指定打包后结果变量的名称。如果未提供，则使用 variable 输入。

```
-byte_pad (struct_level|field_level)
```

指定是否在 8 位边界上打包数据：

- `struct_level`: 首先打包该结构体，然后在 8 位边界上打包。
- `field_level`: 首先在 8 位边界上打包每个字段，然后打包该结构体。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS data_pack variable=<variable> instance=<string>
```

示例

将 `foo` 函数中具有 3 个 8 位字段（`typedef struct {unsigned char R, G, B;} 像素`）的结构体数组 `AB[17]` 打包为新的 24 位数组，其中含 17 个元素。

```
set_directive_data_pack foo AB
#pragma HLS data_pack variable=AB
```

将 `foo` 函数中具有 3 个 8 位字段（`typedef struct {unsigned char R, G, B;} 像素`）的结构体指针 `AB` 打包为新的 24 位指针。

```
set_directive_data_pack foo AB
#pragma HLS data_pack variable=AB
```

set_directive_dependence

描述

Vivado HLS 可监测如下依赖关系：

- 循环内部（独立于循环的依赖关系）；或者
- 循环的不同迭代之间（循环附带的依赖关系）。

这些依赖关系会影响运算的调度时间，尤其是在函数和循环流水线化期间。

- 独立于循环的依赖关系

在同一循环迭代内访问相同元素。

```
for (i=0;i<N;i++) {
    A[i]=x;
    y=A[i];
}
```

- 循环附带的依赖关系

在不同循环迭代内访问相同元素。

```
for (i=0;i<N;i++) {
    A[i]=A[i-1]*2;
}
```

在诸如建立因变量数组索引或需强制实施外部要求（例如，2 项输入从不采用相同索引）之类的情况下，依赖关系分析可能过于保守。`set_directive_dependence` 命令允许显式指定依赖关系并解析假性依赖关系。

语法

```
set_directive_dependence [OPTIONS] <location>
```

- <location> 表示指定依赖关系的位置（格式为 `function[/label]`）。

选项

```
-class (array|pointer)
```

指定其中依赖关系需澄清的变量的类。该选项与 `-variable` 选项互斥。

```
-dependent (true|false)
```

指定需强制实施依赖关系 (`true`) 还是移除依赖关系 (`false`)。默认值为 `true`。

```
-direction (RAW|WAR|WAW)
```

注释： 仅与循环附带的依赖关系有关。

指定依赖关系的方向：

- RAW（先写后读 - 真性依赖关系）
写指令使用的值供读指令使用。
- WAR（先读后写 - 反依赖关系）
读指令获取的值被写指令覆盖。
- WAW（先写后写 - 输出依赖关系）
2 条写指令按顺序写入相同位置。

```
-distance <integer>
```

注释： 仅与循环附带的依赖关系有关，其中 `-dependent` 设置为 `true`。

指定数组访问的迭代间距离。

```
-type (intra|inter)
```

指定依赖关系：

- 位于相同循环迭代内 (`intra`)，或者
- 位于不同循环迭代之间 (`inter`)（默认值）。

```
-variable <variable>
```

指定要考量用于依赖关系指令的特定变量。该选项与 `-class` 选项互斥。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS dependence \
    variable=<variable> \
    <array, pointer> \
    <inter, intra> \
    <RAW, WAR, WAW> \
    distance=<int> \
    <false, true>
```

编译指示中的所有选项均为必需选项。如果不指定 inter/intra 或 false/true，则默认行为为：

```
#pragma HLS DEPENDENCE variable=xxx inter false
```

示例

移除 foo 函数的 loop_1 的相同迭代中的 Var1 之间的依赖关系

```
set_directive_dependence -variable Var1 -type intra \
    -dependent false foo/loop_1
#pragma HLS dependence variable=Var1 intra false
```

foo 函数的 loop_2 中的所有数组的依赖关系将告知 Vivado HLS，在相同循环迭代内，所有读操作都必须发生在写操作之后。

```
set_directive_dependence -class array -type intra \
    -dependent true -direction RAW foo/loop_2
#pragma HLS dependence array inter RAW true
```

set_directive_expression_balance

描述

有时编写的基于 C 的规格包含一连串运算。这可能导致 RTL 中包含一长串运算。如果时钟周期较短，这就可能增加设计时延。

默认情况下，Vivado HLS 会通过结合律和交换律来重新组织运算。通过重新组织可以创建平衡的树结构以缩短运算链，从而以增加硬件为代价来缩短时延。

set_directive_expression_balance 命令支持在指定作用域内开关这种表达式平衡方法。

语法

```
set_directive_expression_balance [OPTIONS] <location>
```

- <location> 是应启用或禁用平衡的位置（格式为 function[/label]）。

选项

```
-off
```

在此位置关闭表达式平衡。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS expression_balance <off>
```

示例

在 My_Func 函数内禁用表达式平衡。

```
set_directive_expression_balance -off My_Func
#pragma HLS expression_balance off
```

在 My_Func2 函数内显式启用表达式平衡。

```
set_directive_expression_balance My_Func2
#pragma HLS expression_balance
```

set_directive_function_instantiate

描述

默认情况下：

- 函数在 RTL 中作为独立层级块保留。
- 同一层级上的任一函数的所有实例都使用相同 RTL 实现（块）。

`set_directive_function_instantiate` 命令用于为函数的每个实例创建唯一的 RTL 实现，允许对每个实例进行最优化。

默认情况下，以下代码可为全部 3 个实例的 `foo_sub` 函数生成单一 RTL 实现。

```
char foo_sub(char inval, char incr)
{
    return inval + incr;
}
void foo(char inval1, char inval2, char inval3,
        char *outval1, char *outval2, char * outval3)
{
    *outval1 = foo_sub(inval1, 1);
    *outval2 = foo_sub(inval2, 2);
    *outval3 = foo_sub(inval3, 3);
}
```

按如下示例部分所示使用指令可生成 3 个版本的 `foo_sub` 函数，每个版本都独立针对 `incr` 变量进行最优化。

语法

```
set_directive_function_instantiate <location> <variable>
```

- `<location>` 表示生成函数唯一实例的位置（格式为 `function[/label]`）。

- `variable <string>` 用于指定将哪个函数实参 `<string>` 指定为常量。

选项

此命令不含任何选项。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS function_instantiate variable=<variable>
```

示例

对于以上所示示例代码，以下 Tcl（或置于 `foo_sub` 函数中的编译指示）支持对 `foo_sub` 函数的每个实例进行与 `incr` 输入相关的独立最优化。

```
set_directive_function_instantiate foo_sub incr  
#pragma HLS function_instantiate variable=incr
```

set_directive_inline

描述

移除层级中作为独立实体的函数。内联后，函数将消隐，不再显示为层级的独立层次。

在某些情况下，将函数内联即可使函数内的运算以更有效的方式与周围运算共享和最优化。已内联的函数无法共享。这可能导致面积增加。

默认情况下，仅在函数层级的下一个层次上执行内联。

语法

```
set_directive_inline [OPTIONS] <location>
```

- `<location>` 是要执行内联的位置（格式为 `function[/label]`）。

选项

```
-off
```

禁用函数内联以防止特定函数发生内联。例如，如果在调用程序函数中使用 `-recursive` 选项，该选项会阻止调用的特定函数进行内联，同时所有其它函数均可进行内联。

```
-recursive
```

默认情况下，仅执行一级函数内联。指定函数内的函数不进行内联。`-recursive` 选项能够以递归方式沿层级向下内联所有函数。

```
-region
```

这样指定区域内的所有函数都将进行内联。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS inline <region | recursive | off>
```

示例

内联 `foo_top` 中的所有函数（不包括任何下级函数）。

```
set_directive_inline -region foo_top
#pragma HLS inline region
```

仅内联 `foo_sub1` 函数。

```
set_directive_inline foo_sub1
#pragma HLS inline
```

以递归方式沿层级向下内联 `foo_top` 中的所有函数，`foo_sub2` 函数除外。第一项编译指示置于 `foo_top` 函数内。第二项编译指示置于 `foo_sub2` 函数内。

```
set_directive_inline -region -recursive foo_top
set_directive_inline -off foo_sub2
#pragma HLS inline region recursive
#pragma HLS inline off
```

set_directive_interface

描述

指定在接口综合期间如何根据函数描述创建 RTL 端口。

RTL 实现中的端口衍生自：

- 指定的任意函数级协议。
- 函数实参
- 全局变量（供顶层函数访问，在其作用域范围外定义）

函数级握手：

- 控制函数何时开始执行运算。
- 指示函数运算何时：
 - 终止
 - 空闲
 - 准备好处理新输入

函数级协议的实现：

- 由 `ap_ctrl_none`、`ap_ctrl_hs` 或 `ap_ctrl_chain` 模式来控制。
- 只需顶层函数名称。

注释：指定用于编译指示的 `return` 函数。

每个函数实参均可指定为包含其自己的 I/O 协议（例如，有效握手或确认握手）。

如果访问全局变量，但所有读写操作均位于设计的本地操作，则在设计中创建该资源。RTL 中无需 I/O 端口。但如果预计全局变量为外部源或目标，请以与标准函数实参相似方式来指定其接口。请参阅以下示例。

在子函数上使用 `set_directive_interface` 时，只能使用 `-register` 选项。在子函数上不支持 `-mode` 选项。

语法

```
set_directive_interface [OPTIONS] <location> <port>
```

- `<location>` 表示要在其中指定函数接口或寄存器的输出的位置（格式为 `function[/label]`）。
- `<port>` 是必须为其综合接口的参数（函数实参或全局变量）。使用 `ap_ctrl_none` 模式或 `ap_ctrl_hs` 模式时，无需指定此项。

选项

`-bundle <string>`：将函数实参分组到各 AXI 端口中。默认情况下，Vivado HLS 将指定为 AXI4-Lite 接口的所有函数实参组合到单一 AXI4-Lite 端口内。同样，Vivado HLS 将指定为 AXI4 接口的所有函数实参都组合到单一 AXI4 端口内。`-bundle` 选项用于将含相同 `<string>` 的所有函数实参显式组合到同一接口端口内，并将 RTL 端口命名为 `<string>`。

```
-mode (ap_none|ap_stable|ap_vld|ap_ack|ap_hs|ap_ovld|ap_fifo| ap_bus|
ap_memory|bram|axis|s_axilite|m_axi|ap_ctrl_none|ap_ctrl_hs |ap_ctrl_chain)
```

以下是有关 Vivado HLS 如何实现 `-mode` 选项的摘要信息。如需了解详细描述，请参阅[接口综合参考](#)。

- `ap_none`：无协议。此接口为数据端口。
- `ap_stable`：无协议。此接口为数据端口。Vivado HLS 假定数据端口复位后始终处于稳定状态，这样即可支持内部最优化移除不必要的寄存器。
- `ap_vld`：用于实现含关联 `valid` 端口的数据端口，以指示何时数据有效且可供读取或写入。
- `ap_ack`：用于实现含关联 `acknowledge` 端口的数据端口，以确认数据已读取或写入。
- `ap_hs`：用于实现含关联 `valid` 端口和 `acknowledge` 端口的数据端口，提供两路握手以指示数据有效且可供读取和写入，并确认数据已读取或写入。
- `ap_ovld`：用于实现含关联 `valid` 端口的输出数据端口，以指示何时数据有效且可供读取或写入。

注释：Vivado HLS 通过 `ap_none` 模式来实现输入实参或任意读取/写入实参的输入部分。

- `ap_fifo`：使用含关联低电平有效 FIFO `empty` 端口和 `full` 端口的数据输入和输出端口来实现含标准 FIFO 接口的端口。

注释：仅限对读取实参和写入实参使用该接口。`ap_fifo` 模式不支持双向读写实参。

- `ap_bus`：用于实现指针和按引用传递端口（作为总线接口）。
- `ap_memory`：用于实现数组实参（作为标准 RAM 接口）。如果在 Vivado IP integrator 中使用 RTL 设计，内存接口会显示为离散端口。
- `bram`：用于实现数组实参（作为标准 RAM 接口）。如果在 Vivado IP integrator 中使用 RTL 设计，内存接口会显示为单端口。
- `axis`：用于实现所有端口（作为 AXI4-Stream 接口）。

- `s_axilite`：用于实现所有端口（作为 AXI4-Lite 接口）。Vivado HLS 会在“导出 RTL”流程期间生成一组关联的 C 驱动文件。
- `m_axi`：用于实现所有端口（作为 AXI4 接口）。您可使用 `config_interface` 命令来指定 32 位（默认）地址端口或 64 位地址端口，并控制任何地址偏移。
- `ap_ctrl_none`：无块级 I/O 协议。

注释：使用 `ap_ctrl_none` 模式可阻止使用 C/RTL 协同仿真功能验证设计。

- `ap_ctrl_hs`：实现块级控制端口以启动 (start) 设计操作，并指示设计何时处于 `idle`、`done` 和 `ready` 状态，以便处理新输入数据。

注释：`ap_ctrl_hs` 模式为默认块级 I/O 协议。

- `ap_ctrl_chain`：实现块级控制端口以启动 (start) 设计操作、`continue` 操作，以及指示设计何时处于 `idle`、`done` 和 `ready` 状态，以便处理新输入数据。

`-name <string>`：该选项用于根据您的规范对端口进行重命名。生成的 RTL 端口将使用此名称

`-depth`：指定供测试激励文件处理的最大采样数。此设置用于指示 Vivado HLS 为 RTL 协同仿真所创建的验证适配器中所需 FIFO 的最大大小。该选项对于使用 `ap_fifo` 模式或 `ap_bus` 模式的指针接口是必需的。

`-register`：寄存信号和任何相关协议信号，并指示信号保持直至至少完成函数执行的最后一个周期为止。该选项适用于顶层函数的以下标量接口：

- `ap_none`
- `ap_ack`
- `ap_vld`
- `ap_ovld`
- `ap_hs`
- `ap_fifo`

`-register_mode (both|forward|reverse|off)`：该选项用于指定寄存器布局到正向路径 (TDATA 和 TVALID)、反向路径 (TREADY) 或同时布局到这两条路径 (TDATA、TVALID 和 TREADY) 上，或者不寄存任何端口信号 (off)。默认值为 `both`。AXI-Stream 旁路信号被视为数据信号，随 TDATA 一起寄存。

`-offset <string>`：控制 AXI4-Lite 接口和 AXI4 接口中的地址偏移。在 AXI4-Lite 接口中，`<string>` 用于指定寄存器映射中的地址。在 AXI 接口中，`<string>` 用于指定以下值：

- `off`：不生成偏移端口。
- `direct`：生成标量输入偏移端口。
- `slave`：生成偏移端口并自动将其映射到 AXI4-Lite 从接口。

`-clock <string>`：默认情况下，AXI4-Lite 接口时钟与系统时钟为相同时钟。该选项用于为 AXI4-Lite 接口指定独立时钟。如果使用 `-bundle` 选项将多个顶层函数实参组合到单一 AXI4-Lite 接口中，那么只需在其中一个捆绑成员上指定时钟选项即可。

`-latency <value>`：该选项可用于 `ap_memory` 和 AXIM 接口。

- 在 `ap_memory` 接口中，接口选项用于指定驱动接口的 RAM 资源的读取时延。默认情况下，使用 1 个时钟周期的读取操作。该选项允许对读取时延超过 1 个时钟周期的外部 RAM 进行建模。
- 在 AXIM 接口中，该选项用于指定 AXI4 接口的期望时延，允许设计发起总线请求的时间比执行期望的读取或写入操作早 `<value>` 个周期（时延）。如果该值太低，设计将过早达成就绪状态，可能停滞并等待总线；如果该值太高，总线访问可能处于空闲状态并等待设计发起访问。

-max_read_burst_length：该选项适用于 AXIM 接口，可指定突发传输事务期间读取的数据值的最大数量。

-max_write_burst_length：该选项适用于 AXIM 接口，可指定突发传输事务期间写入的数据值的最大数量。

-num_read_outstanding：该选项适用于 AXIM 接口，用于指定在设计停滞前可对 AXI4 总线发起的读取请求数量（无响应）。此操作暗示设计中的内部存储空间，且 FIFO 大小为：

```
num_read_outstanding*max_read_burst_length*word_size.
```

-num_write_outstanding：该选项适用于 AXIM 接口，用于指定在设计停滞前可对 AXI4 总线发起的写入请求数量（无响应）。此操作暗示设计中的内部存储空间，且 FIFO 大小为：

```
num_read_outstanding*max_read_burst_length*word_size
```

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS interface <mode> register port=<string>
```

示例

为 foo 函数关闭函数级握手。

```
set_directive_interface -mode ap_ctrl_none foo
#pragma HLS interface ap_ctrl_none port=return
```

foo 函数中的 InData 实参指定为包含 ap_vld 接口，并且输入应寄存。

```
set_directive_interface -mode ap_vld -register foo InData
#pragma HLS interface ap_vld register port=InData
```

公开 foo 函数中使用的 lookup_table 全局函数，该选项在 RTL 设计上用作端口，并带有 ap_memory 接口。

```
set_directive_interface -mode ap_memory foo look_table
```

set_directive_latency

描述

在函数、循环或区域上指定最大和/或最小时延值。

Vivado HLS 始终以最小时延为目标。指定最小和最大时延值时，Vivado HLS 的行为如下所示：

- 时延小于最小值。
如果 Vivado HLS 可达成的时延小于指定的最小时延值，它会将时延延长至指定的值，从而可能增加共享。
- 时延大于最小值。
满足约束。不再执行进一步最优化。
- 时延小于最大值。

满足约束。不再执行进一步最优化。

- 时延大于最大值。

如果 Vivado HLS 无法在最大限制范围内完成调度，它会增加工作量以达成指定约束。如果仍无法满足最大时延，则会发出警告。随后，Vivado HLS 会以可实现的最小时延来生成设计。

语法

```
set_directive_latency [OPTIONS] <location>
```

- <location> 是要约束的函数、循环或区域的位置（格式为 `function[/label]`）。

选项

```
-max <integer>
```

指定最大时延。

```
-min <integer>
```

指定最小时延。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS latency \  
    min=<int> \  
    max=<int>
```

示例

指定 `foo` 函数采用最小时延值 4 和最大时延值 8。

```
set_directive_latency -min=4 -max=8 foo  
#pragma HLS latency min=4 max=8
```

在 `foo` 函数中，指定 `loop_row` 循环采用最大时延 12。将编译指示置于循环主体内。

```
set_directive_latency -max=12 foo/loop_row  
#pragma HLS latency max=12
```

set_directive_loop_flatten

描述

将嵌套循环扁平化为单一循环层级。

在 RTL 实现中，在循环层级内的循环之间移动需耗时 1 个时钟周期。将嵌套循环扁平化即可将其作为单一循环来加以最优化。这样可节省时钟周期，从而进一步对循环主体逻辑进行最优化。



建议：将该指令应用于循环层级中最内层的循环。仅限完美循环和半完美循环才能以此方式加以扁平化。

- 完美循环嵌套
 - 仅限最内层循环才包含循环主体内容。
 - 在循环语句之间不指定任何逻辑。
 - 所有循环边界均为常量。

- 半完美循环嵌套
 - 仅限最内层循环才包含循环主体内容。
 - 在循环语句之间不指定任何逻辑。
 - 最外层的循环边界可采用变量。

- 非完美循环嵌套

当内层循环具有变量边界或者循环主体未完全包含在内层循环内时，请尝试重构代码或者将循环主体中的循环展开以创建完美循环嵌套。

语法

```
set_directive_loop_flatten [OPTIONS] <location>
```

- <location> 表示最内层循环的位置，格式为 `function[/label]`。

选项

```
-off
```

防止执行扁平化。

可防止对某些循环进行扁平化，同时对指定位置的所有其它循环执行扁平化。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS loop_flatten off
```

示例

将 `foo` 函数中的 `loop_1` 及循环层级中位于其上层的所有（完美或半完美）循环扁平化为单一循环。将编译指示置于 `loop_1` 主体内。

```
set_directive_loop_flatten foo/loop_1
#pragma HLS loop_flatten
```

防止在 `foo` 函数的 `loop_2` 中进行循环扁平化。将编译指示置于 `loop_2` 主体内。

```
set_directive_loop_flatten -off foo/loop_2
#pragma HLS loop_flatten off
```

set_directive_loop_merge

描述

将所有循环合并到单一循环中。

合并循环：

- 减少 RTL 中循环主体实现间转换所需时钟周期数。
- 允许并行实现循环（如果可能）。

循环合并规则为：

- 如果循环边界为变量，则值（迭代数）必须相同。
- 如果循环边界为常量，那么最大常量值用作为合并循环的边界。
- 含变量边界和常量边界的循环不可合并。
- 要合并的循环之间的代码不得产生不同结果。多次执行此代码应生成相同结果。
 - 允许 $a=b$
 - 不允许 $a=a+1$
- 包含 FIFO 读取的循环无法合并。合并会更改读取顺序。从 FIFO 或 FIFO 接口执行读取必须始终按顺序进行。

语法

```
set_directive_loop_merge <location>
```

- `<location>` 是循环所在位置（格式为 `function[/label]`）。

选项

```
-force
```

即使 Vivado HLS 发出警告仍强制合并循环。您必须确保合并后的循环功能正常。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS loop_merge force
```

示例

将函数 `foo` 中的所有连续循环都合并到单一循环中。

```
set_directive_loop_merge foo
#pragma HLS loop_merge
```

函数 `foo` 的 `loop_2` 内的所有循环（不包含 `loop_2` 本身）均通过 `-force` 选项合并。将编译指示置于 `loop_2` 主体内。

```
set_directive_loop_merge -force foo/loop_2
#pragma HLS loop_merge force
```

set_directive_loop_tripcount

描述

“循环行程计数”是指循环执行的迭代总数。Vivado HLS 可报告每个循环的总时延（用于执行循环的所有迭代的周期数）。因此，此循环时延为行程计数（循环迭代次数）的函数。

行程计数可为常量值。它取决于循环表达式（例如，`x<y`）中使用的变量值或循环内使用的控制语句。

在某些情况下，Vivado HLS 无法判定行程计数。例如，如果用于判定行程计数的变量属于：

- 输入实参，或者
- 采用动态运算来计算的变量

则在此类情况下，循环时延可能未知。

为帮助执行设计分析来驱动最优化，`set_directive_loop_tripcount` 命令允许您为循环指定最小和最大行程计数。这样您即可通过报告了解循环时延在总设计时延中所占比重。

语法

```
set_directive_loop_tripcount [OPTIONS] <location>
```

- `<location>` 表示循环中指定行程计数的位置（格式为 `function[/label]`）。

选项

```
-avg <integer>
```

指定平均迭代次数。

```
-max <integer>
```

指定最大迭代次数。

```
-min <integer>
```

指定最小迭代次数。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS loop_tripcount \
    min=<int> \
    max=<int>
```


示例

针对 `foo` 函数中的 `loop_1` 指定：

- 最小行程计数为 12
- 最大行程计数为 16

```
set_directive_loop_tripcount -min 12 -max 16 -avg 14 foo/loop_1
#pragma HLS loop_tripcount min=12 max=16 avg=14
```

set_directive_occurrence

描述

当流水线化函数或循环时，指定某个位置的代码的执行速率低于其外层函数或循环中的代码的执行速率。

这使以较低速率执行的代码能以较低速率进行流水线化，并可能在顶层流水线中共享。例如：

- 循环迭代 N 次。
- 循环的一部分受条件语句保护，并且仅执行 M 次，其中 N 是 M 的整数倍。
- 假设受条件保护的代码将执行 N/M 次。

如果 N 以启动时间间隔 II 进行流水线化，则受条件语句保护的任意函数或循环：

- 可能以比 II 更高的启动时间间隔进行流水线化。

注释：

但执行的速度较慢。此代码不常用。

- 可以在外层较高速率的流水线中更好地共享。

标识某一区域，在此区域中流水线化的函数和循环的启动时间间隔可比外层函数或循环更长。

语法

```
set_directive_occurrence [OPTIONS] <location>
```

- `<location>` 指定执行速度较慢的位置。

选项

```
-cycle <int>
```

指定执行次数 N/M ，其中：

- N 是执行外层函数或循环的次数
- M 是执行条件区域的次数。

N 必须是 M 的整数倍。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS occurrence cycle=<int>
```

示例

foo 函数中的 Cond_Region 区域执行 4 次。它的执行速度比其外层的代码慢 4 倍。

```
set_directive_occurrence -cycle 4 foo/Cond_Region
#pragma HLS occurrence cycle=4
```

set_directive_pipeline

描述

指定以下详细信息：

- 函数流水线
- 循环流水线

流水线函数或循环每隔 N 个周期即可处理新输入，其中 N 为启动时间间隔 (II)。默认启动时间间隔为 1，即每个时钟周期均可处理一项新输入，或者可使用 -II 选项来指定该值。

如果 Vivado HLS 无法按指定的 II 创建设计，它将：

- 发出警告。
- 以可能的最低 II 创建设计。

随后，您可利用警告消息来分析此设计，以判定必须采取哪些步骤才能创建满足所需启动时间间隔的设计。

语法

```
set_directive_pipeline [OPTIONS] <location>
```

其中

- <location> 是要流水线化的位置（格式为 function[/label]）。

选项

```
-II <integer>
```

指定期望的流水线启动时间间隔。

Vivado HLS 会尝试满足此请求。根据数据依赖关系，实际结果的 II 可能更大。

```
-enable_flush
```

实现在以下情况下可清空的流水线：当在流水线输入端有效的数据变为不活动时清空。仅针对流水线函数才支持此功能：针对流水线循环不支持。

```
-rewind
```

注释：仅适用于循环。

支持回绕。回绕支持持续性循环流水线操作，但在循环迭代结尾与下一次迭代开始之间没有暂停。

仅当顶层函数内只有一个循环（或完美循环嵌套）时，回绕才有效。循环前的代码段：

- 被视为初始化。
- 在流水线中仅执行一次。
- 不能包含任何条件运算 (if-else)。

```
-off
```

关闭特定循环或函数的流水线。当使用 `config_compile -pipeline_loops` 对循环进行全局流水线化时，可使用该选项。该选项会阻止对特定循环进行流水线化。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS pipeline \
    II=<int> \
    enable_flush \
```

示例

按启动时间间隔为 1 来对函数 `foo` 进行流水线化。

```
set_directive_pipeline foo
#pragma HLS pipeline
```

set_directive_reset

描述

用于为特定状态变量（全局或静态）添加或移除复位。

语法

```
set_directive_reset [OPTIONS] <location> <variable>
```

- `<location>` 表示定义变量的位置（格式为 `function[/label]`）。
- `<variable>` 表示对其应用指令的变量。

选项

```
-off
```

- 如果指定 `-off`，则针对指定变量不生成复位。
- 如果不指定 `-off`，则针对指定变量生成复位。

编译指示

将编译指示置于 C 语言源代码中，位于变量生命周期边界内。

```
#pragma HLS reset variable=a off
```

示例

为 `foo` 函数中的 `static int a` 变量添加复位，即使全局复位设置为 `none` 或 `control` 也是如此。

```
set_directive_reset foo a
#pragma HLS reset variable=a
```

从 `foo` 函数中的 `static int a` 变量移除复位，即使全局复位设置为 `state` 或 `all` 也是如此。

```
set_directive_reset -off foo a
#pragma HLS reset variable=a off
```

set_directive_resource

描述

指定用于在 RTL 中实现变量的资源（核）。可采用以下任意变量：

- 数组
- 算术运算
- 函数实参

Vivado HLS 使用硬件核在代码中实现这些运算。当库中有多个核均可实现运算时，您可通过 `set_directive_resource` 命令来指定要使用的核。要生成核的列表，请使用 `list_core` 命令。如果不指定任何资源，Vivado HLS 会判定要使用的资源。

要指定库中哪些内存元素用于实现数组，请使用 `set_directive_resource` 命令。例如，这样您即可控制数组是作为单端口 RAM 还是作为双端口 RAM 来实现。这种用法对于顶层函数接口上的数组尤为重要，因为与数组关联的内存可用于判定 RTL 中的端口。

您可使用 `-latency` 选项来指定核的时延。对于接口上的块 RAM，`-latency` 选项允许您对接口上的片外非标准 SRAM 进行建模，例如，用于支持时延为 2 或 3 的 SRAM。对于内部运算，`-latency` 选项允许使用多个流水线阶段来实现运算。这些附加的流水线阶段可帮助解决 RTL 综合期间的时序问题。



重要提示！ 要使用 `-latency` 选项，运算必须具有 1 个可用的多阶段核。Vivado HLS 可为所有基本算术运算（加减乘除）、所有浮点运算和所有块 RAM 提供 1 个多阶段核。



建议：为实现最佳结果，赛灵思建议使用 `-std=c99`（针对 C）和 `-fno-builtin`（针对 C 和 C++）。要指定 C 编译选项（例如，`-std=c99`），请使用 Tcl 命令 `add_files` 搭配 `-cflags` 选项。或者，使用“工程设置 (Project Settings)”对话框中的“Edit CFLAGS”按钮。

语法

```
set_directive_resource -core <string> <location> <variable>
```

- <location> 是变量所在位置（格式为 function[/label]）。
- <variable> 变量。

选项

```
-core <string>
```

按技术库中的定义指定核。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS resource \
    variable=<variable> \
    core=<core> \
    latency=<latency>
```

示例

变量 `coeffs[128]` 是顶层函数 `foo_top` 的实参。该指令可指定使用库中的 `RAM_1P` 核来实现 `coeffs`。为访问 `coeffs` 的值而在 RTL 中创建的端口即 `RAM_1P` 核中定义的端口。

```
set_directive_resource -core RAM_1P foo_top coeffs
#pragma HLS resource variable=coeffs core=RAM_1P
```

根据函数 `foo` 中的代码 `Result=A*B`，指定使用双阶段流水线化乘法器核来实现乘法。

```
set_directive_resource -latency 2 foo Result
#pragma HLS RESOURCE variable=Result latency=2
```

要使用 `URAM` 来实现内存，请执行以下操作：

```
#pragma HLS RESOURCE variable=array core=RAM_1P_URAM uram
```

set_directive_stable

描述

`stable` 编译指示用于指示在数据流区域的入口和出口处生成同步时，可忽略某个变量（数据流区域的输入或输出）。

语法

```
set_directive_stable <location> <variable>
```

- <location> 是要在其中约束指令的函数名称或循环名称。
- <variable> 是要约束的数组的名称。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS stable variable=A
```

示例

在以下示例中，不使用 `stable` 编译指示的情况下，`proc1` 与 `proc2` 将同步，以确认读取其输入（包括 A）。使用 `stable` 编译指示后，A 将不再被视为需同步的输入。

```
void dataflow_region(int A[...], int B[...] ...  
#pragma HLS stable variable=A  
#pragma HLS dataflow  
    proc1(...);  
    proc2(A, ...);
```

set_directive_stream

描述

默认情况下，数组变量作为 RAM 来实现：

- 顶层函数数组参数作为 RAM 接口端口来实现。
- 一般数组作为 RAM 来实现以便进行读写访问。
- 在数据流最优化过程中所涉及的子函数中，数组实参使用 RAM 乒乓缓存通道来实现。
- 基于循环的数据流最优化中所涉及的数组作为 RAM 乒乓缓存通道来实现。

如果按顺序来使用或生成数组中的数据，那么更有效的通信机制是使用流传输数据，其中使用 FIFO 代替 RAM。

当顶层函数实参的接口类型被指定为 `ap_fifo` 时，该数组会以流传输方式自动实现。



重要提示！ 要保留访问，可能需要使用 `volatile` 限定符来阻止编译器最优化（尤其是死码删除）。

语法

```
set_directive_stream [OPTIONS] <location> <variable>
```

- `<location>` 是包含数组变量的位置（格式为 `function[/label]`）。
- `<variable>` 是要作为 FIFO 来实现的数组变量。

选项

```
-depth <integer>
```

注释： 仅与数据流通道中的数组流传输相关。

默认情况下，RTL 中实现的 FIFO 的深度与 C 语言代码中指定的数组大小相同。该选项允许您修改 FIFO 的大小。

当在 DATAFLOW 区域内实现数组时，经常使用 `-depth` 选项来减小 FIFO 的大小。例如，在其中所有循环和函数均以 `II=1` 的速率处理数据的 DATAFLOW 区域中，无需大型 FIFO，因为在每个时钟周期内都会生成和使用数据。在此情况下，`-depth` 选项可用于将 FIFO 大小减小为 2 以便大幅减小 RTL 设计的面积。

通过使用 `config_dataflow` 命令搭配 `-depth` 选项即可为 DATAFLOW 区域内所有数组提供该功能。`-depth` 选项搭配 `set_directive_stream` 一起使用即可覆盖使用 `config_dataflow` 指定的默认设置。

```
-dim <int>
```

指定要执行流传输的数组的维度。默认为维度 1。对于一维数组，请将 `dim` 设置为 1。对于二维数组，请将 `dim` 设置为 2。

仅限针对设计内部位于 DATAFLOW 区域内的使用者与生产者模型之间的流传输指定维度。不能将其应用于位于设计接口处的流传输。

```
-off
```

注释：仅与数据流通道中的数组流传输相关。

`config_dataflow -default_channel fifo` 命令用于暗示对设计中的所有数组全局应用 `set_directive_stream`。该选项允许在特定数组上关闭流传输（默认还原为使用基于 RAM 乒乓缓存的通道）。

注释：如果选中 `-off` 选项，`-depth` 选项会设置乒乓的深度（块数）。深度应至少为 2。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS stream
    variable=<variable> \
    off \
    depth=<int>
```

示例

指定 `foo` 函数中要进行流传输的 `A[10]` 数组，并作为 FIFO 实现。

```
set_directive_stream foo A
#pragma HLS STREAM variable=A
```

`foo` 函数中名为 `loop_1` 的循环中的数组 `B` 设置为作为 FIFO（深度为 2）来进行流传输。在此情况下，请将编译指示置于 `loop_1` 内。

```
set_directive_stream -depth 12 foo/loop_1 B
#pragma HLS STREAM variable=B depth=12
```

数组 `C` 已禁用流传输。在此示例中，假定已由 `config_dataflow` 将其启用。

```
set_directive_stream -off foo C
#pragma HLS STREAM variable=C off
```

set_directive_top

描述

为函数添加名称，以便后续可供 `set_top` 命令使用。

此命令通常用于对 C++ 中的某个类的成员函数进行综合。



建议：指定当前处于活动状态的解决方案中的指令。将 `set_top` 命令与新名称搭配使用。

语法

```
set_directive_top [OPTIONS] <location>
```

- `<location>` 表示要重命名的函数。

选项

```
-name <string>
```

指定将供 `set_top` 命令使用的名称。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS top \  
    name=<string>
```

示例

`foo_long_name` 函数重命名为 `DESIGN_TOP`，随后将其指定为顶层函数。如果在代码中放置编译指示，则仍必须在 GUI 工程设置中指定的顶层发出 `set_top` 命令。

```
set_directive_top -name DESIGN_TOP foo_long_name  
#pragma HLS top name=DESIGN_TOP  
set_top DESIGN_TOP
```

set_directive_unroll

描述

通过创建循环主体的副本来变换循环。

循环的迭代执行次数由循环感应变量来指定。迭代次数也可能受到循环主体内的逻辑影响（例如，对任何循环出口变量进行中断或修改）。在 RTL 中，循环按表示循环主体的逻辑块来实现，此类逻辑块会执行相同次数的迭代。

`set_directive_unroll` 命令支持将循环完全展开。展开循环会在 RTL 中创建循环主体副本，副本数量与循环迭代次数相同，或者也可按系数 `N` 部分展开，从而创建循环主体的 `N` 份副本并对循环迭代次数进行相应的调整。

如果用于部分展开的系数 `N` 并非原始循环迭代计数的整数倍，那么必须在循环主体的每个展开片段后检查原始退出条件。

要完全展开循环，在编译时循环边界必须已知。对于部分展开则无需满足此条件。

语法

```
set_directive_unroll [OPTIONS] <location>
```

- <location> 表示要展开的循环的位置（格式为 `format function[/label]`）。

选项

```
-factor <integer>
```

指定非零整数，表示已请求部分展开。

循环主体将按此次数重复。迭代信息将进行相应的调整。

```
-region
```

展开任一循环内的所有循环，不展开封闭循环本身。

请考量以下示例：

- `loop_1` 循环包含多个与循环层级位于相同层次的循环（`loop_2` 循环和 `loop_3` 循环）。
- 指定的循环（如 `loop_1`）同样也对应代码中的区域或位置。
- 一段以 `{ }` 括号扩起的代码。
- 如果在 `<function>/loop_1` 位置指定展开指令，它将展开 `loop_1`。

`-region` 选项可指定该指令仅应用于封闭指定区域的循环。这将生成如下结果：

- `loop_1` 保持“收起”状态。
- 展开 `loop_1` 内的所有循环（`loop_2` 和 `loop_3`）。

```
-skip_exit_check
```

仅当指定系数时才有效（部分展开）。

- “Fixed bounds”
如果迭代计数为系数的倍数，则不执行出口条件检查。
如果迭代计数并非系数的整数倍，则该工具将执行以下操作：
 - 阻止展开。
 - 发出警告，称必须执行出口检查后才能继续。
- “Variable bounds”
移除出口条件检查。您必须确保：
 - 变量边界为系数的整数倍。
 - 实际上无需出口检查。

编译指示

将编译指示置于 C 语言源代码中所需位置的边界内。

```
#pragma HLS unroll \  
    skip_exit_check \  
    factor=<int> \  
    region
```

示例

展开 `foo` 函数内的 L1 循环。将编译指示置于 L1 循环的主体内。

```
set_directive_unroll foo/L1  
#pragma HLS unroll
```

在 `foo` 函数的 L2 循环上指定展开系数 4。移除出口检查。将编译指示置于 L2 循环的主体内。

```
set_directive_unroll -skip_exit_check -factor 4 foo/L2  
#pragma HLS unroll skip_exit_check factor=4
```

展开 `foo` 函数中的 L3 循环内的所有循环，但不展开 L3 循环本身。`-region` 选项用于指定将此位置视为封闭区域，而不是循环标签。

```
set_directive_unroll -region foo/L3  
#pragma HLS unroll region
```

set_part

描述

为当前解决方案设置目标器件。

此命令只能在处于活动状态的解决方案上下文中执行。

语法

```
set_part <device_specification>
```

- `<device_specification>` 是为 Vivado HLS 综合和实现设置目标器件的器件规格。
- `<device_family>` 表示器件系列名称，使用的是系列中的默认器件。
- `<device><package><speed_grade>` 表示目标器件名称，其中包括器件、封装和速度等级信息。

选项

此命令不含任何选项。

编译指示

没有等效的编译指示。

示例

可通过提供器件系列名称，将随 Vivado HLS 提供的 FPGA 库添加到当前解决方案中，如下所示。在此情况下，将使用 Vivado HLS FPGA 库中为此器件系列指定的默认器件、封装和速度等级。

```
set_part virtex7
```

借助随 Vivado HLS 提供的 FPGA 库，可选择指定含有封装和速度等级信息的特定器件。

```
set_part xc6vlx240tff1156-1
```

set_top

描述

定义要综合的顶层函数。

从该函数调用的任一函数都将成为设计的一部分。

语法

```
set_top <top>
```

· <top> 是要综合的函数。

选项

此命令不含任何选项。

编译指示

没有等效的编译指示。

示例

将顶层函数设置为 `foo_top`。

```
set_top foo_top
```

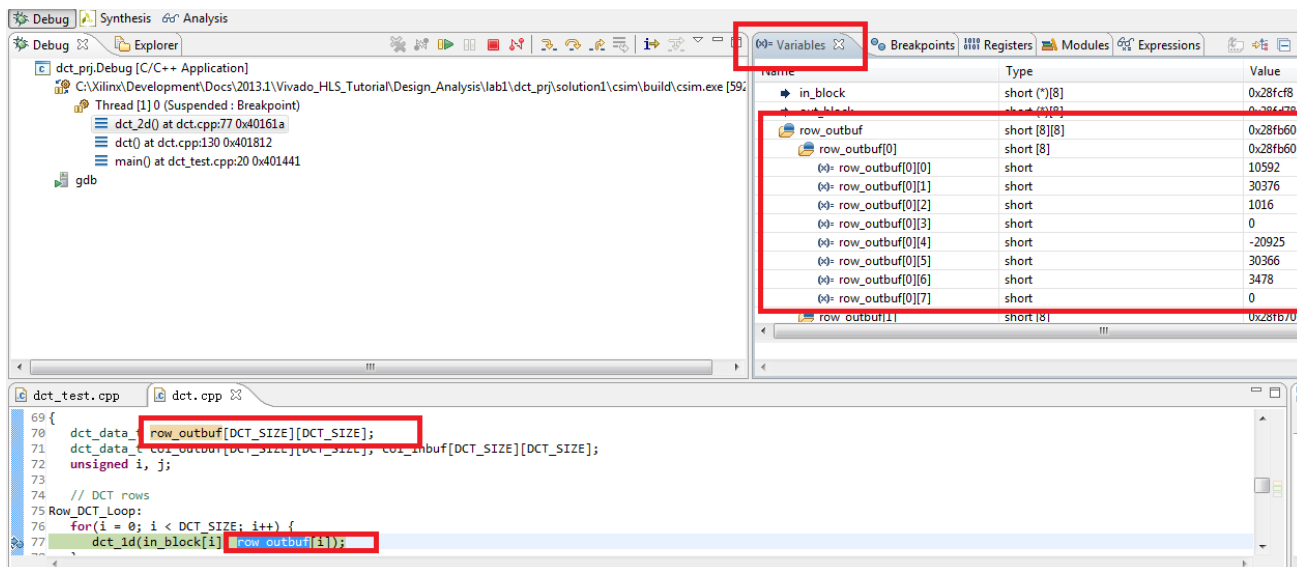
GUI 参考

本参考部分解释了如何使用、控制和自定义 Vivado HLS GUI。

变量监控

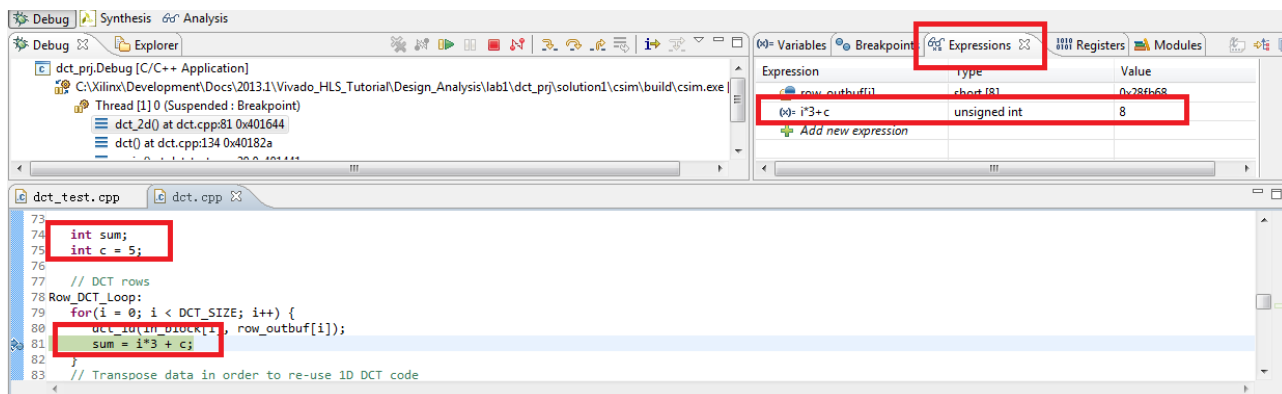
您可直接在“Debug”透视图中查看变量值和表达式。下图显示了监控各个变量值的方式。

图 94：变量监控



您可使用“Expressions”选项卡监控表达式的值。

图 95：表达式监控

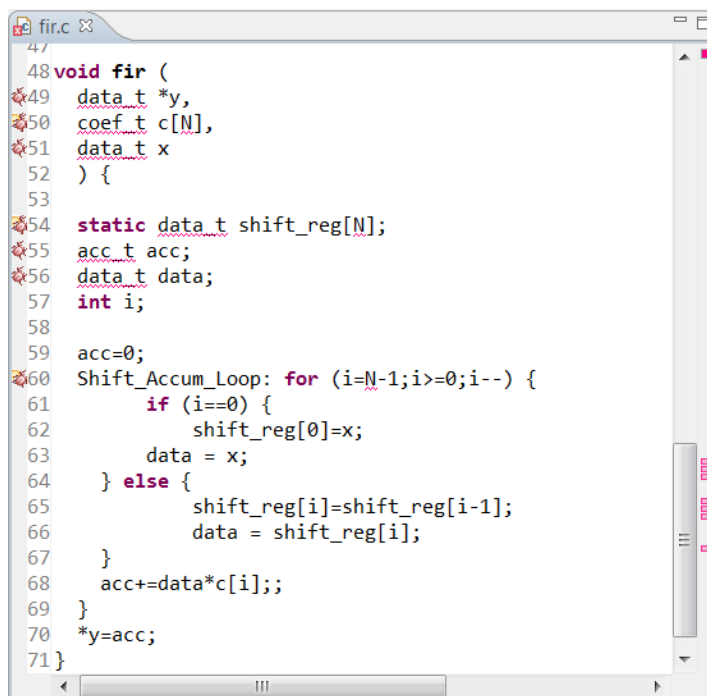


解析头文件信息

默认情况下，Vivado HLS GUI 会持续解析所有头文件，以解析编码引用。GUI 会高亮未解析的引用，如下图所示：

- 左侧边栏：高亮当前视图中未定义的引用。
- 右侧边栏：高亮整个文件中未解析的引用。

图 96：建立 C 语言文件索引



重要提示！ 在执行 C 语言仿真或综合前，移除代码中未定义的引用至关重要。要检查未定义的引用，请参阅代码查看器中的注释，其中会指明某个变量或值处于未知状态或者无法定义。未定义的引用不会显示在指令窗口中。

无法解析头文件（扩展名为 .h 或 .hpp）中定义的代码时，就会发生未定义的引用。未定义的引用的主要原因包括：

- 代码是最近添加到此文件中的。

如果此代码为新代码，请确保头文件已保存。保存头文件后，Vivado HLS 会自动为头文件建立索引，并更新编码引用。

- 头文件不在搜索路径中。

请使用 `include` 语句确保头文件包含在 C 语言代码中、头文件位置位于搜索路径中，并且头文件与添加到工程中的 C 语言文件位于相同目录中。

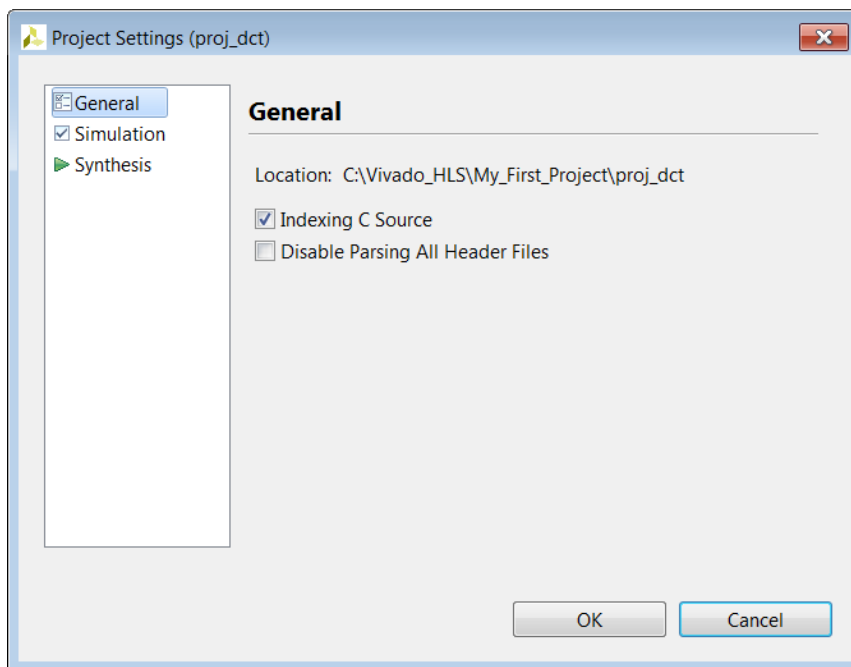
注释：要显式添加搜索路径，请选择“Solution”→“Solution Settings”，单击“Synthesis”或“Simulation”，然后使用“Edit CFLAGS”按钮。如需了解更多信息，请参阅[创建新的综合工程](#)。

- 自动索引已禁用。

请确保 Vivado HLS 自动解析所有头文件。选择“Project”→“Project Settings”以打开“Project Settings”对话框。单击“General”，确保未选中“Disable Parsing All Header Files”，如下图所示。这可能导致 GUI 响应时间缩短，因为 Vivado HLS 会使用 CPU 周期来自动检查头文件。

注释：要手动强制 Vivado HLS 对所有 C 语言文件建立索引，请单击“Index C files”工具栏按钮 .

图 97：控制头文件解析



解决源代码中的注释

在某些本地化文件中，源文件中的非英语注释显示为怪异的字符。可通过如下方式来纠正：

1. 在“Explorer”视图中选中工程。
2. 右键单击并使用“Properties” → “Resource”选择相应的语言编码。在题为“Text file encoding”的部分中，选择“Other”并从下拉菜单中选择相应的编码。

自定义 GUI 行为

在某些情况下，Vivado HLS GUI 的默认设置会阻止显示某些信息，或者默认设置不适合您使用。本节解释了如何自定义下列设置：

- “Console”视图缓存大小。
- 默认按键行为。

自定义“Console”视图

“Console”视图用于显示各项操作（例如，综合和验证）期间发出的消息。

此窗口的默认缓存大小为 80,000 个字符且可更改，或者也可移除此限制，以确保可查看所有消息，方法是使用菜单“Window” → “Preferences” → “Run/Debug” → “Console”。

自定义按键行为

可使用“Windows” → “Preferences”自定义 GUI 的行为，用户定义的新工具设置将予以保存。

按键组合 **Ctrl + Tab** 的默认设置是在源代码与头文件之间切换“信息窗格 (Information Pane)”中的活动选项卡。**Ctrl + Tab** 组合可更改为使每个选项卡依次成为活动选项卡。

- 在“首选项 (Preferences)”菜单中，可通过子菜单“General” → “Keys”选中“Command”列下的“Toggle Source/Header”，并使用“Unbind Command”键来移除 **Ctrl + Tab** 组合。
- 选中“Command”中的“Next Tab”，将光标置于“Binding”对话框中，并依次按下 **Ctrl** 键和 **Tab** 键，这将导致 **Ctrl + Tab** 操作与使下一个选项卡成为活动选项卡的操作关联。

可使用 Microsoft Visual Studio 方案来实现查找下一项的热键。可使用菜单“Window” → “Preference” → “General” → “Keys”，并将“Default”方案替换为“Microsoft Visual Studio”方案来完成此操作。

查看“Preferences”对话框下的子菜单即可对 GUI 环境的每方面进行自定义，以确保最大限度提升工作效率。

接口综合参考

此参考部分对每一种 Vivado HLS 接口模式进行了解释。

块级 I/O 协议

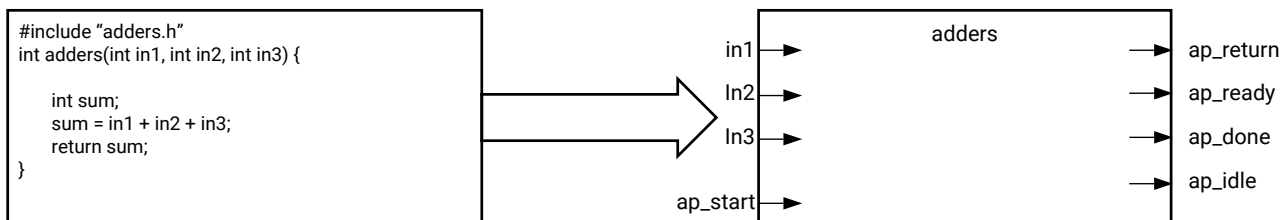
Vivado HLS 使用接口类型 `ap_ctrl_none`、`ap_ctrl_hs` 和 `ap_ctrl_chain` 来指定是否使用块级握手信号实现 RTL。块级握手信号可指定：

- 设计何时开始执行操作
- 操作何时终止
- 设计何时处于空闲状态以及何时准备好处理新输入

您可在函数上或函数返回时指定块级 I/O 协议。如果 C 语言代码不返回值，您仍可在函数返回时指定块级 I/O 协议。如果 C 语言代码使用函数返回，那么 Vivado HLS 会为返回值创建 `ap_return` 输出端口。

`ap_ctrl_hs` 块级 I/O 协议为默认协议。下图显示了 Vivado HLS 对函数实现 `ap_ctrl_hs` 时生成的 RTL 端口和行为。在此示例中，函数使用 `return` 语句返回值，Vivado HLS 在 RTL 设计中创建 `ap_return` 输出端口。如果在 C 语言代码中不包含函数 `return` 语句，则不会创建此端口。

图 98：ap_ctrl_hs 接口示例



X14267

`ap_ctrl_chain` 接口模式类似于 `ap_ctrl_hs`，但可提供额外的 `ap_continue` 输入信号以应用反压。赛灵思建议使用 `ap_ctrl_chain` 块级 I/O 协议将 Vivado HLS 块链接在一起。

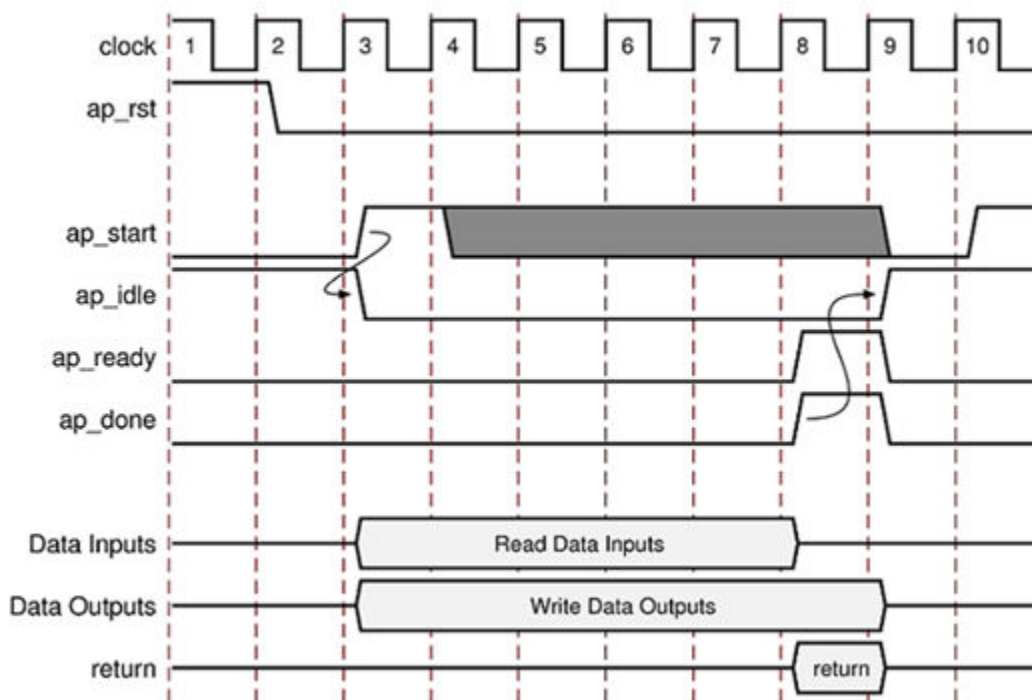
ap_ctrl_none

如果指定 `ap_ctrl_none` 块级 I/O 协议，则不创建块级 I/O 协议中所示的握手信号端口（`ap_start`、`ap_idle`、`ap_ready` 和 `ap_done`）。如果在设计上不指定块级 I/O 协议，那么使用 C/RTL 协同仿真来验证 RTL 设计时，必须遵守接口综合要求中所述条件。

ap_ctrl_hs

下图显示了由 `ap_ctrl_hs` I/O 协议为非流水线化设计创建的块级握手信号的行为。

图 99: `ap_ctrl_hs` 接口的行为



复位后，将执行以下操作：

1. 此块会等待 `ap_start` 达到高电平，然后再开始操作。
2. `ap_idle` 输出会立即变为低电平，以指示设计不再处于空闲状态。
3. `ap_start` 信号必须保持处于高电平状态，直至 `ap_ready` 达到高电平状态。当 `ap_ready` 达到高电平状态后：
 - 如果 `ap_start` 保持高电平，设计将启动下一项传输事务。
 - 如果 `ap_start` 变为低电平，设计将完成当前传输事务，然后停止操作。
4. 可读取输入端口上的数据。
注释：输入端口可使用独立于此块级 I/O 协议的端口级 I/O 协议。欲知详情，请参阅[端口级 I/O 协议](#)。
5. 可将数据写入输出端口。
注释：输出端口可使用独立于此块级 I/O 协议的端口级 I/O 协议。欲知详情，请参阅[端口级 I/O 协议](#)。
6. 当块完成操作后，`ap_done` 输出会变为高电平状态。

注释：如果存在 `ap_return` 端口，那么当 `ap_done` 处于高电平状态时，此端口上的数据将变为有效。因此，`ap_done` 信号还用于指示 `ap_return` 输出上的数据有效。

7. 当设计准备好接受新输入后，`ap_ready` 信号会变为高电平状态。以下是有关 `ap_ready` 信号的其它信息：
 - `ap_ready` 信号处于不活动状态，直至设计开始操作为止。
 - 在非流水线化设计中，`ap_ready` 信号与 `ap_done` 同时断言有效。
 - 在流水线化设计中，当 `ap_start` 采样结果为高电平后，`ap_ready` 信号可能在任意周期变为高电平状态。这取决于设计流水线化方式。
 - 如果 `ap_start` 信号为低电平状态，而 `ap_ready` 为高电平状态，那么设计将持续执行操作，直至 `ap_done` 变为高电平状态后停止操作。
 - 如果 `ap_start` 信号为高电平状态，且 `ap_ready` 为高电平状态，那么下一项传输事务将立即启动，且设计将继续操作。
8. `ap_idle` 信号可用于指示设计何时处于空闲且不执行操作状态。以下是有关 `ap_idle` 信号的其它信息：
 - 如果 `ap_start` 信号为低电平状态，而 `ap_ready` 为高电平状态，那么设计将停止操作，而 `ap_idle` 信号将在达成 `ap_done` 后，再经过 1 个周期后变为高电平状态。
 - 如果 `ap_start` 信号为高电平状态，且 `ap_ready` 为高电平状态，那么设计将继续操作，且 `ap_idle` 信号保持处于低电平状态。

ap_ctrl_chain

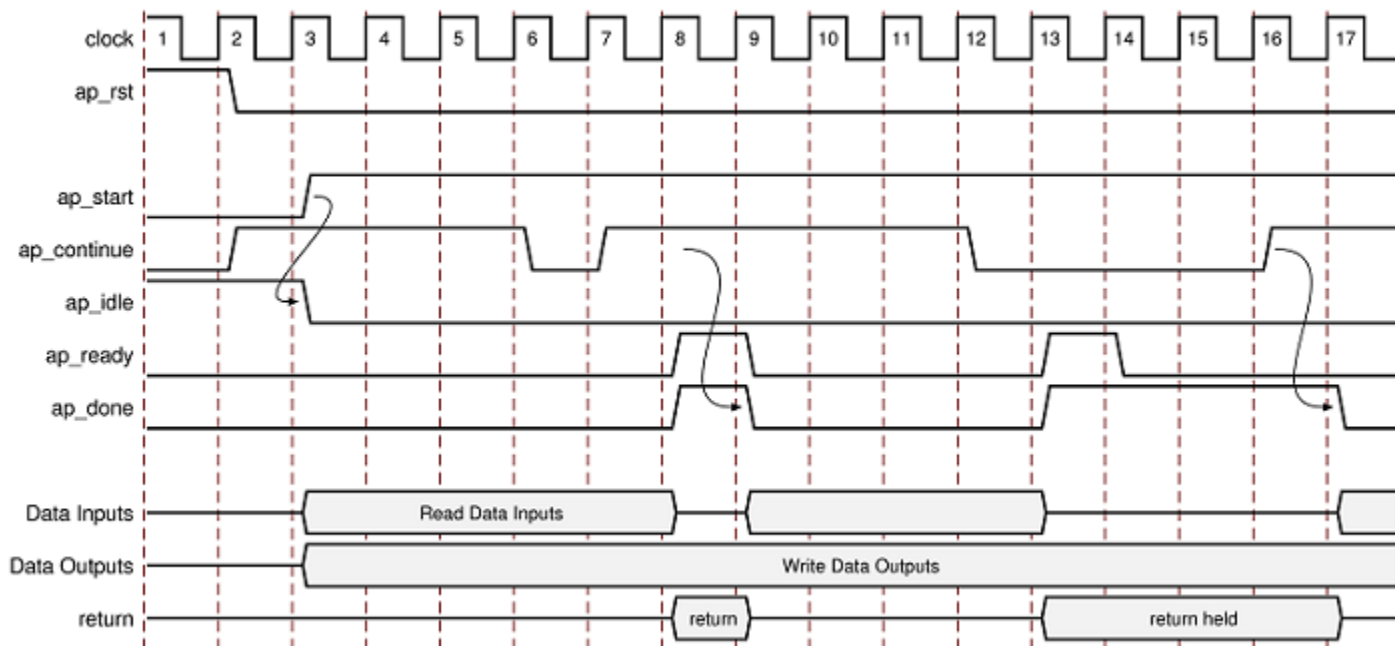
`ap_ctrl_chain` 块级 I/O 协议类似于 `ap_ctrl_hs` 协议，但可提供 1 个额外输入端口，名为 `ap_continue`。处于高电平有效状态的 `ap_continue` 信号可指示使用输出数据的下游块已准备好处理新的数据输入。如果下游块无法使用新数据输入，那么 `ap_continue` 信号处于低电平状态，这将阻止上游块生成更多数据。

下游块的 `ap_ready` 端口可直接驱动 `ap_continue` 端口。以下是有关 `ap_continue` 端口的其它信息：

- 如果 `ap_continue` 信号为高电平状态，且 `ap_done` 为高电平状态，那么设计将继续操作。其它块级 I/O 信号的行为与 `ap_ctrl_hs` 块级 I/O 协议中描述的行为相同。
- 如果 `ap_continue` 信号为低电平状态，而 `ap_done` 为高电平状态，那么设计将停止操作，`ap_done` 信号将保持高电平状态，并且如果存在 `ap_return` 端口，那么 `ap_return` 端口上的数据将保持有效。

在下图中，第 1 项传输事务已完成，第 2 项传输事务立即启动，因为 `ap_continue` 为高电平状态，且 `ap_done` 为高电平状态。但设计在第 2 项传输事务结束后将暂停，直至 `ap_continue` 断言为高电平有效为止。

图 100: ap_ctrl_chain 接口的行为



端口级 I/O 协议

ap_none

`ap_none` 端口级 I/O 协议是最简单的接口类型，没有与之关联的其它信号。输入和输出数据信号都没有关联的控制端口以指示何时读取或写入数据。RTL 设计中仅有的端口是源代码中指定的端口。

`ap_none` 接口无需额外硬件开销。但是，`ap_none` 接口需满足以下条件：

- 生产者块执行以下操作之一：
 - 在正确的时间向输入端口提供数据
 - 在执行传输事务期间保留数据，直到设计完成
- 使用者块在正确的时间读取输出端口

注释： `ap_none` 接口不能与数组实参一起使用。

ap_stable

与 `ap_none` 一样，`ap_stable` 端口级 I/O 协议不会向设计添加任何接口控制端口。`ap_stable` 类型通常用于可更改但在正常操作期间保持稳定的数据，例如提供配置数据的端口。`ap_stable` 类型向 Vivado HLS 发送下列通知：

- 应用于端口的数据在正常操作期间保持稳定，但不是可以最优化的常量值。
- 此端口的扇出无需寄存。

注释： `ap_stable` 类型只能应用于输入端口。当应用于输入输出 (inout) 端口时，仅端口的输入被视为稳定。

ap_hs (ap_ack、ap_vld 和 ap_ovld)

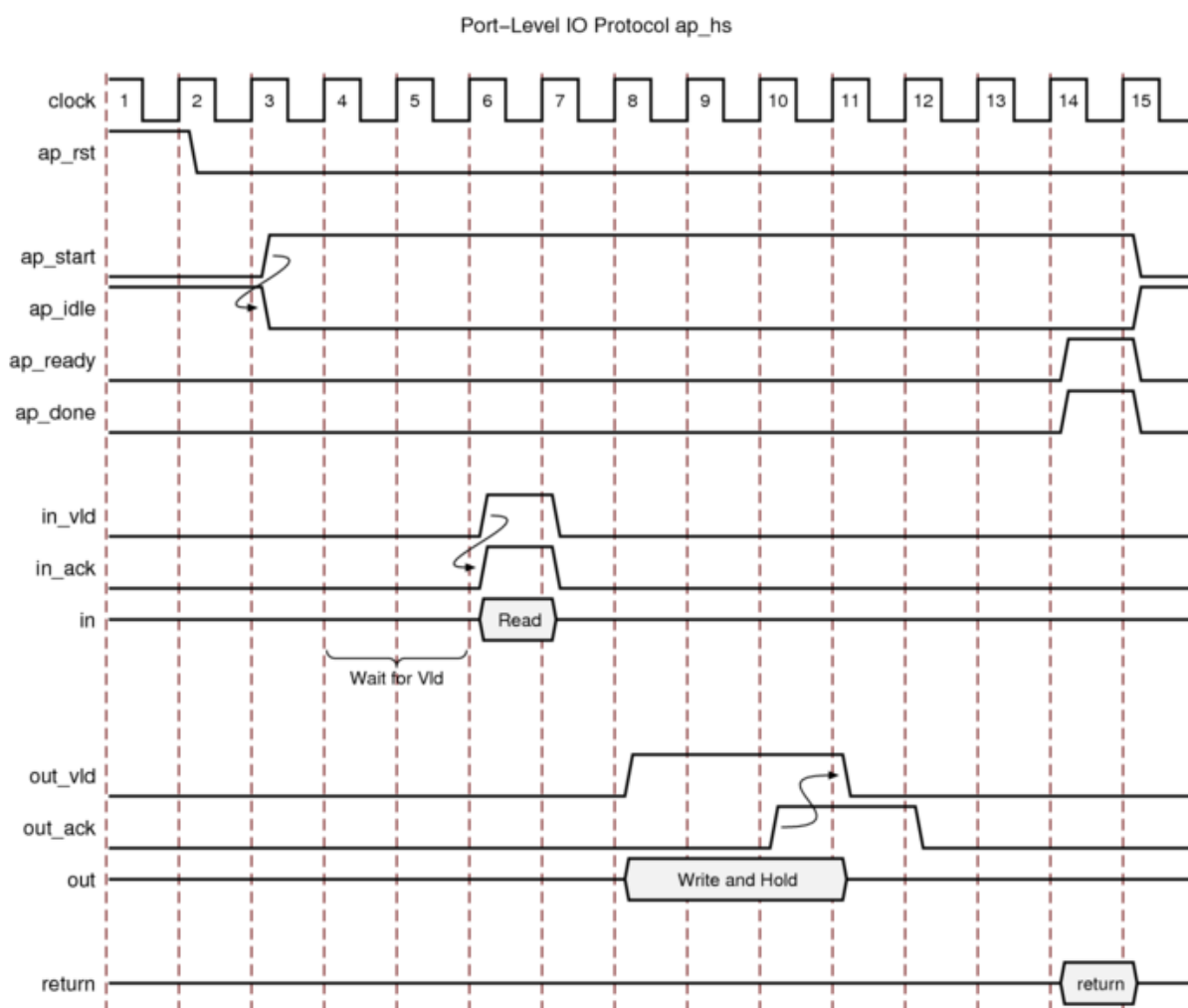
ap_hs 端口级 I/O 协议在开发过程中提供了最大的灵活性，允许采用自下而上和自上而下的设计流程。双向握手可安全执行所有块内通信，无需人为干预或假设即可正确执行。ap_hs 端口级 I/O 协议提供以下信号：

- 数据端口
- 用于指示何时使用数据的确认信号
- 用于指示何时读取数据的 valid 信号

下图显示了 ap_hs 接口对应输入和输出端口的行为。在此示例中，输入端口名为 in，输出端口名为 out。

注释：控制信号名称基于原始端口名称。例如，用于数据输入 in 的 valid 端口名为 in_vld。

图 101：ap_hs 接口的行为



对于输入，将执行以下操作：

- 应用 start 后，该块开始正常操作。

- 如果设计已准备好输入数据，但输入 `valid` 处于低电平状态，则设计将停滞并等待断言输入 `valid` 有效以指示存在新的输入值。

注释：上图显示了此行为。在此示例中，设计已准备好在时钟周期 4 上读取数据输入 `in`、停滞并等待输入 `valid`，然后再读取数据。

- 当输入 `valid` 断言为高电平有效 (High) 时，输出确认将断言为高电平有效 (High)，表明已读取数据。

对于输出，将执行以下操作：

- 应用 `start` 后，该块开始正常操作。
- 写入输出端口时，将同时断言其关联的输出 `valid` 信号有效，以指示端口上存在 `valid` 数据。
- 如果关联的输入确认为低电平有效 (Low)，则设计将停滞并等待断言输入确认有效。
- 当断言输入确认有效后，将在下一个时钟沿断言输出 `valid` 无效。

ap_ack

`ap_ack` 端口级 I/O 协议是 `ap_hs` 接口类型的子集。`ap_ack` 端口级 I/O 协议提供以下信号：

- 数据端口
- 用于指示何时使用数据的确认信号
 - 对于输入实参，设计会在读取输入的周期中生成高电平有效输出确认。
 - 对于输出实参，Vivado HLS 会实现输入确认端口以确认已读取输出。

注释：写入操作之后，设计将停滞并等待，直到输入确认断言为高电平有效为止，这表明使用者块已读取输出。但是，不存在可用于指示何时可以使用数据的关联输出端口。



注意！ 您不能使用 C/RTL 协同仿真来验证在输出端口上使用 `ap_ack` 的设计。

ap_vld

`ap_vld` 是 `ap_hs` 接口类型的子集。`ap_vld` 端口级 I/O 协议提供以下信号：

- 数据端口
- 用于指示何时读取数据的 `valid` 信号
 - 对于输入实参，设计在 `valid` 端口有效后立即读取数据端口。即使设计尚未准备好读取新数据，该设计也会对数据端口进行采样并在内部保留数据，直到需要时为止。
 - 对于输出实参，Vivado HLS 会实现输出 `valid` 端口以指示输出端口上的数据何时为 `valid`。

ap_ovld

`ap_ovld` 是 `ap_hs` 接口类型的子集。`ap_ovld` 端口级 I/O 协议提供以下信号：

- 数据端口
- 用于指示何时读取数据的 `valid` 信号
 - 对于输入实参和输入输出参数中的输入部分，设计默认为 `ap_none` 类型。
 - 对于输出实参和输入输出参数中的输出部分，设计实现 `ap_vld` 类型。

ap_memory, bram

ap_memory 和 bram 接口端口级 I/O 协议用于实现数组实参。当实现要求随机访问内存地址位置时，这种类型的端口级 I/O 协议可以与内存元件（例如，RAM 和 ROM）通信。

注释：如果只需顺序访问内存元件，请改用 ap_fifo 接口。ap_fifo 接口可以减少硬件开销，因为不执行地址生成。

ap_memory 和 bram 接口端口级 I/O 协议相同。唯一的区别是 Vivado IP integrator 显示块的方式：

- ap_memory 接口显示为离散端口。
- bram 接口显示为单一端口（已组合）。在 IP integrator 中，可使用单一连接来创建到所有端口的连接。

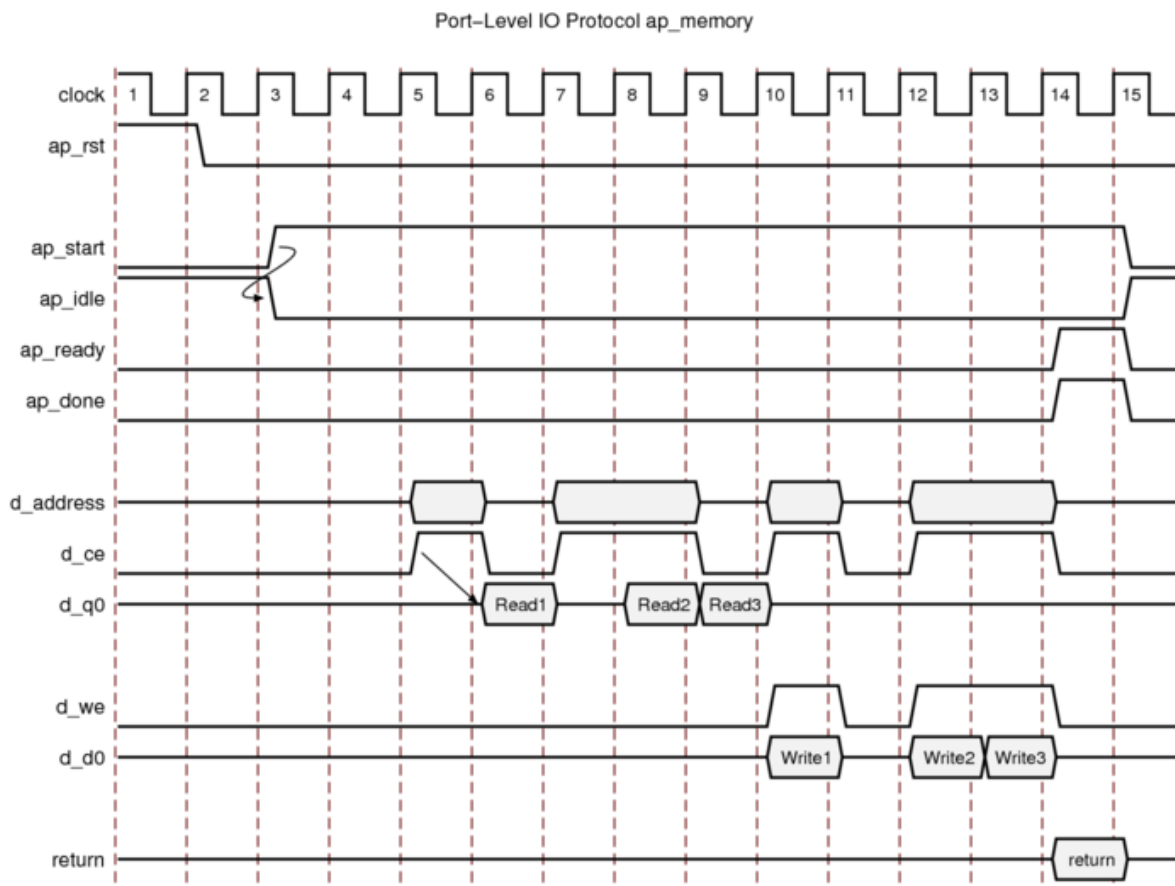
使用 ap_memory 接口时，请使用 RESOURCE 指令指定数组目标。如果没有为数组指定目标，则 Vivado HLS 会判定是使用单端口还是双端口 RAM 接口。



提示：运行综合前，请使用 RESOURCE 指令确保将数组实参定向到正确的内存类型。使用更正后的内存进行重新综合可能会导致调度和 RTL 不同。

下图显示了一个名为 d 的数组，该数组指定为单端口块 RAM。端口名称基于 C 语言函数实参。例如，如果 C 语言实参为 d，则根据 BRAM 的 output/q 端口，芯片使能为 d_ce，输入数据为 d_q0。

图 102：ap_memory 接口的行为



复位后，将执行以下操作：

- 应用 start 后，该块开始正常操作。
- 通过在断言输出信号 d_ce 有效的同时在输出地址端口上应用地址来执行读取。

注释：对于默认的块 RAM，设计期望在下一个时钟周期中，输入数据 d_q0 可用。您可以使用 RESOURCE 指令来指示 RAM 具有更长的读取时延。

- 通过断言输出端口 d_ce 和 d_we 有效并同时应用地址和输出数据 d_d0 来执行写操作。

ap_fifo

写入输出端口时，当设计需要访问内存元件并且访问始终以顺序方式执行时，即不需要随机访问，则与其关联的输出 valid 信号接口是最节省硬件的方法。ap_fifo 端口级 I/O 协议支持以下操作：

- 允许端口连接到 FIFO
- 启用完整的双向 empty-full 通信
- 适用于数组、指针和按引用传递实参类型

注释：可以使用 ap_fifo 接口的函数通常使用指针，并且可能多次访问同一变量。要了解使用这种编码样式时 volatile 限定符的重要性，请参阅[多重访问指针接口：数据流传输](#)。

在以下示例中，in1 是一个指针，该指针访问当前地址，然后访问当前地址上面的两个地址，最后访问下面的一个地址。

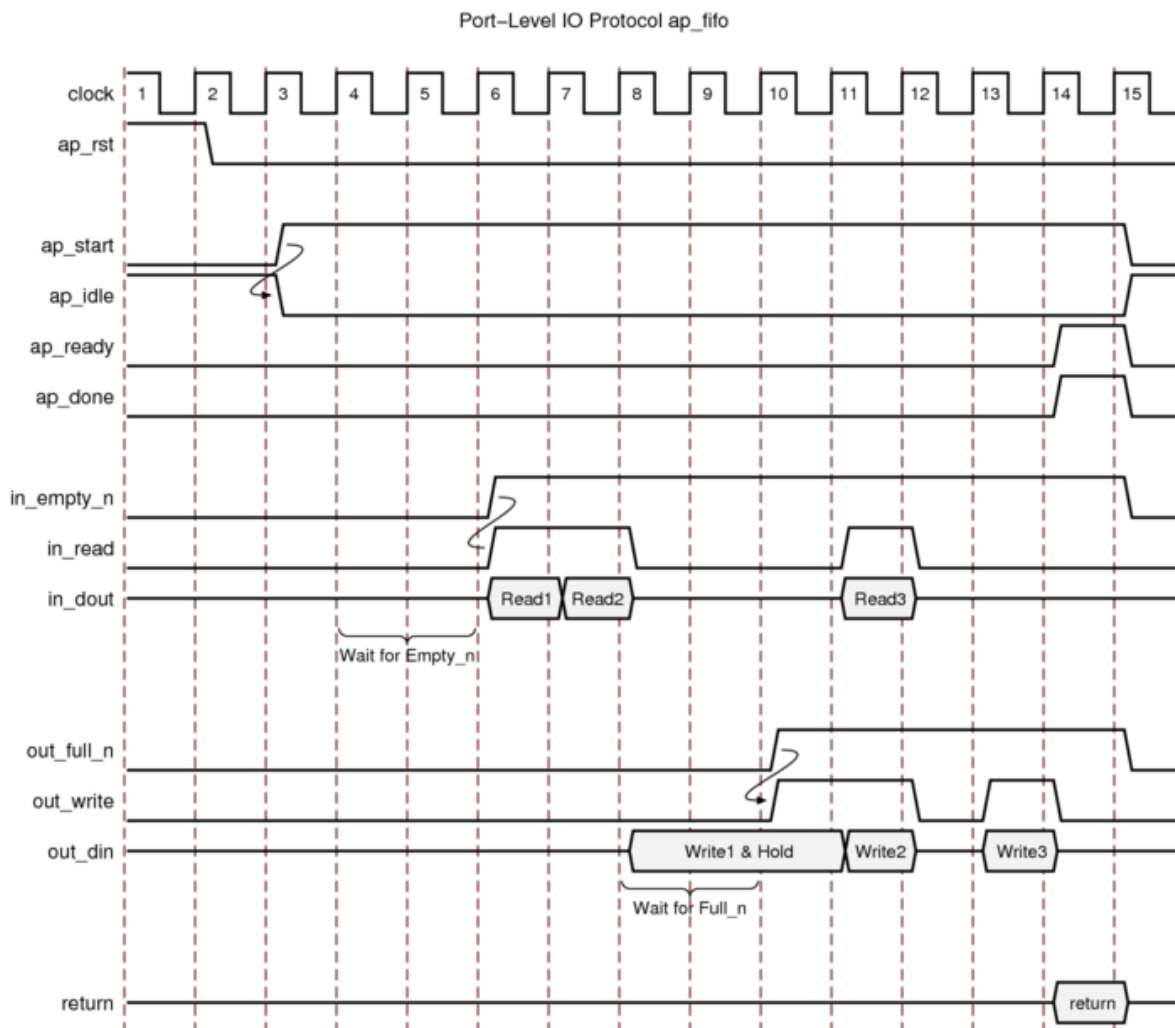
```
void foo(int* in1, ...) {
    int data1, data2, data3;

    ...
    data1= *in1;
    data2= *(in1+2);
    data3= *(in1-1);
    ...
}
```

如果将 in1 指定为 ap_fifo 接口，则 Vivado HLS 会检查访问、判定访问并非按顺序进行，随即发出错误消息并中止。要从非顺序地址读取，请使用 ap_memory 或 bram 接口。

您不能在同时支持读取和写入的实参上指定 ap_fifo 接口。您只能在输入或输出实参上指定 ap_fifo 接口。含输入实参 in 和输出实参 out（指定为 ap_fifo 接口）的设计的行为如下图所示。

图 103: ap_fifo 接口的行为



对于输入，将执行以下操作：

- 应用 start 后，该块开始正常操作。
- 如果输入端口已准备好读取，但输入端口 `in_empty_n` 处于低电平状态，指示 FIFO 为空，则设计将停滞并等待数据可用。
- 当输入端口 `in_empty_n` 处于高电平状态，指示 FIFO 包含数据时，将断言输出确认 `in_read` 为高电平有效以指示当前周期中已读取数据。

对于输出，将执行以下操作：

- 应用 start 后，该块开始正常操作。
- 如果输出端口已准备好写入，但 `out_full_n` 处于低电平状态，指示 FIFO 已满，则数据将置于输出端口上，但设计将停滞并等待 FIFO 中的空间可用。
- 当 `out_full_n` 处于高电平状态，指示 FIFO 中有可用空间时，将断言输出确认信号 `out_write` 有效以指示输出数据为 valid。

- 如果顶层函数或顶层循环已使用 `-rewind` 选项进行流水线化，则 Vivado HLS 将创建另一个后缀为 `_lwr` 的输出端口。当最后一次写入 FIFO 接口完成时，`_lwr` 端口将变为高电平有效。

ap_bus

`ap_bus` 接口可以与总线桥通信。由于 `ap_bus` 接口未遵循特定的总线标准，因此可将此接口用于与系统总线进行通信的总线桥。总线桥必须能够对所有突发写入进行高速缓存。

注释：可以使用 `ap_bus` 接口的函数使用指针，并且可能多次访问同一变量。要了解使用这种编码样式时 `volatile` 限定符的重要性，请参阅[多重访问指针接口：数据流传输](#)。

您可以通过以下方式使用 `ap_bus` 接口：

- 标准模式：此模式执行单独的读操作和写操作，并为每项操作指定地址。
- 突发模式：如果在 C 语言源代码中使用了 C 语言函数 `memcpy`，则此模式将执行数据传输。在突发模式下，该接口指示基址和传输大小。然后以连续的周期传输数据样本。

注释：`memcpy` 函数访问的数组不能分区到寄存器中。

以下示例显示了将 `ap_bus` 接口应用于实参 `d` 时，标准模式下的读写操作的行为。

```
void foo (int *d) {
    static int acc = 0;
    int i;

    for (i=0;i<4;i++) {
        acc += d[i+1];
        d[i] = acc;
    }
}
```

以下示例显示了使用 C `memcpy` 函数和突发模式时的行为。

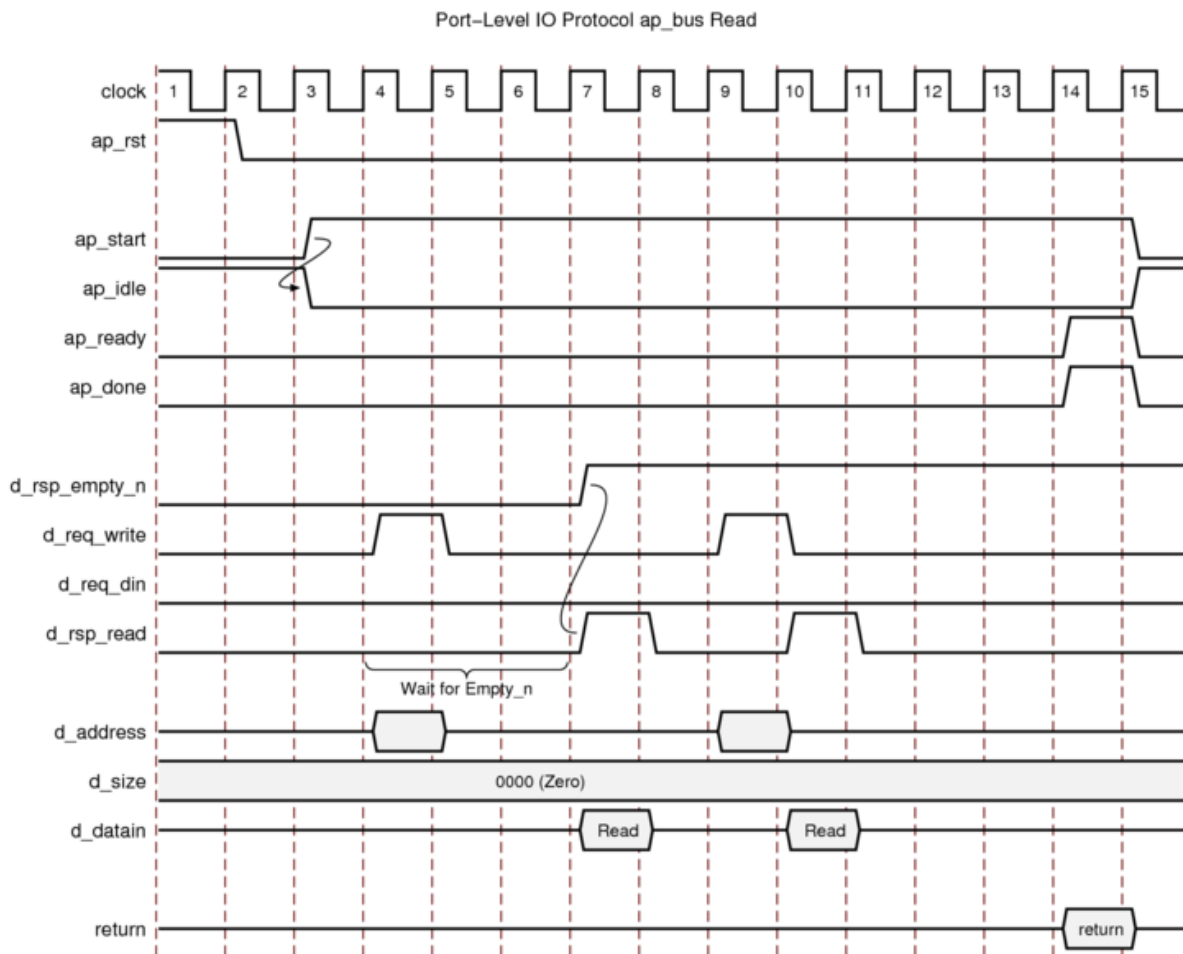
```
void bus (int *d) {
    int buf1[4], buf2[4];
    int i;

    memcpy(buf1,d,4*sizeof(int));

    for (i=0;i<4;i++) {
        buf2[i] = buf1[3-i];
    }

    memcpy(d,buf2,4*sizeof(int));
}
```

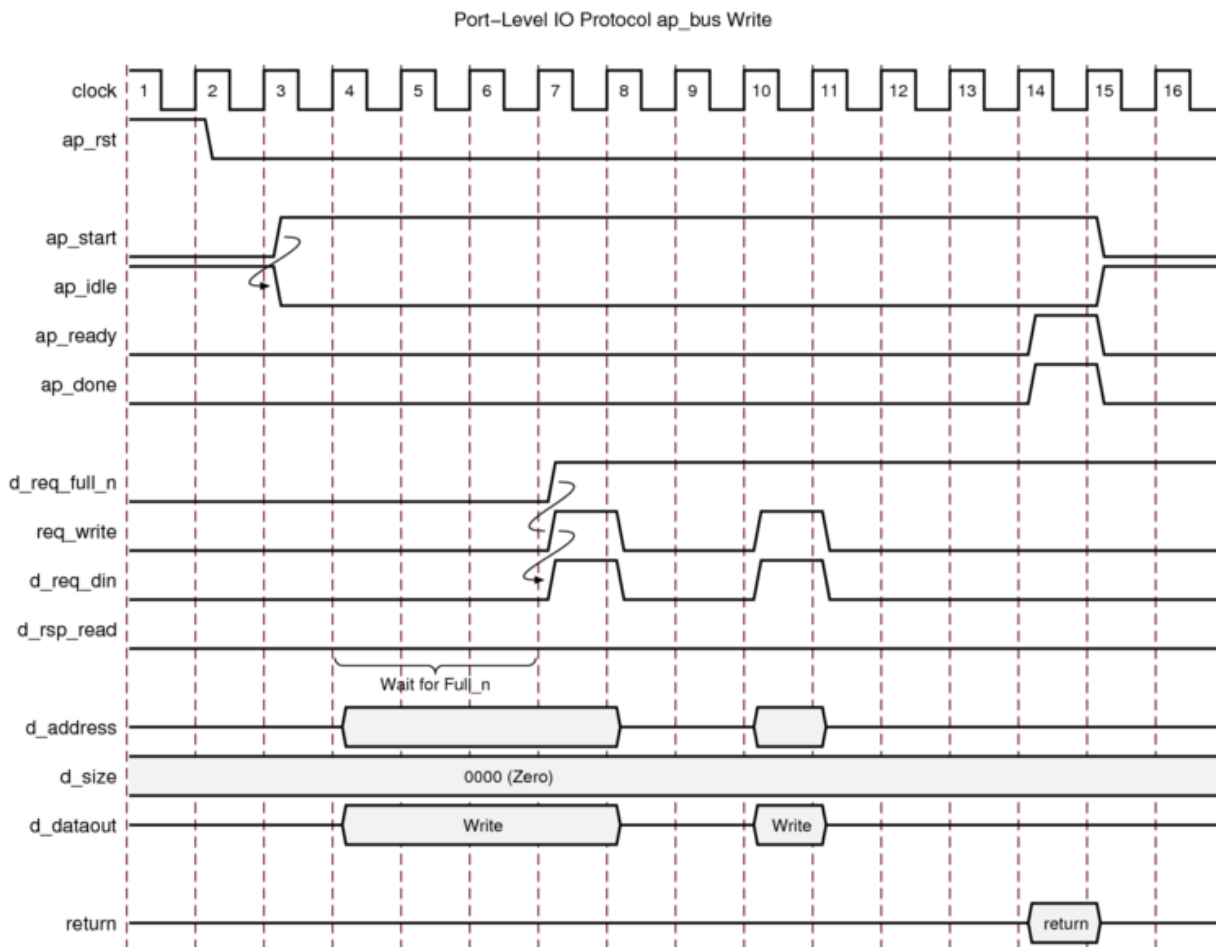

图 104: ap_bus 接口的行为：标准读取



复位后，将执行以下操作：

- 应用 start 后，该块开始正常操作。
- 如果必须执行读取，但 d_rsp_empty_n 为低电平，指示总线桥 FIFO 中没有数据，则会执行以下操作：
 - 断言输出端口 d_req_write 有效，并断言端口 d_req_din 无效，以指示读取操作。
 - 输出地址。
 - 设计停滞并等待数据可用。
- 当数据可用于读取输出信号时，立即断言 d_rsp_read 有效并在下一个时钟沿读取数据。
- 如果必须执行读取并且 d_rsp_empty_n 为高电平，指示总线桥 FIFO 中有可用数据，则会执行以下操作：
 - 断言输出端口 d_req_write 有效并断言端口 d_req_din 无效，以指示读取操作。
 - 输出地址。
 - 在下一个时钟周期断言输出信号 d_rsp_read 有效，并在下一个时钟沿读取数据。

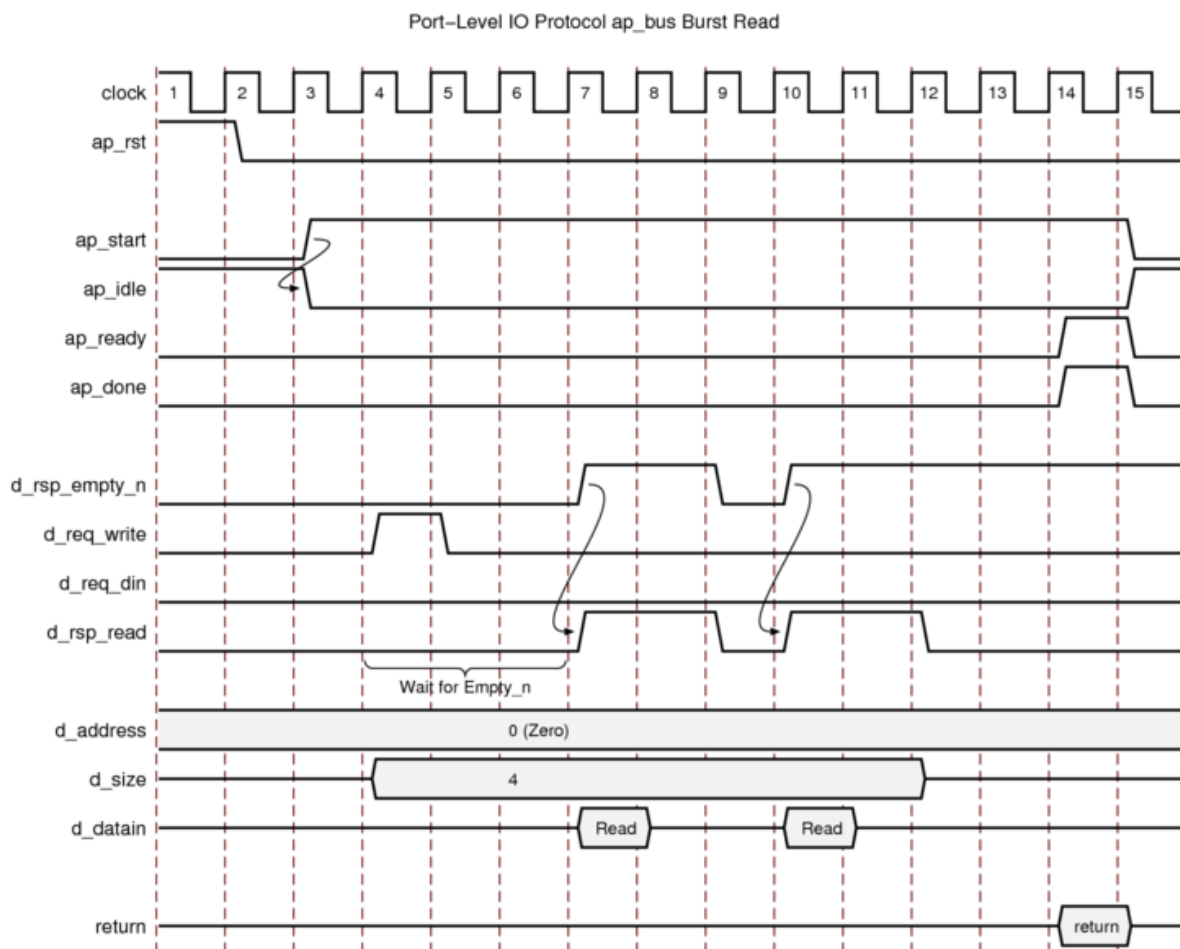
图 105: ap_bus 接口的行为：标准写入



复位后，将执行以下操作：

- 应用 start 后，该块开始正常操作。
- 如果必须执行写入，但 d_req_full_n 为低电平，指示总线桥 FIFO 中没有空间可用，则会执行以下操作：
 - 输出地址和数据。
 - 设计停滞并等待空间可用。
- 当有空间可用于写入时，将执行以下操作：
 - 断言输出端口 d_req_write 和 d_req_din 有效，以指示写操作。
 - 立即断言输出信号 d_req_din 有效以指示该数据在下一个时钟沿有效。
- 如果必须执行写入并且 d_req_full_n 为高电平，指示总线桥 FIFO 中有可用空间，则会执行以下操作：
 - 断言输出端口 d_req_write 和 d_req_din 有效，以指示写操作。
 - 输出地址和数据。
 - 断言输出信号 d_req_din 有效以指示该数据在下一时钟沿有效。

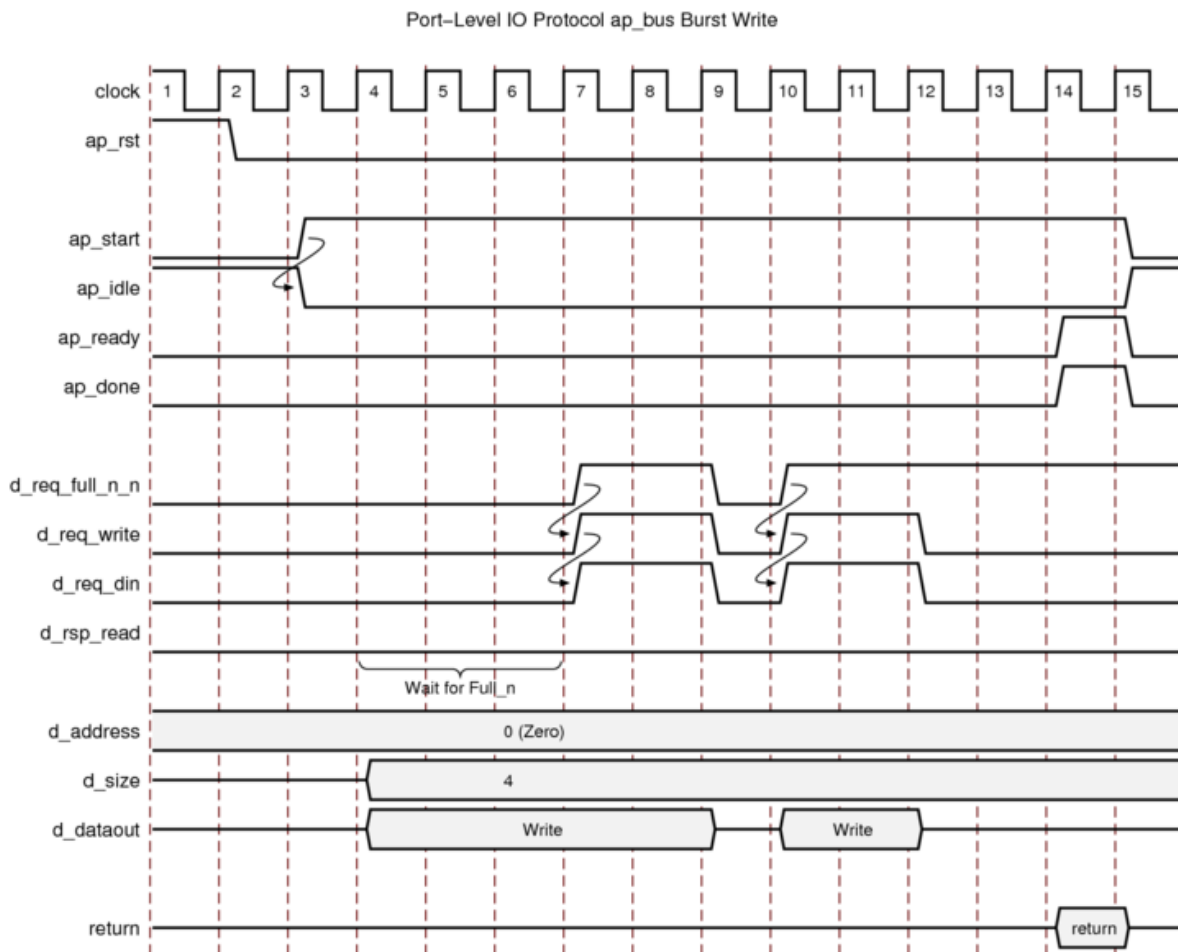
图 106: ap_bus 接口的行为：突发读取



复位后，将执行以下操作：

- 应用 start 后，该块开始正常操作。
- 如果必须执行读取，但 d_rsp_empty_n 为低电平，指示总线桥 FIFO 中没有数据，则会执行以下操作：
 - 断言输出端口 d_req_write 有效，并断言端口 d_req_din 无效，以指示读取操作。
 - 输出传输的基址和大小。
 - 设计停滞并等待数据可用。
- 当数据可用于读取输出信号时，立即断言 d_rsp_read 有效，并在接下来的 N 个时钟沿读取数据，其中 N 是输出端口大小的值。
- 如果总线桥 FIFO 的值已清空，数据传输将立即停止，并等待数据可用后再继续。

图 107: ap_bus 接口的行为：突发写入



复位后，将执行以下操作：

- 应用 start 后，该块开始正常操作。
- 如果必须执行写入，但 d_req_full_n 为低电平，指示总线桥 FIFO 中没有空间可用，则会执行以下操作：
 - 输出基址、传输大小和数据。
 - 设计停滞并等待空间可用。
- 当有空间可用于写入时，将执行以下操作：
 - 断言输出端口 d_req_write 和 d_req_din 有效，以指示写操作。
 - 立即断言输出信号 d_req_din 有效以指示该数据在下一个时钟沿有效。
 - 如果 FIFO 已满，则会立即断言输出信号 d_req_din 无效，并在空间可用时重新断言。
 - 传输于完成 N 个数据值后停止，其中 N 是输出端口大小的值。
- 如果必须执行写入并且 d_rsp_full_n 为高电平，指示总线桥 FIFO 中有可用空间，则传输开始，设计停滞并等待直到 FIFO 已满。

axis

`axis` 模式指定 AXI4-Stream I/O 协议。如需了解有关 AXI4-Stream 接口的完整说明（包括时序和端口），请参阅《Vivado Design Suite: AXI 参考指南》(UG1037)。如需了解有关使用此 I/O 协议的全部功能的信息，请参阅[使用 AXI4 接口](#)。

s_axilite

`s_axilite` 模式指定 AXI4-Lite 从接口 I/O 协议。如需了解有关 AXI4-Lite 从接口的完整说明（包括时序和端口），请参阅《Vivado Design Suite: AXI 参考指南》(UG1037)。如需了解有关使用此 I/O 协议的全部功能的信息，请参阅[使用 AXI4 接口](#)。

m_axi

`m_axi` 模式指定 AXI4 主接口 I/O 协议。如需了解有关 AXI4 主接口的完整说明（包括时序和端口），请参阅《Vivado Design Suite: AXI 参考指南》(UG1037)。如需了解有关使用此 I/O 协议的全部功能的信息，请参阅[使用 AXI4 接口](#)。

AXI4-Lite 从接口 C 驱动参考

将 AXI4-Lite 从接口添加到设计中时，会自动创建一组 C 驱动文件。这些 C 驱动文件可提供一组 API 以集成到 CPU 上运行的任意软件中，并且这些 API 可用于通过 AXI4-Lite 接口与器件进行通信。

API 函数名称衍生自综合的顶层函数。此参考部分假定顶层函数称为 DUT。下表列出了 C 驱动文件中提供的每个 API 函数。

表 47：C 驱动 API 函数

API 函数	描述
<code>XDut_Initialize</code>	此 API 向 InstancePtr 写入值，以供后续在其它 API 中使用。赛灵思建议调用此 API 来初始化器件（系统中使用 MMU 时除外）。
<code>XDut_CfgInitialize</code>	初始化器件配置。在系统中使用 MMU 时，调用此函数前请先将 XDut_Config 变量中的基址替换为虚拟基址。不可用于 Linux 系统。
<code>XDut_LookupConfig</code>	用于按 ID 获取器件的配置信息。配置信息包含物理基址。不可用于 Linux。
<code>XDut_Release</code>	在 Linux 中释放 uio 器件。通过 munmap 删除映射：如果终止此进程，将自动删除映射。仅适用于 Linux 系统。
<code>XDut_Start</code>	启动器件。此函数将在器件上断言 <code>ap_start</code> 端口有效。仅当器件上存在 <code>ap_start</code> 端口时才可用。
<code>XDut_IsDone</code>	检查器件是否已完成先前执行：此函数将返回器件上 <code>ap_done</code> 端口的值。仅当器件上存在 <code>ap_done</code> 端口时才可用。
<code>XDut_IsIdle</code>	检查器件是否处于空闲状态：此函数将返回 <code>ap_idle</code> 端口的值。仅当器件上存在 <code>ap_idle</code> 端口时才可用。
<code>XDut_IsReady</code>	检查器件是否已准备好处理下一项输入：此函数将返回 <code>ap_ready</code> 端口的值。仅当器件上存在 <code>ap_ready</code> 端口时才可用。
<code>XDut_Continue</code>	断言 <code>ap_continue</code> 端口有效。仅当器件上存在 <code>ap_continue</code> 端口时才可用。
<code>XDut_EnableAutoRestart</code>	在器件上启用“自动重启 (auto restart)”。设置此项后，当前传输事务完成时，器件将自动启动下一项传输事务。
<code>XDut_DisableAutoRestart</code>	禁用“auto restart”功能。
<code>XDut_Set_ARG</code>	将值写入端口 ARG（顶层函数的标量实参）。仅当 ARG 为输入端口时才可用。
<code>XDut_Set_ARG_vld</code>	断言 <code>ARG_vld</code> 端口有效。仅当 ARG 为输入端口并使用 <code>ap_hs</code> 或 <code>ap_vld</code> 接口协议实现时才可用。

表 47: C 驱动 API 函数 (续)

API 函数	描述
XDut_Set_ARG_ack	断言 ARG_ack 端口有效。仅当 ARG 为输出端口并使用 ap_hs 或 ap_ack 接口协议实现时才可用。
XDut_Get_ARG	从 ARG 读取值。仅当 ARG 端口为器件上的输出端口时才可用。
XDut_Get_ARG_vld	从 ARG_vld 读取值。仅当 ARG 端口为器件上的输出端口并使用 ap_hs 或 ap_vld 接口协议实现时才可用。
XDut_Get_ARG_ack	从 ARG_ack 读取值。仅当 ARG 端口为器件上的输入端口并使用 ap_hs 或 ap_ack 接口协议实现时才可用。
XDut_Get_ARG_BaseAddress	返回接口内的数组基址。仅当 ARG 为分组到 AXI4-Lite 接口内的数组时才可用。
XDut_Get_ARG_HighAddress	返回数组最上方元素的地址。仅当 ARG 为分组到 AXI4-Lite 接口内的数组时才可用。
XDut_Get_ARG_TotalBytes	返回用于存储数组的字节总数。仅当 ARG 为分组到 AXI4-Lite 接口内的数组时才可用。 如果数组中的元素少于 16 位，Vivado HLS 会将多个元素分组到 AXI4-Lite 接口的位宽为 32 位的数据中。如果元素的位宽超过 32 位，Vivado HLS 会将每个元素存储在多个连续地址上。
XDut_Get_ARG_BitWidth	返回数组中每个元素的位宽。仅当 ARG 为分组到 AXI4-Lite 接口内的数组时才可用。 如果数组中的元素少于 16 位，Vivado HLS 会将多个元素分组到 AXI4-Lite 接口的位宽为 32 位的数据中。如果元素的位宽超过 32 位，Vivado HLS 会将每个元素存储在多个连续地址上。
XDut_Get_ARG_Depth	返回数组中元素总数。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。 如果数组中的元素少于 16 位，Vivado HLS 会将多个元素分组到 AXI4-Lite 接口的位宽为 32 位的数据中。如果元素的位宽超过 32 位，Vivado HLS 会将每个元素存储在多个连续地址上。
XDut_Write_ARG_Words	将长度为 32 位的字词写入 AXI4-Lite 接口的指定地址。此 API 需要来自 BaseAddress 的偏移地址以及要存储的数据长度。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。
XDut_Read_ARG_Words	从数组中读取长度为 32 位的字词。此 API 需要数据目标、来自 BaseAddress 的偏移地址以及要存储的数据长度。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。
XDut_Write_ARG_Bytes	将字节长度写入 AXI4-Lite 接口的指定地址。此 API 需要来自 BaseAddress 的偏移地址以及要存储的数据长度。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。
XDut_Read_ARG_Bytes	从数组读取字节长度。此 API 需要数据目标、来自 BaseAddress 的偏移地址以及要加载的数据长度。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。
XDut_InterruptGlobalEnable	启用中断输出。仅当存在 ap_start 时，中断函数才可用。
XDut_InterruptGlobalDisable	禁用中断输出。
XDut_InterruptEnable	启用中断源。可存在最多 2 个中断源 (source 0 对应 ap_done, source 1 对应 ap_ready)
XDut_InterruptDisable	禁用中断源。
XDut_InterruptClear	清除中断状态。
XDut_InterruptGetEnabled	检查启用的中断源。
XDut_InterruptGetStatus	检查触发的中断源。

以下提供了有关 API 函数的详细信息。

XDut_Initialize

摘要

```
int XDut_Initialize(XDut *InstancePtr, u16 DeviceId);
```

```
int XDut_Initialize(XDut *InstancePtr, const char* InstanceName);
```

描述

int XDut_Initialize(XDut *InstancePtr, u16 DeviceId): 在独立系统上用于器件初始化。此 API 将向 InstancePtr 写入适当的值，以供后续在其它 API 中使用。赛灵思建议调用此 API 来初始化器件，但在系统中使用 MMU 时除外，在此情况下请参阅 XDut_CfgInitialize 函数。

int XDut_Initialize(XDut *InstancePtr, const char* InstanceName): 在 Linux 系统上专用于初始化指定的 uio 器件。使用 sysfs 中的 uio 器件信息创建最多 5 个内存映射并按 mmap 分配从基址。

- “InstancePtr”：指向器件实例的指针。
- “DeviceId”：表示 xparameters.h 中定义的器件 ID。
- “InstanceName”：uio 器件的名称。
- “Return”：XST_SUCCESS 表示成功，否则即为失败。

XDut_CfgInitialize

摘要

```
XDut_CfgInitializeint XDut_CfgInitialize(XDut *InstancePtr, XDut_Config
*ConfigPtr);
```

描述

在系统中使用 MMU 时初始化器件。在此类情况下，AXI4-Lite 从接口的有效地址不同于 xparameters.h 中指定的地址，并且需要使用 API 来初始化器件。

- InstancePtr: 指向器件实例的指针。
- DeviceId: 指向 XDut_Config 的指针。

返回：XST_SUCCESS 表示成功，否则表示失败。

XDut_LookupConfig

摘要

```
XDut_Config* XDut_LookupConfig(u16 DeviceId);
```

描述

此函数用于按 ID 获取器件的配置信息。

- DeviceId: 器件 ID，在 xparameters.h 中指定。

返回：指向 XDut_LookupConfig 变量的指针，此变量包含 ID 为 DeviceId 的器件的配置信息。如果未找到匹配的 DeviceId，则返回 NULL。

XDut_Release

摘要

```
int XDut_Release(XDut *InstancePtr);
```

描述

释放 uio 器件。使用 munmap 删除映射。（如果此进程终止，将自动删除映射）

- InstanceName: uio 器件的名称。
- 返回: XST_SUCCESS 表示成功，否则表示失败。

XDut_Start

摘要

```
void XDut_Start(XDut *InstancePtr);
```

描述

启动器件。此函数将在器件上断言 ap_start 端口有效。仅当器件上存在 ap_start 端口时才可用。

- InstancePtr: 指向器件实例的指针。

XDut_IsDone

摘要

```
void XDut_IsDone(XDut *InstancePtr);
```

描述

检查器件是否已完成先前执行：此函数将返回器件上 ap_done 端口的值。仅当器件上存在 ap_done 端口时才可用。

- InstancePtr: 指向器件实例的指针。

XDut_IsIdle

摘要

```
void XDut_IsIdle(XDut *InstancePtr);
```

描述

检查器件是否处于空闲状态：此函数将返回 ap_idle 端口的值。仅当器件上存在 ap_idle 端口时才可用。

- InstancePtr: 指向器件实例的指针。

XDut_IsReady

摘要

```
void XDut_IsReady(XDut *InstancePtr);
```

描述

检查器件是否已准备好处理下一项输入：此函数将返回 `ap_ready` 端口的值。仅当器件上存在 `ap_ready` 端口时才可用。

- `InstancePtr`：指向器件实例的指针。

XDut_Continue

摘要

```
void XExample_Continue(XExample *InstancePtr);
```

描述

断言 `ap_continue` 端口有效。仅当器件上存在 `ap_continue` 端口时才可用。

- `InstancePtr`：指向器件实例的指针。

XDut_EnableAutoRestart

摘要

```
void XDut_EnableAutoRestart(XDut *InstancePtr);
```

描述

在器件上启用“自动重启 (auto restart)”。启用后，

- 当器件断言 `ap_done` 有效后将立即断言 `ap_start` 端口有效，器件将自动启动下一项传输事务。
- 或者，如果在器件上实现块级 I/O 协议 `ap_ctrl_chain`，那么当器件断言 `ap_ready` 有效后，下一项传输事务将自动重启（将断言 `ap_start`），并且当器件断言 `ap_done` 有效时，将断言 `ap_continue` 有效。

仅当存在 `ap_start` 端口时才可用。

- `InstancePtr`：指向器件实例的指针。

XDut_DisableAutoRestart

摘要

```
void XDut_DisableAutoRestart(XDut *InstancePtr);
```

描述

禁用“自动重启”功能。仅当存在 `ap_start` 端口时才可用。

- `InstancePtr`：指向器件实例的指针。

XDut_Set_ARG

摘要

```
void XDut_Set_ARG(XDut *InstancePtr, u32 Data);
```

描述

将值写入 `ARG` 端口（顶层函数的标量实参）。仅当 `ARG` 为输入端口时才可用。

- `InstancePtr`：指向器件实例的指针。
- `Data`：要写入的值。

XDut_Set_ARG_vld

摘要

```
void XDut_Set_ARG_vld(XDut *InstancePtr);
```

描述

断言 `ARG_vld` 端口有效。仅当 `ARG` 为输入端口并使用 `ap_hs` 或 `ap_vld` 接口协议实现时才可用。

- `InstancePtr`：指向器件实例的指针。

XDut_Set_ARG_ack

摘要

```
void XDut_Set_ARG_ack(XDut *InstancePtr);
```

描述

断言 `ARG_ack` 端口有效。仅当 `ARG` 为输出端口并使用 `ap_hs` 或 `ap_ack` 接口协议实现时才可用。

- `InstancePtr`：指向器件实例的指针。

XDut_Get_ARG

摘要

```
u32 XDut_Get_ARG(XDut *InstancePtr);
```

描述

从 ARG 读取值。仅当 ARG 端口为器件上的输出端口时才可用。

- InstancePtr：指向器件实例的指针。

返回：ARG 的值。

XDut_Get_ARG_vld

摘要

```
u32 XDut_Get_ARG_vld(XDut *InstancePtr);
```

描述

读取 ARG_vld 中的值。仅当 ARG 端口为器件上的输出端口并使用 ap_hs 或 ap_vld 接口协议实现时才可用。

- InstancePtr：指向器件实例的指针。

返回：ARG_vld 的值。

XDut_Get_ARG_ack

摘要

```
u32 XDut_Get_ARG_ack(XDut *InstancePtr);
```

描述

读取 ARG_ack 中的值。仅当 ARG 端口为器件上的输入端口并使用 ap_hs 或 ap_ack 接口协议实现时才可用。

- InstancePtr：指向器件实例的指针。

返回：ARG_ack 的值。

XDut_Get_ARG_BaseAddress

摘要

```
u32 XDut_Get_ARG_BaseAddress(XDut *InstancePtr);
```

描述

返回接口内的数组基址。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。

- InstancePtr：指向器件实例的指针。

返回：数组的基址。

XDut_Get_ARG_HighAddress

摘要

```
u32 XDut_Get_ARG_HighAddress(XDut *InstancePtr);
```

描述

返回数组最上方元素的地址。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。

- InstancePtr：指向器件实例的指针。

返回：数组最上方元素的地址。

XDut_Get_ARG_TotalBytes

摘要

```
u32 XDut_Get_ARG_TotalBytes(XDut *InstancePtr);
```

描述

返回用于存储数组的字节总数。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。

如果数组中的元素少于 16 位，Vivado HLS 会将多个元素分组到 AXI4-Lite 接口的位宽为 32 位的数据中。如果元素的位宽超过 32 位，Vivado HLS 会将每个元素存储在多个连续地址上。

- InstancePtr：指向器件实例的指针。

返回：用于存储数组的字节总数。

XDut_Get_ARG_BitWidth

摘要

```
u32 XDut_Get_ARG_BitWidth(XDut *InstancePtr);
```

描述

返回数组中每个元素的位宽。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。

如果数组中的元素少于 16 位，Vivado HLS 会将多个元素分组到 AXI4-Lite 接口的位宽为 32 位的数据中。如果元素的位宽超过 32 位，Vivado HLS 会将每个元素存储在多个连续地址上。

- InstancePtr：指向器件实例的指针。

返回：数组中每个元素的位宽。

XDut_Get_ARG_Depth

摘要

```
u32 XDut_Get_ARG_Depth(XDut *InstancePtr);
```

描述

返回数组中元素总数。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。

如果数组中的元素少于 16 位，Vivado HLS 会将多个元素分组到 AXI4-Lite 接口的位宽为 32 位的数据中。如果元素的位宽超过 32 位，Vivado HLS 会将每个元素存储在多个连续地址上。

- InstancePtr：指向器件实例的指针。

返回：数组中元素总数。

XDut_Write_ARG_Words

摘要

```
u32 XDut_Write_ARG_Words(XDut *InstancePtr, int offset, int *data, int length);
```

描述

将长度为 32 位的字词写入 AXI4-Lite 接口的指定地址。此 API 需要来自 BaseAddress 的偏移地址以及要存储的数据长度。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。

- InstancePtr：指向器件实例的指针。
- offset：AXI4-Lite 接口中的地址。
- data：指向要存储的数据值的指针。
- length：要存储的数据长度。

返回：从指定地址写入的数据长度。

XDut_Read_ARG_Words

摘要

```
u32 XDut_Read_ARG_Words(XDut *InstancePtr, int offset, int *data, int length);
```

描述

从数组中读取长度为 32 位的字词。此 API 需要数据目标、来自 BaseAddress 的偏移地址以及要存储的数据长度。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。

- InstancePtr：指向器件实例的指针。

- `offset`：ARG 中的地址。
- `data`：指向数据缓存的指针。
- `length`：要存储的数据长度。

返回：从指定地址读取的数据长度。

XDut_Write_ARG_Bytes

摘要

```
u32 XDut_Write_ARG_Bytes(XDut *InstancePtr, int offset, char *data, int length);
```

描述

将字节长度写入 AXI4-Lite 接口的指定地址。此 API 需要来自 `BaseAddress` 的偏移地址以及要存储的数据长度。仅当 ARG 为分组到 AXI4-Lite 接口中的数组时才可用。

- `InstancePtr`：指向器件实例的指针。
- `offset`：ARG 中的地址。
- `data`：指向要存储的数据值的指针。
- `length`：要存储的数据长度。

返回：从指定地址写入的数据长度。

XDut_Read_ARG_Bytes

摘要

```
u32 XDut_Read_ARG_Bytes(XDut *InstancePtr, int offset, char *data, int length);
```

描述

从数组读取字节长度。此 API 需要数据目标、来自 `BaseAddress` 的偏移地址以及要加载的数据长度。仅当 ARG 为分组到 AXI4-Lite 接口内的数组时才可用。

- `InstancePtr`：指向器件实例的指针。
- `offset`：ARG 中的地址。
- `data`：指向数据缓存的指针。
- `length`：要加载的数据长度。

返回：从指定地址读取的数据长度。

XDut_InterruptGlobalEnable

摘要

```
void XDut_InterruptGlobalEnable(XDut *InstancePtr);
```

描述

启用中断输出。仅当有 `ap_start` 时，中断功能才可用。

- `InstancePtr`：指向器件实例的指针。

XDut_InterruptGlobalDisable

摘要

```
void XDut_InterruptGlobalDisable(XDut *InstancePtr);
```

描述

禁用中断输出。

- `InstancePtr`：指向器件实例的指针。

XDut_InterruptEnable

摘要

```
void XDut_InterruptEnable(XDut *InstancePtr, u32 Mask);
```

描述

启用中断源。可存在最多 2 个中断源（`source 0` 对应 `ap_done`，`source 1` 对应 `ap_ready`）。

- `InstancePtr`：指向器件实例的指针。
- `Mask`：位掩码。
 - `Bit n = 1`：启用中断源 `n`。
 - `Bit n = 0`：无更改。

XDut_InterruptDisable

摘要

```
void XDut_InterruptDisable(XDut *InstancePtr, u32 Mask);
```

描述

禁用中断源。

- InstancePtr：指向器件实例的指针。
- Mask：位掩码。
 - Bit n = 1：禁用中断源 n。
 - Bit n = 0：无更改。

XDut_InterruptClear

摘要

```
void XDut_InterruptClear(XDut *InstancePtr, u32 Mask);
```

描述

清除中断状态。

- InstancePtr：指向器件实例的指针。
- Mask：位掩码。
 - Bit n = 1：切换中断状态 n。
 - Bit n = 0：无更改。

XDut_InterruptGetEnabled

摘要

```
u32 XDut_InterruptGetEnabled(XDut *InstancePtr);
```

描述

检查启用的中断源。

- InstancePtr：指向器件实例的指针。
- 返回：位掩码。
- 位 n = 1：已启用。
 - 位 n = 0：已禁用。

XDut_InterruptGetStatus

摘要

```
u32 XDut_InterruptGetStatus(XDut *InstancePtr);
```


描述

检查触发的中断源。

- `InstancePtr`：指向器件实例的指针。
- **返回**：位掩码。
 - Bit n = 1：已触发。
 - Bit n = 0：未触发。

HLS 视频函数库



重要提示！ Vivado® HLS 视频库已迁移到赛灵思 GitHub，可在如下位置找到：<https://github.com/Xilinx/xfopencv>

HLS 线性代数库函数

本节对 Vivado HLS 线性代数处理函数进行了解释。

`matrix_multiply`

摘要

```
template<
class TransposeFormA,
class TransposeFormB,
int RowsA,
int ColsA,
int RowsB,
int ColsB,
int RowsC,
int ColsC,
typename InputType,
typename OutputType>
void matrix_multiply(
    const InputType A[RowsA][ColsA],
    const InputType B[RowsB][ColsB],
    OutputType C[RowsC][ColsC]);
```

描述

$C=AB$

- 计算 2 个矩阵的乘积，返回第 3 个矩阵。
- （可选）输入矩阵的转置（和复数数据类型的共轭转置）。
- 针对已展开的浮点实现提供的替代架构。

参数

表 48：参数

参数	描述
TransposeFormA	矩阵 A 的转置要求；NoTranspose、Transpose 或 ConjugateTranspose。
TransposeFormB	矩阵 B 的转置要求；NoTranspose、Transpose 或 ConjugateTranspose。
RowsA	矩阵 A 中的行数
ColsA	矩阵 A 中的列数
RowsB	矩阵 B 中的行数
ColsB	矩阵 B 中的列数
RowsC	矩阵 C 中的行数
ColsC	矩阵 C 中的列数
InputType	输入数据类型
OutputType	输出数据类型

如果 `ColsA != RowsB`，该函数将抛出断言，并编译失败或综合失败。执行检查前，已解析 A 和 B 的转置要求。

实参

表 49：实参

实参	描述
A	第 1 个输入矩阵
B	第 2 个输入矩阵
C	AB 乘积输出矩阵

返回值

- 不适用 (void 函数)

受支持的数据类型

- `ap_fixed`
- `float`
- `x_complex<ap_fixed>`
- `x_complex<float>`

输入数据假定

- 对于浮点类型，不支持低于正常值的输入值。如果使用此类值，已综合的硬件会将其清零，且行为将有别于软件仿真。

cholesky

摘要

```
template<
bool LowerTriangularL,
int RowsColsA,
typename InputType,
typename OutputType>
int cholesky(
const InputType A[RowsColsA][RowsColsA],
OutputType L[RowsColsA][RowsColsA])
```

描述

$A=LL^*$

- 计算输入矩阵 A 的 Cholesky 分解，返回矩阵 L。
- 根据参数 LowerTriangularL，输出矩阵 L 可能是上三角矩阵或下三角矩阵。
- 矩阵 L 的未使用的部分中的元素将设置为 0。

参数

表 50：参数

参数	描述
RowsColsA	输入和输出矩阵的行列维度
LowerTriangularL	选择期望下三角还是上三角输出矩阵
InputType	输入数据类型
OutputType	输出数据类型

实参

表 51：实参

实参	描述
A	埃尔米特 (Hermitian) 矩阵/对称正定输入矩阵
L	下三角或上三角输出矩阵

返回值

- 0 = 成功
- 1 = 失败。该函数尝试查找负数的平方根，即输入矩阵 A 并非埃尔米特 (Hermitian) 矩阵/对称正定输入矩阵。

受支持的数据类型

- ap_fixed
- float
- x_complex<ap_fixed>

· x_complex<float>

输入数据假定

- 该函数假定输入矩阵为对称正定矩阵（针对复数输入假定为埃尔米特 (Hermitian) 正定矩阵）。
- 对于浮点类型，不支持低于正常值的输入值。如果使用此类值，已综合的硬件会将其清零，且行为将有别于软件仿真。

qrf

摘要

```
template<
bool TransposeQ,
int RowsA,
int ColsA,
typename InputType,
typename OutputType>
void qrf(
const InputType A[RowsA][ColsA],
OutputType Q[RowsA][RowsA],
OutputType R[RowsA][ColsA])
```

描述

A=QR

- 计算输入矩阵 A 的完整 QR 分解，生成正交输出矩阵 Q 和上三角矩阵 R。
- （可选）输出矩阵 Q 可根据参数 TransposeQ 转置。
- 输出矩阵 R 的下三角元素不会清零。
- 不实现瘦 QR 分解（也称为经济型 QR 分解）。

参数

表 52：参数

参数	描述
TransposeQ	选择是否应转置 Q 矩阵。
RowsA	输入矩阵 A 中的行数
ColsA	输入矩阵 A 中的列数
InputType	输入数据类型
OutputType	输出数据类型

- 如果 RowsA < ColsA，该函数将编译失败或综合失败。

实参

表 53：实参

实参	描述
A	输入矩阵
Q	正交输出矩阵
R	上三角输出矩阵

返回值

- 不适用 (void 函数)

受支持的数据类型

- float
- x_complex<float>

输入数据假定

- 对于浮点类型，不支持低于正常值的输入值。如果使用此类值，已综合的硬件会将其清零，且行为将有别于软件仿真。

cholesky_inverse

摘要

```
template <
    int RowsColsA,
    typename InputType,
    typename OutputType>
void cholesky_inverse(const InputType A[RowsColsA][RowsColsA],
                     OutputType InverseA[RowsColsA][RowsColsA],
                     int& cholesky_success)
```

描述

$AA^{-1} = I$

- 使用 Cholesky 分解法计算对称正定输入矩阵 A 的逆矩阵，生成矩阵 InverseA。

参数

表 54：参数

参数	描述
RowsColsA	输入和输出矩阵的行列维度
InputType	输入数据类型
OutputType	输出数据类型

实参

表 55：实参

实参	描述
A	埃尔米特 (Hermitian) 方阵/对称正定输入矩阵
InverseA	输入矩阵的逆矩阵
cholesky_success	0 = 成功 1 = 失败。Cholesky 函数尝试查找负数的平方根。输入矩阵 A 并非对称正定矩阵。

返回值

- 不适用 (void 函数)

受支持的数据类型

- ap_fixed
- float
- x_complex<ap_fixed>
- x_complex<float>

输入数据假定

- 该函数假定输入矩阵为对称正定矩阵（针对复数输入假定为埃尔米特 (Hermitian) 正定矩阵）。
- 对于浮点类型，不支持低于正常值的输入值。如果使用此类值，已综合的硬件会将其清零，且行为将有别于软件仿真。

qr_inverse

摘要

```
template <
    int RowsColsA,
    typename InputType,
    typename OutputType>
void qr_inverse(const InputType A[RowsColsA][RowsColsA],
                OutputType InverseA[RowsColsA][RowsColsA],
                int& A_singular)
```

描述

$$AA^{-1} = I$$

- 使用 QR 分解法计算输入矩阵 A 的逆矩阵，生成矩阵 InverseA。

参数

表 56：参数

参数	描述
RowsColsA	输入和输出矩阵的行列维度。
InputType	输入数据类型
OutputType	输出数据类型

实参

表 57：实参

实参	描述
A	输入矩阵 A
InverseA	输入矩阵的逆矩阵
A_singular	0 = 成功 1 = 矩阵 A 为单数

返回值

- 不适用 (void 函数)

受支持的数据类型

- float
- x_complex<float>

输入数据假定

- 对于浮点类型，不支持低于正常值的输入值。如果使用此类值，已综合的硬件会将其清零，且行为将有别于软件仿真。

svd**摘要**

```
template<
int RowsA,
int ColsA,
typename InputType,
typename OutputType>
void svd(
const InputType A[RowsA][ColsA],
OutputType S[RowsA][ColsA],
OutputType U[RowsA][RowsA],
OutputType V[ColsA][ColsA])
```

描述

$A=USV^*$

- 计算输入矩阵 A 的单数值分解，生成矩阵 U、S 和 V。
- 仅支持方阵。
- 已使用迭代双侧雅可比 (Jacobi) 法实现。

参数

表 58：参数

参数	描述
RowsA	行维度
ColsA	列维度
InputType	输入数据类型
OutputType	输出数据类型

- 如果 `RowsA != ColsA`，该函数将抛出断言，并编译失败或综合失败。

实参

表 59：实参

实参	描述
A	输入矩阵
S	输入矩阵的单数值
U	输入矩阵的左单数矢量
V	输入矩阵的右单数矢量

返回值

- 不适用 (void 函数)

受支持的数据类型

- `float`
- `x_complex<float>`

输入数据假定

- 对于浮点类型，不支持低于正常值的输入值。如果使用此类值，已综合的硬件会将其清零，且行为将有别于软件仿真。

示例

这些示例提供了基本测试激励文件，并演示了如何将每个线性代数函数参数化和例化。在 Vivado HLS 示例目录中为每个函数提供了 1 个或多个示例：

```
<VIVADO_HLS>/examples/design/linear_algebra
```

每个示例都包含以下文件：

- `<example>.cpp`: 用于例化库函数的顶层综合封装。
- `<example>.h`: 用于定义矩阵大小、数据类型以及（如果适用）架构选择的头文件。
- `<example>_tb.cpp`: 用于例化顶层综合封装的基本测试激励文件。
- `run_hls.tcl`: 用于设置示例 Vivado HLS 工程的 Tcl 命令：

```
vivado_hls -f run_hls.tcl
```

- `directives.tcl`: （可选）应用最优化/实现指令的其它 Tcl 命令。

HLS DSP 库函数

HLS DSP 库包含构建块函数，以供在 C++ 内进行 DSP 系统建模，其中重点是 SDR 应用中使用的函数。

HLS DSP 函数

本节对 Vivado HLS DSP 处理函数进行了解释。

awgn

摘要

```
template<
    int OutputWidth>
class awgn {
public:
    typedef ap_ufixed<8,4, AP_RND, AP_SAT> t_ input_scale;
    static const int LFSR_SECTION_WIDTH = 32;
    static const int NUM_NOISE_GENES = 4;
    static const int LFSR_WIDTH = LFSR_SECTION_WIDTH*NUM_NOISE_GENES;
    void awgn(ap_uint<LFSR_WIDTH> seed);
    void ~awgn();
    void operator()(t_input_scale &snr,
        ap_int<OutputWidth> &noise);
```

描述

- 输出由输入信噪比 (SNR) 判定的高斯噪声量级。BPSK 信号为 0 dB 即可生成约 7% 的误码率 (BER)。原因在于当 $E_b/N_0 = 0$ 、 $E_b = 1$ 但 $N_0 / 2 = \text{BPSK 通道的噪声功率}$ 时，生成的噪声离散为信号离散的一半。如需了解更多信息，请参阅 MathWorks 网站上的 AWGN 页面 (<https://www.mathworks.com/help/comm/ug/awgn-channel.html>)。
- SNR 输入表示范围在 [0.0 到 16.0)（以分贝为单位）内的信噪比，以 1/16 分贝递增。
- 如果噪声值超出此配置描述范围，则会以相应的正负最大值达成饱和。
- 该函数使用多个独立噪声发生器来创建输出值，这些噪声发生器可通过求和来利用中心极限理论。默认情况下，这些发生器已流水线化并展开，因为期望的目标应用为高速 BER 测试，在此类测试中期望的时钟速率较高且启动时间间隔为 1。

参数

表 60：参数

模板参数	描述
OutputWidth	输出值中的位数。SNR 可指定相对于软 BPSK 信号的噪声量级，值为 01000 和 11000 范围为 8 位到 32 位。

表 61：构造函数实参

实参	描述
seed	噪声发生器内 LFSR 的种子 (seed) 值。

注释：在 C 语言仿真期间检查参数以验证模板参数配置是否合法。

实参

表 62：实参

实参	描述
snr	信噪比输入
noise	输出噪声

返回值

- 不适用 (void 函数)

受支持的基本数据类型

- 输入
 - ap_ufixed
请参阅头文件 hls_awgn.h 中的 typedef t_input_scale 的定义以获取详细信息。
- 输出
 - ap_int

输入数据假定

- 无

nco**摘要**

```
template<
  int AccumWidth,
  int PhaseAngleWidth,
  int SuperSampleRate,
  int OutputWidth,
```

```
class DualOutputCmpyImpl,
class SingleOutputCmpyImpl,
class SingleOutputNegCmpyImpl>
class nco {
public:
void nco(const ap_uint<AccumWidth> InitPinc,
const ap_uint<AccumWidth> InitPoff);
void ~nco();
void operator()(
stream< ap_uint<AccumWidth> > &pinc,
stream< ap_uint<AccumWidth> > &poff,
stream< t_nco_output_data<SuperSampleRate,OutputWidth> >
&outputData
);
```

描述

- 执行数控振荡器 (NCO) 函数。
- 支持采样率超出时钟速率的超采样率 (SSR)，因此必须在每个时钟周期上输出多个并行数据样本。
- 在 SSR 模式下，更改相位增量 (pinc) 会导致发出内部中断提示。这不会导致干扰输出采样，除非在少于 N 个周期内对 pinc 执行 2 次或更多次更改，其中 N 为 SuperSampleRate/2 +1。

参数

表 63： 参数

模板参数	描述
AccumWidth	相位累加器中的位数。用于判定可综合的频率的精度。范围为 4 到 48。
PhaseAngleWidth	在正弦/余弦查找中直接使用的位数。值越大，输出准确性越高，但会牺牲查找表大小。范围为 4 到 16。
SuperSampleRate	每个时钟周期的输出采样数。范围为 1 到 16。
OutputWidth	每个输出的宽度（正弦和余弦）。范围为 4 到 32。
DualOutputCmpyImpl	选择使用 NcoDualOutputCmpyFiveMult 类或 NcoDualOutputCmpyFourMult 类实现含 5 乘数 (5 DSP48) 或 4 乘数 (6 DSP48) 架构的双输出复数乘法。请参阅 hls_nco.h 以获取详细信息。
SingleOutputCmpyImpl	选择使用 NcoSingleOutputCmpyThreeMult 类或 NcoSingleOutputCmpyFourMult 类实现含 3 乘数 (3 DSP48) 或 4 乘数 (4 DSP48) 架构的单输出复数乘法。请参阅 hls_nco.h 以获取详细信息。
SingleOutputNegCmpyImpl	选择使用 NcoSingleOutputCmpyThreeMult 类或 NcoSingleOutputCmpyFourMult 类实现含 3 乘数 (3 DSP48) 或 4 乘数 (4 DSP48) 架构的单输出负复数乘法。请参阅 hls_nco.h 以获取详细信息。

注释：在 C 语言仿真期间检查参数以验证模板参数配置是否合法。

实参

表 64：实参

实参	描述
pinc	Pinc 表示相位增量 (Phase Increment)。输出相位每次采样的增量为 $pinc/2^{AccumWidth} * 2\pi$ 。
poff	Poff 表示相位偏移 (Phase Offset)。该值与累加相位相加。输出的相位偏移量为 $poff/2^{AccumWidth} * 2\pi$ 。
outputData	正弦和余弦输出。输出部分的量级近似计算方式为 $\cos(\phi) * 2^{OutputWidth-1}$ 和 $\sin(\phi) * 2^{OutputWidth-1}$ ，其中 ϕ 表示相位累加器所描述的相位（经 poff 相应偏移）。

返回值

- 不适用（void 函数）

受支持的基本数据类型

- 输入
 - ap_uint
- 输出
 - std::complex< ap_int >

请参阅 hls_nco.h 中的 t_nco_output_data 结构体定义以获取详细信息。

输入数据假定

- 无

convolution_encoder

摘要

```
template<
    int OutputWidth,
    bool Punctured,
    bool DualOutput,
    int InputRate,
    int OutputRate,
    int ConstraintLength,
    int PunctureCode0,
    int PunctureCode1,
    int ConvolutionCode0,
    int ConvolutionCode1,
    int ConvolutionCode2,
    int ConvolutionCode3,
    int ConvolutionCode4,
    int ConvolutionCode5,
    int ConvolutionCode6>
class convolution_encoder {
```

```
public:
    convolution_encoder();
    ~convolution_encoder();
    void operator()(stream< ap_uint<1> > &inputData,
        stream< ap_uint<OutputWidth> > &outputData);
```

描述

- 基于用户定义的卷积代码和约束长度执行输入数据流的卷积编码
- （可选）数据打孔
- （可选）双通道输出

参数

表 65：参数

模板参数	描述
OutputWidth	定义输出总线中的位数。当 Punctured=true 且 DualOutput=false 时，为 1 位，当 DualOutput=true 时，为 2 位，否则为 OutputRate 位数。
Punctured	当值为 true 时，启用数据打孔。
DualOutput	当值为 true 时，启用含打孔数据的双输出。
InputRate	定义码率的分子。
OutputRate	定义码率的分母。
ConstraintLength	约束长度 K 即编码器中的寄存器数加 1。
PunctureCode0	当 Punctured=true 时，表示输出 0 的打孔代码。长度（二进制）必须等于打孔输入率。两个 PunctureCode 参数中所含数值 1 的总数等于输出率。
PunctureCode1	当 Punctured=true 时，表示输出 1 的打孔代码。长度（二进制）必须等于打孔输入率。两个 PunctureCode 参数中所含数值 1 的总数等于输出率。
ConvolutionCode0	比率 1/2 到 1/7 的卷积代码。 所有卷积代码（如果已用）的长度（二进制）必须等于约束长度值。
ConvolutionCode1	比率 1/2 到 1/7 的卷积代码。
ConvolutionCode2	比率 1/3 到 1/7 的卷积代码。
ConvolutionCode3	比率 1/4 到 1/7 的卷积代码。
ConvolutionCode4	比率 1/5 到 1/7 的卷积代码。
ConvolutionCode5	比率 1/6 到 1/7 的卷积代码。
ConvolutionCode6	比率 1/7 的卷积代码。

注释：在 C 语言仿真期间检查参数以验证模板参数配置是否合法。

实参

表 66：实参

实参	描述
inputData	要编码的单比特数据流。

表 66: 实参 (续)

实参	描述
outputData	已编码的数据流。除非 Punctured=true（位宽为 1）或 DualOutput=true（位宽为 2），否则位宽为 OutputRate。

返回值

- 不适用（void 函数）

受支持的基本数据类型

- ap_uint

输入数据假定

- 无

viterbi_decoder

摘要

```
template<
    int ConstraintLength,
    int TracebackLength,
    bool HasEraseInput,
    bool SoftData,
    int InputDataWidth,
    int SoftDataFormat,
    int OutputRate,
    int ConvolutionCode0,
    int ConvolutionCode1,
    int ConvolutionCode2,
    int ConvolutionCode3,
    int ConvolutionCode4,
    int ConvolutionCode5,
    int ConvolutionCode6>
class viterbi_decoder {
public:
    viterbi_decoder();
    ~viterbi_decoder();
    void operator()(stream<
viterbi_decoder_input<OutputRate, InputDataWidth, HasEraseInput> > &inputData,
    stream< ap_uint<1> > &outputData)
```

描述

- 执行卷积编码数据流的维特比 (Viterbi) 解码
- 支持硬数据或软数据
- 支持软数据格式：偏移二进制码和符号量值
- 支持擦除（打孔）

参数

表 67：参数

模板参数	描述
ConstraintLength	约束长度 K。支持的范围为 3 到 9。
TracebackLength	解码期间要通过网格回溯的状态数。使用至少 ConstraintLength 的 6 倍值，或者针对已打孔的代码，则使用至少 ConstraintLength 的 12 倍值。
HasEraseInput	当值为 true 时，在核上存在 Erase 输入，用于标记已打孔代码中的擦除（空符号）。
SoftData	当值为 true 时，函数接受软（多位）输入数据。
InputDataWidth	指定输入数据的宽度。针对硬数据设置为 1，针对软数据设置为 3-5。
SoftDataFormat	指定软数据格式。0 -> 符号量值，1 -> 偏移二进制码
OutputRate	指定匹配的卷积编码器的输出率。判定解码器的输入总线数量。
ConvolutionCode0	比率 1/2 到 1/7 的卷积代码。 所有卷积代码（如果已用）的长度（二进制）必须等于约束长度值。
ConvolutionCode1	比率 1/2 到 1/7 的卷积代码。
ConvolutionCode2	比率 1/3 到 1/7 的卷积代码。
ConvolutionCode3	比率 1/4 到 1/7 的卷积代码。
ConvolutionCode4	比率 1/5 到 1/7 的卷积代码。
ConvolutionCode5	比率 1/6 到 1/7 的卷积代码。
ConvolutionCode6	比率 1/7 的卷积代码。

注释：在 C 语言仿真期间检查参数以验证模板参数配置是否合法。

实参

表 68：实参

实参	描述
inputData	随附 Erase 信号（如果使用打孔代码）的卷积编码数据流。数据总线为位宽为 OutputRate*InputDataWidth。擦除总线位宽为 OutputRate。
outputData	已解码的单比特数据流。

返回值

- 不适用（void 函数）

受支持的基本数据类型

- 输入
 - ap_uint

请参阅 hls_viterbi_decoder.h 中的 viterbi_decoder_input 结构体定义以获取详细信息。

- 输出
 - ap_uint

输入数据假定

- 无

atan2

摘要

```
template <
    int PhaseFormat,
    int InputWidth,
    int OutputWidth,
    int RoundMode>
void atan2(const typename atan2_input<InputWidth>::cartesian &x,
           typename atan2_output<OutputWidth>::phase &atanX)
```

描述

- 基于 CORDIC 的双实参反正切函数的定点实现
- 可配置的输入和输出宽度
- 可配置的相位格式
- 可配置的舍入模式

参数

表 69：参数

模板参数	描述
PhaseFormat	选择以弧度还是缩放弧度（ $\pi * 1$ 弧度）来表述相位。
InputWidth	定义总体输入数据宽度。
OutputWidth	定义总体输出数据宽度。
RoundMode	选择应用于输出数据的舍入模式： 0 = 截断 (Truncate) 1 = 取最近的正无穷 (Round-to-positive-infinity) 2 = 取最近的正负无穷 (Round-to-positive-and-negative-infinity) 3 = 取最近的偶数 (Round-to-nearest-even)

注释：在 C 语言仿真期间检查参数以验证模板参数配置是否合法。

实参

表 70：实参

实参	描述
x	输入数据含 2 个整数位和 InputWidth-2 个小数位，范围为 [-1,1]。
atanX	x 的四象限反正切值，含 3 个整数位和 OutputWidth-3 个小数位，范围为 [-1,1]。

返回值

- 不适用 (void 函数)

受支持的基本数据类型

- 输入
 - std::complex<ap_fixed>
请参阅 hls_cordic_functions.h 中的 cordic_inputs 结构体的定义和 hls_atan2_cordic.h 中的 atan2_input 结构体定义，以获取详细信息。
- 输出
 - ap_fixed
请参阅 hls_cordic_functions.h 中的 cordic_outputs 结构体定义和 hls_atan2_cordic.h 中的 atan2_output 结构体定义，以获取详细信息。

输入数据假定

- 无

sqrt**摘要**

```
template <
    int DataFormat,
    int InputWidth,
    int OutputWidth,
    int RoundMode>
void sqrt(const typename sqrt_input<InputWidth, DataFormat>::in &x,
          typename sqrt_output<OutputWidth, DataFormat>::out &sqrtX)
```

描述

- 基于 CORDIC 的平方根的定点实现
- 支持无符号小数或无符号整数数据格式
- 可配置的舍入模式

参数

表 71：参数

模板参数	描述
DataFormat	选择无符号小数（整数位宽为 1）或无符号整数格式。
InputWidth	定义总体输入数据宽度。
OutputWidth	定义总体输出数据宽度。
RoundMode	选择应用于输出数据的舍入模式： 0 = 截断 (Truncate) 1 = 取最近的正无穷 (Round-to-positive-infinity) 2 = 取最近的正负无穷 (round-to-positive-and-negative-infinity); 3 = 取最近的偶数 (Round-to-nearest-even)

注释：在 C 语言仿真期间检查参数以验证模板参数配置是否合法。

实参

表 72：实参

实参	描述
x	输入数据。
sqrtX	输入数据的平方根。

返回值

- 不适用（void 函数）

受支持的基本数据类型

- 输入
 - ap_ufixed
 - ap_uint

请参阅 hls_cordic_functions.h 中的 cordic_inputs 结构体定义和 hls_sqrt_cordic.h 中的 sqrt_input 结构体定义，以获取详细信息。
- 输出
 - ap_ufixed
 - ap_uint

请参阅 hls_cordic_functions.h 中的 cordic_inputs 结构体定义和 hls_sqrt_cordic.h 中的 sqrt_input 结构体定义，以获取详细信息。

输入数据假定

- 无

cmpy

摘要

- 标量接口

```
template <
    class Architecture,
    int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1,
    int W2, int I2, ap_q_mode Q2, ap_o_mode O2, int N2>
void cmpy (const ap_fixed<W1, I1, Q1, O1, N1> &ar,
    const ap_fixed<W1, I1, Q1, O1, N1> &ai,
    const ap_fixed<W1, I1, Q1, O1, N1> &br,
    const ap_fixed<W1, I1, Q1, O1, N1> &bi,
    ap_fixed<W2, I2, Q2, O2, N2> &pr,
    ap_fixed<W2, I2, Q2, O2, N2> &pi);
```

- std::complex 接口

```
template <
    class Architecture,
    int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1,
    int W2, int I2, ap_q_mode Q2, ap_o_mode O2, int N2>
void cmpy (const std::complex< ap_fixed<W1, I1, Q1, O1, N1> > &a,
    const std::complex< ap_fixed<W1, I1, Q1, O1, N1> > &b,
    std::complex< ap_fixed<W2, I2, Q2, O2, N2> > &p);
```

描述

- 执行定点复数乘法
- 实现 3 乘数结构或 4 乘数结构
- 支持标量接口或 std::complex 接口

参数

表 73：参数

模板参数	描述
架构	选择 3 乘数架构或 4 乘数架构。指定使用 CmpyThreeMult 结构体或 CmpyFourMult 结构体。
W1、I1、Q1、O1 和 N1	被乘数和乘数定点参数。
W2、I2、Q2、O2 和 N2	乘积的定点参数。

实参

表 74：标量接口实参

实参	描述
ar	被乘数实数部分
ai	被乘数虚数部分
br	乘数实数部分

表 74：标量接口实参 (续)

实参	描述
bi	乘数虚数部分
pr	乘积实数部分
pi	乘积虚数部分

表 75：std::complex 接口实参

实参	描述
a	被乘数
b	乘数
p	产品

返回值

- 不适用 (void 函数)

受支持的基本数据类型

- ap_fixed
- std::complex<ap_fixed>

输入数据假定

- 无

HLS DSP 设计示例

Vivado HLS DSP 设计示例提供了基本测试激励文件，并演示了如何将每个函数参数化和例化。这些设计示例为每个函数提供了 1 个或多个示例。

要从“Welcome”页面打开 Vivado HLS 设计示例，请单击“Open Example Project”。在“示例 (Examples)”向导中，从“Design Examples”→“dsp”文件夹选择设计。

注释：调用 Vivado HLS GUI 时会显示“Welcome”页面。您可随时选择“Help”→“Welcome”来访问此页面。

您还可从以下 Vivado Design Suite 安装位置直接打开设计示例：Vivado_HLS\<version>\examples\design\dsp。

每个示例都包含以下文件：

- <example>.cpp：用于例化库类的顶层综合封装。
- <example>.h：用于定义参数值的头文件。
- <example>_tb.cpp：用于试用顶层综合封装的基本测试激励文件。
- run_hls.tcl：用于设置示例 Vivado HLS 工程的 Tcl 命令：

```
vivado_hls -f run_hls.tcl
```

注释：部分设计示例还包含 directives.tcl 文件，其中提供了额外 Tcl 命令用于应用最优化和实现指令。

HLS SQL 库函数

本节对 Vivado HLS SQL 库函数进行了解释。

hls_alg::sha224

摘要

```
namespace hls_alg {
  template <typename msg_T, typename hash_T>
  void sha224(hls::stream<msg_T>& msg_strm, uint64_t len,
    hls::stream<hash_T>&
    hash_strm);
}
```

描述

- 计算输入消息的 SHA-224 散列值。
- 消息长度以字节数为单位来指定。
- 提供多种可选输入流宽度。

参数

表 76: hls_alg::sha224 参数

参数	描述
msg_T	输入流数据类型。
hash_T	输出流数据类型。

实参

表 77: hls_alg::sha224 实参

实参	描述
msg_strm	输入消息流。
len	消息长度（字节）。
hash_strm	输出散列流。 当 hash_T 为无符号的 char 或无符号的 int 时，调用程序应分别读取该流 28 次或 7 次以获取完整的 224 位消息。

返回值

- 不适用（void 函数）

受支持的数据类型

- msg_T: 无符号的 char。

- hash_T：无符号的 char、无符号的 int 和 ap_uint<224>。

输入数据假定

- 假定 msg_strm 包含充足的数据（len 字节），否则该函数将阻止此数据。

hls_alg::sha256

摘要

```
namespace hls_alg {
template <typename msg_T, typename hash_T>
void sha256(hls::stream<msg_T>& msg_strm, uint64_t len,
hls::stream<hash_T>&
hash_strm);
}
```

描述

- 计算输入消息的 SHA-256 散列值。
- 消息长度以字节数为单位来指定。
- 提供多种可选输入流宽度。

参数

表 78: hls_alg::sha256 参数

参数	描述
msg_T	输入流数据类型。
hash_T	输出流数据类型。

实参

表 79: hls_alg::sha256 实参

实参	描述
msg_strm	输入消息流。
len	消息长度（字节）。
hash_strm	输出散列流。 当 hash_T 为无符号的 char 或无符号的 int 时，调用程序应分别读取该流 32 次或 8 次以获取完整的 256 位消息。

返回值

- 不适用（void 函数）

受支持的数据类型

- msg_T：无符号的 char。
- hash_T：无符号的 char、无符号的 int 和 ap_uint<256>。

输入数据假定

- 假定 msg_strm 包含充足的数据（len 字节），否则该函数将阻止此数据。

hls_alg::sort**摘要**

```
namespace hls_alg {
template <typename T, uint64_t len>
void sort(hls::stream<T>& input, hls::stream<T>& output);
}
```

描述

- 以 hls::stream 格式对输入矢量进行排序。
- 矢量长度为模板参数 len。

参数

表 80: hls_alg::sort

参数	描述
T	流数据类型。
len	矢量长度（stream 格式），应为 2 次幂值。

实参

表 81: hls_alg::sort

实参	描述
input	输入流
output	输出流

返回值

- 不适用（void 函数）

受支持的数据类型

- T：含运算符“<”的定义的类
- len：2 次幂。

C 语言任意精度类型

本节讨论：

- Vivado HLS 为 C 语言设计提供的任意精度 (AP) 类型。
- `C int#w` 类型的关联函数。

编译 [u]int#W 类型

要使用 [u]int#W 类型，必须将 `ap_cint.h` 头文件包含在引用 [u]int#W 变量的所有源文件中。

编译使用这些类型的软件模型时，可能需要指定 Vivado HLS 头文件的位置，例如，为 `gcc` 编译添加 “`-I/<HLS_HOME>/include`” 选项。

声明/定义 [u]int#W 变量

现有有符号的 C 类型和无符号的 C 类型分别是：

- `int#w`
- `uint#w`

其中

- `#w` 用于指定要声明的变量总宽度。

用户定义的类型可使用 C/C++ ‘`typedef`’ 声明来创建，如下示例所示：

```
include "ap_cint.h" // use [u]int#W types

typedef uint128 uint128_t; // 128-bit user defined type
int96 my_wide_var; // a global variable declaration
```

允许的最大宽度为 1024 位。

从常量初始化和赋值（字面值）

[u]int#W 变量可通过针对原生整数数据类型支持的相同整数常量来加以初始化。此类常量为扩展至 [u]int#W 变量的完整宽度的 0 或符号。

```
#include "ap_cint.h"

uint15 a = 0;
uint52 b = 1234567890U;
uint52 c = 0o12345670UL;
uint96 d = 0x123456789ABCDEFULL;
```

如果位宽超过 64 位，则可使用以下函数。

apint_string2bits()

本节还介绍了相关函数的使用：

- `apint_string2bits_bin()`
- `apint_string2bits_oct()`


```
· apint_string2bits_hex()
```

这些函数用于将常量数字字符串转换为给定位宽 *N* 的对应值，该字符串是在基数（十进制、二进制、八进制、十六进制）的约束内指定的。对于任何基数，数值前带有减号用于表示负值。

```
int#W apint_string2bits[_radix](const char*, int N)
```

这用于构造所含值大于 C 语言允许的值的整数常量。虽然也可以使用较小的值，但此类值更易于通过现有 C 语言常量值构造来指定。

```
#include <stdio.h>
#include "ap_cint.h"

int128 a;

// Set a to the value hex 000000000000000000000000123456789ABCDEF0
a = apint_string2bits_hex( "-123456789ABCDEF", 128);
```

值也可从字符串内直接赋值。

apint_vstring2bits()

此函数可将数字字符串转换为给定位宽 *N* 的对应值，该字符串是在十六进制基数约束内指定的。该数值前可带有减号以表示负值。

这用于构造所含值大于 C 语言允许的值的整数常量。该函数通常在测试激励文件中用于读取来自文件的信息。

给定的文件 `test.dat` 包含以下数据：

```
123456789ABCDEF
-123456789ABCDEF
-5
```

测试记录文件中使用的函数提供以下值：

```
#include <stdio.h>
#include "ap_cint.h"

typedef data_t;

int128 test (
    int128 t a
) {
    return a+1;
}

int main () {
    FILE *fp;
    char vstring[33];

    fp = fopen(test.dat, r);

    while (fscanf(fp, %s, vstring) == 1) {

        // Supply function "test" with the following values
```

控制台 I/O 支持 (打印)

```
#include "ap_cint.h"

uint164 c = 0x123456789ABCDEFULL;

printf(  d%40d\n,c); // Signed integer in decimal format
// d -1985229329
printf( hd%40hd\n,c); // Short integer
// hd -12817
printf( ld%40ld\n,c); // Long integer
// ld 81985529216486895
printf(lld%40lld\n,c); // Long long integer
// lld 81985529216486895

printf(  u%40u\n,c); // Unsigned integer in decimal format
// u 2309737967
printf( hu%40hu\n,c);
// hu 52719
printf( lu%40lu\n,c);
// lu 81985529216486895
printf(llu%40llu\n,c);
// llu 81985529216486895

printf(  o%40o\n,c); // Unsigned integer in octal format
// o 21152746757
printf( ho%40ho\n,c);
// ho 146757
printf( lo%40lo\n,c);
// lo 4432126361152746757
printf(llo%40llo\n,c);
// llo 4432126361152746757

printf(  x%40x\n,c); // Unsigned integer in hexadecimal format [0-9a-f]
// x 89abcdef
printf( hx%40hx\n,c);
// hx cdef
printf( lx%40lx\n,c);
// lx 123456789abcdef
printf(llx%40llx\n,c);
// llx 123456789abcdef

printf(  X%40X\n,c); // Unsigned integer in hexadecimal format [0-9A-F]
// X 89ABCDEF
}
```

apint_print()

```
void apint_print(int#N value, int radix)
```

[illegible]

```
void apint_fprint(FILE* file, int#N value, int radix)
```

china.xilinx.com
435

标准二进制整数算术运算符通过重载可提供任意精度算法。以下运算符可采用 2 个 `[u]int#W` 操作数，或者也可采用 1 个 `[u]int#W` 类型和 1 个 C/C++ 基本整数数据类型（例如，`char`、`short` 或 `int`）。

在基于目标变量（或表达式）宽度应用符号位扩展、0 值填充或截断前，生成的值的宽度和符号有无取决于操作数的宽度和符号类型。对应每个运算符提供返回值的详细描述。

当表达式混用 `ap-[u]int` 与 C/C++ 基本整数类型时，C++ 类型假定采用如下宽度：

- `char`：8 位
- `short`：16 位
- `int`：32 位
- `long`：32 位
- `long long`：64 位

加法

```
[u]int#W::RType [u]int#W::operator + ([u]int#W op)
```

生成 2 个 `ap-[u]int` 的和值，或者生成 1 个 `ap-[u]int` 和 1 个 C/C++ 整数类型的和值。

和值的宽度为：

- 比 2 个操作数的宽度多 1 位
- 2 位（仅当较宽的操作数无符号而较窄的操作数有符号时）

如果任一操作数或两个操作数均为有符号类型，则所得的和作为有符号来处理。

减法

```
[u]int#W::RType [u]int#W::operator - ([u]int#W op)
```

- 生成 2 个整数的差值。
- 差值的宽度为：
 - 比 2 个操作数的宽度多 1 位
 - 2 位（仅当较宽的操作数无符号而较窄的操作数有符号时）
- 在赋值前已基于目标变量的宽度执行了符号位扩展、0 值填充或截断的情况下，适用此条件。
- 差值作为有符号来处理，与操作数符号类型无关。

乘法

```
[u]int#W::RType [u]int#W::operator * ([u]int#W op)
```

- 返回 2 个整数值的积。
- 积的宽度是操作数的宽度之和。
- 如果任一操作数类型为有符号，则积作为有符号类型来处理。

除法

```
[u]int#W::RType [u]int#W::operator / ([u]int#W op)
```

- 返回 2 个整数值 的商。
- 商的宽度为被除数的宽度（前提是除数为无符号类型）；否则为被除数的宽度加 1。
- 如果任一操作数类型为有符号，则商作为有符号类型来处理。

模数

```
[u]int#W::RType [u]int#W::operator % ([u]int#W op)
```

- 返回 2 个整数值 的整数除法的模数或余数。
- 模数的宽度为操作数宽度的最小值（前提是 2 个操作数符号类型相同）；如果除数类型为无符号，而被除数类型为有符号，则宽度为除数宽度加 1。
- 商的符号类型与被除数相同。

按位逻辑运算符

按位逻辑运算符返回的值的宽度全部为 2 个操作数的最大宽度。仅当 2 个操作数均为无符号时，才将其作为无符号来处理。否则将其作为有符号类型来处理。

根据表达式（而不是目标变量）的符号类型，可能发生符号位扩展（或 0 值填充）。

按位 OR

```
[u]int#W::RType [u]int#W::operator | ([u]int#W op)
```

返回 2 个操作数的按位 OR。

按位 AND

```
[u]int#W::RType [u]int#W::operator & ([u]int#W op)
```

返回 2 个操作数的按位 AND。

按位 XOR

```
[u]int#W::RType [u]int#W::operator ^ ([u]int#W op)
```

返回 2 个操作数的按位 XOR。

移位运算符

每个移位运算符都带有 2 个版本：1 个对应无符号右侧 (RHS) 操作数，另 1 个对应有符号 RHS。

向有符号 RHS 版本提供负值将导致移位运算方向反转，即按 RHS 操作数绝对值发生反向移位。

移位运算符返回的值的宽度与左侧 (LHS) 操作数相同。就像 C/C++ 一样，如果右移的 LHS 操作数为有符号类型，那么符号位将复制到最高有效位的位置，并保留 LHS 操作数的符号。

无符号的整数右移

```
[u]int#W [u]int#W::operator >>(ap_uint<int_W2> op)
```

整数右移

```
[u]int#W [u]int#W::operator >>(ap_int<int_W2> op)
```

无符号的整数左移

```
[u]int#W [u]int#W::operator <<(ap_uint<int_W2> op)
```

整数左移

```
[u]int#W [u]int#W::operator <<(ap_int<int_W2> op)
```



注意！ 将左移运算符结果赋值到更宽的目标变量时，可能丢失部分或全部信息。赛灵思建议您将移位表达式显式强制转换为目标类型以避免发生意外行为。

复合赋值运算符

Vivado HLS 支持复合赋值运算符：

- *=
- /=
- %=
- +=
- -=
- <<=
- >>=
- &=
- ^=
- =

首先对 RHS 表达式求值，然后作为 RHS 操作数提供给基本运算符。结果将重新赋值给 LHS 变量。表达式大小、符号类型 and 可能的符号位扩展或截断规则适用方式与上述相关运算所述规则相同。

关系运算符

Vivado HLS 支持所有关系运算符。根据比较结果，这些运算符会返回布尔值。您可将 `ap_[u]int` 类型的变量与含有以下运算符的 C/C++ 基本整数类型进行比较。

等于

```
bool [u]int#W::operator == ([u]int#W op)
```

不等于

```
bool [u]int#W::operator != ([u]int#W op)
```

小于

```
bool [u]int#W::operator < ([u]int#W op)
```

大于

```
bool [u]int#W::operator > ([u]int#W op)
```

小于或等于

```
bool [u]int#W::operator <= ([u]int#W op)
```

大于或等于

```
bool [u]int#W::operator >= ([u]int#W op)
```

位级运算：支持函数

`[u]int#W` 类型允许以位级精确性表述变量。通常最好使用硬件算法来执行位级运算。Vivado HLS 可提供以下函数来启用此操作。

位操作

以下方法有助于对 `ap_[u]int` 类型变量中存储的值执行常见位级运算。

长度

`apint_bitwidthof()`

```
int apint_bitwidthof(type_or_value)
```

返回整数值，以任意精度整数值提供位数。可配合类型或值一起使用。

```
int5 Var1, Res1;

Var1 = -1;
Res1 = apint_bitwidthof(Var1); // Res1 is assigned 5
Res1 = apint_bitwidthof(int7); // Res1 is assigned 7
```

串联

apint_concatenate()

```
int#(N+M) apint_concatenate(int#N first, int#M second)
```

串联 2 个 [u]int#W 变量。返回的值的宽度为操作数的宽度之和。

High 和 Low 实参分别置于结果的高阶位和低阶位。



建议：为避免意外结果，请在串联前将 C 原生类型（包括整数字面值）强制转换为相应的 [u]int#W 类型。

位选择

apint_get_bit()

```
int apint_get_bit(int#N source, int index)
```

从任意精度整数值中选择 1 个位，并将其返回。

源变量必须为 [u]int#W 类型。索引实参必须为 int 值。它用于指定要选择的位的索引。最低有效位为索引 0。允许的最高索引为此 [u]int#W 的位宽减 1。

设置位值

apint_set_bit()

```
int#N apint_set_bit(int#N source, int index, int value)
```

- 将 [u]int#W 实例源的指定位和索引设置为指定的值（0 或 1）。

范围选择

apint_get_range()

```
int#N apint_get_range(int#N source, int high, int low)
```

- 返回以实参指定的位的范围来表示的值。
- High 实参用于指定此范围的最高有效位 (MSB) 的位置。
- Low 实参用于指定此范围的最低有效位 (LSB) 的位置。

- 源变量的 LSB 为位置 0。如果 High 实参的值小于 Low 的值，则会逆序返回这些位。

设置范围值

apint_set_range()

```
int#N apint_set_range(int#N source, int high, int low, int#M part)
```

- 将 High 与 Low 之间的源变量指定位数指定为 part 的值。

位减缩

AND 减缩

apint_and_reduce()

```
int apint_and_reduce(int#N value)
```

- 对值中所有的位应用 AND 运算。
- 返回生成的单个位作为整数值（可强制转换为布尔值）。

```
int5 Var1, Res1;

Var1= -1;
Res1 = apint_and_reduce(Var1); // Res1 is assigned 1

Var1= 1;
Res1 = apint_and_reduce(Var1); // Res1 is assigned 0
```

- 等效于与 -1 相比较。如果匹配，则返回 1。如果不匹配，则返回 0。另一种解释是检查所有位是否均为 1。

OR 减缩

apint_or_reduce()

```
int apint_or_reduce(int#N value)
```

- 对值中所有的位应用 OR 运算。
- 返回生成的单个位作为整数值（可强制转换为布尔值）。
- 等效于与 0 比较，如果匹配，则返回 0，否则返回 1。

```
int5 Var1, Res1;

Var1= 1;
Res1 = apint_or_reduce(Var1); // Res1 is assigned 1

Var1= 0;
Res1 = apint_or_reduce(Var1); // Res1 is assigned 0
```

XOR 减缩

apint_xor_reduce()

```
int apint_xor_reduce(int#N value)
```

- 对值中所有的位应用 XOR 运算。
- 返回生成的单个位作为整数值（可强制转换为布尔值）。
- 等效于计算字词中 1 的数量。此运算：
 - 如果存在偶数，则返回 0。
 - 如果存在奇数（偶数奇偶校验），则返回 1。

```
int5 Var1, Res1;

Var1= 0;
Res1 = apint_xor_reduce(Var1); // Res1 is assigned 0

Var1= 1;
Res1 = apint_xor_reduce(Var1); // Res1 is assigned 1
```

NAND 减缩

apint_nand_reduce()

```
int apint_nand_reduce(int#N value)
```

- 对值中所有的位应用 NAND 运算。
- 返回生成的单个位作为整数值（可强制转换为布尔值）。
- 等效于将该值与 -1 比较（全部为 1），如果匹配，则返回 false，否则返回 true。

```
int5 Var1, Res1;

Var1= 1;
Res1 = apint_nand_reduce(Var1); // Res1 is assigned 1

Var1= -1;
Res1 = apint_nand_reduce(Var1); // Res1 is assigned 0
```

NOR 减缩

apint_nor_reduce()

```
int apint_nor_reduce(int#N value)
```

- 对值中所有的位应用 NOR 运算。
- 返回生成的单个位作为整数值（可强制转换为布尔值）。

- 等效于将该值与 0 比较（全部为 0），如果匹配，则返回 true，否则返回 false。

```
int5 Var1, Res1;

Var1= 0;
Res1 = apint_nor_reduce(Var1); // Res1 is assigned 1

Var1= 1;
Res1 = apint_nor_reduce(Var1); // Res1 is assigned 0
```

XNOR 减缩

apint_xnor_reduce()

```
int apint_xnor_reduce(int#N value)
```

- 对值中所有的位应用 XNOR 运算。
- 返回生成的单个位作为整数值（可强制转换为布尔值）。
- 等效于计算字词中 1 的数量。
- 此运算：
 - 如果存在奇数，则返回 0。
 - 如果存在偶数（奇数奇偶校验），则返回 1。

```
int5 Var1, Res1;

Var1= 0;
Res1 = apint_xnor_reduce(Var1); // Res1 is assigned 1

Var1= 1;
Res1 = apint_xnor_reduce(Var1); // Res1 is assigned 0
```

C++ 任意精度类型

Vivado HLS 可提供 C++ 模板类 `ap_[u]int<>` 以实现任意精度（或位精确）整数数据类型，软件与硬件建模之间可实现一致的位精确行为。

该类可提供原生 C 整数类型允许的所有算术、按位、逻辑和关系运算符。此外，该类还可提供多种方法用于处理某些实用的硬件操作，例如允许对宽度大于 64 位的变量进行初始化和转换。以下提供了所有运算符和类方法的详细信息。

编译 `ap_[u]int<>` 类型

要使用 `ap_[u]int<>` 类，必须将 `ap_int.h` 头文件包含在引用 `ap_[u]int<>` 变量的所有源文件中。

编译使用这些类的软件模型时，可能需要指定 Vivado HLS 头文件的位置，例如，为 g++ 编译添加 `-I/<HLS_HOME>/include` 选项。

声明/定义 ap_[u] 变量

有符号的类与无符号的类不同：

- ap_int<int_W> (有符号)
- ap_uint<int_W> (无符号)

int_W 模板参数可指定要声明的变量的总宽度。

用户定义的类型可使用 C/C++ typedef 声明来创建，如以下示例所示：

```
include "ap_int.h" // use ap_[u]fixed<> types

typedef ap_uint<128> uint128_t; // 128-bit user defined type
ap_int<96> my_wide_var; // a global variable declaration
```

允许的默认最大宽度为 1024 位。此默认设置可通过在包含 ap_int.h 头文件之前定义含正整数（小于或等于 32768）的 AP_INT_MAX_W 宏来覆盖。



注意！ AP_INT_MAX_W 的值设置过高可能导致软件编译缓慢、运行时间延长。

以下是覆盖 AP_INT_MAX_W 的示例：

```
#define AP_INT_MAX_W 4096 // Must be defined before next line
#include "ap_int.h"

ap_int<4096> very_wide_var;
```

从常量初始化和赋值（字面值）

类构造函数和赋值运算符重载后，允许使用标准 C/C++ 整数字面值对 ap_[u]fixed<> 变量进行初始化和赋值。

这种向 ap_[u]fixed<> 变量赋值的方法受到 C++ 限制和运行软件的系统的影响。这通常导致整数字面值（例如，LL 或 ULL 后缀的整数字面值）限制为 64 位。

为支持使用宽度超过 64 位的值进行赋值，ap_[u]fixed<> 类提供了允许从任意长度（小于或等于变量宽度）的字符串进行初始化的构造函数。

默认情况下，提供的字符串解释为十六进制值，前提是它仅包含有效的十六进制数字（即，0-9 和 a-f）。要从此类字符串进行赋值，需对字符串进行显式 C++ 样式强制转换，将其转换为适当的类型。

以下是初始化和赋值示例（包括大于 64 位的值）：

```
ap_int<42> a_42b_var(-1424692392255LL); // long long decimal format
a_42b_var = 0x14BB648B13FLL; // hexadecimal format

a_42b_var = -1; // negative int literal sign-extended to full width

ap_uint<96> wide_var( "76543210fedcba9876543210" , 16); // Greater than 64-bit
wide_var = ap_int<96>( "0123456789abcdef01234567" , 16);
```

注释：为避免协同仿真期间出现意外行为，请勿初始化 `ap_uint<N> a = {0}`。

可显式指示 `ap_[u]<>` 构造函数将该字符串解释为以基数 2、8、10 或 16 进制格式来显示数字。具体方法为在构造函数调用中添加相应的基数值作为第 2 个参数。

如果字符串字面值包含的任何字符对于指定的基数属于无效数字，那么会发生编译错误。

以下示例使用不同基数格式：

```
ap_int<6> a_6bit_var( "101010" , 2); // 42d in binary format
a_6bit_var = ap_int<6>( "40" , 8); // 32d in octal format
a_6bit_var = ap_int<6>( "55" , 10); // decimal format
a_6bit_var = ap_int<6>( "2A" , 16); // 42d in hexadecimal format

a_6bit_var = ap_int<6>( "42" , 2); // COMPILER-TIME ERROR! "42" is not
binary
```

字符串中已编码的数字的基数还可通过构造函数来推断，前提是以 1 个零 (0) 作为其前缀，后接以下任一字符：b、o 或 x。前缀 0b、0o 和 0x 分别对应于二进制、八进制和十六进制格式。

以下示例使用其它初始化程序字符串格式：

```
ap_int<6> a_6bit_var( "0b101010" , 2); // 42d in binary format
a_6bit_var = ap_int<6>( "0o40" , 8); // 32d in octal format
a_6bit_var = ap_int<6>( "0x2A" , 16); // 42d in hexadecimal format

a_6bit_var = ap_int<6>( "0b42" , 2); // COMPILER-TIME ERROR! "42" is not
binary
```

如果位宽大于 53 位，必须使用如下字符串对 `ap_[u]fixed` 值进行初始化：

```
ap_ufixed<72,10> Val( "2460508560057040035.375" );
```

控制台 I/O 支持（打印）

正如 `ap_[u]fixed<>` 变量初始化和赋值一样，Vivado HLS 支持打印显示超 64 位的值。

使用 C++ 标准输出流

输出存储在 `ap_[u]int` 变量中的任意值的最简单方法是使用 C++ 标准输出流：

```
std::cout (#include <iostream> 或 <iostream.h>)
```

流插入运算符 (<<) 已重载，可正确输出任意给定 `ap_[u]fixed` 变量的全部可能值。以下流操控程序同样受支持：

- dec（十进制）
- hex（十六进制）
- oct（八进制）

这些操控程序允许按指定方式对值进行格式化。

以下示例使用 `cout` 来打印值：

```
#include <iostream.h>
// Alternative: #include <iostream>

ap_ufixed<72> Val( "10fedcba9876543210" );

cout << Val << endl; // Yields: "313512663723845890576"
cout << hex << val << endl; // Yields: "10fedcba9876543210"
cout << oct << val << endl; // Yields: "41773345651416625031020"
```

使用标准 C 语言库

您还可使用标准 C 语言库 (`#include <stdio.h>`) 来打印大于 64 位的值：

1. 使用 `ap_[u]fixed` 类方法 `to_string()` 将值转换为 C++ `std::string`。
2. 使用 `std::string` 类方法 `c_str()` 将结果转换为以 `null` 结尾的 C 语言字符串。

可选实参 1（指定基数）

您可为 `ap_[u]int::to_string()` 方法传递 1 个可选实参以指定期望的数字格式的基数。有效的基数实参值包括：

- 2（二进制，默认值）
- 8（八进制）
- 10（十进制）
- 16（十六进制）

可选实参 2（打印为有符号的值）

`ap_[u]int::to_string()` 的第 2 个可选实参可指定是否将非十进制格式打印为有符号的值。该实参为布尔值。默认值为 `false`，导致非十进制格式打印为无符号的值。

以下示例使用 `printf` 来打印值：

```
ap_int<72> Val( "80fedcba9876543210" );

printf( "%s\n", Val.to_string().c_str()); // => "80FEDCBA9876543210"
printf( "%s\n", Val.to_string(10).c_str()); // => "-2342818482890329542128"
printf( "%s\n", Val.to_string(8).c_str()); // =>
"401773345651416625031020"
printf( "%s\n", Val.to_string(16, true).c_str()); // =>
"-7F0123456789ABCDF0"
```

涉及 `ap_[u]<>` 类型的表达式

类型为 `ap_[u]<>` 的变量通常可在涉及 C/C++ 运算符的表达式中随意使用。这可能导致某些意外行为。此类行为将在下文中详细讲解。

从较窄的变量赋值到较宽的变量时的零位扩展和符号位扩展

将位宽较窄的有符号 (`ap_int<>`) 变量赋值到较宽的变量时，该值将采用符号位扩展以扩展至目标变量的宽度，这与其是否有符号无关。

同样，无符号的源变量在赋值前将采用零位扩展。

可能需要进行显式源变量类型强制转换，以确保赋值行为符合期望。请参阅以下示例：

```
ap_uint<10> Result;  
  
ap_int<7> Val1 = 0x7f;  
ap_uint<6> Val2 = 0x3f;  
  
Result = Val1; // Yields: 0x3ff (sign-extended)  
Result = Val2; // Yields: 0x03f (zero-padded)  
  
Result = ap_uint<7>(Val1); // Yields: 0x07f (zero-padded)  
Result = ap_int<6>(Val2); // Yields: 0x3ff (sign-extended)
```

从较宽的变量赋值到较窄的变量时的截断

将较宽的源变量赋值到较窄的变量会导致值被截断。超出目标变量的最高有效位 (MSB) 位置的所有位都将丢失。

截断期间对于符号信息不进行任何特殊处理。这可能导致出现意外行为。显式强制类型转换可有助于避免此类意外行为。

方法与运算符分类

`ap_[u]int` 类型不支持从 `wide ap_[u]int` (>64 位) 隐式转换为内置 C/C++ 整数类型。例如，由于在 `if` 语句中从 `ap_int[65]` 到 `bool` 的隐式强制转换返回 0，因此以下代码示例返回 `s1`。

```
bool nonzero(ap_uint<65> data) {  
    return data; // This leads to implicit truncation to 64b int  
}  
  
int main() {  
    if (nonzero((ap_uint<65>)1 << 64)) {  
        return 0;  
    }  
    printf(FAIL"\n");  
    return 1;  
}
```

要将较宽的 `ap_[u]int` 类型转换为内置整数，请使用 `ap_[u]int` 类型包含的显式转换函数：

- `to_int()`
- `to_long()`
- `to_bool()`

一般情况下，支持针对 `ap_[u]int` 类型使用运算符重载来执行在原生 C/C++ 整数数据类型上执行的任意有效操作。

除这些重载运算符外，还包含部分特定于类的运算符和方法以简化位级运算。

二进制算术运算符

标准二进制整数算术运算符通过重载可提供任意精度算法。这些运算符采用：

- 2 个 `ap_[u]int` 操作数，或者
- 1 个 `ap_[u]int` 类型和 1 个 C/C++ 基本整数数据类型

例如：

- `char`
- `short`
- `int`

在基于目标变量（或表达式）宽度应用符号位扩展、0 值填充或截断前，生成的值的宽度和符号有无取决于操作数的宽度和符号类型。对应每个运算符提供返回值的详细描述。

当表达式混用 `ap_[u]int` 与 C/C++ 基本整数类型时，C++ 类型假定采用如下宽度：

- `char` (8 位)
- `short` (16 位)
- `int` (32 位)
- `long` (32 位)
- `long long` (64 位)

加法

```
ap_(u)int::RType ap_(u)int::operator + (ap_(u)int op)
```

返回以下值之和：

- 2 个 `ap_[u]int`，或
- 1 个 `ap_[u]int` 和 1 个 C/C++ 整数类型

和值的宽度为：

- 比 2 个操作数的宽度多 1 位，或者
- 2 位（仅当较宽的操作数无符号而较窄的操作数有符号时）

如果任一操作数或两个操作数均为有符号类型，则所得的和作为有符号来处理。

减法

```
ap_(u)int::RType ap_(u)int::operator - (ap_(u)int op)
```

返回 2 个整数之差。

差值的宽度为：

- 比 2 个操作数的宽度多 1 位，或者
- 2 位（仅当较宽的操作数无符号而较窄的操作数有符号时）

在赋值前已基于目标变量的宽度执行了符号位扩展、0 值填充或截断的情况下，可满足此条件。

差值作为有符号来处理，与操作数符号类型无关。

乘法

```
ap_(u)int::RType ap_(u)int::operator * (ap_(u)int op)
```

返回 2 个整数值积。

积的宽度是操作数的宽度之和。

如果任一操作数类型为有符号，则积作为有符号类型来处理。

除法

```
ap_(u)int::RType ap_(u)int::operator / (ap_(u)int op)
```

返回 2 个整数值商。

商的宽度为被除数的宽度（前提是除数为无符号类型）。否则，其宽度为被除数宽度加 1。

如果任一操作数类型为有符号，则商作为有符号类型来处理。

模数

```
ap_(u)int::RType ap_(u)int::operator % (ap_(u)int op)
```

返回 2 个整数值整数除法的模数或余数。

模数的宽度为操作数宽度的最小值（前提是 2 个操作数符号类型相同）。

如果除数类型为无符号，而被除数类型为有符号，则宽度为除数宽度加 1。

商的符号类型与被除数相同。



重要提示！ Vivado HLS 对模数 (%) 运算符进行综合会导致对生成的 RTL 中已适当参数化的赛灵思 LogiCORE 被除数核进行例化。

以下是算术运算符的示例：

```
ap_uint<71> Rslt;

ap_uint<42> Val1 = 5;
ap_int<23> Val2 = -8;

Rslt = Val1 + Val2; // Yields: -3 (43 bits) sign-extended to 71 bits
Rslt = Val1 - Val2; // Yields: +3 sign extended to 71 bits
Rslt = Val1 * Val2; // Yields: -40 (65 bits) sign extended to 71 bits
Rslt = 50 / Val2; // Yields: -6 (33 bits) sign extended to 71 bits
Rslt = 50 % Val2; // Yields: +2 (23 bits) sign extended to 71 bits
```

按位逻辑运算符

按位逻辑运算符返回的值的宽度全部为 2 个操作数的最大宽度。仅当 2 个操作数均为无符号时，才将其作为无符号来处理。否则，将其作为有符号类型来处理。

根据表达式（而不是目标变量）的符号类型，可能发生符号位扩展（或 0 值填充）。

按位 OR

```
ap_(u)int::RType ap_(u)int::operator | (ap_(u)int op)
```

返回 2 个操作数的按位 OR。

按位 AND

```
ap_(u)int::RType ap_(u)int::operator & (ap_(u)int op)
```

返回 2 个操作数的按位 AND。

按位 XOR

```
ap_(u)int::RType ap_(u)int::operator ^ (ap_(u)int op)
```

返回 2 个操作数的按位 XOR。

一元运算符

加法

```
ap_(u)int ap_(u)int::operator + ()
```

返回 `ap_[u]int` 操作数的自我复制。

减法

```
ap_(u)int::RType ap_(u)int::operator - ()
```

运行以下操作：

- 操作数的负值，位宽相同（前提是其类型为有符号）；或者
- 如果操作数无符号，则位宽为其位宽加 1。

返回值类型始终为有符号。

按位反转

```
ap_(u)int::RType ap_(u)int::operator ~ ()
```

返回操作数的按位 NOT，位宽相同，符号类型相同。

逻辑反转

```
bool ap_(u)int::operator ! ()
```

仅当操作数不等于零 (0) 时，返回布尔值 `false`。

如果操作数等于零 (0)，则返回布尔值 `true`。

三元运算符

对标准 C 语言 `int` 类型使用三元运算符时，必须从一种类型显式强制转换为另一种类型，以确保 2 个结果的类型相同。例如：

```
// Integer type is cast to ap_int type
ap_int<32> testc3(int a, ap_int<32> b, ap_int<32> c, bool d) {
    return d?ap_int<32>(a):b;
}
// ap_int type is cast to an integer type
ap_int<32> testc4(int a, ap_int<32> b, ap_int<32> c, bool d) {
    return d?a+1:(int)b;
}
// Integer type is cast to ap_int type
ap_int<32> testc5(int a, ap_int<32> b, ap_int<32> c, bool d) {
    return d?ap_int<33>(a):b+1;
}
```

移位运算符

每个移位运算符都带有 2 个版本：

- 1 个版本对应于无符号的右侧 (RHS) 操作数
- 另 1 个版本对应于有符号的右侧 (RHS) 操作数

对有符号的 RHS 版本提供负值将导致移位操作方向反转。即按 RHS 操作数的绝对值发生反方向移位。

移位运算符返回的值的宽度与左侧 (LHS) 操作数相同。就像 C/C++ 一样，如果右移的 LHS 操作数为有符号类型，那么符号位将复制到最高有效位的位置，并保留 LHS 操作数的符号。

无符号的整数右移

```
ap_(u)int ap_(u)int::operator << (ap_uint<int_W2> op)
```

整数右移

```
ap_(u)int ap_(u)int::operator << (ap_int<int_W2> op)
```

无符号的整数左移

```
ap_(u)int ap_(u)int::operator >> (ap_uint<int_W2> op)
```

整数左移

```
ap_(u)int ap_(u)int::operator >> (ap_int<int_W2> op)
```



注意！ 将左移运算符结果赋值到更宽的目标变量时，可能丢失部分或全部信息。赛灵思建议您将移位表达式显式强制转换为目标类型以避免发生意外行为。

以下是移位运算示例：

```
ap_uint<13> Rslt;

ap_uint<7> Val1 = 0x41;

Rslt = Val1 << 6; // Yields: 0x0040, i.e. msb of Val1 is lost
Rslt = ap_uint<13>(Val1) << 6; // Yields: 0x1040, no info lost

ap_int<7> Val2 = -63;
Rslt = Val2 >> 4; //Yields: 0x1ffc, sign is maintained and extended
```

复合赋值运算符

Vivado HLS 支持复合赋值运算符：

- *=
- /=
- %=
- +=
- -=
- <<=
- >>=
- &=
- ^=
- |=

RHS 表达式首先进行求值，然后作为 RHS 操作数提供给基本运算符，其结果重新赋值给 LHS 变量。表达式大小、符号类型和可能的符号位扩展或截断规则适用方式与上述相关运算所述规则相同。

```
ap_uint<10> Val1 = 630;
ap_int<3> Val2 = -3;
ap_uint<5> Val3 = 27;

Val1 += Val2 - Val3; // Yields: 600 and is equivalent to:

// Val1 = ap_uint<10>(ap_int<11>(Val1) +
// ap_int<11>((ap_int<6>(Val2) -
// ap_int<6>(Val3))));
```

递增运算符和递减运算符

递增运算符和递减运算符均已提供。返回值的宽度全都与操作数相同，仅当 2 个操作数类型均为无符号时，返回值类型为无符号，否则为有符号。

递增前

```
ap_(u)int& ap_(u)int::operator ++ ()
```

返回操作数的递增值。

赋值操作数的递增值。

递增后

```
const ap_(u)int ap_(u)int::operator ++ (int)
```

返回将递增值赋值给操作数变量之前的操作数的值。

递减前

```
ap_(u)int& ap_(u)int::operator -- ()
```

返回操作数的递减值并将递减值赋值给操作数。

递减后

```
const ap_(u)int ap_(u)int::operator -- (int)
```

返回将递减值赋值给操作数变量之前的操作数的值。

关系运算符

Vivado HLS 支持所有关系运算符。根据比较结果，这些运算符会返回布尔值。您可将 `ap_[u]int` 类型的变量与含有以下运算符的 C/C++ 基本整数类型进行比较。

等于

```
bool ap_(u)int::operator == (ap_(u)int op)
```

不等于

```
bool ap_(u)int::operator != (ap_(u)int op)
```

小于

```
bool ap_(u)int::operator < (ap_(u)int op)
```

大于

```
bool ap_(u)int::operator > (ap_(u)int op)
```

小于或等于

```
bool ap_(u)int::operator <= (ap_(u)int op)
```

大于或等于

```
bool ap_(u)int::operator >= (ap_(u)int op)
```

其它类方法、运算符和数据成员

以下章节探讨了其它类方法、运算符和数据成员。

位级运算

以下方法有助于对 `ap_[u]int` 类型变量中存储的值执行常见位级运算。

长度

```
int ap_(u)int::length ()
```

返回整数，以提供 `ap_[u]int` 变量中的总位数。

串联

```
ap_concat_ref ap_(u)int::concat (ap_(u)int low)
ap_concat_ref ap_(u)int::operator , (ap_(u)int high, ap_(u)int low)
```

串联 2 个 `ap_[u]int` 变量，返回的值的宽度为操作数宽度之和。

High 和 Low 实参分别置于结果的高阶位和低阶位；`concat()` 方法用于将实参置于低阶位。

使用已重载的逗号运算符时，括号是必需的。逗号运算符版本还可显示在赋值的 LHS 上。



建议：为避免意外结果，请在串联前将 C/C++ 原生类型（包括整数字面值）强制转换为相应的 `ap_[u]int` 类型。

```
ap_uint<10> Rslt;

ap_int<3> Val1 = -3;
ap_int<7> Val2 = 54;

Rslt = (Val2, Val1); // Yields: 0x1B5
Rslt = Val1.concat(Val2); // Yields: 0x2B6
(Val1, Val2) = 0xAB; // Yields: Val1 == 1, Val2 == 43
```

位选择

```
ap_bit_ref ap_(u)int::operator [] (int bit)
```

从任意精度整数值中选择 1 个位，并将其返回。

返回的值为可用于设置或清除此 `ap_[u]int` 中的对应位的参考值。

位实参必须为 `int` 值。它用于指定要选择的位的索引。最低有效位为索引 0。允许的最高索引为此 `ap_[u]int` 的位宽减 1。

生成的 `ap_bit_ref` 类型表示引用该 `ap_[u]int` 实例的 1 个位（按位指定）。

范围选择

```
ap_range_ref ap_(u)int::range (unsigned Hi, unsigned Lo)
ap_range_ref ap_(u)int::operator () (unsigned Hi, unsigned Lo)
```

返回以实参指定的位的范围来表示的值。

H_i 实参用于指定此范围的最高有效位 (MSB) 的位置， L_o 用于指定最低有效位 (LSB)。

源变量的 LSB 为位置 0。如果 H_i 实参的值小于 L_o 的值，则会逆序返回这些位。

```
ap_uint<4> Rslt;

ap_uint<8> Val1 = 0x5f;
ap_uint<8> Val2 = 0xaa;

Rslt = Val1.range(3, 0); // Yields: 0xF
Val1(3,0) = Val2(3, 0); // Yields: 0x5A
Val1(3,0) = Val2(4, 1); // Yields: 0x55
Rslt = Val1.range(4, 7); // Yields: 0xA; bit-reversed!
```

AND 减缩

```
bool ap_(u)int::and_reduce ()
```

- 对此 $ap_ (u)int$ 中的所有位应用 AND 运算。
- 返回生成的单个位。
- 等效于将该值与 -1 比较（全部为 1），如果匹配，则返回 true，否则返回 false。

OR 减缩

```
bool ap_(u)int::or_reduce ()
```

- 对此 $ap_ (u)int$ 中的所有位应用 OR 运算。
- 返回生成的单个位。
- 等效于将该值与 0 比较（全部为 0），如果匹配，则返回 false，否则返回 true。

XOR 减缩

```
bool ap_(u)int::xor_reduce ()
```

- 对此 ap_int 中的所有位应用 XOR 运算。
- 返回生成的单个位。
- 等效于计算该值中 1 位的数量，如果计数结果为偶数，则返回 false，如果为奇数，则返回 true。

NAND 减缩

```
bool ap_(u)int::nand_reduce ()
```

- 对此 `ap_int` 中的所有位应用 NAND 运算。
- 返回生成的单个位。
- 等效于将该值与 -1 比较（全部为 1），如果匹配，则返回 `false`，否则返回 `true`。

NOR 减缩

```
bool ap_int::nor_reduce ()
```

- 对此 `ap_int` 中的所有位应用 NOR 运算。
- 返回生成的单个位。
- 等效于将该值与 0 比较（全部为 0），如果匹配，则返回 `true`，否则返回 `false`。

XNOR 减缩

```
bool ap_(u)int::xnor_reduce ()
```

- 对此 `ap_(u)int` 中的所有位应用 XNOR 运算。
- 返回生成的单个位。
- 等效于计算该值中 1 位的数量，如果计数结果为偶数，则返回 `true`，如果为奇数，则返回 `false`。

位减缩方法示例

```
ap_uint<8> Val = 0xaa;

bool t = Val.and_reduce(); // Yields: false
t = Val.or_reduce();       // Yields: true
t = Val.xor_reduce();      // Yields: false
t = Val.nand_reduce();     // Yields: true
t = Val.nor_reduce();      // Yields: false
t = Val.xnor_reduce();     // Yields: true
```

位反转

```
void ap_(u)int::reverse ()
```

将 `ap_[u]int` 实例的内容反转：

- LSB 变为 MSB。
- MSB 变为 LSB。

反转方法示例

```
ap_uint<8> Val = 0x12;
Val.reverse(); // Yields: 0x48
```

测试位值

```
bool ap_(u)int::test (unsigned i)
```

检查 `ap_(u)int` 实例的指定位是否是 1。

如果是，则返回 `true`，如果否，则返回 `false`。

测试方法示例

```
ap_uint<8> Val = 0x12;
bool t = Val.test(5); // Yields: true
```

设置位值

```
void ap_(u)int::set (unsigned i, bool v)
void ap_(u)int::set_bit (unsigned i, bool v)
```

将 `ap_(u)int` 实例的指定位设置为整数 `v` 的值。

设置位（设为 1）

```
void ap_(u)int::set (unsigned i)
```

将 `ap_(u)int` 实例的指定位设置为值 1（一）。

清除位（设为 0）。

```
void ap_(u)int::clear(unsigned i)
```

将 `ap_(u)int` 实例的指定位设置为值 0（零）。

对位取反

```
void ap_(u)int::invert(unsigned i)
```

对 `ap_(u)int` 实例的函数实参中指定的位取反。指定的位原始值如果为 1，则变为 0，反之亦然。

位设置、清除和取反方法示例：

```
ap_uint<8> Val = 0x12;
Val.set(0, 1); // Yields: 0x13
Val.set_bit(4, false); // Yields: 0x03
Val.set(7); // Yields: 0x83
Val.clear(1); // Yields: 0x81
Val.invert(4); // Yields: 0x91
```

右转

```
void ap_(u)int:: rrotate(unsigned n)
```

将 `ap_(u)int` 实例向右旋转 `n` 位。

左转

```
void ap_(u)int:: lrotate(unsigned n)
```

将 `ap_(u)int` 实例向左旋转 `n` 位。

```
ap_uint<8> Val = 0x12;
Val.rrotate(3); // Yields: 0x42
Val.lrotate(6); // Yields: 0x90
```

按位 NOT

```
void ap_(u)int:: b_not()
```

- 补足 `ap_(u)int` 实例的每个位。

```
ap_uint<8> Val = 0x12;
Val.b_not(); // Yields: 0xED
```

按位 NOT 示例

测试符号

```
bool ap_int:: sign()
```

- 检查 `ap_(u)int` 实例是否为负。
- 如果为负，则返回 `true`。
- 如果为正，则返回 `false`。

显式转换方法

转换为 C/C++ (u)int 型

```
int ap_(u)int::to_int ()
unsigned ap_(u)int::to_uint ()
```

- 返回 ap_[u]int 中包含的值对应的原生 C/C++（大部分系统上为 32 位）整数。
- 如果值大于 [unsigned] int 可显示的值，则发生截断。

转换为 C/C++ 64 位 (u)int 型

```
long long ap_(u)int::to_int64 ()
unsigned long long ap_(u)int::to_uint64 ()
```

- 返回 ap_[u]int 中包含的值对应的原生 C/C++ 64 位整数。
- 如果值大于 [unsigned] int 可显示的值，则发生截断。

转换为 C/C++ double 型

```
double ap_(u)int::to_double ()
```

- 返回 ap_[u]int 中包含的值对应的原生 C/C++ double 64 位浮点表示法。
- 如果 ap_[u]int 宽度超过 53 位（double 的尾数中的位数），生成的 double 所包含的值可能与期望的值不同。



建议：赛灵思建议显式调用成员函数，而不是使用 C 样式强制转换来将 ap_[u]int 转换为其它数据类型。

Sizeof

标准 C++ sizeof() 函数不应搭配 ap_[u]int 或者对象的其它类或实例一起使用。ap_int<> 数据类型为类，sizeof 会返回类或实例对象使用的存储空间。sizeof(ap_int<N>) 始终返回所使用的字节数。例如：

```
sizeof(ap_int<127>)=16
sizeof(ap_int<128>)=16
sizeof(ap_int<129>)=24
sizeof(ap_int<130>)=24
```

编译时间访问数据类型属性

ap_[u]int<> 类型随静态成员一并提供，此静态成员允许在编译时判定变量大小。数据类型随静态常量成员 width 一并提供，数据类型的宽度将自动赋值给该成员：

```
static const int width = _AP_W;
```

您可使用 `width` 数据成员来提取现有 `ap_[u]int<>` 数据类型的数据宽度，以在编译时创建另一个 `ap_[u]int<>` 数据类型。以下示例显示 `Res` 变量的大小定义为比 `Val1` 变量和 `Val2` 变量位宽大 1 位：

```
// Definition of basic data type
#define INPUT_DATA_WIDTH 8
typedef ap_int<INPUT_DATA_WIDTH> data_t;
// Definition of variables
data_t Val1, Val2;
// Res is automatically sized at compile-time to be 1-bit greater than data
type
data_t
ap_int<data_t::width+1> Res = Val1 + Val2;
```

这样可确保 Vivado HLS 对加法导致的位增长进行正确建模，即使为 `data_t` 更新 `INPUT_DATA_WIDTH` 的值也是如此。

C++ 任意精度定点类型

Vivado HLS 支持定点类型，定点类型支持轻松处理小数运算。定点运算的优势如下示例所示。

```
ap_fixed<11, 6> Var1 = 22.96875; // 11-bit signed word, 5 fractional bits
ap_ufixed<12,11> Var2 = 512.5; // 12-bit word, 1 fractional bit
ap_fixed<16,11> Res1; // 16-bit signed word, 5 fractional bits

Res1 = Var1 + Var2; // Result is 535.46875
```

即使 `Var1` 与 `Var2` 精度不同，但定点类型在执行运算前（此处为加法）仍可确保小数点正确对齐。您无需在 C 语言代码中执行任何操作来使小数点对齐。

用于存储任意定点运算结果的类型大小（包括整数位和小数位）必须足以存储完整结果。

否则，`ap_fixed` 类型会执行以下操作：

- 使处理溢出（前提是结果所含 MSB 多于指定类型支持的数量）
- 量化（或舍入，前提是结果所含 LSB 少于指定类型支持的数量）

`ap_[u]fixed` 类型可提供有关溢出和量化执行方式的多种不同选项。这些选项如下所述。

ap_[u]fixed 表示法

在 `ap[u]fixed` 类型中，定点值以位序列的形式来表示，具有对应二进制小数点的指定位置。

- 二进制小数点左侧的位表示值的整数部分。
- 二进制小数点右侧的位表示值的小数部分。

ap_[u]fixed type 定义如下：

```
ap_[u]fixed<int W,
int I,
ap_q_mode Q,
ap_o_mode O,
ap_sat_bits N>;
```

量化模式

舍入到正无穷	AP_RND
舍入到 0	AP_RND_ZERO
舍入到负无穷	AP_RND_MIN_INF
舍入到无穷	AP_RND_INF
收敛舍入	AP_RND_CONV
截断	AP_TRN
截断到 0	AP_TRN_ZERO

AP_RND

- 将值舍入到特定 ap_[u]fixed 类型最接近的可表示值。

```
ap_fixed<3, 2, AP_RND, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.5
ap_fixed<3, 2, AP_RND, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.0
```

AP_RND_ZERO

- 将值舍入到最接近的可表示值。
- 向 0 舍入。
 - 对于正值，删除冗余位。
 - 对于负值，添加最低有效位以获取最接近的可表示值。

```
ap_fixed<3, 2, AP_RND_ZERO, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.0
ap_fixed<3, 2, AP_RND_ZERO, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.0
```

AP_RND_MIN_INF

- 将值舍入到最接近的可表示值。
- 向负无穷舍入。
 - 对于正值，删除冗余位。
 - 对于负值，添加最低有效位。

```
ap_fixed<3, 2, AP_RND_MIN_INF, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.0
ap_fixed<3, 2, AP_RND_MIN_INF, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.5
```

AP_RND_INF

- 将值舍入到最接近的可表示值。
- 舍入方式取决于最低有效位。
 - 对于正值，如果已设置最低有效位，则向正无穷舍入。否则，向负无穷舍入。
 - 对于负值，如果已设置最低有效位，则向负无穷舍入。否则，向正无穷舍入。

```
ap_fixed<3, 2, AP_RND_INF, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.5
ap_fixed<3, 2, AP_RND_INF, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.5
```

AP_RND_CONV

- 将值舍入到最接近的可表示值。
- 舍入方式取决于最低有效位。
 - 如果已设置最低有效位，则向正无穷舍入。
 - 否则，向负无穷舍入。

```
ap_fixed<3, 2, AP_RND_CONV, AP_SAT> UAPFixed4 = 0.75; // Yields: 1.0
ap_fixed<3, 2, AP_RND_CONV, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.0
```

AP_TRN

- 始终将值向负无穷舍入。

```
ap_fixed<3, 2, AP_TRN, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.0
ap_fixed<3, 2, AP_TRN, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.5
```

AP_TRN_ZERO

将值舍入为：

- 对于正值，舍入方式与 AP_TRN 模式相同。
- 对于负值，向 0 舍入。

```
ap_fixed<3, 2, AP_TRN_ZERO, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.0
ap_fixed<3, 2, AP_TRN_ZERO, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.0
```

溢出模式

饱和	AP_SAT
饱和到 0	AP_SAT_ZERO
对称饱和	AP_SAT_SYM
卷绕	AP_WRAP
符号量值卷绕	AP_WRAP_SM

AP_SAT

使值达到饱和。

- 设为最大值，以防溢出。
- 设为负最大值，以防负溢出。

```
ap_fixed<4, 4, AP_RND, AP_SAT> UAPFixed4 = 19.0; // Yields: 7.0
ap_fixed<4, 4, AP_RND, AP_SAT> UAPFixed4 = -19.0; // Yields: -8.0
ap_ufixed<4, 4, AP_RND, AP_SAT> UAPFixed4 = 19.0; // Yields: 15.0
ap_ufixed<4, 4, AP_RND, AP_SAT> UAPFixed4 = -19.0; // Yields: 0.0
```

AP_SAT_ZERO

强制值为 0，以防溢出或负溢出。

```
ap_fixed<4, 4, AP_RND, AP_SAT_ZERO> UAPFixed4 = 19.0; // Yields: 0.0
ap_fixed<4, 4, AP_RND, AP_SAT_ZERO> UAPFixed4 = -19.0; // Yields: 0.0
ap_ufixed<4, 4, AP_RND, AP_SAT_ZERO> UAPFixed4 = 19.0; // Yields: 0.0
ap_ufixed<4, 4, AP_RND, AP_SAT_ZERO> UAPFixed4 = -19.0; // Yields: 0.0
```

AP_SAT_SYM

使值饱和为：

- 最大值，以防溢出。
- 最小值，以防负溢出。
 - 针对有符号的 `ap_fixed` 类型，设为负最大值
 - 针对无符号的 `ap_ufixed` 类型，设为 0

```
ap_fixed<4, 4, AP_RND, AP_SAT_SYM> UAPFixed4 = 19.0; // Yields: 7.0
ap_fixed<4, 4, AP_RND, AP_SAT_SYM> UAPFixed4 = -19.0; // Yields: -7.0
ap_ufixed<4, 4, AP_RND, AP_SAT_SYM> UAPFixed4 = 19.0; // Yields: 15.0
ap_ufixed<4, 4, AP_RND, AP_SAT_SYM> UAPFixed4 = -19.0; // Yields: 0.0
```

AP_WRAP

将值卷绕，以防溢出。

```
ap_fixed<4, 4, AP_RND, AP_WRAP> UAPFixed4 = 31.0; // Yields: -1.0
ap_fixed<4, 4, AP_RND, AP_WRAP> UAPFixed4 = -19.0; // Yields: -3.0
ap_ufixed<4, 4, AP_RND, AP_WRAP> UAPFixed4 = 19.0; // Yields: 3.0
ap_ufixed<4, 4, AP_RND, AP_WRAP> UAPFixed4 = -19.0; // Yields: 13.0
```

如果 N 的值设为 0（默认溢出模式）：

- 删除超出范围的所有 MSB 位。
- 针对无符号的数值：达到最大值后，卷绕为 0。
- 针对有符号的数值：达到最大值后，卷绕至最小值。

如果 N>0：

- 当 N > 0 时，N 个 MSB 位饱和为 1 或设为 1。
- 保留符号位，因此正数仍保留为正数，负数保留为负数。
- 复制不饱和的位，从 LSB 侧开始。

AP_WRAP_SM

应以符号量值卷绕该值。

```
ap_fixed<4, 4, AP_RND, AP_WRAP_SM> UAPFixed4 = 19.0; // Yields: -4.0
ap_fixed<4, 4, AP_RND, AP_WRAP_SM> UAPFixed4 = -19.0; // Yields: 2.0
```

如果 N 的值设为 0（默认溢出模式）：

- 此模式使用符号量值卷绕。
- 符号位设置为已删除的最低有效位的值。
- 如果剩余最高有效位不同于原 MSB，那么所有剩余位全部取反。
- 如果 MSB 相同，那么将复制其它位。
 1. 删除冗余 MSB。
 2. 新符号位是已删除位的最低有效位。在此情况下即为 0。
 3. 将新的符号位与新的值的符号进行比较。
- 如果不同，则对所有数值取反。在此情况下两者不同。

如果 N>0：

- 使用符号量级饱和
- N 个 MSB 饱和为 1。
- 与 N = 0 情况下的行为类似，但正数仍保留为正数，负数保留为负数。

编译 ap_[u]fixed<> 类型

要使用 ap_[u]fixed<> 类，必须将 ap_fixed.h 头文件包含在引用 ap_[u]fixed<> variables 的所有源文件中。

编译使用这些类的软件模型时，可能需要指定 Vivado HLS 头文件的位置，例如，为 g++ 编译添加 “-I/<HLS_HOME>/include” 选项。

声明和定义 ap_[u]fixed<> 变量

有符号的类与无符号的类不同：

- ap_fixed<W,I>（有符号）
- ap_ufixed<W,I>（无符号）

您可使用 C/C++ typedef 语句创建用户定义的类型：

```
#include "ap_fixed.h" // use ap_[u]fixed<> types

typedef ap_ufixed<128,32> uint128_t; // 128-bit user defined type,
// 32 integer bits
```

用户定义的类型示例

从常量初始化和赋值（字面值）

您可使用常用 C/C++ 宽度的正常浮点常量来初始化 `ap_[u]fixed` 变量：

- 32 位（针对 `float` 类型）
- 64 位（针对 `double` 类型）

即通常为单精度类型或双精度形式的浮点值。

请注意分配到定点变量的值将受到常量精度的限制。按[从常量初始化和赋值（字面值）](#)中所述使用字符串初始化可确保根据字符串所描述的精度来填充定点值的所有位。

```
#include <ap_fixed.h>

ap_ufixed<30, 15> my15BitInt = 3.1415;
ap_fixed<42, 23> my42BitInt = -1158.987;
ap_ufixed<99, 40> = 287432.0382911;
ap_fixed<36, 30> = -0x123.456p-1;
```

在 `std::complex` 类型的数组中使用，`ap_[u]fixed` 类型不支持初始化。

```
typedef ap_fixed<DIN_W, 1, AP_TRN, AP_SAT> coeff_t; // MUST have IW >= 1
std::complex<coeff_t> twid_rom[REAL_SZ/2] = {{ 1, -0 }, { 0.9, -0.006 }, etc.}
```

初始化值必须首先强制转换为 `std::complex` 类型：

```
typedef ap_fixed<DIN_W, 1, AP_TRN, AP_SAT> coeff_t; // MUST have IW >= 1
std::complex<coeff_t> twid_rom[REAL_SZ/2] = {std::complex<coeff_t>( 1,
-0 ),
std::complex<coeff_t>(0.9, -0.006 ), etc.}
```

控制台 I/O 支持（打印）

正如 `ap_[u]fixed<>` 变量初始化和赋值一样，Vivado HLS 支持打印显示超 64 位的值。

输出存储在 `ap_[u]fixed` 变量中的任意值的最简单方法是使用 C++ 标准输出流：`std::cout`（`#include <iostream>` 或 `<iostream.h>`）。流插入运算符 `<<` 已重载，可正确输出任意给定 `ap_[u]fixed` 变量的全部可能值。以下流传输操控程序同样受支持，以便按所示方式对该值进行格式化。

- `dec`（十进制）
- `hex`（十六进制）
- `oct`（八进制）

```
#include <iostream.h>
// Alternative: #include <iostream>

ap_fixed<6, 3, AP_RND, AP_WRAP> Val = 3.25;

cout << Val << endl; // Yields: 3.25
```

使用标准 C 语言库

您还可使用标准 C 语言库 (`#include <stdio.h>`) 来打印大于 64 位的值：

1. 使用 `ap_[u]fixed` 类方法 `to_string()` 将值转换为 C++ `std::string`。
2. 使用 `std::string` 类方法 `c_str()` 将结果转换为以 `null` 结尾的 C 语言字符串。

可选实参 1（指定基数）

您可为 `ap[u]int::to_string()` 方法传递 1 个可选实参以指定期望的数字格式的基数。有效的基数实参值包括：

- 2（二进制）
- 8（八进制）
- 10（十进制）
- 16（十六进制，默认值）

可选实参 2（打印为有符号的值）

`ap_[u]int::to_string()` 的第 2 个可选实参可指定是否将非十进制格式打印为有符号的值。该实参为布尔值。默认值为 `false`，导致非十进制格式打印为无符号的值。

```
ap_fixed<6,3, AP_RND, AP_WRAP> Val = 3.25;

printf("%s \n", in2.to_string().c_str()); // Yields: 0b011.010
printf("%s \n", in2.to_string(10).c_str()); //Yields: 3.25
```

以下 C++ 操控程序函数支持 `ap_[u]fixed` 类型：

- `setprecision`
- `setw`
- `setfill`

`setprecision` 操控程序用于设置要使用的小数位精度。需 1 个参数 `f` 作为小数位精度的值，其中 `n` 用于指定要显示的有意义的最大位数（计算小数点前后的总位数）。

`f` 默认值为 6，与原生 C 浮点类型一致。

```
ap_fixed<64, 32> f =3.14159;
cout << setprecision (5) << f << endl;
cout << setprecision (9) << f << endl;
f = 123456;
cout << setprecision (5) << f << endl;
```

以上示例显示了以下结果，其中当实际精度超过指定精度时，将舍入打印的结果：

```
3.1416
3.14159
1.2346e+05
```

`setw` 操控程序：

- 用于设置要用作为字段宽度的字符数。
- 需 1 个参数 `w` 作为宽度值

其中

- `w` 用于判定在某些输出表示法中将写入的最小字符数。

如果该表示法的标准宽度小于字段宽度，那么将以填充字符来填充该表示法。填充字符由 `setfill` 操控程序来控制，需 1 个参数 `f` 作为填充字符。

例如，给定：

```
ap_fixed<65,32> aa = 123456;
int precision = 5;
cout<<setprecision(precision)<<setw(13)<<setfill('T')<<a<<endl;
```

输出为：

```
TTT1.2346e+05
```

涉及 `ap_[u]fixed<>` 类型的表达式

任意精度定点值可用于使用受 C/C++ 支持的任意运算符的表达式。定义任意精度定点类型或变量后，其使用方式与 C/C++ 语言中的任意浮点类型或变量相同。

请注意以下附加说明：

- 0 值和符号扩展

根据源值符号，较小位宽的所有值均为 0 值或符号扩展值。您可能需要插入强制类型转换以在将较小位宽的值赋值到较大位宽的值时获取替代符号。

- 截断

如果赋值的任意精度定点的位宽大于目标变量，则会发生截断。

类方法、运算符和数据成员

一般情况下，支持针对 `ap_[u]fixed` 类型使用运算符重载来执行在原生 C/C++ 整数数据类型上执行的任意有效操作。除这些重载运算符外，还包含部分特定于类的运算符和方法以简化位级运算。

二进制算术运算符

加法

```
ap_[u]fixed::RType ap_[u]fixed::operator + (ap_[u]fixed op)
```

以任意精度定点加上给定操作数 `op`

该操作数可采用以下任一整数类型：

- `ap_[u]fixed`
- `ap_[u]int`
- C/C++

返回类型 `ap_[u]fixed::RType` 取决于 2 个操作数的类型信息。

```
ap_fixed<76, 63> Result;

ap_fixed<5, 2> Val1 = 1.125;
ap_fixed<75, 62> Val2 = 6721.35595703125;

Result = Val1 + Val2; //Yields 6722.480957
```

由于 `Val2` 的整数部分和小数部分位宽均较大，因此返回类型采用相同位宽加 1 以便存储所有可能的结果值。

指定数据宽度可通过使用幂函数来控制资源，如下所示。在此类情况下，赛灵思建议指定存储的结果的宽度，而不是指定定点运算的宽度。

```
ap_ufixed<16,6> x=5;
ap_ufixed<16,7>y=hl::rsqrt<16,6>(x+x);
```

减法

```
ap_[u]fixed::RType ap_[u]fixed::operator - (ap_[u]fixed op)
```

以任意精度定点减去给定操作数 `op`

返回类型 `ap_[u]fixed::RType` 取决于 2 个操作数的类型信息。

```
ap_fixed<76, 63> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val2 - Val1; // Yields 6720.23057
```

由于 `Val2` 的整数部分和小数部分位宽均较大，因此返回类型采用相同位宽加 1 以便存储所有可能的结果值。

乘法

```
ap_[u]fixed::RType ap_[u]fixed::operator * (ap_[u]fixed op)
```

以任意精度定点乘以给定操作数 `op`

```
ap_fixed<80, 64> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val1 * Val2; // Yields 7561.525452
```

这将显示 `Val1` 与 `Val2` 的乘法。返回类型为其整数部分位宽与其小数部分位宽之和。

除法

```
ap_[u]fixed::RType ap_[u]fixed::operator / (ap_[u]fixed op)
```

以任意精度定点除以给定操作数 `op`

```
ap_fixed<84, 66> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Val2 / Val1; // Yields 5974.538628
```

这将显示 `Val1` 与 `Val2` 的除法。为保留足够精度：

- 返回类型的整数位宽为 `Val2` 的整数位宽与 `Val1` 的小数位宽之和。
- 返回类型的小数位宽等于 `Val2` 的小数位宽。

按位逻辑运算符

按位 OR

```
ap_[u]fixed::RType ap_[u]fixed::operator | (ap_[u]fixed op)
```

对任意精度定点和给定操作数 `op` 应用按位运算。

```
ap_fixed<75, 62> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val1 | Val2; // Yields 6271.480957
```

按位 AND

```
ap_[u]fixed::RType ap_[u]fixed::operator & (ap_[u]fixed op)
```

对任意精度定点和给定操作数 `op` 应用按位运算。

```
ap_fixed<75, 62> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val1 & Val2; // Yields 1.00000
```

按位 XOR

```
ap_[u]fixed::RType ap_[u]fixed::operator ^ (ap_[u]fixed op)
```

对任意精度定点和给定操作数 `op` 应用 `xor` 按位运算。

```
ap_fixed<75, 62> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val1 ^ Val2; // Yields 6720.480957
```

递增运算符和递减运算符

递增前

```
ap_[u]fixed ap_[u]fixed::operator ++ ()
```

此运算符函数前缀会将任意精度定点变量加 1。

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = ++Val1; // Yields 6.125000
```

递增后

```
ap_[u]fixed ap_[u]fixed::operator ++ (int)
```

此运算符函数后缀：

- 会将任意精度定点变量加 1。
- 返回此任意精度定点的原始 val。

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = Val1++; // Yields 5.125000
```

递减前

```
ap_[u]fixed ap_[u]fixed::operator -- ()
```

此运算符函数前缀会将任意精度定点变量减 1。

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = --Val1; // Yields 4.125000
```

递减后

```
ap_[u]fixed ap_[u]fixed::operator -- (int)
```

此运算符函数后缀：

- 会将任意精度定点变量减 1。
- 返回此任意精度定点的原始 val。

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = Val1--; // Yields 5.125000
```

一元运算符

加法

```
ap_[u]fixed ap_[u]fixed::operator + ()
```

返回任意精度定点变量的自拷贝。

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = +Val1; // Yields 5.125000
```

减法

```
ap_[u]fixed::RType ap_[u]fixed::operator - ()
```

返回任意精度定点变量的负值。

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = -Val1; // Yields -5.125000
```

等于 0

```
bool ap_[u]fixed::operator ! ()
```

此运算符函数：

- 将任意精度定点变量与 0 比较。
- 返回结果。

```
bool Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = !Val1; // Yields false
```

按位反转

```
ap_[u]fixed::RType ap_[u]fixed::operator ~ ()
```

返回任意精度定点变量的按位补码。

```
ap_fixed<25, 15> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = ~Val1; // Yields -5.25
```

移位运算符

无符号的左移

```
ap_[u]fixed ap_[u]fixed::operator << (ap_uint<_W2> op)
```

此运算符函数：

- 按给定整数操作数左移。
- 返回结果。

此操作数可采用以下 C/C++ 整数类型：

- char
- short
- int
- long

左移运算的返回类型与移位的类型的宽度相同。

注释：移位不支持溢出或量化模式。

```
ap_fixed<25, 15> Result;
ap_fixed<8, 5> Val = 5.375;

ap_uint<4> sh = 2;

Result = Val << sh; // Yields -10.5
```

结果的位宽为 (W = 25, I = 15)。由于左移运算返回类型与 Val 类型相同：

- Val 的高阶 2 位将移出。
- 结果为 -10.5。

如果需结果为 21.5, Val 必须首先强制转换为 ap_fixed<10, 7> - 例如, ap_ufixed<10, 7>(Val)。

有符号的左移

```
ap_[u]fixed ap_[u]fixed::operator << (ap_int<_W2> op)
```

此运算符：

- 按给定整数操作数左移。
- 返回结果。

移位方向取决于操作数为正还是为负。

- 如果操作数为正, 则执行右移。
- 如果操作数为负, 则执行左移 (反方向)。

此操作数可采用以下 C/C++ 整数类型：

- char

- short
- int
- long

右移运算的返回类型与移位的类型的宽度相同。

```
ap_fixed<25, 15, false> Result;
ap_uint<8, 5> Val = 5.375;

ap_int<4> Sh = 2;
Result = Val << sh; // Shift left, yields -10.25

Sh = -2;
Result = Val << sh; // Shift right, yields 1.25
```

无符号的右移

```
ap_[u]fixed ap_[u]fixed::operator >> (ap_uint<_W2> op)
```

此运算符函数：

- 按给定整数操作数右移。
- 返回结果。

此操作数可采用以下 C/C++ 整数类型：

- char
- short
- int
- long

右移运算的返回类型与移位的类型的宽度相同。

```
ap_fixed<25, 15> Result;
ap_fixed<8, 5> Val = 5.375;

ap_uint<4> sh = 2;

Result = Val >> sh; // Yields 1.25
```

如需保留所有有效位，请首先扩展 Val 的小数部分的位宽，例如，ap_fixed<10, 5>(Val)。

有符号的右移

```
ap_[u]fixed ap_[u]fixed::operator >> (ap_int<_W2> op)
```

此运算符：

- 按给定整数操作数右移。
- 返回结果。

移位方向取决于操作数为正还是为负。

- 如果操作数为正，则执行右移。
- 如果操作数为负，则执行左移（反方向）。

此操作数可采用任一 C/C++ 整数类型（char、short、int 或 long）。

右移运算的返回类型与移位的类型的宽度相同。例如：

```
ap_fixed<25, 15, false> Result;
ap_uint<8, 5> Val = 5.375;

ap_int<4> Sh = 2;
Result = Val >> sh; // Shift right, yields 1.25

Sh = -2;
Result = Val >> sh; // Shift left, yields -10.5

1.25
```

关系运算符

等于

```
bool ap_[u]fixed::operator == (ap_[u]fixed op)
```

此运算符会将任意精度定点变量与给定操作数进行比较。

如果两者相等，则返回 true；如果不相等，则返回 false。

此 op 操作数的类型可以是 ap_[u]fixed、ap_int 或 C/C++ 整数类型。例如：

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 == Val2; // Yields true
Result = Val1 == Val3; // Yields false
```

不等于

```
bool ap_[u]fixed::operator != (ap_[u]fixed op)
```

此运算符会将此任意精度定点变量与给定操作数进行比较。

如果两者不相等，则返回 true；如果相等，则返回 false。

op 操作数的类型可以是：

- ap_[u]fixed
- ap_int
- C 或 C++ 整数类型

例如：

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 != Val2; // Yields false
Result = Val1 != Val3; // Yields true
```

大于或等于

```
bool ap_[u]fixed::operator >= (ap_[u]fixed op)
```

此操作数会将变量与给定操作数比较。

如果两者相等或者如果变量大于运算符，则返回 `true`，否则返回 `false`。

此 `op` 操作数的类型可以是 `ap_[u]fixed`、`ap_int` 或 C/C++ 整数类型。

例如：

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 >= Val2; // Yields true
Result = Val1 >= Val3; // Yields false
```

小于或等于

```
bool ap_[u]fixed::operator <= (ap_[u]fixed op)
```

此运算符会将变量与给定操作数比较，如果变量等于或小于操作数，则返回 `true`，否则返回 `false`。

此 `op` 操作数的类型可以是 `ap_[u]fixed`、`ap_int` 或 C/C++ 整数类型。

例如：

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 <= Val2; // Yields true
Result = Val1 <= Val3; // Yields true
```

大于

```
bool ap_[u]fixed::operator > (ap_[u]fixed op)
```

此运算符会将变量与给定操作数比较，如果变量大于运算符，则返回 `true`，否则返回 `false`。

此 `op` 操作数的类型可以是 `ap_[u]fixed`、`ap_int` 或 C/C++ 整数类型。

例如：

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 > Val2; // Yields false
Result = Val1 > Val3; // Yields false
```

小于

```
bool ap_[u]fixed::operator < (ap_[u]fixed op)
```

此运算符会将变量与给定操作数比较，如果变量小于运算符，则返回 `true`，否则返回 `false`。

此 `op` 操作数的类型可以是 `ap_[u]fixed`、`ap_int` 或 C/C++ 整数类型。例如：

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 < Val2; // Yields false
Result = Val1 < Val3; // Yields true
```

位运算符

位选择和位设置

```
af_bit_ref ap_[u]fixed::operator [] (int bit)
```

此运算符从任意精度定点值中选择 1 个位，并将其返回。

返回的值为可用于设置或清除 `ap_[u]fixed` 变量中的对应位的参考值。位实参必须为整数值，它用于指定要选择的位的索引。最低有效位索引为 0。允许的最高索引为此 `ap_[u]fixed` 变量的位宽减 1。

结果类型为 `af_bit_ref`，值为 0 或 1。例如：

```
ap_int<8, 5> Value = 1.375;

Value[3]; // Yields 1
Value[4]; // Yields 0

Value[2] = 1; // Yields 1.875
Value[3] = 0; // Yields 0.875
```

位范围

```
af_range_ref af_(u)fixed::range (unsigned Hi, unsigned Lo)
af_range_ref af_(u)fixed::operator [] (unsigned Hi, unsigned Lo)
```

此运算类似于位选择运算符 [], 但它对某一范围内的位而不是对单个位进行运算。

它可从任意精度定点变量中选择一组位。Hi 实参可提供要选择的位范围的上半部分。Lo 实参可提供要选择的最低有效位。如果 Lo 大于 Hi, 那么所选的位将按逆序返回。

返回类型 `af_range_ref` 表示引用 Hi 和 Lo 所指定的 `ap_[u]fixed` 变量范围。例如：

```
ap_uint<4> Result = 0;
ap_ufixed<4, 2> Value = 1.25;
ap_uint<8> Repl = 0xAA;

Result = Value.range(3, 0); // Yields: 0x5
Value(3, 0) = Repl(3, 0); // Yields: -1.5

// when Lo > Hi, return the reverse bits string
Result = Value.range(0, 3); // Yields: 0xA
```

范围选择

```
af_range_ref af_(u)fixed::range ()
af_range_ref af_(u)fixed::operator []
```

此运算符是范围选择运算符 [] 的特例。它会按正常顺序选择该任意精度定点值中的所有位。

返回类型 `af_range_ref` 表示对 Hi = W - 1 和 Lo = 0 所指定的范围的引用例如：

```
ap_uint<4> Result = 0;

ap_ufixed<4, 2> Value = 1.25;
ap_uint<8> Repl = 0xAA;

Result = Value.range(); // Yields: 0x5
Value() = Repl(3, 0); // Yields: -1.5
```

长度

```
int ap_[u]fixed::length ()
```

此函数返回的整数可提供任意精度定点值内的位数。可配合类型或值一起使用。例如：

```
ap_ufixed<128, 64> My128APFixed;

int bitwidth = My128APFixed.length(); // Yields 128
```

显式转换方法

定点到双精度

```
double ap_[u]fixed::to_double ()
```

此成员函数以 IEEE 双精度格式返回此定点值。例如：

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
double Result;

Result = MyAPFixed.to_double(); // Yields 333.789
```

定点到浮点

```
float ap_[u]fixed::to_float()
```

此成员函数以 IEEE 浮点格式返回此定点值。例如：

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
float Result;

Result = MyAPFixed.to_float(); // Yields 333.789
```

定点到半精度浮点

```
half ap_[u]fixed::to_half()
```

此成员函数以 HLS 半精度（16 位）浮点精度格式返回此定点值。例如：

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
half Result;

Result = MyAPFixed.to_half(); // Yields 333.789
```

定点到 ap_int

```
ap_int ap_[u]fixed::to_ap_int ()
```

此成员函数将此定点值显式转换为 ap_int 以捕获所有整数位（小数位将被截断）。例如：

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
ap_uint<77> Result;

Result = MyAPFixed.to_ap_int(); //Yields 333
```

定点到整数

```
int ap_[u]fixed::to_int ()
unsigned ap_[u]fixed::to_uint ()
ap_slong ap_[u]fixed::to_int64 ()
ap_ulong ap_[u]fixed::to_uint64 ()
```

此成员函数将此定点值显式转换为 C 内置整数类型。例如：

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
unsigned int Result;

Result = MyAPFixed.to_uint(); //Yields 333

unsigned long long Result;
Result = MyAPFixed.to_uint64(); //Yields 333
```



建议：赛灵思建议显式调用成员函数，而不是使用 C 样式强制转换来将 ap_[u]fixed 转换为其它数据类型。

编译时间访问数据类型属性

`ap_[u]fixed<>` 类型将随多个静态成员一起提供，以便在编译时判定数据类型的大小和配置。数据类型将随 `static const` 成员一起提供：`width`、`iwidth`、`qmode` 和 `omode`：

```
static const int width = _AP_W;
static const int iwidth = _AP_I;
static const ap_q_mode qmode = _AP_Q;
static const ap_o_mode omode = _AP_O;
```

您可使用这些数据成员从任何现有 `ap_[u]fixed<>` 数据类型提取以下信息：

- `width`：数据类型的宽度。
- `iwidth`：数据类型的整数部分的宽度。
- `qmode`：数据类型的量化模式。
- `omode`：数据类型的溢出模式。

例如，您可使用这些数据成员来抽取现有 `ap_[u]fixed<>` 数据类型的数据宽度，以在编译时创建另一个 `ap_[u]fixed<>` 数据类型。

以下示例显示 `Res` 变量的大小自动定义为比使用相同量化模式的 `Val1` 变量和 `Val2` 变量位宽大 1 位：

```
// Definition of basic data type
#define INPUT_DATA_WIDTH 12
#define IN_INTG_WIDTH 6
#define IN_QMODE AP_RND_ZERO
#define IN_OMODE AP_WRAP
typedef ap_fixed<INPUT_DATA_WIDTH, IN_INTG_WIDTH, IN_QMODE, IN_OMODE>
data_t;
// Definition of variables
data_t Val1, Val2;
// Res is automatically sized at run-time to be 1-bit greater than
INPUT_DATA_WIDTH
// The bit growth in Res will be in the integer bits
ap_int<data_t::width+1, data_t::iwidth+1, data_t::qmode, data_t::omode> Res
= Val1 +
Val2;
```

这样可确保 Vivado HLS 对加法导致的位增长进行正确建模，即使为 `data_t` 更新 `INPUT_DATA_WIDTH`、`IN_INTG_WIDTH` 或量化模式的值也是如此。

SystemC 类型与 Vivado HLS 类型之比较

Vivado HLS 类型与 SystemC 类型在几乎所有情况下都相似且兼容，使用 Vivado HLS 类型编写的代码通常可移植到 SystemC 语言设计，且反之亦然。

在 Vivado HLS 类型与 SystemC 类型的行为之间存在一些差异。本节中探讨了这些差异，其中涵盖如下主题。

- 默认构造函数
- 整数除法
- 整数取模

- 负移位
- 过左移
- 范围运算
- 定点除法
- 定点右移
- 定点左移

默认构造函数

在 SystemC 中，以下类型的构造函数会在执行程序前将值初始化为 0：

- `sc_[u]int`
- `sc_[u]bignint`
- `sc_[u]fixed`

构造函数不会对以下 Vivado HLS 类型进行初始化：

- `ap_[u]int`
- `ap_[u]fixed`

Vivado HLS 位精确数据类型：

- `ap_[u]int`
无默认初始化
- `ap_[u]fixed`
无默认初始化

SystemC 位精确数据类型：

- `sc_[u]int`
默认初始化为 0
- `sc_big[u]int`
默认初始化为 0
- `sc_[u]fixed`
默认初始化为 0



注意！ 将 SystemC 类型移植到 Vivado HLS 类型时，请确保在写入条件前不读取或使用条件中的任何值。

SystemC 启动时可能显示所有输出默认值均为 0，无论是否已写入输出都是如此。表述为 Vivado HLS 类型的相同变量在写入前均保持处于未知状态。

整数除法

使用整数除法时，Vivado HLS 类型与 `sc_big[u]int` 类型保持一致，但其行为与 `sc_[u]int` 类型不同。下图显示了 1 个示例。

图 108：整数除法差异

ap_(u)int	=	sc_big(u)int	≠	sc_(u)int
<pre>#include "ap_int.h" ap_uint<15>dividend = 32757; ap_int<15>divisor = -2; ap_int<21>ret = dividend/divisor;</pre>		<pre>#include "systemc.h" sc_biguint<15>dividend = 32757; sc_biguint<15>divisor = -2; sc_biguint<21>ret = dividend/divisor;</pre>		<pre>#include "systemc.h" sc_uint<15>dividend = 32757; sc_int<15>divisor = -2; sc_int<21>ret = dividend/divisor;</pre>
dividend divisor ret 7 F F F 32757 7 F F E -2 1 F C 0 0 1 -16383		dividend divisor ret 7 F F F 32757 7 F F E -2 1 F C 0 0 1 -16383		dividend divisor ret 7 F F F 32757 7 F F E -2 0 0 0 0 0 0 0

X14224

当无符号整数除以有符号的整数负值时，SystemC `sc_int` 类型返回 0 值。Vivado HLS 类型（如 SystemC `sc_biguint` 类型）将显示负值结果。

整数取模

使用取模运算符时，Vivado HLS 类型与 `sc_big[u]int` 类型保持一致，但其行为与 `sc_[u]int` 类型不同。下图显示了 1 个示例。

图 109：整数取模差异

ap_(u)int	=	sc_big(u)int	≠	sc_(u)int
<pre>#include "ap_int.h" ap_uint<15>dividend = 18; ap_int<15>divisor = -5; ap_int<21>ret = dividend%divisor;</pre>		<pre>#include "systemc.h" sc_biguint<15>dividend = 18; sc_biguint<15>divisor = -5; sc_biguint<21>ret = dividend%divisor;</pre>		<pre>#include "systemc.h" sc_uint<15>dividend = 18; sc_int<15>divisor = -5; sc_int<21>ret = dividend%divisor;</pre>
dividend divisor ret 0 0 1 2 18 7 F F B -5 0 0 0 0 0 3 3		dividend divisor ret 0 0 1 2 18 7 F F B -5 0 0 0 0 0 3 3		dividend divisor ret 0 0 1 2 18 7 F F B -5 0 0 0 0 1 2 18

X14225

在以下情况下，SystemC `sc_int` 类型会返回取模运算的被除数值：

- 被除数为无符号的整数，以及
- 被除数为有符号的整数负值。

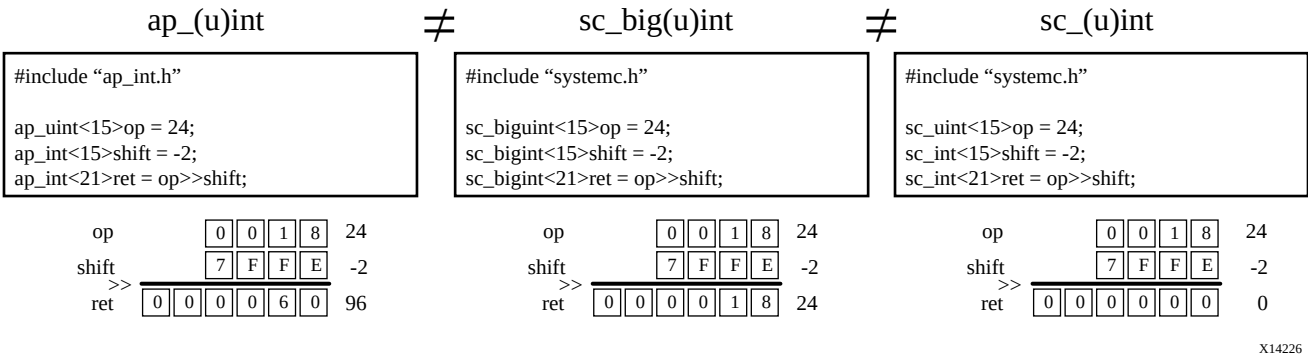
Vivado HLS 类型（例如 SystemC `sc_biguint` 类型）会返回取模运算的正值结果。

负相移

当移位运算的值为负数时，Vivado HLS `ap_[u]int` 类型将沿相反方向移位。例如，对于右移操作，它会执行左移。

在这种情况下，SystemC 的 `sc_[u]int` 类型与 `sc_big[u]int` 类型的行为方式将有所不同。下图显示了针对 Vivado HLS 和 SystemC 类型的此操作示例。

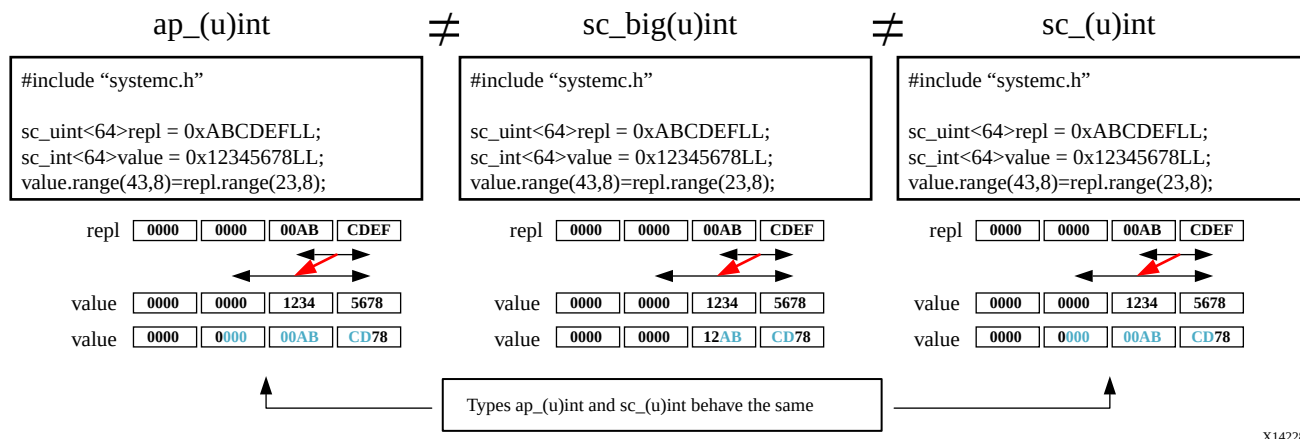
图 110：负相移差异



范围运算

使用范围运算时，如果源范围与目标范围大小不同，则行为会产生差异。下图显示了针对 Vivado HLS 类型和 SystemC 类型执行此操作的示例。请参阅以下综述。

图 112：范围运算差异



X14228

- Vivado HLS `ap_[u]int` 类型和 SystemC `sc_big[u]int` 类型会替换指定范围并扩展，以 0 值填充目标范围。
- SystemC `sc_big[u]int` 类型则仅更新源的范围。

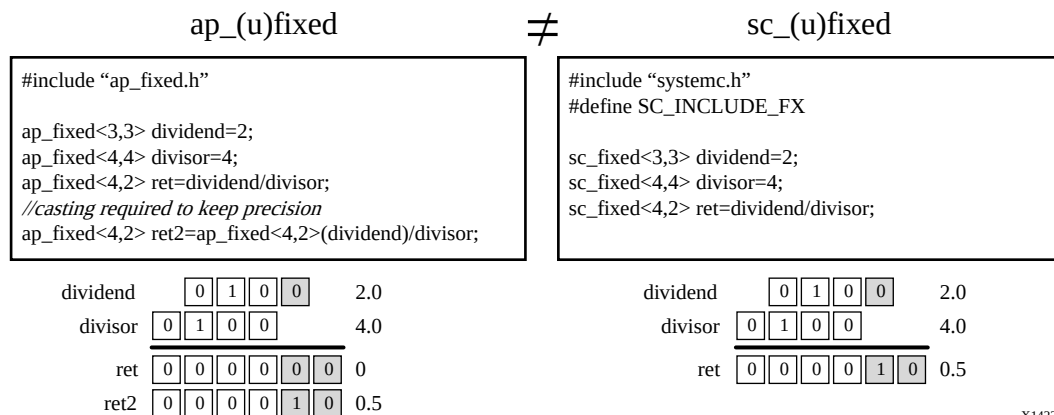
除法和定点类型

以不同大小的定点类型变量执行除法时，在 Vivado HLS 类型与 SystemC 类型之间分配小数值的方式有所不同。

对于 `ap_[u]fixed` 类型，小数不大于被除数的小数。SystemC `sc_[u]fixed` 类型保留除法的小数精度。使用 `ap_[u]fixed` 类型时可通过在赋值前将其类型强制转换为新变量宽度来保留小数部分。

下图显示了针对 Vivado HLS 类型和 SystemC 类型执行此操作的示例。

图 113：定点除法差异



X14229

右移和定点类型

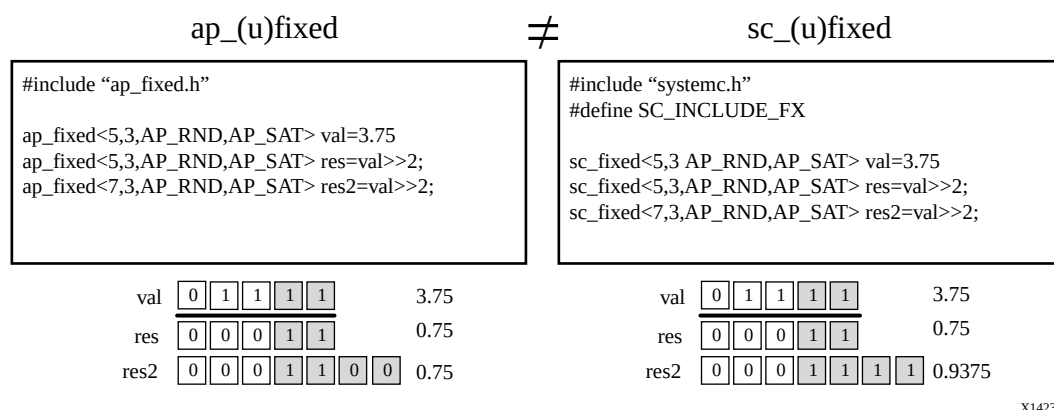
Vivado HLS 和 SystemC 执行右移操作时的行为不同。

- 对于 Vivado HLS 定点类型，先执行移位，然后赋值。
- 对于 SystemC 定点类型，先赋值，然后执行移位。

当结果为带有更多小数位数的定点类型时，SystemC 类型可保留额外的准确性。

下图显示了针对 Vivado HLS 类型和 SystemC 类型执行此操作的示例。

图 114：定点与右移的差异



X14230

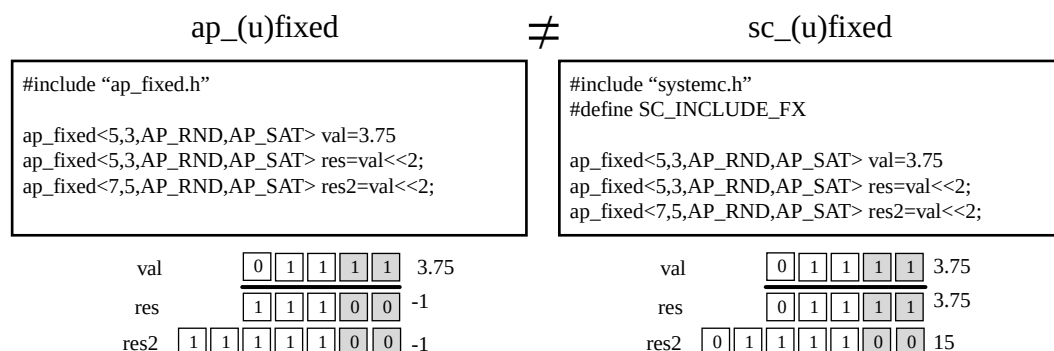
量化模式的类型不影响 ap_[u]fixed 右移的结果。赛灵思建议您先为结果类型的大小赋值，然后再执行移位操作。

左移和定点类型

使用 ap_[u]fixed 类型执行左移运算时，操作数按顺序执行符号位扩展、移位，然后赋值。SystemC sc_[u]fixed 类型则先赋值，再移位。在此情况下，Vivado HLS 类型会保留所有符号意向。

下图显示了针对 Vivado HLS 类型和 SystemC 类型执行此操作的示例。

图 115：定点与左移的差异



X14231

RTL 黑盒 JSON 文件

JSON 文件示例

本节提供了有关手动编写 RTL 黑盒所需的 JSON 文件的详细信息。以下是 JSON 文件示例：

```
{
  "c_function_name" : "foo",
  "rtl_top_module_name" : "foo",
  "c_files" :
  [
    {
      "c_file" : "../..a/top.cpp",
      "cflag" : ""
    },
    {
      "c_file" : "xx.cpp",
      "cflag" : "-D KF"
    }
  ],
  "rtl_files" : [
    "../..foo.v",
    "xx.v"
  ],
  "c_parameters" : [{
    "c_name" : "a",
    "c_port_direction" : "in",
    "rtl_ports" : {
      "data_read_in" : "a"
    }
  },
  {
    "c_name" : "b",
    "c_port_direction" : "in",
    "rtl_ports" : {
      "data_read_in" : "b"
    }
  },
  {
    "c_name" : "c",
    "c_port_direction" : "out",
    "rtl_ports" : {
      "data_write_out" : "c",
      "data_write_valid" : "c_ap_vld"
    }
  },
  {
    "c_name" : "d",
    "c_port_direction" : "inout",
    "rtl_ports" : {
      "data_read_in" : "d_i",
      "data_write_out" : "d_o",
      "data_write_valid" : "d_o_ap_vld"
    }
  },
  {
    "c_name" : "e",
    "c_port_direction" : "in",
    "rtl_ports" : {
      "FIFO_empty_flag" : "e_empty_n",
```

```

        "FIFO_read_enable" : "e_read",
        "FIFO_data_read_in" : "e"
    },
    {
        "c_name" : "f",
        "c_port_direction" : "out",
        "rtl_ports" : {
            "FIFO_full_flag" : "f_full_n",
            "FIFO_write_enable" : "f_write",
            "FIFO_data_write_out" : "f"
        }
    },
    {
        "c_name" : "g",
        "c_port_direction" : "in",
        "RAM_type" : "RAM_1P",
        "rtl_ports" : {
            "RAM_address" : "g_address0",
            "RAM_clock_enable" : "g_ce0",
            "RAM_data_read_in" : "g_q0"
        }
    },
    {
        "c_name" : "h",
        "c_port_direction" : "out",
        "RAM_type" : "RAM_1P",
        "rtl_ports" : {
            "RAM_address" : "h_address0",
            "RAM_clock_enable" : "h_ce0",
            "RAM_write_enable" : "h_we0",
            "RAM_data_write_out" : "h_d0"
        }
    },
    {
        "c_name" : "i",
        "c_port_direction" : "inout",
        "RAM_type" : "RAM_1P",
        "rtl_ports" : {
            "RAM_address" : "i_address0",
            "RAM_clock_enable" : "i_ce0",
            "RAM_write_enable" : "i_we0",
            "RAM_data_write_out" : "i_d0",
            "RAM_data_read_in" : "i_q0"
        }
    },
    {
        "c_name" : "j",
        "c_port_direction" : "in",
        "RAM_type" : "RAM_T2P",
        "rtl_ports" : {
            "RAM_address" : "j_address0",
            "RAM_clock_enable" : "j_ce0",
            "RAM_data_read_in" : "j_q0",
            "RAM_address_snd" : "j_address1",
            "RAM_clock_enable_snd" : "j_ce1",
            "RAM_data_read_in_snd" : "j_q1"
        }
    },
    {
        "c_name" : "k",
        "c_port_direction" : "out",
        "RAM_type" : "RAM_T2P",

```

```

        "rtl_ports" : {
            "RAM_address" : "k_address0",
            "RAM_clock_enable" : "k_ce0",
            "RAM_write_enable" : "k_we0",
            "RAM_data_write_out" : "k_d0",
            "RAM_address_snd" : "k_address1",
            "RAM_clock_enable_snd" : "k_ce1",
            "RAM_write_enable_snd" : "k_we1",
            "RAM_data_write_out_snd" : "k_d1"
        },
    },
    {
        "c_name" : "l",
        "c_port_direction" : "inout",
        "RAM_type" : "RAM_T2P",
        "rtl_ports" : {
            "RAM_address" : "l_address0",
            "RAM_clock_enable" : "l_ce0",
            "RAM_write_enable" : "l_we0",
            "RAM_data_write_out" : "l_d0",
            "RAM_data_read_in" : "l_q0",
            "RAM_address_snd" : "l_address1",
            "RAM_clock_enable_snd" : "l_ce1",
            "RAM_write_enable_snd" : "l_we1",
            "RAM_data_write_out_snd" : "l_d1",
            "RAM_data_read_in_snd" : "l_q1"
        },
    },
    "c_return" : {
        "c_port_direction" : "out",
        "rtl_ports" : {
            "data_write_out" : "ap_return"
        },
    },
    "rtl_common_signal" : {
        "module_clock" : "ap_clk",
        "module_reset" : "ap_rst",
        "module_clock_enable" : "ap_ce",
        "ap_ctrl_chain_protocol_idle" : "ap_idle",
        "ap_ctrl_chain_protocol_start" : "ap_start",
        "ap_ctrl_chain_protocol_ready" : "ap_ready",
        "ap_ctrl_chain_protocol_done" : "ap_done",
        "ap_ctrl_chain_protocol_continue" : "ap_continue"
    },
    "rtl_performance" : {
        "latency" : "6",
        "II" : "2"
    },
    "rtl_resource_usage" : {
        "FF" : "0",
        "LUT" : "0",
        "BRAM" : "0",
        "URAM" : "0",
        "DSP" : "0"
    }
}

```

附加资源与法律提示

赛灵思资源

如需了解答复记录、技术文档、下载以及论坛等支持性资源，请参阅[赛灵思技术支持](#)。

Documentation Navigator 与设计中心

赛灵思 Documentation Navigator (DocNav) 提供了访问赛灵思文档、视频和支持资源的渠道，您可以在其中筛选搜索信息。打开 DocNav 的方法：

- 在 Vivado[®] IDE 中，单击 “Help” → “Documentation and Tutorials”。
- 在 Windows 中，单击 “Start” → “All Programs” → “Xilinx Design Tools” → “DocNav”。
- 在 Linux 命令提示中输入 “docnav”。

赛灵思设计中心提供了根据设计任务和其他话题整理的文档链接，您可以使用链接了解关键概念以及常见问题解答。访问设计中心：

- 在 DocNav 中，单击 “Design Hub View” 标签。
- 在赛灵思网站上，查看[设计中心](#)页面。

注释：如需了解更多有关 DocNav 的信息，请参阅赛灵思网站上的 [Documentation Navigator](#)。

参考资料

以下技术文档是非常实用的补充资料，可配合本指南一起使用：

1. 《采用 Vivado 高层次综合开展 FPGA 设计介绍》([UG998](#))
2. 《Vivado Design Suite 教程：高层次综合》([UG871](#))
3. 《Vivado Design Suite 用户指南：版本说明、安装和许可》([UG973](#))
4. 《使用 Vivado HLS 进行浮点设计》([XAPP599](#))
5. 《LogiCORE IP Fast Fourier Transform 产品指南》([PG109](#))
6. 《LogiCORE IP FIR Compiler 产品指南》([PG149](#))

7. 《LogiCORE IP DDS Compiler 产品指南》(PG141)
8. 《Vivado Design Suite: AXI 参考指南》(UG1037)
9. 《使用 Vivado HLS 视频库配合 Zynq-7000 SoC 加速 OpenCV 应用》(XAPP1167)
10. 《UltraFast 高效设计方法指南》(UG1197)
11. GCC 网站上的“Option Summary”页面 (gcc.gnu.org/onlinedocs/gcc/Option-Summary.html)
12. Accellera 网站 (<http://www.accellera.org/>)
13. MathWorks 网站上的 AWGN 页面 (<http://www.mathworks.com/help/comm/ug/awgn-channel.html>)
14. Vivado® Design Suite 文档

请阅读：重要法律提示

本文向贵司/您所提供的信息（下称“资料”）仅在对赛灵思产品进行选择和使用参考。在适用法律允许的最大范围内：（1）资料均按“现状”提供，且不保证不存在任何瑕疵，赛灵思在此声明对资料及其状况不作任何保证或担保，无论是明示、暗示还是法定的保证，包括但不限于对适销性、非侵权性或任何特定用途的适用性的保证；且（2）赛灵思对任何因资料发生的或与资料有关的（含对资料的使用）任何损失或赔偿（包括任何直接、间接、特殊、附带或连带损失或赔偿，如数据、利润、商誉的损失或任何因第三方行为造成的任何类型的损失或赔偿），均不承担责任，不论该等损失或者赔偿是何种类或性质，也不论是基于合同、侵权、过失或是其他责任认定原理，即便该损失或赔偿可以合理预见或赛灵思事前被告知有发生该损失或赔偿的可能。赛灵思无义务纠正资料中包含的任何错误，也无义务对资料或产品说明书发生的更新进行通知。未经赛灵思公司的事先书面许可，贵司/您不得复制、修改、分发或公开展示本资料。部分产品受赛灵思有限保证条款的约束，请参阅赛灵思销售条款：<https://china.xilinx.com/legal.htm#tos>；IP 核可能受赛灵思向贵司/您签发的许可证中所包含的保证与支持条款的约束。赛灵思产品并非为故障安全保护目的而设计，也不具备此故障安全保护功能，不能用于任何需要专门故障安全保护性能的用途。如果把赛灵思产品应用于此类特殊用途，贵司/您将自行承担风险和责任。请参阅赛灵思销售条款：<https://china.xilinx.com/legal.htm#tos>。

关于与汽车相关用途的免责声明

如将汽车产品（部件编号中含“XA”字样）用于部署安全气囊或用于影响车辆控制的应用（“安全应用”），除非有符合 ISO 26262 汽车安全标准的安全概念或冗余特性（“安全设计”），否则不在质保范围内。客户应在使用或分销任何包含产品的系统之前为了安全的目的全面地测试此类系统。在未采用安全设计的条件下将产品用于安全应用的所有风险，由客户自行承担，并且仅在适用的法律法规对产品责任另有规定的情况下，适用该等法律法规的规定。

商标

© Copyright 2012-2020 赛灵思公司版权所有。Xilinx、赛灵思标识、Alveo、Artix、Kintex、Spartan、Versal、Virtex、Vivado、Zynq 本文提到的其它指定品牌均为赛灵思在美国及其它国家的商标。“OpenCL”和“OpenCL”标识均为 Apple Inc. 的商标，经 Khronos 许可后方可使用。“PCI”、“PCIe”和“PCI Express”均为 PCI-SIG 拥有的商标，且经授权使用。“AMBA”、“AMBA Designer”、“Arm”、“ARM1176JZ-SV”、“CoreSight”、“Cortex”、“PrimeCell”、“Mali”和“MPCore”为 Arm Limited 在欧盟及其它国家的注册商标。所有其它商标均为各自所有方所属财产。